

Documentation Ligne par Ligne

Explication du code, instruction par instruction, projet par projet

Dossier Talent-Performer-Projects-main — projets 8 a 36

Ce document explique le code source de chaque projet ligne par ligne : pour (quasiment) chaque ligne de code utile de chaque fichier principal, une explication precise indique ce que cette ligne fait precisement. Chaque projet est traite de maniere totalement independante des autres — aucune comparaison ni mise en relation entre projets n'est faite dans ce document, meme lorsque plusieurs projets partagent des outils ou des schemas similaires.

Les lignes strictement vides ne sont pas listees. Le fichier 10-Workflow_Builder/gestionclient.json (JSON tres repetitif) beneficie d'un traitement adapte, explique dans sa propre section. Les projets 12, 13, 15, 17, 19 et 28 n'existent pas dans le depot (numerotation non continue).

Sommaire

#	Projet
8	8-mistral_OCR
9	9-Arize
10	10-Workflow_Builder
11	11-Notion_RAG
14	14-Multi_AgentAGNO
16	16-Whatsapp
18	18-Memory_agent
20	20-Agno-Team
21	21-SDV-synthetise
22	22-Cyber_Workflow
23	23-Whatsapp-lette
24	24-ResearchAnalyst
25	25-PriceAndPositioning
26	26-PlateDetection
27	27-ImportExportSpecialist
29	29-Investment_insight
30	30-compliance_agent
31	31-Procurement-agent
32	32-Advance-Research-agent
33	33-FreightAgent

#	Projet
34	34-ImportExportSpecialist
35	35-HR-Assistant
36	36-rfp-Agent

Projet 8 — 8-mistral_OCR

Legal Copilot : OCR de document juridique + questions-reponses

Objectif

Permet d'uploader un PDF juridique (contrat, reglement...) via une interface Streamlit, d'en extraire le texte avec l'OCR de Mistral, puis de poser des questions en langage naturel sur son contenu ou d'en generer un resume. Le systeme retrouve les passages les plus pertinents par recherche vectorielle avant de generer chaque reponse.

Code source, ligne par ligne

8-mistral_OCR/legal_copilot.py

Classe OCRService qui centralise tous les appels au SDK Mistral (OCR, embeddings, chat) et la recherche vectorielle FAISS.

83 lignes de code annotatees ci-dessous.

2 import os

→ Importe le module standard pour lire les variables d'environnement et verifier l'existence de fichiers.

3 from dotenv import load_dotenv

→ Importe la fonction qui charge les variables du fichier .env dans l'environnement du processus. Ce fichier .env est la ou l'on met les cles API (secretes) plutot que de les ecrire en dur dans le code : ca evite de les publier par accident si le code est partage ou mis sur GitHub.

4 import time

→ Importe le module standard utilise plus bas pour espacer les appels API (time.sleep), afin de ne pas depasser la limite de requetes par seconde/minute imposee par l'API Mistral.

5 import faiss

→ Importe FAISS (Facebook AI Similarity Search), une librairie qui sait, etant donne un grand nombre de vecteurs numeriques, retrouver tres rapidement ceux qui ressemblent le plus a un vecteur donne. C'est le moteur de recherche utilise plus bas pour l'etape de recherche du systeme RAG (voir process_question).

6 import numpy as np

→ Importe NumPy, utilise pour manipuler les tableaux de vecteurs d'embeddings sous la forme attendue par FAISS.

7 import chardet

→ Importe la librairie de detection automatique d'encodage de texte, utilisee pour lire correctement le fichier Markdown genere par l'OCR meme si son encodage exact n'est pas connu a l'avance.

8 from mistralai.client import Mistral

→ Importe la classe client du SDK Mistral, le point d'entree unique de tous les appels a l'API Mistral (OCR, embeddings, chat) utilises dans ce fichier.

9 import os

→ Deuxieme import du meme module os (redondant avec la ligne 2, sans effet supplementaire).

11 load_dotenv()

→ Lit le fichier .env du projet et place ses variables (cles API) dans l'environnement du processus, pour que les lignes suivantes puissent les recuperer avec os.getenv().

```
13 MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
```

→ Recupere la cle API Mistral depuis les variables d'environnement, ou None si elle est absente. Cette cle est ce qui authentifie et facture les appels a l'API Mistral (OCR, embeddings, chat) faits par ce projet.

```
14 OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

→ Recupere une eventuelle cle OpenAI depuis l'environnement (non utilisee ailleurs dans ce fichier, sans doute un reste d'une version anterieure qui utilisait OpenAI avant de passer entierement a Mistral).

```
15 class OCRService:
```

→ Declare la classe qui regroupe toutes les operations OCR, embeddings et chat de ce projet. Regrouper ces operations dans une classe permet de partager un seul client Mistral authentifie entre toutes les methodes.

```
16     def __init__(self):
```

→ Definit le constructeur de la classe, appele une fois a la creation d'un objet OCRService.

```
17         self.client = Mistral(api_key=MISTRAL_API_KEY)
```

→ Cree le client Mistral authentifie avec la cle API, stocke sur l'instance pour etre reutilise partout. Recreer un client a chaque appel serait inutilement couteux, d'ou ce choix de le construire une seule fois.

```
19     def ocr_pdf(self, file_path):
```

→ Definit la methode qui envoie un PDF a l'API OCR de Mistral et sauvegarde le resultat sur disque. L'OCR (Optical Character Recognition, reconnaissance optique de caracteres) sert ici a transformer un PDF -- qui peut contenir du texte scanne, des tableaux, une mise en page complexe -- en texte structure exploitable par un modele de langage, ce qu'une simple extraction de texte PDF classique ne saurait pas toujours faire aussi fidelement pour un document juridique.

```
20         uploaded_pdf = self.client.files.upload(
```

→ Demarre l'appel qui telecharge le fichier PDF vers les serveurs Mistral : l'API OCR de Mistral fonctionne sur des fichiers deja heberges cote serveur, pas sur des octets envoyes directement dans la requete OCR.

```
21             file={
```

→ Ouvre le dictionnaire decrivant le fichier a envoyer.

```
22                 "file_name": file_path.split("/")[-1],
```

→ Extrait uniquement le nom de fichier (dernier segment du chemin) pour l'envoyer comme nom.

```
23                 "content": open(file_path, "rb"),
```

→ Ouvre le fichier PDF en mode binaire et fournit son contenu a l'upload.

```
24             },
```

→ Ferme le dictionnaire "file".

```
25             purpose="ocr"
```

→ Indique a l'API que ce fichier est destine a un traitement OCR (et non, par exemple, a du fine-tuning).

```
26         )
```

→ Ferme l'appel a client.files.upload et recupere l'objet representant le fichier televerse.

```
28         signed_url = self.client.files.get_signed_url(file_id=uploaded_pdf.id)
```

→ Demande a l'API une URL temporaire et signee permettant d'acceder au fichier televerse. Cette URL signee est une mesure de securite standard : elle donne un acces limite dans le temps au fichier prive, sans avoir a le rendre public ni a transmettre la cle API elle-meme dans l'URL.

```
30         ocr_response = self.client.ocr.process(
```

→ Demarre l'appel qui lance reellement le traitement OCR du document.

```
31             model="mistral-ocr-latest",
```

→ Precise le modele OCR de Mistral a utiliser pour l'extraction.

```
32             document={
```

→ Ouvre le dictionnaire décrivant le document à traiter.

```
33         "type": "document_url",
```

→ Indique que le document est fourni sous forme d'URL plutôt que de contenu brut.

```
34         "document_url": signed_url.url,
```

→ Fournit l'URL signée obtenue à la ligne 28 comme source du document.

```
35     },
```

→ Ferme le dictionnaire "document".

```
36         include_image_base64=True
```

→ Demande à l'API d'inclure aussi les images du PDF encodées en base64 dans la réponse, utile si le document contient des schémas ou des signatures scannées à préserver.

```
37     )
```

→ Ferme l'appel à `client.ocr.process` et récupère la réponse OCR complète.

```
39     with open("ocr_response.md", "w", encoding="utf-8") as f:
```

→ Ouvre (ou crée) le fichier `ocr_response.md` en écriture, avec un encodage UTF-8 explicite.

```
40         f.write("\n".join([page.markdown for page in ocr_response.pages]))
```

→ Concatène le Markdown de chaque page OCRisée avec un saut de ligne et écrit le résultat dans le fichier. Mistral OCR renvoie le texte extrait au format Markdown (avec titres, listes, tableaux structurés) plutôt qu'en texte brut, ce qui préserve une partie de la structure logique du document juridique original.

```
42     with open("ocr_meta.txt", "w") as meta:
```

→ Ouvre (ou crée) le fichier `ocr_meta.txt` qui sert de mémoire du dernier fichier traité.

```
43         meta.write(file_path)
```

→ Écrit le chemin du fichier PDF traité. C'est un mécanisme de cache simple : il permet à `ocr_is_valid()` (plus bas) de savoir, sans refaire l'OCR (qui coûte de l'argent et du temps), si le résultat déjà sauvegardé correspond toujours au PDF actuellement uploadé dans l'interface.

```
45     def read_markdown_file(self, path):
```

→ Définit une méthode utilitaire qui lit un fichier texte en détectant automatiquement son encodage.

```
46         with open(path, 'rb') as f:
```

→ Ouvre le fichier en mode binaire, nécessaire pour que `chardet` puisse analyser les octets bruts.

```
47         raw_data = f.read()
```

→ Lit l'intégralité du contenu binaire du fichier en mémoire.

```
48         result = chardet.detect(raw_data)
```

→ Analyse les octets pour deviner l'encodage de texte le plus probable (ex. utf-8, latin-1). C'est une protection défensive : si l'OCR ou l'utilisateur produit un fichier dans un encodage inattendu, le lire avec un encodage fixe (par exemple toujours utf-8) provoquerait une erreur ; `chardet` le devine à la volée.

```
49         encoding = result['encoding']
```

→ Extrait le nom de l'encodage détecté depuis le résultat de `chardet`.

```
50         return raw_data.decode(encoding, errors='replace')
```

→ Décode les octets bruts en texte avec l'encodage détecté, en remplaçant les caractères invalides plutôt que de faire planter tout le programme sur un seul caractère mal formé.

```
52     def get_text_embedding(self, input):
```

→ Définit la méthode qui transforme un morceau de texte en vecteur numérique via l'API Mistral. Un "embedding" est une représentation numérique (un vecteur de plusieurs centaines de nombres) qui capture le sens d'un texte : deux textes de sens proche ont des vecteurs proches, ce qui permet ensuite de comparer des textes mathématiquement plutôt qu'en cherchant des

mots-cles exacts.

```
53         embeddings_batch_response = self.client.embeddings.create(  
    → Demarre l'appel a l'API d'embeddings de Mistral.  
  
54         model="mistral-embed",  
    → Precise le modele d'embedding a utiliser.  
  
55         inputs=input  
    → Fournit le texte (ou la liste de textes) a transformer en vecteur.  
  
56     )  
    → Ferme l'appel et recupere la reponse contenant le ou les vecteurs generes.  
  
57     return embeddings_batch_response.data[0].embedding  
    → Retourne uniquement le vecteur du premier (et ici unique) texte envoye.  
  
59     def run_mistral(self, user_message, model="mistral-large-latest"):  
    → Definit la methode generique qui envoie un message a un modele de chat Mistral et recupere sa reponse. C'est le modele  
de langage (LLM) qui redige en langage naturel le resume ou la reponse finale a partir du contexte qu'on lui fournit -- il ne "sait"  
rien du document par lui-meme, tout ce qu'il sait vient du texte qu'on lui injecte dans le prompt (voir process_question plus bas).  
  
60         messages = [{"role": "user", "content": user_message}]  
    → Construit la liste de messages attendue par l'API de chat, avec un seul message utilisateur.  
  
61         chat_response = self.client.chat.complete(model=model, messages=messages)  
    → Envoie la requete de chat complete au modele demande (mistral-large-latest par default).  
  
62         return chat_response.choices[0].message.content  
    → Extrait et retourne uniquement le texte de la premiere reponse generee par le modele.  
  
64     def ocr_is_valid(self, file_path):  
    → Definit la methode qui verifie si le cache OCR existant correspond toujours au fichier actuellement uploadé, pour eviter de  
relancer un OCR couteux si ce n'est pas necessaire.  
  
65         if not os.path.exists("ocr_meta.txt"):  
    → Verifie si aucun OCR n'a jamais ete effectue (fichier de metadonnees absent).  
  
66         return False  
    → Si le fichier de metadonnees n'existe pas, indique que le cache n'est pas valide.  
  
67         with open("ocr_meta.txt", "r") as meta:  
    → Ouvre le fichier de metadonnees en lecture pour connaitre le dernier PDF traite.  
  
68         last_file = meta.read().strip()  
    → Lit le chemin du dernier fichier traite et retire les espaces/retours a la ligne superflus.  
  
69         return file_path == last_file  
    → Retourne vrai seulement si le fichier actuellement uploadé est exactement celui du dernier OCR effectue.  
  
71     def process_question(self, question, file_path):  
    → Definit la methode centrale du fichier : elle met en oeuvre un systeme RAG (Retrieval-Augmented Generation, "generation  
augmentee par recuperation"). L'idee du RAG : plutot que de demander directement au modele de langage de repondre (qui  
risquerait d'halluciner, ou de ne pas connaitre le contenu specifique de CE document), on va d'abord RETROUVER (retrieval) les  
passages du document les plus pertinents pour la question, puis on les injecte dans le prompt du modele pour qu'il GENERE  
(generation) sa reponse a partir de ce contexte precis et verifiable, plutot que de sa seule "memoire" generale.  
  
72         text = self.read_markdown_file("ocr_response.md")  
    → Recharge le texte Markdown complet issu de l'OCR precedemment sauvegarde.
```

73 chunk_size = 2048

→ Fixe la taille (en caracteres) de chaque morceau de texte a decouper. Un document entier est generalement trop long pour tenir dans un seul embedding utile ou dans le contexte du modele : on le decoupe donc en "chunks" (morceaux) plus petits et geres individuellement.

74 chunks = [text[i:i + chunk_size] for i in range(0, len(text), chunk_size)]

→ Decoupe le texte entier en une liste de morceaux successifs de 2048 caracteres chacun.

76 text_embeddings = []

→ Initialise la liste vide qui va accueillir le vecteur de chaque morceau de texte.

77 for chunk in chunks:

→ Demarre une boucle qui traite chaque morceau de texte un par un.

78 embedding = self.get_text_embedding(chunk)

→ Calcule le vecteur d'embedding du morceau de texte courant via l'API Mistral (c'est l'etape d'"indexation" du RAG : transformer chaque morceau en vecteur comparable).

79 text_embeddings.append(embedding)

→ Ajoute ce vecteur a la liste des embeddings.

80 time.sleep(2)

→ Attend 2 secondes avant le prochain appel API, pour respecter les limites de debit de l'API Mistral (un appel par chunk, donc un document de 20 chunks ferait 20 appels rapproches sans cette pause).

82 text_embeddings = np.array(text_embeddings)

→ Convertit la liste Python de vecteurs en tableau NumPy, format attendu par FAISS.

83 question_embedding = np.array([self.get_text_embedding(question)])

→ Calcule le vecteur de la question posee et l'enveloppe dans un tableau NumPy a une seule ligne, pour pouvoir le comparer aux vecteurs des morceaux du document dans le meme espace vectoriel.

85 d = text_embeddings.shape[1]

→ Recupere la dimension (longueur) des vecteurs d'embedding, necessaire pour initialiser l'index FAISS.

86 index = faiss.IndexFlatL2(d)

→ Cree un index FAISS qui compare les vecteurs par distance euclidienne (L2, "a vol d'oiseau" dans l'espace des vecteurs), sans compression : c'est la variante la plus simple de FAISS, exacte mais adaptee ici car le nombre de chunks d'un seul document reste petit (une compression approximative ne serait utile qu'a tres grande echelle, avec des millions de vecteurs).

87 index.add(text_embeddings)

→ Insere tous les vecteurs des morceaux de texte dans l'index de recherche.

89 D, I = index.search(question_embedding, k=2)

→ Recherche dans l'index les 2 vecteurs les plus proches du vecteur de la question ; D=distances, I=indices. C'est l'etape de "recuperation" (retrieval) du RAG : au lieu de donner tout le document au modele, on ne donne que les 2 passages statistiquement les plus proches en sens de la question posee.

90 retrieved_chunk = [chunks[i] for i in I.tolist()[0]]

→ Recupere le texte original des 2 morceaux les plus pertinents a partir de leurs indices.

92 prompt = f"""

→ Commence la construction du prompt final envoye au modele de chat, sous forme de chaine multi-lignes. C'est l'etape de "generation" du RAG : le prompt combine explicitement le contexte retrouve et la question, pour guider le modele a repondre a partir de CE contexte plutot que de connaissances generales potentiellement fausses ou obsoletes (ce qu'on appelle une hallucination).

93 Context information is below.

→ Ligne fixe du prompt introduisant le contexte fourni au modele.

94 -----

→ Separateur visuel dans le prompt, avant le contenu du contexte.

95 {retrieved_chunk}

→ Insere dans le prompt les 2 morceaux de texte retrouves par la recherche vectorielle.

96 -----

→ Second separateur visuel, marquant la fin du bloc de contexte.

97 Given the context information and not prior knowledge, answer the query.

→ Instruction explicite au modele de repondre uniquement a partir du contexte fourni, pas de connaissances generales -- c'est cette instruction qui reduit concretement le risque d'hallucination du modele.

98 Query: {question}

→ Insere la question de l'utilisateur telle quelle dans le prompt.

99 Answer :

→ Invite le modele a commencer sa reponse juste apres ce mot-cle.

100 """

→ Ferme la chaine multi-lignes du prompt.

102 return self.run_mistral(prompt)

→ Envoie le prompt complet au modele de chat Mistral et retourne sa reponse finale a l'appelant.

8-mistral_OCR/app.py

Interface Streamlit qui pilote la classe OCRService : upload du PDF, lancement de l'OCR, resume et questions.

50 lignes de code annotees ci-dessous.

1 import streamlit as st

→ Importe Streamlit, une librairie qui permet de construire une interface web interactive en pur Python (sans ecrire de HTML/CSS/JavaScript) : chaque widget (bouton, champ de texte...) est une simple fonction Python, et Streamlit ré-execute tout le script a chaque interaction de l'utilisateur.

2 import os

→ Importe le module standard, utilise ici pour creer un dossier et verifier l'existence de fichiers.

3 from legal_copilot import OCRService

→ Importe la classe metier definie dans legal_copilot.py qui contient toute la logique OCR/IA. Separer l'interface (app.py) de la logique metier (legal_copilot.py) permet de tester ou reutiliser cette logique independamment de Streamlit.

5 st.set_page_config(page_title="Legal Copilot", layout="wide")

→ Configure le titre de l'onglet du navigateur et demande une mise en page large de la page Streamlit.

6 st.title("Legal Copilot - PDF OCR and Legal QA")

→ Affiche le titre principal visible en haut de la page.

8 ocr_service = OCRService()

→ Instancie une seule fois le service OCR (et donc le client Mistral) pour toute la session Streamlit.

9 uploaded_file = st.file_uploader("Upload a legal PDF file", type=["pdf"])

→ Affiche un composant d'upload de fichier limite aux PDF et stocke le fichier choisi par l'utilisateur.

10 ocr_performed = False

→ Initialise un indicateur local a False, mis a True seulement si l'OCR est effectivement relance dans ce cycle.


```

12 if uploaded_file:
    → N'exécute le bloc suivant que si un fichier a été effectivement uploadé.

13     os.makedirs("pdf", exist_ok=True)
    → Crée le dossier "pdf" s'il n'existe pas encore, sans erreur s'il existe déjà.

14     file_path = f"pdf/{uploaded_file.name}"
    → Construit le chemin local où le PDF uploadé sera sauvegardé, à partir de son nom d'origine.

16     with open(file_path, "wb") as f:
        → Ouvre (ou crée) ce fichier local en écriture binaire.

17         f.write(uploaded_file.read())
        → Lit les octets du fichier reçu par Streamlit (qui les garde en mémoire, pas sur disque par défaut) et les écrit sur le disque local, car l'API OCR de Mistral (voir ocr_pdf) attend un chemin de fichier ouvrable.

19     if st.button("Run OCR"):
        → Affiche un bouton ; le bloc suivant ne s'exécute que si l'utilisateur clique dessus lors de ce cycle. Streamlit re-exécute tout le script à chaque interaction : sans ce bouton, l'OCR (couteux) se relancerait à chaque re-rendu de la page.

20         try:
            → Démarre un bloc protégé pour intercepter toute erreur pendant l'OCR.

21             with st.spinner("OCR en cours..."):
                → Affiche un indicateur de chargement tant que le bloc interne s'exécute.

22                 ocr_service.ocr_pdf(file_path)
                → Appelle la méthode qui envoie le PDF à l'API OCR de Mistral et sauvegarde le résultat.

23                 ocr_performed = True
                → Marque que l'OCR vient d'être effectué avec succès durant ce cycle d'exécution.

24         except Exception as e:
            → Capture toute exception survenue pendant l'appel OCR.

25             st.error(f"Error during OCR: {e}")
            → Affiche le message d'erreur à l'utilisateur dans l'interface.

27 if uploaded_file and ocr_service.ocr_is_valid(file_path) and st.checkbox("Show extracted OCR text"):
    → N'affiche la case à cocher (et son contenu) que si un fichier est uploadé et que le cache OCR correspond à ce fichier (voir ocr_is_valid dans legal_copilot.py, qui évite de montrer un texte OCR périmé).

28     try:
        → Démarre un bloc protégé pour la lecture du fichier Markdown OCRisé.

29         text = ocr_service.read_markdown_file("ocr_response.md")
        → Recharge le texte Markdown extrait par l'OCR depuis le disque.

30         st.markdown(text)
        → Affiche ce texte Markdown formaté directement dans la page.

31     except Exception:
        → Capture toute erreur de lecture du fichier (ex. fichier absent ou corrompu).

32         st.warning("Error reading OCR content.")
        → Affiche un avertissement à l'utilisateur en cas d'échec de lecture.

```

```
33 elif not uploaded_file:
    → Sinon, si aucun fichier n'a été uploadé du tout.

34     st.info("Upload a PDF file to enable OCR analysis.")
    → Invite l'utilisateur à uploader un PDF pour activer l'analyse.

36 st.markdown("### Document Summary")
    → Affiche un sous-titre de section pour la partie résumé du document.

37 if uploaded_file and ocr_service.ocr_is_valid(file_path) and st.button("Generate Summary"):
    → Affiche le bouton de résumé et n'exécute le bloc que si un fichier valide est présent et que le bouton est cliqué.

38     text = ocr_service.read_markdown_file("ocr_response.md")
    → Recharge le texte Markdown complet du document depuis le fichier OCRise.

39     prompt = f"Summarize this legal document clearly and concisely:\n\n{text}"
    → Construit un prompt demandant un résumé clair et concis, en y insérant le texte complet du document. À la différence de process_question (qui utilise le RAG pour ne sélectionner que 2 chunks pertinents), ici le document entier est envoyé d'un bloc car un résumé doit couvrir tout le contenu, pas seulement un passage.

40     try:
        → Démarre un bloc protégé pour l'appel de génération du résumé.

41         summary = ocr_service.run_mistral(prompt)
        → Envoie le prompt au modèle de chat Mistral et récupère le résumé généré.

42         st.markdown("#### Summary:")
        → Affiche un petit sous-titre avant le résumé.

43         st.markdown(summary)
        → Affiche le texte du résumé généré, avec mise en forme Markdown.

44     except Exception as e:
        → Capture toute erreur survenue pendant la génération du résumé.

45         st.error(f"Error during summarization: {e}")
        → Affiche le message d'erreur correspondant à l'utilisateur.

47 st.markdown("### Ask a question about the document content")
    → Affiche le sous-titre de la section questions-réponses.

48 question = st.text_input("Your question:")
    → Affiche un champ de saisie texte et stocke la question tapée par l'utilisateur.

50 if question:
    → N'exécute le bloc suivant que si l'utilisateur a effectivement saisi une question (chaîne non vide).

51     if not os.path.exists("ocr_response.md") or not ocr_service.ocr_is_valid(file_path):
        → Vérifie qu'un OCR valide existe déjà avant de tenter de répondre à une question.

52         st.error("No valid OCR data found. Please run OCR first.")
        → Si aucun OCR valide n'est disponible, affiche une erreur invitant à lancer l'OCR d'abord.

53     else:
        → Sinon, si un OCR valide existe bien pour ce fichier.

54         try:
            → Démarre un bloc protégé pour le traitement de la question.
```

```

55         response = ocr_service.process_question(question, file_path)
    → Appelle la methode RAG complete : decoupage, embeddings, recherche FAISS, puis generation de la reponse.

56         st.markdown("### Response")
    → Affiche un sous-titre avant la reponse generee.

57         st.markdown(response)
    → Affiche la reponse generee par le modele, avec mise en forme Markdown.

58     except Exception as e:
    → Capture toute erreur survenue pendant le traitement de la question.

59         st.error(f"Error while processing: {e}")
    → Affiche le message d'erreur correspondant a l'utilisateur.

```

9-Arize/app.py

Interface Streamlit qui pilote le meme moteur partage (policy_pipeline.py) avec une barre de progression et un export CSV.

78 lignes de code annotatees ci-dessous.

```

1 import streamlit as st
    → Importe Streamlit, la librairie utilisee pour construire l'interface web en pur Python.

3 from policy_pipeline import (
    → Commence l'import de plusieurs elements depuis le module moteur partage.

4     ARIZE_TRACING_ACTIF,
    → Importe le boolean indiquant si le tracing Arize est actif.

5     GROQ_API_KEY,
    → Importe la cle API Groq pour verifier sa presence avant de lancer le pipeline.

6     run_auto_evaluation,
    → Importe la fonction qui declenche l'auto-evaluation Phoenix.

7     run_multi_agent_pipeline,
    → Importe la fonction qui execute le pipeline multi-agents complet.

8 )
    → Ferme la liste des imports depuis policy_pipeline.

10 st.set_page_config(page_title="Politique Agricole - Multi-Agents", page_icon="", layout="wide")
    → Configure le titre d'onglet, l'icone et la mise en page large de la page Streamlit.

11 st.title("Analyse Multi-Agents de Politique Agricole")
    → Affiche le titre principal de la page.

12 st.caption(
    → Commence l'affichage d'un sous-titre explicatif discret sous le titre.

13     "Un Gestionnaire delegue a 3 agents specialises – Recherche (web), Analyse, Evaluation d'impact – "
    → Premiere partie du texte du sous-titre, decrivant l'architecture du pipeline.

14     "puis un second modele (Phoenix) juge independamment si le rapport final est impactant."
    → Suite et fin du texte du sous-titre, decrivant l'auto-evaluation.

```

15)
→ Ferme l'appel a st.caption.

17 if not GROQ_API_KEY:
→ Verifie si la cle API Groq est absente ou vide -- une verification faite tot, avant de laisser l'utilisateur lancer un pipeline qui echouerait de toute facon a son premier appel.

18 st.error("GROQ_API_KEY est introuvable. Ajoute-le dans le fichier .env a la racine du projet.")
→ Affiche un message d'erreur invitant a configurer la cle manquante.

19 st.stop()
→ Arrete immediatement l'execution du script Streamlit si la cle est absente.

21 with st.sidebar:
→ Ouvre un bloc de contenu affiche dans la barre laterale de l'application.

22 st.header("Parametres")
→ Affiche un en-tete de section dans la barre laterale.

23 if ARIZE_TRACING_ACTIF:
→ Verifie si le tracing Arize est actif.

24 st.success("Tracing Arize actif")
→ Affiche un message de succes si le tracing est actif.

25 else:
→ Sinon, si le tracing Arize n'est pas configure.

26 st.info("Tracing Arize desactive (ARIZE_SPACE_ID / ARIZE_API_KEY absents). Le pipeline fonctionne quand meme.")
→ Affiche un message informatif precisant que le pipeline reste fonctionnel sans tracing.

28 st.caption(
→ Commence l'affichage d'un avertissement discret sur la duree d'execution.

29 "Ce pipeline enchaîne plusieurs appels Groq (recherche web incluse) et peut prendre "
→ Premiere partie du texte d'avertissement sur la duree.

30 "***3 a 8 minutes**", selon la charge du quota gratuit Groq (8000 tokens/minute)."
→ Suite et fin du texte d'avertissement, donnant une estimation de duree.

31)
→ Ferme l'appel a st.caption dans la barre laterale.

33 DEFAULT_QUESTION = (
→ Commence la definition de la question par default proposee a l'utilisateur.

34 "Analyser l'etat actuel et les implications futures des politiques agricoles "
→ Premiere partie du texte de la question par default.

35 "britanniques et leur impact sur les communautes agricoles"
→ Suite et fin du texte de la question par default.

36)
→ Ferme la definition de DEFAULT_QUESTION.

38 saisie_utilisateur = st.text_area("Sujet a analyser", value=DEFAULT_QUESTION, height=100)
→ Affiche une zone de texte modifiable pre-remplie avec la question par default.

```

39 lancer = st.button("Lancer l'analyse multi-agents", use_container_width=True)
    → Affiche le bouton de lancement du pipeline, sur toute la largeur disponible.

41 if "resultat" not in st.session_state:
    → Verifie si l'etat de session Streamlit ne contient pas encore de resultat stocke. st.session_state est la memoire persistante de
    Streamlit entre deux re-executions du script (par default, toute variable normale serait reinitialisee a chaque interaction de
    l'utilisateur).

42     st.session_state.resultat = None
    → Initialise la variable de session resultat a None lors du premier chargement de la page.

44 if lancer:
    → N'execute le bloc suivant que si le bouton de lancement a ete clique lors de ce cycle.

45     if not saisie_utilisateur.strip():
        → Verifie si le champ de sujet est vide une fois les espaces superflus retires.

46         st.warning("Decris un sujet a analyser avant de lancer le pipeline.")
        → Affiche un avertissement si aucun sujet n'a ete saisi.

47     else:
        → Sinon, si un sujet a bien ete saisi.

48         progress_placeholder = st.empty()
        → Cree un emplacement vide reutilisable dans la page, destine a afficher la progression.

49         progress_placeholder.info("Demarrage du pipeline...")
        → Affiche un message initial de demarrage dans cet emplacement.

50         try:
            → Demarre un bloc protege pour intercepter toute erreur pendant l'execution du pipeline.

51             contenu_final = run_multi_agent_pipeline(
                → Lance l'execution du pipeline multi-agents complet.

52                 saisie_utilisateur,
                → Transmet le sujet saisi par l'utilisateur comme entree du pipeline.

53                 progress_callback=lambda label: progress_placeholder.info(label),
                → Fournit un callback qui met a jour l'emplacement de progression a chaque etape deleguee.

54             )
            → Ferme l'appel a run_multi_agent_pipeline.

55             progress_placeholder.info("Auto-evaluation du rapport (Phoenix)...")
            → Met a jour l'emplacement de progression pour indiquer le passage a l'auto-evaluation.

56             df_eval = run_auto_evaluation(saisie_utilisateur, contenu_final)
            → Lance l'auto-evaluation Phoenix sur le rapport final genere.

57             progress_placeholder.empty()
            → Vide l'emplacement de progression une fois le traitement termine.

58             st.session_state.resultat = {
                → Commence la construction du dictionnaire de resultats stocke en session.

59                 "saisie": saisie_utilisateur,

```

→ Stocke le sujet analyse dans le dictionnaire de resultats.

```
60         "contenu_final": contenu_final,
```

→ Stocke le rapport final genere dans le dictionnaire de resultats.

```
61         "label": df_eval["label"].iloc[0],
```

→ Extrait et stocke l'etiquette d'evaluation (premiere ligne du DataFrame).

```
62         "explanation": df_eval["explanation"].iloc[0],
```

→ Extrait et stocke l'explication d'evaluation correspondante.

```
63         "df": df_eval,
```

→ Stocke le DataFrame complet, reutilise plus bas pour l'export CSV.

```
64     }
```

→ Ferme le dictionnaire de resultats et l'affecte a st.session_state.resultat.

```
65     except Exception as e:
```

→ Capture toute exception survenue pendant l'execution du pipeline ou de l'evaluation.

```
66         progress_placeholder.empty()
```

→ Vide l'emplacement de progression meme en cas d'erreur.

```
67         st.error(f"Erreur pendant le pipeline : {e}")
```

→ Affiche le message d'erreur a l'utilisateur.

```
69 resultat = st.session_state.resultat
```

→ Relit le resultat stocke en session dans une variable locale, a chaque execution du script.

```
70 if resultat is None:
```

→ Verifie si aucun resultat n'a encore ete genere.

```
71     st.info("Decris un sujet puis clique sur **Lancer l'analyse multi-agents**.")
```

→ Affiche un message d'invite si aucun resultat n'existe encore.

```
72     st.stop()
```

→ Arrete l'execution du script ici tant qu'aucun resultat n'est disponible.

```
74 st.divider()
```

→ Affiche une ligne de separation visuelle horizontale.

```
76 label = resultat["label"]
```

→ Recupere l'etiquette d'evaluation du resultat stocke.

```
77 if label == "impactful":
```

→ Verifie si l'etiquette d'evaluation est positive.

```
78     st.success(f"Evaluation automatique : **{label}**")
```

→ Affiche l'etiquette en vert avec un message de succes si elle est positive.

```
79 else:
```

→ Sinon, si l'etiquette d'evaluation est negative.

```
80     st.warning(f"Evaluation automatique : **{label}**")
```

→ Affiche l'etiquette en orange avec un message d'avertissement si elle est negative.

```
81 st.markdown(f"**Justification : ** {resultat['explanation']}**")
```

→ Affiche l'explication textuelle justifiant la decision d'evaluation.

83 `st.download_button(`
→ Affiche un bouton permettant de telecharger les resultats.

84 `"Exporter les resultats en CSV",`
→ Definit le libelle affiche sur le bouton de telechargement.

85 `data=resultat["df"].to_csv(index=False).encode("utf-8-sig"),`
→ Convertit le DataFrame en CSV puis l'encode en UTF-8 avec BOM, pour une compatibilite correcte avec Excel (qui, sans le BOM, interprete parfois mal les accents dans un CSV UTF-8).

86 `file_name="evaluation_analyse_politique.csv",`
→ Definit le nom du fichier propose au telechargement.

87 `mime="text/csv",`
→ Precise le type MIME du fichier telecharge, pour que le navigateur le reconnaisse comme un CSV.

88 `)`
→ Ferme l'appel a `st.download_button`.

90 `st.divider()`
→ Affiche une seconde ligne de separation visuelle horizontale.

91 `st.markdown("### Rapport final")`
→ Affiche le sous-titre de la section du rapport final.

92 `st.markdown(resultat["contenu_final"])`
→ Affiche le texte complet du rapport final, avec mise en forme Markdown.

Projet 9 — 9-Arize

Pipeline multi-agents (Recherche -> Analyse -> Evaluation) avec auto-evaluation independante

Objectif

Analyse un sujet de politique publique via un pipeline de 3 agents specialises qui se relaient : un agent collecte des donnees recentes sur le web, un second les synthetise en rapport structure, et un troisieme juge si ce rapport est impactant. Un modele totalement independant rejuge ensuite le rapport final de maniere impartiale.

Code source, ligne par ligne

9-Arize/policy_pipeline.py

Moteur partage definissant les 3 agents specialises, le Team orchestrateur, et l'auto-evaluation Phoenix, reutilise par le CLI et par Streamlit.

288 lignes de code annotatees ci-dessous.

- 1 `"""Moteur partage : pipeline multi-agents (Recherche -> Analyse -> Evaluation)`
→ Premiere ligne de la docstring du module, qui decrit le role general du fichier.
- 2 `coordonne par un agno.team.Team, plus l'auto-evaluation Phoenix (llm_classify).`
→ Precise que l'orchestration se fait via un Team agno et que l'auto-evaluation utilise Phoenix -- ce sont les deux concepts cles de ce fichier, expliques en detail plus bas a leur premiere apparition.
- 4 `Utilise a la fois par le script CLI (evaluation_agent.py) et par l'interface`
→ Indique que ce moteur est partage entre deux points d'entree differents.
- 5 `Streamlit (app.py), pour eviter de dupliquer la definition des agents et la`
→ Continue la phrase precedente : justifie le partage de code.
- 6 `logique d'execution entre les deux.`
→ Termine la phrase justifiant le partage de code entre CLI et interface web.
- 7 `"""`
→ Ferme la docstring du module.
- 9 `import os`
→ Importe le module standard pour lire les variables d'environnement.
- 10 `import sys`
→ Importe le module standard donnant acces aux flux stdout/stderr du processus.
- 12 `# La console Windows utilise par default l'encodage cp1252, qui ne supporte pas`
→ Commentaire expliquant pourquoi la reconfiguration UTF-8 qui suit est necessaire.
- 13 `# certains caracteres Unicode que les logs internes d'Agno ou les reponses du`
→ Suite du commentaire : precise la source des caracteres problematiques.
- 14 `# modele peuvent contenir (ex. espace fine insecable " ") – ca faisait`
→ Suite du commentaire : donne un exemple concret de caractere posant probleme.
- 15 `# planter le script en plein milieu de l'execution.`
→ Fin du commentaire : decrit le symptome observe avant la correction.

16 for _stream in (sys.stdout, sys.stderr):

→ Demarre une boucle sur les deux flux de sortie standard et d'erreur du processus.

17 if hasattr(_stream, "reconfigure"):

→ Verifie que le flux dispose bien de la methode reconfigure (presente sur Python 3.7+).

18 _stream.reconfigure(encoding="utf-8", errors="replace")

→ Force l'encodage du flux en UTF-8 et remplace les caracteres non representables au lieu de planter.

20 from dotenv import load_dotenv

→ Importe la fonction qui charge les variables du fichier .env, ou sont stockees les cles API (Groq, Arize).

21 import pandas as pd

→ Importe pandas, une librairie de manipulation de tableaux de donnees, utilisee ici pour structurer les donnees envoyees a l'evaluation Phoenix (une ligne = un rapport a juger, des colonnes = input/sortie/attendu).

22 from textwrap import dedent

→ Importe la fonction qui retire l'indentation commune d'un bloc de texte multi-lignes -- pratique pour ecrire de longs textes d'instructions indentes proprement dans le code source sans que cette indentation ne se retrouve dans le texte final envoye au modele.

23 from agno.agent import Agent

→ Importe la classe Agent du framework agno. Un "Agent" y represente un assistant IA autonome : un modele de langage combine a des instructions, eventuellement des outils (recherche web, base de connaissances...) qu'il peut appeler lui-meme pour accomplir une tache, plutot qu'un simple echange question-reponse.

24 from agno.team import Team

→ Importe la classe Team d'agno. Un "Team" coordonne plusieurs Agents : il recoit une tache, decide quel agent membre est le mieux place pour s'en charger (ou enchaîne plusieurs agents dans un ordre precis), et recompose leurs reponses -- c'est le mecanisme utilise ici pour batir un pipeline Recherche -> Analyse -> Evaluation.

25 from agno.models.groq import Groq

→ Importe la classe wrapper qui connecte agno a l'API Groq, un fournisseur d'inference qui execute des modeles de langage open-source (dont celui utilise ici) tres rapidement, avec un tier gratuit limite en nombre de tokens par minute.

26 from agno.tools.duckduckgo import DuckDuckGoTools

→ Importe l'outil de recherche web DuckDuckGo utilisable par un agent : concretement, ca donne a l'agent une fonction qu'il peut appeler lui-meme pendant son raisonnement pour interroger le moteur de recherche et recuperer des resultats recents, plutot que de se limiter a ses connaissances figees au moment de son entraînement.

27 from arize.otel import register

→ Importe la fonction d'enregistrement OpenTelemetry fournie par Arize. OpenTelemetry est un standard d'observabilite : il capture des "traces" (qui a appele quoi, avec quels arguments, combien de temps ca a pris) pendant l'execution d'un programme. Arize Phoenix est une plateforme specialisee dans l'observabilite des applications IA : elle permet de visualiser ces traces pour comprendre et deboguer le comportement des agents.

28 from openinference.instrumentation.openai import OpenAIInstrumentor

→ Importe l'instrumenteur qui capture automatiquement les appels compatibles API OpenAI (dont Groq fait partie, voir plus bas) et les transforme en traces OpenTelemetry.

29 from openinference.instrumentation.agno import AgnoInstrumentor

→ Importe l'instrumenteur qui capture automatiquement les executions d'agents agno (quel agent a tourne, avec quel prompt, quelle reponse) et les transforme elles aussi en traces OpenTelemetry.

30 from phoenix.evals import OpenAIModel, llm_classify

→ Importe les outils Phoenix utilises pour faire juger un texte par un modele de langage. C'est le pattern "LLM-as-judge" (un modele de langage comme juge) : plutot que d'ecrire des regles manuelles pour evaluer la qualite d'un texte genere, on demande a un second modele de le classer selon un critere donne.

```

32 load_dotenv()
    → Charge les variables du fichier .env dans l'environnement du processus.

34 # ----- Configuration OpenTelemetry -----
    → Commentaire de section marquant le debut du bloc de configuration du tracing.

35 # register() exige des identifiants non vides (ValueError sinon) : le tracing Arize
    → Explique que la fonction register leve une erreur si les identifiants sont vides.

36 # est donc optionnel ici et ne s'active que si ARIZE_SPACE_ID/ARIZE_API_KEY sont definis,
    → Explique la condition sous laquelle le tracing est active.

37 # pour que le pipeline multi-agents reste utilisable sans compte Arize.
    → Justifie le choix : le pipeline doit fonctionner meme sans compte Arize configure -- l'observabilite est une fonctionnalite en plus, pas une dependance obligatoire pour que le coeur du pipeline tourne.

38 ARIZE_SPACE_ID = os.getenv("ARIZE_SPACE_ID")
    → Recupere l'identifiant d'espace Arize depuis l'environnement, ou None si absent.

39 ARIZE_API_KEY = os.getenv("ARIZE_API_KEY")
    → Recupere la cle API Arize depuis l'environnement, ou None si absente.

40 ARIZE_TRACING_ACTIF = bool(ARIZE_SPACE_ID and ARIZE_API_KEY)
    → Calcule un booleen vrai seulement si les deux identifiants Arize sont presents.

42 if ARIZE_TRACING_ACTIF:
    → N'active le tracing que si les identifiants Arize sont bien configures.

43     tracer_provider = register(
        → Demarre l'enregistrement du fournisseur de traces OpenTelemetry aupres d'Arize.

44         space_id=ARIZE_SPACE_ID,
        → Transmet l'identifiant d'espace Arize a la fonction register.

45         api_key=ARIZE_API_KEY,
        → Transmet la cle API Arize a la fonction register.

46         project_name="agent-politique-agricole",
        → Nomme le projet sous lequel les traces apparaitront dans le tableau de bord Arize.

47     )
    → Ferme l'appel a register et recupere l'objet tracer_provider.

48     AgnoInstrumentor().instrument(tracer_provider=tracer_provider)
    → Active l'instrumentation automatique des executions agno, connectee au fournisseur de traces.

49     OpenAIInstrumentor().instrument(tracer_provider=tracer_provider)
    → Active l'instrumentation automatique des appels compatibles API OpenAI, connectee au meme fournisseur.

51 # Le script original utilisait OpenAI (gpt-4o) ; remplace par Groq (compatible
    → Commentaire expliquant le remplacement du modele OpenAI original par Groq.

52 # avec l'API OpenAI) pour reutiliser la cle GROQ_API_KEY deja disponible.
    → Fin du commentaire : justifie le choix par la disponibilite de la cle Groq.

53 GROQ_API_KEY = os.getenv("GROQ_API_KEY")
    → Recupere la cle API Groq depuis l'environnement.

```

54 GROQ_MODEL_ID = "openai/gpt-oss-120b"

→ Definit le nom du modele Groq utilise par default par les agents specialises -- un modele de 120 milliards de parametres, plus capable mais aussi plus lent/couteux en tokens qu'une variante plus legere (voir plus bas).

56 # ----- Agent 1: Recherche -----

→ Commentaire de section marquant le debut de la definition de l'agent de recherche.

57 AgentRecherche = Agent(

→ Commence la creation de l'agent charge de collecter des donnees via le web.

58 name="AgentRecherche",

→ Donne un nom identifiant a cet agent, utilise notamment par le Team pour le designer.

59 model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),

→ Attache a l'agent le modele Groq 120B configure plus haut : c'est le "cerveau" qui va lire les instructions ci-dessous et decider quoi faire (dont quand appeler l'outil de recherche web).

60 tools=[DuckDuckGoTools()],

→ Donne a l'agent l'acces a l'outil de recherche web DuckDuckGo : pendant son raisonnement, le modele peut decider d'appeler cette fonction avec une requete de recherche, recevoir les resultats, et continuer a raisonner avec ces resultats -- c'est ce qui permet a l'agent d'aller chercher de l'information au-dela de ce qu'il "connait" deja.

61 description=dedent("""

→ Commence la description du role de l'agent, avec dedent pour retirer l'indentation du bloc.

62 Vous etes un assistant de recherche politique specialise dans la collecte de donnees recentes et credibles sur :

→ Decrit le role general de l'agent : collecter des donnees recentes et credibles.

63 - Les politiques agricoles du Royaume-Uni, les programmes de subventions, les schemas de developpement rural et les reglementations environnementales

→ Precise le perimetre thematique exact que l'agent doit couvrir.

64 - Votre objectif est de recuperer des informations exploitables et des rapports verifies a partir de :

→ Introduit la liste des types de sources a utiliser.

66 - Publications DEFRA et NFU

→ Cite une premiere categorie de source officielle a privilegier.

67 - Organismes de recherche scientifique et groupes de reflexion politique

→ Cite une deuxieme categorie de source a privilegier.

68 - Journaux commerciaux et journaux reputes du Royaume-Uni

→ Cite une troisieme categorie de source a privilegier.

69 - Plateformes gouvernementales officielles (.gov.uk)

→ Cite une quatrieme categorie de source a privilegier.

70 """),

→ Ferme le bloc de texte de la description et l'appel a dedent.

71 instructions=dedent("""

→ Commence le bloc d'instructions detaillees donnees a l'agent -- la "description" dit QUI est l'agent, les "instructions" disent COMMENT il doit s'y prendre concretement, etape par etape.

72 1. Decouverte des Sources

→ Titre de la premiere etape de la methode de travail de l'agent.

73 - Rechercher les documents, rapports et articles les plus recents lies a la legislation agricole britannique.
→ Detaille la premiere sous-tache de l'etape "Decouverte des Sources".

74 - Prioriser les donnees de DEFRA, etudes academiques, NFU et sources politiques de confiance.
→ Detaille la seconde sous-tache : prioriser certaines sources fiables.

76 2. Filtrage des Informations
→ Titre de la deuxieme etape de la methode de travail.

77 - Eliminer le contenu duplique ou obsolete.
→ Detaille la premiere sous-tache de filtrage.

78 - Se concentrer sur les changements politiques les plus recents et regionalement pertinents.
→ Detaille la seconde sous-tache de filtrage.

80 3. Extraction du Contexte Brut
→ Titre de la troisieme etape de la methode de travail.

81 - Extraire les faits cles, mises a jour politiques et contexte pertinent en texte brut.
→ Detaille la tache d'extraction du contexte.

82 - Ne pas effectuer d'analyse - se concentrer uniquement sur la recuperation d'informations.
→ Precise une limite stricte : cet agent ne doit pas analyser, seulement collecter -- cette separation des responsabilites (un agent collecte, un autre analyse) est le principe meme du pipeline multi-agents.

84 Limite-toi a 4-5 recherches cibles maximum, puis redige la synthese avec ce que tu as trouve.
→ Impose une limite explicite du nombre de recherches, pour eviter une execution trop longue.

85 Ne cherche pas l'exhaustivite : mieux vaut une synthese utile rapidement qu'une recherche infinie.
→ Renforce l'instruction precedente en justifiant le compromis rapidite/exhaustivite.

86 """),
→ Ferme le bloc d'instructions et l'appel a dedent.

87 expected_output=dedent("""
→ Commence la definition du format de sortie attendu de l'agent -- donner un gabarit precis au modele augmente la chance qu'il produise une reponse structuree et exploitable par les etapes suivantes du pipeline.

88 # {Resume de la Politique Agricole du Royaume-Uni Contexte Brut}
→ Modele du titre principal attendu dans la sortie Markdown de l'agent.

90 ## Donnees Collectees
→ Modele d'un sous-titre attendu, introduisant la liste des donnees collectees.

91 - **Source 1:** {...}
→ Emplacement type pour la premiere source citee, a remplir par l'agent.

92 - **Source 2:** {...}
→ Emplacement type pour la deuxieme source citee.

93 - **Source 3:** {...}
→ Emplacement type pour la troisieme source citee.

95 ## Politiques et Programmes Pertinents
→ Modele d'un sous-titre attendu pour la liste des politiques pertinentes.

96 - Politique A: {...}
→ Emplacement type pour la premiere politique decrite.

97 - Politique B: {...}
→ Emplacement type pour la deuxieme politique decrite.

98 - Schema C: {...}
→ Emplacement type pour un schema ou programme decrit.

100 ## Notes Environnementales et Economiques
→ Modele d'un sous-titre attendu pour les notes environnementales/economiques.

101 - Mises a jour des initiatives climatiques: {...}
→ Emplacement type pour les mises a jour climatiques.

102 - Tendances des revenus agricoles: {...}
→ Emplacement type pour les tendances de revenus.

104 ---
→ Separateur visuel Markdown avant la ligne de signature du rapport.

105 Compile par l'Agent de Recherche - Derniere Verification: {heure_actuelle}
→ Modele de ligne de signature indiquant l'auteur et l'heure de generation.

106 """),
→ Ferme le bloc expected_output et l'appel a dedent.

107 markdown=True,
→ Indique a agno que la sortie de l'agent doit etre interpretee/rendue comme du Markdown.

108 debug_mode=True,
→ Active les logs de debogage detailles pour cet agent.

109 # Les resultats bruts de web_search (plusieurs recherches x 10 resultats JSON
→ Commentaire expliquant pourquoi la compression des resultats d'outils est necessaire.

110 # complets) font rapidement gonfler le contexte au-dela de la limite Groq
→ Suite du commentaire : precise la cause du depassement de contexte.

111 # (8000 tokens/minute en tier gratuit) : on les compresse avant qu'ils ne
→ Suite du commentaire : donne le chiffre exact de la limite Groq concerne.

112 # s'accumulent dans la conversation.
→ Fin du commentaire justifiant compress_tool_results.

113 compress_tool_results=True,
→ Active la compression automatique des resultats d'outils avant qu'ils n'entrent dans le contexte du modele : concretement, agno resume/tronque les reponses volumineuses de l'outil de recherche web avant de les repasser au modele, pour rester sous la limite de tokens autorisee par minute par le fournisseur.

114 # Sans limite, l'agent peut enchaîner des recherches web quasi indefiniment sur
→ Commentaire expliquant le probleme observe sans limite de recherches.

115 # un sujet aussi vaste (observe : 20+ min sans jamais conclure). On plafonne le
→ Suite du commentaire : donne le temps d'execution observe sans plafond.

116 # nombre d'appels d'outils pour garantir un temps d'execution raisonnable.
→ Fin du commentaire justifiant tool_call_limit.

117 tool_call_limit=6,

→ Limite l'agent a 6 appels d'outils maximum par execution, une garde-fou simple contre une boucle de recherche qui ne se termine jamais.

118)

→ Ferme la construction de l'objet AgentRecherche.

120 # ----- Agent 2: Analyse -----

→ Commentaire de section marquant le debut de la definition de l'agent d'analyse.

121 AgentAnalyse = Agent(

→ Commence la creation de l'agent charge de synthetiser les donnees brutes en analyse structuree.

122 name="AgentAnalyse",

→ Donne un nom identifiant a cet agent.

123 model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),

→ Attache a l'agent le modele Groq 120B. Contrairement a AgentRecherche, cet agent n'a pas d'outil (pas de tools=[...]) : il n'est pas censé aller chercher de nouvelles informations, seulement raisonner sur ce que AgentRecherche lui aura transmis.

124 description=dedent("""

→ Commence la description du role de l'agent d'analyse.

125 Vous etes un analyste senior de politique agricole qui synthetise des evaluations structurees

→ Decrit le role general : analyste senior produisant des evaluations structurees.

126 basees sur des documents politiques et des donnees d'intervenants. Vous fournissez des perspectives

→ Precise la base de travail : documents politiques et donnees d'intervenants.

127 completes, factuelles et regionalement conscientes couvrant :

→ Introduit la liste des axes que l'analyse doit couvrir.

129 - Implications politiques sur l'agriculture et la biodiversite

→ Premier axe d'analyse attendu.

130 - Evaluations d'impact economique

→ Deuxieme axe d'analyse attendu.

131 - Recommandations strategiques pour le gouvernement et les parties prenantes

→ Troisieme axe d'analyse attendu.

132 """),

→ Ferme le bloc description.

133 instructions=dedent("""

→ Commence le bloc d'instructions detaillees de l'agent d'analyse.

134 1. Revision Politique

→ Titre de la premiere etape de la methode de travail.

135 - Analyser les donnees brutes et identifier les principaux moteurs politiques.

→ Detaille la premiere sous-tache de revision.

136 - Relier les conclusions a la productivite agricole, au bien-etre des exploitations et a la durabilite.

→ Detaille la seconde sous-tache de revision.

138 2. Evaluation des Resultats

→ Titre de la deuxieme etape de la methode de travail.

139 - Quantifier l'impact sur les communautés rurales et les chaînes d'approvisionnement alimentaire.

→ Détaille la première sous-tâche d'évaluation.

140 - Intégrer les tendances environnementales, commerciales et économiques.

→ Détaille la seconde sous-tâche d'évaluation.

142 3. Recommandations

→ Titre de la troisième étape de la méthode de travail.

143 - Proposer des plans d'action multi-échelles (immédiat, moyen terme, long terme).

→ Détaille la première sous-tâche de recommandation.

144 - Structurer les suggestions selon le type d'exploitation, la géographie et la zone prioritaire.

→ Détaille la seconde sous-tâche de recommandation.

146 4. Format et Clarté

→ Titre de la quatrième étape de la méthode de travail.

147 - Utiliser un formatage Markdown clair avec des en-têtes structurés.

→ Détaille la première exigence de mise en forme.

148 - Assurer un flux logique de l'observation à la recommandation.

→ Détaille la seconde exigence de mise en forme.

149 ""),

→ Ferme le bloc instructions.

150 expected_output=dedent("""

→ Commence la définition du format de sortie attendu de l'agent d'analyse.

151 # {Analyse de Politique Agricole du Royaume-Uni}

→ Modèle du titre principal attendu.

153 ## Resume Executif

→ Modèle de sous-titre pour le résumé exécutif.

154 {Aperçu concis du paysage agricole actuel et des défis clés}

→ Emplacement type pour le contenu du résumé exécutif.

156 | Region | Types d'Exploitations | Enjeux Clés | Schémas de Soutien |

→ Ligne d'en-tête du tableau Markdown attendu dans le rapport.

157 |-----|-----|-----|-----|

→ Ligne de séparation Markdown définissant les colonnes du tableau.

158 | Angleterre| ... | ... | ... |

→ Ligne d'exemple de contenu attendu pour l'Angleterre.

159 | Pays de Galles | ... | ... | ... |

→ Ligne d'exemple de contenu attendu pour le Pays de Galles.

160 | ... | ... | ... | ... |

→ Ligne indiquant que d'autres régions doivent suivre le même format.

162 ## Conclusions Cles
→ Modele de sous-titre pour les conclusions cles.

163 - ****Impact Environnemental:**** {...}
→ Emplacement type pour la conclusion sur l'impact environnemental.

164 - ****Viabilite Economique:**** {...}
→ Emplacement type pour la conclusion sur la viabilite economique.

165 - ****Developpement Rural:**** {...}
→ Emplacement type pour la conclusion sur le developpement rural.

167 ## Analyse de Marche
→ Modele de sous-titre pour l'analyse de marche.

168 {Tendances actuelles de l'agriculture britannique et implications commerciales
internationales}
→ Emplacement type pour le contenu de l'analyse de marche.

170 ## Recommandations
→ Modele de sous-titre pour les recommandations.

171 1. ****Actions Immediates:**** {...}
→ Emplacement type pour les actions immediates recommandees.

172 2. ****Strategie a Moyen Terme:**** {...}
→ Emplacement type pour la strategie a moyen terme.

173 3. ****Vision a Long Terme:**** {...}
→ Emplacement type pour la vision a long terme.

175 ## Sources de Donnees
→ Modele de sous-titre pour la liste des sources utilisees.

176 {Liste numerotee des references avec dates et pertinence}
→ Emplacement type pour la liste des sources et references.

178 ---
→ Separateur visuel avant la ligne de signature.

179 Prepare par l'Analyste de Politique Agricole - Publie : {date_actuelle} - Derniere Mise a Jour
: {heure_actuelle}
→ Modele de ligne de signature avec dates de publication et de mise a jour.

180 """),
→ Ferme le bloc expected_output et l'appel a dedent.

181 markdown=True,
→ Indique que la sortie de cet agent doit etre traitee comme du Markdown.

182)
→ Ferme la construction de l'objet AgentAnalyse.

184 # ----- Agent 3: Evaluation -----
→ Commentaire de section marquant le debut de la definition de l'agent evaluateur.

185 AgentEvalueur = Agent(
→ Commence la creation de l'agent charge de juger si le rapport final est impactant.

186 name="AgentEvalueur",

→ Donne un nom identifiant a cet agent.

```
187     model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),
```

→ Attache a l'agent le modele Groq 120B.

```
188     description=dedent("""
```

→ Commence la description du role de l'agent evaluateur.

```
189         Vous etes un auditeur d'impact politique évaluant si un rapport de politique agricole
britannique
```

→ Decrit le role general : auditeur d'impact politique.

```
190         repond de maniere significative aux besoins des "communautes agricoles". Vous evaluez si
l'analyse
```

→ Precise le critere central de l'evaluation : la reponse aux besoins des communautes agricoles.

```
191         est impactante basee sur :
```

→ Introduit la liste des criteres d'evaluation.

```
193         - Pertinence et couverture des defis agricoles actuels
```

→ Premier critere d'evaluation cite.

```
194         - Profondeur des perspectives sur les resultats economiques et environnementaux
```

→ Deuxieme critere d'evaluation cite.

```
195         - Qualite et realisme des recommandations politiques proposees
```

→ Troisieme critere d'evaluation cite.

```
196     """),
```

→ Ferme le bloc description.

```
197     instructions=dedent("""
```

→ Commence le bloc d'instructions detaillees de l'agent evaluateur.

```
198         1. Tache d'Evaluation
```

→ Titre de la premiere etape de la methode de travail.

```
199         - Evaluer l'utilite globale du rapport politique.
```

→ Detaille la premiere sous-tache d'evaluation.

```
200         - Comparer le contenu aux preoccupations agricoles du monde reel.
```

→ Detaille la seconde sous-tache d'evaluation.

```
202         2. Decision d'Etiquetage
```

→ Titre de la deuxieme etape de la methode de travail.

```
203         - Utiliser uniquement "impactant" ou "non impactant" comme etiquette finale.
```

→ Impose les deux seules etiquettes possibles pour la decision finale -- forcer un choix binaire simplifie grandement l'exploitation automatique de cette sortie en aval (voir run_auto_evaluation plus bas).

```
204         - Etre strict : n'attribuer "impactant" que si l'analyse est approfondie et axee sur la
communaute.
```

→ Impose un critere strict pour attribuer l'etiquette positive.

```
206         3. Explication du Raisonnement
```

→ Titre de la troisieme etape de la methode de travail.

```
207         - Apres l'etiquetage, rediger une explication etape par etape.
```

→ Demande une explication detaillee apres la decision d'etiquetage.

208 - Souligner ce qui rend le rapport fort ou faible d'un point de vue d'impact politique.
→ Precise le contenu attendu de cette explication.

210 La sortie doit commencer par un seul mot : impactant ou non impactant. Aucun autre caractere sur la premiere ligne.
→ Impose un format de sortie strict pour que la premiere ligne soit facilement extractible par du code, sans avoir besoin d'un parsing complexe.

211 `"""),`
→ Ferme le bloc instructions.

212 `expected_output=dedent("""`
→ Commence la definition du format de sortie attendu de l'agent evaluateur.

213 `impactant`
→ Exemple du mot-cle attendu en toute premiere ligne de la sortie.

215 L'analyse demontre profondeur et pertinence. Elle couvre la biodiversite, l'economie rurale et les risques de chaine d'approvisionnement.
→ Exemple de phrase de justification suivant le mot-cle.

216 La structure s'aligne avec les formats politiques et inclut des recommandations basees sur des donnees.
→ Suite de l'exemple de justification.

217 Les differences regionales et les impacts socioeconomiques sont correctement abordes.
→ Fin de l'exemple de justification.

219 `---`
→ Separateur visuel avant la ligne de signature.

220 `Evalue par l'Auditeur d'Impact Politique - Verifie : {date_actuelle}`
→ Modele de ligne de signature avec la date de verification.

221 `"""),`
→ Ferme le bloc `expected_output` et l'appel a `dedent`.

222 `markdown=True,`
→ Indique que la sortie de cet agent doit etre traitee comme du Markdown.

223 `)`
→ Ferme la construction de l'objet `AgentEvalueur`.

225 # ----- Agent 4: Gestionnaire -----
→ Commentaire de section marquant le debut de la definition du Team orchestrateur.

226 # Le script original passait AgentRecherche/AgentAnalyse/AgentEvalueur directement
→ Commentaire expliquant l'ancienne approche (pattern agent-comme-outil) desormais obsolete.

227 # dans tools=[...] d'un Agent (pattern "agent-comme-outil"). Ce pattern n'existe plus
→ Suite du commentaire : precise que ce pattern n'est plus supporte par la version installee d'agno.

228 # dans la version installee d'agno (2.8.7) : un Agent ne delegue plus automatiquement
→ Suite du commentaire : precise la version d'agno concerne.

229 # a d'autres Agents passes en tools, il se contentait d'halluciner un faux plan en
→ Suite du commentaire : decrit le symptome observe avec l'ancien pattern.

230 # texte sans jamais les appeler. Le remplacement officiel pour orchestrer plusieurs

→ Suite du commentaire : introduit la solution de remplacement.

231 # agents est agno.team.Team, qui delegue reellement a ses "members".

→ Fin du commentaire : nomme la classe Team comme solution retenue.

232 AgentGestionnaire = Team(

→ Commence la creation du Team qui orchestre reellement les 3 agents specialises. Un Team est lui-meme pilote par un modele de langage (son "leader") qui decide, a chaque etape, a quel membre deleguer la suite de la tache -- c'est cette couche de decision qui manque a un simple Agent isole.

233 name="AgentGestionnaire",

→ Donne un nom identifiant a ce Team.

234 # Le leader du Team doit relire/synthetiser en un seul appel les rapports complets

→ Commentaire expliquant pourquoi un modele plus leger est utilise pour le leader du Team.

235 # de AgentRecherche + AgentAnalyse (souvent >8000 tokens a eux deux), ce qui depasse

→ Suite du commentaire : quantifie le volume de tokens combine depassant la limite.

236 # la limite Groq du modele 120B (8000 tokens/minute en tier gratuit). Le modele 20B,

→ Suite du commentaire : cite le chiffre exact de la limite Groq concerne.

237 # plus leger, dispose d'un quota TPM plus eleve sur ce meme tier et suffit pour ce

→ Suite du commentaire : explique pourquoi le modele 20B est mieux adapte ici.

238 # role de routage/synthese (les agents specialises gardent le modele 120B).

→ Fin du commentaire : precise que seuls les agents specialises gardent le modele 120B.

239 model=Groq(id="openai/gpt-oss-20b", api_key=GROQ_API_KEY),

→ Attache au Team le modele Groq 20B, une version plus legere (moins de parametres, donc moins couteuse en tokens et plus rapide, au prix d'un raisonnement moins fin) que le 120B utilise par les agents specialises -- un compromis delibere car le role du leader (router les taches, synthetiser) est moins exigeant que celui de redaction fine des agents.

240 members=[AgentRecherche, AgentAnalyse, AgentEvalueur],

→ Declare les 3 agents specialises comme membres que le Team peut deleguer.

241 description="Un agent orchestrateur coordonnant le flux d'evaluation complet de la politique agricole britannique.",

→ Decrit en une phrase le role global du Team.

242 instructions=dedent("""

→ Commence le bloc d'instructions donnees au Team pour orchestrer les delegations.

243 Etapes :

→ Introduit la liste des etapes que le Team doit suivre.

244 1. Appeler AgentRecherche pour rassembler le contexte pertinent de la politique agricole britannique.

→ Premiere etape imposee au Team : deleguer d'abord a AgentRecherche.

245 2. Transmettre le contexte brut a AgentAnalyse pour produire une analyse structuree.

→ Deuxieme etape imposee au Team : deleguer ensuite a AgentAnalyse.

246 3. Demander a AgentEvalueur d'evaluer l'impact du rapport politique.

→ Troisieme etape imposee au Team : deleguer enfin a AgentEvalueur.

248 Ne pas generer de texte vous-meme. Router chaque tache vers l'agent correct.

→ Instruction cruciale interdisant au Team de rediger lui-meme une reponse au lieu de deleguer -- sans cette consigne explicite, le modele leader tend a repondre directement plutot que d'appeler ses membres.

249 """),
→ Ferme le bloc instructions et l'appel a dedent.

250 expected_output="Rapport politique complet suivi d'une etiquette d'evaluation d'impact formelle et
d'une explication.",
→ Decrit en une phrase le format global de sortie attendu du Team.

251 markdown=True,
→ Indique que la sortie finale du Team doit etre traitee comme du Markdown.

252 debug_mode=True,
→ Active les logs de debogage detailles pour le Team.

253 # Idem : le Team accumule les sorties completes de AgentRecherche + AgentAnalyse
→ Commentaire reexpliquant pourquoi la compression des resultats d'outils est necessaire au niveau du Team.

254 # pour synthetiser le rapport final, ce qui a fait depasser la limite Groq
→ Suite du commentaire : precise la consequence observee sans compression.

255 # (8774 tokens demandes pour 8000 autorises/minute) lors d'un premier test.
→ Fin du commentaire : donne les chiffres exacts observes lors d'un test reel.

256 compress_tool_results=True,
→ Active la compression des resultats d'outils au niveau du Team egalement.

257)
→ Ferme la construction de l'objet AgentGestionnaire (le Team).

259 MEMBER_LABELS = {
→ Debut un dictionnaire associant l'identifiant technique de chaque membre a un libelle lisible pour l'interface.

260 "agentrecherche": "Recherche en cours (web) – AgentRecherche...",
→ Associe l'identifiant "agentrecherche" a son libelle de progression affiche a l'utilisateur.

261 "agentanalyse": "Analyse structuree en cours – AgentAnalyse...",
→ Associe l'identifiant "agentanalyse" a son libelle de progression.

262 "agentevalateur": "Evaluation d'impact en cours – AgentEvalueur...",
→ Associe l'identifiant "agentevalateur" a son libelle de progression.

263 }
→ Ferme le dictionnaire MEMBER_LABELS.

266 def run_multi_agent_pipeline(saisie_utilisateur: str, progress_callback=None) -> str:
→ Definit la fonction publique qui execute le pipeline complet et retourne le contenu final.

267 """"Execute le pipeline Recherche -> Analyse -> Evaluation via le Team, en
→ Premiere ligne de la docstring de la fonction, decrivant son role general.

268 streamant les evenements pour suivre la progression reelle (quel agent est
→ Suite de la docstring : explique le mecanisme de suivi de progression.

269 en train de tourner), et retourne le contenu final synthetise.
→ Fin de la premiere partie de la docstring : precise la valeur de retour.

271 progress_callback(label: str), si fourni, est appele a chaque fois qu'un
→ Documente le parametre progress_callback et quand il est invoque.

272 nouveau membre du Team est delegue.
→ Fin de la documentation du parametre progress_callback.

273 """
→ Ferme la docstring de la fonction.

274 final_content = None
→ Initialise la variable qui contiendra la reponse finale, a None tant qu'elle n'est pas trouvee.

275 for event in AgentGestionnaire.run(saisie_utilisateur, stream=True, stream_events=True):
→ Lance l'execution du Team en mode streaming et itere sur chaque evenement emis au fil de l'execution. Le "streaming" consiste a recevoir la reponse (et ici, les evenements intermediaires) au fur et a mesure qu'elle est produite, plutot que d'attendre que tout le pipeline soit termine -- c'est ce qui permet ensuite d'afficher une barre de progression en temps reel plutot qu'un simple ecran de chargement bloquant.

276 tool = getattr(event, "tool", None)
→ Recupere l'attribut "tool" de l'evenement s'il existe, sinon None, sans lever d'erreur.

277 if tool is not None and tool.tool_name == "delegate_task_to_member" and progress_callback:
→ Verifie si cet evenement correspond a une delegation de tache a un membre, et qu'un callback est fourni.
"delegate_task_to_member" est le nom interne de l'outil qu'agno donne au Team pour deleguer a un membre : en surveillant cet evenement precis, on peut savoir en temps reel quel agent le Team vient de choisir.

278 member_id = (tool.tool_args or {}).get("member_id", "")
→ Extrait l'identifiant du membre concerne par cette delegation depuis les arguments de l'outil.

279 progress_callback(MEMBER_LABELS.get(member_id, f"{member_id} en cours..."))
→ Appelle le callback de progression avec le libelle lisible correspondant a ce membre (ou un libelle generique).

281 # "TeamRunCompleted" (avec le prefixe Team) n'est emis que par le Team
→ Commentaire expliquant comment distinguer l'evenement final du Team des evenements des membres.

282 # lui-meme, jamais par les membres delegates (qui emettent "RunCompleted")
→ Suite du commentaire : precise que les membres emettent un evenement au nom different.

283 # sans prefixe) : ca suffit a isoler la reponse finale synthetisee.
→ Fin du commentaire : conclut que ce nom d'evenement suffit a isoler la reponse finale.

284 if getattr(event, "event", None) == "TeamRunCompleted" and getattr(event, "content", None):
→ Verifie si l'evenement courant est bien l'evenement final du Team et qu'il contient du contenu.

285 final_content = event.content
→ Stocke le contenu de cet evenement comme reponse finale du pipeline.

287 return final_content or "(Aucune reponse generee.)"
→ Retourne le contenu final trouve, ou un message par default si aucun contenu n'a ete capture.

290 MODELE_EVAL_ROUTER = """
→ Commence la definition du gabarit de prompt utilise pour l'auto-evaluation Phoenix.

291 Vous etes un bot d'aide IA qui verifie l'impact de l'analyse de politique agricole britannique sur les
communautes agricoles. Votre tache est d'evaluer si l'analyse fournie est importante.
→ Decrit le role attendu du modele qui va executer ce gabarit de jugement -- c'est le "juge" du pattern LLM-as-judge introduit plus haut, un modele distinct de ceux du Team, dont le seul travail est d'evaluer.

293 Voici les donnees :
→ Introduit la section contenant les donnees a evaluer.

294 [DEBUT DONNEES]

→ Marqueur textuel signalant le debut du bloc de donnees insere dans le prompt.

295 *****

→ Separateur visuel dans le gabarit de prompt.

296 [Input]: Ci-dessous se trouve l'entree qui contient la saisie utilisateur et l'historique de conversation.

→ Introduit l'emplacement ou sera insere le texte de la requete utilisateur d'origine.

297 {input}

→ Emplacement de substitution (placeholder) ou le texte de la requete utilisateur sera injecte.

299 [Sortie Reelle]: Ci-dessous se trouve la sortie reelle generee par l'agent.

→ Introduit l'emplacement ou sera insere le rapport final reellement genere par le pipeline.

300 {actual_output}

→ Emplacement de substitution ou le rapport final genere sera injecte.

302 [Sortie Attendue]: Ci-dessous se trouve le format de sortie attendu qui devrait contenir une analyse des politiques agricoles britanniques et leur impact sur les communauts agricoles.

→ Introduit l'emplacement ou sera insere le format de sortie attendu defini pour AgentAnalyse.

303 {expected_output}

→ Emplacement de substitution ou le format de sortie attendu sera injecte.

304 *****

→ Second separateur visuel dans le gabarit de prompt.

305 [FIN DONNEES]

→ Marqueur textuel signalant la fin du bloc de donnees insere dans le prompt.

307 Determiner si l'analyse est impactante ou non impactante base sur l'entree, la sortie reelle et le format attendu.

→ Instruction finale demandant explicitement au modele juge de rendre son verdict.

308 """

→ Ferme la chaine de caracteres du gabarit MODELE_EVAL_ROUTER.

311 def run_auto_evaluation(saisie_utilisateur: str, contenu_final: str) -> pd.DataFrame:

→ Definit la fonction publique qui declenche l'auto-evaluation Phoenix et retourne un DataFrame de resultats.

312 """Fait juger le rapport final par un second modele (Phoenix llm_classify),

→ Premiere ligne de la docstring : decrit le mecanisme de jugement independant utilise.

313 independamment de AgentEvalueur, et retourne le DataFrame de resultat

→ Suite de la docstring : precise que ce jugement est independant de AgentEvalueur -- l'interet d'avoir un second juge distinct du Team lui-meme est d'obtenir un avis moins susceptible d'etre biaise en faveur du propre travail de l'equipe (un agent qui s'auto-evalue a tendance a se noter genereusement).

314 (colonnes 'label' et 'explanation' notamment).

→ Fin de la docstring : precise le contenu du DataFrame retourne.

315 """

→ Ferme la docstring de la fonction.

316 donnees_reponse = {

→ Commence la construction du dictionnaire de donnees qui servira a creer le DataFrame pandas.

```

317         "input": [saisie_utilisateur],
    → Place la requete utilisateur d'origine dans une liste a un seul element (format attendu par pandas).

318         "actual_output": [contenu_final],
    → Place le rapport final genere dans une liste a un seul element.

319         "expected_output": [AgentAnalyse.expected_output],
    → Place le format de sortie attendu defini sur AgentAnalyse, utilise comme reference de comparaison.

320     }
    → Ferme le dictionnaire donnees_reponse.

321     df_reponse = pd.DataFrame(donnees_reponse)
    → Convertit le dictionnaire en DataFrame pandas d'une seule ligne.

323     def format_template(row):
    → Definit une fonction interne qui construit le prompt final pour une ligne du DataFrame.

324         return MODELE_EVAL_ROUTER.format(
    → Demarre le remplissage du gabarit de prompt avec les valeurs de la ligne.

325             input=row["input"],
    → Insere la valeur de la colonne "input" de la ligne dans le placeholder {input}.

326             actual_output=row["actual_output"],
    → Insere la valeur de la colonne "actual_output" dans le placeholder {actual_output}.

327             expected_output=row["expected_output"],
    → Insere la valeur de la colonne "expected_output" dans le placeholder {expected_output}.

328         )
    → Ferme l'appel a .format() et retourne le prompt final complete.

330     df_reponse["formatted_prompt"] = df_reponse.apply(format_template, axis=1)
    → Applique format_template a chaque ligne du DataFrame et stocke le resultat dans une nouvelle colonne.

332     return llm_classify(
    → Demarre l'appel a la fonction Phoenix qui fait juger les donnees par un modele de langage.

333         dataframe=df_reponse,
    → Fournit le DataFrame contenant les donnees et le prompt formate a llm_classify.

334         template=MODELE_EVAL_ROUTER,
    → Fournit le gabarit de prompt utilise pour structurer la demande au modele juge.

335         model=OpenAIModel(
    → Commence la configuration du modele qui va effectuer le jugement, via l'interface compatible OpenAI de Phoenix.

336             api_key=GROQ_API_KEY,
    → Utilise la cle API Groq pour authentifier ce modele de jugement.

337             base_url="https://api.groq.com/openai/v1",
    → Pointe vers l'endpoint compatible OpenAI expose par Groq, plutot que vers OpenAI directement -- Groq reproduit
    volontairement le format d'API d'OpenAI, ce qui permet de reutiliser des librairies comme OpenAIModel de Phoenix sans jamais
    toucher a un compte OpenAI.

338             model=GROQ_MODEL_ID,
    → Precise le modele Groq 120B a utiliser pour le jugement.

339         ),

```

→ Ferme la configuration de l'objet OpenAIModel.

```
340 rails=["not impactful", "impactful"],
```

→ Definit les deux seules etiquettes possibles que le modele juge peut attribuer.

```
341 provide_explanation=True,
```

→ Demande au modele juge de fournir egalement une explication textuelle de sa decision.

```
342 concurrency=1,
```

→ Limite l'execution a un seul appel concurrent, pour rester dans les limites de debit de l'API.

```
343 )
```

→ Ferme l'appel a llm_classify et retourne son resultat (un DataFrame avec les colonnes label/explanation).

9-Arize/evaluation_agent.py

Script CLI qui lance le pipeline multi-agents puis l'auto-evaluation, et affiche/sauvegarde les resultats en console.

26 lignes de code annotatees ci-dessous.

```
1 """Script CLI : lance le pipeline multi-agents (Recherche -> Analyse -> Evaluation)
```

→ Premiere ligne de la docstring du module, decrivant son role de point d'entree CLI.

```
2 puis l'auto-evaluation Phoenix, et affiche/sauvegarde les resultats.
```

→ Suite de la docstring : precise les deux etapes executees et leur sortie.

```
4 Toute la logique (agents, Team, appels API) vit dans policy_pipeline.py, partagee
```

→ Precise que toute la logique metier est deleguee au module partage.

```
5 avec l'interface Streamlit (app.py).
```

→ Fin de la docstring : indique que ce module est partage avec l'interface Streamlit.

```
6 """
```

→ Ferme la docstring du module.

```
8 from policy_pipeline import (
```

→ Commence l'import de plusieurs elements depuis le module moteur partage.

```
9     ARIZE_TRACING_ACTIF,
```

→ Importe le booleen indiquant si le tracing Arize est actif.

```
10     run_auto_evaluation,
```

→ Importe la fonction qui declenche l'auto-evaluation Phoenix.

```
11     run_multi_agent_pipeline,
```

→ Importe la fonction qui execute le pipeline multi-agents complet.

```
12 )
```

→ Ferme la liste des imports depuis policy_pipeline.

```
14 if ARIZE_TRACING_ACTIF:
```

→ Verifie si le tracing Arize a ete active au chargement du module partage.

```
15     print("Tracing Arize actif.")
```

→ Affiche en console que le tracing Arize est actif.

```
16 else:
```


→ Sinon, si le tracing Arize n'est pas configure.

```
17 print("ARIZE_SPACE_ID / ARIZE_API_KEY absents : tracing Arize desactive, le pipeline continue sans observabilite.")
```

→ Affiche en console que le pipeline continuera sans observabilite Arize (fonctionnalite optionnelle, pas un pre-requis pour que le pipeline lui-meme fonctionne).

```
19 saisie_utilisateur = "Analyser l'etat actuel et les implications futures des politiques agricoles britanniques et leur impact sur les communautés agricoles"
```

→ Definit en dur le sujet d'analyse fixe utilise par ce script CLI.

```
21 print("Generation d'analyse politique multi-agents...")
```

→ Affiche un message indiquant le demarrage du pipeline.

```
22 print("=" * 80)
```

→ Affiche une ligne de separation visuelle de 80 caracteres "=".

```
24 contenu_final = run_multi_agent_pipeline(saisie_utilisateur, progress_callback=lambda label: print(f"-> {label}"))
```

→ Lance le pipeline complet, avec un callback qui affiche chaque etape de progression en console.

```
25 print("\n===== SORTIE FINALE =====")
```

→ Affiche un separateur avant le rapport final.

```
26 print(contenu_final)
```

→ Affiche le rapport final synthetise par le Team.

```
28 # ----- Evaluation automatique -----
```

→ Commentaire de section marquant le debut de l'etape d'auto-evaluation.

```
29 df_eval_router = run_auto_evaluation(saisie_utilisateur, contenu_final)
```

→ Lance l'auto-evaluation Phoenix sur le rapport final et recupere le DataFrame de resultats.

```
31 print("\n===== RESULTATS D'EVALUATION =====")
```

→ Affiche un separateur avant les resultats d'evaluation.

```
32 print(df_eval_router[["label", "explanation"]])
```

→ Affiche uniquement les colonnes label et explanation du DataFrame de resultats.

```
33 print("Resultats sauvegardes dans 'evaluation_analyse_politique.csv'")
```

→ Affiche un message confirmant l'emplacement du fichier CSV de sauvegarde.

```
34 df_eval_router.to_csv("evaluation_analyse_politique.csv", index=False)
```

→ Exporte le DataFrame complet dans un fichier CSV, sans ecrire la colonne d'index pandas.

8-mistral_OCR/app.py

Interface Streamlit qui pilote la classe OCRService : upload du PDF, lancement de l'OCR, resume et questions.

50 lignes de code annotees ci-dessous.

```
1 import streamlit as st
```

→ Importe Streamlit, une librairie qui permet de construire une interface web interactive en pur Python (sans ecrire de HTML/CSS/JavaScript) : chaque widget (bouton, champ de texte...) est une simple fonction Python, et Streamlit ré-execute tout le script a chaque interaction de l'utilisateur.

```
2 import os
```

→ Importe le module standard, utilise ici pour creer un dossier et verifier l'existence de fichiers.

```

3 from legal_copilot import OCRService
    → Importe la classe metier definie dans legal_copilot.py qui contient toute la logique OCR/IA. Separer l'interface (app.py) de la
logique metier (legal_copilot.py) permet de tester ou reutiliser cette logique independamment de Streamlit.

5 st.set_page_config(page_title="Legal Copilot", layout="wide")
    → Configure le titre de l'onglet du navigateur et demande une mise en page large de la page Streamlit.

6 st.title("Legal Copilot - PDF OCR and Legal QA")
    → Affiche le titre principal visible en haut de la page.

8 ocr_service = OCRService()
    → Instancie une seule fois le service OCR (et donc le client Mistral) pour toute la session Streamlit.

9 uploaded_file = st.file_uploader("Upload a legal PDF file", type=["pdf"])
    → Affiche un composant d'upload de fichier limite aux PDF et stocke le fichier choisi par l'utilisateur.

10 ocr_performed = False
    → Initialise un indicateur local a False, mis a True seulement si l'OCR est effectivement relance dans ce cycle.

12 if uploaded_file:
    → N'execute le bloc suivant que si un fichier a ete effectivement uploadé.

13     os.makedirs("pdf", exist_ok=True)
    → Cree le dossier "pdf" s'il n'existe pas encore, sans erreur s'il existe deja.

14     file_path = f"pdf/{uploaded_file.name}"
    → Construit le chemin local ou le PDF uploadé sera sauvegarde, a partir de son nom d'origine.

16     with open(file_path, "wb") as f:
    → Ouvre (ou cree) ce fichier local en ecriture binaire.

17         f.write(uploaded_file.read())
    → Lit les octets du fichier recu par Streamlit (qui les garde en memoire, pas sur disque par default) et les ecrit sur le disque
local, car l'API OCR de Mistral (voir ocr_pdf) attend un chemin de fichier ouvrable.

19     if st.button("Run OCR"):
    → Affiche un bouton ; le bloc suivant ne s'execute que si l'utilisateur clique dessus lors de ce cycle. Streamlit re-execute tout le
script a chaque interaction : sans ce bouton, l'OCR (couteux) se relancerait a chaque re-rendu de la page.

20         try:
    → Demarre un bloc protege pour intercepter toute erreur pendant l'OCR.

21             with st.spinner("OCR en cours..."):
    → Affiche un indicateur de chargement tant que le bloc interne s'execute.

22                 ocr_service.ocr_pdf(file_path)
    → Appelle la methode qui envoie le PDF a l'API OCR de Mistral et sauvegarde le resultat.

23                 ocr_performed = True
    → Marque que l'OCR vient d'etre effectuee avec succes durant ce cycle d'execution.

24             except Exception as e:
    → Capture toute exception survenue pendant l'appel OCR.

25                 st.error(f"Error during OCR: {e}")
    → Affiche le message d'erreur a l'utilisateur dans l'interface.

27 if uploaded_file and ocr_service.ocr_is_valid(file_path) and st.checkbox("Show extracted OCR text"):

```

→ N'affiche la case a cocher (et son contenu) que si un fichier est uploadé et que le cache OCR correspond a ce fichier (voir `ocr_is_valid` dans `legal_copilot.py`, qui évite de montrer un texte OCR perime).

28 `try:`

→ Demarre un bloc protege pour la lecture du fichier Markdown OCRise.

29 `text = ocr_service.read_markdown_file("ocr_response.md")`

→ Recharge le texte Markdown extrait par l'OCR depuis le disque.

30 `st.markdown(text)`

→ Affiche ce texte Markdown formate directement dans la page.

31 `except Exception:`

→ Capture toute erreur de lecture du fichier (ex. fichier absent ou corrompu).

32 `st.warning("Error reading OCR content.")`

→ Affiche un avertissement a l'utilisateur en cas d'echec de lecture.

33 `elif not uploaded_file:`

→ Sinon, si aucun fichier n'a été uploadé du tout.

34 `st.info("Upload a PDF file to enable OCR analysis.")`

→ Invite l'utilisateur a uploader un PDF pour activer l'analyse.

36 `st.markdown("### Document Summary")`

→ Affiche un sous-titre de section pour la partie resume du document.

37 `if uploaded_file and ocr_service.ocr_is_valid(file_path) and st.button("Generate Summary"):`

→ Affiche le bouton de resume et n'exécute le bloc que si un fichier valide est present et que le bouton est cliqué.

38 `text = ocr_service.read_markdown_file("ocr_response.md")`

→ Recharge le texte Markdown complet du document depuis le fichier OCRise.

39 `prompt = f"Summarize this legal document clearly and concisely:\n\n{text}"`

→ Construit un prompt demandant un resume clair et concis, en y inserant le texte complet du document. A la difference de `process_question` (qui utilise le RAG pour ne selectionner que 2 chunks pertinents), ici le document entier est envoyé d'un bloc car un resume doit couvrir tout le contenu, pas seulement un passage.

40 `try:`

→ Demarre un bloc protege pour l'appel de generation du resume.

41 `summary = ocr_service.run_mistral(prompt)`

→ Envoie le prompt au modele de chat Mistral et recupere le resume genere.

42 `st.markdown("#### Summary:")`

→ Affiche un petit sous-titre avant le resume.

43 `st.markdown(summary)`

→ Affiche le texte du resume genere, avec mise en forme Markdown.

44 `except Exception as e:`

→ Capture toute erreur survenue pendant la generation du resume.

45 `st.error(f"Error during summarization: {e}")`

→ Affiche le message d'erreur correspondant a l'utilisateur.

47 `st.markdown("### Ask a question about the document content")`

→ Affiche le sous-titre de la section questions-reponses.

```

48 question = st.text_input("Your question:")
→ Affiche un champ de saisie texte et stocke la question tapee par l'utilisateur.

50 if question:
→ N'exécute le bloc suivant que si l'utilisateur a effectivement saisi une question (chaîne non vide).

51     if not os.path.exists("ocr_response.md") or not ocr_service.ocr_is_valid(file_path):
→ Vérifie qu'un OCR valide existe déjà avant de tenter de répondre à une question.

52         st.error("No valid OCR data found. Please run OCR first.")
→ Si aucun OCR valide n'est disponible, affiche une erreur invitant à lancer l'OCR d'abord.

53     else:
→ Sinon, si un OCR valide existe bien pour ce fichier.

54         try:
→ Démarre un bloc protégé pour le traitement de la question.

55             response = ocr_service.process_question(question, file_path)
→ Appelle la méthode RAG complète : découpage, embeddings, recherche FAISS, puis génération de la réponse.

56             st.markdown("### Response")
→ Affiche un sous-titre avant la réponse générée.

57             st.markdown(response)
→ Affiche la réponse générée par le modèle, avec mise en forme Markdown.

58         except Exception as e:
→ Capture toute erreur survenue pendant le traitement de la question.

59             st.error(f"Error while processing: {e}")
→ Affiche le message d'erreur correspondant à l'utilisateur.

```

9-Arize/app.py

Interface Streamlit qui pilote le même moteur partagé (policy_pipeline.py) avec une barre de progression et un export CSV.

78 lignes de code annotées ci-dessous.

```

1 import streamlit as st
→ Importe Streamlit, la librairie utilisée pour construire l'interface web en pur Python.

3 from policy_pipeline import (
→ Commence l'import de plusieurs éléments depuis le module moteur partagé.

4     ARIZE_TRACING_ACTIF,
→ Importe le booléen indiquant si le tracing Arize est actif.

5     GROQ_API_KEY,
→ Importe la clé API Groq pour vérifier sa présence avant de lancer le pipeline.

6     run_auto_evaluation,
→ Importe la fonction qui déclenche l'auto-évaluation Phoenix.

7     run_multi_agent_pipeline,
→ Importe la fonction qui exécute le pipeline multi-agents complet.

```

8)

→ Ferme la liste des imports depuis policy_pipeline.

10 st.set_page_config(page_title="Politique Agricole - Multi-Agents", page_icon="", layout="wide")

→ Configure le titre d'onglet, l'icone et la mise en page large de la page Streamlit.

11 st.title("Analyse Multi-Agents de Politique Agricole")

→ Affiche le titre principal de la page.

12 st.caption(

→ Commence l'affichage d'un sous-titre explicatif discret sous le titre.

13 "Un Gestionnaire delegue a 3 agents specialises – Recherche (web), Analyse, Evaluation d'impact – "

→ Premiere partie du texte du sous-titre, decrivant l'architecture du pipeline.

14 "puis un second modele (Phoenix) juge independamment si le rapport final est impactant."

→ Suite et fin du texte du sous-titre, decrivant l'auto-evaluation.

15)

→ Ferme l'appel a st.caption.

17 if not GROQ_API_KEY:

→ Verifie si la cle API Groq est absente ou vide -- une verification faite tot, avant de laisser l'utilisateur lancer un pipeline qui echouerait de toute facon a son premier appel.

18 st.error("GROQ_API_KEY est introuvable. Ajoute-le dans le fichier .env a la racine du projet.")

→ Affiche un message d'erreur invitant a configurer la cle manquante.

19 st.stop()

→ Arrete immediatement l'execution du script Streamlit si la cle est absente.

21 with st.sidebar:

→ Ouvre un bloc de contenu affiche dans la barre laterale de l'application.

22 st.header("Parametres")

→ Affiche un en-tete de section dans la barre laterale.

23 if ARIZE_TRACING_ACTIF:

→ Verifie si le tracing Arize est actif.

24 st.success("Tracing Arize actif")

→ Affiche un message de succes si le tracing est actif.

25 else:

→ Sinon, si le tracing Arize n'est pas configure.

26 st.info("Tracing Arize desactive (ARIZE_SPACE_ID / ARIZE_API_KEY absents). Le pipeline fonctionne quand meme.")

→ Affiche un message informatif precisant que le pipeline reste fonctionnel sans tracing.

28 st.caption(

→ Commence l'affichage d'un avertissement discret sur la duree d'execution.

29 "Ce pipeline enchaine plusieurs appels Groq (recherche web incluse) et peut prendre "

→ Premiere partie du texte d'avertissement sur la duree.

30 `***3 a 8 minutes**`, selon la charge du quota gratuit Groq (8000 tokens/minute)."

→ Suite et fin du texte d'avertissement, donnant une estimation de duree.

31)

→ Ferme l'appel a `st.caption` dans la barre laterale.

33 `DEFAULT_QUESTION = (`

→ Commence la definition de la question par default proposee a l'utilisateur.

34 `"Analyser l'etat actuel et les implications futures des politiques agricoles "`

→ Premiere partie du texte de la question par default.

35 `"britanniques et leur impact sur les communauts agricoles"`

→ Suite et fin du texte de la question par default.

36)

→ Ferme la definition de `DEFAULT_QUESTION`.

38 `saisie_utilisateur = st.text_area("Sujet a analyser", value=DEFAULT_QUESTION, height=100)`

→ Affiche une zone de texte modifiable pre-remplie avec la question par default.

39 `lancer = st.button("Lancer l'analyse multi-agents", use_container_width=True)`

→ Affiche le bouton de lancement du pipeline, sur toute la largeur disponible.

41 `if "resultat" not in st.session_state:`

→ Verifie si l'etat de session Streamlit ne contient pas encore de resultat stocke. `st.session_state` est la memoire persistante de Streamlit entre deux re-executions du script (par default, toute variable normale serait reinitialisee a chaque interaction de l'utilisateur).

42 `st.session_state.resultat = None`

→ Initialise la variable de session resultat a None lors du premier chargement de la page.

44 `if lancer:`

→ N'execute le bloc suivant que si le bouton de lancement a ete clique lors de ce cycle.

45 `if not saisie_utilisateur.strip():`

→ Verifie si le champ de sujet est vide une fois les espaces superflus retires.

46 `st.warning("Decris un sujet a analyser avant de lancer le pipeline.")`

→ Affiche un avertissement si aucun sujet n'a ete saisi.

47 `else:`

→ Sinon, si un sujet a bien ete saisi.

48 `progress_placeholder = st.empty()`

→ Cree un emplacement vide reutilisable dans la page, destine a afficher la progression.

49 `progress_placeholder.info("Demarrage du pipeline...")`

→ Affiche un message initial de demarrage dans cet emplacement.

50 `try:`

→ Demarre un bloc protege pour intercepter toute erreur pendant l'execution du pipeline.

51 `contenu_final = run_multi_agent_pipeline(`

→ Lance l'execution du pipeline multi-agents complet.

52 `saisie_utilisateur,`

→ Transmet le sujet saisi par l'utilisateur comme entree du pipeline.

53 `progress_callback=lambda label: progress_placeholder.info(label),`

→ Fournit un callback qui met a jour l'emplacement de progression a chaque etape deleguee.

54)

→ Ferme l'appel a run_multi_agent_pipeline.

55 progress_placeholder.info("Auto-evaluation du rapport (Phoenix)...")

→ Met a jour l'emplacement de progression pour indiquer le passage a l'auto-evaluation.

56 df_eval = run_auto_evaluation(saisie_utilisateur, contenu_final)

→ Lance l'auto-evaluation Phoenix sur le rapport final genere.

57 progress_placeholder.empty()

→ Vide l'emplacement de progression une fois le traitement termine.

58 st.session_state.resultat = {

→ Commence la construction du dictionnaire de resultats stocke en session.

59 "saisie": saisie_utilisateur,

→ Stocke le sujet analyse dans le dictionnaire de resultats.

60 "contenu_final": contenu_final,

→ Stocke le rapport final genere dans le dictionnaire de resultats.

61 "label": df_eval["label"].iloc[0],

→ Extrait et stocke l'etiquette d'evaluation (premiere ligne du DataFrame).

62 "explanation": df_eval["explanation"].iloc[0],

→ Extrait et stocke l'explication d'evaluation correspondante.

63 "df": df_eval,

→ Stocke le DataFrame complet, reutilise plus bas pour l'export CSV.

64 }

→ Ferme le dictionnaire de resultats et l'affecte a st.session_state.resultat.

65 except Exception as e:

→ Capture toute exception survenue pendant l'execution du pipeline ou de l'evaluation.

66 progress_placeholder.empty()

→ Vide l'emplacement de progression meme en cas d'erreur.

67 st.error(f"Erreur pendant le pipeline : {e}")

→ Affiche le message d'erreur a l'utilisateur.

69 resultat = st.session_state.resultat

→ Relit le resultat stocke en session dans une variable locale, a chaque execution du script.

70 if resultat is None:

→ Verifie si aucun resultat n'a encore ete genere.

71 st.info("Decris un sujet puis clique sur **Lancer l'analyse multi-agents**.")

→ Affiche un message d'invite si aucun resultat n'existe encore.

72 st.stop()

→ Arrete l'execution du script ici tant qu'aucun resultat n'est disponible.

74 st.divider()

→ Affiche une ligne de separation visuelle horizontale.

76 label = resultat["label"]
→ Recupere l'etiquette d'evaluation du resultat stocke.

77 if label == "impactful":
→ Verifie si l'etiquette d'evaluation est positive.

78 st.success(f"Evaluation automatique : **{label}**")
→ Affiche l'etiquette en vert avec un message de succes si elle est positive.

79 else:
→ Sinon, si l'etiquette d'evaluation est negative.

80 st.warning(f"Evaluation automatique : **{label}**")
→ Affiche l'etiquette en orange avec un message d'avertissement si elle est negative.

81 st.markdown(f"***Justification : ** {resultat['explanation']}**")
→ Affiche l'explication textuelle justifiant la decision d'evaluation.

83 st.download_button(
→ Affiche un bouton permettant de telecharger les resultats.

84 "Exporter les resultats en CSV",
→ Definit le libelle affiche sur le bouton de telechargement.

85 data=resultat["df"].to_csv(index=False).encode("utf-8-sig"),
→ Convertit le DataFrame en CSV puis l'encode en UTF-8 avec BOM, pour une compatibilite correcte avec Excel (qui, sans le BOM, interprete parfois mal les accents dans un CSV UTF-8).

86 file_name="evaluation_analyse_politique.csv",
→ Definit le nom du fichier propose au telechargement.

87 mime="text/csv",
→ Precise le type MIME du fichier telecharge, pour que le navigateur le reconnaisse comme un CSV.

88)
→ Ferme l'appel a st.download_button.

90 st.divider()
→ Affiche une seconde ligne de separation visuelle horizontale.

91 st.markdown("### Rapport final")
→ Affiche le sous-titre de la section du rapport final.

92 st.markdown(resultat["contenu_final"])
→ Affiche le texte complet du rapport final, avec mise en forme Markdown.

Projet 10 — 10-Workflow_Builder

Workflows d'automatisation no-code (n8n / Activepieces) pour la gestion clients

Objectif

Construit, sans écrire de backend, des workflows d'automatisation pour un scénario CRM complet : qualification automatique des leads entrants (formulaire, score, routage), séquences d'emails d'onboarding programmées pour les nouveaux clients, envoi d'enquêtes de satisfaction après un service terminé, et relance progressive des factures échues. Chaque workflow est un graphe de nœuds décrit dans un fichier JSON (gestionclient.json), importé et exécuté par un outil no-code (n8n).

Note de méthode : gestionclient.json contient 21 nœuds, dont plusieurs partagent exactement la même structure JSON (seuls les champs de valeur changent). Pour rester lisible, l'annotation ligne par ligne complète ci-dessous couvre les 3 premières formes de nœud rencontrées (webhook, code, switch/routeur) ; les 18 nœuds suivants sont ensuite décrits individuellement mais de façon condensée, en renvoyant à la forme JSON déjà détaillée. En revanche, le vrai code métier de ce projet — le JavaScript exécuté par les 4 nœuds de type "code" — est intégralement annoté ligne par ligne, reformulé avec de vrais retours à la ligne.

gestionclient.json

Définition complète du workflow n8n : 21 nœuds et leurs connexions.

Structure JSON — en-tête et 3 premiers nœuds (formes webhook / code / switch)

1 {

→ Ouvre l'objet JSON racine qui décrit le workflow entier. n8n est un outil d'automatisation "no-code" : plutôt que d'écrire du code Python/JavaScript classique, on décrit visuellement (dans l'éditeur n8n) une suite d'étapes connectées entre elles, appelées des "nœuds" (nodes). Ce fichier JSON est la représentation exportable de ce graphe visuel : n8n peut l'importer pour recréer exactement le même workflow.

2 "name": "Gestion Clients - Hub Principal",

→ Nom du workflow tel qu'affiche dans la liste des workflows n8n.

3 "nodes": [

→ Ouvre le tableau qui va contenir les 21 nœuds (étapes) du workflow. Chaque nœud représente une action ou une décision (recevoir un événement, exécuter du code, appeler une API, attendre un déclencheur temporel...) ; c'est la section "connections" plus bas qui précise dans quel ordre ces nœuds s'enchaînent.

4 {

→ Ouvre l'objet JSON du premier nœud du workflow.

5 "parameters": {

→ Ouvre le sous-objet des paramètres propres à ce type de nœud (ici, un webhook).

6 "path": "lead-qualification",

→ Définit le segment d'URL du webhook : ce nœud sera joignable sur .../webhook/lead-qualification.

7 "options": {}

→ Sous-objet d'options avancées du webhook, laisse vide ici.

8 },

→ Ferme le sous-objet "parameters".

- 9 `"id": "webhook-lead-qualification",`
→ Identifiant technique unique du noeud, utilise en interne par n8n pour le référencer.
- 10 `"name": "Nouveau Lead (Formulaire)",`
→ Nom affiche du noeud dans l'editeur visuel n8n.
- 11 `"type": "n8n-nodes-base.webhook",`
→ Type de noeud : un declencheur ("trigger") qui ecoute les requetes HTTP entrantes et demarre le workflow. Un "webhook" est simplement une URL exposee par n8n : quand un systeme externe (ici, vraisemblablement un formulaire web de capture de leads) envoie une requete HTTP a cette URL, n8n declenche automatiquement l'execution de ce workflow avec les donnees recues -- c'est le mecanisme standard pour connecter deux systemes independants sans que l'un ait besoin d'interroger l'autre en boucle (polling) : c'est le systeme source qui previent n8n des qu'il y a du nouveau.
- 12 `"typeVersion": 1,`
→ Version du schema de ce type de noeud, utilisee par n8n pour la compatibilite.
- 13 `"position": [`
→ Ouvre le tableau des coordonnees [x, y] du noeud dans le canevas visuel de l'editeur.
- 14 `240,`
→ Coordonnee horizontale (x) du noeud dans le canevas (purement visuel, sans effet sur l'execution).
- 15 `300`
→ Coordonnee verticale (y) du noeud dans le canevas.
- 16 `],`
→ Ferme le tableau de coordonnees.
- 17 `"webhookId": "lead-qualification-webhook"`
→ Identifiant interne du webhook utilise par n8n pour router les requetes HTTP entrantes vers ce noeud.
- 18 `},`
→ Ferme l'objet JSON de ce premier noeud.
- 19 `{`
→ Ouvre l'objet JSON du deuxieme noeud (calcul du score de qualification du lead).
- 20 `"parameters": {`
→ Ouvre le sous-objet des parametres propres a ce type de noeud (ici, un noeud de code).
- 21 `"jsCode": "... (voir le JavaScript reformate plus bas) ..."`
→ Contient tout le code JavaScript execute par ce noeud, sous forme de chaine echappee tenant sur une seule ligne JSON (tous les retours a la ligne du script sont encodes en `\n`). Ce JavaScript est reformate avec de vrais retours a la ligne et explique instruction par instruction juste apres cette section.
- 22 `},`
→ Ferme le sous-objet "parameters".
- 23 `"id": "lead-scoring",`
→ Identifiant technique unique du noeud.
- 24 `"name": "Calcul Score Lead",`
→ Nom affiche du noeud dans l'editeur visuel n8n.
- 25 `"type": "n8n-nodes-base.code",`
→ Type de noeud : execute du JavaScript arbitraire sur les donnees recues en entree.
- 26 `"typeVersion": 2,`
→ Version du schema de ce type de noeud.

27 "position": [
→ Ouvre le tableau des coordonnees [x, y] du noeud dans le canevas.

28 460,
→ Coordonnee horizontale (x) du noeud dans le canevas.

29 300
→ Coordonnee verticale (y) du noeud dans le canevas.

30]
→ Ferme le tableau de coordonnees (pas de virgule : c'est la derniere propriete de ce noeud).

31 },
→ Ferme l'objet JSON de ce deuxieme noeud.

32 {
→ Ouvre l'objet JSON du troisieme noeud (routage selon la qualification du lead).

33 "parameters": {
→ Ouvre le sous-objet des parametres propres a ce type de noeud (ici, un switch/routeur).

34 "conditions": {
→ Ouvre le sous-objet decrivant les conditions de routage a evaluer.

35 "options": {
→ Ouvre les options de comparaison des conditions.

36 "caseSensitive": true,
→ Impose une comparaison de texte sensible a la casse.

37 "leftValue": "",
→ Laisse vide la valeur de gauche par default au niveau global (definie par condition ci-dessous).

38 "typeValidation": "strict"
→ Impose une validation stricte des types lors de la comparaison.

39 },
→ Ferme le sous-objet "options" des conditions.

40 "conditions": [
→ Ouvre le tableau des conditions individuelles a evaluer (une branche par condition).

41 {
→ Ouvre la premiere condition de routage.

42 "id": "qualification-hot",
→ Identifiant de cette condition (branche "lead chaud").

43 "leftValue": "={{ \$json.qualification_status }}",
→ Valeur a comparer : le champ qualification_status calcule par le noeud de scoring precedent.

44 "rightValue": "hot",
→ Valeur attendue pour que cette condition soit vraie : la chaine "hot".

45 "operator": {
→ Ouvre la definition de l'operateur de comparaison utilise.

46 "type": "string",
→ Precise que la comparaison porte sur des chaines de caracteres.

```

47         "operation": "equals"
    → Precise que l'operation est un test d'egalite stricte.

48     }
    → Ferme la definition de l'opérateur.

49 },
    → Ferme la premiere condition (branche "hot").

50 {
    → Ouvre la deuxieme condition de routage.

51     "id": "qualification-warm",
    → Identifiant de cette condition (branche "lead tiede").

52     "leftValue": "={{ $json.qualification_status }}",
    → Meme champ compare que pour la premiere condition.

53     "rightValue": "warm",
    → Valeur attendue pour cette condition : la chaine "warm".

54     "operator": {
    → Ouvre la definition de l'opérateur de comparaison utilise.

55         "type": "string",
    → Precise que la comparaison porte sur des chaines de caracteres.

56         "operation": "equals"
    → Precise que l'operation est un test d'egalite stricte.

57     }
    → Ferme la definition de l'opérateur.

58 }
    → Ferme la deuxieme condition (pas de virgule : derniere condition du tableau).

59 ],
    → Ferme le tableau des conditions.

60     "combinator": "or"
    → Indique que le noeud route vers la premiere branche des lors qu'AU MOINS UNE des conditions est vraie.

61 },
    → Ferme le sous-objet "conditions".

62     "options": {}
    → Options avancees du noeud switch, laissees vides.

63 },
    → Ferme le sous-objet "parameters".

64     "id": "qualification-switch",
    → Identifiant technique unique du noeud.

65     "name": "Routage par Qualification",
    → Nom affiche du noeud dans l'editeur visuel n8n.

```

66 "type": "n8n-nodes-base.switch",

→ Type de noeud : un "switch" (aiguillage) evalue des conditions et route les donnees vers une branche de sortie differente selon le resultat -- l'equivalent visuel d'un if/elif dans un langage de programmation classique. C'est ce qui permet a un workflow n8n de ne pas etre une simple suite lineaire d'etapes, mais un veritable graphe de decision : ici, un lead "cold" et un lead "hot" ne suivent pas le meme chemin apres ce noeud.

67 "typeVersion": 3,

→ Version du schema de ce type de noeud.

68 "position": [

→ Ouvre le tableau des coordonnees [x, y] du noeud dans le canevas.

69 680,

→ Coordonnee horizontale (x) du noeud dans le canevas.

70 300

→ Coordonnee verticale (y) du noeud dans le canevas.

71]

→ Ferme le tableau de coordonnees.

72 },

→ Ferme l'objet JSON de ce troisieme noeud.

Les 18 noeuds suivants (memes formes JSON, contenu propre a chacun)

- **crm-create-lead (lignes 73-147, type httpRequest, methode POST)** : Envoie une requete HTTP POST vers l'API du CRM (URL et jeton d'authentification lus depuis les variables d'environnement n8n \$vars.CRM_API_URL / \$vars.CRM_API_TOKEN) pour creer une fiche lead, avec un corps de requete qui reprend tous les champs calcules par les noeuds precedents (nom, email, score, qualification, priorite, action suivante). Meme structure JSON qu'un noeud webhook/code deja detaille, seuls les champs "parameters" (methode, url, authentication, en-tetes, corps) different.
- **sales-assignment (lignes 148-160, type code)** : Noeud de code JavaScript qui determine automatiquement le commercial a assigner au lead selon son secteur et sa priorite, et calcule une echeance de reponse (SLA). Le JavaScript complet est detaille ligne par ligne dans la section "JS-2" ci-dessous.
- **slack-notification (lignes 161-188, type httpRequest, methode POST)** : Envoie une requete HTTP POST vers une URL de webhook Slack (variable \$vars.SLACK_WEBHOOK_SALES) pour publier un message formate (blocs Slack) annoncant l'arrivee d'un nouveau lead qualifie et assigne, avec son nom, son score, sa priorite et le commercial assigne. Meme structure httpRequest que le noeud crm-create-lead, seul le contenu du corps de requete change.
- **cron-onboarding-check (lignes 189-208, type cron)** : Declencheur temporel qui reveille le workflow toutes les 6 heures (expression cron "0 */6 * * *") pour verifier s'il existe de nouveaux clients a onboarding. Un noeud cron n'a presque pas de parametres : uniquement l'expression de planification, un identifiant, un nom, un type et une position.
- **fetch-new-clients (lignes 209-251, type httpRequest, methode GET)** : Envoie une requete HTTP GET vers l'API du CRM pour recuperer la liste des clients dont le statut est "new_client" et l'onboarding "not_started", cree depuis moins de 6 heures. Contrairement aux noeuds httpRequest POST deja vus, celui-ci utilise des parametres de requete ("queryParameters") plutot qu'un corps de requete, puisqu'il s'agit d'une simple lecture de donnees.
- **split-clients (lignes 252-262, type splitInBatches)** : Decoupe la liste de clients recuperee en lots traites un par un par la suite du workflow, plutot que d'envoyer toute la liste d'un coup au noeud suivant. Ce type de noeud n'a aucun parametre propre : sa seule fonction est de repartir les elements recus en plusieurs executions successives des noeuds en aval.

- **generate-email-sequence (lignes 263-275, type code)** : Noeud de code JavaScript qui construit une sequence de 4 emails d'onboarding personnalises et echelonnees dans le temps pour un client donne. Le JavaScript complet est detaille ligne par ligne dans la section "JS-3" ci-dessous.
- **batch-emails (lignes 276-290, type merge)** : Recombine en un seul flux les emails generes individuellement par le noeud precedent (mode "combine"), avant de les transmettre groupes au noeud de planification suivant. Comme le noeud splitInBatches, un noeud merge n'a que quelques parametres de configuration du mode de fusion, pas de logique metier propre.
- **schedule-email-sequence (lignes 291-341, type httpRequest, methode POST)** : Envoie une requete HTTP POST vers un service d'emailing externe (variable \$vars.EMAIL_SERVICE_URL) pour programmer l'envoi differe de la sequence d'emails d'onboarding generee plus haut, en transmettant l'identifiant client et la liste complete des emails avec leurs delais. Meme structure httpRequest POST que les noeuds crm-create-lead et slack-notification.
- **cron-satisfaction-survey (lignes 342-361, type cron)** : Declencheur temporel qui reveille le workflow chaque jour a 10h (expression cron "0 10 * * *") pour lancer l'envoi des enquetes de satisfaction. Meme structure minimale qu'un noeud cron deja detaille.
- **fetch-completed-services (lignes 362-400, type httpRequest, methode GET)** : Recupere via l'API du CRM la liste des prestations terminees depuis la veille et n'ayant pas encore recu d'enquete de satisfaction. Meme structure httpRequest GET avec parametres de requete que le noeud fetch-new-clients.
- **split-services (lignes 401-411, type splitInBatches)** : Decoupe la liste de prestations terminees en lots traites un par un. Meme structure minimale qu'un noeud splitInBatches deja detaille.
- **create-satisfaction-survey (lignes 412-466, type httpRequest, methode POST)** : Envoie une requete HTTP POST vers une plateforme d'enquetes externe pour creer un questionnaire de satisfaction post-service, avec une liste de questions notees (qualite, professionnalisme, respect des delais, communication, recommandation NPS) et les informations du client concerne. Meme structure httpRequest POST que les noeuds precedents.
- **slack-survey-notification (lignes 467-488, type n8n-nodes-base.slack, authentication OAuth2)** : Publie un message dans un canal Slack via le noeud Slack natif de n8n (authentifie par OAuth2 plutot que par une simple URL de webhook comme le noeud slack-notification), annoncant qu'une enquete de satisfaction a ete envoyee a un client, avec un lien vers l'enquete et un rappel de la relance automatique prevue sous 3 jours en l'absence de reponse.
- **cron-invoice-check (lignes 489-508, type cron)** : Declencheur temporel qui reveille le workflow chaque jour a 9h (expression cron "0 9 * * *") pour verifier les factures en retard de paiement. Meme structure minimale qu'un noeud cron deja detaille.
- **fetch-overdue-invoices (lignes 509-547, type httpRequest, methode GET)** : Recupere via l'API de facturation la liste des factures dont le statut est "overdue" (en retard), avec les donnees client associees. Meme structure httpRequest GET que les noeuds fetch-new-clients et fetch-completed-services.
- **split-overdue-invoices (lignes 548-558, type splitInBatches)** : Decoupe la liste de factures en retard en lots traites une par une par le noeud de calcul de relance suivant. Meme structure minimale qu'un noeud splitInBatches deja detaille.
- **invoice-reminder-logic (lignes 559-571, type code)** : Noeud de code JavaScript, le plus complexe des 4 noeuds de code du workflow : il calcule le nombre de jours de retard d'une facture, en deduit un niveau de relance progressif (douce / ferme / urgente / finale), calcule une penalite financiere au-dela de 15 jours de retard, et applique une regle anti-spam limitant les relances a une par 24 heures. Le JavaScript complet est detaille ligne par ligne dans la section "JS-4" ci-dessous.

Le vrai code metier : les 4 scripts JavaScript embarques, ligne par ligne

JS-1 — noeud "Calcul Score Lead" (lead-scoring)

JavaScript reformate avec de vrais retours a la ligne (dans le fichier JSON original, tout ce script tient sur une seule ligne, encode dans le champ "jsCode").

```
1 // Calcul du score de qualification lead
  → Commentaire decrivant le role du script pour un lecteur humain dans l'editeur n8n.

2 const data = $input.first().json;
  → Recupere les donnees JSON du premier element reçu en entree de ce noeud (celles envoyees par le webhook).

4 let score = 0;
  → Initialise le score du lead a zero avant de l'accumuler.

5 const weights = {
  → Ouvre la table de ponderation qui associe a chaque critere et chaque valeur possible un nombre de points.

6   company_size: { 'startup': 2, 'pme': 5, 'entreprise': 10 },
  → Points attribues selon la taille de l'entreprise du lead.

7   budget: { 'low': 1, 'medium': 5, 'high': 10 },
  → Points attribues selon le budget declare par le lead.

8   urgency: { 'low': 1, 'medium': 3, 'high': 7 },
  → Points attribues selon le niveau d'urgence declare.

9   decision_maker: { 'yes': 8, 'influence': 4, 'no': 1 },
  → Points attribues selon que le contact est decideur, a de l'influence, ou non decideur.

10  industry_match: { 'perfect': 10, 'good': 6, 'average': 3 }
  → Points attribues selon l'adequation entre le secteur du lead et l'offre de l'entreprise.

11 };
  → Ferme la table de ponderation.

13 // Calcul du score
  → Commentaire introduisant la boucle de calcul du score total.

14 Object.keys(weights).forEach(criteria => {
  → Parcourt chacun des 5 criteres definis dans la table de ponderation.

15   if (data[criteria] && weights[criteria][data[criteria]]) {
  → Verifie que le lead a bien une valeur pour ce critere, et que cette valeur existe dans la table de points.

16     score += weights[criteria][data[criteria]];
  → Ajoute au score total les points correspondant a la valeur de ce critere pour ce lead.

17   }
  → Ferme le bloc conditionnel du critere courant.

18 });
  → Ferme la boucle sur les criteres de ponderation.

20 // Qualification
  → Commentaire introduisant la conversion du score numerique en categorie.

21 let qualification = 'cold';
```

→ Initialise la qualification par défaut à "froid" (score insuffisant).

```
22 if (score >= 35) qualification = 'hot';
```

→ Reclasse le lead en "chaud" si son score atteint au moins 35 points.

```
23 else if (score >= 20) qualification = 'warm';
```

→ Sinon, reclasse le lead en "tiède" si son score atteint au moins 20 points.

```
25 // Priorite
```

→ Commentaire introduisant le calcul du niveau de priorité commerciale.

```
26 let priority = 'low';
```

→ Initialise la priorité par défaut à "basse".

```
27 if (score >= 40) priority = 'high';
```

→ Passe la priorité à "haute" si le score atteint au moins 40 points.

```
28 else if (score >= 25) priority = 'medium';
```

→ Sinon, passe la priorité à "moyenne" si le score atteint au moins 25 points.

```
30 return {
```

→ Commence la construction de la valeur retournée par ce nœud, transmise au nœud suivant.

```
31   json: {
```

→ Ouvre l'objet JSON qui constituera les données de sortie de ce nœud.

```
32     ...data,
```

→ Reprend l'intégralité des champs reçus en entrée, pour ne rien perdre en aval du workflow.

```
33     lead_score: score,
```

→ Ajoute le score numérique calculé aux données du lead.

```
34     qualification_status: qualification,
```

→ Ajoute la catégorie de qualification ("hot"/"warm"/"cold") aux données du lead.

```
35     priority: priority,
```

→ Ajoute le niveau de priorité commerciale aux données du lead.

```
36     qualified_at: new Date().toISOString(),
```

→ Ajoute un horodatage indiquant à quel moment ce lead a été qualifié.

```
37     next_action: qualification === 'hot' ? 'immediate_contact' :
```

→ Détermine l'action suivante à mener : contact immédiat si le lead est chaud.

```
38         qualification === 'warm' ? 'nurture_sequence' : 'marketing_automation'
```

→ Sinon, une séquence de nurturing si tiède, ou une automatisation marketing standard si froide.

```
39   }
```

→ Ferme l'objet json de sortie.

```
40 };
```

→ Ferme l'objet retourné par le nœud, qui devient les données d'entrée du nœud suivant.

JS-2 — noeud "Attribution Commercial" (sales-assignment)

JavaScript reformate avec de vrais retours a la ligne (dans le fichier JSON original, tout ce script tient sur une seule ligne, encode dans le champ "jsCode").

```
1 // Attribution automatique du commercial
  → Commentaire decrivant le role du script.

2 const data = $input.first().json;
  → Recupere les donnees JSON recues en entree (le lead enrichi par les noeuds precedents).

3 const leadData = data.body || data;
  → Utilise le sous-objet "body" des donnees si present (cas d'un appel API), sinon les donnees telles quelles.

5 // Regles d'attribution par secteur et priorite
  → Commentaire introduisant la table de regles d'attribution.

6 const salesAssignment = {
  → Ouvre la table associant secteur et priorite au commercial a assigner.

7   'tech': {
    → Ouvre les regles d'attribution pour le secteur technologique.

8     'high': 'commercial_tech_senior',
    → Assigne un commercial senior tech aux leads tech de priorite haute.

9     'medium': 'commercial_tech_junior',
    → Assigne un commercial junior tech aux leads tech de priorite moyenne.

10    'low': 'marketing_automation'
    → Route les leads tech de faible priorite vers l'automatisation marketing plutot qu'un commercial humain.

11  },
  → Ferme les regles du secteur tech.

12  'retail': {
    → Ouvre les regles d'attribution pour le secteur retail.

13    'high': 'commercial_retail_senior',
    → Assigne un commercial senior retail aux leads retail de priorite haute.

14    'medium': 'commercial_retail_junior',
    → Assigne un commercial junior retail aux leads retail de priorite moyenne.

15    'low': 'marketing_automation'
    → Route les leads retail de faible priorite vers l'automatisation marketing.

16  },
  → Ferme les regles du secteur retail.

17  'finance': {
    → Ouvre les regles d'attribution pour le secteur finance.

18    'high': 'commercial_finance_senior',
    → Assigne un commercial senior finance aux leads finance de priorite haute.

19    'medium': 'commercial_finance_junior',
    → Assigne un commercial junior finance aux leads finance de priorite moyenne.

20    'low': 'marketing_automation'
```

→ Route les leads finance de faible priorite vers l'automatisation marketing.

21 },

→ Ferme les regles du secteur finance.

22 'default': {

→ Ouvre les regles d'attribution par default, utilisees pour tout secteur non liste explicitement.

23 'high': 'commercial_general_senior',

→ Assigne un commercial senior generaliste par default aux leads de priorite haute.

24 'medium': 'commercial_general_junior',

→ Assigne un commercial junior generaliste par default aux leads de priorite moyenne.

25 'low': 'marketing_automation'

→ Route les leads de faible priorite et secteur inconnu vers l'automatisation marketing.

26 }

→ Ferme les regles par default.

27 };

→ Ferme la table complete des regles d'attribution.

29 const industry = leadData.industry || 'default';

→ Recupere le secteur du lead, ou "default" si le champ est absent.

30 const priority = leadData.priority || 'low';

→ Recupere la priorite du lead calculee plus tot, ou "low" par default si absente.

31 const assignedTo = salesAssignment[industry]?.[priority] || salesAssignment['default'][priority];

→ Cherche le commercial correspondant au secteur et a la priorite ; si le secteur n'existe pas dans la table, retombe sur la regle par default pour cette priorite.

33 // Calcul du SLA (temps de reponse attendu)

→ Commentaire introduisant le calcul du delai de reponse garanti (SLA).

34 const slaHours = {

→ Ouvre la table associant chaque niveau de priorite a un delai de reponse en heures.

35 'high': 2,

→ Delai de reponse garanti de 2 heures pour un lead de priorite haute.

36 'medium': 24,

→ Delai de reponse garanti de 24 heures (1 jour) pour un lead de priorite moyenne.

37 'low': 72

→ Delai de reponse garanti de 72 heures (3 jours) pour un lead de priorite basse.

38 };

→ Ferme la table des delais de SLA.

40 const responseDeadline = new Date();

→ Cree un objet date initialise a l'instant present, qui servira de base au calcul de l'echeance.

41 responseDeadline.setHours(responseDeadline.getHours() + slaHours[priority]);

→ Ajoute a l'heure courante le nombre d'heures de SLA correspondant a la priorite du lead, pour obtenir l'echeance de reponse.

43 return {

→ Commence la construction de la valeur retournee par ce noeud.

44 json: {

→ Ouvre l'objet JSON de sortie.

45 ...leadData,

→ Reprend l'integralite des champs du lead recus en entree.

46 assigned_to: assignedTo,

→ Ajoute l'identifiant du commercial (ou de l'automatisation) assigne a ce lead.

47 assignment_date: new Date().toISOString(),

→ Ajoute un horodatage indiquant a quel moment l'attribution a ete faite.

48 response_deadline: responseDeadline.toISOString(),

→ Ajoute l'echeance de reponse calculee plus haut, au format ISO.

49 sla_hours: slaHours[priority],

→ Ajoute le nombre d'heures de SLA applicable, pour reference.

50 assignment_reason: `Industry: \${industry}, Priority: \${priority}`

→ Ajoute une courte justification textuelle de l'attribution, utile pour l'audit ou l'affichage.

51 }

→ Ferme l'objet json de sortie.

52 };

→ Ferme l'objet retourne par le noeud.

JS-3 — noeud "Generation Sequence Emails" (generate-email-sequence)

JavaScript reformate avec de vrais retours a la ligne (dans le fichier JSON original, tout ce script tient sur une seule ligne, encode dans le champ "jsCode").

```
1 // Generation sequence emails onboarding personnalisee
  → Commentaire decrivant le role du script.

2 const client = $input.first().json;
  → Recupere les donnees JSON du client reçu en entree (un des nouveaux clients a onboorder).

4 const emailSequence = [
  → Ouvre le tableau qui va contenir les 4 emails programmes de la sequence d'onboarding.

5   {
  → Ouvre la definition du premier email de la sequence.

6     delay_hours: 0,
  → Ce premier email est envoye immediatement (0 heure de delai).

7     template: 'welcome_email',
  → Nom du gabarit d'email de bienvenue a utiliser.

8     subject: `Bienvenue ${client.first_name} ! Votre onboarding commence`,
  → Construit l'objet de l'email en y inserant dynamiquement le prenom du client.

9     variables: {
  → Ouvre les variables qui seront injectees dans le gabarit d'email par le service d'emailing.

10      client_name: client.first_name,
  → Variable du prenom du client pour la personnalisation du gabarit.

11      company: client.company,
  → Variable du nom de l'entreprise du client.

12      account_manager: client.assigned_account_manager,
  → Variable du nom du charge de compte assigne au client.

13      onboarding_link: `${process.env.PORTAL_URL}/onboarding/${client.id}`
  → Construit un lien personnalise vers le portail d'onboarding du client, a partir d'une variable d'environnement et de l'identifiant client.

14    },
  → Ferme les variables du premier email.

15  },
  → Ferme la definition du premier email.

16  {
  → Ouvre la definition du deuxieme email de la sequence.

17    delay_hours: 24,
  → Ce deuxieme email est programme 24 heures apres le premier.

18    template: 'setup_guide',
  → Nom du gabarit de guide de configuration a utiliser.

19    subject: 'Etape 1 : Configuration de votre compte',
  → Objet fixe du deuxieme email (pas de personnalisation ici).
```

```

20     variables: {
    → Ouvre les variables du deuxieme email.

21     setup_checklist: [
    → Ouvre la liste des etapes de configuration a afficher dans l'email.

22         'Verification profil entreprise',
    → Premiere etape de la checklist de configuration.

23         'Configuration preferences',
    → Deuxieme etape de la checklist.

24         'Import donnees existantes',
    → Troisieme etape de la checklist.

25         'Parametrage utilisateurs'
    → Quatrieme et derniere etape de la checklist.

26     ]
    → Ferme la liste des etapes de configuration.

27 }
    → Ferme les variables du deuxieme email.

28 },
    → Ferme la definition du deuxieme email.

29 {
    → Ouvre la definition du troisieme email de la sequence.

30     delay_hours: 72,
    → Ce troisieme email est programme 72 heures (3 jours) apres le premier.

31     template: 'training_invitation',
    → Nom du gabarit d'invitation a la formation.

32     subject: 'Votre formation personnalisee est prete !',
    → Objet fixe du troisieme email.

33     variables: {
    → Ouvre les variables du troisieme email.

34         training_date: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000).toISOString(),
    → Calcule une date de formation proposee, fixee a 7 jours dans le futur a partir de l'instant d'execution.

35         training_type: client.package_type === 'enterprise' ? 'advanced' : 'standard',
    → Choisit le type de formation (avancee ou standard) selon l'offre souscrite par le client.

36         calendar_link: `${process.env.CALENDAR_URL}/book-training/${client.id}`
    → Construit un lien personnalise de reservation de creneau de formation.

37     }
    → Ferme les variables du troisieme email.

38 },
    → Ferme la definition du troisieme email.

39 {
    → Ouvre la definition du quatrieme et dernier email de la sequence.

```

```

40     delay_hours: 120,
    → Ce quatrieme email est programme 120 heures (5 jours) apres le premier.

41     template: 'progress_check',
    → Nom du gabarit de suivi de progression.

42     subject: 'Comment se passe votre prise en main ?',
    → Objet fixe du quatrieme email.

43     variables: {
    → Ouvre les variables du quatrieme email.

44         progress_survey: `${process.env.SURVEY_URL}/onboarding-feedback/${client.id}`,
    → Construit un lien personnalise vers un sondage de satisfaction sur l'onboarding.

45         support_contact: client.assigned_account_manager
    → Rappelle le contact support (le charge de compte) dans l'email.

46     }
    → Ferme les variables du quatrieme email.

47 }
    → Ferme la definition du quatrieme email (pas de virgule : dernier element du tableau).

48 ];
    → Ferme le tableau des 4 emails de la sequence.

50 return emailSequence.map(email => ({
    → Transforme chaque email de la sequence en un element de sortie distinct du noeud (un item n8n par email).

51     json: {
    → Ouvre l'objet JSON de sortie pour cet email.

52         client_id: client.id,
    → Rattache l'identifiant du client a chaque email genere, pour pouvoir le retrouver en aval.

53         client_email: client.email,
    → Rattache l'adresse email du client, necessaire pour l'envoi.

54         client_name: client.first_name,
    → Rattache le prenom du client, reutilisable en aval.

55         ...email,
    → Reprend tous les champs de la definition de cet email (delay_hours, template, subject, variables).

56         scheduled_for: new Date(Date.now() + email.delay_hours * 60 * 60 * 1000).toISOString()
    → Calcule l'horodatage exact d'envoi de cet email, en ajoutant son delai en heures a l'instant d'execution.

57     }
    → Ferme l'objet json de cet email.

58 }));
    → Ferme l'appel a .map() : le noeud retourne donc 4 items en sortie, un par email programme.

```

JS-4 — noeud "Calcul Niveau de Relance" (invoice-reminder-logic)

JavaScript reformate avec de vrais retours a la ligne (dans le fichier JSON original, tout ce script tient sur une seule ligne, encode dans le champ "jsCode").

```
1 // Logique de relance progressive des factures
  → Commentaire decrivant le role du script.

2 const invoice = $input.first().json;
  → Recupere les donnees JSON de la facture recue en entree.

3 const today = new Date();
  → Cree un objet date representant l'instant present, utilise pour calculer le retard.

4 const dueDate = new Date(invoice.due_date);
  → Convertit la date d'echance de la facture (texte) en objet date exploitable.

5 const daysOverdue = Math.floor((today - dueDate) / (1000 * 60 * 60 * 24));
  → Calcule le nombre de jours de retard en divisant la difference de temps (en millisecondes) par le nombre de millisecondes dans une journee.

7 // Definir le niveau de relance selon les jours de retard
  → Commentaire introduisant la logique de classification du niveau de relance.

8 let reminderLevel = 'none';
  → Initialise le niveau de relance par default a "aucun".

9 let emailTemplate = '';
  → Initialise le gabarit d'email a une chaine vide.

10 let escalation = false;
  → Initialise l'indicateur d'escalade (notification comptable) a faux.

11 let accountAction = 'none';
  → Initialise l'action sur le compte client a "aucune".

13 if (daysOverdue >= 1 && daysOverdue <= 7) {
  → Verifie si le retard est compris entre 1 et 7 jours.

14   reminderLevel = 'gentle';
  → Fixe le niveau de relance a "douce" pour un retard de 1 a 7 jours.

15   emailTemplate = 'gentle_reminder';
  → Selectionne le gabarit d'email de relance douce.

16 } else if (daysOverdue >= 8 && daysOverdue <= 15) {
  → Sinon, verifie si le retard est compris entre 8 et 15 jours.

17   reminderLevel = 'firm';
  → Fixe le niveau de relance a "ferme" pour un retard de 8 a 15 jours.

18   emailTemplate = 'firm_reminder';
  → Selectionne le gabarit d'email de relance ferme.

19 } else if (daysOverdue >= 16 && daysOverdue <= 30) {
  → Sinon, verifie si le retard est compris entre 16 et 30 jours.

20   reminderLevel = 'urgent';
  → Fixe le niveau de relance a "urgent" pour un retard de 16 a 30 jours.
```

```
21  emailTemplate = 'urgent_reminder';  
    → Selectionne le gabarit d'email de relance urgente.  
  
22  escalation = true; // Notification comptable  
    → Active l'escalade : le service comptable est notifié a partir de ce niveau de retard.  
  
23 } else if (daysOverdue > 30) {  
    → Sinon, si le retard dépasse 30 jours.  
  
24  reminderLevel = 'final';  
    → Fixe le niveau de relance a "final" (dernier avertissement) au-delà de 30 jours.  
  
25  emailTemplate = 'final_notice';  
    → Selectionne le gabarit d'email de mise en demeure finale.  
  
26  escalation = true;  
    → Maintient l'escalade active pour ce niveau critique.  
  
27  accountAction = 'suspend'; // Suspension compte  
    → Déclenche la suspension du compte client au-delà de 30 jours de retard.  
  
28 }  
    → Ferme le bloc de classification du niveau de relance.  
  
30 // Calcul des penalites si applicable  
    → Commentaire introduisant le calcul des penalites financieres.  
  
31 let penalty = 0;  
    → Initialise le montant de la penalite a zero.  
  
32 if (daysOverdue > 15) {  
    → Verifie si le retard dépasse 15 jours, seuil a partir duquel une penalite s'applique.  
  
33  penalty = invoice.amount * 0.02; // 2% de penalite  
    → Calcule une penalite egale a 2% du montant de la facture.  
  
34 }  
    → Ferme le bloc de calcul de la penalite.  
  
36 // Historique des relances  
    → Commentaire introduisant la lecture de l'historique des relances deja envoyees.  
  
37 const reminderHistory = invoice.reminder_history || [];  
    → Recupere l'historique des relances precedentes, ou un tableau vide s'il n'existe pas encore.  
  
38 const lastReminder = reminderHistory[reminderHistory.length - 1];  
    → Recupere la derniere relance envoyee (dernier element du tableau), ou undefined si aucune relance n'a encore ete  
    envoyee.  
  
40 // Verifier si on doit envoyer une relance (pas plus d'une par jour)  
    → Commentaire introduisant la regle anti-spam limitant les relances.  
  
41 let shouldSendReminder = true;  
    → Initialise l'indicateur d'envoi a vrai par default.  
  
42 if (lastReminder) {  
    → Verifie si une relance precedente existe deja pour cette facture.  
  
43  const lastReminderDate = new Date(lastReminder.sent_date);
```


→ Convertit la date de la dernière relance en objet date exploitable.

```
44 const hoursSinceLastReminder = (today - lastReminderDate) / (1000 * 60 * 60);
```

→ Calcule le nombre d'heures écoulées depuis la dernière relance envoyée.

```
45 if (hoursSinceLastReminder < 24) {
```

→ Vérifie si moins de 24 heures se sont écoulées depuis la dernière relance.

```
46     shouldSendReminder = false;
```

→ Empêche l'envoi d'une nouvelle relance si la précédente date de moins de 24 heures.

```
47 }
```

→ Ferme le bloc de vérification du délai.

```
48 }
```

→ Ferme le bloc de vérification de l'existence d'une relance précédente.

```
50 return {
```

→ Commence la construction de la valeur retournée par ce nœud.

```
51   json: {
```

→ Ouvre l'objet JSON de sortie.

```
52     ...invoice,
```

→ Reprend l'intégralité des champs de la facture reçus en entrée.

```
53     days_overdue: daysOverdue,
```

→ Ajoute le nombre de jours de retard calculé.

```
54     reminder_level: reminderLevel,
```

→ Ajoute le niveau de relance déterminé (douce/ferme/urgente/finale/aucune).

```
55     email_template: emailTemplate,
```

→ Ajoute le nom du gabarit d'email à utiliser pour la relance.

```
56     should_send_reminder: shouldSendReminder,
```

→ Ajoute l'indicateur final décidant si une relance doit effectivement partir maintenant.

```
57     escalation_required: escalation,
```

→ Ajoute l'indicateur d'escalade vers le service comptable.

```
58     account_action: accountAction,
```

→ Ajoute l'action à appliquer sur le compte client (aucune ou suspension).

```
59     penalty_amount: penalty,
```

→ Ajoute le montant de la pénalité calculée.

```
60     total_amount_due: invoice.amount + penalty,
```

→ Ajoute le montant total du à présent (facture initiale plus pénalité).

```
61     next_escalation_date: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000).toISOString(),
```

→ Calcule une date indicative de prochaine escalade, fixée à 7 jours dans le futur.

```
62     reminder_count: reminderHistory.length
```

→ Ajoute le nombre total de relances déjà envoyées pour cette facture.

```
63 }
```

→ Ferme l'objet json de sortie.

64 };

→ Ferme l'objet retourne par le noeud.

Connexions entre noeuds (section "connections" du JSON, condensee)

Cette section du fichier (lignes 573 a 765) associe mecaniquement, pour chaque noeud, le nom du noeud suivant dans le flux ; sa structure JSON est purement repetitive (un objet imbrique par connexion) et n'apporte pas d'information supplementaire au-dela du lien lui-meme : elle est donc presentee ici sous forme de tableau plutot que ligne de JSON par ligne de JSON.

Noeud source	Noeud(s) cible(s)	Role de la connexion
Nouveau Lead (Formulaire)	Calcul Score Lead	Le webhook transmet les donnees brutes du formulaire au noeud de scoring.
Calcul Score Lead	Routage par Qualification	Le score et la qualification calcules sont transmis au noeud de routage.
Routage par Qualification	Creation Lead CRM (branche hot/warm) ; aucune suite (branche cold)	Seuls les leads chauds ou tiesdes continuent vers la creation dans le CRM ; les leads froids s'arretent ici.
Creation Lead CRM	Attribution Commercial	Le lead cree dans le CRM est transmis au noeud d'attribution commerciale.
Attribution Commercial	Notification Slack	Le commercial assigne et l'echance SLA sont transmis a la notification Slack.
Check Nouveaux Clients (cron)	Recup Nouveaux Clients	Le declenchement horaire lance la recuperation des nouveaux clients via l'API.
Recup Nouveaux Clients	Split Clients	La liste de clients recuperee est transmise au decoupage par lots.
Split Clients	Generation Sequence Emails (branche courante) ; boucle (lot suivant)	Chaque client du lot courant declenche une generation de sequence d'emails, puis le lot suivant est traite.
Generation Sequence Emails	Batch Emails	Les emails generes individuellement sont transmis au noeud de fusion.
Batch Emails	Programmer Sequence	Les emails fusionnes sont transmis a la programmation d'envoi differe.
Surveys Satisfaction 10h (cron)	Services Termes Hier	Le declenchement quotidien lance la recuperation des services recemment termes.
Services Termes Hier	Split Services	La liste de services recuperee est transmise au decoupage par lots.
Split Services	Creation Survey (branche courante) ; boucle (lot suivant)	Chaque service du lot courant declenche la creation d'une enquete de satisfaction, puis le lot suivant est traite.
Creation Survey	Notif Survey Slack	L'enquete creee declenche une notification Slack a l'equipe.
Check Factures 9h (cron)	Factures Echues	Le declenchement quotidien lance la recuperation des factures en retard.
Factures Echues	Split Factures	La liste de factures en retard est transmise au decoupage par lots.

Noeud source	Noeud(s) cible(s)	Role de la connexion
Split Factures	Calcul Niveau de Relance (branche courante) ; boucle (lot suivant)	Chaque facture du lot courant declenche le calcul de son niveau de relance, puis le lot suivant est traite.
Etat final : fonctionnel — workflow importe et visualise dans n8n ; stack Activepieces demarree via Docker Compose.		

Projet 11 — 11-Notion_RAG

Chat en ligne de commande sur une base de connaissances Notion d'entreprise

Objectif

Assistant en ligne de commande qui permet aux employés de poser des questions en langage naturel à une base de connaissances interne stockée dans une base de données Notion. L'agent interroge Notion à chaque question, retrouve les entrées les plus proches, et répond en citant la source.

Code source, ligne par ligne

11-Notion_RAG/noton_rag.py

Classe NotionRAG : agent agno interrogeant une base de connaissances Notion via un retriever personnalisée, en mode chat CLI.

123 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard pour lire les variables d'environnement.

2 from notion_client import Client

→ Importe le client officiel de l'API Notion. Notion est un outil de prise de notes/base de connaissances d'entreprise ; ce client permet de lire par programme le contenu d'une base de données Notion (ici, une liste de paires question/réponse) plutôt que de le consulter manuellement dans l'application Notion.

3 from agno.agent import Agent

→ Importe la classe Agent d'agno, utilisée pour créer l'agent conversationnel.

4 from agno.models.groq import Groq

→ Importe la classe wrapper qui connecte agno à l'API Groq, le fournisseur d'inférence utilisé ici pour exécuter le modèle de langage qui rédige les réponses.

5 from dotenv import load_dotenv

→ Importe la fonction qui charge les variables du fichier .env.

7 load_dotenv()

→ Charge les variables du fichier .env dans l'environnement du processus.

9 GROQ_API_KEY = os.getenv("GROQ_API_KEY")

→ Récupère la clé API Groq depuis les variables d'environnement.

11 class NotionRAG:

→ Déclare la classe qui encapsule l'agent et la logique de recherche dans Notion. Le nom "NotionRAG" fait référence au pattern RAG (Retrieval-Augmented Generation) : au lieu de laisser le modèle de langage répondre de mémoire, on va d'abord RETROUVER les entrées Notion pertinentes pour la question, puis les injecter dans le contexte du modèle pour qu'il GÉNÈRE sa réponse à partir de ces informations vérifiées de l'entreprise.

12 def __init__(self):

→ Définit le constructeur de la classe, appelé une fois à la création d'un objet NotionRAG.

13 self.notion = Client(auth=os.getenv("NOTION_TOKEN"))

→ Crée le client Notion authentifié avec le jeton d'intégration lu dans l'environnement. Ce jeton (à créer depuis les paramètres "intégrations" de l'espace Notion) autorise ce script à lire une base Notion précise, sans jamais transmettre le mot de passe du

compte Notion lui-meme.

```
14 self.database_id = os.getenv("NOTION_DATABASE_ID")
```

→ Recupere l'identifiant de la base de donnees Notion a interroger.

```
15 self.agent = self._create_agent()
```

→ Construit et stocke l'agent conversationnel via la methode privree _create_agent.

```
17 def _create_agent(self):
```

→ Definit la methode privree qui construit et configure l'agent agno.

```
18 """Create agent with Notion retriever"""
```

→ Docstring de la methode, decrivant son role en une phrase.

```
19 return Agent(
```

→ Commence la creation et le retour de l'objet Agent.

```
20 # Le script original utilisait OpenAI (gpt-4o) ; remplace par Groq pour
```

→ Commentaire expliquant le remplacement du modele OpenAI original par Groq.

```
21 # reutiliser la cle GROQ_API_KEY deja disponible.
```

→ Fin du commentaire justifiant ce remplacement.

```
22 model=Groq(id="openai/gpt-oss-120b", api_key=GROQ_API_KEY),
```

→ Attache a l'agent le modele Groq 120B utilise pour generer les reponses -- c'est le modele de langage, le composant qui redige effectivement la reponse en francais/anglais naturel a partir du contexte fourni.

```
23 knowledge=None,
```

→ Indique explicitement qu'aucune base de connaissances pre-indexee classique (comme une base vectorielle de type FAISS ou PgVector utilisee dans d'autres projets) n'est utilisee ici : la recherche se fait autrement, via le knowledge_retriever personnalise defini plus bas.

```
24 search_knowledge=True,
```

→ Active la capacite de l'agent a interroger une source de connaissances avant de repondre.

```
25 # Le parametre s'appelait "retriever" dans une version anterieure d'agno ;
```

→ Commentaire expliquant le renommage du parametre dans les versions recentes d'agno.

```
26 # il a ete renomme "knowledge_retriever" (meme signature d'appel).
```

→ Fin du commentaire : precise que la signature de la fonction reste inchangee.

```
27 knowledge_retriever=self._notion_retriever
```

→ Fournit la methode _notion_retriever comme fonction de recherche personnalisee appelee a chaque question. Le "retriever" (recupereur) est le composant charge, dans une architecture RAG, d'aller chercher les informations pertinentes avant de les donner au modele de langage : ici, au lieu d'interroger une base vectorielle par similarite d'embeddings, il interroge directement l'API Notion en temps reel et compare les mots-cles -- une approche plus simple, sans indexation prealable a maintenir, adaptee a une petite FAQ.

```
28 )
```

→ Ferme la construction de l'objet Agent et le retourne a l'appelant.

```
30 def _notion_retriever(self, agent, query, num_documents=None, **kwargs):
```

→ Definit la fonction de recherche personnalisee, avec la signature attendue par agno (agent, query, num_documents...). C'est agno lui-meme qui appelle cette fonction automatiquement des que l'agent decide qu'il a besoin de consulter la base de connaissances pour repondre.

```
31 """Search Notion database for relevant Q&A pairs"""
```

→ Docstring de la methode, decrivant son role en une phrase.

```
32 try:
```

→ Demarre un bloc protege pour intercepter toute erreur d'appel a l'API Notion.

```
33         results = self.notion.databases.query(database_id=self.database_id)
```

→ Interroge l'API Notion pour recuperer toutes les pages de la base de connaissances configuree.

```
35         matching_documents = []
```

→ Initialise la liste vide qui accueillera les entrees Notion jugees pertinentes.

```
36         query_words = set(query.lower().split())
```

→ Decoupe la question de l'utilisateur en un ensemble de mots minuscules, sans doublons.

```
38         for page in results["results"]:
```

→ Demarre une boucle sur chaque page (entree) renvoyee par la requete Notion.

```
39             # Recuperer la question (gerer differentes casses)
```

→ Commentaire introduisant la gestion des variantes de casse des noms de proprietes Notion.

```
40             question_data = None
```

→ Initialise la variable qui contiendra les donnees brutes de la propriete "question" de cette page.

```
41             if "Question" in page["properties"]:
```

→ Verifie si la page possede une propriete nommee exactement "Question" (avec majuscule). Cette verification defensive existe car, dans Notion, les colonnes d'une base sont creees manuellement par un humain : leur nom exact (casse, orthographe) peut varier d'une base a l'autre, d'ou ces multiples variantes testees.

```
42                 question_data = page["properties"]["Question"]["title"]
```

→ Recupere le contenu du champ "Question" (de type titre Notion) si present.

```
43             elif "question" in page["properties"]:
```

→ Sinon, verifie si la page possede une propriete nommee "question" en minuscules.

```
44                 question_data = page["properties"]["question"]["title"]
```

→ Recupere le contenu du champ "question" (variante minuscule) si present.

```
46                 question = question_data[0]["plain_text"] if question_data and question_data else ""
```

→ Extrait le texte brut de la question depuis la structure Notion, ou une chaine vide si absent.

```
48             # Recuperer la reponse (gerer differentes casses)
```

→ Commentaire introduisant la gestion des variantes de casse pour la propriete reponse.

```
49             answer_data = None
```

→ Initialise la variable qui contiendra les donnees brutes de la propriete "reponse" de cette page.

```
50             if "Response" in page["properties"]:
```

→ Verifie si la page possede une propriete nommee "Response".

```
51                 answer_data = page["properties"]["Response"]["rich_text"]
```

→ Recupere le contenu du champ "Response" (texte enrichi Notion) si present.

```
52             elif "response" in page["properties"]:
```

→ Sinon, verifie si la page possede une propriete nommee "response" en minuscules.

```
53                 answer_data = page["properties"]["response"]["rich_text"]
```

→ Recupere le contenu du champ "response" (variante minuscule) si present.

```
54             elif "answer" in page["properties"]:
```

→ Sinon, verifie si la page possede une propriete nommee "answer".

```
55                 answer_data = page["properties"]["answer"]["rich_text"]
```

→ Recupere le contenu du champ "answer" (variante alternative) si present.

```
57         answer = answer_data[0]["plain_text"] if answer_data and answer_data else ""
```

→ Extrait le texte brut de la reponse depuis la structure Notion, ou une chaine vide si absent.

```
59         # Better matching: check if most query words are in question or answer
```

→ Commentaire introduisant la logique de correspondance par mots-cles.

```
60         question_words = set(question.lower().split())
```

→ Decoupe la question Notion en un ensemble de mots minuscules.

```
61         answer_words = set(answer.lower().split())
```

→ Decoupe la reponse Notion en un ensemble de mots minuscules.

```
62         all_words = question_words.union(answer_words)
```

→ Fusionne les mots de la question et de la reponse Notion en un seul ensemble.

```
64         # If at least 60% of query words match, it's a hit
```

→ Commentaire precisant le seuil de correspondance retenu.

```
65         matches = len(query_words.intersection(all_words))
```

→ Compte le nombre de mots communs entre la question de l'utilisateur et l'entree Notion.

```
66         match_ratio = matches / len(query_words) if query_words else 0
```

→ Calcule la proportion de mots de la question utilisateur retrouves dans l'entree Notion. C'est une forme de recherche lexicale (par recouvrement de mots), plus simple qu'une recherche semantique par embeddings : elle ne comprend pas les synonymes ou reformulations, mais ne necessite aucune infrastructure supplementaire (pas de base vectorielle a maintenir), ce qui convient a une petite FAQ interne.

```
68         if match_ratio >= 0.6: # 60% match threshold
```

→ Ne considere l'entree comme pertinente que si au moins 60% des mots de la question correspondent.

```
69         # Recuperer les metadonnees (gerer differentes casses)
```

→ Commentaire introduisant la recuperation des metadonnees (departement, tags).

```
70         department_data = None
```

→ Initialise la variable qui contiendra les donnees brutes de la propriete "departement".

```
71         if "Departement" in page["properties"]:
```

→ Verifie si la page possede une propriete nommee "Departement".

```
72         department_data = page["properties"]["Departement"]["select"]
```

→ Recupere la valeur du champ "Departement" (liste deroulante Notion) si present.

```
73         elif "departement" in page["properties"]:
```

→ Sinon, verifie la variante minuscule "departement".

```
74         department_data = page["properties"]["departement"]["select"]
```

→ Recupere la valeur du champ "departement" (variante minuscule) si present.

```
75         elif "Department" in page["properties"]:
```

→ Sinon, verifie la variante anglaise "Department".

```
76         department_data = page["properties"]["Department"]["select"]
```

→ Recupere la valeur du champ "Department" (variante anglaise) si present.

```
78         department = department_data["name"] if department_data else "Unknown"
```

→ Extrait le nom du departement selectionne, ou "Unknown" si aucune valeur n'est definie.


```

80         tags_data = None
    → Initialise la variable qui contiendra les donnees brutes de la propriete "tags".

81         if "Tags" in page["properties"]:
    → Verifie si la page possede une propriete nommee "Tags".

82         tags_data = page["properties"]["Tags"]["multi_select"]
    → Recupere la liste des tags (champ a selection multiple Notion) si presente.

83         elif "tags" in page["properties"]:
    → Sinon, verifie la variante minuscule "tags".

84         tags_data = page["properties"]["tags"]["multi_select"]
    → Recupere la liste des tags (variante minuscule) si presente.

86         tags = [tag["name"] for tag in tags_data] if tags_data else []
    → Construit une liste simple des noms de tags a partir de la structure Notion, ou une liste vide.

88         content = f"Q: {question}\nA: {answer}"
    → Construit le texte du document combine, affichant la question puis la reponse. C'est ce texte qui sera injecte dans le
    contexte du modele de langage : la structure "Q: ... A: ..." aide le modele a comprendre qu'il s'agit d'une paire question-reponse
    verifiee, pas d'un texte libre.

89         if tags:
    → Verifie si au moins un tag a ete trouve pour cette entree.

90         content += f"\nTags: {' , '.join(tags)}"
    → Ajoute la liste des tags, separees par des virgules, a la fin du contenu.

92         matching_documents.append({

93             "content": content,
    → Stocke le texte combine question/reponse/tags comme contenu du document.

94             "meta_data": {

95                 "source": "Company Knowledge Base",
    → Indique la source du document, affichee eventuellement comme citation par l'agent -- c'est ce qui permet a l'utilisateur de
    savoir que la reponse vient bien de la base interne de l'entreprise, pas d'une invention du modele.

96                 "department": department,
    → Ajoute le departement associe a cette entree dans les metadonnees.

97                 "tags": tags,
    → Ajoute la liste des tags associes a cette entree dans les metadonnees.

98                 "match_score": match_ratio
    → Ajoute le score de correspondance calcule, utilise pour le tri plus bas.

99             }
    → Ferme le dictionnaire de metadonnees.

100         })
    → Ferme le dictionnaire du document et l'appel a append.

102         # Sort by match score (best matches first)
    → Commentaire introduisant le tri des resultats par pertinence.

```

103 matching_documents.sort(key=lambda x: x["meta_data"]["match_score"], reverse=True)
→ Trie la liste des documents trouves par score de correspondance decroissant (meilleurs resultats en premier).

105 print(f"Found {len(matching_documents)} matching documents")
→ Affiche en console le nombre de documents pertinents trouves, utile pour le debogage.

107 if matching_documents:
→ Verifie si au moins un document pertinent a ete trouve.

108 return matching_documents[:num_documents] if num_documents else matching_documents
→ Retourne les documents trouves, limites a num_documents si cette limite est fournie par agno.

109 else:
→ Sinon, si aucun document pertinent n'a ete trouve.

110 return [{"content": "No matching entry found in the knowledge base.",
→ Commence la construction d'un document de secours indiquant qu'aucune correspondance n'a ete trouvee.

111 "meta_data": {"source": "Company Knowledge Base"}}]
→ Termine ce document de secours avec ses metadonnees minimales, et le retourne dans une liste.

113 except Exception as e:
→ Capture toute exception survenue pendant l'appel a l'API Notion ou le traitement des donnees.

114 print(f"Error: {e}")
→ Affiche l'erreur en console pour faciliter le diagnostic.

115 return [{"content": f"Error: {str(e)}", "meta_data": {"source": "Error"}}]
→ Retourne un document contenant le message d'erreur, pour que l'agent puisse informer l'utilisateur.

117 def ask(self, question):
→ Definit la methode publique qui pose une question unique a l'agent et affiche la reponse.

118 """Ask a question to the knowledge base"""
→ Docstring de la methode, decrivant son role en une phrase.

119 print(f"Searching knowledge base for: {question}")
→ Affiche en console la question qui va etre recherchee.

120 self.agent.print_response(question)
→ Demande a l'agent de traiter la question et d'afficher directement sa reponse en console.

122 def chat(self):
→ Definit la methode publique qui lance une boucle de conversation interactive en console.

123 """Interactive chat mode"""
→ Docstring de la methode, decrivant son role en une phrase.

124 print("Notion Knowledge Base Chat")
→ Affiche un message d'accueil au demarrage du mode chat.

125 print("Type 'quit' to exit")
→ Indique a l'utilisateur comment quitter la boucle de conversation.

127 while True:
→ Demarre une boucle infinie qui continue tant que l'utilisateur ne demande pas explicitement de quitter.

128 question = input("\nYour question: ")
→ Attend et lit une ligne de texte saisie par l'utilisateur dans le terminal.

```
129         if question.lower() in ["quit", "exit", "q"]:  
            → Verifie si l'utilisateur a tape l'un des mots-cles de sortie (insensible a la casse).  
  
130             break  
            → Sort de la boucle infinie si un mot-cle de sortie a ete saisi.  
  
132         self.ask(question)  
            → Sinon, transmet la question saisie a la methode ask() pour obtenir et afficher une reponse.  
  
134 def main():  
    → Definit la fonction principale, point d'entree du script en ligne de commande.  
  
135     """Simple main function"""  
    → Docstring de la fonction, decrivant son role en une phrase.  
  
136     print("Notion RAG Knowledge Base")  
    → Affiche un message de demarrage du programme.  
  
137     print("-" * 40)  
    → Affiche une ligne de separation visuelle de 40 tirets.  
  
139     try:  
        → Demarre un bloc protege pour intercepter toute erreur d'initialisation.  
  
140         # Initialize the system  
        → Commentaire indiquant le debut de l'initialisation du systeme.  
  
141         rag = NotionRAG()  
        → Cree une instance de la classe NotionRAG, ce qui initialise le client Notion et l'agent.  
  
143         # Interactive chat  
        → Commentaire indiquant le lancement du mode chat interactif.  
  
144         rag.chat()  
        → Demarre la boucle de conversation interactive en console.  
  
146     except Exception as e:  
        → Capture toute exception survenue pendant l'initialisation ou la conversation.  
  
147         print(f"Error: {e}")  
        → Affiche le message d'erreur brut en console.  
  
148         print("Make sure you have:")  
        → Introduit une checklist d'aide au diagnostic pour l'utilisateur, listant les variables d'environnement necessaires pour que le  
        programme puisse s'authentifier aupres de Notion et Groq.  
  
149         print("1. NOTION_TOKEN in .env")  
        → Rappelle que le jeton d'integration Notion doit etre present dans le fichier .env.  
  
150         print("2. NOTION_DATABASE_ID in .env")  
        → Rappelle que l'identifiant de la base de donnees Notion doit etre present dans le fichier .env.  
  
151         print("3. GROQ_API_KEY in .env")  
        → Rappelle que la cle API Groq doit etre presente dans le fichier .env.  
  
153 if __name__ == "__main__":  
    → Verifie que le script est execute directement (et non importe comme module).
```

154 `main()`

→ Appelle la fonction principale, ce qui démarre effectivement le programme.

Projet 14 — 14-Multi_AgentAGNO

Equipe d'agents finance (recherche + reasoning) servie via Playground

Objectif

Assemble une équipe de deux agents spécialisés en finance -- l'un dédié à la recherche de données de marché en ligne, l'autre au raisonnement et au calcul -- coordonnées pour répondre à des questions d'analyse financière. L'équipe est exposée via l'interface de développement Playground d'agno, consultable directement dans un navigateur.

Code source, ligne par ligne

14-Multi_AgentAGNO/reasoning_finance_team.py

Ce fichier configure et lance une équipe de deux agents IA (recherche web et finance) coordonnés par un agent superviseur, avec mémoire persistante en base MySQL/Postgres, exposée via une interface web Playground.

152 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard `os`, qui donne accès aux variables d'environnement du système via `os.getenv`. C'est la méthode standard en Python pour lire de la configuration externe (clés API, identifiants de base de données) sans jamais l'écrire en dur dans le code source, ce qui évite de divulguer des secrets si le code est partagé ou versionné.

2 from agno.agent import Agent

→ Importe la classe `Agent` du framework Agno. Agno est un framework Python dédié à la construction d'agents IA : il encapsule un modèle de langage (LLM), lui associe des outils (des fonctions que le modèle peut décider d'appeler), une mémoire et des instructions, pour obtenir un agent capable d'agir de façon autonome plutôt qu'un simple chatbot qui ne fait que répondre au texte reçu.

3 from agno.memory.v2.db.postgres import PostgresMemoryDb

→ Importe `PostgresMemoryDb`, l'adaptateur qui relie le système de mémoire long-terme d'Agno (version 2) à une base PostgreSQL. Sans cette couche de persistance, les souvenirs extraits des conversations resteraient uniquement en mémoire vive et seraient perdus à chaque redémarrage du script.

4 from agno.memory.v2.memory import Memory

→ Importe la classe `Memory`, qui gère la mémoire "long terme" d'un agent. Contrairement à l'historique brut d'une conversation, elle extrait et conserve des faits durables sur l'utilisateur (préférences, informations récurrentes) afin que l'agent puisse s'en souvenir même lors d'une session ultérieure, des jours après.

5 from agno.models.openai import OpenAIChat

→ Importe `OpenAIChat`, la classe qui encapsule un grand modèle de langage (LLM) OpenAI de type conversationnel (ici GPT-4o) pour qu'Agno puisse l'utiliser comme "cerveau" d'un agent. Le LLM est le composant qui comprend le langage naturel, raisonne et décide quelles actions entreprendre.

6 from agno.storage.agent.postgres import PostgresAgentStorage

→ Importe `PostgresAgentStorage`, qui persiste l'historique des sessions (les échanges de conversation bruts) dans Postgres. C'est complémentaire à `Memory` : le storage conserve la trace complète des conversations, tandis que la mémoire retient des faits synthétisés et durables.

7 from agno.team.team import Team

→ Importe la classe `Team`, qui permet de regrouper plusieurs agents individuels en une équipe pilotée par un modèle superviseur. C'est un pattern d'architecture multi-agents : plutôt qu'un seul agent généraliste, on spécialise plusieurs agents (ici recherche web et finance) et un coordinateur répartit les tâches entre eux puis assemble le résultat final.

```
8 from agno.tools.duckduckgo import DuckDuckGoTools
```

→ Importe l'outil ``DuckDuckGoTools``, qui donne à un agent la capacité d'effectuer de vraies recherches web via le moteur DuckDuckGo. Un "tool" (outil), en IA agentique, est une fonction que le LLM peut décider d'invoquer pour obtenir une information ou réaliser une action qu'il ne peut pas faire seul : ici, accéder à des données récentes du web, que le modèle ne connaît pas par cœur puisque son entraînement s'arrête à une date donnée.

```
9 from agno.tools.reasoning import ReasoningTools
```

→ Importe ``ReasoningTools``, un outil qui pousse l'agent à exposer un raisonnement structuré, étape par étape, avant de conclure (une forme de "chaîne de pensée" outillée et traçable). Cela améliore la fiabilité des réponses sur des tâches complexes, notamment lorsqu'il faut croiser et pondérer plusieurs sources d'information avant de trancher.

```
10 from agno.tools.yfinance import YFinanceTools
```

→ Importe ``YFinanceTools``, un outil qui encapsule la bibliothèque ``yfinance`` pour aller chercher des données financières réelles (cours de bourse, ratios, recommandations d'analystes) sur Yahoo Finance. Il permet à l'agent de répondre avec des chiffres à jour plutôt qu'avec des connaissances figées à la date d'entraînement du modèle.

```
11 from agno.playground import Playground, serve_playground_app
```

→ Importe ``Playground`` et ``serve_playground_app``, qui fournissent une interface web de type chat déjà prête à l'emploi pour tester et démontrer des agents ou équipes sans avoir à développer son propre frontend. ``serve_playground_app`` démarre ensuite le serveur qui l'expose sur le réseau.

```
12 from dotenv import load_dotenv
```

→ Importe ``load_dotenv``, fonction de la librairie python-dotenv qui charge les variables définies dans un fichier ``.env`` local vers l'environnement du processus Python. C'est la pratique standard pour garder les secrets (clés API, mots de passe de base de données) hors du code source, donc hors d'un dépôt Git par exemple.

```
14 load_dotenv()
```

→ Exécute le chargement effectif du fichier ``.env`` : à partir de cette ligne, ``os.getenv(...)`` peut lire les valeurs qu'il contient.

```
15 # SÉCURITÉ: Utiliser les variables d'environnement
```

→ Commentaire rappelant la bonne pratique de sécurité : ne jamais coder une clé API en dur dans le fichier, toujours passer par une variable d'environnement chargée dynamiquement.

```
16 OPENAI_API_KEY = os.getenv("OPENAI_API_KEY") or os.getenv("OPENAI_KEY_API")
```

→ Récupère la clé API OpenAI en testant successivement deux noms de variable possibles, ce qui rend le script tolérant à une éventuelle incohérence de nommage dans le fichier ``.env``. Cette clé sert à authentifier chaque appel fait au modèle GPT-4o utilisé plus loin par les agents.

```
18 if not OPENAI_API_KEY:
```

→ Vérifie qu'une clé a bien été trouvée avant d'aller plus loin. Sans clé, tout appel au modèle échouerait plus tard de façon plus obscure ; le script préfère donc échouer immédiatement avec un message clair, un principe appelé "fail fast" qui simplifie le diagnostic.

```
19     print("ERREUR: Clé OpenAI manquante dans .env")
```

→ Affiche un message d'erreur explicite indiquant que la clé est absente.

```
20     exit(1)
```

→ Arrête l'exécution du script avec le code de sortie 1, convention Unix signalant une erreur, utile si ce script est lancé par un outil d'orchestration qui surveille les codes de retour.

```
22 MYSQL_HOST = os.getenv("MYSQL_HOST", "localhost")
```

→ Récupère l'hôte MySQL depuis l'environnement, avec "localhost" comme valeur par défaut. Le deuxième argument de ``getenv`` fournit une valeur de repli, ce qui permet au script de fonctionner "out of the box" en développement local même sans configuration explicite.

```
23 MYSQL_PORT = os.getenv("MYSQL_PORT", "3306")
```

→ Récupère le port MySQL depuis l'environnement, avec "3306" (le port par défaut de MySQL) comme valeur de repli.

```

24 MYSQL_USER = os.getenv("MYSQL_USER", "root")
    → Récupère l'utilisateur MySQL depuis l'environnement, avec "root" comme valeur par défaut.

25 MYSQL_PASSWORD = os.getenv("MYSQL_PASSWORD", "")
    → Récupère le mot de passe MySQL depuis l'environnement, vide par défaut.

26 MYSQL_DATABASE = os.getenv("MYSQL_DATABASE", "agno_finance_db")
    → Récupère le nom de la base MySQL depuis l'environnement, avec une valeur par défaut dédiée à ce projet.

28 if MYSQL_PASSWORD:
    → Vérifie si un mot de passe MySQL a été fourni, pour construire l'URL de connexion en conséquence.

29     db_url =
f"mysql+mysqlconnector://{MYSQL_USER}:{MYSQL_PASSWORD}@{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DATABASE}"
    → Construit l'URL de connexion MySQL au format attendu par SQLAlchemy
(dialecte+driver://utilisateur:motdepasse@hôte:port/base), en incluant le mot de passe. Cette URL unique sera réutilisée par
toutes les couches de stockage et de mémoire des agents plus bas dans le fichier.

30 else:
    → Cas où aucun mot de passe n'est défini (par exemple en développement local avec un utilisateur MySQL sans mot de
passe).

31     db_url = f"mysql+mysqlconnector://{MYSQL_USER}@{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DATABASE}"
    → Construit l'URL de connexion MySQL sans mot de passe, en évitant un ":" vide qui pourrait perturber certains pilotes de
connexion.

33 print(f"MySQL: {MYSQL_USER}@{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DATABASE}")
    → Affiche les paramètres de connexion MySQL utilisés, à titre informatif pour faciliter le diagnostic au démarrage.

35 def test_mysql_connection():
    → Définit une fonction qui teste la connexion à la base MySQL avant de démarrer l'application. Ce type de "health check" au
démarrage évite de lancer des agents qui échoueraient silencieusement plus tard faute de pouvoir sauvegarder leur mémoire ou
leur historique.

36     try:
    → Démarre un bloc try/except, mécanisme Python de gestion des erreurs qui permet de capturer une exception plutôt que de
laisser le script planter brutalement.

37         from sqlalchemy import create_engine, text
    → Importe localement les fonctions de SQLAlchemy, la bibliothèque Python de référence pour interagir avec des bases de
données relationnelles. `create_engine` construit un objet gérant le pool de connexions, `text` permet d'écrire du SQL brut de
façon sûre (évite les injections).

38         engine = create_engine(db_url)
    → Crée un moteur SQLAlchemy à partir de l'URL de connexion MySQL construite plus haut ; ce moteur ne se connecte pas
encore réellement, il prépare seulement la configuration.

39         with engine.connect() as connection:
    → Ouvre une connexion effective à la base de données dans un gestionnaire de contexte (`with`), qui garantit que la connexion
sera proprement fermée même en cas d'erreur.

40             connection.execute(text("SELECT 1"))
    → Exécute une requête SQL triviale pour vérifier concrètement que la connexion fonctionne, indépendamment de l'existence
de tables applicatives.

41             result = connection.execute(text("SELECT DATABASE()"))
    → Exécute une requête pour récupérer le nom de la base de données actuellement active sur la connexion.

42             current_db = result.fetchone()[0]

```

→ Extrait la première valeur de la première ligne du résultat, c'est-à-dire le nom de la base.

```
43         print(f"MySQL OK - Base active: {current_db}")
```

→ Affiche un message confirmant la connexion et la base active, utile pour repérer une mauvaise configuration.

```
44         return True
```

→ Retourne True pour signaler que le test de connexion a réussi.

```
45     except Exception as e:
```

→ Capture toute exception survenue pendant le test (base injoignable, identifiants invalides, etc.).

```
46         print(f"Erreur MySQL: {e}")
```

→ Affiche le message d'erreur capturé, pour aider à diagnostiquer le problème de connexion.

```
47         return False
```

→ Retourne False pour signaler l'échec de la connexion à l'appelant.

```
49 if not test_mysql_connection():
```

→ Appelle la fonction de test et vérifie si la connexion a échoué.

```
50     exit(1)
```

→ Arrête le script avec un code d'erreur si la connexion MySQL a échoué, là encore selon le principe "fail fast" : mieux vaut ne pas démarrer que démarrer dans un état cassé.

```
52 memory = Memory(
```

→ Commence la création de l'objet `Memory` partagé, qui sera ensuite attaché aux deux agents et à l'équipe. Le partager entre plusieurs agents leur permet d'accéder aux mêmes souvenirs sur l'utilisateur, au lieu que chacun ait sa propre mémoire cloisonnée.

```
53     model=OpenAIChat(api_key=OPENAI_API_KEY, id="gpt-4o"), # CORRECTION: Utiliser la variable
```

→ Configure le modèle OpenAI GPT-4o utilisé en interne pour analyser les conversations et en extraire des souvenirs pertinents, avec la clé API récupérée plus haut. Le commentaire "CORRECTION" indique qu'une version antérieure du code utilisait probablement la clé en dur au lieu de la variable d'environnement.

```
54     db=PostgresMemoryDb(table_name="user_memories", db_url=db_url),
```

→ Configure la base de stockage des souvenirs sur la table "user_memories", en réutilisant la même URL de connexion que le reste de l'application (ici pointant en réalité vers MySQL malgré le nom "Postgres" de la classe).

```
55     delete_memories=False,
```

→ Désactive la suppression automatique des souvenirs existants lors des mises à jour de mémoire, pour ne pas perdre d'historique utilisateur.

```
56     clear_memories=False,
```

→ Désactive l'effacement complet de la mémoire au démarrage de l'application, garantissant la persistance des souvenirs d'une exécution à l'autre.

```
57 )
```

→ Ferme l'appel du constructeur `Memory`.

```
59 web_agent = Agent(
```

→ Commence la création du premier agent spécialisé de l'équipe, chargé de la recherche web générale.

```
60     name="Web Search Agent",
```

→ Définit le nom affiché de cet agent, utile notamment dans l'interface Playground.

```
61     role="Handle web search requests and general research",
```

→ Décrit le rôle de l'agent : ce texte sert aussi au coordinateur de l'équipe (plus bas) pour savoir à quel agent déléguer quelle tâche.

62 `agent_id="web_agent",`
→ Définit l'identifiant technique unique de cet agent, utilisé notamment pour distinguer ses données de session dans le stockage.

63 `model=OpenAIChat(api_key=OPENAI_API_KEY, id="gpt-4o"), # CORRECTION: Utiliser la variable`
→ Configure le modèle OpenAI GPT-4o utilisé par cet agent pour comprendre les requêtes et générer ses réponses, avec la clé API en variable d'environnement.

64 `tools=[DuckDuckGoTools()],`
→ Fournit à l'agent l'outil de recherche web DuckDuckGo défini plus haut : c'est ce qui lui permet concrètement d'aller chercher de l'information à jour sur internet plutôt que de se fier uniquement à ses connaissances internes figées.

65 `storage=PostgresAgentStorage(db_url=db_url, table_name="web_agent_sessions"),`
→ Configure le stockage des sessions de cet agent dans sa propre table "web_agent_sessions", ce qui permet de reprendre une conversation là où elle s'était arrêtée.

66 `memory=memory,`
→ Associe l'objet mémoire partagé, créé plus haut, à cet agent, afin qu'il puisse lire et écrire dans la même base de souvenirs que le reste de l'équipe.

67 `instructions=[`
→ Commence la liste des instructions textuelles données à l'agent. Ces instructions forment le "prompt système" : un texte injecté avant chaque échange pour cadrer le comportement du modèle, une technique clé de prompt engineering.

68 `"Search for current and relevant information on financial topics",`
→ Instruction demandant de chercher des informations financières actuelles et pertinentes.

69 `"Always include sources and publication dates",`
→ Instruction demandant de toujours indiquer les sources et dates de publication, ce qui renforce la vérifiabilité des réponses générées par le LLM.

70 `"Focus on reputable financial news sources",`
→ Instruction demandant de privilégier des sources d'actualité financière fiables.

71 `"Provide context and background information",`
→ Instruction demandant de fournir du contexte et des informations de fond.

72 `"Remember user preferences and previous searches",`
→ Instruction demandant explicitement à l'agent d'exploiter sa mémoire long-terme pour se souvenir des préférences et recherches précédentes de l'utilisateur.

73 `],`
→ Ferme la liste des instructions de l'agent.

74 `markdown=True,`
→ Active le formatage Markdown pour les réponses de l'agent, ce qui permet d'afficher des titres, listes et tableaux lisibles dans l'interface Playground.

75 `enable_agentic_memory=True,`
→ Active la "mémoire agentic" : c'est l'agent lui-même, via le LLM, qui décide quoi retenir, mettre à jour ou oublier dans sa mémoire long-terme, plutôt qu'une logique fixe codée manuellement. Cela rend la gestion des souvenirs plus flexible mais dépend de la qualité du modèle.

76 `add_history_to_messages=True,`
→ Active l'ajout de l'historique récent de la conversation aux messages envoyés au modèle, pour qu'il garde le contexte des échanges précédents dans la même session.

77 `add_datetime_to_instructions=True,`
→ Active l'ajout automatique de la date et l'heure courantes aux instructions, pour que l'agent soit "conscient" du moment

présent (utile par exemple pour évaluer si une information financière est récente).

78)

→ Ferme l'appel du constructeur de `web\_agent`.

80 finance_agent = Agent(

→ Commence la création du second agent spécialisé de l'équipe, chargé des données financières.

81 name="Finance Agent",

→ Définit le nom affiché de cet agent.

82 role="Handle financial data requests and market analysis",

→ Décrit le rôle de l'agent, utilisé par le coordinateur de l'équipe pour router les demandes appropriées vers lui.

83 agent_id="finance_agent",

→ Définit l'identifiant technique unique de cet agent.

84 model=OpenAIChat(api_key=OPENAI_API_KEY, id="gpt-4o"), # CORRECTION: Utiliser la variable

→ Configure le modèle OpenAI GPT-4o utilisé par cet agent avec la clé API.

85 tools=[YFinanceTools(

→ Commence la configuration de l'outil `YFinanceTools`, qui donne accès aux données financières de Yahoo Finance ; chaque paramètre ci-dessous active une catégorie de données spécifique.

86 stock_price=True,

→ Active la récupération du prix courant des actions.

87 company_info=True,

→ Active la récupération des informations générales sur l'entreprise (secteur, description, etc.).

88 stock_fundamentals=True,

→ Active la récupération des données fondamentales de l'action (résultats financiers de base de l'entreprise).

89 key_financial_ratios=True,

→ Active la récupération des ratios financiers clés (par exemple P/E, marges), utilisés pour évaluer la santé d'une entreprise.

90 analyst_recommendations=True,

→ Active la récupération des recommandations des analystes financiers professionnels sur le titre.

91]),

→ Ferme la configuration de l'outil `YFinanceTools` et la liste `tools`.

92 storage=PostgresAgentStorage(db_url=db_url, table_name="finance_agent_sessions"),

→ Configure le stockage des sessions de cet agent dans sa propre table "finance_agent_sessions".

93 memory=memory,

→ Associe le même objet mémoire partagé à cet agent, pour cohérence avec `web\_agent`.

94 instructions=[

→ Commence la liste des instructions textuelles données à l'agent.

95 "You are a financial data specialist and your goal is to generate comprehensive and accurate financial reports.",

→ Instruction définissant le rôle de spécialiste des données financières de l'agent.

96 "Use tables to display stock prices, fundamentals (P/E, Market Cap, Revenue), and recommendations.",

→ Instruction demandant d'utiliser des tableaux Markdown pour présenter les données financières de façon lisible.

97 "Clearly state the company name and ticker symbol.",

→ Instruction demandant de toujours préciser le nom et le symbole boursier (ticker) de l'entreprise, pour éviter toute ambiguïté.

98 "Include key financial ratios and metrics in your analysis.",

→ Instruction demandant d'inclure les ratios et métriques financiers clés dans l'analyse.

99 "Focus on delivering actionable financial insights.",

→ Instruction demandant de fournir des analyses financières exploitables, pas seulement descriptives.

100 "Delegate tasks and run tools in parallel if needed.",

→ Instruction autorisant l'agent à déléguer des tâches et exécuter plusieurs outils en parallèle si besoin, pour accélérer la production de rapports complexes.

101 "Remember user's investment preferences and risk tolerance.",

→ Instruction demandant de s'appuyer sur la mémoire long-terme pour se souvenir des préférences d'investissement et de la tolérance au risque de l'utilisateur.

102],

→ Ferme la liste des instructions de l'agent.

103 markdown=True,

→ Active le formatage Markdown pour les réponses de l'agent.

104 enable_agentic_memory=True,

→ Active la gestion autonome de la mémoire par l'agent lui-même, comme pour `web\_agent`.

105 add_history_to_messages=True,

→ Active l'ajout de l'historique de conversation aux messages envoyés au modèle.

106 add_datetime_to_instructions=True,

→ Active l'ajout automatique de la date et l'heure courantes aux instructions.

107)

→ Ferme l'appel du constructeur de `finance\_agent`.

109 reasoning_finance_team = Team(

→ Commence la création de l'équipe qui coordonne les deux agents précédents. C'est la classe `Team` d'Agno qui matérialise l'architecture multi-agents : un modèle superviseur reçoit la requête utilisateur, décide comment la répartir entre `web\_agent` et `finance\_agent`, puis consolide leurs réponses.

110 name="Reasoning Finance Team",

→ Définit le nom affiché de l'équipe.

111 mode="coordinate",

→ Définit le mode de fonctionnement de l'équipe sur "coordinate" : dans ce mode, un agent-superviseur (piloté par le `model` défini plus bas) analyse la demande, délègue des sous-tâches aux membres pertinents, puis synthétise leurs réponses en une seule sortie cohérente, au lieu de simplement transférer la conversation à un seul membre ("route") ou de tout exécuter en groupe libre ("collaborate").

112 team_id="reasoning_finance_team",

→ Définit l'identifiant technique unique de l'équipe.

113 model=OpenAIChat(api_key=OPENAI_API_KEY, id="gpt-4o"), # CORRECTION: Utiliser la variable

→ Configure le modèle OpenAI GPT-4o utilisé par le superviseur de l'équipe pour orchestrer et coordonner les deux agents membres.

114 members=[web_agent, finance_agent],

→ Définit les deux agents membres de l'équipe, ceux entre lesquels le superviseur pourra répartir le travail.

115 tools=[ReasoningTools(add_instructions=True)],

→ Fournit à l'équipe l'outil de raisonnement structuré `ReasoningTools`, avec des instructions additionnelles automatiques activées, pour que le superviseur explicite ses étapes de réflexion avant de rendre son verdict final.

116 instructions=[

→ Commence la liste des instructions textuelles données à l'équipe (le prompt système du superviseur).

117 "Collaborate to provide comprehensive financial and investment insights",

→ Instruction demandant à l'équipe de collaborer pour fournir des analyses financières complètes.

118 "Consider both fundamental analysis and market sentiment",

→ Instruction demandant de considérer à la fois l'analyse fondamentale (chiffres de l'entreprise) et le sentiment de marché (actualités, perception).

119 "Provide actionable investment recommendations with clear rationale",

→ Instruction demandant de fournir des recommandations d'investissement claires et justifiées.

120 "Use tables and charts to display data clearly and professionally",

→ Instruction demandant d'utiliser tableaux et graphiques pour présenter les données.

121 "Ensure all claims are supported by data and sources",

→ Instruction demandant que toute affirmation soit appuyée par des données et sources, pour limiter le risque d'hallucination du modèle.

122 "Present findings in a structured format",

→ Instruction demandant de présenter les résultats de façon structurée.

123 "Only output the final consolidated analysis",

→ Instruction demandant de ne produire que l'analyse finale consolidée, sans exposer les échanges intermédiaires entre les deux agents membres.

124 "Don't use emojis in formal financial reports",

→ Instruction interdisant l'usage d'émojis dans les rapports financiers formels, pour garder un ton professionnel.

125 "Build on previous team analyses and preferences stored in memory",

→ Instruction demandant de s'appuyer sur les analyses et préférences précédentes stockées en mémoire long-terme, pour assurer une continuité entre les sessions.

126],

→ Ferme la liste des instructions de l'équipe.

127 storage=PostgresAgentStorage(db_url=db_url, table_name="reasoning_finance_team_sessions"),

→ Configure le stockage des sessions de l'équipe dans sa propre table "reasoning_finance_team_sessions", distincte de celles des agents individuels.

128 memory=memory,

→ Associe le même objet mémoire partagé à l'équipe, complétant le triptyque web_agent / finance_agent / team qui partagent tous la même base de souvenirs utilisateur.

129 markdown=True,

→ Active le formatage Markdown pour les réponses de l'équipe.

130 enable_agentic_memory=True,

→ Active la gestion autonome de la mémoire par l'équipe elle-même.

131 enable_agentic_context=True,

→ Active la gestion autonome du contexte partagé par l'équipe : le superviseur peut décider quelles informations transmettre à quel agent membre, plutôt que de tout leur donner en bloc.

132 `add_datetime_to_instructions=True,`
→ Active l'ajout automatique de la date et l'heure courantes aux instructions de l'équipe.

133 `enable_team_history=True,`
→ Active la conservation de l'historique des échanges de l'équipe, distinct de l'historique de chaque agent pris individuellement.

134 `success_criteria="The team has provided a complete financial analysis with data, visualizations, risk assessment, and actionable recommendations.",`
→ Définit en langage naturel le critère de succès attendu pour une tâche accomplie par l'équipe : le superviseur peut s'en servir pour s'auto-évaluer et décider s'il doit continuer à approfondir avant de rendre sa réponse finale.

135)
→ Ferme l'appel du constructeur de ``reasoning_finance_team``.

137 `app = Playground(`
→ Commence la création de l'application ``Playground``, l'interface web fournie par Agno pour interagir avec les agents et équipes définis plus haut sans écrire de frontend personnalisé.

138 `app_id="multi-agent-reasoning-app-mysql",`
→ Définit l'identifiant de l'application Playground.

139 `name="Multi Agent Reasoning App (MySQL)",`
→ Définit le nom affiché de l'application Playground.

140 `agents=[web_agent, finance_agent],`
→ Enregistre les deux agents individuels dans l'application Playground, pour pouvoir les tester séparément.

141 `teams=[reasoning_finance_team],`
→ Enregistre l'équipe coordonnée dans l'application Playground, pour pouvoir tester le comportement collectif.

142)`.get_app()`
→ Construit et récupère l'application web sous-jacente (une application FastAPI) du Playground, prête à être servie par un serveur ASGI.

144 `def show_database_info():`
→ Définit une fonction utilitaire de diagnostic qui affiche l'état des tables de la base MySQL au démarrage, utile pour vérifier rapidement que la persistance fonctionne comme attendu.

145 `try:`
→ Démarre un bloc try pour capturer les erreurs d'accès à la base sans faire planter l'application.

146 `from sqlalchemy import create_engine, text`
→ Importe localement les fonctions SQLAlchemy nécessaires (voir ligne 37 pour le rôle de SQLAlchemy).

147 `engine = create_engine(db_url)`
→ Crée un moteur SQLAlchemy à partir de l'URL de connexion MySQL.

148 `print("État DB MySQL:")`
→ Affiche un en-tête annonçant l'état de la base de données.

149 `with engine.connect() as connection:`
→ Ouvre une connexion à la base de données dans un contexte géré.

150 `tables = connection.execute(text("SHOW TABLES")).fetchall()`
→ Exécute la commande SQL MySQL listant toutes les tables de la base et récupère tous les résultats.

151 `if tables:`
→ Vérifie si au moins une table existe dans la base.

```
152         for table in tables:
    → Parcourt chaque table trouvée dans la base.

153         table_name = table[0]
    → Extrait le nom de la table depuis la ligne de résultat retournée par SQLAlchemy.

154         count = connection.execute(text(f"SELECT COUNT(*) FROM
`{table_name}`")).fetchone()[0]
    → Compte le nombre d'enregistrements présents dans cette table, ce qui donne un aperçu rapide de l'activité déjà stockée
(sessions, souvenirs...).

155         print(f"{table_name}: {count} enregistrements")
    → Affiche le nom de la table et son nombre d'enregistrements.

156     else:
    → Cas où aucune table n'a été trouvée (première exécution, base vide).

157         print("Aucune table trouvée")
    → Affiche un message indiquant l'absence de table dans la base.

158     except Exception as e:
    → Capture toute exception survenue lors de l'accès à la base.

159         print(f"Impossible d'afficher la base: {e}")
    → Affiche le message d'erreur capturé lors de l'accès à la base.

161 if __name__ == "__main__":
    → Vérifie que le script est exécuté directement (et non importé comme module), condition standard en Python pour délimiter le
point d'entrée du programme.

162     print("Démarrage Application Multi-Agents d'Analyse Financière")
    → Affiche un message annonçant le démarrage de l'application.

163     print(f"Interface: http://localhost:7777")
    → Affiche l'URL locale où l'interface web Playground sera accessible une fois le serveur démarré.

164     print(f"Base: MySQL - {MYSQL_DATABASE}")
    → Affiche le nom de la base de données MySQL utilisée.

165     show_database_info()
    → Appelle la fonction de diagnostic qui affiche l'état des tables de la base avant de démarrer le serveur.

166     serve_playground_app("__main__:app", port=7777)
    → Démarre le serveur web qui expose le Playground sur le port 7777 : la chaîne "__main__:app" indique au serveur ASGI où
trouver l'objet `app` (module courant, variable `app` définie plus haut), un peu comme pour lancer une application
Uvicorn/FastAPI classique.
```

Projet 16 — 16-Whatsapp

Bot WhatsApp gestionnaire de todo-list (Todoist)

Objectif

Bot conversationnel joignable sur WhatsApp qui gère une liste de tâches personnelle : l'utilisateur écrit en langage naturel et l'agent traduit la demande en actions sur un espace Todoist connecté. Le message WhatsApp entrant est reçu via un webhook, traité par un agent IA qui choisit l'outil Todoist à appeler, puis la réponse est renvoyée sur WhatsApp.

Code source, ligne par ligne

16-Whatsapp/AI_TODO_List/main.py

Ce fichier définit le serveur web FastAPI qui reçoit les webhooks WhatsApp, vérifie leur authenticité, et déclenche le traitement des messages entrants en tâche de fond.

106 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard `os` pour lire les variables d'environnement (identifiants et jetons de l'API WhatsApp).

2 import json

→ Importe le module standard `json` pour manipuler des données au format JSON, format utilisé par l'API WhatsApp pour tous ses échanges.

3 import uvicorn

→ Importe Uvicorn, un serveur ASGI (Asynchronous Server Gateway Interface) : c'est le programme qui fait réellement tourner l'application FastAPI et écoute les connexions réseau entrantes, FastAPI ne définissant que la logique applicative.

4 import logging

→ Importe le module standard `logging` pour journaliser des événements (informations, avertissements, erreurs) de façon structurée, plus adaptée en production qu'un simple `print`.

5 import requests

→ Importe `requests`, la bibliothèque Python la plus courante pour effectuer des appels HTTP sortants, ici utilisée pour appeler l'API WhatsApp/Facebook Graph.

6 from threading import Thread

→ Importe la classe `Thread` du module standard `threading`, permettant d'exécuter une fonction en parallèle du fil principal, sans bloquer le programme appelant.

7 from dotenv import load_dotenv

→ Importe `load\_dotenv`, qui charge les variables définies dans un fichier `.env` local vers l'environnement du processus, pour garder les secrets hors du code source.

8 from fastapi import FastAPI, Request, HTTPException

→ Importe les briques principales de FastAPI, un framework web Python moderne et asynchrone : `FastAPI` est la classe d'application, `Request` représente une requête HTTP entrante, `HTTPException` permet de répondre avec un code d'erreur HTTP standard.

9 from fastapi.responses import PlainTextResponse, JSONResponse

→ Importe les classes de réponse HTTP en texte brut et en JSON, utilisées pour formater précisément ce que le serveur renvoie selon ce qu'attend WhatsApp à chaque étape.

10 from whatsapp_utils import process_whatsapp_message, is_valid_whatsapp_message

→ Importe les fonctions utilitaires définies dans `whatsapp\_utils.py` : l'une valide qu'un message reçu correspond bien au format attendu de l'API WhatsApp, l'autre orchestre son traitement complet (voir ce fichier plus bas pour le détail).

12 load_dotenv()

→ Charge les variables d'environnement définies dans le fichier .env du projet.

14 VERSION = os.getenv('VERSION')

→ Récupère la version de l'API Graph WhatsApp utilisée (par exemple "v18.0"), Meta versionnant son API et exigeant de préciser la version dans chaque URL d'appel.

15 PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')

→ Récupère l'identifiant du numéro de téléphone WhatsApp Business associé à l'application, fourni par Meta lors de la configuration du compte professionnel.

16 WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')

→ Récupère le jeton d'accès (access token) à l'API WhatsApp Cloud, qui authentifie chaque requête envoyée à Meta au nom de l'application.

17 RECIPIENT_PHONE_NUMBER = os.getenv('RECIPIENT_PHONE_NUMBER')

→ Récupère le numéro de téléphone destinataire par défaut, utilisé par les routes de test d'envoi de message plus bas dans le fichier.

18 VERIFY_TOKEN = os.getenv('VERIFY_TOKEN')

→ Récupère le jeton de vérification défini par le développeur : c'est une valeur secrète propre à ce projet, que Meta renverra lors de la configuration du webhook pour prouver que la requête de vérification vient bien de Meta et que le développeur contrôle bien cet endpoint.

20 app = FastAPI()

→ Crée l'instance principale de l'application FastAPI, l'objet central autour duquel toutes les routes ("endpoints") du serveur seront déclarées.

22 @app.get("/")

→ Déclare, via un décorateur FastAPI, une route HTTP GET sur le chemin racine "/" de l'API. Les décorateurs `@app.get`/`@app.post` sont la façon idiomatique de FastAPI d'associer une fonction Python à une route web.

23 def read_root():

→ Définit la fonction gérant les requêtes vers la route racine.

24 return {"message": "Welcome to the portal!"}

→ Retourne un dictionnaire Python que FastAPI convertit automatiquement en réponse JSON, message de bienvenue simple confirmant que le serveur tourne.

26 # Required webhook verification for WhatsApp

→ Commentaire indiquant que cette fonction gère la vérification requise du webhook, une étape imposée par Meta lors de la configuration de tout endpoint WhatsApp/Facebook.

27 async def verify(request: Request):

→ Définit une fonction `async` (asynchrone) qui vérifie l'authenticité d'une requête de webhook. Le mot-clé `async` permet à FastAPI de traiter cette requête sans bloquer le reste du serveur pendant son exécution, ce qui améliore le débit sous charge.

28 params = dict(request.query_params)

→ Convertit les paramètres de requête présents dans l'URL (query string) en dictionnaire Python standard, plus simple à manipuler.

29 mode = params.get("hub.mode")

→ Extrait le paramètre "hub.mode" envoyé par Meta lors du handshake de vérification du webhook (attendu à la valeur "subscribe").


```
30 token = params.get("hub.verify_token")
```

→ Extrait le paramètre "hub.verify_token" envoyé par Meta, à comparer avec le `VERIFY\_TOKEN` secret défini côté serveur.

```
31 challenge = params.get("hub.challenge")
```

→ Extrait le paramètre "hub.challenge", une valeur aléatoire que Meta demande de lui renvoyer telle quelle pour prouver que le serveur a bien reçu et validé la requête.

```
33 if mode and token:
```

→ Vérifie que les paramètres mode et token sont bien présents dans la requête.

```
34 if mode == "subscribe" and token == VERIFY_TOKEN:
```

→ Vérifie que le mode est "subscribe" et que le token reçu correspond exactement au token secret configuré côté serveur : c'est ce contrôle qui empêche un tiers non autorisé de faire valider un faux webhook.

```
35 logging.info("WEBHOOK_VERIFIED")
```

→ Journalise un message confirmant que le webhook a été vérifié avec succès.

```
36 return PlainTextResponse(content=challenge, status_code=200)
```

→ Retourne le challenge reçu en texte brut avec un code de statut 200 : c'est exactement le format que l'API Meta exige en retour pour finaliser l'activation du webhook.

```
37 else:
```

→ Cas où le mode ou le token ne correspondent pas aux valeurs attendues.

```
38 logging.warning("VERIFICATION_FAILED")
```

→ Journalise un avertissement signalant l'échec de la vérification.

```
39 raise HTTPException(status_code=403, detail="Verification failed")
```

→ Lève une exception HTTP 403 (Interdit) indiquant l'échec de la vérification ; FastAPI transforme automatiquement cette exception en réponse HTTP correspondante.

```
40 else:
```

→ Cas où le mode ou le token sont absents des paramètres.

```
41 logging.warning("MISSING_PARAMETER")
```

→ Journalise un avertissement signalant l'absence de paramètres requis.

```
42 raise HTTPException(status_code=400, detail="Missing parameters")
```

→ Lève une exception HTTP 400 (Requête invalide) indiquant que des paramètres attendus sont manquants.

```
44 async def handle_message(request: Request):
```

→ Définit une fonction asynchrone qui traite le corps d'une requête webhook entrante, c'est-à-dire les notifications que Meta envoie en POST à chaque nouveau message ou événement WhatsApp.

```
45 try:
```

→ Démarre un bloc try pour capturer les erreurs de décodage JSON.

```
46 body = await request.json()
```

→ Lit et décode le corps de la requête au format JSON de manière asynchrone (`await`), sans bloquer le serveur pendant la lecture du flux réseau.

```
47 except json.JSONDecodeError:
```

→ Capture l'exception levée si le corps reçu n'est pas un JSON valide.

```
48 logging.error("Failed to decode JSON")
```

→ Journalise une erreur indiquant l'échec du décodage JSON.

```

49         return JsonResponse(status_code=400, content={"status": "error", "message": "Invalid JSON
provided"})
    → Retourne une réponse JSON d'erreur avec le code de statut 400.

51     if (
    → Commence une condition composée, écrite sur plusieurs lignes pour la lisibilité, qui va vérifier une chaîne d'accès imbriquée
dans le JSON reçu.

52         body.get("entry", [{}])[0]
    → Récupère le premier élément de la liste "entry" du corps JSON, avec un dictionnaire vide par défaut si la clé est absente : ce
style défensif évite un `KeyError`/`IndexError` sur une charge utile malformée.

53         .get("changes", [{}])[0]
    → Récupère le premier élément de la liste "changes" de cet élément, avec la même logique de valeur par défaut.

54         .get("value", {})
    → Récupère le dictionnaire "value" de cet élément, avec valeur par défaut vide.

55         .get("statuses")
    → Récupère le champ "statuses" du dictionnaire value, s'il existe : ce champ signale des accusés de réception/lecture
WhatsApp plutôt qu'un vrai message texte.

56     ):
    → Ferme la condition composée et ouvre le bloc conditionnel.

57     logging.info("Received a WhatsApp status update.")
    → Journalise que la requête reçue est une simple mise à jour de statut WhatsApp (par exemple "message livré" ou "message
lu"), pas un message à traiter.

58     return JsonResponse(content={"status": "ok"})
    → Retourne une réponse JSON de confirmation sans traiter davantage la requête, pour éviter de déclencher inutilement une
réponse de l'agent.

60     if is_valid_whatsapp_message(body):
    → Vérifie, via la fonction utilitaire importée, si le corps reçu correspond bien à un message WhatsApp valide et exploitable.

61     try:
    → Démarre un bloc try pour capturer les erreurs de lancement du thread.

62         # Run message processing in a background thread
    → Commentaire indiquant que le traitement du message se fait dans un thread en arrière-plan.

63         Thread(target=process_whatsapp_message, args=(body,)).start()
    → Crée et démarre immédiatement un thread qui exécute `process_whatsapp_message(body)` en parallèle. C'est un choix
délibéré : l'API WhatsApp impose de répondre très rapidement (quelques secondes) au webhook sous peine de le considérer en
échec et de renvoyer la notification en boucle ; déléguer le traitement (qui peut être lent, car il appelle un LLM) à un thread permet
de répondre immédiatement sans attendre la fin du traitement.

64     except Exception as e:
    → Capture toute exception survenue lors du démarrage du thread.

65     logging.error(f"Error starting background message thread: {e}")
    → Journalise l'erreur survenue lors du démarrage du thread en arrière-plan.

66     return JsonResponse(content={"status": "ok"})
    → Retourne immédiatement une réponse JSON de confirmation à Meta, indépendamment du résultat final du traitement en
arrière-plan.

67     else:

```

→ Cas où le corps reçu n'est pas un événement WhatsApp valide.

```
68         return JsonResponse(status_code=404, content={"status": "error", "message": "Not a WhatsApp API event"})
```

→ Retourne une réponse JSON d'erreur 404 indiquant que ce n'est pas un événement WhatsApp reconnu.

```
72 @app.get("/test")
```

→ Déclare une route GET sur le chemin "/test", route de commodité pour vérifier manuellement que l'envoi de message fonctionne, en dehors de tout webhook entrant.

```
73 def test():
```

→ Définit la fonction gérant la route de test d'envoi de message.

```
74     url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
```

→ Construit l'URL de l'API Graph de Facebook/Meta pour l'envoi de messages WhatsApp, qui suit le schéma `graph.facebook.com/{version}/{id\_du\_numéro}/messages`.

```
75     headers = {
```

→ Commence la définition des en-têtes HTTP de la requête.

```
76         'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
```

→ Ajoute l'en-tête d'autorisation avec le jeton d'accès WhatsApp, au format standard "Bearer <token>" utilisé par la majorité des API REST modernes.

```
77         'Content-Type': 'application/json'
```

→ Ajoute l'en-tête précisant que le contenu envoyé est au format JSON.

```
78     }
```

→ Ferme le dictionnaire des en-têtes HTTP.

```
79     data = {
```

→ Commence la définition du corps de la requête à envoyer.

```
80         'messaging_product': 'whatsapp',
```

→ Précise que le produit de messagerie utilisé est WhatsApp, champ requis par le format de l'API Graph.

```
81         'to': RECIPIENT_PHONE_NUMBER,
```

→ Définit le numéro de téléphone destinataire du message.

```
82         'type': 'template',
```

→ Précise que le type de message envoyé est un modèle prédéfini ("template") : WhatsApp impose d'utiliser un modèle pré-approuvé par Meta pour initier une conversation avec un utilisateur qui ne vous a pas écrit en premier depuis plus de 24h.

```
83         'template': {
```

→ Commence la définition du modèle de message à utiliser.

```
84             'name': 'hello_world',
```

→ Précise le nom du modèle de message, "hello_world", un modèle de démonstration fourni par défaut par Meta pour tester l'intégration.

```
85             'language': {
```

→ Commence la définition de la langue du modèle.

```
86                 'code': 'en_US'
```

→ Précise le code de langue utilisé pour le modèle, l'anglais américain.

```
87         }
```

→ Ferme le dictionnaire de la langue.

```

88         }
    → Ferme le dictionnaire du modèle de message.

89     }
    → Ferme le dictionnaire des données de la requête.

90     response = requests.post(url, headers=headers, json=data)
    → Envoie la requête HTTP POST à l'API WhatsApp avec les en-têtes et les données définis ; le paramètre `json=data`
    sérialise automatiquement le dictionnaire Python en JSON.

91     return {"message": "Test successful!", "response": response.json()}
    → Retourne un message de succès accompagné de la réponse JSON brute renvoyée par l'API WhatsApp, utile pour
    déboguer.

93 @app.get("/send_message")
    → Déclare une route GET sur le chemin "/send_message" pour envoyer un message texte libre de test.

94 def send_message():
    → Définit la fonction gérant la route d'envoi de message texte.

95     url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
    → Construit l'URL de l'API Graph de Facebook pour l'envoi de messages WhatsApp.

96     headers = {
    → Commence la définition des en-têtes HTTP de la requête.

97         'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
    → Ajoute l'en-tête d'autorisation avec le jeton d'accès WhatsApp.

98         'Content-Type': 'application/json'
    → Ajoute l'en-tête précisant que le contenu envoyé est au format JSON.

99     }
    → Ferme le dictionnaire des en-têtes HTTP.

100     data = {
    → Commence la définition du corps de la requête à envoyer.

101         'messaging_product': 'whatsapp',
    → Précise que le produit de messagerie utilisé est WhatsApp.

102         'to': RECIPIENT_PHONE_NUMBER,
    → Définit le numéro de téléphone destinataire du message.

103         'type': 'text',
    → Précise que le type de message envoyé est un texte libre plutôt qu'un modèle : cela ne fonctionne que si la fenêtre de
    conversation de 24h avec l'utilisateur est encore ouverte.

104         'text': {
    → Commence la définition du contenu textuel du message.

105             'body': 'Hello, how are you?'
    → Définit le texte du message envoyé.

106         }
    → Ferme le dictionnaire du contenu textuel.

107     }
    → Ferme le dictionnaire des données de la requête.

```

```

108     response = requests.post(url, headers=headers, json=data)
    → Envoie la requête POST à l'API WhatsApp avec les en-têtes et les données définis.

109     return {"message": "Message sent successfully!", "response": response.json()}
    → Retourne un message de succès accompagné de la réponse JSON de l'API WhatsApp.

111 @app.get("/webhook")
    → Déclare une route GET sur le chemin "/webhook" : c'est celle que Meta appelle une seule fois, lors de la configuration initiale, pour valider que le serveur contrôle bien cet endpoint (voir la fonction `verify`).

112 async def webhook_get(request: Request):
    → Définit la fonction asynchrone gérant les requêtes GET du webhook.

113     print("GET webhook called!")
    → Affiche un message indiquant que le webhook a été appelé en GET, utile pour confirmer visuellement que Meta a bien atteint le serveur pendant la configuration.

114     return await verify(request)
    → Délègue le traitement de la requête à la fonction `verify` définie plus haut, en attendant (`await`) son résultat puisqu'elle est elle-même asynchrone.

116 @app.post("/webhook")
    → Déclare une route POST sur le chemin "/webhook" : c'est celle que Meta appelle à chaque nouveau message ou événement WhatsApp entrant, en continu après la configuration initiale.

117 async def webhook_post(request: Request):
    → Définit la fonction asynchrone gérant les requêtes POST du webhook.

118     print("POST webhook called!")
    → Affiche un message indiquant que le webhook a été appelé en POST.

119     return await handle_message(request)
    → Délègue le traitement de la requête à la fonction `handle_message` définie plus haut, de manière asynchrone.

121 if __name__ == "__main__":
    → Vérifie que le script est exécuté directement, et non importé comme module.

122     uvicorn.run(app, host="0.0.0.0", port=8000)
    → Démarre le serveur Uvicorn qui fait tourner l'application FastAPI. L'hôte "0.0.0.0" signifie qu'il écoute sur toutes les interfaces réseau de la machine (pas seulement en local), ce qui est nécessaire pour qu'un service externe comme Meta puisse atteindre ce serveur (typiquement via un tunnel ou un déploiement public), sur le port 8000.

```

16-Whatsapp/AI_TODO_List/agent_service.py

Ce fichier configure un agent IA Agno capable de consulter et créer des tâches Todoist en fonction de dates relatives, et expose une fonction simple pour obtenir sa réponse à un message.

99 lignes de code annotées ci-dessous.

```

1 import os
    → Importe le module standard `os` pour lire les variables d'environnement (clés API OpenAI et Todoist).

2 import requests
    → Importe `requests`, la bibliothèque standard pour effectuer des appels HTTP, utilisée ici pour interroger l'API REST de Todoist et une API d'horloge externe.

```

```
3 from datetime import datetime, timedelta
```

→ Importe les classes ``datetime`` (représentation d'une date/heure) et ``timedelta`` (représentation d'une durée) du module standard, utilisées pour calculer des dates relatives comme "demain".

```
4 from textwrap import dedent
```

→ Importe ``dedent``, fonction du module standard ``textwrap`` qui retire l'indentation commune d'un bloc de texte multiligne. Elle est très utile pour écrire un long prompt/instruction directement indenté dans le code (donc lisible) sans que cette indentation ne se retrouve, parasite, dans le texte final envoyé au modèle.

```
5 from agno.agent import Agent
```

→ Importe la classe ``Agent`` du framework Agno, utilisée pour créer un agent IA capable d'appeler des outils (voir le fichier ``reasoning_finance_team.py`` pour une présentation plus détaillée d'Agno).

```
6 from dotenv import load_dotenv
```

→ Importe ``load_dotenv``, qui charge les variables du fichier ``.env`` dans l'environnement du processus, pour garder les clés API hors du code source.

```
7 from agno.models.openai import OpenAIChat
```

→ Importe ``OpenAIChat``, la classe qui encapsule un modèle de langage OpenAI pour qu'Agno puisse l'utiliser comme moteur de raisonnement de l'agent.

```
9 load_dotenv()
```

→ Charge les variables d'environnement définies dans le fichier ``.env`` du projet.

```
11 OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

→ Récupère la clé API OpenAI depuis les variables d'environnement.

```
12 TODOIST_API_KEY = os.getenv("TODOIST_API_KEY")
```

→ Récupère la clé API Todoist depuis les variables d'environnement. Todoist est un service de gestion de tâches qui expose une API REST permettant de lister, créer ou modifier des tâches par programme.

```
13 HEADERS = {
```

→ Commence la définition des en-têtes HTTP réutilisés pour tous les appels à l'API Todoist.

```
14     "Authorization": f"Bearer {TODOIST_API_KEY}",
```

→ Ajoute l'en-tête d'autorisation avec la clé API Todoist, au format standard "Bearer <token>".

```
15     "Content-Type": "application/json"
```

→ Ajoute l'en-tête précisant que le contenu échangé est au format JSON.

```
16 }
```

→ Ferme le dictionnaire des en-têtes HTTP.

```
18 def get_tasks():
```

→ Définit une fonction qui récupère la liste de toutes les tâches Todoist de l'utilisateur. Cette fonction sera plus bas transformée en "outil" utilisable par l'agent IA.

```
19     response = requests.get("https://api.todoist.com/rest/v2/tasks", headers=HEADERS)
```

→ Envoie une requête HTTP GET à l'endpoint REST de Todoist qui liste les tâches, en s'authentifiant via les en-têtes définis plus haut.

```
20     if response.ok:
```

→ Vérifie si la requête a réussi (code de statut HTTP dans la plage 200-299).

```
21         return response.json()
```

→ Retourne le contenu JSON de la réponse, désérialisé en structure Python (liste de dictionnaires représentant les tâches).

```
22     return []
```

→ Retourne une liste vide si la requête a échoué, pour que le code appelant n'ait pas à gérer un cas d'erreur spécifique.

```

24 def add_task(content, due_datetime=None):
    → Définit une fonction qui ajoute une nouvelle tâche Todoist, avec une échéance optionnelle (paramètre par défaut à `None`).

25     payload = {"content": content}
    → Construit le corps de la requête avec le texte de la tâche, seul champ obligatoire pour l'API Todoist.

26     if due_datetime:
    → Vérifie si une date d'échéance a été fournie par l'appelant.

27         payload["due_datetime"] = due_datetime
    → Ajoute la date d'échéance au corps de la requête uniquement si elle est fournie, pour laisser Todoist créer une tâche sans échéance sinon.

28     response = requests.post("https://api.todoist.com/rest/v2/tasks", headers=HEADERS, json=payload)
    → Envoie une requête HTTP POST à l'API Todoist pour créer la tâche avec les données du payload.

29     return response.ok
    → Retourne un booléen indiquant si la création de la tâche a réussi, forme de retour simple adaptée à un outil consommé par un LLM.

31 def get_current_datetime():
    → Définit une fonction qui récupère la date et l'heure actuelles via une API externe, plutôt que de se fier à l'horloge locale du modèle ou du serveur (un LLM n'a par nature aucune notion fiable de l'heure actuelle).

32     url = 'https://timeapi.io/api/Time/current/zone?timeZone=Africa/Casablanca'
    → Définit l'URL d'un service web public (timeapi.io) donnant l'heure actuelle pour le fuseau horaire de Casablanca, pertinent pour la localisation de l'utilisateur visé par ce projet.

33     response = requests.get(url)
    → Envoie une requête GET à l'API d'heure pour obtenir l'heure courante.

34     if response.status_code == 200:
    → Vérifie si la requête a réussi avec le code de statut 200.

35         data = response.json()
    → Convertit le contenu de la réponse en dictionnaire Python via désérialisation JSON.

36         return data["dateTime"]
    → Retourne le champ "dateTime" contenant la date et l'heure actuelles au format texte fourni par l'API.

37     else:
    → Cas où la requête n'a pas réussi.

38     return None
    → Retourne None si la récupération de l'heure a échoué, pour signaler l'absence de valeur exploitable à l'appelant.

40 # NOUVELLES FONCTIONS D'AIDE POUR L'AGENT
    → Commentaire indiquant que les fonctions suivantes sont des fonctions d'aide ("helpers") ajoutées spécifiquement pour être exposées comme outils à l'agent.

41 def get_tasks_for_date(date_str):
    → Définit une fonction qui filtre, parmi toutes les tâches, celles correspondant à une date donnée.

42     """Récupère les tâches pour une date spécifique (format: 2025-08-30)"""
    → Docstring décrivant le rôle de la fonction et le format de date attendu. Cette docstring compte particulièrement ici car, avec le nom de la fonction, c'est ce que le LLM lira pour décider quand et comment appeler cet outil.

43     all_tasks = get_tasks()

```

→ Récupère l'ensemble des tâches Todoist existantes en réutilisant la fonction ``get_tasks`` définie plus haut.

```
44 tasks_for_date = []
```

→ Initialise une liste vide qui contiendra les tâches correspondant à la date recherchée.

```
46 for task in all_tasks:
```

→ Parcourt chaque tâche de la liste complète des tâches.

```
47     if task.get("due") and task["due"].get("date") == date_str:
```

→ Vérifie que la tâche a une échéance définie et que sa date correspond exactement à la date recherchée (comparaison de chaînes au format "AAAA-MM-JJ").

```
48         tasks_for_date.append({
```

→ Commence l'ajout d'un dictionnaire résumant la tâche filtrée à la liste résultat, plutôt que de renvoyer l'objet Todoist complet (allègement utile pour ce que le LLM devra ensuite lire).

```
49             "content": task["content"],
```

→ Ajoute le texte de la tâche au dictionnaire résumé.

```
50             "time": task["due"].get("datetime", "Pas d'heure spécifique")
```

→ Ajoute l'heure de la tâche si disponible, sinon un texte par défaut lisible par l'utilisateur final.

```
51         })
```

→ Ferme le dictionnaire ajouté à la liste des tâches filtrées.

```
53     return tasks_for_date
```

→ Retourne la liste des tâches correspondant à la date demandée.

```
55 def get_today_date():
```

→ Définit une fonction qui retourne la date du jour au format attendu par Todoist et par les autres fonctions de ce fichier.

```
56     """Retourne la date d'aujourd'hui au format 2025-08-29"""
```

→ Docstring décrivant le rôle et le format de retour de la fonction, lue par le LLM comme description de l'outil.

```
57     current_dt = get_current_datetime()
```

→ Récupère la date et l'heure actuelles via l'API externe définie plus haut.

```
58     if current_dt:
```

→ Vérifie que la récupération de la date/heure a réussi.

```
59         # Extraire juste la date (YYYY-MM-DD)
```

→ Commentaire indiquant que seule la partie date doit être extraite de la chaîne datetime complète.

```
60         return current_dt.split("T")[0]
```

→ Découpe la chaîne datetime (format ISO 8601, ex. "2025-08-29T14:30:00") sur le séparateur "T" et retourne uniquement la partie date qui précède.

```
61     return None
```

→ Retourne None si la récupération de la date/heure a échoué.

```
63 def get_tomorrow_date():
```

→ Définit une fonction qui retourne la date du lendemain au format attendu.

```
64     """Retourne la date de demain au format 2025-08-30"""
```

→ Docstring décrivant le rôle et le format de retour de la fonction.

```
65     today = get_today_date()
```

→ Récupère la date du jour en appelant la fonction correspondante.


```

66     if today:
    → Vérifie que la date du jour a bien été obtenue.

67         # Convertir en date et ajouter 1 jour
    → Commentaire indiquant que la date va être convertie en objet manipulable puis incrémentée d'un jour.

68         today_dt = datetime.strptime(today, "%Y-%m-%d")
    → Convertit la chaîne de date du jour en objet `datetime` selon le format attendu, via `strptime` ("string parse time"), l'inverse de `strftime`.

69         tomorrow_dt = today_dt + timedelta(days=1)
    → Ajoute une durée d'un jour (`timedelta`) à la date du jour pour obtenir la date de demain : c'est l'arithmétique de dates standard en Python, plus fiable que de manipuler des chaînes de caractères à la main.

70         return tomorrow_dt.strftime("%Y-%m-%d")
    → Formate la date de demain en chaîne de caractères au format attendu ("strftime" = "string format time").

71     return None
    → Retourne None si la date du jour n'a pas pu être obtenue.

73 agent = Agent(
    → Commence la création de l'agent IA Agno chargé de gérer les tâches de l'utilisateur via le langage naturel.

74     model=OpenAIChat(id="gpt-4o", api_key=OPENAI_API_KEY),
    → Configure le modèle OpenAI GPT-4o utilisé par l'agent avec la clé API, le LLM qui va interpréter les messages de l'utilisateur et décider quels outils appeler.

75     instructions = dedent("""
    → Commence la définition des instructions (prompt système) de l'agent, en enveloppant le texte multiligne dans `dedent` pour retirer l'indentation Python parasite (voir import ligne 4).

76     Tu es un assistant personnel qui gère les tâches via WhatsApp.
    → Ligne du texte d'instruction précisant le rôle général de l'assistant.

78     TES OUTILS DISPONIBLES:
    → Ligne d'instruction introduisant la liste des outils disponibles pour l'agent. Décrire explicitement les outils dans le prompt, en plus de leur enregistrement technique via `tools=[...]` plus bas, aide le modèle à mieux savoir quand et comment les utiliser.

79     - get_current_datetime() : heure actuelle complète
    → Décrit l'outil `get_current_datetime` dans les instructions de l'agent.

80     - get_today_date() : date d'aujourd'hui (format: 2025-08-29)
    → Décrit l'outil `get_today_date` dans les instructions de l'agent.

81     - get_tomorrow_date() : date de demain (format: 2025-08-30)
    → Décrit l'outil `get_tomorrow_date` dans les instructions de l'agent.

82     - get_tasks() : toutes les tâches
    → Décrit l'outil `get_tasks` dans les instructions de l'agent.

83     - get_tasks_for_date(date) : tâches pour une date précise
    → Décrit l'outil `get_tasks_for_date` dans les instructions de l'agent.

84     - add_task(content, due_datetime) : ajouter une tâche
    → Décrit l'outil `add_task` dans les instructions de l'agent.

86     COMMENT RÉPONDRE:
    → Ligne d'instruction introduisant des scénarios concrets de réponse attendus, une technique de prompt engineering appelée

```

"few-shot" : donner des exemples de raisonnement pas à pas plutôt que de simples règles abstraites, pour guider plus fidèlement le modèle.

88 Pour "J'ai combien de tâches demain?":

→ Ligne d'instruction introduisant le scénario de comptage des tâches de demain.

89 1. Appeler `get_tomorrow_date()`

→ Décrit la première étape du scénario : appeler ``get_tomorrow_date``.

90 2. Appeler `get_tasks_for_date(date_de_demain)`

→ Décrit la deuxième étape du scénario : appeler ``get_tasks_for_date`` avec la date de demain obtenue à l'étape précédente.

91 3. Compter et lister les tâches

→ Décrit la troisième étape du scénario : compter et lister les tâches obtenues.

93 Pour "Mes tâches aujourd'hui":

→ Ligne d'instruction introduisant le scénario des tâches du jour.

94 1. Appeler `get_today_date()`

→ Décrit la première étape du scénario : appeler ``get_today_date``.

95 2. Appeler `get_tasks_for_date(date_aujourd_hui)`

→ Décrit la deuxième étape du scénario : appeler ``get_tasks_for_date`` avec la date du jour.

96 3. Lister les tâches

→ Décrit la troisième étape du scénario : lister les tâches obtenues.

98 Pour ajouter une tâche demain à 15h:

→ Ligne d'instruction introduisant le scénario d'ajout de tâche pour demain à 15h.

99 1. Appeler `get_tomorrow_date()` "2025-08-30"

→ Décrit la première étape du scénario avec un exemple concret de résultat attendu.

100 2. Créer `datetime: "2025-08-30T15:00:00Z"`

→ Décrit la deuxième étape du scénario : construire la chaîne `datetime` complète au format ISO 8601 attendu par Todoist.

101 3. Appeler `add_task("contenu", "2025-08-30T15:00:00Z")`

→ Décrit la troisième étape du scénario : appeler ``add_task`` avec le contenu et la date construits.

103 EXEMPLES DE RÉPONSES:

→ Ligne d'instruction introduisant des exemples de formulation de réponses attendues, autre application du "few-shot prompting".

104 - " Vous avez 2 tâches demain : Courses (pas d'heure), Appel (15h)"

→ Exemple de réponse formatée pour une liste de tâches de demain.

105 - " Tâche ajoutée pour demain !"

→ Exemple de réponse formatée confirmant l'ajout d'une tâche.

106 - " Aucune tâche prévue demain"

→ Exemple de réponse formatée indiquant l'absence de tâche prévue.

108 Sois simple, direct et utilise des emojis appropriés.

→ Ligne d'instruction demandant un style de réponse simple, direct, avec émojis, adapté au ton attendu sur WhatsApp.

109 """),

→ Ferme le texte des instructions et l'appel à ``dedent``.

110 `markdown=True,`

→ Active le formatage Markdown pour les réponses de l'agent (peu utile sur WhatsApp qui ne le rend pas, mais laissé par

défaut par Agno).

```
111 tools=[get_current_datetime, get_today_date, get_tomorrow_date, get_tasks, get_tasks_for_date, add_task]
```

→ Fournit à l'agent la liste des fonctions Python définies plus haut, utilisables comme outils : c'est cette ligne qui, techniquement, transforme de simples fonctions en capacités que le LLM peut choisir d'invoquer pendant son raisonnement (appelé "function calling" ou "tool calling").

```
112 )
```

→ Ferme l'appel du constructeur de l'agent.

```
114 def get_response(message):
```

→ Définit une fonction qui obtient la réponse de l'agent à un message donné, point d'entrée simple réutilisé par le reste de l'application (voir `whatsapp\_utils.py`).

```
115     response = agent.run(message)
```

→ Exécute l'agent sur le message reçu : Agno envoie le message au LLM avec les instructions et les outils disponibles, laisse le modèle enchaîner d'éventuels appels d'outils, puis récupère l'objet réponse final.

```
116     return response.content
```

→ Retourne uniquement le contenu textuel de la réponse de l'agent, en ignorant les métadonnées (traces d'appels d'outils, etc.) que contient l'objet réponse complet.

```
118 if __name__ == "__main__":
```

→ Vérifie que le script est exécuté directement, et non importé.

```
119     print(get_response("J'ai combien de tâches demain?"))
```

→ Affiche la réponse de l'agent à une question de test sur les tâches de demain, utile pour tester le comportement de l'agent en isolation, sans passer par WhatsApp.

16-Whatsapp/AI_TODO_List/whatsapp_utils.py

Ce fichier fournit les fonctions utilitaires pour valider, construire et envoyer des messages via l'API WhatsApp, ainsi que pour orchestrer le traitement complet d'un message entrant.

61 lignes de code annotées ci-dessous.

```
1 import requests
```

→ Importe `requests`, la bibliothèque standard pour effectuer des appels HTTP, utilisée ici pour envoyer des messages via l'API Graph de WhatsApp.

```
2 import json
```

→ Importe le module standard `json` pour manipuler des données au format JSON.

```
3 import logging
```

→ Importe le module standard `logging` pour journaliser les événements de façon structurée.

```
4 import re
```

→ Importe le module standard des expressions régulières (non utilisé plus loin dans les lignes annotées, mais disponible pour un éventuel traitement de texte).

```
5 from fastapi.responses import JSONResponse
```

→ Importe la classe de réponse JSON du framework FastAPI, utilisée ici pour retourner des erreurs structurées en cas d'échec d'envoi de message.

```
6 from dotenv import load_dotenv
```

→ Importe `load\_dotenv`, qui charge les variables du fichier `.env` dans l'environnement du processus.

```
7 from agent_service import get_response
```

→ Importe la fonction `get\_response` définie dans `agent\_service.py`, celle qui interroge l'agent IA Agno pour obtenir une réponse à un message utilisateur.

8 import os

→ Importe le module standard `os` pour lire les variables d'environnement.

10 load_dotenv()

→ Charge les variables d'environnement définies dans le fichier .env du projet.

12 WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')

→ Récupère le jeton d'accès à l'API WhatsApp Cloud depuis l'environnement, utilisé pour authentifier les envois de messages sortants.

13 VERSION = os.getenv('VERSION')

→ Récupère la version de l'API Graph WhatsApp depuis les variables d'environnement.

14 PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')

→ Récupère l'identifiant du numéro de téléphone WhatsApp Business depuis l'environnement.

16 def log_http_response(response):

→ Définit une fonction qui journalise les détails d'une réponse HTTP, utile pour déboguer les échanges avec l'API WhatsApp sans avoir à insérer des `print` ad hoc partout.

17 logging.info(f"Status: {response.status_code}")

→ Journalise le code de statut HTTP de la réponse.

18 logging.info(f"Content-type: {response.headers.get('content-type')}")

→ Journalise le type de contenu de la réponse HTTP.

19 logging.info(f"Body: {response.text}")

→ Journalise le corps textuel brut de la réponse HTTP.

22 def get_text_message_input(recipient, text):

→ Définit une fonction qui construit le payload JSON attendu par l'API WhatsApp pour envoyer un message texte simple, en centralisant ce format à un seul endroit du code.

23 return {

→ Commence la construction du dictionnaire retourné représentant le message.

24 "messaging_product": "whatsapp",

→ Précise que le produit de messagerie utilisé est WhatsApp, champ requis par l'API Graph.

25 "recipient_type": "individual",

→ Précise que le destinataire est un individu et non un groupe.

26 "to": recipient,

→ Définit le numéro de téléphone destinataire du message, passé en paramètre.

27 "type": "text",

→ Précise que le type de message envoyé est un texte libre.

28 "text": {"preview_url": False, "body": text},

→ Définit le contenu textuel du message et désactive l'aperçu automatique des liens éventuellement présents dans le texte.

29 }

→ Ferme le dictionnaire retourné par la fonction.

```

32 def generate_response(message):
    → Définit une fonction générant une réponse simple à partir d'un message, non utilisée dans le flux principal (qui passe par `get_response` de l'agent IA) mais laissée comme utilitaire de secours ou de test.

33     # Dummy response for now
    → Commentaire indiquant qu'il s'agit d'une réponse factice ("dummy") temporaire.

34     return message.upper()
    → Retourne le message reçu converti en majuscules, comportement volontairement trivial pour une fonction de test.

37 def send_message(data):
    → Définit la fonction qui envoie effectivement un message construit (via `get_text_message_input`) vers l'API WhatsApp.

38     headers = {
    → Commence la définition des en-têtes HTTP de la requête.

39         "Content-type": "application/json",
    → Ajoute l'en-tête précisant que le contenu envoyé est au format JSON.

40         "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
    → Ajoute l'en-tête d'autorisation avec le jeton d'accès WhatsApp.

41     }
    → Ferme le dictionnaire des en-têtes HTTP.

42     url = f"https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages"
    → Construit l'URL de l'API Graph de Facebook/Meta pour l'envoi de messages WhatsApp.

44     try:
    → Démarre un bloc try pour capturer les erreurs réseau lors de l'envoi.

45         response = requests.post(url, json=data, headers=headers, timeout=10)
    → Envoie la requête POST du message avec un délai maximal (`timeout`) de 10 secondes : sans cette limite, un problème réseau côté Meta pourrait bloquer indéfiniment le thread appelant.

46         response.raise_for_status()
    → Lève automatiquement une exception si le code de statut HTTP indique une erreur (4xx ou 5xx), méthode `requests` pratique pour éviter de vérifier manuellement chaque code de statut.

47         log_http_response(response)
    → Journalise les détails de la réponse HTTP obtenue, via la fonction définie plus haut.

48         return response
    → Retourne l'objet réponse HTTP en cas de succès.

49     except requests.Timeout:
    → Capture spécifiquement l'exception levée en cas de dépassement du délai d'attente défini (`timeout=10`).

50         logging.error("Timeout occurred while sending message")
    → Journalise une erreur indiquant que l'envoi a expiré.

51         return JSONResponse({"status": "error", "message": "Request timed out"}, status_code=408)
    → Retourne une réponse JSON d'erreur avec le code de statut HTTP 408 (Timeout).

52     except requests.RequestException as e:
    → Capture toute autre exception liée à la requête réseau (erreur de connexion, statut d'erreur levé par `raise_for_status`, etc.), classe mère de la plupart des exceptions de `requests`.

53         logging.error(f"Request failed due to: {e}")
    → Journalise l'erreur détaillée survenue lors de la requête.

```

```

51         return JsonResponse({"status": "error", "message": "Failed to send message"}, status_code=500)
    → Retourne une réponse JSON d'erreur avec le code de statut 500 (erreur serveur générique).

52 def is_valid_whatsapp_message(body: dict):
    → Définit une fonction qui vérifie que le corps JSON reçu correspond bien à la structure imbriquée attendue d'un message
    WhatsApp entrant, avant de tenter de le traiter.

53     return (
    → Commence l'expression booléenne composée à retourner.

54         body.get("object")
    → Vérifie que le champ "object" est présent dans le corps (identifie le type d'entité notifiée par Meta).

55         and body.get("entry")
    → Vérifie que le champ "entry" est présent dans le corps.

56         and body["entry"][0].get("changes")
    → Vérifie que le premier élément de "entry" contient un champ "changes".

57         and body["entry"][0]["changes"][0].get("value")
    → Vérifie que le premier élément de "changes" contient un champ "value".

58         and body["entry"][0]["changes"][0]["value"].get("messages")
    → Vérifie que "value" contient un champ "messages".

59         and body["entry"][0]["changes"][0]["value"]["messages"][0]
    → Vérifie que le premier élément de "messages" existe et est non vide (un message effectif, pas une simple mise à jour de
    statut).

60     )
    → Ferme l'expression booléenne composée retournée par la fonction. Grâce au court-circuit du `and` en Python, si un des
    champs manque, l'expression s'arrête et retourne une valeur "fausse" (None ou liste vide) sans lever d'exception, ce qui rend la
    fonction sûre même face à un payload incomplet.

61 def process_whatsapp_message(body: dict):
    → Définit la fonction principale qui traite un message WhatsApp entrant de bout en bout : extraction des données, appel de
    l'agent IA, puis envoi de la réponse. C'est cette fonction qui est exécutée dans le thread en arrière-plan lancé par `main.py`.

62     value = body["entry"][0]["changes"][0]["value"]
    → Extrait le dictionnaire "value" contenant les données du message reçu, une fois sa validité déjà confirmée par
    `is_valid_whatsapp_message`.

63     wa_id = value["contacts"][0]["wa_id"]
    → Extrait l'identifiant WhatsApp (numéro de téléphone) de l'expéditeur du message, utilisé ensuite comme destinataire de la
    réponse.

64     name = value["contacts"][0]["profile"]["name"]
    → Extrait le nom du profil de l'expéditeur du message (non réutilisé plus loin dans ces lignes, mais disponible pour
    personnaliser une réponse).

65     message_body = value["messages"][0]["text"]["body"]
    → Extrait le texte du message envoyé par l'utilisateur, c'est-à-dire la requête en langage naturel qui va être transmise à l'agent
    IA.

66     response_text = get_response(message_body)
    → Obtient la réponse générée par l'agent IA (défini dans `agent_service.py`) pour le texte du message reçu : c'est le point de
    jonction entre la couche WhatsApp et la couche agent.

```

75 `data = get_text_message_input(wa_id, response_text)`

→ Construit le payload du message de réponse à destination de l'expéditeur, au format attendu par l'API WhatsApp.

76 `send_message(data)`

→ Envoie le message de réponse via l'API WhatsApp, bouclant ainsi le cycle complet : message reçu, traité par l'IA, réponse renvoyée à l'utilisateur sur WhatsApp.

Projet 18 — 18-Memory_agent

Assistants sante / immobilier avec memoire persistante (Mem0)

Objectif

Deux assistants conversationnels independants (l'un pour un cabinet de sante, l'autre pour une agence immobiliere) qui se souviennent des echanges precedents avec chaque utilisateur grace a une memoire persistante, au lieu de repartir de zero a chaque conversation.

Code source, ligne par ligne

18-Memory_agent/Health-care/agent.py

Ce fichier définit un agent IA de santé (via Google ADK) doté d'une mémoire persistante Mem0 (avec repli local), capable d'enregistrer des informations patient, de les retrouver et de programmer des rendez-vous.

111 lignes de code annotées ci-dessous.

```
1 import os
  → Importe le module standard `os` pour lire les variables d'environnement (clé API Mem0).

2 from mem0 import MemoryClient
  → Importe `MemoryClient`, le client Python du service Mem0. Mem0 est une plateforme de mémoire pour agents IA : au lieu de gérer soi-même une base vectorielle et la logique d'extraction de faits, on délègue à Mem0 le stockage et la recherche sémantique de souvenirs (préférences, informations personnelles) associés à un utilisateur, avec une API simple `add`/`search`.

3 from google.adk.agents import LlmAgent
  → Importe `LlmAgent`, la classe d'agent principale du Google Agent Development Kit (ADK), un framework de Google pour construire des agents pilotés par un LLM, avec gestion des outils, des instructions et du cycle de vie de l'agent.

4 from google.adk.models.lite_llm import LiteLlm
  → Importe `LiteLlm`, l'adaptateur qui permet à Google ADK d'utiliser des modèles de langage d'autres fournisseurs que Google (ici OpenAI) via LiteLLM, une bibliothèque qui offre une interface unifiée pour appeler de nombreux fournisseurs de LLM avec une syntaxe commune (par exemple "openai/gpt-4o-mini").

5 from dotenv import load_dotenv
  → Importe `load_dotenv`, qui charge les variables du fichier `.env` dans l'environnement du processus, pour garder la clé API Mem0 hors du code source.

7 # Charger les variables d'environnement
  → Commentaire indiquant que les variables d'environnement vont être chargées.

8 load_dotenv()
  → Charge les variables d'environnement définies dans le fichier .env du projet.

10 # Configuration
  → Commentaire introduisant la section de configuration des constantes.

11 USER_ID = "Isma"
  → Définit l'identifiant utilisateur fixe utilisé pour associer les souvenirs à ce patient dans Mem0 ; en production, cette valeur serait typiquement dynamique (un identifiant par patient réel).

12 APP_NAME = "agent"
  → Définit le nom de l'application utilisé en interne.
```


13 `SESSION_ID_SCHEMA_AGENT = "session_agent"`

→ Définit l'identifiant de session fixe pour l'agent.

14 `MEMO_API_KEY = os.getenv("MEMO_API_KEY")`

→ Récupère la clé API Mem0 depuis les variables d'environnement, nécessaire pour authentifier les appels au service de mémoire.

16 `class PatientService:`

→ Déclare la classe qui gère les informations des patients avec mémoire persistante, en encapsulant toute la logique d'accès à Mem0 (ou à son repli local) derrière une interface simple, orientée métier (santé), plutôt que d'exposer directement les appels Mem0 bruts à l'agent.

17 `"""Service pour gérer les informations des patients avec mémoire persistante"""`

→ Docstring décrivant le rôle général de la classe `PatientService`.

19 `def __init__(self, user_id) -> None:`

→ Définit le constructeur de la classe, prenant l'identifiant utilisateur en paramètre.

20 `"""Initialise le service avec l'ID utilisateur"""`

→ Docstring décrivant le rôle du constructeur.

21 `self.user_id = user_id`

→ Stocke l'identifiant utilisateur reçu comme attribut de l'instance, réutilisé dans chaque appel à Mem0.

22 `if MEMO_API_KEY:`

→ Vérifie si une clé API Mem0 a été fournie, pour décider entre utiliser le vrai service Mem0 ou un mode de secours local.

23 `self.memo_client = MemoryClient(api_key=MEMO_API_KEY)`

→ Crée un client Mem0 authentifié avec la clé API, si elle existe : c'est cet objet qui sera utilisé pour tous les appels `add`/`search` vers le service de mémoire.

24 `self.use_mem0 = True`

→ Marque que le service utilisera Mem0 comme backend de mémoire.

25 `else:`

→ Cas où aucune clé API Mem0 n'est disponible.

26 `self.memo_client = None`

→ Initialise le client Mem0 à `None` en l'absence de clé API.

27 `self.use_mem0 = False`

→ Marque que le service n'utilisera pas Mem0 comme backend de mémoire.

28 `self.local_memory = {}`

→ Initialise un dictionnaire de mémoire locale servant de repli ("fallback") : c'est un choix de conception qui rend le service fonctionnel même sans compte Mem0 configuré, au prix d'une mémoire non persistante (perdue au redémarrage) et non sémantique (pas de recherche par similarité de sens, uniquement un stockage brut).

30 `def save_patient_info(self, information: str) -> dict:`

→ Définit la méthode qui enregistre une information patient en mémoire ; cette méthode sera exposée plus bas comme un outil que le LLM de l'agent peut invoquer.

31 `"""Sauvegarde les informations importantes du patient en mémoire."""`

→ Docstring décrivant le rôle de la méthode `save\_patient\_info`. Avec Google ADK, cette docstring et la signature typée de la méthode sont utilisées pour générer automatiquement la description de l'outil fournie au LLM.

32 `try:`

→ Démarre un bloc try pour capturer les erreurs de sauvegarde.

```

33         if self.use_mem0 and self.memo_client:
            → Vérifie si Mem0 est disponible et configuré pour cette instance.

34         response = self.memo_client.add(
            → Commence l'appel à l'API Mem0 pour ajouter une nouvelle information à la mémoire de l'utilisateur.

35             [{"role": "user", "content": information}],
            → Passe l'information sous forme de message au format "chat" (rôle "user" + contenu) : c'est le format attendu par Mem0, qui analyse en interne ce type de message pour en extraire des faits pertinents à mémoriser.

36             user_id=self.user_id,
            → Associe l'enregistrement à l'identifiant utilisateur courant, ce qui permet à Mem0 de cloisonner les souvenirs par utilisateur.

37             run_id="healthcare_session1",
            → Associe l'enregistrement à un identifiant de session fixe "healthcare_session1", permettant à Mem0 de regrouper les souvenirs d'une même conversation/contexte.

38             metadata={"type": "patient_info"}
            → Ajoute des métadonnées précisant le type d'information enregistrée, utile pour filtrer ou catégoriser les souvenirs ultérieurement.

39         )
            → Ferme l'appel à la méthode `add` du client Mem0.

40         return {"status": "success", "message": "Information sauvegardée"}
            → Retourne un dictionnaire confirmant le succès de la sauvegarde via Mem0.

41     else:
        → Cas où Mem0 n'est pas disponible pour cette instance.

42         # Fallback local
        → Commentaire indiquant que la sauvegarde se fait localement en secours, faute de clé Mem0 configurée.

43         if self.user_id not in self.local_memory:
            → Vérifie si l'utilisateur n'a pas encore d'entrée dans la mémoire locale.

44             self.local_memory[self.user_id] = []
            → Crée une liste vide pour cet utilisateur dans la mémoire locale.

45             self.local_memory[self.user_id].append(information)
            → Ajoute l'information à la liste locale de cet utilisateur, stockage en mémoire vive simple sans recherche sémantique.

46         return {"status": "success", "message": "Information sauvegardée localement"}
            → Retourne un dictionnaire confirmant le succès de la sauvegarde locale.

47     except Exception as e:
        → Capture toute exception survenue pendant la sauvegarde.

48         return {"status": "error", "message": f"Erreur: {str(e)}"}
            → Retourne un dictionnaire d'erreur contenant le message de l'exception, plutôt que de laisser l'exception remonter et interrompre l'agent.

50     def retrieve_patient_info(self, query: str) -> dict:
        → Définit la méthode qui récupère des informations patient pertinentes en mémoire, à partir d'une requête en langage naturel.

51         """Récupère les informations pertinentes du patient depuis la mémoire."""
        → Docstring décrivant le rôle de la méthode `retrieve_patient_info`, également utilisée par ADK comme description de l'outil.

```

```

52         try:
    → Démarre un bloc try pour capturer les erreurs de récupération.

53         if self.use_mem0 and self.memo_client:
    → Vérifie si Mem0 est disponible et configuré pour cette instance.

54             results = self.memo_client.search(
    → Commence l'appel à l'API Mem0 pour rechercher des souvenirs pertinents : contrairement à une simple recherche par
mot-clé, Mem0 effectue en interne une recherche sémantique (par similarité de sens des vecteurs d'embedding), ce qui permet
de retrouver une information même formulée différemment de la requête.

55                 query,
    → Passe le texte de la requête de recherche en langage naturel.

56                 user_id=self.user_id,
    → Restreint la recherche à l'identifiant utilisateur courant, pour ne jamais mélanger les souvenirs de deux patients différents.

57                 limit=5,
    → Limite le nombre de résultats retournés à 5, pour ne pas surcharger le contexte envoyé ensuite au LLM.

58                 threshold=0.7,
    → Définit le seuil de pertinence minimal (score de similarité) des résultats à 0.7 : en dessous, un souvenir est jugé trop peu
pertinent par rapport à la requête pour être renvoyé.

59                 output_format="v1.1"
    → Précise le format de sortie attendu de l'API Mem0 (version du schéma de réponse).

60             )
    → Ferme l'appel à la méthode `search` du client Mem0.

62             if results and len(results) > 0:
    → Vérifie que des résultats ont été retournés et que la liste n'est pas vide.

63                 memories = [memory["memory"] for memory in results.get("results", [])]
    → Extrait, via une liste en compréhension, le texte de chaque souvenir depuis la liste de résultats structurés renvoyés par
Mem0.

64                 return {"status": "success", "memories": memories, "count": len(memories)}
    → Retourne un dictionnaire de succès contenant les souvenirs trouvés et leur nombre.

65             else:
    → Cas où aucun résultat n'a été trouvé.

66                 return {"status": "no_results", "memories": [], "count": 0}
    → Retourne un dictionnaire indiquant l'absence de résultats.

67             else:
    → Cas où Mem0 n'est pas disponible pour cette instance.

68             # Fallback local
    → Commentaire indiquant que la recherche se fait localement en secours.

69             if self.user_id in self.local_memory and self.local_memory[self.user_id]:
    → Vérifie que l'utilisateur a des entrées non vides dans la mémoire locale.

70             memories = self.local_memory[self.user_id]
    → Récupère la liste des informations stockées localement pour cet utilisateur ; à la différence de Mem0, ceci retourne tout
l'historique local sans filtrage par pertinence sémantique.

```

```

71         return {"status": "success", "memories": memories, "count": len(memories)}
    → Retourne un dictionnaire de succès contenant les souvenirs locaux et leur nombre.

72     else:
    → Cas où l'utilisateur n'a aucune entrée locale.

73         return {"status": "no_results", "memories": [], "count": 0}
    → Retourne un dictionnaire indiquant l'absence de résultats locaux.

74     except Exception as e:
    → Capture toute exception survenue pendant la récupération.

75         return {"status": "error", "message": f"Erreur: {str(e)}", "memories": [], "count": 0}
    → Retourne un dictionnaire d'erreur avec le message de l'exception et des listes vides, pour garder une structure de réponse cohérente même en cas d'échec.

77     def schedule_appointment(self, date: str, time: str, reason: str) -> dict:
    → Définit la méthode qui programme un rendez-vous médical ; comme les autres méthodes publiques de cette classe, elle sera exposée à l'agent comme un outil actionnable.

78         """Programme un rendez-vous médical."""
    → Docstring décrivant le rôle de la méthode `schedule_appointment`.

79         try:
    → Démarre un bloc try pour capturer les erreurs de programmation du rendez-vous.

80             appointment_id = f"APPT-{hash(date + time) % 10000}"
    → Génère un identifiant de rendez-vous simple en hachant la concaténation de la date et de l'heure, puis en réduisant le résultat à 4 chiffres via un modulo : une génération d'identifiant simplifiée à but de démonstration, plutôt qu'un vrai système d'agenda connecté.

81             return {
    → Commence la construction du dictionnaire de réponse retourné.

82                 "status": "success",
    → Indique que la programmation du rendez-vous a réussi.

83                 "appointment_id": appointment_id,
    → Inclut l'identifiant généré du rendez-vous dans la réponse.

84                 "confirmation": f"Rendez-vous programmé le {date} à {time} pour {reason}",
    → Inclut un message de confirmation récapitulant la date, l'heure et le motif.

85                 "message": "Merci d'arriver 15 minutes en avance."
    → Inclut une consigne demandant d'arriver 15 minutes en avance.

86             }
    → Ferme le dictionnaire de réponse retourné.

87     except Exception as e:
    → Capture toute exception survenue pendant la programmation du rendez-vous.

88         return {"status": "error", "message": f"Erreur: {str(e)}"}
    → Retourne un dictionnaire d'erreur contenant le message de l'exception.

90 # Initialiser le service
    → Commentaire indiquant que le service patient va être instancié une seule fois, au niveau du module.

```

91 patient_service = PatientService(USER_ID)

→ Crée une instance unique de `PatientService` pour l'utilisateur défini par `USER\_ID`, instance qui sera partagée par toutes les méthodes-outils enregistrées sur l'agent plus bas.

93 # IMPORTANT: Créer l'agent au niveau du module pour ADK

→ Commentaire précisant que l'agent doit être défini comme variable de niveau module (et non à l'intérieur d'une fonction) pour être détecté par les outils de Google ADK (CLI ou interface de développement), qui recherchent une variable nommée `root\_agent`.

94 root_agent = LlmAgent(

→ Commence la création de l'agent LLM racine de l'application, via la classe `LlmAgent` de Google ADK.

95 name="healthcare_assistant",

→ Définit le nom de l'agent, "healthcare_assistant".

96 model=LiteLlm(model="openai/gpt-4o-mini"),

→ Configure le modèle utilisé par l'agent via LiteLLM : la chaîne "openai/gpt-4o-mini" indique le fournisseur ("openai") et le modèle ("gpt-4o-mini", une version plus légère et moins coûteuse de GPT-4o), ce qui permet à un agent ADK (nativement pensé pour les modèles Gemini de Google) d'utiliser un modèle OpenAI de façon transparente.

97 description="Assistant IA Santé pour accompagner les patients",

→ Définit la description courte de l'agent.

99 instruction="""

→ Commence la définition de l'instruction système multiligne de l'agent (le prompt qui cadre son comportement).

100 Vous êtes un assistant IA de santé professionnel et bienveillant.

→ Ligne d'instruction définissant la posture générale de l'agent.

102 Quand on vous salue, répondez chaleureusement en vous présentant.

→ Ligne d'instruction précisant le comportement attendu lors d'une salutation.

104 Vos responsabilités:

→ Ligne d'instruction introduisant la liste des responsabilités de l'agent.

105 1. Enregistrer les informations de santé avec save_patient_info

→ Décrit la première responsabilité : enregistrer les informations via l'outil `save\_patient\_info`. Nommer explicitement l'outil dans le prompt aide le LLM à associer la responsabilité déclarée à l'action concrète qu'il peut exécuter.

106 2. Récupérer l'historique avec retrieve_patient_info

→ Décrit la deuxième responsabilité : récupérer l'historique via l'outil `retrieve\_patient\_info`.

107 3. Programmer des rendez-vous avec schedule_appointment

→ Décrit la troisième responsabilité : programmer des rendez-vous via l'outil `schedule\_appointment`.

109 Comportement:

→ Ligne d'instruction introduisant la liste des règles de comportement.

110 - Ton empathique et professionnel

→ Règle de comportement demandant un ton empathique et professionnel.

111 - Ne donnez JAMAIS de diagnostic médical

→ Règle de comportement interdisant de fournir un diagnostic médical : un garde-fou essentiel pour un agent de santé, car un LLM n'est pas un professionnel médical et une fausse assurance de diagnostic pourrait être dangereuse.

112 - Encouragez à consulter un professionnel pour les symptômes sérieux

→ Règle de comportement encourageant la consultation d'un professionnel pour les cas sérieux, complément direct de la règle précédente.

113 - Respectez la confidentialité

→ Règle de comportement demandant le respect de la confidentialité des informations de santé échangées.

115 Exemple de salutation: "Bonjour ! Je suis votre assistant santé IA. Comment puis-je vous aider aujourd'hui ?"

→ Fournit un exemple concret de phrase de salutation attendue, technique de "few-shot prompting" pour guider précisément le ton du modèle.

116 """

→ Ferme le texte de l'instruction système de l'agent.

118 tools=[

→ Commence la liste des outils fournis à l'agent : chaque élément est une méthode liée (bound method) de l'instance `patient\_service`, ce qui permet à Google ADK d'en extraire automatiquement le nom, la signature typée et la docstring pour générer la description de l'outil transmise au LLM.

119 patient_service.save_patient_info,

→ Ajoute la méthode `save\_patient\_info` du service patient comme outil.

120 patient_service.retrieve_patient_info,

→ Ajoute la méthode `retrieve\_patient\_info` du service patient comme outil.

121 patient_service.schedule_appointment,

→ Ajoute la méthode `schedule\_appointment` du service patient comme outil.

122],

→ Ferme la liste des outils de l'agent.

123)

→ Ferme l'appel du constructeur de `root\_agent`.

125 # Code d'initialisation optionnel

→ Commentaire indiquant que le code suivant est une initialisation optionnelle, exécutée seulement si le fichier est lancé directement (pas si l'agent est chargé par l'outillage ADK).

126 if __name__ == "__main__":

→ Vérifie que le script est exécuté directement, et non importé.

127 print("Assistant IA Santé prêt")

→ Affiche un message confirmant que l'assistant est prêt.

128 print(f"Utilisateur: {USER_ID}")

→ Affiche l'identifiant de l'utilisateur configuré.

18-Memory_agent/Real-Estate/agent.py.py

Ce fichier définit un agent IA immobilier (via Google ADK) combinant mémoire persistante Mem0 et recherche vectorielle FAISS sur des annonces, capable d'enregistrer des préférences, retrouver l'historique, chercher des biens et programmer des visites.

237 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard `os` pour lire les variables d'environnement et manipuler des chemins de fichiers.

2 import json

→ Importe le module standard `json` pour charger le fichier de données des propriétés immobilières.

3 import faiss

→ Importe FAISS (Facebook AI Similarity Search), une bibliothèque open-source développée par Meta pour effectuer une recherche de similarité très rapide parmi de grandes quantités de vecteurs numériques (embeddings). C'est un composant central de nombreux systèmes de RAG (Retrieval-Augmented Generation, ou "génération augmentée par récupération") : au lieu de tout injecter dans le prompt, on indexe les données, on retrouve seulement les éléments les plus pertinents par similarité vectorielle, puis on les transmet au modèle.

4 import warnings

→ Importe le module standard `warnings` pour gérer l'affichage des avertissements Python.

5 from mem0 import MemoryClient

→ Importe `MemoryClient`, le client du service Mem0, une plateforme de mémoire pour agents IA qui gère le stockage et la recherche sémantique de souvenirs (voir aussi `18-Memory\_agent/Health-care/agent.py` pour une présentation détaillée de Mem0).

6 from dotenv import load_dotenv

→ Importe `load\_dotenv`, qui charge les variables du fichier `.env` dans l'environnement du processus, pour garder les clés API hors du code source.

7 from google.adk.agents import LlmAgent

→ Importe `LlmAgent`, la classe d'agent principale du Google Agent Development Kit (ADK), framework de Google pour construire des agents pilotés par un LLM.

8 from google.adk.models.lite_llm import LiteLlm

→ Importe `LiteLlm`, l'adaptateur qui permet à Google ADK d'utiliser un modèle d'un autre fournisseur (ici OpenAI) via LiteLLM, bibliothèque offrant une interface unifiée pour appeler de nombreux fournisseurs de LLM.

9 from sentence_transformers import SentenceTransformer

→ Importe `SentenceTransformer`, une classe de la bibliothèque du même nom qui charge un modèle pré-entraîné capable de convertir une phrase en vecteur numérique ("embedding") représentant son sens. C'est ce vecteur qui permettra ensuite de comparer la similarité sémantique entre une requête utilisateur et chaque annonce immobilière, indépendamment du choix exact des mots employés.

11 # Supprimer les avertissements de dépréciation

→ Commentaire indiquant que les avertissements de dépréciation vont être masqués, pour garder une sortie console lisible.

12 warnings.filterwarnings("ignore", category=DeprecationWarning)

→ Configure le filtre d'avertissements pour ignorer les `DeprecationWarning`, souvent émis par les dépendances (FAISS, SentenceTransformer) sans impact sur le fonctionnement réel du script.

14 # Charger les variables d'environnement

→ Commentaire indiquant que les variables d'environnement vont être chargées.

15 load_dotenv()

→ Charge les variables d'environnement définies dans le fichier .env du projet.

17 OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

→ Récupère la clé API OpenAI depuis les variables d'environnement.

18 MEM0_API_KEY = os.getenv("MEM0_API_KEY")

→ Récupère la clé API Mem0 depuis les variables d'environnement.

19 USER_ID = "Ismo"

→ Définit l'identifiant utilisateur fixe utilisé pour associer les souvenirs à ce client dans Mem0.

21 class MemoryTools:

→ Déclare la classe qui regroupe à la fois les outils de mémoire long-terme (Mem0) et de recherche vectorielle (FAISS), formant ainsi la couche "connaissances" complète de l'agent immobilier.

22 `"""Classe pour gérer la mémoire persistante et la recherche vectorielle"""`
→ Docstring décrivant le rôle général de la classe `MemoryTools`.

24 `def __init__(self):`
→ Définit le constructeur de la classe `MemoryTools`.

25 `"""Initialisation du client mémoire et des outils de recherche"""`
→ Docstring décrivant le rôle du constructeur.

26 `self.mem0_client = MemoryClient(MEM0_API_KEY)`
→ Crée un client Mem0 initialisé avec la clé API récupérée, utilisé plus bas pour sauvegarder et rechercher les préférences des clients.

28 `# Initialisation du modèle d'embedding et de l'index FAISS`
→ Commentaire annonçant l'initialisation du système de recherche vectorielle sur les annonces immobilières.

29 `self._initialize_search_system()`
→ Appelle la méthode interne qui met en place le système de recherche FAISS (chargement des données, calcul des embeddings, construction de l'index).

31 `def _initialize_search_system(self):`
→ Définit la méthode privée (préfixée par `\_`, convention Python signalant un usage interne à la classe) qui initialise le système de recherche vectorielle FAISS.

32 `"""Initialise le système de recherche vectorielle FAISS"""`
→ Docstring décrivant le rôle de la méthode `\_initialize\_search\_system`.

33 `# Chargement des données JSON des propriétés`
→ Commentaire annonçant le chargement du fichier JSON des propriétés immobilières.

34 `BASE_DIR = os.path.dirname(os.path.abspath(__file__))`
→ Calcule le chemin absolu du répertoire contenant ce fichier source, pour pouvoir localiser le fichier de données JSON de façon fiable quel que soit le répertoire depuis lequel le script est lancé.

36 `try:`
→ Démarre un bloc try pour capturer une éventuelle absence du fichier de données.

37 `with open(os.path.join(BASE_DIR, "agency_property_data.json"), "r") as f:`
→ Ouvre en lecture le fichier JSON des propriétés situé dans le répertoire du script, dans un gestionnaire de contexte qui garantit sa fermeture propre.

38 `self.data = json.load(f)`
→ Charge et déséréalise le contenu JSON du fichier dans l'attribut `data` de l'instance : c'est la base brute des annonces immobilières, source des données qui seront indexées par FAISS.

39 `except FileNotFoundError:`
→ Capture l'exception levée si le fichier de données n'existe pas.

40 `print(" Fichier de données immobilières non trouvé")`
→ Affiche un avertissement indiquant que le fichier de données est introuvable.

41 `self.data = []`
→ Initialise l'attribut `data` à une liste vide en l'absence du fichier, pour que le reste du code continue de fonctionner (sans résultats de recherche) plutôt que de planter.

43 `# Initialisation du modèle d'embedding`
→ Commentaire annonçant l'initialisation du modèle d'embedding de phrases.


```

44         self.embed_model = SentenceTransformer("all-MiniLM-L6-v2")
    → Charge le modèle pré-entraîné "all-MiniLM-L6-v2", un modèle SentenceTransformer compact et rapide très utilisé pour la
recherche sémantique, qui convertit un texte en vecteur de nombres capturant son sens.

46         # Extraction des textes pour l'embedding
    → Commentaire annonçant l'extraction des textes descriptifs des propriétés.

47         self.texts = self._extract_property_texts()
    → Appelle la méthode qui construit, pour chaque propriété, un texte descriptif concaténé, et stocke la liste résultante : c'est ce
texte qui sera transformé en vecteur d'embedding.

49         # Calcul des embeddings
    → Commentaire annonçant le calcul des embeddings des textes de propriétés.

50         if self.texts:
    → Vérifie que la liste des textes de propriétés n'est pas vide avant de tenter de calculer des embeddings.

51         embeddings = self.embed_model.encode(self.texts, convert_to_numpy=True)
    → Calcule les vecteurs d'embedding de tous les textes en une seule passe, retournés sous forme de tableau NumPy
('convert_to_numpy=True'), format attendu par FAISS pour construire son index.

53         # Création de l'index FAISS
    → Commentaire annonçant la création de l'index de recherche FAISS.

54         dim = embeddings.shape[1]
    → Récupère la dimension des vecteurs d'embedding générés (le nombre de nombres composant chaque vecteur), nécessaire
pour configurer l'index FAISS avec la bonne taille.

55         self.faiss_index = faiss.IndexFlatL2(dim)
    → Crée un index FAISS de type `IndexFlatL2` : un index de recherche "à plat" (exhaustif, sans approximation) utilisant la
distance euclidienne (L2) entre vecteurs pour mesurer leur proximité. C'est l'option la plus simple et la plus précise de FAISS,
adaptée à un petit volume de données comme un catalogue d'annonces.

56         self.faiss_index.add(embeddings)
    → Ajoute tous les vecteurs d'embedding des propriétés à l'index FAISS, le rendant prêt à être interrogé.

57         else:
    → Cas où aucun texte de propriété n'a été trouvé.

58         self.faiss_index = None
    → Laisse l'index FAISS à `None` en l'absence de données à indexer.

60     def _extract_property_texts(self) -> list:
    → Définit la méthode privée qui construit les textes descriptifs de chaque propriété, à partir des champs structurés du JSON,
en vue de leur transformation en embeddings.

61         """Extrait et formate les textes des propriétés pour l'embedding"""
    → Docstring décrivant le rôle de la méthode `_extract_property_texts`.

62         texts = []
    → Initialise une liste vide qui contiendra les textes descriptifs générés.

64         for item in self.data:
    → Parcourt chaque propriété présente dans les données chargées.

65         # Création d'un texte descriptif complet pour chaque propriété
    → Commentaire annonçant la construction du texte descriptif de la propriété courante.

66         property_text = (

```

→ Commence la construction de la chaîne de caractères descriptive, en concaténant plusieurs f-strings sur plusieurs lignes (Python concatène automatiquement des littéraux de chaîne adjacents). Regrouper tous les champs pertinents dans un seul texte permet à l'embedding de capturer une représentation globale de l'annonce plutôt que de devoir indexer chaque champ séparément.

```
67         f"{item.get('title', '')} "
```

→ Ajoute le titre de la propriété au texte descriptif.

```
68         f"{item.get('description', '')} "
```

→ Ajoute la description de la propriété au texte descriptif.

```
69         f"{item.get('location', '')} "
```

→ Ajoute la localisation de la propriété au texte descriptif.

```
70         f"{str(item.get('price', ''))} "
```

→ Ajoute le prix de la propriété, converti en chaîne, au texte descriptif.

```
71         f"{item.get('currency', '')} "
```

→ Ajoute la devise du prix de la propriété au texte descriptif.

```
72         f"{str(item.get('surface_area', ''))} "
```

→ Ajoute la surface de la propriété, convertie en chaîne, au texte descriptif.

```
73         f"{str(item.get('rooms', ''))} "
```

→ Ajoute le nombre de pièces de la propriété, converti en chaîne, au texte descriptif.

```
74         f"{str(item.get('bedrooms', ''))} "
```

→ Ajoute le nombre de chambres de la propriété, converti en chaîne, au texte descriptif.

```
75         f"{str(item.get('bathrooms', ''))} "
```

→ Ajoute le nombre de salles de bain de la propriété, converti en chaîne, au texte descriptif.

```
76         f"{' '.join(item.get('features', []))} "
```

→ Ajoute la liste des caractéristiques de la propriété (features), jointes par des espaces, au texte descriptif.

```
77         f"{item.get('status', '')}"
```

→ Ajoute le statut de la propriété au texte descriptif.

```
78     )
```

→ Ferme la chaîne de caractères descriptive concaténée.

```
79     texts.append(property_text)
```

→ Ajoute le texte descriptif de cette propriété à la liste des textes.

```
81     return texts
```

→ Retourne la liste complète des textes descriptifs des propriétés, dans le même ordre que `self.data`, ordre essentiel pour pouvoir ensuite retrouver la propriété d'origine à partir de l'indice renvoyé par FAISS.

```
83     def save_info(self, information: str) -> dict:
```

→ Définit la méthode qui enregistre une information client (préférences, budget, etc.) en mémoire long-terme via Mem0 ; elle sera exposée comme outil à l'agent.

```
84     """Sauvegarde les préférences importantes du client en mémoire"""
```

→ Docstring décrivant le rôle de la méthode `save\_info`, utilisée par Google ADK pour générer la description de l'outil.

```
85     try:
```

→ Démarre un bloc try pour capturer les erreurs de sauvegarde.

```
86         # Stockage dans Mem0
```

→ Commentaire annonçant le stockage de l'information dans Mem0.

```
87         response = self.mem0_client.add(
```

→ Commence l'appel à l'API Mem0 pour ajouter une nouvelle information à la mémoire du client.

```
88         [{"role": "user", "content": information}],
```

→ Passe l'information sous forme de message au format "chat" (rôle "user" + contenu), format attendu par Mem0 pour en extraire des faits pertinents.

```
89         user_id=USER_ID,
```

→ Associe l'enregistrement à l'identifiant utilisateur global `USER\_ID`.

```
90         run_id="real_estate_session",
```

→ Associe l'enregistrement à un identifiant de session fixe "real_estate_session".

```
91         metadata={"type": "property_info"})
```

→ Ajoute des métadonnées précisant le type d'information enregistrée.

```
92     )
```

→ Ferme l'appel à la méthode `add` du client Mem0.

```
94     return {
```

→ Commence la construction du dictionnaire de réponse en cas de succès.

```
95         "status": "success",
```

→ Indique que la sauvegarde de l'information a réussi.

```
96         "message": "Informations client sauvegardées avec succès",
```

→ Inclut un message confirmant la sauvegarde des informations client.

```
97         "response": response
```

→ Inclut la réponse brute retournée par l'API Mem0, utile pour un éventuel débogage ou traçabilité.

```
98     }
```

→ Ferme le dictionnaire de réponse en cas de succès.

```
100     except Exception as e:
```

→ Capture toute exception survenue pendant la sauvegarde.

```
101     return {
```

→ Commence la construction du dictionnaire de réponse en cas d'erreur.

```
102         "status": "error",
```

→ Indique que la sauvegarde de l'information a échoué.

```
103         "message": f"Erreur lors de la sauvegarde: {str(e)}"
```

→ Inclut le message d'erreur décrivant l'exception survenue.

```
104     }
```

→ Ferme le dictionnaire de réponse en cas d'erreur.

```
106     def retrieve_info(self, query: str) -> dict:
```

→ Définit la méthode qui récupère des informations client pertinentes en mémoire, à partir d'une requête en langage naturel.

```
107     """Récupère les informations pertinentes du client depuis la mémoire"""
```

→ Docstring décrivant le rôle de la méthode `retrieve\_info`.

```

108         try:
    → Démarre un bloc try pour capturer les erreurs de récupération.

109         # Recherche dans Mem0
    → Commentaire annonçant la recherche d'informations dans Mem0.

110         results = self.mem0_client.search(
    → Commence l'appel à l'API Mem0 pour rechercher des souvenirs pertinents via recherche sémantique (par similarité de sens
    plutôt que par mots-clés exacts).

111         query,
    → Passe le texte de la requête de recherche.

112         user_id=USER_ID,
    → Restreint la recherche à l'identifiant utilisateur global `USER_ID`.

113         limit=5,
    → Limite le nombre de résultats retournés à 5.

114         threshold=0.7, # Seuil plus élevé pour des résultats plus pertinents
    → Définit le seuil de pertinence minimal des résultats à 0.7 : un score de similarité plus élevé signifie que seuls les souvenirs
    les plus proches sémantiquement de la requête sont retenus, réduisant le bruit dans les réponses.

115         output_format="v1.1"
    → Précise le format de sortie attendu de l'API Mem0.

116     )
    → Ferme l'appel à la méthode `search` du client Mem0.

118     # Formatage et retour des résultats
    → Commentaire annonçant le formatage des résultats obtenus.

119     if results and len(results) > 0:
    → Vérifie que des résultats ont été retournés et que la liste n'est pas vide.

120         memories = [memory["memory"] for memory in results.get("results", [])]
    → Extrait le texte de chaque souvenir depuis la liste de résultats structurés renvoyés par Mem0.

121         return {
    → Commence la construction du dictionnaire de réponse en cas de résultats trouvés.

122             "status": "success",
    → Indique que la récupération des informations a réussi.

123             "memories": memories,
    → Inclut la liste des souvenirs extraits dans la réponse.

124             "count": len(memories)
    → Inclut le nombre de souvenirs trouvés dans la réponse.

125         }
    → Ferme le dictionnaire de réponse en cas de résultats trouvés.

126     else:
    → Cas où aucun résultat n'a été trouvé.

127         return {
    → Commence la construction du dictionnaire de réponse en l'absence de résultats.

128             "status": "no_results",

```

→ Indique qu'aucun résultat n'a été trouvé.

```
129         "memories": [],
```

→ Inclut une liste vide de souvenirs dans la réponse.

```
130         "count": 0
```

→ Inclut un compte de zéro souvenir dans la réponse.

```
131     }
```

→ Ferme le dictionnaire de réponse en l'absence de résultats.

```
133     except Exception as e:
```

→ Capture toute exception survenue pendant la récupération.

```
134         return {
```

→ Commence la construction du dictionnaire de réponse en cas d'erreur.

```
135         "status": "error",
```

→ Indique que la récupération des informations a échoué.

```
136         "message": f"Erreur lors de la récupération: {str(e)}",
```

→ Inclut le message d'erreur décrivant l'exception survenue.

```
137         "memories": [],
```

→ Inclut une liste vide de souvenirs dans la réponse d'erreur.

```
138         "count": 0
```

→ Inclut un compte de zéro souvenir dans la réponse d'erreur.

```
139     }
```

→ Ferme le dictionnaire de réponse en cas d'erreur.

```
141     def search_faiss(self, query: str, top_k: int = 5) -> dict:
```

→ Définit la méthode qui recherche les propriétés les plus proches sémantiquement d'une requête via FAISS ; c'est le cœur du mécanisme de type RAG de cet agent : au lieu de connaître les annonces "par cœur", l'agent les retrouve dynamiquement en fonction de la demande du client.

```
142     """Recherche FAISS et retourne les top_k résultats"""
```

→ Docstring décrivant le rôle de la méthode `search\_faiss`, avec `top\_k` comme nombre de résultats à retourner.

```
143         try:
```

→ Démarre un bloc try pour capturer les erreurs de recherche.

```
144             if not self.faiss_index or not self.texts:
```

→ Vérifie si l'index FAISS ou la liste de textes n'est pas disponible (cas où le fichier de données était absent à l'initialisation).

```
145             return {
```

→ Commence la construction du dictionnaire de réponse en cas d'index non initialisé.

```
146             "status": "error",
```

→ Indique que la recherche a échoué faute d'index disponible.

```
147             "message": "Index FAISS non initialisé",
```

→ Inclut un message précisant que l'index FAISS n'est pas initialisé.

```
148             "results": [],
```

→ Inclut une liste vide de résultats dans la réponse d'erreur.

```
149             "count": 0
```

→ Inclut un compte de zéro résultat dans la réponse d'erreur.

```
150         }
```

→ Ferme le dictionnaire de réponse en cas d'index non initialisé.

```
152         # Encodage de la requête
```

→ Commentaire annonçant l'encodage vectoriel de la requête de recherche.

```
153         query_vec = self.embed_model.encode([query], convert_to_numpy=True)
```

→ Calcule le vecteur d'embedding de la requête utilisateur, avec le même modèle SentenceTransformer que celui utilisé pour indexer les propriétés : c'est essentiel, car la comparaison de similarité n'a de sens que si la requête et les données indexées sont projetées dans le même espace vectoriel.

```
155         # Recherche dans l'index FAISS
```

→ Commentaire annonçant la recherche des voisins les plus proches dans l'index FAISS.

```
156         D, I = self.faiss_index.search(query_vec, top_k)
```

→ Recherche les `top\_k` vecteurs les plus proches du vecteur de la requête et retourne deux tableaux : `D` les distances (plus la valeur est faible, plus la propriété est proche sémantiquement) et `I` les indices des propriétés correspondantes dans les données d'origine.

```
158         # Récupération des résultats originaux
```

→ Commentaire annonçant la récupération des données originales correspondant aux indices trouvés.

```
159         results = []
```

→ Initialise une liste vide qui contiendra les propriétés trouvées.

```
160         for idx in I[0]:
```

→ Parcourt les indices des résultats trouvés pour la première (et unique) requête envoyée à `search` (d'où `I[0]`).

```
161             if idx < len(self.data): # Vérification des limites
```

→ Vérifie que l'indice trouvé est valide par rapport à la taille des données, garde-fou défensif contre un indice hors limites que FAISS pourrait renvoyer (par exemple -1 si moins de résultats que `top\_k` sont disponibles).

```
162             results.append(self.data[idx])
```

→ Ajoute la propriété originale (avec tous ses champs) correspondant à cet indice à la liste des résultats, en revenant des vecteurs abstraits aux données métier exploitables.

```
164         return {
```

→ Commence la construction du dictionnaire de réponse en cas de succès.

```
165             "status": "success",
```

→ Indique que la recherche FAISS a réussi.

```
166             "results": results,
```

→ Inclut la liste des propriétés trouvées dans la réponse.

```
167             "count": len(results)
```

→ Inclut le nombre de propriétés trouvées dans la réponse.

```
168         }
```

→ Ferme le dictionnaire de réponse en cas de succès.

```
170     except Exception as e:
```

→ Capture toute exception survenue pendant la recherche FAISS.

```
171     return {
```

→ Commence la construction du dictionnaire de réponse en cas d'erreur.

```

172         "status": "error",
    → Indique que la recherche FAISS a échoué.

173         "message": f"Erreur lors de la recherche: {str(e)}",
    → Inclut le message d'erreur décrivant l'exception survenue.

174         "results": [],
    → Inclut une liste vide de résultats dans la réponse d'erreur.

175         "count": 0
    → Inclut un compte de zéro résultat dans la réponse d'erreur.

176     }
    → Ferme le dictionnaire de réponse en cas d'erreur.

178     def schedule_appointment(self, date: str, time: str, reason: str) -> dict:
    → Définit la méthode qui programme un rendez-vous (visite immobilière) ; elle sera exposée comme outil à l'agent.

179         """Programme un rendez-vous"""
    → Docstring décrivant le rôle de la méthode `schedule_appointment`.

180         try:
    → Démarre un bloc try pour capturer les erreurs de programmation du rendez-vous.

181             # Dans une vraie application, ceci se connecterait à un système de planification
    → Commentaire précisant qu'une application réelle se connecterait à un vrai système de planification (agenda partagé, CRM immobilier), cette implémentation étant simplifiée à but de démonstration.

182             appointment_id = f"APPT-{hash(date + time) % 10000}"
    → Génère un identifiant de rendez-vous simple en hachant la concaténation de la date et de l'heure, réduit à 4 chiffres via un modulo.

183         return {

184             "status": "success",
    → Indique que la programmation du rendez-vous a réussi.

185             "appointment_id": appointment_id,
    → Inclut l'identifiant généré du rendez-vous dans la réponse.

186             "confirmation": f"Appointment scheduled for {date} at {time} for {reason}"
    → Inclut un message de confirmation récapitulant la date, l'heure et le motif du rendez-vous.

187         }
    → Ferme le dictionnaire de réponse retourné.

188     except Exception as e:
    → Capture toute exception survenue pendant la programmation du rendez-vous.

189     return {

190         "status": "error",
    → Indique que la programmation du rendez-vous a échoué.

191         "message": f"Erreur lors de la planification: {str(e)}"
    → Inclut le message d'erreur décrivant l'exception survenue.

192     }

```

→ Ferme le dictionnaire de réponse en cas d'erreur.

```
196     def get_contact_info(self) -> dict:
```

→ Définit la méthode qui retourne les informations de contact de l'agence, dernier outil "statique" (sans dépendance à Memo0 ou FAISS) mis à disposition de l'agent.

```
197         """Retourne les informations de contact de l'agence"""
```

→ Docstring décrivant le rôle de la méthode `get\_contact\_info`.

```
198         return {
```

→ Commence la construction du dictionnaire de réponse retourné.

```
199             "status": "success",
```

→ Indique que la récupération des informations de contact a réussi.

```
200             "contact_info": "You can contact the agency at +33 6 12 34 56 78 or by email at  
realestateagency@mail.com"
```

→ Inclut le texte des coordonnées de contact de l'agence, téléphone et email, en dur puisqu'il s'agit d'une information fixe qui n'a pas besoin de mémoire ni de recherche.

```
201         }
```

→ Ferme le dictionnaire de réponse retourné.

```
204 # =====
```

→ Ligne de séparation visuelle commentée introduisant une nouvelle section du fichier.

```
205 # CRÉATION DE L'AGENT IMMOBILIER
```

→ Commentaire titrant la section de création de l'agent immobilier.

```
206 # =====
```

→ Ligne de séparation visuelle commentée fermant le titre de section.

```
207 def create_real_estate_agent():
```

→ Définit la fonction qui construit et configure l'agent IA immobilier complet, en l'encapsulant dans une fonction (plutôt qu'au niveau module comme dans le projet Health-care) pour pouvoir, par exemple, la ré-invoquer ou la tester indépendamment.

```
208     """Crée et configure l'agent IA immobilier professionnel"""
```

→ Docstring décrivant le rôle de la fonction `create\_real\_estate\_agent`.

```
210     # Initialisation des outils mémoire
```

→ Commentaire annonçant l'initialisation des outils de mémoire et de recherche.

```
211     memory_tools = MemoryTools()
```

→ Crée une instance de `MemoryTools`, ce qui déclenche au passage le chargement des données immobilières, le calcul des embeddings et la construction de l'index FAISS (voir le constructeur plus haut).

```
213     # Création de l'agent avec LiteLlm pour OpenAI
```

→ Commentaire annonçant la création de l'agent utilisant LiteLLM pour accéder à un modèle OpenAI depuis Google ADK.

```
214     root_agent = LlmAgent(
```

→ Commence la création de l'agent LLM, local à cette fonction (variable locale, à ne pas confondre avec un éventuel `root\_agent` de niveau module attendu par l'outillage ADK).

```
215         name="real_estate_assistant",
```

→ Définit le nom de l'agent, "real_estate_assistant".

```
216         model=LiteLlm(model="openai/gpt-4o-mini"),
```

→ Configure le modèle utilisé par l'agent via LiteLLM, gpt-4o-mini d'OpenAI (voir ligne 8 pour le rôle de LiteLLM).

218 `description=""`

→ Commence la définition de la description multiligne de l'agent.

219 `AI-powered Real Estate Assistant for a reputable agency, supporting clients by recording preferences,`

→ Ligne de description présentant l'agent comme assistant immobilier alimenté par l'IA.

220 `retrieving past inquiries, searching property listings, scheduling viewings, and providing professional guidance.`

→ Ligne de description énumérant les fonctionnalités de récupération, recherche, planification et conseil.

221 `""`,

→ Ferme le texte de la description de l'agent.

223 `instruction=""`

→ Commence la définition de l'instruction système multiligne de l'agent (le prompt qui cadre son comportement et son ton).

224 `You are a professional Real Estate Assistant representing a top-tier agency in France. Always speak in a courteous, confident, and expert manner—just like a senior agent at our firm.`

→ Ligne d'instruction définissant la posture professionnelle et le ton de l'agent.

226 `When you greet a user:`

→ Ligne d'instruction introduisant le comportement attendu lors d'une salutation.

227 `- Use a formal opening (e.g., "Good day. We are a Real Estate Agency. How may I assist you with finding your next property?")`

→ Précise le type de formule d'accueil formelle à utiliser, avec un exemple concret, technique de "few-shot prompting".

229 `Core responsibilities:`

→ Ligne d'instruction introduisant la liste des responsabilités principales de l'agent, chacune associée explicitement à l'outil correspondant.

230 `1. **Recording client preferences and details** using the `save_info` tool (e.g., preferred neighborhoods, budgets, must-have features).`

→ Décrit la responsabilité d'enregistrer les préférences client via l'outil `save\_info`, qui écrit dans Mem0.

231 `2. **Retrieving past inquiries or saved preferences** via `retrieve_info` so the conversation feels consistent and personalized.`

→ Décrit la responsabilité de récupérer les demandes ou préférences passées via `retrieve\_info`, qui interroge Mem0 par recherche sémantique.

232 `3. **Searching property listings** with `search_faiss` to provide concise summaries of matching properties, including location, price (formatted in EUR), surface area, bedrooms, and standout features.`

→ Décrit la responsabilité de rechercher des annonces via `search\_faiss`, l'outil de recherche vectorielle défini plus haut, et d'en résumer les caractéristiques.

233 `4. **Scheduling property viewings or appointments** via `schedule_appointment` once the client has identified properties of interest.`

→ Décrit la responsabilité de programmer des visites via `schedule\_appointment`.

234 `5. **Providing agency contact information** by invoking `get_contact_info` whenever the user requests it or if you need them to have our phone/email.`

→ Décrit la responsabilité de fournir les coordonnées de l'agence via `get\_contact\_info`.

236 `GUIDELINES & BEHAVIOR:`

→ Ligne d'instruction introduisant la liste des directives de comportement.

237 `- Maintain a professional, friendly, and helpful tone at all times—like a high-end real estate`

broker.

→ Directive demandant un ton professionnel, amical et serviable en toute circonstance.

238 - Do NOT offer legal, financial, or investment advice. Instead, suggest they consult qualified professionals for those matters.

→ Directive interdisant de donner des conseils juridiques, financiers ou d'investissement, garde-fou important pour un agent IA qui ne doit pas se substituer à des professionnels réglementés.

239 - Always ask for missing contact details of the client using `get\_contact\_info` when appropriate (e.g., "May I share our agency's phone number with you so you can call directly?").

→ Directive demandant de proposer les coordonnées de l'agence quand c'est pertinent.

240 - Respect client privacy—never expose or share their personal info outside the scope of this conversation.

→ Directive demandant de respecter la confidentialité des informations personnelles du client.

241 - Before requesting any new information, check existing memory with `retrieve\_info`.

→ Directive demandant de vérifier la mémoire existante avant de redemander une information déjà connue, pour éviter de faire répéter le client et améliorer la fluidité de l'expérience.

242 - Offer only the most relevant listings to avoid overwhelming the client.

→ Directive demandant de ne proposer que les annonces les plus pertinentes, cohérente avec le paramètre `top\_k` et le seuil de similarité utilisés côté recherche.

243 - Confirm actions (saving info, scheduling viewings, offering contact info) clearly and politely.

→ Directive demandant de confirmer clairement et poliment chaque action effectuée.

245 COMPLIANCE & PRIVACY:

→ Ligne d'instruction introduisant la section conformité et confidentialité.

246 - Treat all client data as strictly confidential.

→ Directive demandant de traiter toutes les données client comme strictement confidentielles.

248 Your goal is to emulate a boutique real estate agency: knowledgeable, professional, and fully attentive to each client's needs.

→ Ligne d'instruction résumant l'objectif général de l'agent en tant qu'agence boutique.

249 """

→ Ferme le texte de l'instruction système de l'agent.

251 # Outils disponibles pour l'agent

→ Commentaire annonçant la liste des outils fournis à l'agent.

252 tools=[

→ Commence la liste des outils fournis à l'agent : chaque élément est une méthode liée de l'instance `memory\_tools`, dont Google ADK extrait automatiquement nom, signature et docstring pour construire la description transmise au LLM.

253 memory_tools.save_info,

→ Ajoute la méthode `save\_info` de l'instance `memory\_tools` comme outil.

254 memory_tools.retrieve_info,

→ Ajoute la méthode `retrieve\_info` de l'instance `memory\_tools` comme outil.

255 memory_tools.search_faiss,

→ Ajoute la méthode `search\_faiss` de l'instance `memory\_tools` comme outil.

256 memory_tools.schedule_appointment,

→ Ajoute la méthode `schedule\_appointment` de l'instance `memory\_tools` comme outil.

```

257         memory_tools.get_contact_info,
    → Ajoute la méthode `get_contact_info` de l'instance `memory_tools` comme outil.

258     ],
    → Ferme la liste des outils de l'agent.

259 )
    → Ferme l'appel du constructeur de `root_agent` local à la fonction.

261     return root_agent
    → Retourne l'agent immobilier configuré à l'appelant.

264 # =====
    → Ligne de séparation visuelle commentée introduisant une nouvelle section.

265 # POINT D'ENTRÉE PRINCIPAL
    → Commentaire titrant la section du point d'entrée principal du script.

266 # =====
    → Ligne de séparation visuelle commentée fermant le titre de section.

267 if __name__ == "__main__":
    → Vérifie que le script est exécuté directement, et non importé.

268     try:
    → Démarre un bloc try pour capturer les erreurs d'initialisation au lancement.

269         # Initialisation des outils mémoire
    → Commentaire annonçant l'initialisation des outils de mémoire pour le point d'entrée.

270         memory_tools = MemoryTools()
    → Crée une instance de `MemoryTools` pour le test en exécution directe du script, redondante avec celle créée dans
    `create_real_estate_agent()` mais utile ici pour afficher des informations de diagnostic (nombre de propriétés chargées) avant de
    créer l'agent.

272         # Créer l'agent immobilier
    → Commentaire annonçant la création de l'agent immobilier complet.

273         real_estate_agent = create_real_estate_agent()
    → Appelle la fonction qui construit l'agent immobilier configuré, prêt à être utilisé.

275         print(" Assistant IA Immobilier initialisé avec succès!")
    → Affiche un message confirmant l'initialisation réussie de l'assistant.

276         print(f" Utilisateur: {USER_ID}")
    → Affiche l'identifiant de l'utilisateur configuré.

277         print(" Prêt à accompagner vos clients...")
    → Affiche un message indiquant que l'assistant est prêt à accompagner des clients.

279         # Informations sur les données chargées
    → Commentaire annonçant l'affichage d'informations sur les données chargées.

280         if hasattr(memory_tools, 'data') and memory_tools.data:
    → Vérifie que l'instance `memory_tools` possède bien un attribut `data` et que celui-ci n'est pas vide, avant d'en afficher la
    taille.

281         print(f" {len(memory_tools.data)} propriétés chargées dans la base")
    → Affiche le nombre de propriétés chargées dans la base de données, confirmant que l'indexation FAISS a bien eu de la

```

matière à traiter.

282 `else:`

→ Cas où aucune donnée de propriété n'a été chargée.

283 `print(" Aucune donnée de propriété chargée")`

→ Affiche un avertissement indiquant l'absence de données de propriété, ce qui signifie que `search\_faiss` ne pourra retourner aucun résultat.

285 `except Exception as e:`

→ Capture toute exception survenue pendant l'initialisation au lancement.

286 `print(f" Erreur lors de l'initialisation: {e}")`

→ Affiche un message d'erreur décrivant l'exception survenue lors de l'initialisation.

287 `print("Vérifiez vos clés API et dépendances")`

→ Affiche un conseil invitant à vérifier les clés API et les dépendances installées.

Projet 20 — 20-Agno-Team

Analyse de contrats par 3 agents specialises coordonnees

Objectif

Analyse un contrat PDF uploadé via une interface Streamlit, en le faisant traiter par plusieurs agents specialises coordonnees (extraction des clauses, analyse des risques, verification de conformite) qui produisent ensemble un rapport structure. L'utilisateur peut ensuite poser des questions complementaires sur le contrat directement dans l'interface.

Code source, ligne par ligne

20-Agno-Team/contract_analysis.py

Ce fichier définit, avec le framework agno, trois agents IA spécialisés dans l'analyse de contrats (structure, juridique, négociation) ainsi qu'une équipe (Team) qui les orchestre et synthétise leurs rapports en un document final unique.

165 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard os, qui donne accès aux fonctionnalités du système d'exploitation ; ici il servira à lire les variables d'environnement (comme la clé API) via os.getenv.

2 from dotenv import load_dotenv

→ Importe load_dotenv de la bibliothèque python-dotenv. Cet outil lit un fichier .env local (non versionné dans git) et injecte ses paires clé=valeur comme variables d'environnement du processus. Le but est de garder les secrets (clés API, mots de passe) hors du code source, pour ne pas les exposer accidentellement dans un dépôt Git.

3 from agno.agent import Agent

→ Importe la classe Agent du framework agno. Agno est un framework Python pour construire des agents IA : il encapsule un modèle de langage (LLM) avec un rôle, des instructions et éventuellement des outils, afin que le modèle puisse accomplir une tâche de façon autonome plutôt que de se contenter de répondre à un seul prompt isolé.

4 from agno.team.team import Team

→ Importe la classe Team d'agno, qui permet d'orchestrer plusieurs agents ensemble. Plutôt que d'enchaîner manuellement des appels à différents agents dans le code, Team gère la distribution du travail entre agents membres et la synthèse de leurs réponses par un agent "leader". C'est utile quand une tâche se découpe naturellement en sous-tâches spécialisées, comme ici l'analyse structurelle, juridique et de négociation d'un contrat.

5 from agno.models.groq import Groq

→ Importe le connecteur Groq, qui permet à un Agent agno d'utiliser un modèle de langage hébergé sur l'infrastructure Groq. Groq est un fournisseur d'inférence connu pour sa rapidité (matériel LPU dédié à l'inférence de LLM) ; il héberge notamment des modèles à poids ouverts (open-weight) comme la famille gpt-oss utilisée ici. C'est ce connecteur qui définit quel "cerveau" LLM chaque agent utilisera.

6 from textwrap import dedent

→ Importe dedent, un utilitaire de la bibliothèque standard qui retire l'indentation commune d'une chaîne multi-lignes. Il permet d'écrire de longues instructions indentées proprement dans le code Python (pour rester lisible) sans que cette indentation ne se retrouve dans le texte réellement envoyé au modèle comme prompt.

7 from datetime import datetime

→ Importe la classe datetime, mais elle n'est en réalité utilisée nulle part dans ce fichier : c'est un import mort (résidu du script original ou d'une fonctionnalité retirée).

8 import utils

→ Importe le module local utils.py, qui centralise le chargement du document et l'extraction de son texte (PDF, Word, texte). Séparer cette logique dans son propre module permet à la fois aux agents (ce fichier) et à l'interface (app.py) de partager le même document "courant" chargé en mémoire.

10 load_dotenv()

→ Exécute le chargement effectif du fichier .env dans les variables d'environnement du processus. Cet appel doit avoir lieu avant tout appel à os.getenv, sinon la clé ne serait pas encore disponible.

12 # Le script original utilisait OpenAI (gpt-4o) ; remplacé par Groq pour

→ Commentaire de documentation expliquant un choix d'architecture : le script d'origine appelait l'API OpenAI (modèle gpt-4o), remplacée ici par Groq.

13 # réutiliser la clé GROQ_API_KEY déjà disponible.

→ Suite du commentaire : la raison du remplacement est pragmatique, réutiliser une clé d'API déjà configurée pour ce projet plutôt que de gérer un deuxième compte et une deuxième clé (OpenAI).

14 GROQ_API_KEY = os.getenv("GROQ_API_KEY")

→ Lit la clé d'API Groq depuis les variables d'environnement (chargées juste avant par load_dotenv) et la stocke dans une constante module. Utiliser une variable d'environnement plutôt qu'une clé écrite en dur dans le code est une pratique de sécurité de base : la clé ne se retrouve jamais dans l'historique Git ni visible à l'écran.

15 GROQ_MODEL_ID = "openai/gpt-oss-120b"

→ Définit l'identifiant du modèle par défaut utilisé par les trois agents spécialisés. "gpt-oss-120b" est un modèle de langage à poids ouverts (open-weight, publié par OpenAI mais librement réutilisable, distinct des modèles propriétaires GPT-4/GPT-4o) comptant 120 milliards de paramètres ; il est ici simplement hébergé et servi rapidement par l'infrastructure Groq plutôt que par OpenAI.

17 def get_document():

→ Déclare une fonction locale nommée get_document. Elle sera passée aux agents comme "outil" (tool) : un LLM ne peut pas lire un fichier tout seul, donc on lui fournit une fonction Python qu'il peut décider d'appeler pour obtenir le texte du contrat à analyser. C'est le mécanisme de "tool use" / "function calling" : le modèle reçoit la description de la fonction et choisit, pendant son raisonnement, de l'invoquer si besoin.

18 """

→ Ouvre la docstring décrivant le rôle de la fonction.

19 Récupère le document à analyser depuis le module utils

→ Texte de la docstring expliquant que la fonction délègue au module utils.

20 """

→ Ferme la docstring de la fonction.

21 return utils.get_document()

→ Appelle la fonction get_document du module utils (qui lit le fichier actuellement chargé en mémoire et en extrait le texte) et retourne son résultat tel quel à l'agent appelant.

23 def create_structure_agent():

→ Déclare la fonction "usine" (factory) qui construit et configure l'agent chargé de l'analyse structurelle du contrat. Utiliser une fonction plutôt qu'une instance globale permet de recréer un agent frais à chaque analyse.

24 """

→ Ouvre la docstring de la fonction create_structure_agent.

25 Crée l'agent spécialisé dans l'analyse structurelle des contrats

→ Texte de la docstring décrivant le rôle de cet agent.

26 """

→ Ferme la docstring de la fonction.

27 return Agent(

→ Commence la construction et le retour d'une instance de la classe Agent : c'est ici que le rôle, le modèle et les capacités de l'agent sont assemblés.

28 model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),

→ Configure le modèle Groq utilisé par cet agent, avec l'identifiant de modèle et la clé d'API définis plus haut. C'est ce paramètre qui détermine quel LLM effectuera réellement le raisonnement de l'agent.

29 name='Agent Structure',

→ Donne le nom affiché "Agent Structure" à cet agent (utile pour les logs et l'affichage dans l'équipe).

30 role='Expert en Structuration de Contrats',

→ Définit le rôle métier de l'agent. Dans une Team agno, ce rôle sert de repère au leader pour savoir à qui déléguer quoi.

31 instructions=dedent("""

→ Débute le paramètre instructions avec une chaîne multi-lignes dont l'indentation sera retirée par dedent. Ce paramètre agit comme le "system prompt" de l'agent : c'est le texte qui façonne durablement son comportement, contrairement à un message utilisateur ponctuel.

32 Vous êtes un Expert en Structuration de Contrats. Votre rôle est d'évaluer la structure

→ Première phrase des instructions : présente le rôle d'expert en structuration à l'agent.

33 d'un contrat et de suggérer des améliorations ou de construire une structure appropriée

→ Suite de la phrase précisant les deux missions possibles de l'agent.

34 à partir de zéro.

→ Fin de la phrase introductive des instructions.

36 Vous utiliserez l'outil `get\_document` pour récupérer le texte complet du contrat.

→ Indique explicitement à l'agent qu'il dispose de l'outil get_document et qu'il doit l'utiliser pour obtenir le texte du contrat, plutôt que d'inventer ou de supposer un contenu.

37 Votre tâche est d'analyser le contrat et de déterminer s'il est structuré de manière

→ Précise la tâche d'analyse de la clarté de la structure du contrat.

38 claire, complète et juridiquement appropriée.

→ Fin de la phrase décrivant les critères d'évaluation de la structure.

40 Vous devez identifier les sections manquantes ou peu claires :

→ Introduit la liste des consignes sur les sections manquantes ou peu claires.

41 - S'il manque de structure, suggérez une structure complète en utilisant des en-têtes

→ Première consigne : proposer une structure complète avec des en-têtes standards si absente.

42 de section standard (ex: Définitions, Conditions, Obligations, Résiliation, Loi applicable).

→ Donne des exemples concrets d'en-têtes de section standards à utiliser.

43 - Évitez l'interprétation juridique - concentrez-vous uniquement sur l'organisation,

→ Deuxième consigne : ne pas faire d'interprétation juridique, se concentrer sur l'organisation. Cette séparation des rôles (structure vs juridique vs négociation) évite les chevauchements entre les trois agents et rend chaque rapport plus ciblé.

44 la clarté et le flux logique.

→ Complète la consigne précédente en ajoutant clarté et flux logique comme critères.

45 - Soyez concis mais clair dans votre analyse.

→ Troisième consigne : demander une analyse concise et claire.

47 Produisez une structure au format markdown si vous créez une nouvelle structure,
→ Précise le format de sortie attendu (markdown) en cas de création d'une nouvelle structure ; markdown est choisi car il s'affiche proprement à la fois en console et dans l'interface Streamlit (voir app.py).

48 ou des commentaires sous forme de puces si vous évaluez une structure existante.
→ Précise le format de sortie attendu (puces) en cas d'évaluation d'une structure existante.

51 """),
→ Ferme la chaîne multi-lignes et l'appel à dedent pour le paramètre instructions.

52 tools=[get_document],
→ Fournit à l'agent la fonction get_document comme outil qu'il peut appeler pendant son raisonnement (function calling). Sans cette liste, l'agent n'aurait aucun moyen d'accéder au texte réel du contrat.

53 markdown=True,
→ Active le rendu markdown pour les réponses de cet agent, cohérent avec le format demandé dans les instructions.

54)
→ Ferme l'appel au constructeur Agent et donc la fonction create_structure_agent.

56 def create_legal_agent():
→ Déclare la fonction qui construit l'agent chargé de l'analyse juridique du contrat, sur le même principe que create_structure_agent.

57 """
→ Ouvre la docstring de la fonction create_legal_agent.

58 Crée l'agent spécialisé dans l'analyse juridique des contrats
→ Texte de la docstring décrivant le rôle de cet agent.

59 """
→ Ferme la docstring de la fonction.

60 return Agent(
→ Commence la construction et le retour d'une instance de la classe Agent.

61 model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),
→ Configure le modèle Groq utilisé par cet agent avec l'identifiant et la clé définis plus haut.

62 name='Agent Juridique',
→ Donne le nom affiché "Agent Juridique" à cet agent.

63 role='Analyste des Questions Juridiques',
→ Définit le rôle métier de l'agent, utilisé dans la coordination d'équipe.

64 instructions=dedent("""
→ Débute le paramètre instructions avec une chaîne multi-lignes dont l'indentation sera retirée.

65 Vous êtes un Analyste du Cadre Juridique chargé d'identifier les problèmes juridiques,
→ Présente le rôle d'analyste juridique et sa mission d'identifier les problèmes juridiques.

66 les risques et les principes juridiques clés dans le contrat téléchargé.
→ Complète la mission en ajoutant les risques et principes juridiques clés à identifier.

68 Utilisez l'outil `get\_document` pour accéder au texte complet du contrat.
→ Indique à l'agent d'utiliser l'outil get_document pour accéder au texte du contrat.

- 69 Pour chaque problème juridique ou observation, vous DEVEZ :
→ Introduit la liste des obligations pour chaque problème ou observation signalés.
- 71 - Citer la clause, phrase ou paragraphe exact du contrat sur lequel votre point est basé.
→ Première obligation : citer textuellement la clause exacte sur laquelle repose le point soulevé. Exiger une citation exacte est une technique de "grounding" (ancrage) : elle force le modèle à s'appuyer sur le texte réel du contrat plutôt que sur des connaissances générales, ce qui réduit le risque d'hallucination (affirmation plausible mais inventée).
- 72 - Commencer une nouvelle ligne avec 'Problème :' suivi d'une explication courte
→ Deuxième obligation : commencer une nouvelle ligne par le mot-clé 'Problème :', un marqueur de formatage qui facilitera la lecture et l'agrégation du rapport final par l'agent manager.
- 73 et claire de la préoccupation ou du principe juridique.
→ Complète l'obligation précédente en précisant le contenu attendu de l'explication.
- 74 - Faire clairement référence au titre de section, en-tête ou numéro de paragraphe
→ Troisième obligation : référencer le titre de section ou le numéro de paragraphe concerné.
- 75 si disponible. Sinon, décrire son emplacement (ex: "section commençant par [texte]...").
→ Donne l'alternative à utiliser si aucun titre ou numéro de section n'est disponible.
- 76 - NE PAS faire d'évaluation ou commentaire juridique à moins qu'il ne soit directement
→ Quatrième obligation : interdit tout commentaire juridique non directement justifié, renforçant encore la contrainte de grounding sur le texte source.
- 77 soutenu par une citation du contrat.
→ Complète l'interdiction précédente en exigeant qu'un commentaire soit soutenu par une citation.
- 79 Votre tâche :
→ Introduit la liste des tâches concrètes assignées à l'agent juridique.
- 80 - Identifier le domaine juridique du contrat (ex: droit commercial, emploi, NDA, etc.)
→ Première tâche : identifier le domaine juridique du contrat avec des exemples.
- 81 - Déterminer la juridiction probable ou la loi applicable
→ Deuxième tâche : déterminer la juridiction ou la loi applicable au contrat.
- 82 - Souligner tout problème juridique potentiel ou clause problématique
→ Troisième tâche : signaler tout problème juridique potentiel ou clause à risque.
- 83 - Vérifier la conformité RGPD si applicable
→ Quatrième tâche : vérifier la conformité au RGPD (règlement européen sur la protection des données) lorsque cela s'applique.
- 84 - Identifier les déséquilibres de responsabilité et de responsabilité
→ Cinquième tâche : repérer les déséquilibres liés à la responsabilité contractuelle.
- 86 Formatez chaque constatation comme suit :
→ Introduit le modèle de formatage exigé pour chaque constatation juridique ; imposer un gabarit fixe rend la sortie plus facile à parser et à synthétiser ensuite par l'agent manager.
- 87 Clause :
→ Premier élément du gabarit de formatage : l'étiquette précédant la clause citée.
- 88 "Texte cité du contrat ici."
→ Exemple de texte de clause entre guillemets dans le gabarit de formatage.
- 90 Section : (Titre de section ou emplacement)
→ Deuxième élément du gabarit : l'étiquette indiquant la section ou l'emplacement de la clause.

92 Problème :

→ Troisième élément du gabarit : l'étiquette précédant l'explication du problème juridique.

93 Votre brève analyse de pourquoi cette clause peut présenter une préoccupation juridique.

→ Exemple du contenu attendu après l'étiquette Problème dans le gabarit.

95 """),

→ Ferme la chaîne multi-lignes et l'appel à dedent pour le paramètre instructions.

96 tools=[get_document],

→ Fournit à l'agent la fonction get_document comme outil qu'il peut appeler.

97 debug_mode=False,

→ Désactive le mode de débogage pour cet agent : quand debug_mode est actif, agno affiche des logs détaillés (appels d'outils, échanges internes) utiles pendant le développement mais bruyants en usage normal.

98 markdown=True,

→ Active le rendu markdown pour les réponses de cet agent.

99)

→ Ferme l'appel au constructeur Agent et donc la fonction create_legal_agent.

101 def create_negotiation_agent():

→ Déclare la fonction qui construit l'agent chargé des stratégies de négociation du contrat.

102 """

→ Ouvre la docstring de la fonction create_negotiation_agent.

103 Crée l'agent spécialisé dans les stratégies de négociation contractuelle

→ Texte de la docstring décrivant le rôle de cet agent.

104 """

→ Ferme la docstring de la fonction.

105 return Agent(

→ Commence la construction et le retour d'une instance de la classe Agent.

106 model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),

→ Configure le modèle Groq utilisé par cet agent avec l'identifiant et la clé définis plus haut.

107 name='Agent Négociation',

→ Donne le nom affiché "Agent Négociation" à cet agent.

108 role='Stratège en Négociation de Contrats',

→ Définit le rôle métier de l'agent, utilisé dans la coordination d'équipe.

109 instructions=dedent("""

→ Débute le paramètre instructions avec une chaîne multi-lignes dont l'indentation sera retirée.

110 Vous êtes un Stratège en Négociation de Contrats.

→ Présente à l'agent son rôle de stratège en négociation de contrats.

112 Votre travail est d'identifier les parties d'un contrat qui sont couramment négociables

→ Décrit la mission d'identifier les parties du contrat couramment négociables.

113 ou potentiellement déséquilibrées. Vous DEVEZ :

→ Complète la mission en ajoutant la détection des déséquilibres, puis introduit des obligations.

115 - Toujours citer le paragraphe ou la clause exacte à laquelle vous faites référence.
→ Première obligation : toujours citer le paragraphe ou la clause exacte référencée, même technique de grounding par citation que pour l'agent juridique.

116 - Expliquer clairement pourquoi elle peut être négociable ou nécessite un ajustement.
→ Deuxième obligation : expliquer pourquoi la clause est négociable ou doit être ajustée.

117 - Suggérer une contre-offre ou une formulation alternative.
→ Troisième obligation : proposer une contre-offre ou une formulation alternative.

119 Structurez votre analyse comme ceci :
→ Introduit le plan structurel attendu pour chaque point de l'analyse.

120 1. **Clause Citée** (texte exact du contrat)
→ Premier point du plan : la clause citée textuellement.

121 2. **Pourquoi elle est négociable ou problématique**
→ Deuxième point du plan : la justification de la négociabilité ou du problème.

122 3. **Exemple de stratégie ou contre-suggestion**
→ Troisième point du plan : un exemple de stratégie ou de contre-suggestion.

124 NE FAITES PAS de commentaires généraux. Chaque point que vous faites doit être soutenu
→ Interdit les commentaires généraux et exige que chaque point soit soutenu par une preuve.

125 par une citation directe du contrat, et votre sortie doit clairement montrer quelle
→ Complète l'exigence de citation directe et de clarté sur la partie du contrat analysée.

126 partie du contrat vous analysez.
→ Fin de la phrase précisant que la sortie doit montrer la partie du contrat analysée.

128 """),
→ Ferme la chaîne multi-lignes et l'appel à dedent pour le paramètre instructions.

129 tools=[get_document],
→ Fournit à l'agent la fonction get_document comme outil qu'il peut appeler.

130 debug_mode=False,
→ Désactive le mode de débogage pour cet agent.

131 markdown=True,
→ Active le rendu markdown pour les réponses de cet agent.

132)
→ Ferme l'appel au constructeur Agent et donc la fonction create_negotiation_agent.

134 def create_manager_team():
→ Déclare la fonction qui construit l'équipe (Team) coordonnant les trois agents spécialisés créés plus haut.

135 """
→ Ouvre la docstring de la fonction create_manager_team.

136 Crée l'équipe de management qui coordonne tous les agents spécialisés
→ Texte de la docstring décrivant le rôle général de la fonction.

138 Returns:
→ Section Returns de la docstring, introduisant la description de la valeur renvoyée.

139 Team: Équipe coordonnée d'agents pour l'analyse complète de contrats
→ Décrit le type et le contenu de la valeur renvoyée par la fonction.

140 """
→ Ferme la docstring de la fonction.

142 # Création des agents spécialisés
→ Commentaire introduisant la section de création des trois agents spécialisés.

143 structure_agent = create_structure_agent()
→ Instancie l'agent d'analyse structurelle en appelant sa fonction de création.

144 legal_agent = create_legal_agent()
→ Instancie l'agent d'analyse juridique en appelant sa fonction de création.

145 negotiate_agent = create_negotiation_agent()
→ Instancie l'agent de négociation en appelant sa fonction de création.

147 # Équipe coordonnée par un agent manager
→ Commentaire introduisant la construction de l'équipe coordonnée par un agent manager.

148 manager_agent = Team(
→ Commence la construction d'une instance de la classe Team, stockée dans manager_agent. C'est ce "leader" qui va déléguer aux trois agents membres puis fusionner leurs réponses en un rapport unique.

149 members=[structure_agent, legal_agent, negotiate_agent],
→ Définit les trois agents créés précédemment comme membres de l'équipe : c'est la liste des sous-agents que le leader pourra solliciter.

150 # Le leader doit synthétiser en un seul appel les 3 rapports complets
→ Commentaire expliquant pourquoi un modèle différent est utilisé pour le leader de l'équipe.

151 # (Structure + Légal + Négociation), ce qui dépasse la limite Groq du
→ Suite du commentaire sur la limite de tokens Groq dépassée par la synthèse des 3 rapports.

152 # modèle 120B (8000 tokens/minute) ; le modèle 20B a un quota plus élevé.
→ Fin du commentaire précisant le quota du modèle 120B et la raison du choix du modèle 20B. C'est une contrainte de "rate limiting" typique des API de LLM : chaque compte/modèle a un débit maximal de tokens autorisé par minute, et le dépasser fait échouer l'appel ; ici, comme le leader doit lire trois rapports complets d'un coup (gros volume de tokens en entrée), un modèle avec un quota plus généreux est nécessaire même s'il est plus petit.

153 model=Groq(id="openai/gpt-oss-20b", api_key=GROQ_API_KEY),
→ Configure le modèle Groq 20B (variante plus petite du même modèle, avec un quota plus élevé) comme moteur du leader d'équipe, pour éviter l'erreur de limite de débit décrite dans le commentaire ci-dessus.

154 mode='coordinate',
→ Définit le mode de fonctionnement de l'équipe sur 'coordinate' : dans ce mode agno, le leader distribue une tâche à ses membres puis attend et fusionne leurs réponses en une synthèse finale, plutôt que de simplement router la requête vers un seul agent (mode 'route') ou de laisser tous les agents discuter librement.

155 # Force la délégation systématique aux 3 agents (le rapport a besoin des
→ Commentaire expliquant pourquoi la délégation systématique aux 3 agents est forcée.

156 # 3 apports) plutôt que de laisser le leader décider s'il délègue ou non.
→ Suite du commentaire précisant que le leader ne décide pas lui-même de déléguer.

157 delegate_to_all_members=True,
→ Force l'équipe à toujours déléguer la tâche à tous les membres. Sans ce paramètre, le leader (un LLM) pourrait décider par lui-même de ne consulter qu'un ou deux agents, ce qui casserait l'exigence du rapport final de contenir les trois volets (structure,

juridique, négociation) ; ce paramètre rend le comportement déterministe plutôt que laissé au jugement du modèle.

158 # success_criteria n'existe plus comme paramètre séparé de Team : son contenu

→ Commentaire expliquant que success_criteria a été fusionné dans le paramètre instructions.

159 # est fusionné dans instructions ci-dessous.

→ Suite du commentaire précisant où se trouve désormais ce contenu (trace d'une évolution de version de la bibliothèque agno).

160 instructions=dedent("""

→ Débute le paramètre instructions du manager avec une chaîne multi-lignes dédentée ; ce texte joue ici le rôle de prompt système pour l'étape finale de synthèse.

161 Critère de réussite : un résumé bien organisé et traçable du contrat qui inclut

→ Décrit le critère de réussite attendu : un résumé organisé et traçable du contrat.

162 un contexte juridique soulignant les problèmes juridiques potentiels avec le texte

→ Précise que le résumé doit inclure un contexte juridique avec les problèmes potentiels.

163 du contrat cité comme preuve, une revue structurelle avec des suggestions de clarté

→ Ajoute l'exigence de citer le contrat comme preuve et d'inclure une revue structurelle.

164 et de formatage, des stratégies de négociation directement liées à des paragraphes

→ Ajoute l'exigence d'inclure des stratégies de négociation liées à des paragraphes précis.

165 ou clauses spécifiques, une évaluation des risques consolidée avec des recommandations

→ Ajoute l'exigence d'une évaluation des risques consolidée avec des recommandations.

166 actionnables, et un classement de priorité clair de tous les problèmes identifiés.

→ Termine la description du critère de réussite avec un classement de priorité des problèmes.

168 Vous êtes le Coordinateur principal et résumeur. Vous devez combiner les apports de :

→ Attribue au leader le rôle de coordinateur et résumeur combinant les apports des agents.

169 1. Agent Structure

→ Premier élément de la liste des agents dont les apports doivent être combinés.

170 2. Agent Juridique

→ Deuxième élément de la liste des agents dont les apports doivent être combinés.

171 3. Agent Négociation

→ Troisième élément de la liste des agents dont les apports doivent être combinés.

173 IMPORTANT : Ne demandez JAMAIS à l'utilisateur de fournir le texte du contrat.

→ Interdit explicitement de demander à l'utilisateur de fournir le texte du contrat : comme l'application est automatisée (voir app.py), il n'y a personne pour répondre à une telle question, donc le workflow bloquerait sans cette consigne.

174 Déléguez immédiatement la tâche aux trois agents ci-dessus : chacun a accès à

→ Demande de déléguer immédiatement la tâche aux trois agents spécialisés.

175 l'outil get_document() et récupérera lui-même le contrat déjà chargé en mémoire.

→ Explique que chaque agent récupère lui-même le contrat via l'outil get_document(), ce qui rappelle que le mécanisme de récupération de document (outil) est partagé par tous les agents et repose sur l'état global du module utils.

177 Exigences clés :

→ Introduit la liste des exigences clés pour la synthèse finale du manager.

178 - Pour tous les points juridiques et de négociation, préservez les clauses citées

→ Première exigence : préserver les clauses citées pour les points juridiques et de négociation, afin que la synthèse finale reste elle aussi traçable jusqu'au texte du contrat.

179 du contrat comme preuve.
→ Complète l'exigence précédente en précisant que les clauses servent de preuve.

180 - L'Agent Juridique devrait souligner des problèmes juridiques spécifiques,
→ Deuxième exigence : l'agent juridique doit souligner des problèmes juridiques spécifiques.

181 suivis d'une courte explication 'Problème :'.
→ Complète l'exigence précédente en imposant le format 'Problème :.' pour l'explication.

182 - Chaque extrait cité doit être suivi de l'explication ou recommandation de l'agent.
→ Troisième exigence : chaque citation doit être suivie de l'explication ou recommandation associée.

183 - Consolidez tous les scores en une évaluation globale des risques
→ Quatrième exigence : consolider tous les scores en une évaluation globale des risques.

184 - Fournissez un résumé exécutif clair avec des étapes actionnables suivantes
→ Cinquième exigence : fournir un résumé exécutif clair avec des étapes actionnables.

185 - Assurez-vous de la traçabilité complète de toutes les contributions des agents
→ Sixième exigence : garantir la traçabilité complète des contributions de chaque agent.

187 Format final :
→ Introduit la structure imposée pour le rapport final.

188 1. Résumé Exécutif
→ Première section imposée du format final : le résumé exécutif.

189 2. Contexte Juridique (Avec clauses citées et explications des problèmes)
→ Deuxième section imposée du format final : le contexte juridique avec clauses et explications.

190 3. Retours sur la Structure du Contrat
→ Troisième section imposée du format final : les retours sur la structure du contrat.

191 4. Recommandations de Négociation
→ Quatrième section imposée du format final : les recommandations de négociation.

192 """),
→ Ferme la chaîne multi-lignes et l'appel à dedent pour le paramètre instructions du manager.

193 debug_mode=True,
→ Active le mode de débogage pour l'équipe manager : contrairement aux trois agents spécialisés, ici debug_mode reste actif, ce qui permet de voir dans les logs comment le leader délègue et synthétise les rapports — utile pour diagnostiquer la coordination, l'étape la plus complexe du pipeline.

194 markdown=True,
→ Active le rendu markdown pour les réponses de l'équipe.

195)
→ Ferme l'appel au constructeur Team.

197 return manager_agent
→ Retourne l'objet Team construit, prêt à être utilisé (via .run(...)) pour analyser un contrat de bout en bout.

Ce fichier gère le chargement d'un fichier de contrat et l'extraction de son texte selon son format (PDF, Word, texte), et sert de pont entre l'interface (app.py) et les agents (contract_analysis.py) via un état global partagé.

51 lignes de code annotées ci-dessous.

```
1 from pathlib import Path
    → Importe la classe Path du module standard pathlib, une API orientée objet pour manipuler des chemins de fichiers de façon portable entre systèmes d'exploitation (au lieu de manipuler des chaînes de caractères avec os.path).

2 import fitz # PyMuPDF
    → Importe la bibliothèque PyMuPDF (dont le nom du module Python est "fitz"), une bibliothèque de traitement de documents PDF rapide et légère. Elle permet ici d'ouvrir un PDF et d'en extraire le texte page par page.

4 # Variable globale
    → Commentaire annonçant la déclaration d'une variable globale du module.

5 current_file_path = None
    → Initialise la variable globale qui stockera le chemin du fichier actuellement chargé. Utiliser une variable au niveau du module est une façon simple de partager un état ("quel est le document actif ?") entre app.py qui le définit et contract_analysis.py qui le lit, sans avoir à le faire transiter explicitement à travers tous les appels d'agents.

7 def set_document_file(file_path):
    → Déclare la fonction qui définit quel fichier doit être analysé.

8     """Définit le fichier à analyser"""
    → Docstring décrivant en une ligne le rôle de la fonction.

9     global current_file_path
    → Indique que la fonction va modifier la variable globale current_file_path plutôt que de créer une variable locale du même nom (nécessaire en Python dès qu'on réassigne une variable globale à l'intérieur d'une fonction).

10    current_file_path = file_path
    → Stocke le chemin du fichier reçu en paramètre dans la variable globale.

12 def get_document():
    → Déclare la fonction qui retourne le contenu du document actuellement chargé ; c'est cette fonction que get_document() de contract_analysis.py appelle, et donc celle que les agents invoquent indirectement comme "outil".

13     """Retourne le contenu du fichier"""
    → Docstring décrivant en une ligne le rôle de la fonction.

14    global current_file_path
    → Indique que la fonction va lire la variable globale current_file_path (le mot-clé global n'est pas strictement nécessaire ici puisqu'il n'y a pas de réassignation, mais il documente l'intention).

16    if not current_file_path:
    → Vérifie si aucun fichier n'a été défini au préalable (cas où get_document serait appelé avant tout upload).

17        return {"error": "Aucun fichier", "content": ""}
    → Retourne un dictionnaire d'erreur structuré si aucun fichier n'a été défini, plutôt que de lever une exception : cela permet à l'agent qui appelle cet outil de recevoir une réponse exploitable (texte) au lieu de faire planter tout le programme.

19    file_path = Path(current_file_path)
    → Convertit le chemin stocké (une simple chaîne) en objet Path pour faciliter les manipulations suivantes, notamment la lecture de l'extension.

20    extension = file_path.suffix.lower()
    → Extrait l'extension du fichier (ex: ".pdf") et la met en minuscules, pour que la détection du format ne dépende pas de la casse (".PDF" vs ".pdf").
```

```

22     try:
    → Ouvre un bloc try pour capturer les erreurs pouvant survenir pendant l'extraction (fichier corrompu, permissions, etc.).

23         if extension == '.pdf':
    → Vérifie si l'extension correspond à un fichier PDF.

24             content = extract_pdf(file_path)
    → Appelle la fonction d'extraction PDF et stocke le texte obtenu.

25         elif extension in ['.docx', '.doc']:
    → Vérifie si l'extension correspond à un fichier Word (.docx ou .doc).

26             content = extract_word_simple(file_path)
    → Appelle la fonction d'extraction Word et stocke le texte obtenu.

27         elif extension == '.txt':
    → Vérifie si l'extension correspond à un fichier texte brut.

28             content = extract_text(file_path)
    → Appelle la fonction d'extraction de texte brut et stocke le texte obtenu.

29     else:
    → Cas où l'extension ne correspond à aucun format géré.

30         return {"error": "Format non supporté", "content": ""}
    → Retourne un dictionnaire d'erreur signalant un format de fichier non supporté.

31
32         return {"content": content, "error": None}
    → Retourne le contenu extrait avec succès et une erreur nulle ; ce format de retour homogène ({"content", "error"}) permet à
    l'appelant de vérifier facilement si l'opération a réussi.

33
34     except Exception as e:
    → Capture toute exception survenue pendant l'extraction du contenu (garde-fou générique).

35         return {"error": str(e), "content": ""}
    → Retourne un dictionnaire d'erreur contenant le message de l'exception capturée, gardant le même format que les autres
    retours de la fonction.

36
37 def extract_pdf(file_path):
    → Déclare la fonction qui extrait le texte d'un fichier PDF.

38     """Extrait le texte d'un PDF"""
    → Docstring décrivant en une ligne le rôle de la fonction.

39     doc = fitz.open(file_path)
    → Ouvre le fichier PDF avec PyMuPDF et retourne un objet document qui donne accès à ses pages.

40     text = ""
    → Initialise une chaîne vide qui accumulera le texte extrait.

41     for page in doc:
    → Parcourt chaque page du document PDF (PyMuPDF rend l'objet document itérable page par page).

42         text += page.get_text()
    → Ajoute le texte extrait de la page courante à la chaîne accumulée ; get_text() récupère le texte "brut" de la page sans sa
    mise en forme visuelle.

43     doc.close()

```


→ Ferme le document PDF pour libérer les ressources associées (fichier, mémoire).

44 return text

→ Retourne le texte complet extrait de toutes les pages du PDF.

46 def extract_word_simple(file_path):

→ Déclare la fonction qui extrait le texte d'un fichier Word via PyMuPDF.

47 """Extrait le texte d'un fichier Word avec PyMuPDF"""

→ Docstring décrivant en une ligne le rôle de la fonction.

48 try:

→ Ouvre un bloc try pour capturer les erreurs de lecture du fichier Word.

49 # PyMuPDF peut aussi lire les fichiers Word !

→ Commentaire signalant que PyMuPDF, bien que pensé pour le PDF, sait aussi ouvrir certains fichiers Word — ce qui évite d'ajouter une dépendance supplémentaire (comme python-docx) rien que pour ce cas.

50 doc = fitz.open(file_path)

→ Ouvre le fichier Word avec PyMuPDF et retourne un objet document.

51 text = ""

→ Initialise une chaîne vide qui accumulera le texte extrait.

52 for page in doc:

→ Parcourt chaque page (ou section) du document Word.

53 text += page.get_text()

→ Ajoute le texte extrait de la page courante à la chaîne accumulée.

54 doc.close()

→ Ferme le document Word pour libérer les ressources associées.

55 return text

→ Retourne le texte complet extrait du fichier Word en cas de succès.

56 except:

→ Capture toute exception survenue pendant l'ouverture ou la lecture du fichier Word (certains .doc/.docx ne sont pas correctement supportés par PyMuPDF).

57 return "Erreur: Impossible de lire ce fichier Word. Convertissez-le en PDF."

→ Retourne un message d'erreur texte invitant l'utilisateur à convertir le fichier en PDF, un format que PyMuPDF gère de façon fiable.

59 def extract_text(file_path):

→ Déclare la fonction qui extrait le contenu d'un fichier texte brut.

60 """Extrait le texte d'un fichier texte"""

→ Docstring décrivant en une ligne le rôle de la fonction.

61 with open(file_path, 'r', encoding='utf-8') as file:

→ Ouvre le fichier en lecture avec l'encodage UTF-8 explicite (pour gérer correctement les accents et caractères spéciaux), dans un bloc contextuel (with) qui garantit la fermeture automatique du fichier même en cas d'erreur.

62 return file.read()

→ Lit et retourne l'intégralité du contenu du fichier texte.

20 - Agno - Team/app.py

Ce fichier définit l'interface web Streamlit permettant d'uploader un contrat, de lancer son analyse par l'équipe d'agents et d'afficher le rapport résultant.

45 lignes de code annotées ci-dessous.

1 `import streamlit as st`

→ Importe la bibliothèque Streamlit, sous l'alias conventionnel `st`. Streamlit permet de construire une interface web interactive en pur Python (sans HTML/CSS/JS à écrire) : chaque appel comme `st.title`, `st.button`, etc. ajoute un élément visuel à la page. Son principe de fonctionnement est de ré-exécuter tout le script du haut vers le bas à chaque interaction utilisateur (clic, upload...), ce qui explique la structure du fichier.

2 `import tempfile`

→ Importe le module standard `tempfile`, qui permet de créer des fichiers temporaires sur le disque. Il est nécessaire ici car les bibliothèques d'extraction (PyMuPDF dans `utils.py`) attendent un chemin de fichier réel, alors que Streamlit ne fournit le fichier téléversé qu'en mémoire (bytes) : on doit donc l'écrire temporairement sur disque pour pouvoir le traiter.

3 `from pathlib import Path`

→ Importe la classe `Path` pour manipuler des chemins de fichiers de façon portable, notamment pour récupérer l'extension du fichier téléversé.

4 `from utils import get_document, set_document_file`

→ Importe depuis le module `utils` les fonctions de récupération et de définition du document ; `set_document_file` sera utilisée pour enregistrer le fichier temporaire comme document actif avant analyse.

5 `from contract_analysis import create_manager_team`

→ Importe la fonction qui construit l'équipe d'agents d'analyse de contrat, définie dans `contract_analysis.py`.

7 `def main():`

→ Déclare la fonction principale qui définit l'interface et la logique de l'application ; Streamlit exécute ce fichier comme un script, donc regrouper la logique dans `main()` est une convention de structuration plutôt qu'une contrainte du framework.

8 `st.title(" Analyseur de Contrats")`

→ Affiche le titre de la page avec une icône de document, en haut de l'interface.

10 `# Upload du fichier`

→ Commentaire introduisant la section de téléversement du fichier.

11 `uploaded_file = st.file_uploader(`

→ Crée un composant Streamlit de téléversement de fichier : un widget web (glisser-déposer ou sélection) qui renvoie l'objet fichier téléversé (ou `None` tant que rien n'a été choisi).

12 `"Choisir un contrat",`

→ Définit le libellé affiché au-dessus du composant de téléversement.

13 `type=['pdf', 'docx', 'txt'])`

→ Restreint les types de fichiers acceptés aux formats pdf, docx et txt, en cohérence avec les formats gérés par `utils.py`.

14 `)`

→ Ferme l'appel à `st.file_uploader`.

16 `if uploaded_file:`

→ Vérifie qu'un fichier a bien été téléversé avant de poursuivre (`uploaded_file` vaut `None` sinon).

17 `# Sauvegarder temporairement`

→ Commentaire introduisant la sauvegarde temporaire du fichier téléversé.

18 `with tempfile.NamedTemporaryFile(delete=False, suffix=Path(uploaded_file.name).suffix) as`

`tmp_file:`

→ Ouvre un fichier temporaire réel sur le disque. `delete=False` évite qu'il soit supprimé automatiquement à la fermeture du bloc `with` (il doit rester disponible pour que `utils.py` puisse le lire plus tard) ; suffix reprend l'extension d'origine du fichier téléversé pour que `utils.py` puisse détecter correctement son format.

19 `tmp_file.write(uploaded_file.getvalue())`

→ Écrit le contenu binaire du fichier téléversé (récupéré en mémoire via `getvalue()`) dans le fichier temporaire sur disque.

20 `temp_file_path = tmp_file.name`

→ Récupère le chemin du fichier temporaire créé et le stocke dans une variable, pour pouvoir le référencer après la fermeture du bloc `with`.

22 `# Bouton d'analyse`

→ Commentaire introduisant le bouton déclenchant l'analyse.

23 `if st.button(" Analyser le contrat", type="primary"):`

→ Affiche un bouton Streamlit stylé en couleur primaire et exécute le bloc suivant uniquement lors du re-run déclenché par le clic de l'utilisateur.

25 `with st.spinner("Analyse en cours..."):`

→ Affiche un indicateur de chargement animé pendant l'exécution du bloc suivant, pour donner un retour visuel à l'utilisateur pendant que les agents (potentiellement longs, car ils font plusieurs appels LLM) travaillent.

26 `try:`

→ Ouvre un bloc `try` pour capturer les erreurs pouvant survenir pendant l'analyse (par exemple un problème d'API ou d'extraction).

27 `# Charger le document`

→ Commentaire introduisant le chargement du document dans le module `utils`.

28 `set_document_file(temp_file_path)`

→ Enregistre le chemin du fichier temporaire comme document courant à analyser, dans l'état global partagé de `utils.py` ; c'est ce qui permettra ensuite à l'outil `get_document()` des agents de retrouver ce fichier.

30 `# Créer l'équipe d'agents`

→ Commentaire introduisant la création de l'équipe d'agents.

31 `manager_team = create_manager_team()`

→ Construit l'équipe d'agents coordonnée en appelant `create_manager_team` ; une nouvelle équipe est recrée à chaque analyse plutôt que d'en garder une seule en mémoire.

33 `# Lancer l'analyse`

→ Commentaire introduisant le lancement de l'analyse par l'équipe.

34 `result = manager_team.run("Analysez ce contrat complètement.")`

→ Exécute l'équipe d'agents avec une instruction textuelle simple ; c'est cet appel `.run()` qui déclenche en interne la délégation aux trois agents spécialisés puis la synthèse par le leader, et stocke l'objet résultat retourné.

36 `# Afficher les résultats`

→ Commentaire introduisant l'affichage des résultats de l'analyse.

37 `st.success(" Analyse terminée!")`

→ Affiche un message de succès (encadré vert) indiquant que l'analyse est terminée.

39 `if hasattr(result, 'content'):`

→ Vérifie si l'objet résultat possède un attribut `content` ; cette vérification défensive gère le cas où la forme exacte de l'objet retourné par `agno` varierait selon la version de la bibliothèque.

40 `analysis_text = result.content`

→ Si l'attribut `content` existe, l'utilise comme texte d'analyse (c'est le cas normal attendu, contenant la synthèse markdown du

leader).

41 else:

→ Cas où l'objet résultat ne possède pas d'attribut content.

42 analysis_text = str(result)

→ Convertit le résultat en chaîne de caractères comme texte d'analyse de secours (repli défensif).

44 st.markdown(analysis_text)

→ Affiche le texte d'analyse dans la page en le rendant comme du markdown (titres, listes, gras...), cohérent avec markdown=True configuré côté agents.

46 # Bouton de téléchargement

→ Commentaire introduisant le bouton de téléchargement du rapport.

47 st.download_button(

→ Crée un bouton Streamlit permettant à l'utilisateur de télécharger un fichier généré côté serveur directement depuis le navigateur.

48 " Télécharger le rapport",

→ Définit le libellé affiché sur le bouton de téléchargement.

49 analysis_text,

→ Fournit le texte d'analyse comme contenu du fichier à télécharger.

50 file_name=f"analyse_{uploaded_file.name}.md",

→ Définit le nom du fichier téléchargé en incluant le nom du fichier original, avec l'extension .md (markdown).

51 mime="text/markdown"

→ Définit le type MIME du fichier téléchargé comme du markdown, pour que le navigateur/système le traite correctement.

52)

→ Ferme l'appel à st.download_button.

54 except Exception as e:

→ Capture toute exception survenue pendant le bloc d'analyse (appel LLM, extraction, etc.).

55 st.error(f" Erreur: {str(e)}")

→ Affiche un message d'erreur (encadré rouge) contenant le détail de l'exception capturée, plutôt que de laisser l'application planter silencieusement.

57 if __name__ == "__main__":

→ Vérifie que le script est exécuté directement et non importé comme module (convention Python standard) ; Streamlit exécute effectivement le fichier comme script principal via la commande `streamlit run`.

58 main()

→ Appelle la fonction principale pour démarrer l'application Streamlit.

Projet 21 — 21-SDV-synthetise

Generation de donnees synthetiques (SDV + GPT-2)

Objectif

Genere un jeu de donnees synthetique qui reproduit les proprietes statistiques d'un jeu de donnees reel (memes distributions, memes correlations entre colonnes) sans jamais exposer les enregistrements originaux, en combinant une librairie de synthese tabulaire (SDV) et un modele de langage pour generer du texte synthetique coherent dans certains champs.

Code source, ligne par ligne

21-SDV-synthetise/data_synthesizer.py

Ce script en ligne de commande génère des données synthétiques (tabulaires et narratives) à partir d'un dataset réel avec la bibliothèque SDV, puis propose des rapports et visualisations pour en évaluer la qualité.

178 lignes de code annotées ci-dessous.

1 import sys

→ Importe le module standard sys, qui donne accès notamment aux flux de sortie standard (stdout) et d'erreur (stderr) du processus.

3 # La console Windows utilise par défaut l'encodage cp1252, qui ne supporte pas

→ Commentaire expliquant le problème d'encodage de la console Windows.

4 # les emojis utilisés dans les messages de ce script (ex:) – ça faisait

→ Suite du commentaire précisant que les emojis provoquaient un plantage.

5 # planter le script juste après avoir sauvegardé les résultats.

→ Fin du commentaire expliquant le moment où le plantage se produisait.

6 for _stream in (sys.stdout, sys.stderr):

→ Parcourt les deux flux standard de sortie et d'erreur.

7 if hasattr(_stream, "reconfigure"):

→ Vérifie si le flux courant possède une méthode reconfigure (disponible sur les flux texte de Python 3.7+, mais pas garantie dans tous les contextes d'exécution, d'où cette vérification défensive).

8 _stream.reconfigure(encoding="utf-8", errors="replace")

→ Reconfigure le flux pour utiliser l'encodage UTF-8 (qui supporte les emojis et caractères spéciaux) et remplacer les caractères non représentables plutôt que de lever une exception. Ce correctif contourne une limitation historique de la console Windows, dont l'encodage par défaut (cp1252) ne sait pas afficher certains emojis utilisés plus loin dans les print().

10 import pandas as pd

→ Importe la bibliothèque pandas, sous l'alias conventionnel pd. Pandas est la bibliothèque de référence en Python pour manipuler des données tabulaires (façon feuille de calcul) via sa structure principale, le DataFrame ; elle est utilisée ici pour charger le CSV source et manipuler les données réelles et synthétiques.

11 import warnings

→ Importe le module standard warnings pour contrôler l'affichage des avertissements (non bloquants) émis par les bibliothèques utilisées.

12 import os

→ Importe le module standard `os` pour interagir avec le système de fichiers et les variables d'environnement.

14 # transformers essaie de charger TensorFlow en plus de PyTorch, et la version de

→ Commentaire expliquant le conflit entre transformers, TensorFlow et protobuf.

15 # TensorFlow installée sur cette machine est incompatible avec la version de

→ Suite du commentaire décrivant l'incompatibilité de versions.

16 # protobuf présente. On n'a besoin que du backend PyTorch pour le pipeline GPT-2.

→ Fin du commentaire précisant que seul le backend PyTorch est nécessaire. transformers (Hugging Face) peut s'appuyer indifféremment sur PyTorch ou TensorFlow comme moteur de calcul ; ici on force PyTorch pour éviter un plantage au chargement dû à une incompatibilité de versions avec TensorFlow.

17 os.environ.setdefault("USE_TF", "0")

→ Définit la variable d'environnement `USE_TF` à "0" si elle n'est pas déjà définie, ce qui indique à la bibliothèque transformers de ne pas tenter de charger TensorFlow, uniquement PyTorch.

19 from sdv.metadata import Metadata

→ Importe la classe Metadata de la bibliothèque SDV (Synthetic Data Vault). SDV est une bibliothèque spécialisée dans la génération de données synthétiques tabulaires : Metadata décrit la structure d'un jeu de données (types de colonnes, contraintes, clés) — étape préalable nécessaire pour que le synthétiseur sache comment générer des données statistiquement cohérentes.

20 from sdv.single_table import GaussianCopulaSynthesizer

→ Importe le synthétiseur de données basé sur une copule gaussienne, pour tables uniques. Une "copule gaussienne" est une technique statistique qui modélise à la fois la distribution de chaque colonne et les corrélations entre colonnes, afin de générer de nouvelles lignes qui reproduisent les mêmes tendances statistiques que les données réelles sans être des copies des lignes d'origine (donc sans exposer les vraies données individuelles).

21 from sdv.evaluation.single_table import run_diagnostic, evaluate_quality, get_column_plot

→ Importe les fonctions d'évaluation de SDV : `run_diagnostic` vérifie que les données générées sont structurellement valides, `evaluate_quality` mesure à quel point elles ressemblent statistiquement aux données réelles, et `get_column_plot` produit un graphique comparatif colonne par colonne.

22 from transformers import pipeline

→ Importe la fonction pipeline de la bibliothèque transformers (Hugging Face). Un "pipeline" transformers est une interface haut niveau qui encapsule un modèle pré-entraîné pour une tâche donnée (ici la génération de texte) en une simple fonction appelable, sans avoir à gérer manuellement le tokenizer, le modèle et le post-traitement.

24 # Ignorer les messages d'avertissement pour garder une sortie propre

→ Commentaire expliquant le but de la ligne suivante.

25 warnings.filterwarnings("ignore")

→ Configure le module warnings pour ignorer tous les messages d'avertissement (souvent émis par SDV ou transformers) afin de garder une sortie console lisible.

27 # Dossier du script : les chemins sont résolus par rapport à lui, pas au

→ Commentaire expliquant que les chemins sont relatifs à l'emplacement du script.

28 # répertoire courant, pour que le script fonctionne peu importe d'où il est lancé.

→ Suite du commentaire précisant l'avantage de cette approche.

29 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

→ Calcule le chemin absolu du dossier contenant ce script, en partant du chemin du fichier courant (`__file__`).

30 DATA_DIR = os.path.join(SCRIPT_DIR, "Data")

→ Construit le chemin du sous-dossier "Data" situé à côté du script, de façon portable entre systèmes d'exploitation grâce à `os.path.join`.

```

32 # Créer un dossier nommé "Data" s'il n'existe pas déjà
    → Commentaire expliquant le but de la ligne suivante.

33 os.makedirs(DATA_DIR, exist_ok=True)
    → Crée le dossier Data s'il n'existe pas encore ; exist_ok=True évite qu'une erreur soit levée si le dossier existe déjà.

35 # Charger le dataset original (premières 500 lignes pour la démo)
    → Commentaire expliquant que seules 500 lignes du dataset sont chargées pour la démonstration.

36 real_data = pd.read_csv(os.path.join(DATA_DIR, "E-Commerce Dataset.csv"), nrows=500)
    → Charge les 500 premières lignes du fichier CSV du dataset e-commerce dans un DataFrame pandas ; limiter à 500 lignes
accélère l'entraînement du synthétiseur pour la démonstration, au prix d'un modèle statistique moins riche que sur le dataset
complet.

38 # Initialiser les variables pour stocker les données et résultats
    → Commentaire annonçant l'initialisation des variables globales du module.

39 metadata = None
    → Initialise la variable globale metadata à None avant sa détection ultérieure.

40 synthetic_data = None
    → Initialise la variable globale synthetic_data à None avant sa génération ultérieure.

41 quality_report_result = None
    → Initialise la variable globale quality_report_result à None avant son calcul ultérieur.

43 def synthesize_data(realism=False):
    → Déclare la fonction de génération de données synthétiques, avec un paramètre realism par défaut à False (paramètre
présent dans la signature mais non exploité plus loin dans le corps de la fonction, comme l'indique le code ci-dessous).

44     """
    → Ouvre la docstring de la fonction synthesize_data.

45     Générer des données synthétiques basées sur le dataset original
    → Première ligne de la docstring décrivant le rôle de la fonction.

47     Args:
    → Section Args de la docstring, introduisant la description des paramètres.

48     realism (bool): Si True, active le mode réalisme pour plus de variété
    → Décrit le paramètre realism et son effet lorsqu'il vaut True.

49     """
    → Ferme la docstring de la fonction.

50     global metadata, synthetic_data
    → Indique que la fonction va modifier les variables globales metadata et synthetic_data.

52     # Détecter automatiquement la structure et les types depuis le dataset original
    → Commentaire expliquant le but de la ligne suivante.

53     metadata = Metadata.detect_from_dataframe(real_data)
    → Détecte automatiquement la structure et les types de colonnes du DataFrame réel (numérique, catégoriel, identifiant...) sans
que le développeur ait à les décrire manuellement colonne par colonne ; c'est cette métadonnée qui guide ensuite le synthétiseur
sur la façon de modéliser chaque colonne.

55     # Initialiser le synthétiseur avec les options sélectionnées
    → Commentaire expliquant le but de la ligne suivante.

56     synthesizer = GaussianCopulaSynthesizer(metadata)

```

→ Instancie le synthétiseur à copule gaussienne avec les métadonnées détectées, prêt à être entraîné.

```
58 # Entraîner le synthétiseur sur le dataset réel
```

→ Commentaire expliquant le but de la ligne suivante.

```
59 synthesizer.fit(real_data)
```

→ Entraîne le synthétiseur sur les données réelles chargées : il apprend les distributions statistiques de chaque colonne ainsi que leurs corrélations, afin de pouvoir ensuite générer des données synthétiques qui suivent les mêmes propriétés statistiques.

```
61 # Générer des échantillons synthétiques
```

→ Commentaire expliquant le but de la ligne suivante.

```
62 synthetic_data = synthesizer.sample(num_rows=500)
```

→ Génère 500 lignes de données synthétiques (aucune ligne n'est copiée du dataset réel : chaque ligne est tirée du modèle statistique appris) et les stocke dans la variable globale.

```
64 # Sauvegarder les données générées dans un fichier CSV
```

→ Commentaire expliquant le but de la ligne suivante.

```
65 synthetic_data.to_csv(os.path.join(DATA_DIR, "synthetic_data.csv"), index=False)
```

→ Écrit les données synthétiques générées dans un fichier CSV, sans colonne d'index (index=False évite d'ajouter une colonne numérique superflue au fichier).

```
66 print("\n Données synthétiques sauvegardées dans Data/synthetic_data.csv")
```

→ Affiche un message confirmant la sauvegarde des données synthétiques.

```
68 def generate_narrative_data():
```

→ Déclare la fonction de génération de données narratives synthétiques (texte libre, par opposition aux données tabulaires structurées ci-dessus).

```
69 """
```

→ Ouvre la docstring de la fonction generate_narrative_data.

```
70 Générer des données narratives synthétiques en utilisant GPT-2
```

→ Texte de la docstring décrivant le rôle de la fonction.

```
71 """
```

→ Ferme la docstring de la fonction.

```
72 global narrative_data
```

→ Indique que la fonction va créer/modifier la variable globale narrative_data.

```
74 # Créer un pipeline de génération de texte
```

→ Commentaire expliquant le but de la ligne suivante.

```
75 generator = pipeline("text-generation", model="gpt2")
```

→ Crée un pipeline transformers de génération de texte basé sur le modèle GPT-2. GPT-2 est un modèle de langage pré-entraîné publié par OpenAI en 2019 (petit comparé aux LLM actuels, mais léger et gratuit à exécuter localement) : ici il n'est pas utilisé pour dialoguer, mais purement pour produire du texte plausible (des messages fictifs de support client) servant de données synthétiques narratives.

```
77 # Générer des messages synthétiques
```

→ Commentaire expliquant le but des lignes suivantes.

```
78 prompts = ["Message support client:" for _ in range(10)]
```

→ Construit une liste de 10 amorces de texte identiques, qui serviront de point de départ à GPT-2 pour générer 10 messages différents.

```
79 messages = [
```


→ Commence une compréhension de liste destinée à stocker les messages générés.

```
80         generator(prompt, max_new_tokens=40, truncation=True)[0]["generated_text"].replace(prompt,
""").strip()
```

→ Génère du texte à partir de l'amorce (au maximum 40 nouveaux tokens, une limite de longueur pour rester rapide), récupère le texte généré (le pipeline retourne une liste de résultats, dont on prend le premier), retire l'amorce du résultat pour ne garder que la partie générée, et supprime les espaces superflus.

```
81     for prompt in prompts
```

→ Itère sur chaque amorce de la liste prompts pour alimenter la compréhension de liste.

```
82     ]
```

→ Ferme la compréhension de liste des messages générés.

```
84     # Créer et sauvegarder le DataFrame
```

→ Commentaire expliquant le but des lignes suivantes.

```
85     narrative_data = pd.DataFrame({
```

→ Commence la construction d'un DataFrame pandas à partir d'un dictionnaire de colonnes.

```
86         "id": [f"MSG-{i+1}" for i in range(len(messages))],
```

→ Construit la colonne id avec des identifiants du type MSG-1, MSG-2, etc.

```
87         "message": messages
```

→ Assigne la liste des messages générés à la colonne message du DataFrame.

```
88     })
```

→ Ferme le dictionnaire et l'appel au constructeur DataFrame.

```
89     narrative_data.to_csv(os.path.join(DATA_DIR, "narrative_data.csv"), index=False)
```

→ Écrit le DataFrame narratif dans un fichier CSV, sans colonne d'index.

```
90     print("\n Données narratives synthétiques sauvegardées dans Data/narrative_data.csv")
```

→ Affiche un message confirmant la sauvegarde des données narratives.

```
92 def diagnostic_report():
```

→ Déclare la fonction qui exécute un diagnostic de validité sur les données synthétiques.

```
93     """
```

→ Ouvre la docstring de la fonction diagnostic_report.

```
94     Exécuter un diagnostic pour s'assurer que les données sont valides
```

→ Texte de la docstring décrivant le rôle de la fonction.

```
95     """
```

→ Ferme la docstring de la fonction.

```
96     diagnostic = run_diagnostic(
```

→ Commence l'appel à la fonction run_diagnostic de SDV, qui vérifie des propriétés structurelles basiques (plages de valeurs respectées, formats cohérents...) plutôt que la qualité statistique fine ; c'est un contrôle de "validité" avant d'aller plus loin dans l'évaluation.

```
97         real_data=real_data,
```

→ Fournit les données réelles en argument nommé à run_diagnostic.

```
98         synthetic_data=synthetic_data,
```

→ Fournit les données synthétiques en argument nommé à run_diagnostic.

```
99         metadata=metadata
```

→ Fournit les métadonnées en argument nommé à `run_diagnostic`.

```
100     )
```

→ Ferme l'appel à la fonction `run_diagnostic`.

```
101     print("\n Rapport de diagnostic généré")
```

→ Affiche un message confirmant la génération du rapport de diagnostic.

```
102     return diagnostic
```

→ Retourne l'objet diagnostic obtenu.

```
104 def quality_report():
```

→ Déclare la fonction qui mesure la qualité statistique des données synthétiques.

```
105     """
```

→ Ouvre la docstring de la fonction `quality_report`.

```
106     Mesurer la qualité des données ou la similarité statistique entre les vraies et fausses données
```

→ Texte de la docstring décrivant le rôle de la fonction.

```
107     """
```

→ Ferme la docstring de la fonction.

```
108     global quality_report_result
```

→ Indique que la fonction va modifier la variable globale `quality_report_result`.

```
109     quality_report_result = evaluate_quality(
```

→ Commence l'appel à `evaluate_quality` de SDV, qui calcule un score de similarité statistique entre données réelles et synthétiques (par exemple, à quel point les distributions par colonne et les corrélations entre colonnes se ressemblent), et stocke le résultat dans la variable globale.

```
110         real_data,
```

→ Fournit les données réelles en premier argument positionnel à `evaluate_quality`.

```
111         synthetic_data,
```

→ Fournit les données synthétiques en deuxième argument positionnel à `evaluate_quality`.

```
112         metadata
```

→ Fournit les métadonnées en troisième argument positionnel à `evaluate_quality`.

```
113     )
```

→ Ferme l'appel à la fonction `evaluate_quality`.

```
114     print("\n Rapport de qualité généré")
```

→ Affiche un message confirmant la génération du rapport de qualité.

```
115     print(f"Score de qualité global: {quality_report_result.get_score():.3f}")
```

→ Affiche le score de qualité global (un nombre entre 0 et 1 résumant la ressemblance statistique globale) formaté avec trois décimales.

```
117 def column_plot():
```

→ Déclare la fonction qui affiche les détails de qualité colonne par colonne.

```
118     """
```

→ Ouvre la docstring de la fonction `column_plot`.

```
119     Afficher les détails de qualité par colonne
```

→ Texte de la docstring décrivant le rôle de la fonction.

```

120     """
    → Ferme la docstring de la fonction.

121     global quality_report_result
    → Indique que la fonction va lire et potentiellement modifier la variable globale quality_report_result.

122     if quality_report_result is None:
    → Vérifie si le rapport de qualité n'a pas encore été généré (protection contre un appel de column_plot avant quality_report).

123         print("Rapport de qualité pas encore généré. Génération en cours...")
    → Affiche un message indiquant que le rapport va être généré maintenant.

124         quality_report()
    → Appelle la fonction quality_report pour générer le rapport manquant à la volée.

125     print("Colonnes classées de la meilleure à la moins bonne qualité:")
    → Affiche un en-tête annonçant le classement des colonnes par qualité.

126     print("-----")
    → Affiche une ligne de séparation visuelle sous forme de tirets.

127     print(quality_report_result.get_details("Column Shapes"))
    → Affiche les détails de qualité par colonne issus du rapport SDV, filtrés sur la métrique "Column Shapes" (qui compare la distribution de chaque colonne prise individuellement entre données réelles et synthétiques).

129 def visualize_data():
    → Déclare la fonction qui affiche une visualisation comparant une colonne réelle et synthétique.

130     """
    → Ouvre la docstring de la fonction visualize_data.

131     Visualiser la comparaison d'une colonne entre les données réelles et synthétiques
    → Texte de la docstring décrivant le rôle de la fonction.

132     """
    → Ferme la docstring de la fonction.

133     # Visualiser en comparant une colonne (PreferredOrderCat) des vraies données aux synthétiques
    → Commentaire précisant la colonne utilisée pour la comparaison visuelle.

134     fig = get_column_plot(
    → Commence l'appel à get_column_plot de SDV, qui produit un graphique (histogramme ou courbe de distribution) superposant les valeurs réelles et synthétiques d'une colonne donnée, et stocke la figure retournée.

135         real_data=real_data,
    → Fournit les données réelles en argument nommé à get_column_plot.

136         synthetic_data=synthetic_data,
    → Fournit les données synthétiques en argument nommé à get_column_plot.

137         column_name="PreferredOrderCat",
    → Spécifie le nom de la colonne à comparer, ici PreferredOrderCat.

138         metadata=metadata
    → Fournit les métadonnées en argument nommé à get_column_plot.

139     )
    → Ferme l'appel à la fonction get_column_plot.

140     fig.show()

```

→ Affiche la figure générée (un graphique interactif Plotly, format utilisé par SDV) dans une fenêtre ou un navigateur.

```
141     print("\n Graphique de comparaison affiché")
```

→ Affiche un message confirmant que le graphique a été affiché.

```
143 def post_synthesis_menu():
```

→ Déclare la fonction affichant le menu interactif après la génération des données.

```
144     """
```

→ Ouvre la docstring de la fonction post_synthesis_menu.

```
145     Menu d'analyse post-synthèse permettant d'évaluer les résultats
```

→ Texte de la docstring décrivant le rôle de la fonction.

```
146     """
```

→ Ferme la docstring de la fonction.

```
147     print("\nDonnées synthétisées avec succès, consultez le fichier 'Data/synthetic_data.csv'")
```

→ Affiche un message confirmant le succès de la synthèse et l'emplacement du fichier.

```
148     print("""
```

→ Commence l'affichage d'une chaîne multi-lignes listant les options du menu.

```
149 Que voulez-vous faire ensuite ?
```

→ Ligne du menu demandant à l'utilisateur ce qu'il souhaite faire ensuite.

```
150 1. Voir le rapport de diagnostic
```

→ Première option du menu : consulter le rapport de diagnostic.

```
151 2. Voir le rapport de qualité
```

→ Deuxième option du menu : consulter le rapport de qualité.

```
152 3. Voir le graphique des colonnes
```

→ Troisième option du menu : consulter le graphique des colonnes.

```
153 4. Visualiser les données
```

→ Quatrième option du menu : visualiser les données.

```
154 5. Quitter
```

→ Cinquième option du menu : quitter le programme.

```
155 """)
```

→ Ferme la chaîne multi-lignes et l'appel à print affichant le menu.

```
157     while True:
```

→ Démarre une boucle infinie attendant un choix valide de l'utilisateur ; c'est le pattern classique de menu interactif en ligne de commande.

```
158         choice = input("Tapez votre choix : ").strip()
```

→ Lit la saisie de l'utilisateur et retire les espaces superflus en début et fin.

```
159         if choice == "1":
```

→ Vérifie si l'utilisateur a choisi l'option 1.

```
160             diagnostic_report()
```

→ Appelle la fonction diagnostic_report pour afficher le rapport de diagnostic.

```
161             elif choice == "2":
```

→ Vérifie si l'utilisateur a choisi l'option 2.

```

162         quality_report()
    → Appelle la fonction quality_report pour afficher le rapport de qualité.

163         elif choice == "3":
    → Vérifie si l'utilisateur a choisi l'option 3.

164         column_plot()
    → Appelle la fonction column_plot pour afficher les détails de qualité par colonne.

165         elif choice == "4":
    → Vérifie si l'utilisateur a choisi l'option 4.

166         visualize_data()
    → Appelle la fonction visualize_data pour afficher le graphique de comparaison.

167         elif choice == "5":
    → Vérifie si l'utilisateur a choisi l'option 5.

168         print(" Fermeture du programme.")
    → Affiche un message indiquant la fermeture du programme.

169         exit()
    → Termine l'exécution du programme.

170     else:
    → Cas où la saisie ne correspond à aucune option valide.

171         print(" Entrée invalide. Veuillez entrer un nombre de 1 à 5.")
    → Affiche un message d'erreur indiquant que la saisie est invalide.

173 # Menu principal
    → Commentaire annonçant le point d'entrée principal du script.

174 if __name__ == "__main__":
    → Vérifie que le script est exécuté directement et non importé comme module.

175     print("""
    → Commence l'affichage d'une chaîne multi-lignes présentant le menu initial des objectifs.

176 Quel est l'objectif des données que vous voulez synthétiser (tapez le numéro de l'objectif) :
    → Ligne du menu demandant à l'utilisateur de choisir l'objectif de la synthèse.

177 1. Réplication de schéma
    → Première option du menu initial : réplication de schéma.

178 2. Réalisme + variété
    → Deuxième option du menu initial : réalisme et variété.

179 3. Champs narratifs (Ex: avis, tickets de support)
    → Troisième option du menu initial : champs narratifs, avec des exemples.

180 Vous voulez connaître la différence entre les objectifs ? Tapez 'aide'
    → Ligne indiquant comment obtenir de l'aide sur la différence entre les objectifs.

181 """)
    → Ferme la chaîne multi-lignes et l'appel à print affichant le menu initial.

```

183 while True:
→ Démarre une boucle infinie attendant une saisie valide de l'utilisateur.

184 user_input = input("Tapez votre choix : ")
→ Lit la saisie brute de l'utilisateur sans la nettoyer.

185 if user_input == "aide":
→ Vérifie si l'utilisateur a demandé de l'aide.

186 print("""
→ Commence l'affichage d'une chaîne multi-lignes expliquant les trois objectifs.

187 1. Réplication de schéma : S'assure que les données synthétiques correspondent à la structure
→ Première ligne explicative sur l'objectif de réplication de schéma.

188 (colonnes, types de données, contraintes) du dataset original.
→ Suite de l'explication précisant ce que couvre la réplication de schéma.

190 2. Réalisme + variété : Se concentre sur la création de données synthétiques réalistes et
→ Première ligne explicative sur l'objectif de réalisme et variété.

191 diversifiées, capturant les motifs et la variabilité observés dans les vraies données.
→ Suite de l'explication précisant ce que couvre le réalisme et la variété.

193 3. Champs narratifs : Implique la génération de texte cohérent et humain pour des champs
→ Première ligne explicative sur l'objectif des champs narratifs.

194 comme les avis ou tickets de support, nécessitant compréhension et génération du langage naturel.
→ Suite de l'explication précisant les exemples de champs narratifs concernés.

195 """)
→ Ferme la chaîne multi-lignes et l'appel à print affichant l'explication d'aide.

196 elif user_input == "1":
→ Vérifie si l'utilisateur a choisi l'option 1 (réplication de schéma).

197 synthesize_data()
→ Appelle la fonction synthesize_data sans activer le mode réalisme.

198 post_synthesis_menu()
→ Affiche le menu d'analyse post-synthèse après la génération des données.

199 elif user_input == "2":
→ Vérifie si l'utilisateur a choisi l'option 2 (réalisme et variété).

200 synthesize_data(realism=True)
→ Appelle la fonction synthesize_data en activant le mode réalisme (paramètre transmis mais non exploité dans le corps actuel de la fonction, comme vu plus haut).

201 post_synthesis_menu()
→ Affiche le menu d'analyse post-synthèse après la génération des données.

202 elif user_input == "3":
→ Vérifie si l'utilisateur a choisi l'option 3 (champs narratifs).

203 generate_narrative_data()
→ Appelle la fonction generate_narrative_data pour générer les messages synthétiques via GPT-2.

204 # Note: Pas de menu post-synthèse pour les données narratives dans le code original

→ Commentaire signalant l'absence volontaire de menu post-synthèse pour les données narratives (les fonctions de rapport/qualité de SDV s'appliquent aux données tabulaires, pas au texte libre généré par GPT-2).

```
205         print("\nDonnées narratives générées. Tapez 'quit' pour quitter ou recommencez.")
```

→ Affiche un message confirmant la génération des données narratives.

```
206     else:
```

→ Cas où la saisie ne correspond à aucune option valide ni à 'aide'.

```
207         print("Entrée invalide. Veuillez entrer 1, 2, 3 ou 'aide'.")
```

→ Affiche un message d'erreur indiquant que la saisie est invalide.

```
208         continue
```

→ Reprend immédiatement la boucle pour redemander une saisie à l'utilisateur.

Projet 22 — 22-Cyber_Workflow

Reconnaissance + tests de securite passifs sur un domaine

Objectif

Automatise une phase de reconnaissance et de tests de securite passifs sur un nom de domaine donne (sous-domaines, ports ouverts, en-tetes de securite manquants...) et compile les resultats dans un rapport, a des fins d'audit de securite defensif sur un perimetre autorise.

Code source, ligne par ligne

22-Cyber_Workflow/OffCyberTest.py

Ce fichier définit, avec le framework agno, un workflow de test de cybersécurité offensif qui enchaîne un agent de reconnaissance passive et un agent de test outillé sur un domaine cible.

134 lignes de code annotées ci-dessous.

```
1 import sys
    → Importe le module standard sys pour accéder aux flux de sortie standard et d'erreur.

3 # La console Windows utilise par défaut l'encodage cp1252, qui ne supporte pas
    → Commentaire expliquant le problème d'encodage de la console Windows.

4 # certains caractères utilisés dans le rendu rich (icônes, box-drawing).
    → Suite du commentaire précisant les caractères spéciaux concernés par le problème.

5 for _stream in (sys.stdout, sys.stderr):
    → Parcourt les deux flux standard de sortie et d'erreur.

6     if hasattr(_stream, "reconfigure"):
        → Vérifie si le flux courant possède une méthode reconfigure.

7         _stream.reconfigure(encoding="utf-8", errors="replace")
        → Reconfigure le flux pour utiliser l'encodage UTF-8 et remplacer les caractères non supportés ; même correctif que dans
data_synthetizer.py, nécessaire ici car la bibliothèque rich (utilisée plus bas pour l'affichage terminal) dessine des cadres et
icônes qui ne passent pas dans l'encodage par défaut de la console Windows.

9 import os
    → Importe le module standard os pour accéder aux variables d'environnement.

10 import whois
    → Importe la bibliothèque whois, qui interroge les serveurs WHOIS publics pour récupérer les informations d'enregistrement
d'un nom de domaine (propriétaire, date de création, registrar...). C'est un outil classique de reconnaissance passive en
cybersécurité : il ne fait que consulter des informations publiques, sans interagir avec la cible elle-même.

11 import requests
    → Importe la bibliothèque requests, la bibliothèque HTTP la plus utilisée en Python pour envoyer des requêtes web (GET,
HEAD, POST...) et lire les réponses ; elle sert ici à interroger les en-têtes HTTP du site cible.

12 import dns.resolver
    → Importe le module de résolution DNS (de la bibliothèque dnspython), qui permet d'interroger directement les serveurs DNS
pour obtenir les enregistrements associés à un domaine (ici les enregistrements de type A, c'est-à-dire les adresses IP).
```


13 from textwrap import dedent

→ Importe la fonction dedent qui retire l'indentation commune d'un texte multi-lignes, utilisée ici pour les prompts des agents comme dans contract_analysis.py.

14 from typing import Iterator

→ Importe le type générique Iterator du module typing, utilisé pour annoter les fonctions qui retournent un générateur (produisant des valeurs une à une via yield) plutôt qu'une valeur unique.

15 from dotenv import load_dotenv

→ Importe la fonction qui charge les variables définies dans un fichier .env, pour garder la clé d'API hors du code source.

16 from agno.utils.log import logger

→ Importe l'objet logger du framework agno, un journal d'événements préconfiguré qui permet de tracer le déroulement du workflow (ex: "Running Recon for domain...") de façon structurée, plutôt que d'utiliser de simples print().

17 from agno.workflow import Workflow

→ Importe la classe Workflow du framework agno. Contrairement à un Team (qui laisse un agent-leader décider dynamiquement de la coordination), un Workflow permet d'écrire en code Python classique la logique d'enchaînement entre agents et outils (étape 1 puis étape 2, avec conditions et transformations explicites) : on garde ainsi un contrôle déterministe sur l'ordre des opérations tout en réutilisant des Agents comme "briques" à l'intérieur.

18 from agno.agent import Agent, RunOutput as RunResponse

→ Importe la classe Agent et la classe RunOutput renommée en RunResponse (via l'alias `as`, probablement pour rester compatible avec un nom utilisé dans une version antérieure de la bibliothèque agno).

19 from agno.models.groq import Groq

→ Importe la classe Groq qui permet d'utiliser un modèle hébergé sur Groq comme moteur LLM.

20 from agno.utils.pprint import pprint_run_response

→ Importe la fonction utilitaire pprint_run_response d'agno, qui affiche proprement dans le terminal la réponse (ou le flux de réponses) d'un agent, avec mise en forme.

21 from tools import check_security_headers, check_common_directories, find_js_urls, check_robots_txt, check_sitemap, check_env_exposure, check_server_header, check_http_redirect, check_github_mentions

→ Importe depuis le module local tools les neuf fonctions d'outils de test de sécurité. Ce sont ces fonctions Python "brutes" qui seront ensuite données au Test Agent comme outils (tool use) : le LLM ne les exécute pas lui-même, il décide lesquelles appeler et le framework agno les exécute réellement pour lui.

23 load_dotenv()

→ Charge les variables d'environnement du fichier .env dans le processus courant.

25 # Le script original utilisait OpenAI (gpt-4o) ; remplacé par Groq pour

→ Commentaire expliquant que le modèle OpenAI d'origine a été remplacé par Groq.

26 # réutiliser la clé GROQ_API_KEY déjà disponible.

→ Suite du commentaire précisant la raison du remplacement par Groq.

27 GROQ_API_KEY = os.getenv("GROQ_API_KEY")

→ Lit la clé d'API Groq depuis les variables d'environnement et la stocke dans une constante.

29 def extract_domain(url: str) -> str:

→ Déclare une fonction qui extrait le nom de domaine à partir d'une URL complète (utile si l'utilisateur saisit une URL complète comme "https://exemple.com/page" au lieu d'un simple domaine).

30 """Extract domain from a full URL."""

→ Docstring décrivant en une ligne le rôle de la fonction.

31 return url.split("//")[-1].split("/")[0]

→ Découpe l'URL sur "/" (pour retirer le protocole http:// ou https://) puis sur "/" (pour retirer tout chemin après le domaine) et

retourne le premier segment restant, à savoir le nom de domaine.

```
33 def get_whois(domain: str) -> str:
```

→ Déclare une fonction qui récupère les informations WHOIS d'un domaine.

```
34     try:
```

→ Ouvre un bloc try pour capturer les erreurs de la requête WHOIS (serveur WHOIS injoignable, domaine inexistant...).

```
35         w = whois.whois(domain)
```

→ Effectue la requête WHOIS sur le domaine et stocke le résultat (un objet contenant les champs d'enregistrement du domaine).

```
36         return "\n".join(f"{k}: {v}" for k, v in w.items())
```

→ Formate chaque paire clé-valeur du résultat WHOIS sur une ligne ("clé: valeur") et les joint par des retours à la ligne, pour obtenir un texte lisible à injecter ensuite dans le prompt de l'agent de reconnaissance.

```
37     except Exception as e:
```

→ Capture toute exception survenue pendant la requête WHOIS.

```
38         return f"WHOIS lookup failed: {e}"
```

→ Retourne un message d'erreur incluant le détail de l'exception capturée, plutôt que de faire planter tout le workflow si la recherche WHOIS échoue.

```
40 def get_dns_records(domain: str) -> str:
```

→ Déclare une fonction qui récupère les enregistrements DNS de type A d'un domaine.

```
41     try:
```

→ Ouvre un bloc try pour capturer les erreurs de la résolution DNS.

```
42         answers = dns.resolver.resolve(domain, 'A')
```

→ Résout le domaine pour obtenir ses enregistrements DNS de type A (les adresses IPv4 associées au domaine).

```
43         return "\n".join(rdata.address for rdata in answers)
```

→ Extrait l'adresse IP de chaque enregistrement retourné et les joint par des retours à la ligne.

```
44     except Exception as e:
```

→ Capture toute exception survenue pendant la résolution DNS.

```
45         return f"DNS lookup failed: {e}"
```

→ Retourne un message d'erreur incluant le détail de l'exception capturée.

```
47 def get_http_headers(domain: str) -> str:
```

→ Déclare une fonction qui récupère les en-têtes HTTP d'un domaine.

```
48     url = f"https://{domain}"
```

→ Construit l'URL HTTPS complète à partir du nom de domaine.

```
49     try:
```

→ Ouvre un bloc try pour capturer les erreurs de la requête HTTP.

```
50         response = requests.head(url, timeout=5)
```

→ Envoie une requête HTTP HEAD (qui ne récupère que les en-têtes, pas le corps de la page, donc plus légère qu'un GET) vers l'URL avec un délai d'attente de 5 secondes.

```
51         headers = response.headers
```

→ Récupère les en-têtes HTTP de la réponse.

```
52         return "\n".join(f"{k}: {v}" for k, v in headers.items())
```

→ Formate chaque en-tête sur une ligne et les joint par des retours à la ligne.

53 `except Exception as e:`

→ Capture toute exception survenue pendant la requête HTTP.

54 `return f"HTTP header fetch failed: {e}"`

→ Retourne un message d'erreur incluant le détail de l'exception capturée.

57 `class OffensiveCybersecurityTester(Workflow):`

→ Déclare la classe du workflow, héritant de la classe Workflow du framework agno décrite plus haut : c'est cette classe qui va combiner récupération de données (WHOIS/DNS/HTTP), un agent de reconnaissance et un agent de test dans une séquence de code explicite.

58 `recon_agent: Agent = Agent(`

→ Déclare un attribut de classe recon_agent, typé Agent, et commence sa construction. Contrairement à contract_analysis.py où les agents sont créés par des fonctions "usine", ici l'agent est construit directement comme attribut de classe : il est donc instancié une seule fois au chargement de la classe et partagé par toutes les instances de OffensiveCybersecurityTester.

59 `name = 'Recon Agent',`

→ Donne le nom affiché "Recon Agent" à cet agent.

60 `model=Groq(id='openai/gpt-oss-120b', api_key=GROQ_API_KEY),`

→ Configure le modèle Groq 120B (modèle à poids ouverts gpt-oss, servi par l'infrastructure rapide de Groq) avec la clé d'API comme moteur de cet agent.

61 `description=dedent("""`

→ Débute le paramètre description avec une chaîne multi-lignes dédentée. Contrairement à instructions (le "comment faire"), description sert généralement à résumer en langage naturel le rôle général de l'agent, utile notamment pour l'affichage ou pour qu'un autre agent/leader sache à quoi sert celui-ci.

62 `Gather detailed information about the target URL to understand its environment and potential vulnerabilities.`

→ Décrit la mission de l'agent : collecter des informations détaillées sur l'URL cible.

63 `This agent performs passive reconnaissance by collecting public data without interacting aggressively with the target.`

→ Précise que l'agent effectue une reconnaissance passive : il ne fait que lire des informations publiquement accessibles (WHOIS, DNS, en-têtes) sans envoyer de requêtes intrusives, ce qui reste légal et sans risque même sans autorisation explicite du propriétaire du site.

64 `"""),`

→ Ferme la chaîne multi-lignes et l'appel à dedent pour le paramètre description.

65 `instructions = dedent("""`

→ Débute le paramètre instructions avec une chaîne multi-lignes dédentée.

66 `You are a Recon Agent. You will receive reconnaissance data about a target domain.`

→ Présente à l'agent son rôle et précise qu'il recevra des données de reconnaissance sur un domaine.

68 `Your tasks:`

→ Introduit la liste des tâches assignées à l'agent de reconnaissance.

69 `1. Analyze the WHOIS, DNS, and any available metadata.`

→ Première tâche : analyser les données WHOIS, DNS et toute métadonnée disponible.

70 `2. Based on patterns (e.g. tech stack, DNS structure, WHOIS info), **categorize** the target:`

→ Deuxième tâche : catégoriser la cible en se basant sur les motifs observés.

71 `- Is it an e-commerce site, blog, SaaS app, API backend, login portal, etc.?`

→ Précise les catégories possibles de cible à identifier.

72 3. Identify notable or suspicious findings (e.g., short registration, strange DNS, tech used).

→ Troisième tâche : identifier les constats notables ou suspects, avec des exemples.

73 4. Based on the category, ****suggest specific types of tests**** that could be safely conducted later.

→ Quatrième tâche : suggérer des types de tests spécifiques pouvant être menés en toute sécurité ; c'est cette sortie qui sera transmise telle quelle au Test Agent, qui décidera ensuite quels outils utiliser en fonction de ces suggestions (première étape du pipeline en deux agents).

74 Example: If it's an e-commerce site, suggest testing cart, search, login, etc.

→ Donne un exemple concret de suggestion de test pour un site e-commerce.

76 Your output should include:

→ Introduit la liste des éléments attendus dans la sortie de l'agent.

77 - Target category

→ Premier élément attendu dans la sortie : la catégorie de la cible.

78 - Key observations

→ Deuxième élément attendu dans la sortie : les observations clés.

79 - List of recommended test types (with short reasons)

→ Troisième élément attendu dans la sortie : la liste des types de tests recommandés avec justification.

81 Do not make up data. Only use what's in the input.

→ Interdit à l'agent d'inventer des données et l'oblige à se limiter aux données d'entrée fournies (WHOIS/DNS/en-têtes réels) : une consigne anti-hallucination, essentielle dans un contexte de sécurité où une fausse information pourrait orienter des tests inutiles ou dangereux.

82 """),

→ Ferme la chaîne multi-lignes et l'appel à dedent pour le paramètre instructions.

83)

→ Ferme l'appel au constructeur Agent pour recon_agent.

85 test_agent: Agent = Agent(

→ Déclare un attribut de classe test_agent, typé Agent, et commence sa construction.

86 name = 'Test Agent',

→ Donne le nom affiché "Test Agent" à cet agent.

87 model=Groq(id='openai/gpt-oss-120b', api_key=GROQ_API_KEY),

→ Configure le modèle Groq 120B avec la clé d'API comme moteur de cet agent.

88 description=dedent("""

→ Débute le paramètre description avec une chaîne multi-lignes dédentée.

89 You are a Test Agent. You will receive an Agent Summary output of recommended tests to conduct on the target domain.

→ Décrit la mission de l'agent : recevoir un résumé de tests recommandés à mener sur le domaine cible (produit par le Recon Agent).

90 """),

→ Ferme la chaîne multi-lignes et l'appel à dedent pour le paramètre description.

91 instructions = dedent("""

→ Débute le paramètre instructions avec une chaîne multi-lignes dédentée.

- 92 You are the Tester Agent in an offensive cybersecurity workflow.
→ Présente à l'agent son rôle de testeur dans un workflow de cybersécurité offensive.
- 94 You are equipped with the following ****safe and legal tools****:
→ Introduit la liste des outils sûrs et légaux mis à disposition de l'agent ; le prompt documente chaque outil en langage naturel pour que le LLM sache quand et pourquoi l'utiliser, en complément de la description technique fournie automatiquement par le paramètre tools plus bas.
- 96 1. check_security_headers - Inspect CSP, HSTS, and X-Frame headers.
→ Décrit l'outil 1 : inspection des en-têtes de sécurité CSP, HSTS et X-Frame.
- 97 2. check_common_directories - Scan for /admin, /login, /config, etc.
→ Décrit l'outil 2 : recherche de répertoires communs comme /admin ou /login.
- 98 3. find_js_urls - List JS file URLs from the homepage.
→ Décrit l'outil 3 : liste des URLs de fichiers JavaScript de la page d'accueil.
- 99 4. check_robots_txt - Look for robots.txt and list contents.
→ Décrit l'outil 4 : recherche et affichage du contenu du fichier robots.txt.
- 100 5. check_sitemap - Look for sitemap.xml and print summary.
→ Décrit l'outil 5 : recherche et résumé du fichier sitemap.xml.
- 101 6. check_env_exposure - Check for exposed .env file.
→ Décrit l'outil 6 : vérification de l'exposition d'un fichier .env.
- 102 7. check_server_header - Read the 'Server' HTTP header.
→ Décrit l'outil 7 : lecture de l'en-tête HTTP 'Server'.
- 103 8. check_http_redirect - Check if HTTP redirects to HTTPS.
→ Décrit l'outil 8 : vérification de la redirection HTTP vers HTTPS.
- 104 9. check_github_mentions - Provide Google search link for GitHub leaks.
→ Décrit l'outil 9 : fourniture d'un lien de recherche Google pour des fuites sur GitHub.
- 106 Your tasks:
→ Introduit la liste des tâches assignées à l'agent testeur.
- 108 1. Read the ****Recon Agent's output**** (which includes recon analysis and test recommendations).
→ Première tâche : lire la sortie de l'agent de reconnaissance.
- 109 2. Based on those recommendations, ****select only the relevant tools**** to run.
→ Deuxième tâche : sélectionner uniquement les outils pertinents en fonction des recommandations ; c'est l'agent lui-même (le LLM), pas le code Python, qui décide dynamiquement quels outils appeler parmi les neuf disponibles, en fonction du contexte de la cible.
- 110 3. Perform safe, non-intrusive tests using only those selected tools.
→ Troisième tâche : effectuer des tests sûrs et non intrusifs avec les outils sélectionnés.
- 111 4. Return a structured response including:
→ Quatrième tâche : retourner une réponse structurée incluant certains éléments listés ensuite.
- 113 - Which tools were used and why (brief reasoning).
→ Premier élément de la réponse structurée : les outils utilisés et leur justification brève.
- 114 - Which tools were skipped and why (e.g. "not relevant" or "not recommended").
→ Deuxième élément : les outils ignorés et la raison de leur exclusion.

115 - A brief summary of findings from each test used.
→ Troisième élément : un résumé bref des constats de chaque test utilisé.

116 - A security risk score: Low / Medium / High (based on number and severity of issues found).
→ Quatrième élément : un score de risque de sécurité Faible/Moyen/Élevé basé sur les problèmes trouvés.

117 - (Optional) Suggest other safe checks to run later if helpful.
→ Cinquième élément, optionnel : suggérer d'autres vérifications sûres à effectuer plus tard.

119 Rules:
→ Introduit la liste des règles strictes que l'agent doit respecter.

120 - Never run tools unless explicitly recommended by the recon phase.
→ Première règle : ne jamais exécuter d'outils sauf s'ils sont explicitement recommandés par la reconnaissance ; cette règle relie strictement les deux agents entre eux, empêchant le Test Agent d'agir de façon indépendante ou imprévisible.

121 - Never test destructively. Only public, passive, and educational analysis is allowed.
→ Deuxième règle : ne jamais tester de façon destructive, seule l'analyse publique et passive est autorisée — un garde-fou éthique et légal essentiel, puisque tous les outils fournis (voir tools.py) se limitent effectivement à de la lecture d'informations publiques, sans tentative d'exploitation.

122 """),
→ Ferme la chaîne multi-lignes et l'appel à dedent pour le paramètre instructions.

123 tools=[check_security_headers, check_common_directories, find_js_urls, check_robots_txt, check_sitemap, check_env_exposure, check_server_header, check_http_redirect, check_github_mentions],
→ Fournit à l'agent la liste des neuf fonctions d'outils qu'il peut appeler (function calling / tool use) ; c'est ce paramètre qui rend techniquement possible l'exécution des outils décrits en langage naturel dans les instructions ci-dessus — sans lui, l'agent pourrait en parler mais pas réellement les exécuter.

124)
→ Ferme l'appel au constructeur Agent pour test_agent.

126 def run(self, domain: str) -> Iterator[RunResponse]:
→ Déclare la méthode run du workflow, qui reçoit un domaine et retourne un itérateur de réponses (Iterator[RunResponse]) : le type de retour indique que cette méthode est un générateur Python (elle contient des instructions yield plus bas), permettant de produire les réponses progressivement plutôt que d'attendre la fin de tout le traitement pour renvoyer un seul résultat.

127 domain = extract_domain(domain)
→ Nettoie le domaine reçu en extrayant uniquement le nom de domaine de l'URL éventuelle.

128 whois_info = get_whois(domain)
→ Récupère les informations WHOIS du domaine et les stocke dans une variable locale.

129 dns_records = get_dns_records(domain)
→ Récupère les enregistrements DNS du domaine et les stocke dans une variable locale.

130 http_headers = get_http_headers(domain)
→ Récupère les en-têtes HTTP du domaine et les stocke dans une variable locale.

131 full_context = f""
→ Débute la construction d'une chaîne de contexte multi-lignes interpolée regroupant les données collectées ; c'est ce texte qui sera envoyé comme entrée au Recon Agent, à la place de lui donner un accès direct aux fonctions WHOIS/DNS/HTTP (ici la collecte de données est faite en code Python classique, pas via des outils de l'agent, contrairement au Test Agent plus loin).

132 Domain: {domain}
→ Insère le nom du domaine dans le texte de contexte.

133 WHOIS info:
→ Insère un intitulé annonçant les informations WHOIS dans le texte de contexte.

134 {whois_info}
→ Insère le contenu des informations WHOIS collectées dans le texte de contexte.

135 DNS records:
→ Insère un intitulé annonçant les enregistrements DNS dans le texte de contexte.

136 {dns_records}
→ Insère le contenu des enregistrements DNS collectés dans le texte de contexte.

137 HTTP headers:
→ Insère un intitulé annonçant les en-têtes HTTP dans le texte de contexte.

138 {http_headers}
→ Insère le contenu des en-têtes HTTP collectés dans le texte de contexte.

139 ""
→ Ferme la chaîne de contexte multi-lignes interpolée assignée à full_context.

140 logger.info(f'Running Recon for domain: {domain}')

→ Journalise un message d'information indiquant le démarrage de la reconnaissance pour ce domaine, via le logger agno importé plus haut.

141 recon_response: Iterator[RunResponse] = self.recon_agent.run(full_context)
→ Exécute l'agent de reconnaissance avec le contexte complet (WHOIS + DNS + en-têtes) et stocke sa réponse ; c'est l'appel .run() qui déclenche réellement l'appel au LLM via Groq.

142 if recon_response is None or not recon_response.content:
→ Vérifie si la réponse de reconnaissance est absente ou si son contenu est vide (garde-fou en cas d'échec de l'appel LLM).

143 yield RunResponse(
→ Commence la production (yield) d'une réponse d'erreur si la reconnaissance a échoué. Le mot-clé yield, propre aux générateurs Python, produit une valeur au fil de l'exécution sans arrêter définitivement la fonction, contrairement à return.

144 run_id=self.run_id,
→ Associe l'identifiant d'exécution courant du workflow à la réponse d'erreur produite.

145 content="Sorry, could not get the Recon analysis report.",
→ Définit le contenu textuel de la réponse d'erreur signalant l'échec de la reconnaissance.

146)
→ Ferme la construction de l'objet RunResponse d'erreur.

147 return
→ Arrête l'exécution de la méthode run après avoir produit la réponse d'erreur (un return sans valeur dans un générateur termine simplement l'itération).

148 logger.info('Testing the target domain')

→ Journalise un message d'information indiquant le démarrage de la phase de test.

149 yield from self.test_agent.run(recon_response.content, stream=True)
→ Délègue la production des réponses à l'agent testeur : stream=True fait que l'agent produit sa réponse progressivement (token par token ou par blocs) plutôt que d'un seul bloc final, et yield from relaie chacun de ces morceaux directement vers l'appelant du workflow au fur et à mesure, permettant un affichage en temps réel côté utilisateur.

```
151 if __name__ == '__main__':
```

→ Vérifie que le script est exécuté directement et non importé comme module.

```
152     from rich.prompt import Prompt
```

→ Importe la classe Prompt de la bibliothèque rich, une bibliothèque de mise en forme de terminal (couleurs, tableaux, barres de progression...) ; Prompt fournit une saisie interactive en ligne de commande plus soignée qu'un simple input().

```
154     workflow = OffensiveCybersecurityTester()
```

→ Instancie la classe du workflow de test de cybersécurité offensif.

```
155     domain = Prompt.ask("Enter the domain to test")
```

→ Demande interactivement à l'utilisateur de saisir le domaine à tester, via l'invite stylée de rich.

```
156     response: Iterator[RunResponse] = workflow.run(domain=domain)
```

→ Lance l'exécution du workflow sur le domaine saisi et stocke l'itérateur de réponses retourné (rien n'est encore exécuté à ce stade : un générateur Python ne s'exécute que lorsqu'on itère dessus).

```
157     pprint_run_response(response, markdown=True)
```

→ Affiche joliment les réponses du workflow au fur et à mesure qu'elles sont produites (grâce au flux streamé), avec un rendu markdown dans le terminal via rich.

22-Cyber_Workflow/tools.py

Ce fichier fournit un ensemble de fonctions d'outils de test de sécurité passif (en-têtes, cookies, SSL, répertoires, fichiers exposés) destinées à être utilisées comme outils par l'agent testeur du workflow, individuellement ou via un scan complet en ligne de commande.

183 lignes de code annotées ci-dessous.

```
1 import requests
```

→ Importe la bibliothèque requests pour effectuer des requêtes HTTP.

```
2 from bs4 import BeautifulSoup
```

→ Importe la classe BeautifulSoup de la bibliothèque bs4 (Beautiful Soup), une bibliothèque de parsing HTML/XML qui permet de naviguer facilement dans la structure d'une page web (retrouver des balises, leurs attributs...) sans écrire d'expressions régulières fragiles.

```
3 import ssl
```

→ Importe le module standard ssl, qui donne accès aux fonctions bas niveau de gestion des connexions et certificats TLS/SSL de Python.

```
4 import socket
```

→ Importe le module standard socket, qui permet d'établir des connexions réseau de bas niveau (TCP/IP), nécessaire ici pour ouvrir une connexion TLS brute afin d'inspecter le certificat du serveur.

```
5 from datetime import datetime
```

→ Importe la classe datetime pour manipuler les dates d'expiration de certificats.

```
7 def check_http_redirect(domain: str) -> str:
```

→ Déclare la fonction qui vérifie si le trafic HTTP redirige vers HTTPS. C'est un test de sécurité de base : un site qui ne redirige pas automatiquement vers HTTPS expose ses visiteurs à des connexions non chiffrées.

```
8     """Vérifie si HTTP redirige vers HTTPS"""
```

→ Docstring décrivant en une ligne le rôle de la fonction.

```
9     try:
```

→ Ouvre un bloc try pour capturer les erreurs de la requête HTTP.


```

10     r = requests.get(f"http://{domain}", timeout=5, allow_redirects=True)
    → Envoie une requête HTTP GET en HTTP simple (non chiffré) sur le domaine, en suivant automatiquement les redirections (allow_redirects=True), avec un délai de 5 secondes.

11     if r.url.startswith("https://"):
    → Vérifie si l'URL finale après redirection commence par https://.

12         return " HTTP redirects to HTTPS."
    → Retourne un message confirmant que HTTP redirige correctement vers HTTPS.

13     else:
    → Cas où l'URL finale ne commence pas par https://.

14         return " HTTP does not redirect securely. Final URL: {r.url}"
    → Retourne un message signalant l'absence de redirection sécurisée, avec l'URL finale (à noter : il manque le préfixe f devant cette chaîne, donc {r.url} s'affiche littéralement au lieu d'être interpolé — un petit bug du code d'origine, sans conséquence fonctionnelle grave).

15     except Exception as e:
    → Capture toute exception survenue pendant la requête HTTP.

16         return f"Redirect check failed: {e}"
    → Retourne un message d'erreur incluant le détail de l'exception capturée.

18 def check_github_mentions(domain: str) -> str:
    → Déclare la fonction qui prépare une recherche des mentions GitHub du domaine. Elle ne fait pas de scraping automatique de GitHub (ce qui nécessiterait une API et poserait des questions de rate limiting), mais génère simplement un lien de recherche Google que l'utilisateur humain peut consulter manuellement — une façon simple et sans risque de repérer des fuites de code ou de secrets liés au domaine.

19     """Recherche les mentions GitHub du domaine"""
    → Docstring décrivant en une ligne le rôle de la fonction.

20     try:
    → Ouvre un bloc try, bien qu'aucune opération risquée ne soit effectuée dans ce cas.

21         query = f'"{domain}" site:github.com'
    → Construit une requête de recherche restreinte au domaine github.com (opérateur de recherche "site:") pour le nom de domaine cible entre guillemets, afin de cibler les mentions exactes.

22         return f" Search manually for GitHub mentions:\\nhttps://www.google.com/search?q={query.replace(' ', '+')}}"
    → Retourne un lien de recherche Google avec la requête encodée en remplaçant les espaces par des symboles plus (encodage minimal d'URL).

23     except Exception as e:
    → Capture toute exception qui pourrait survenir pendant la construction de la requête.

24         return f"GitHub mention check failed: {e}"
    → Retourne un message d'erreur incluant le détail de l'exception capturée.

26 def check_ssl_certificate(domain: str) -> str:
    → Déclare la fonction qui vérifie la validité du certificat SSL/TLS du domaine. Un certificat expiré ou sur le point d'expirer est un problème de sécurité opérationnel classique (le site devient inaccessible en HTTPS, ou pire, vulnérable si mal renouvelé).

27     """Vérifie le certificat SSL"""
    → Docstring décrivant en une ligne le rôle de la fonction.

28     try:
    → Ouvre un bloc try pour capturer les erreurs de connexion ou de certificat.

```

```
29         context = ssl.create_default_context()
```

→ Crée un contexte SSL par défaut avec les paramètres de sécurité standards du système (vérification de la chaîne de certificats, algorithmes recommandés...).

```
30         with socket.create_connection((domain, 443), timeout=5) as sock:
```

→ Établit une connexion TCP brute vers le domaine sur le port 443 (le port standard HTTPS), avec un délai de 5 secondes, dans un bloc contextuel qui fermera automatiquement la socket.

```
31         with context.wrap_socket(sock, server_hostname=domain) as ssock:
```

→ Enveloppe la connexion TCP dans une couche TLS (négociation du handshake SSL) en vérifiant que le nom d'hôte du serveur correspond bien au certificat présenté (protection contre les certificats usurpés).

```
32         cert = ssock.getpeercert()
```

→ Récupère les informations du certificat SSL présenté par le serveur distant (dates de validité, émetteur, sujet...).

```
33         expiry = datetime.strptime(cert['notAfter'], "%b %d %H:%M:%S %Y %Z")
```

→ Analyse la date d'expiration du certificat (champ 'notAfter', une chaîne au format texte) pour la convertir en objet datetime manipulable.

```
34         days_left = (expiry - datetime.utcnow()).days
```

→ Calcule le nombre de jours restants avant l'expiration du certificat, en soustrayant la date actuelle (UTC) de la date d'expiration.

```
35         return f"SSL Certificate is valid. Expires in {days_left} days."
```

→ Retourne un message confirmant la validité du certificat et le nombre de jours restants.

```
36     except Exception as e:
```

→ Capture toute exception survenue pendant la connexion ou la lecture du certificat.

```
37         return f"SSL certificate check failed: {e}"
```

→ Retourne un message d'erreur incluant le détail de l'exception capturée.

```
39 def check_api_endpoints(domain: str) -> str:
```

→ Déclare la fonction qui recherche des endpoints API communs sur le domaine (technique de "reconnaissance de surface d'attaque" : repérer des points d'entrée non documentés publiquement, comme une API ou une interface d'administration).

```
40     """Vérifie les endpoints API communs"""
```

→ Docstring décrivant en une ligne le rôle de la fonction.

```
41     common_paths = ["/api", "/api/v1", "/graphql", "/swagger", "/admin", "/api-docs"]
```

→ Définit la liste des chemins d'API courants à tester (approche par "wordlist", une liste de chemins fréquemment utilisés par les applications web).

```
42     found = []
```

→ Initialise une liste vide qui accumulera les endpoints trouvés.

```
44     for path in common_paths:
```

→ Parcourt chaque chemin de la liste des chemins d'API courants.

```
45         url = f"https://{domain}{path}"
```

→ Construit l'URL complète en combinant le domaine et le chemin courant.

```
46         try:
```

→ Ouvre un bloc try pour capturer les erreurs de requête sur ce chemin précis.

```
47             r = requests.get(url, timeout=3)
```

→ Envoie une requête HTTP GET vers l'URL construite, avec un délai de 3 secondes (plus court que pour les requêtes uniques, car ce test en enchaîne plusieurs).

```

48         if r.status_code in [200, 403, 401]:
    → Vérifie si le code de statut retourné indique une ressource présente (200) ou protégée mais existante (403 Forbidden, 401 Unauthorized) — dans les deux cas, cela révèle que quelque chose existe à ce chemin, même si l'accès est refusé.

49         found.append(f"{url} - Status {r.status_code}")
    → Ajoute l'URL et son code de statut à la liste des endpoints trouvés.

50     except:
    → Capture silencieusement toute exception survenue pour ce chemin (timeout, DNS, etc.), pour ne pas interrompre le balayage des autres chemins.

51     continue
    → Passe au chemin suivant de la boucle sans interrompre le traitement.

53     if found:
    → Vérifie si au moins un endpoint a été trouvé.

54     return " Possible API endpoints:\n" + "\n".join(found)
    → Retourne la liste des endpoints trouvés, préfixée d'un avertissement.

55     else:
    → Cas où aucun endpoint n'a été trouvé.

56     return " No common API endpoints found."
    → Retourne un message confirmant qu'aucun endpoint API commun n'a été trouvé.

58 def check_cookie_flags(domain: str) -> str:
    → Déclare la fonction qui vérifie les indicateurs de sécurité des cookies du domaine.

59     """Vérifie les flags de sécurité des cookies"""
    → Docstring décrivant en une ligne le rôle de la fonction.

60     try:
    → Ouvre un bloc try pour capturer les erreurs de la requête HTTP.

61         r = requests.get(f"https://{domain}", timeout=5)
    → Envoie une requête HTTP GET vers le domaine en HTTPS, avec un délai de 5 secondes.

62         cookies = r.headers.get("Set-Cookie", "")
    → Récupère l'en-tête Set-Cookie de la réponse, ou une chaîne vide si absent.

63         issues = []
    → Initialise une liste vide qui accumulera les problèmes détectés sur les cookies.

65         if cookies:
    → Vérifie si des cookies ont effectivement été renvoyés par le serveur.

66         if "Secure" not in cookies:
    → Vérifie si l'indicateur "Secure" est absent des cookies. L'attribut Secure d'un cookie garantit qu'il n'est envoyé que sur des connexions chiffrées HTTPS ; son absence expose le cookie à une interception en clair.

67         issues.append("Missing 'Secure'")
    → Ajoute un message signalant l'absence de l'indicateur "Secure" à la liste des problèmes.

68         if "HttpOnly" not in cookies:
    → Vérifie si l'indicateur "HttpOnly" est absent des cookies. L'attribut HttpOnly empêche un script JavaScript côté navigateur d'accéder au cookie, ce qui limite l'impact d'une éventuelle faille XSS (injection de script) sur le site.

```

```

69         issues.append("Missing 'HttpOnly'")
    → Ajoute un message signalant l'absence de l'indicateur "HttpOnly" à la liste des problèmes.

71         if issues:
    → Vérifie si au moins un problème a été détecté sur les cookies.

72         return " Cookie flags issue: " + ", ".join(issues)
    → Retourne la liste des problèmes détectés sur les cookies, séparés par des virgules.

73         return " Cookies have Secure and HttpOnly flags."
    → Retourne un message confirmant que les cookies possèdent les deux indicateurs de sécurité.

74     except Exception as e:
    → Capture toute exception survenue pendant la requête ou l'analyse des cookies.

75         return f"Cookie security check failed: {e}"
    → Retourne un message d'erreur incluant le détail de l'exception capturée.

77 def check_security_headers(domain: str) -> str:
    → Déclare la fonction qui vérifie la présence des en-têtes de sécurité HTTP du domaine.

78     """Vérifie les en-têtes de sécurité"""
    → Docstring décrivant en une ligne le rôle de la fonction.

79     try:
    → Ouvre un bloc try pour capturer les erreurs de la requête HTTP.

80         url = f"https://{domain}"
    → Construit l'URL HTTPS complète à partir du nom de domaine.

81         response = requests.get(url, timeout=5)
    → Envoie une requête HTTP GET vers l'URL avec un délai de 5 secondes.

82         headers = response.headers
    → Récupère les en-têtes HTTP de la réponse.

83         results = []
    → Initialise une liste vide qui accumulera les résultats de la vérification pour chaque en-tête.

84         required = ['Content-Security-Policy', 'X-Frame-Options', 'Strict-Transport-Security']
    → Définit la liste des en-têtes de sécurité requis à vérifier. Content-Security-Policy (CSP) restreint les sources de scripts/ressources qu'une page peut charger (protection contre les injections XSS) ; X-Frame-Options empêche le site d'être intégré dans une iframe sur un autre site (protection contre le clickjacking) ; Strict-Transport-Security (HSTS) force le navigateur à toujours utiliser HTTPS pour ce domaine.

86         for h in required:

87             if h in headers:
    → Vérifie si l'en-tête courant est présent dans les en-têtes de la réponse.

88                 results.append(f" {h} found")
    → Ajoute un message confirmant la présence de l'en-tête à la liste des résultats.

89             else:
    → Cas où l'en-tête courant est absent des en-têtes de la réponse.

90                 results.append(f" {h} missing")
    → Ajoute un message signalant l'absence de l'en-tête à la liste des résultats.

```

```

92         return "\\n".join(results)
    → Retourne l'ensemble des résultats de vérification, un par ligne.

93     except Exception as e:
    → Capture toute exception survenue pendant la requête ou l'analyse des en-têtes.

94         return f"Header check failed: {e}"
    → Retourne un message d'erreur incluant le détail de l'exception capturée.

96 def check_common_directories(domain: str) -> str:
    → Déclare la fonction qui recherche des répertoires communs potentiellement sensibles sur le domaine (même technique de "wordlist" que check_api_endpoints, appliquée cette fois à des chemins d'administration classiques).

97     """Vérifie les répertoires communs"""
    → Docstring décrivant en une ligne le rôle de la fonction.

98     url_base = f"https://{domain}"
    → Construit l'URL de base HTTPS à partir du nom de domaine.

99     wordlist = ["/admin", "/login", "/config", "/.git", "/backup"]
    → Définit la liste des chemins de répertoires courants à tester ; /.git exposé publiquement est particulièrement critique car il peut permettre de reconstituer tout le code source et l'historique d'un site.

100     found = []
    → Initialise une liste vide qui accumulera les répertoires trouvés.

102     for path in wordlist:
    → Parcourt chaque chemin de la liste de répertoires courants.

103         try:
    → Ouvre un bloc try pour capturer les erreurs de requête sur ce chemin précis.

104             full_url = url_base + path
    → Construit l'URL complète en concaténant l'URL de base et le chemin courant.

105             r = requests.get(full_url, timeout=3)
    → Envoie une requête HTTP GET vers l'URL construite, avec un délai de 3 secondes.

106             if r.status_code in [200, 301, 403]:
    → Vérifie si le code de statut retourné indique une ressource présente (200), redirigée (301) ou protégée (403).

107                 found.append(f" Found: {full_url} [{r.status_code}]")
    → Ajoute l'URL trouvée et son code de statut à la liste des répertoires trouvés.

108         except:
    → Capture silencieusement toute exception survenue pour ce chemin.

109             continue
    → Passe au chemin suivant de la boucle sans interrompre le traitement.

111     return "\\n".join(found) if found else "No common directories found."
    → Retourne la liste des répertoires trouvés, ou un message par défaut si aucun n'a été trouvé.

114 def find_js_urls(domain: str) -> str:
    → Déclare la fonction qui recherche les URLs de fichiers JavaScript référencés par la page d'accueil. Lister les fichiers JS d'un site est une étape courante de reconnaissance : le code JavaScript côté client peut révéler des endpoints d'API, des clés ou des logiques internes non censées être publiques.

```

115 """Trouve les fichiers JavaScript"""

→ Docstring décrivant en une ligne le rôle de la fonction.

116 try:

→ Ouvre un bloc try pour capturer les erreurs de la requête ou du parsing HTML.

117 url = f"https://{domain}"

→ Construit l'URL HTTPS complète à partir du nom de domaine.

118 r = requests.get(url, timeout=5)

→ Envoie une requête HTTP GET vers l'URL avec un délai de 5 secondes.

119 soup = BeautifulSoup(r.text, "html.parser")

→ Analyse le HTML de la réponse avec BeautifulSoup, en utilisant le parseur intégré à Python ("html.parser"), pour pouvoir ensuite chercher facilement des balises spécifiques dans le document.

120 js_urls = []

→ Initialise une liste vide qui accumulera les URLs de fichiers JavaScript trouvées.

122 for script in soup.find_all("script", src=True):

→ Parcourt toutes les balises <script> possédant un attribut src (donc chargeant un fichier externe) dans la page ; find_all est la méthode principale de BeautifulSoup pour rechercher des éléments HTML par balise et attributs.

123 src = script["src"]

→ Récupère la valeur de l'attribut src de la balise script courante.

124 if src.startswith("http"):

→ Vérifie si la valeur src est déjà une URL absolue commençant par http.

125 js_urls.append(src)

→ Ajoute directement l'URL absolue à la liste des URLs JavaScript trouvées.

126 else:

→ Cas où la valeur src est une URL relative.

127 js_urls.append(f"{url.rstrip('/')}/{src.lstrip('/')}")

→ Construit une URL absolue en combinant l'URL de base et le chemin relatif (en nettoyant les slashes superflus des deux côtés), puis l'ajoute à la liste.

129 return "\\n".join(js_urls) if js_urls else "No JS files found."

→ Retourne la liste des URLs JavaScript trouvées, ou un message par défaut si aucune n'a été trouvée.

130 except Exception as e:

→ Capture toute exception survenue pendant la requête ou l'analyse HTML.

131 return f"JS scan failed: {e}"

→ Retourne un message d'erreur incluant le détail de l'exception capturée.

133 def check_robots_txt(domain: str) -> str:

→ Déclare la fonction qui vérifie la présence et le contenu du fichier robots.txt. Ce fichier, censé lister les zones du site que les robots d'indexation ne doivent pas visiter, révèle parfois involontairement des chemins sensibles (zones d'administration, sections privées) aux attaquants qui savent où regarder.

134 """Vérifie le fichier robots.txt"""

→ Docstring décrivant en une ligne le rôle de la fonction.

135 try:

→ Ouvre un bloc try pour capturer les erreurs de la requête HTTP.

```

136         url = f"https://{domain}/robots.txt"
    → Construit l'URL du fichier robots.txt à partir du nom de domaine (chemin standardisé, toujours à la racine du site).

137         r = requests.get(url, timeout=3)
    → Envoie une requête HTTP GET vers l'URL du fichier robots.txt, avec un délai de 3 secondes.

138         if r.status_code == 200:
    → Vérifie si le fichier robots.txt a été trouvé avec succès.

139             return f" robots.txt found:\\n{r.text[:500]}"
    → Retourne un message de succès avec les 500 premiers caractères du contenu du fichier (limite pour éviter un rapport trop volumineux).

140         else:
    → Cas où le fichier robots.txt n'a pas été trouvé avec succès.

141             return f" robots.txt not found (status {r.status_code})"
    → Retourne un message signalant l'absence du fichier avec le code de statut obtenu.

142     except Exception as e:
    → Capture toute exception survenue pendant la requête HTTP.

143         return f"robots.txt check failed: {e}"
    → Retourne un message d'erreur incluant le détail de l'exception capturée.

145 def check_sitemap(domain: str) -> str:
    → Déclare la fonction qui vérifie la présence et le contenu du fichier sitemap.xml. Ce fichier, destiné aux moteurs de recherche, liste souvent l'intégralité des pages publiques d'un site, ce qui en fait une source pratique pour cartographier rapidement sa surface d'attaque.

146     """Vérifie le fichier sitemap.xml"""
    → Docstring décrivant en une ligne le rôle de la fonction.

147     try:
    → Ouvre un bloc try pour capturer les erreurs de la requête HTTP.

148         url = f"https://{domain}/sitemap.xml"
    → Construit l'URL du fichier sitemap.xml à partir du nom de domaine.

149         r = requests.get(url, timeout=3)
    → Envoie une requête HTTP GET vers l'URL du fichier sitemap.xml, avec un délai de 3 secondes.

150         if r.status_code == 200:
    → Vérifie si le fichier sitemap.xml a été trouvé avec succès.

151             return f" sitemap.xml found:\\n{r.text[:500]}"
    → Retourne un message de succès avec les 500 premiers caractères du contenu du fichier.

152         else:
    → Cas où le fichier sitemap.xml n'a pas été trouvé avec succès.

153             return f" sitemap.xml not found (status {r.status_code})"
    → Retourne un message signalant l'absence du fichier avec le code de statut obtenu.

154     except Exception as e:
    → Capture toute exception survenue pendant la requête HTTP.

155         return f"sitemap check failed: {e}"

```

→ Retourne un message d'erreur incluant le détail de l'exception capturée.

```
157 def check_env_exposure(domain: str) -> str:
```

→ Déclare la fonction qui vérifie si un fichier .env est exposé publiquement sur le domaine. Un fichier .env contient typiquement des secrets (clés d'API, mots de passe de base de données) ; s'il est accessible directement via une URL, c'est une faille de configuration critique et fréquente.

```
158     """Vérifie l'exposition du fichier .env"""
```

→ Docstring décrivant en une ligne le rôle de la fonction.

```
159     try:
```

→ Ouvre un bloc try pour capturer les erreurs de la requête HTTP.

```
160         url = f"https://{domain}/.env"
```

→ Construit l'URL du fichier .env à partir du nom de domaine.

```
161         r = requests.get(url, timeout=3)
```

→ Envoie une requête HTTP GET vers l'URL du fichier .env, avec un délai de 3 secondes.

```
162         if "APP_KEY" in r.text or "DB_PASSWORD" in r.text:
```

→ Vérifie si le contenu de la réponse contient des indices de secrets applicatifs exposés (noms de variables typiques des fichiers .env de frameworks comme Laravel).

```
163             return f"Exposed .env file found:\\n{r.text[:300]}"
```

→ Retourne un avertissement avec les 300 premiers caractères du fichier .env exposé.

```
164         elif r.status_code == 200:
```

→ Vérifie si le fichier existe (code 200) mais sans qu'aucune des chaînes recherchées à la ligne précédente n'ait été détectée.

```
165             return ".env file exists but no sensitive data detected."
```

→ Retourne un avertissement indiquant que le fichier existe sans donnée sensible détectée (l'exposition du fichier reste en soi un problème de configuration, même sans secret repéré par cette vérification simplifiée).

```
166         else:
```

→ Cas où le fichier .env n'est pas accessible.

```
167             return ".env file not accessible."
```

→ Retourne un message confirmant que le fichier .env n'est pas accessible.

```
168     except Exception as e:
```

→ Capture toute exception survenue pendant la requête HTTP.

```
169         return f".env check failed: {e}"
```

→ Retourne un message d'erreur incluant le détail de l'exception capturée.

```
171 def check_server_header(domain: str) -> str:
```

→ Déclare la fonction qui lit l'en-tête Server renvoyé par le serveur du domaine. Cet en-tête révèle souvent la technologie serveur utilisée (nginx, Apache, IIS...) et parfois sa version précise, une information utile pour rechercher des vulnérabilités connues associées à cette version.

```
172     """Vérifie l'en-tête Server"""
```

→ Docstring décrivant en une ligne le rôle de la fonction.

```
173     try:
```

→ Ouvre un bloc try pour capturer les erreurs de la requête HTTP.

```
174         url = f"https://{domain}"
```

→ Construit l'URL HTTPS complète à partir du nom de domaine.


```

175         r = requests.get(url, timeout=5)
    → Envoie une requête HTTP GET vers l'URL avec un délai de 5 secondes.

176         server = r.headers.get("Server", "Not found")
    → Récupère la valeur de l'en-tête Server, ou "Not found" si absent.

177         return f" Server header: {server}"
    → Retourne un message affichant la valeur de l'en-tête Server détectée.

178     except Exception as e:
    → Capture toute exception survenue pendant la requête HTTP.

179         return f"Server header check failed: {e}"
    → Retourne un message d'erreur incluant le détail de l'exception capturée.

181 # Fonction principale pour exécuter tous les tests
    → Commentaire annonçant la fonction principale qui orchestre tous les tests de sécurité.

182 def run_security_scan(domain: str):
    → Déclare la fonction qui exécute l'ensemble des vérifications de sécurité sur un domaine, séquentiellement. Cette fonction
    constitue un mode d'usage alternatif du fichier tools.py : un scan complet en ligne de commande, indépendant du workflow piloté
    par agent (OffCyberTest.py), qui appelle lui les fonctions individuellement via l'agent testeur.

183     """Exécute un scan de sécurité complet"""
    → Docstring décrivant en une ligne le rôle de la fonction.

184     print(f"=== Security Scan for {domain} ===\n")
    → Affiche un en-tête annonçant le début du scan de sécurité pour ce domaine.

186     checks = [
    → Commence la construction d'une liste de tuples associant un nom de test à sa fonction ; cette structure "table de dispatch"
    permet de boucler simplement sur tous les tests sans dupliquer le code d'affichage pour chacun.

187         ("HTTP to HTTPS Redirect", check_http_redirect),
    → Ajoute à la liste le test de redirection HTTP vers HTTPS avec sa fonction associée.

188         ("SSL Certificate", check_ssl_certificate),
    → Ajoute à la liste le test du certificat SSL avec sa fonction associée.

189         ("Security Headers", check_security_headers),
    → Ajoute à la liste le test des en-têtes de sécurité avec sa fonction associée.

190         ("Cookie Security", check_cookie_flags),
    → Ajoute à la liste le test de sécurité des cookies avec sa fonction associée.

191         ("API Endpoints", check_api_endpoints),
    → Ajoute à la liste le test des endpoints API avec sa fonction associée.

192         ("Common Directories", check_common_directories),
    → Ajoute à la liste le test des répertoires communs avec sa fonction associée.

193         ("JavaScript Files", find_js_urls),
    → Ajoute à la liste le test de recherche des fichiers JavaScript avec sa fonction associée.

194         ("robots.txt", check_robots_txt),
    → Ajoute à la liste le test du fichier robots.txt avec sa fonction associée.

195         ("sitemap.xml", check_sitemap),

```

→ Ajoute à la liste le test du fichier sitemap.xml avec sa fonction associée.

```
196         ("Environment File", check_env_exposure),
```

→ Ajoute à la liste le test d'exposition du fichier .env avec sa fonction associée.

```
197         ("Server Header", check_server_header),
```

→ Ajoute à la liste le test de l'en-tête Server avec sa fonction associée.

```
198         ("GitHub Mentions", check_github_mentions)
```

→ Ajoute à la liste le test des mentions GitHub avec sa fonction associée.

```
199     ]
```

→ Ferme la liste des tuples nom de test / fonction.

```
201     for check_name, check_function in checks:
```

→ Parcourt chaque paire nom de test / fonction de la liste checks, en déballant (unpacking) le tuple directement dans la boucle.

```
202         print(f"--- {check_name} ---")
```

→ Affiche un séparateur avec le nom du test en cours d'exécution.

```
203         result = check_function(domain)
```

→ Exécute la fonction de test courante sur le domaine (appel dynamique via la référence de fonction stockée dans le tuple) et stocke son résultat.

```
204         print(result)
```

→ Affiche le résultat retourné par la fonction de test.

```
205         print()
```

→ Affiche une ligne vide pour séparer visuellement les résultats des différents tests.

```
207 # Exemple d'utilisation
```

→ Commentaire annonçant le point d'entrée d'exemple du script.

```
208 if __name__ == "__main__":
```

→ Vérifie que le script est exécuté directement et non importé comme module.

```
209     domain = input("Enter domain to scan: ")
```

→ Demande interactivement à l'utilisateur de saisir le domaine à scanner.

```
210     run_security_scan(domain)
```

→ Lance le scan de sécurité complet sur le domaine saisi.

Projet 23 — 23-Whatsapp-lette

Bot WhatsApp relie a un agent Letta

Objectif

Bot conversationnel joignable sur WhatsApp qui joue le role d'un compagnon de conversation dote d'une memoire a long terme grace a une integration avec un agent Letta : contrairement a un chatbot classique, il retient des informations sur l'utilisateur d'une conversation a l'autre, meme apres un redemarrage.

Code source, ligne par ligne

23-Whatsapp-lette/CompanionAgent/main.py

Serveur FastAPI qui expose les routes webhook WhatsApp (vérification et réception de messages) et des endpoints de test pour l'envoi de messages via l'API Cloud WhatsApp de Meta.

104 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard os, qui donne accès au système d'exploitation. Ici il sert surtout à lire des variables d'environnement (os.getenv) : c'est la manière standard de fournir des secrets (clés API, tokens) à un programme sans jamais les écrire en dur dans le code source, ce qui évite de les exposer si le code est partagé ou versionné.

2 import json

→ Importe le module standard json, qui sait convertir entre le format texte JSON (utilisé par toutes les API web, dont celle de WhatsApp) et les structures Python (dict, list). Il est utilisé ici pour détecter les erreurs de décodage quand le corps d'une requête n'est pas un JSON valide.

3 import uvicorn

→ Importe uvicorn, un serveur ASGI (Asynchronous Server Gateway Interface). FastAPI décrit seulement comment répondre aux requêtes, mais il faut un vrai serveur pour écouter un port réseau et transmettre les requêtes HTTP entrantes à l'application : uvicorn est le serveur standard et très performant utilisé avec FastAPI, capable de gérer des routes asynchrones (comme les fonctions "async def" de ce fichier).

4 import logging

→ Importe le module standard de journalisation (logging). Contrairement à de simples print(), il permet de catégoriser les messages (info, warning, error) et est la pratique recommandée pour un serveur en production, afin de pouvoir diagnostiquer plus tard pourquoi une vérification de webhook a échoué par exemple.

5 import requests

→ Importe la bibliothèque tierce requests, le client HTTP le plus utilisé en Python. Elle sert ici à appeler l'API Graph de Facebook/Meta (l'API REST derrière WhatsApp Business) pour envoyer des messages sortants.

6 from threading import Thread

→ Importe la classe Thread du module standard threading, qui permet d'exécuter une fonction en parallèle, dans un fil d'exécution séparé, sans bloquer le programme principal. C'est utilisé plus bas pour lancer le traitement d'un message WhatsApp en arrière-plan : Meta exige une réponse HTTP rapide au webhook (sinon il considère l'appel en échec et le renvoie), donc on ne peut pas laisser l'appel à l'agent IA (potentiellement long) bloquer la réponse.

7 from dotenv import load_dotenv

→ Importe la fonction load_dotenv de la bibliothèque python-dotenv. Elle permet de charger des paires clé=valeur depuis un fichier .env local vers les variables d'environnement du processus. C'est la façon standard de garder les secrets (tokens, clés API) hors du code source et hors de tout dépôt Git, tout en les rendant accessibles via os.getenv.

```
8 from fastapi import FastAPI, Request, HTTPException
```

→ Importe trois éléments du framework FastAPI. FastAPI est un framework web Python moderne, conçu autour d'async/await, qui sert ici à construire l'API HTTP recevant les webhooks WhatsApp. Request donne accès à la requête HTTP entrante (paramètres, corps). HTTPException est la façon standard de FastAPI de répondre avec un code d'erreur HTTP (403, 400, etc.) accompagné d'un message.

```
9 from fastapi.responses import PlainTextResponse, JSONResponse
```

→ Importe deux classes de réponse. JSONResponse renvoie un corps JSON structuré, le format habituel d'une API. PlainTextResponse renvoie du texte brut : elle est nécessaire spécifiquement pour la vérification du webhook WhatsApp, car Meta exige que le serveur renvoie littéralement la valeur "challenge" en texte brut, pas encapsulée dans du JSON.

```
10 from whatsapp_utils import process_whatsapp_message, is_valid_whatsapp_message
```

→ Importe deux fonctions utilitaires définies dans un module local séparé (whatsapp_utils.py). Séparer la logique métier (validation, traitement des messages) du fichier qui définit les routes HTTP est une pratique d'architecture courante qui garde chaque fichier focalisé sur une responsabilité.

```
12 load_dotenv()
```

→ Exécute le chargement des variables du fichier .env au démarrage du programme, avant toute lecture via os.getenv plus bas.

```
14 VERSION = os.getenv('VERSION')
```

→ Lit la version de l'API Graph de Meta à utiliser dans les URL (par exemple "v18.0"). Meta versionne son API et exige d'indiquer explicitement la version dans chaque appel, ce qui permet de contrôler quand migrer vers une nouvelle version sans rupture surprise.

```
15 PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')
```

→ Lit l'identifiant technique du numéro de téléphone WhatsApp Business enregistré dans la console développeur Meta. Ce n'est pas le numéro lui-même mais un ID interne requis dans l'URL de chaque appel pour indiquer depuis quel numéro l'envoi doit se faire.

```
16 WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')
```

→ Lit le jeton d'accès (Bearer token) qui authentifie les appels à l'API WhatsApp. Sans ce jeton valide, l'API refuse toute requête d'envoi de message.

```
17 RECIPIENT_PHONE_NUMBER = os.getenv('RECIPIENT_PHONE_NUMBER')
```

→ Lit un numéro de téléphone destinataire par défaut, utilisé par les routes de test /test et /send_message.

```
18 VERIFY_TOKEN = os.getenv('VERIFY_TOKEN')
```

→ Lit un jeton de vérification distinct du token d'accès : c'est un secret partagé uniquement utilisé une fois, lors de la configuration du webhook dans la console Meta, pour prouver que le serveur qui répond est bien le vôtre (voir la fonction verify plus bas).

```
20 app = FastAPI()
```

→ Crée l'instance principale de l'application FastAPI, l'objet central auquel on accroche ensuite toutes les routes (via les décorateurs @app.get, @app.post).

```
22 @app.get("/")
```

→ Décorateur FastAPI qui associe la fonction définie juste en dessous à la méthode HTTP GET sur le chemin "/". C'est le mécanisme central de FastAPI pour définir des routes de façon déclarative.

```
23 def read_root():
```

→ Définit la fonction gérant les requêtes GET sur la racine, utilisée ici comme simple route de contrôle (« l'API est-elle en ligne ? »).

```
24     return {"message": "Welcome to the portal!"}
```

→ Retourne un dictionnaire Python ; FastAPI le sérialise automatiquement en JSON dans la réponse HTTP, sans code supplémentaire à écrire.

```
26 # Required webhook verification for WhatsApp
```

→ Commentaire indiquant que la fonction suivante gère la vérification du webhook.

27 `async def verify(request: Request):`

→ Définit une fonction asynchrone (async def) qui vérifie la requête de webhook WhatsApp. Le mot-clé async permet à FastAPI de traiter cette requête sans bloquer le serveur pendant l'attente d'opérations d'entrée/sortie, ce qui est important pour un serveur qui doit rester réactif face à de nombreux appels de Meta.

28 `params = dict(request.query_params)`

→ Convertit les paramètres de la requête (la partie après le "?" dans l'URL) en dictionnaire Python standard pour pouvoir y accéder facilement avec .get().

29 `mode = params.get("hub.mode")`

→ Récupère le paramètre hub.mode envoyé par WhatsApp. C'est une convention imposée par Meta dans le protocole de vérification de webhook (« handshake ») : Meta appelle l'URL configurée avec ces trois paramètres hub.* pour valider qu'elle appartient bien au développeur avant de commencer à y envoyer des événements réels.

30 `token = params.get("hub.verify_token")`

→ Récupère le jeton de vérification envoyé par WhatsApp, à comparer au VERIFY_TOKEN secret défini côté serveur.

31 `challenge = params.get("hub.challenge")`

→ Récupère la valeur "challenge", une chaîne aléatoire que le serveur doit renvoyer telle quelle pour prouver qu'il contrôle bien cette URL.

33 `if mode and token:`

→ Vérifie que les paramètres mode et token sont bien présents avant d'aller plus loin.

34 `if mode == "subscribe" and token == VERIFY_TOKEN:`

→ Vérifie que le mode est "subscribe" (le seul mode utilisé par WhatsApp pour l'abonnement au webhook) et que le jeton reçu correspond exactement au secret VERIFY_TOKEN configuré côté serveur.

35 `logging.info("WEBHOOK_VERIFIED")`

→ Écrit un message d'information confirmant la vérification du webhook, utile pour suivre dans les logs que la configuration côté Meta s'est bien déroulée.

36 `return PlainTextResponse(content=challenge, status_code=200)`

→ Renvoie la valeur challenge en texte brut avec un code 200 : c'est exactement ce que Meta attend pour valider l'URL du webhook et commencer à y envoyer des notifications.

37 `else:`

→ Cas où le mode ou le jeton ne correspondent pas aux valeurs attendues (tentative de vérification invalide, par exemple avec un mauvais secret).

38 `logging.warning("VERIFICATION_FAILED")`

→ Écrit un message d'avertissement signalant l'échec de la vérification.

39 `raise HTTPException(status_code=403, detail="Verification failed")`

→ Lève une exception HTTP 403 (Forbidden) indiquant l'échec de la vérification ; FastAPI convertit automatiquement cette exception en réponse HTTP avec ce code de statut.

40 `else:`

→ Cas où le paramètre mode ou token est totalement absent de la requête (requête malformée, pas un vrai appel de vérification Meta).

41 `logging.warning("MISSING_PARAMETER")`

→ Écrit un message d'avertissement signalant un paramètre manquant.

42 `raise HTTPException(status_code=400, detail="Missing parameters")`

→ Lève une exception HTTP 400 (Bad Request) indiquant que des paramètres obligatoires sont manquants.

44 `async def handle_message(request: Request):`

→ Définit une fonction asynchrone traitant les messages entrants du webhook, distincte de `verify()` : `verify()` ne gère que la vérification initiale unique, `handle_message()` gère chaque événement réel envoyé ensuite par WhatsApp (messages, statuts...).

45 `try:`

→ Démarre un bloc pour intercepter les erreurs de décodage JSON, au cas où Meta enverrait un corps de requête corrompu ou vide.

46 `body = await request.json()`

→ Lit et décode le corps de la requête en JSON de façon asynchrone (`await`) : FastAPI/Starlette lit le flux réseau sans bloquer le reste du serveur pendant cette opération d'entrée/sortie.

47 `except json.JSONDecodeError:`

→ Intercepte l'erreur spécifique levée si le corps de la requête n'est pas un JSON syntaxiquement valide.

48 `logging.error("Failed to decode JSON")`

→ Écrit un message d'erreur signalant l'échec du décodage JSON.

49 `return JSONResponse(status_code=400, content={"status": "error", "message": "Invalid JSON provided"})`

→ Renvoie une réponse JSON d'erreur 400 en cas de JSON invalide, sans planter le serveur.

51 `if (`

→ Débute une condition composée sur plusieurs lignes vérifiant le contenu du corps.

52 `body.get("entry", [{}])[0]`

→ Récupère le premier élément de la liste "entry" (ou un dictionnaire vide par défaut si la clé n'existe pas). L'usage systématique de `.get(clé, valeur_par_défaut)` au lieu de `body["entry"]` est une technique défensive : elle évite qu'une clé manquante fasse planter le serveur avec une `KeyError` sur un payload WhatsApp inattendu.

53 `.get("changes", [{}])[0]`

→ Récupère le premier élément de la liste "changes" imbriquée dans cet entry, avec le même filet de sécurité.

54 `.get("value", {})`

→ Récupère le dictionnaire "value" contenu dans changes, qui porte les données réelles de l'événement.

55 `.get("statuses")`

→ Récupère le champ "statuses" de valeur s'il existe. WhatsApp envoie sur la même URL de webhook plusieurs types d'événements : les vrais messages entrants, mais aussi des mises à jour de statut (accusé d'envoi, de remise, de lecture) ; ce champ permet de distinguer les deux.

56 `):`

→ Ferme la condition composée débutée à la ligne 51 : si le chemin complet mène à une valeur non vide, c'est qu'il s'agit d'une mise à jour de statut, pas d'un vrai message.

57 `logging.info("Received a WhatsApp status update.")`

→ Écrit un message d'information signalant la réception d'une mise à jour de statut.

58 `return JSONResponse(content={"status": "ok"})`

→ Renvoie une réponse JSON ok sans traiter davantage la requête, car il n'y a pas de message utilisateur à faire répondre par l'agent.

60 `if is_valid_whatsapp_message(body):`

→ Vérifie via la fonction utilitaire importée que le corps correspond bien à un message WhatsApp entrant valide et exploitable.

61 `try:`

→ Démarre un bloc pour intercepter une erreur au démarrage du thread d'arrière-plan.

62 `# Run message processing in a background thread`

→ Commentaire précisant que le traitement du message se fait dans un thread séparé.

```
63 Thread(target=process_whatsapp_message, args=(body,)).start()
```

→ Lance un nouveau thread qui exécute `process_whatsapp_message` avec le corps reçu, puis continue immédiatement sans attendre sa fin. C'est le point clé de robustesse du webhook : appeler l'agent conversationnel peut prendre plusieurs secondes, alors que Meta attend une réponse HTTP rapide (sous quelques secondes) sous peine de considérer l'appel en échec et de le réémettre en double ; déléguer le traitement à un thread permet de répondre "ok" tout de suite pendant que le vrai travail se poursuit en tâche de fond.

```
64 except Exception as e:
```

→ Intercepte toute exception survenue lors du démarrage du thread (cas rare, mais évite de faire planter la route).

```
65 logging.error(f"Error starting background message thread: {e}")
```

→ Écrit un message d'erreur détaillant l'exception rencontrée.

```
66 return JsonResponse(content={"status": "ok"})
```

→ Renvoie tout de même une réponse JSON ok au serveur WhatsApp, pour accuser réception de l'événement même si un souci est survenu au lancement du thread.

```
67 else:
```

→ Cas où le corps ne correspond pas à un message WhatsApp entrant valide.

```
68 return JsonResponse(status_code=404, content={"status": "error", "message": "Not a WhatsApp API event"})
```

→ Renvoie une réponse JSON d'erreur 404 indiquant un événement non reconnu.

```
72 @app.get("/test")
```

→ Déclare une route GET de test sur le chemin `/test`, utile pendant le développement pour vérifier manuellement que l'intégration avec l'API WhatsApp fonctionne, sans passer par le webhook.

```
73 def test():
```

→ Définit la fonction gérant la route de test d'envoi de message.

```
74 url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
```

→ Construit l'URL de l'API Graph de Facebook/Meta pour l'envoi de messages, en y insérant dynamiquement la version d'API et l'identifiant du numéro expéditeur.

```
75 headers = {
```

→ Débute la construction du dictionnaire des en-têtes HTTP à envoyer avec la requête.

```
76     'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
```

→ Ajoute l'en-tête d'autorisation avec le jeton d'accès WhatsApp, au format standard "Bearer <token>" utilisé par la plupart des API REST modernes pour authentifier l'appelant.

```
77     'Content-Type': 'application/json'
```

→ Ajoute l'en-tête indiquant que le contenu envoyé est du JSON, pour que le serveur de Meta sache comment interpréter le corps de la requête.

```
78 }
```

→ Ferme le dictionnaire des en-têtes HTTP.

```
79 data = {
```

→ Débute la construction du corps de la requête à envoyer, qui suit le format attendu par l'API WhatsApp Cloud pour un message.

```
80     'messaging_product': 'whatsapp',
```

→ Précise que le produit de messagerie utilisé est whatsapp (champ requis identique pour tous les appels de cette API).

```
81     'to': RECIPIENT_PHONE_NUMBER,
```

→ Indique le numéro de téléphone destinataire du message.

```
82         'type': 'template',
```

→ Précise que le message envoyé est de type modèle (template).

```
83         'template': {
```

→ Débute la définition du modèle de message à envoyer.

```
84             'name': 'hello_world',
```

→ Indique que le modèle utilisé s'appelle "hello_world", le modèle d'exemple fourni par défaut par Meta. Les "templates" WhatsApp sont des messages pré-approuvés par Meta (texte figé, éventuellement avec variables) : ils sont nécessaires pour qu'une entreprise puisse initier une conversation vers un utilisateur qui ne lui a pas encore écrit, contrairement à un message texte libre.

```
85             'language': {
```

→ Débute la définition de la langue du modèle.

```
86                 'code': 'en_US'
```

→ Précise le code de langue anglais américain pour le modèle, qui doit correspondre à une des langues dans lesquelles le template a été approuvé par Meta.

```
87             }
```

→ Ferme le dictionnaire de la langue.

```
88         }
```

→ Ferme le dictionnaire du modèle.

```
89     }
```

→ Ferme le dictionnaire data.

```
90     response = requests.post(url, headers=headers, json=data)
```

→ Envoie une requête POST à l'API WhatsApp avec les en-têtes et les données ; le paramètre json=data indique à requests de sérialiser automatiquement le dictionnaire Python en JSON et de fixer le bon en-tête Content-Type.

```
91     return {"message": "Test successful!", "response": response.json()}
```

→ Retourne un message de succès accompagné de la réponse JSON brute renvoyée par l'API Meta, utile pour déboguer directement depuis le navigateur.

```
93 @app.get("/send_message")
```

→ Déclare une route GET pour envoyer un message texte de démonstration, cette fois en texte libre plutôt qu'en template.

```
94 def send_message():
```

→ Définit la fonction gérant la route d'envoi de message texte.

```
95     url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
```

→ Construit l'URL de l'API Graph de Facebook pour l'envoi de messages.

```
96     headers = {
```

→ Débute la construction du dictionnaire des en-têtes HTTP.

```
97         'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
```

→ Ajoute l'en-tête d'autorisation avec le jeton d'accès WhatsApp.

```
98         'Content-Type': 'application/json'
```

→ Ajoute l'en-tête indiquant que le contenu envoyé est du JSON.

```
99     }
```

→ Ferme le dictionnaire des en-têtes HTTP.

100 `data = {`
→ Début la construction du corps de la requête à envoyer.

101 `'messaging_product': 'whatsapp',`
→ Précise que le produit de messagerie utilisé est whatsapp.

102 `'to': RECIPIENT_PHONE_NUMBER,`
→ Indique le numéro de téléphone destinataire du message.

103 `'type': 'text',`
→ Précise que le message envoyé est de type texte simple (par opposition au type "template" utilisé dans la route précédente).

104 `'text': {`
→ Début la définition du contenu textuel du message.

105 `'body': 'Hello, how are you?'`
→ Définit le texte du message envoyé au destinataire.

106 `}`
→ Ferme le dictionnaire text.

107 `}`
→ Ferme le dictionnaire data.

108 `response = requests.post(url, headers=headers, json=data)`
→ Envoie une requête POST à l'API WhatsApp avec les en-têtes et les données.

109 `return {"message": "Message sent successfully!", "response": response.json()}`
→ Retourne un message de succès accompagné de la réponse JSON de l'API.

111 `@app.get("/webhook")`
→ Déclare une route GET sur /webhook, utilisée une seule fois par Meta lors de la configuration initiale pour effectuer la vérification (handshake) du webhook.

112 `async def webhook_get(request: Request):`
→ Définit la fonction asynchrone gérant la vérification GET du webhook.

113 `return await verify(request)`
→ Délègue le traitement à la fonction `verify` définie plus haut et retourne son résultat ; séparer `verify()` en fonction indépendante permet de garder la logique métier isolée de la déclaration de route.

115 `@app.post("/webhook")`
→ Déclare une route POST sur /webhook, celle que Meta appellera en continu pour transmettre chaque événement réel (messages, statuts) une fois le webhook validé.

116 `async def webhook_post(request: Request):`
→ Définit la fonction asynchrone gérant les notifications POST du webhook.

117 `return await handle_message(request)`
→ Délègue le traitement à la fonction `handle_message` et retourne son résultat.

119 `if __name__ == "__main__":`
→ Vérifie que le script est exécuté directement (par exemple via "python main.py") et non importé comme module par un autre fichier ; c'est la convention Python standard pour distinguer un script exécutable d'une bibliothèque.

120 `uvicorn.run(app, host="0.0.0.0", port=8000)`
→ Démarre le serveur uvicorn qui fait tourner l'application FastAPI. `host="0.0.0.0"` signifie que le serveur écoute sur toutes les interfaces réseau de la machine (et pas seulement en local), ce qui est nécessaire pour qu'il soit accessible depuis l'extérieur (par exemple via un tunnel comme ngrok), condition indispensable pour que les serveurs de Meta puissent atteindre ce webhook.

23-Whatsapp-lette/CompanionAgent/agent_service.py

Module qui envoie les messages (texte ou image) de l'utilisateur à l'agent Letta et récupère sa réponse.

51 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard os pour lire les variables d'environnement (clé API, identifiant d'agent).

2 import base64

→ Importe le module standard base64, qui encode des données binaires (comme une image) en une chaîne de texte composée uniquement de caractères ASCII. C'est nécessaire ici car le format JSON, utilisé pour envoyer le message à l'API de l'agent, ne peut transporter que du texte, pas des octets binaires bruts : encoder l'image en base64 est la façon standard de faire transiter un fichier binaire dans un payload JSON.

3 from letta_client import Letta

→ Importe la classe client de Letta. Letta est un framework/plateforme pour construire des agents conversationnels avec état persistant (« stateful agents ») : contrairement à un simple appel à un modèle de langage sans mémoire, un agent Letta conserve son historique de conversation et sa mémoire à long terme côté serveur entre les appels, ce qui lui permet de se souvenir du contexte d'un utilisateur d'une session à l'autre.

4 from dotenv import load_dotenv

→ Importe la fonction permettant de charger les variables depuis un fichier .env, pour garder les secrets hors du code source.

6 load_dotenv()

→ Charge les variables d'environnement définies dans le fichier .env.

9 LETTA_TOKEN = os.getenv('LETTA_API_KEY')

→ Lit la clé API Letta depuis les variables d'environnement, nécessaire pour authentifier les appels vers la plateforme Letta.

10 AGENT_ID = os.getenv('AGENT_ID')

→ Lit l'identifiant de l'agent Letta déjà créé sur la plateforme, auquel les messages doivent être envoyés. Un agent Letta est créé une fois (souvent via l'interface ou un template) puis référencé par cet identifiant à chaque appel.

12 client = Letta(

→ Débute la création du client Letta, l'objet Python qui encapsule les appels HTTP vers l'API Letta et sert de point d'entrée pour dialoguer avec l'agent.

13 token=LETTA_TOKEN,

→ Passe le jeton d'authentification au client Letta, pour que chaque appel soit reconnu comme provenant du compte autorisé.

14 # timeout=15,

→ Ligne commentée qui aurait défini un délai d'expiration de 15 secondes pour les appels réseau, laissée en place comme réglage optionnel désactivé.

15)

→ Ferme l'appel de construction du client Letta.

17 # create agent

→ Commentaire indiquant que le bloc suivant concernait la création d'un agent.

18 # response = client.templates.agents.create(

→ Ligne commentée d'appel désactivé qui aurait créé un nouvel agent à partir d'un template Letta prédéfini (au lieu de réutiliser un agent existant via AGENT_ID) ; laissée en commentaire comme trace de la méthode de création initiale de l'agent.

19 # project="default-project",

→ Ligne commentée précisant le projet Letta dans lequel l'agent aurait été créé (Letta organise les agents par projet).

20 # template_version="Companion:latest",

→ Ligne commentée précisant que l'agent aurait été instancié depuis le template "Companion" en version la plus récente : un template Letta encapsule une configuration d'agent réutilisable (personnalité, mémoire initiale, outils).

21 #)

→ Ligne commentée fermant l'appel de création d'agent désactivé.

23 # message_agent

→ Commentaire indiquant que la fonction suivante envoie un message à l'agent.

24 def get_response(message):

→ Définit la fonction qui envoie un message (texte ou image) à l'agent Letta et retourne sa réponse ; c'est le point d'entrée principal utilisé par le reste de l'application pour dialoguer avec l'IA.

25 message = message

→ Réaffecte le paramètre message à lui-même, ligne sans effet réel (probable reste de refactorisation).

26 print('-----Agent triggered-----')

→ Affiche dans la console un séparateur signalant le déclenchement de l'agent, utile pour suivre l'activité pendant le développement.

27 if isinstance(message, dict):

→ Vérifie si le message reçu est un dictionnaire Python, ce qui, par convention dans cette fonction, indique qu'il s'agit d'une image accompagnée d'une légende (plutôt qu'une simple chaîne de texte).

28 with open(message.get("image_path"), "rb") as image_file:

→ Ouvre en lecture binaire ("rb") le fichier image dont le chemin est fourni dans le message ; le mode binaire est indispensable pour lire correctement les octets d'un fichier image sans corruption.

29 encoded_string = base64.b64encode(image_file.read()).decode('utf-8')

→ Lit le contenu binaire du fichier image, l'encode en base64 (octets vers texte), puis décode ce résultat en chaîne de caractères utf-8 exploitable directement dans un JSON.

30 message = [

→ Reconstitue la variable message sous forme de liste, au format de « contenu multimodal » attendu par l'API de l'agent : une liste de blocs, chacun typé (image, texte...), ce qui permet à un même message d'associer plusieurs types de contenu.

31 {

→ Débute le premier élément de la liste représentant l'image.

32 "type": "image",

→ Indique que cet élément du message est une image.

33 "source": {

→ Débute la description de la source de l'image encodée.

34 "type": "base64",

→ Précise que la source de l'image est encodée en base64, plutôt que fournie par exemple sous forme d'URL.

35 "media_type": "image/jpeg",

→ Précise le type MIME de l'image comme étant jpeg, information nécessaire pour que le modèle sache comment décoder les octets.

36 "data": encoded_string,

→ Fournit les données binaires de l'image encodées en base64.

37 },

→ Ferme le dictionnaire source de l'image.

```

38         },
    → Ferme le premier élément de la liste représentant l'image.

39     {
    → Débute le second élément de la liste représentant le texte.

40         "type": "text",
    → Indique que cet élément du message est du texte.

41         "text": message.get("caption")
    → Récupère la légende de l'image comme contenu textuel associé, envoyé au même agent en même temps que l'image pour lui donner du contexte.

42     }
    → Ferme le dictionnaire représentant l'élément texte.

43 ]
    → Ferme la liste message contenant l'image et le texte.

44     try:
    → Démarre un bloc pour intercepter les erreurs lors de l'appel à l'agent (réseau, quota, mauvaise configuration, etc.), afin que l'application n'arrête pas de fonctionner si un appel échoue ponctuellement.

45         answer = client.agents.messages.create(
    → Envoie le message à l'agent Letta via le client et récupère sa réponse. C'est l'appel réseau central du fichier : il transmet le contenu (texte simple ou liste multimodale) à l'agent identifié par AGENT_ID et attend sa réponse générée.

46             agent_id=AGENT_ID,
    → Précise l'identifiant de l'agent Letta auquel le message est destiné.

47             messages=[{"role": "user", "content": message}],
    → Construit la liste des messages envoyés, avec le rôle "user" (convention standard des API de chat pour indiquer que ce message vient de l'utilisateur humain, par opposition à "assistant" ou "system") et le contenu préparé plus haut.

48         )
    → Ferme l'appel de création du message à l'agent.

49         for msg in answer.messages:
    → Parcourt la liste des messages retournés par l'agent (un agent Letta peut renvoyer plusieurs messages internes, par exemple des étapes de raisonnement, avant le message final destiné à l'utilisateur).

50             if hasattr(msg, "content") and msg.content:
    → Vérifie que le message possède un attribut content non vide, pour ne garder que les messages porteurs d'un vrai contenu exploitable.

51                 return msg.content
    → Retourne le contenu du premier message non vide trouvé, considéré comme la réponse utile de l'agent.

52         return "Agent responded with no content."
    → Retourne un message par défaut si aucun contenu n'a été trouvé dans la réponse de l'agent, pour éviter de renvoyer une valeur vide ou None à l'appelant.

53     except Exception as e:
    → Intercepte toute exception survenue pendant l'appel à l'agent.

54         print("ERROR:", e)
    → Affiche l'erreur rencontrée dans la console, utile pour le diagnostic.

```

```
55         return "Sorry, I ran into an issue. Please try again! "
```

→ Retourne un message d'erreur convivial à destination de l'utilisateur final plutôt que de laisser l'exception remonter et casser le flux WhatsApp.

```
58 if __name__ == '__main__':
```

→ Vérifie que le script est exécuté directement et non importé, pour permettre un test rapide en ligne de commande.

```
59     print(get_response('hello'))
```

→ Appelle la fonction `get_response` avec le mot "hello" et affiche le résultat, un test manuel simple de la connexion à l'agent.

23-Whatsapp-lette/CompanionAgent/whatsapp_utils.py

Fonctions utilitaires de validation, d'envoi et de traitement des messages WhatsApp entrants, y compris les images.

80 lignes de code annotées ci-dessous.

```
1 import requests
```

→ Importe la bibliothèque `requests`, le client HTTP utilisé pour appeler l'API WhatsApp Cloud (envoi de messages, récupération d'images).

```
2 import json
```

→ Importe le module standard `json` pour manipuler des données JSON.

```
3 import logging
```

→ Importe le module standard de journalisation pour enregistrer des messages de log en cas d'erreur réseau.

```
4 import re
```

→ Importe le module standard des expressions régulières (`re`), disponible pour d'éventuels traitements de texte, bien qu'il ne soit pas activement utilisé plus loin dans ce fichier.

```
5 from fastapi.responses import JSONResponse
```

→ Importe la classe de réponse JSON du framework FastAPI, réutilisée ici pour renvoyer des erreurs structurées en cas d'échec d'envoi, de la même façon que dans `main.py`.

```
6 from dotenv import load_dotenv
```

→ Importe la fonction permettant de charger les variables depuis un fichier `.env`.

```
7 from agent_service import get_response
```

→ Importe la fonction `get_response` qui interroge l'agent conversationnel Letta, définie dans le module voisin `agent_service.py` : c'est le pont entre la logique WhatsApp (ce fichier) et la logique IA.

```
8 import os
```

→ Importe le module standard `os` pour lire les variables d'environnement.

```
9 from time import sleep
```

→ Importe la fonction `sleep` du module standard `time`, qui permet de mettre le programme en pause pendant un nombre de secondes donné (non utilisée activement plus loin dans le fichier, probablement prévue pour un futur throttling des appels API).

```
11 load_dotenv()
```

→ Charge les variables d'environnement définies dans le fichier `.env`.

```
13 WHATSAPP_ACCESS_TOKEN = os.getenv("WHATSAPP_ACCESS_TOKEN")
```

→ Lit le jeton d'accès à l'API WhatsApp depuis l'environnement.

```
14 VERSION = os.getenv('VERSION')
```

→ Lit la variable d'environnement `VERSION` (version de l'API Graph de Meta) et la stocke dans une constante.

```
15 PHONE_NUMBER_ID = os.getenv("PHONE_NUMBER_ID")
```

→ Lit l'identifiant du numéro de téléphone WhatsApp depuis l'environnement.

17 `def log_http_response(response):`

→ Définit une fonction qui journalise les détails d'une réponse HTTP, pour centraliser et uniformiser le format des logs de débogage réseau utilisés à plusieurs endroits du fichier.

18 `logging.info(f"Status: {response.status_code}")`

→ Écrit dans les logs le code de statut de la réponse HTTP (200, 400, etc.), premier indicateur de succès ou d'échec d'un appel API.

19 `logging.info(f"Content-type: {response.headers.get('content-type')}")`

→ Écrit dans les logs le type de contenu de la réponse HTTP.

20 `logging.info(f"Body: {response.text}")`

→ Écrit dans les logs le corps textuel brut de la réponse HTTP, utile pour voir le détail d'une erreur renvoyée par l'API Meta.

23 `def get_text_message_input(recipient, text):`

→ Définit une fonction qui construit le payload JSON attendu par l'API WhatsApp pour un message texte, afin de ne pas dupliquer cette structure à chaque endroit où l'on envoie un message.

24 `return {`

→ Débute la construction du dictionnaire représentant le message.

25 `"messaging_product": "whatsapp",`

→ Précise que le produit de messagerie utilisé est whatsapp, champ requis par l'API.

26 `"recipient_type": "individual",`

→ Précise que le message est destiné à un destinataire individuel (par opposition à un groupe, non supporté par cette API pour l'envoi programmatique).

27 `"to": recipient,`

→ Indique le numéro de téléphone du destinataire du message.

28 `"type": "text",`

→ Précise que le type de message envoyé est du texte.

29 `"text": {"preview_url": False, "body": text},`

→ Définit le contenu textuel du message et désactive l'aperçu automatique d'URL (preview_url), qui sinon générerait une carte de prévisualisation si le texte contient un lien.

30 `}`

→ Ferme le dictionnaire représentant le message retourné.

33 `def generate_response(message):`

→ Définit une fonction générant une réponse factice à partir du message, probablement une fonction de test ou un vestige antérieur à l'intégration de l'agent Letta réel.

34 `# Dummy response for now`

→ Commentaire indiquant qu'il s'agit d'une réponse temporaire de test.

35 `return message.upper()`

→ Retourne le message reçu converti en majuscules, comme réponse factice non liée à l'IA.

38 `def send_message(data):`

→ Définit une fonction qui envoie un message via l'API WhatsApp, centralisant la logique d'appel réseau (en-têtes, gestion des erreurs) utilisée pour toute réponse envoyée à un utilisateur.

39 `headers = {`

→ Débute la construction du dictionnaire des en-têtes HTTP.

```

40         "Content-type": "application/json",
    → Ajoute l'en-tête indiquant que le contenu envoyé est du JSON.

41         "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
    → Ajoute l'en-tête d'autorisation avec le jeton d'accès WhatsApp.

42     }
    → Ferme le dictionnaire des en-têtes HTTP.

43     url = f"https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages"
    → Construit l'URL de l'API Graph de Facebook pour l'envoi de messages.

45     try:
    → Démarre un bloc pour intercepter les erreurs réseau lors de l'envoi de la requête (délai dépassé, panne, etc.), afin qu'un problème d'appel API n'interrompe pas tout le programme.

46         response = requests.post(url, json=data, headers=headers, timeout=10)
    → Envoie une requête POST à l'API WhatsApp avec un délai maximal (timeout) de 10 secondes : fixer un timeout explicite est une bonne pratique réseau, car sans lui une requête HTTP peut rester bloquée indéfiniment si le serveur distant ne répond jamais.

47         response.raise_for_status()
    → Lève une exception si le code de statut HTTP indique une erreur (4xx ou 5xx), méthode standard de la bibliothèque requests pour transformer un code d'erreur HTTP en exception Python gérable par un except.

48         log_http_response(response)
    → Journalise les détails de la réponse HTTP reçue via la fonction utilitaire définie plus haut.

49         return response
    → Retourne l'objet réponse HTTP obtenu en cas de succès.

50     except requests.Timeout:
    → Intercepte spécifiquement l'exception levée par requests en cas de dépassement du délai d'attente défini.

51         logging.error("Timeout occurred while sending message")
    → Écrit un message d'erreur signalant le dépassement du délai d'attente.

52         return JSONResponse({"status": "error", "message": "Request timed out"}, status_code=408)
    → Renvoie une réponse JSON d'erreur avec le code HTTP 408 (Request Timeout), reflétant fidèlement la nature du problème.

53     except requests.RequestException as e:
    → Intercepte toute autre exception liée à la requête HTTP (erreur réseau générique, DNS, connexion refusée...), RequestException étant la classe de base de toutes les erreurs de la bibliothèque requests.

54         logging.error(f"Request failed due to: {e}")
    → Écrit un message d'erreur détaillant la cause de l'échec de la requête.

55         return JSONResponse({"status": "error", "message": "Failed to send message"}, status_code=500)
    → Renvoie une réponse JSON d'erreur 500 indiquant l'échec de l'envoi.

58 def is_valid_whatsapp_message(body: dict):
    → Définit une fonction vérifiant que le corps reçu contient bien la structure imbriquée d'un message WhatsApp entrant valide, afin de filtrer les payloads incomplets ou d'un autre type avant de tenter de les traiter.

59     return (
    → Débute une expression composée retournant un résultat booléen combiné (grâce à l'opérateur "and" qui court-circuite dès qu'une condition est fausse).

```

60 `body.get("object")`

→ Vérifie que le corps contient un champ "object" (WhatsApp l'utilise pour identifier le type d'objet notifié, ici "whatsapp_business_account").

61 `and body.get("entry")`

→ Vérifie que le corps contient un champ entry non vide.

62 `and body["entry"][0].get("changes")`

→ Vérifie que le premier entry contient un champ changes.

63 `and body["entry"][0]["changes"][0].get("value")`

→ Vérifie que le premier changes contient un champ value.

64 `and body["entry"][0]["changes"][0]["value"].get("messages")`

→ Vérifie que value contient un champ messages (présent uniquement pour un vrai message entrant, absent pour une mise à jour de statut).

65 `and body["entry"][0]["changes"][0]["value"]["messages"][0]`

→ Vérifie que le premier élément de messages existe et est non vide.

66 `)`

→ Ferme l'expression composée et retourne le résultat booléen final de la chaîne de "and".

69 `def process_whatsapp_message(body: dict):`

→ Définit la fonction principale traitant un message WhatsApp entrant, appelée en arrière-plan depuis main.py (dans un thread séparé) : elle extrait le message, le transmet à l'agent IA, puis renvoie la réponse à l'utilisateur.

70 `value = body["entry"][0]["changes"][0]["value"]`

→ Extrait le dictionnaire value contenant les données du message. Ici, contrairement à `is_valid_whatsapp_message`, l'accès direct par crochets (`[...]`) est utilisé sans `.get()` de secours : c'est acceptable car cette fonction n'est appelée qu'après validation préalable par `is_valid_whatsapp_message`.

71 `wa_id = value["contacts"][0]["wa_id"]`

→ Extrait l'identifiant WhatsApp du contact expéditeur (`wa_id`), utilisé ensuite comme destinataire pour la réponse.

72 `name = value["contacts"][0]["profile"]["name"]`

→ Extrait le nom du profil du contact expéditeur.

73 `message = value["messages"][0]`

→ Extrait le premier message contenu dans value.

74 `if message["type"] == "text":`

→ Vérifie si le message reçu est de type texte.

75 `message_body = message["text"]["body"]`

→ Extrait le texte du corps du message.

76 `elif message["type"] == "image":`

→ Vérifie si le message reçu est de type image.

77 `message_body = message["image"]`

→ Extrait le dictionnaire décrivant l'image reçue (il ne contient pas l'image elle-même, seulement des métadonnées comme son identifiant).

78 `id = message_body.get("id")`

→ Récupère l'identifiant de l'image fourni par WhatsApp, nécessaire pour aller la télécharger séparément : l'API WhatsApp ne transmet jamais le fichier binaire directement dans le webhook, seulement une référence à récupérer via un second appel.


```

79         headers = {
    → Début la construction du dictionnaire des en-têtes HTTP pour la requête de métadonnées.

80         "Content-type": "application/json",
    → Ajoute l'en-tête indiquant que le contenu attendu est du JSON.

81         "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
    → Ajoute l'en-tête d'autorisation avec le jeton d'accès WhatsApp.

82     }
    → Ferme le dictionnaire des en-têtes HTTP.

83     metadata_res = requests.get(f"https://graph.facebook.com/{VERSION}/{id}", headers=headers)
    → Envoie une requête GET pour récupérer les métadonnées de l'image via son identifiant : première étape du téléchargement
    en deux temps imposé par l'API WhatsApp pour les médias.

84     image_url = metadata_res.json().get("url")
    → Extrait l'URL réelle et temporaire de l'image depuis la réponse JSON des métadonnées ; cette URL n'est valide que peu de
    temps et requiert elle aussi une authentification.

85     image_res = requests.get(image_url, headers={"Authorization": f"Bearer
{WHATSAPP_ACCESS_TOKEN}"})
    → Télécharge le fichier image depuis l'URL obtenue en utilisant à nouveau le jeton d'accès : seconde étape du
    téléchargement, celle-ci récupère les octets binaires réels de l'image.

86     with open(f"images/{id}.jpg", "wb") as f:
    → Ouvre en écriture binaire ("wb") un fichier local nommé d'après l'identifiant de l'image, pour la sauvegarder sur le disque
    avant de la transmettre à l'agent.

87         f.write(image_res.content)
    → Écrit le contenu binaire de l'image téléchargée dans le fichier local.

88     if message_body.get("caption"):
    → Vérifie si l'image reçue est accompagnée d'une légende (texte que l'utilisateur peut joindre à une photo WhatsApp).

89         message_body = {'image_path': f'images/{id}.jpg', 'caption': message_body.get("caption")}
    → Remplace message_body par un dictionnaire contenant le chemin local de l'image et sa légende : c'est exactement le
    format attendu par get_response dans agent_service.py pour reconnaître un message image (via isinstance(message, dict)).

90     else:
    → Cas où l'image ne comporte aucune légende associée.

91         message_body = {'image_path': f'images/{id}.jpg', 'caption': ''}
    → Remplace message_body par un dictionnaire avec une légende vide, pour garder une structure homogène même sans
    légende.

93     response_text = get_response(message_body)
    → Envoie le contenu du message (texte simple ou dictionnaire image+légende) à l'agent Letta via get_response et récupère sa
    réponse textuelle.

94     data = get_text_message_input(wa_id, response_text)
    → Construit le payload du message de réponse à envoyer au contact, en réutilisant la fonction utilitaire dédiée à cet effet.

95     send_message(data)
    → Envoie le message de réponse au contact via l'API WhatsApp, bouclant ainsi le cycle complet : message reçu -> traité par
    l'agent -> réponse renvoyée sur WhatsApp.

```

Projet 24 — 24-ResearchAnalyst

Reproduit un rapport pour un autre pays a partir d'un PDF source

Objectif

Prend en entree un rapport de recherche existant au format PDF portant sur un pays donne, et produit automatiquement un nouveau rapport suivant la meme structure et methodologie mais applique a un autre pays choisi par l'utilisateur, en recherchant les donnees actualisees necessaires sur le web.

Code source, ligne par ligne

24-ResearchAnalyst/WorkflowAgent.py

Définit le workflow à deux agents qui effectue l'OCR d'un PDF, en analyse la structure, puis génère un rapport équivalent pour un autre pays.

114 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard os pour lire les variables d'environnement et vérifier l'existence de fichiers.

2 from textwrap import dedent

→ Importe la fonction dedent du module standard textwrap, qui supprime l'indentation commune d'une chaîne de caractères multi-lignes. Elle est utilisée ici pour écrire de longs blocs d'instructions destinés au modèle de langage directement indentés comme le reste du code Python (donc lisibles), tout en obtenant une chaîne finale propre sans les espaces d'indentation parasites.

3 from typing import Iterator

→ Importe le type Iterator du module standard typing, utilisé uniquement pour l'annotation de type des fonctions qui produisent un flux de réponses au fur et à mesure (streaming) plutôt qu'un résultat unique.

4 from mistralai.client import Mistral

→ Importe le client officiel de Mistral AI. Il est utilisé ici non pas pour du texte, mais pour son service d'OCR (reconnaissance optique de caractères) : Mistral propose une API capable d'extraire le texte structuré (y compris la mise en page en markdown) à partir d'un document PDF, y compris scanné.

5 from agno.workflow import Workflow

→ Importe la classe de base Workflow du framework agno. Agno est un framework Python pour construire des agents et des workflows d'agents IA : un Workflow orchestre plusieurs agents ou étapes dans un ordre défini par du code Python classique (contrairement à un simple agent isolé), ce qui permet de construire des pipelines multi-étapes comme celui de ce fichier (OCR -> analyse -> recherche).

6 from agno.models.groq import Groq

→ Importe la classe de modèle Groq d'agno, un wrapper qui permet à un Agent agno d'utiliser un modèle de langage hébergé par Groq. Groq est un fournisseur d'inférence connu pour la vitesse exceptionnelle à laquelle il exécute des modèles de langage (grâce à du matériel dédié, les LPU), ce qui le rend intéressant pour des workflows où la latence compte.

7 from agno.agent import Agent, RunOutput as RunResponse

→ Importe la classe Agent, brique de base d'agno représentant un agent IA autonome (modèle + instructions + éventuels outils), et renomme RunOutput (le type représentant le résultat d'une exécution d'agent) en RunResponse pour la suite du fichier.

8 from agno.utils.pprint import pprint_run_response

→ Importe une fonction utilitaire d'affichage formaté ("pretty print") des réponses d'agent dans la console, non utilisée directement dans ce fichier mais disponible pour du débogage.

```
9 from agno.tools.duckduckgo import DuckDuckGoTools
```

→ Importe l'outil de recherche DuckDuckGo intégré à agno. Un "tool" (outil), dans le contexte des agents IA, est une fonction que le modèle de langage peut décider d'appeler lui-même pendant son raisonnement (function calling) pour aller chercher une information qu'il ne connaît pas nativement ; DuckDuckGoTools donne ainsi à l'agent la capacité de faire de vraies recherches web en direct plutôt que de se limiter à ses connaissances figées à l'entraînement.

```
10 from dotenv import load_dotenv
```

→ Importe la fonction permettant de charger les variables depuis un fichier .env.

```
12 load_dotenv()
```

→ Charge les variables d'environnement définies dans le fichier .env.

```
14 mistral_key = os.getenv("MISTRAL_API_KEY")
```

→ Lit la clé API Mistral depuis les variables d'environnement, nécessaire pour authentifier les appels au service OCR.

```
15 # Le script original utilisait OpenAI (gpt-4o-mini) ; remplacé par Groq pour
```

→ Commentaire expliquant que le modèle OpenAI original a été remplacé par Groq.

```
16 # réutiliser la clé GROQ_API_KEY déjà disponible.
```

→ Suite du commentaire précisant la raison du remplacement par Groq : réutiliser une clé API déjà en possession du projet plutôt que d'en configurer une nouvelle.

```
17 groq_api_key = os.getenv("GROQ_API_KEY")
```

→ Lit la clé API Groq depuis les variables d'environnement, utilisée par les deux agents du workflow pour interroger leur modèle de langage.

```
18 client = Mistral(api_key=mistral_key)
```

→ Crée une instance du client Mistral avec la clé API fournie, prête à être utilisée pour l'OCR du PDF.

```
20 # OCR the pdf and return the text
```

→ Commentaire indiquant que la fonction suivante effectue l'OCR d'un PDF.

```
21 def ocr_pdf(pdf_path):
```

→ Définit la fonction qui effectue l'OCR (reconnaissance optique de caractères) d'un fichier PDF et retourne son texte. L'OCR est la technique qui convertit une image de document (texte figé en pixels, comme un PDF scanné) en texte réellement exploitable par du code ou un modèle de langage.

```
22     # check if the file exists
```

→ Commentaire indiquant que la ligne suivante vérifie l'existence du fichier.

```
23     if not os.path.exists(pdf_path):
```

→ Vérifie si le chemin du fichier PDF fourni n'existe pas, pour échouer proprement plutôt que de laisser une erreur bas niveau se produire plus loin.

```
24         return "PDF file not found"
```

→ Retourne un message d'erreur si le fichier PDF est introuvable.

```
26     uploaded_pdf = client.files.upload(
```

→ Débute l'envoi du fichier PDF vers l'infrastructure de Mistral : contrairement à un simple appel de texte, l'OCR sur un gros document nécessite d'abord de téléverser le fichier lui-même, qui sera ensuite référencé par un identifiant.

```
27         file={
```

→ Débute la construction du dictionnaire décrivant le fichier à envoyer.

```
28         "file_name": pdf_path,
```

→ Indique le nom (chemin local) du fichier à envoyer.

```

29         "content": open(pdf_path, "rb"),
    → Ouvre le fichier PDF en lecture binaire pour le transmettre tel quel dans la requête de téléversement.

30     },
    → Ferme le dictionnaire décrivant le fichier.

31     purpose="ocr"
    → Précise que le fichier est envoyé dans le but explicite de faire de l'OCR, information que Mistral utilise pour router le traitement vers le bon service.

32 )
    → Ferme l'appel d'envoi du fichier vers Mistral.

34     signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)
    → Récupère une URL signée (temporaire et sécurisée) permettant d'accéder au fichier téléversé : c'est cette URL, plutôt que le fichier lui-même, qui sera fournie au moteur d'OCR à l'étape suivante.

36     ocr_response = client.ocr.process(
    → Débute l'appel qui lance le traitement OCR proprement dit du document sur les serveurs de Mistral.

37         model="mistral-ocr-latest",
    → Précise le modèle OCR à utiliser pour le traitement, ici la dernière version disponible du modèle d'OCR de Mistral (distinct de leurs modèles de langage généralistes).

38         document={
    → Débute la description du document à analyser.

39             "type": "document_url",
    → Précise que le document est fourni sous forme d'URL plutôt que de contenu brut inline.

40             "document_url": signed_url.url,
    → Fournit l'URL signée du document à analyser.

41         },
    → Ferme le dictionnaire décrivant le document.

42         include_image_base64=True
    → Demande à inclure les images détectées dans le document, encodées en base64, dans la réponse (utile si le PDF contient des schémas ou photos à conserver, même si ce script n'exploite ici que le texte markdown).

43     )
    → Ferme l'appel de traitement OCR.

45     with open("ocr_response.md", "w") as f:
    → Ouvre en écriture un fichier local destiné à recevoir le résultat de l'OCR au format markdown, pour le conserver et le réutiliser sans refaire l'OCR (voir app.py qui relit ce fichier).

46         f.write("\n".join([page.markdown for page in ocr_response.pages]))
    → Écrit le texte markdown de toutes les pages OCRisées, concaténées et séparées par des retours à la ligne ; le résultat de l'OCR de Mistral est fourni page par page, chacune déjà convertie en markdown structuré (titres, listes...).

48     return ocr_response.pages
    → Retourne la liste des pages résultant du traitement OCR.

50 # Main Agents Workflow
    → Commentaire indiquant que la classe suivante définit le workflow principal des agents.

51 class ResearchAnalystWorkflow(Workflow):
    → Définit la classe du workflow en héritant de la classe Workflow d'agno : hériter de Workflow donne accès aux mécanismes internes d'agno (gestion du run_id, du cycle de vie de l'exécution) tout en laissant la méthode run() entièrement personnalisable

```

en Python classique.

52 # Targeted report structure (Golden structure)

→ Commentaire indiquant que l'agent suivant définit la structure cible du rapport.

53 structure_and_analysis_agent: Agent = Agent(

→ Débute la déclaration du premier agent du pipeline, chargé d'analyser la structure et le contenu du document source. C'est le motif de conception central de ce fichier : un pipeline à deux agents spécialisés, chacun avec un rôle précis (ici : analyser), plutôt qu'un seul agent générique censé tout faire.

54 name = 'Structure and Analysis Agent',

→ Attribue un nom à cet agent d'analyse de structure, utile pour l'identifier dans les logs et l'interface de suivi d'agno.

55 model=Groq(id="openai/gpt-oss-120b", api_key=groq_api_key),

→ Configure l'agent pour utiliser, via Groq, le modèle open-weight gpt-oss-120b (120 milliards de paramètres) servi à très grande vitesse par l'infrastructure Groq, avec la clé API fournie.

56 instructions = dedent("""

→ Débute la chaîne multi-lignes contenant les instructions système de l'agent, avec indentation supprimée par dedent. Ces "instructions" jouent le rôle de prompt système : elles définissent le rôle, les règles et le format de sortie attendu du modèle, un des leviers principaux pour contrôler le comportement d'un agent IA.

57 You are an intelligent document analysis agent. You are given the raw text of a PDF (OCR processed). Your task is to analyze the content and produce a structured summary of the document's subject, key points, and organization. The report discusses a specific country, which you must identify.

→ Ligne de texte des instructions décrivant le rôle général de l'agent d'analyse.

58 Follow these steps:

→ Ligne de texte des instructions introduisant la liste des étapes à suivre.

60 1. Detect and clearly state the COUNTRY being discussed (call this "Country X").

→ Ligne de texte des instructions demandant d'identifier le pays traité dans le document.

61 2. Identify the MAIN TOPIC of the document (e.g. energy policy, health care reforms, education quality).

→ Ligne de texte des instructions demandant d'identifier le sujet principal du document.

62 3. Define the DOCUMENT STRUCTURE as a clean numbered outline with sections and subsections based on the logical flow of the text.

→ Ligne de texte des instructions demandant de définir la structure du document sous forme de plan numéroté.

63 4. Extract the KEY ENTITIES, KEY FIGURES (if any), and any important ORGANIZATIONS or AGENCIES mentioned.

→ Ligne de texte des instructions demandant d'extraire les entités, chiffres et organisations clés.

64 5. Summarize the KEY POINTS and INSIGHTS under each major section, focusing on the data, trends, problems, or conclusions made about Country X.

→ Ligne de texte des instructions demandant de résumer les points clés de chaque section.

66 Output format:

→ Ligne de texte des instructions introduisant le format de sortie attendu. Imposer un format de sortie strict dans le prompt est essentiel ici car la sortie de cet agent sert directement d'entrée structurée au second agent du pipeline.

68 COUNTRY: <Country X>

→ Ligne de texte du format de sortie indiquant l'emplacement du nom du pays.

69 TOPIC: <main subject of the document>

→ Ligne de texte du format de sortie indiquant l'emplacement du sujet principal.

71 STRUCTURE:

→ Ligne de texte du format de sortie introduisant la section structure du rapport.

72 1. <Section Title>

→ Ligne de texte du format de sortie illustrant le titre de la première section.

73 - <Summary or bullet points>

→ Ligne de texte du format de sortie illustrant les puces de résumé sous une section.

74 2. <Section Title>

→ Ligne de texte du format de sortie illustrant le titre d'une deuxième section.

75 - <Summary or bullet points>

→ Ligne de texte du format de sortie illustrant à nouveau les puces de résumé.

76 ...

→ Ligne de texte du format de sortie indiquant que la liste des sections peut continuer.

78 KEY POINTS AND INSIGHTS:

→ Ligne de texte du format de sortie introduisant la section des points clés.

79 - Bullet points summarizing major findings, arguments, statistics, etc.

→ Ligne de texte du format de sortie précisant le contenu attendu des puces.

80 - Must reflect what is emphasized in the document

→ Ligne de texte du format de sortie demandant de refléter ce qui est mis en avant dans le document.

81 - Keep this detailed but concise

→ Ligne de texte du format de sortie demandant un contenu détaillé mais concis.

83 This output will be used by a second agent to replicate a similar report for a different country. Be clear, complete, and well-structured.

→ Ligne de texte des instructions précisant explicitement que la sortie servira à un second agent pour un autre pays : donner ce contexte au modèle l'aide à produire une sortie exploitable programmatiquement plutôt qu'un texte narratif libre.

84 """),

→ Ferme la chaîne multi-lignes des instructions et l'appel à dedent.

85)

→ Ferme l'appel de construction de l'agent structure_and_analysis_agent.

87 def run(self, ocr_text: str, user_country: str) -> Iterator[RunResponse]:

→ Définit la méthode run du workflow, le point d'entrée appelé pour exécuter tout le pipeline. Elle prend le texte OCR du document source et le pays cible demandé par l'utilisateur, et retourne un Iterator[RunResponse], c'est-à-dire un générateur qui produit les réponses au fur et à mesure (streaming) plutôt qu'un résultat unique attendu en bloc.

88 analysis_response: Iterator[RunResponse] = self.structure_and_analysis_agent.run(ocr_text)

→ Exécute le premier agent (analyse de structure) sur le texte OCR et stocke sa réponse ; c'est la première étape du pipeline séquentiel.

89 if analysis_response is None or not analysis_response.content:

→ Vérifie si l'agent d'analyse n'a retourné aucune réponse ou un contenu vide, cas d'échec qu'il faut gérer avant de lancer le second agent sur une entrée invalide.

90 yield RunResponse(

→ Produit (yield) une réponse de workflow signalant l'échec de l'analyse ; "yield" fait de cette méthode un générateur, permettant au code appelant de recevoir les résultats progressivement au lieu d'attendre la fin complète du traitement.

91 run_id=self.run_id,

→ Associe la réponse produite à l'identifiant d'exécution du workflow, pour pouvoir tracer quelle réponse appartient à quel run.

92 content="Sorry, could not get the analysis report.",
→ Définit le message de contenu indiquant l'échec de l'obtention du rapport d'analyse.

93)
→ Ferme l'appel de construction de la réponse RunResponse.

94 return
→ Arrête l'exécution du générateur si l'analyse a échoué, empêchant le second agent de s'exécuter inutilement sur une entrée vide.

96 research_agent: Agent = Agent(
→ Débute la déclaration du second agent du pipeline, chargé de produire le rapport final pour le nouveau pays demandé, en s'appuyant sur la sortie structurée du premier agent.

97 name = 'Research Agent',
→ Attribue un nom à cet agent de recherche.

98 model=Groq(id="openai/gpt-oss-120b", api_key=groq_api_key),
→ Configure l'agent de recherche pour utiliser le même modèle Groq que le premier agent, avec la clé API fournie.

99 instructions = dedent(f"""
→ Débute la chaîne multi-lignes formatée (f-string) contenant les instructions système de l'agent de recherche. L'usage d'une f-string ici permet d'injecter dynamiquement la valeur de user_country directement dans le prompt, pour que les instructions soient personnalisées à chaque exécution.

100 You are a research agent. You are given a structured summary of a document about a topic in a specific country. Your task is to generate a new report on the **same topic** but focused on COUNTRY: {user_country}.
→ Ligne de texte des instructions décrivant le rôle de l'agent de recherche et le pays cible.

102 You may use the GoogleSearchTools() tool to search for real-world information related to Country {user_country}.
→ Ligne de texte des instructions mentionnant un outil de recherche disponible pour le pays cible ; à noter que le prompt évoque GoogleSearchTools() alors que l'outil réellement attaché à l'agent plus bas est DuckDuckGoTools() — une petite incohérence entre le texte du prompt et la configuration réelle du code, sans impact fonctionnel car le modèle utilise simplement l'outil qui lui est effectivement fourni.

104 Instructions:
→ Ligne de texte des instructions introduisant la liste détaillée des étapes à suivre.

106 1. Read the input summary carefully. It contains:
→ Ligne de texte des instructions demandant de lire attentivement le résumé fourni en entrée.

107 - COUNTRY: <Country X>
→ Ligne de texte des instructions listant le champ COUNTRY présent dans le résumé d'entrée.

108 - TOPIC: <Main Topic>
→ Ligne de texte des instructions listant le champ TOPIC présent dans le résumé d'entrée.

109 - STRUCTURE: <Numbered outline of the report>
→ Ligne de texte des instructions listant le champ STRUCTURE présent dans le résumé d'entrée.

110 - KEY POINTS AND INSIGHTS from the original document
→ Ligne de texte des instructions listant les points clés du document original.

112 2. Understand the structure and content focus. Your output must use **the same section headings** and address **the same themes**, but for **Country {user_country}**.
→ Ligne de texte des instructions demandant de conserver la même structure pour le nouveau pays.

114 3. For each section, use `GoogleSearchTools()` to find reliable and up-to-date information specific to `Country {user_country}`. For example:

→ Ligne de texte des instructions demandant d'utiliser l'outil de recherche pour chaque section, afin d'ancrer le rapport dans des informations réelles et à jour plutôt que dans les seules connaissances figées du modèle.

115 - Local policies

→ Ligne de texte des instructions donnant un exemple d'information à rechercher, les politiques locales.

116 - Trends

→ Ligne de texte des instructions donnant un exemple d'information à rechercher, les tendances.

117 - Statistics

→ Ligne de texte des instructions donnant un exemple d'information à rechercher, les statistiques.

118 - Institutional efforts

→ Ligne de texte des instructions donnant un exemple d'information à rechercher, les efforts institutionnels.

119 - Challenges and opportunities

→ Ligne de texte des instructions donnant un exemple d'information à rechercher, les défis et opportunités.

120 - Relevant events

→ Ligne de texte des instructions donnant un exemple d'information à rechercher, les événements pertinents.

122 4. Rebuild the report using the exact structure, now filled with data and insights from `Country {user_country}`. The tone, depth, and organization should match the original as closely as possible.

→ Ligne de texte des instructions demandant de reconstruire le rapport avec la même structure pour le nouveau pays.

124 Output format:

→ Ligne de texte des instructions introduisant le format de sortie attendu.

126 TITLE: <Same topic> in <Country {user_country}>

→ Ligne de texte du format de sortie indiquant le titre attendu incluant le pays cible.

128 1. <Section Title from original>

→ Ligne de texte du format de sortie illustrant le titre repris de la première section originale.

129 - <Findings related to Country {user_country}>

→ Ligne de texte du format de sortie illustrant les résultats à fournir pour le pays cible.

130 2. <Next Section>

→ Ligne de texte du format de sortie illustrant le titre d'une section suivante.

131 - <Continue same structure>

→ Ligne de texte du format de sortie demandant de continuer selon la même structure.

133 Cite or refer to sources found via search where helpful.

→ Ligne de texte des instructions demandant de citer les sources trouvées par la recherche, pour renforcer la fiabilité perçue du rapport généré.

135 IMPORTANT: You must always stay exactly within the subject defined in the original document. Do not go broader, do not narrow the scope, and do not add or remove themes. Reproduce the same structure and content areas, adapted only to `Country {user_country}`. Nothing more, nothing less.

→ Ligne de texte des instructions insistant sur le respect strict du périmètre du sujet original, une contrainte de prompt engineering destinée à éviter que le modèle "dérive" et invente des sections hors sujet.

136 """),

→ Ferme la chaîne multi-lignes des instructions et l'appel à dedent.


```

137         tools=[DuckDuckGoTools()],
    → Fournit à l'agent l'outil de recherche DuckDuckGo qu'il peut invoquer lui-même pendant son raisonnement. C'est ce paramètre "tools" qui donne concrètement à l'agent la capacité d'aller chercher des informations réelles sur le web, au-delà de ses connaissances d'entraînement figées dans le temps.

138         # Les résultats bruts de web_search font gonfler le contexte au-delà de la
    → Commentaire expliquant que les résultats bruts de recherche satureront le contexte du modèle.

139         # limite Groq (8000 tokens/minute en tier gratuit) : on les compresse.
    → Suite du commentaire précisant la limite de tokens Groq justifiant la compression des résultats. Un "token" est l'unité de texte que compte un modèle de langage (environ un mot ou un fragment de mot) ; la plupart des fournisseurs d'API facturent et limitent l'usage en tokens par minute, donc des résultats de recherche web bruts et volumineux peuvent rapidement dépasser ce quota.

140         compress_tool_results=True,
    → Active la compression des résultats des outils : agno résume ou réduit automatiquement ce que l'outil de recherche renvoie avant de l'injecter dans le contexte du modèle, pour rester sous la limite de tokens imposée par Groq tout en conservant l'information utile.

141     )
    → Ferme l'appel de construction de l'agent reasearch_agent.

142     yield from reasearch_agent.run(analysis_response.content, stream=True)
    → Délègue au générateur du second agent la production en flux (streaming) du rapport final, à partir du contenu produit par le premier agent. "yield from" propage directement les éléments produits par le générateur interne de l'agent vers l'appelant de run(), et stream=True indique à agno de renvoyer la réponse du modèle morceau par morceau au fur et à mesure qu'elle est générée plutôt que d'attendre le texte complet, ce qui permet un affichage progressif côté interface (voir app.py).

```

24-ResearchAnalyst/app.py

Interface Streamlit permettant de téléverser un PDF, lancer son OCR puis afficher en flux le rapport généré par le workflow de recherche.

39 lignes de code annotées ci-dessous.

```

1 import streamlit as st
    → Importe la bibliothèque Streamlit, un framework Python qui permet de construire une interface web interactive (formulaires, boutons, affichage de texte) en quelques lignes de code pur Python, sans avoir à écrire de HTML/CSS/JavaScript. C'est ce qui sert ici de façade utilisateur au workflow d'analyse défini dans WorkflowAgent.py.

2 from typing import Iterator
    → Importe le type Iterator utilisé pour annoter le flux de réponses retourné par le workflow.

3 from agno.agent import RunOutput as RunResponse
    → Importe la classe RunOutput du framework agno, renommée en RunResponse, pour annoter le type des éléments reçus depuis le workflow en streaming.

4 from WorkflowAgent import ResearchAnalystWorkflow, ocr_pdf
    → Importe la classe de workflow et la fonction d'OCR définies dans le fichier voisin WorkflowAgent.py : cette interface Streamlit ne fait qu'orchestrer visuellement ce pipeline déjà défini, sans dupliquer sa logique.

6 # Main function
    → Commentaire indiquant que le bloc suivant constitue le point d'entrée principal.

7 if __name__ == "__main__":
    → Vérifie que le script est exécuté directement ; par convention, une application Streamlit se lance avec la commande "streamlit run app.py", qui exécute ce fichier comme script principal.

```

```
8 ocr_text = ""
```

→ Initialise la variable contenant le texte OCR à une chaîne vide, pour qu'elle existe même si l'utilisateur n'a encore rien soumis.

```
9 user_country = ""
```

→ Initialise la variable contenant le pays choisi par l'utilisateur à une chaîne vide.

```
10 pdf_path = ""
```

→ Initialise la variable contenant le fichier PDF téléversé à une chaîne vide.

```
11 st.title("Research Analyst Multi-Agent System")
```

→ Affiche le titre principal de la page Streamlit.

```
12 with st.form("input_form"):
```

→ Débute un formulaire Streamlit nommé input_form. Un st.form regroupe plusieurs champs de saisie de façon à ce que Streamlit ne relance le script (son comportement par défaut à chaque interaction) qu'une seule fois, au moment du clic sur le bouton de soumission, plutôt qu'à chaque frappe ou changement de champ.

```
13 pdf_path = st.file_uploader("Upload a PDF file", type=["pdf"])
```

→ Affiche un composant de téléversement de fichier, restreint aux fichiers portant l'extension .pdf via le paramètre type, pour éviter que l'utilisateur ne soumette un format non supporté par le pipeline d'OCR.

```
14 user_country = st.text_input("Enter your desired research country")
```

→ Affiche un champ de texte pour saisir le pays de recherche souhaité, la donnée transmise ensuite au second agent du workflow.

```
15 submitted = st.form_submit_button("Submit")
```

→ Affiche le bouton de soumission du formulaire et stocke, sous forme de booléen, l'état indiquant s'il vient d'être cliqué.

```
16 if submitted and pdf_path and user_country:
```

→ Vérifie que le formulaire a été soumis et que les deux champs requis (fichier PDF et pays) ont bien été renseignés, avant de lancer tout traitement coûteux.

```
17 with st.spinner("Processing PDF..."):
```

→ Affiche un indicateur de chargement animé (spinner) pendant toute la durée du bloc, pour donner un retour visuel à l'utilisateur pendant une opération potentiellement longue comme l'OCR.

```
18 # save the pdf locally
```

→ Commentaire indiquant que le bloc suivant enregistre le PDF localement.

```
19 with open(f'UploadedPDF/{pdf_path.name}', "wb") as f:
```

→ Ouvre en écriture binaire un fichier local nommé d'après le PDF téléversé, dans le dossier UploadedPDF ; il faut réécrire le fichier sur disque car st.file_uploader ne fournit qu'un objet fichier en mémoire, alors que la fonction ocr_pdf du workflow attend un chemin de fichier réel.

```
20 f.write(pdf_path.read())
```

→ Écrit le contenu binaire du fichier téléversé sur le disque.

```
21 st.success("PDF file uploaded successfully")
```

→ Affiche un message de succès (encadré vert) confirmant le téléversement du PDF.

```
22 ocr_pdf(f'UploadedPDF/{pdf_path.name}')
```

→ Appelle la fonction d'OCR importée de WorkflowAgent.py sur le fichier PDF enregistré localement, ce qui déclenche l'appel à l'API OCR de Mistral et écrit le résultat dans ocr_response.md.

```
23 # read the ocr_response.md file
```

→ Commentaire indiquant que le bloc suivant lit le fichier de résultat OCR.

24 with open("ocr_response.md", "r") as f:
→ Ouvre en lecture le fichier markdown généré par l'OCR, écrit précédemment par ocr_pdf.

25 ocr_text = f.read()
→ Lit le contenu textuel complet du fichier OCR dans la variable ocr_text.

26 st.success("OCR completed successfully")
→ Affiche un message de succès confirmant la fin de l'OCR.

27 with st.spinner("Running research workflow..."):
→ Affiche un indicateur de chargement pendant l'exécution du workflow de recherche à deux agents, l'étape la plus longue du pipeline.

28 # run the workflow
→ Commentaire indiquant que la ligne suivante lance le workflow.

29 workflow = ResearchAnalystWorkflow()
→ Instancie la classe du workflow d'analyse et de recherche définie dans WorkflowAgent.py.

30 # Display a placeholder for streaming
→ Commentaire indiquant que la ligne suivante crée un espace réservé pour l'affichage en flux.

31 output_placeholder = st.empty()
→ Crée un conteneur Streamlit vide et réutilisable (st.empty()), destiné à afficher le texte au fur et à mesure de sa génération : sans ce conteneur dédié, chaque mise à jour créerait un nouveau bloc de texte au lieu de remplacer le précédent.

32 full_output = ""
→ Initialise une chaîne vide qui accumulera progressivement tout le texte de sortie reçu du workflow en streaming.

33 # Run the workflow and stream the output
→ Commentaire indiquant que la ligne suivante exécute le workflow en streaming.

34 response: Iterator[RunResponse] = workflow.run(ocr_text=ocr_text,
user_country=user_country)
→ Lance le workflow avec le texte OCR et le pays choisi ; comme run() est un générateur (voir WorkflowAgent.py), cet appel ne bloque pas encore : il retourne immédiatement un itérateur qui ne produira les réponses qu'au fur et à mesure de la boucle for suivante.

35 for run_response in response:
→ Parcourt chaque fragment de réponse produit au fur et à mesure par le workflow en streaming, plutôt que d'attendre la réponse complète avant de commencer à afficher quoi que ce soit.

36 if run_response.content:
→ Vérifie que la réponse courante contient du texte (certains événements de streaming peuvent être vides ou porter uniquement des métadonnées).

37 full_output += run_response.content
→ Ajoute le nouveau morceau de texte reçu au texte de sortie accumulé jusqu'ici.

38 output_placeholder.markdown(full_output)
→ Met à jour l'affichage Streamlit avec le texte accumulé, rendu en markdown : à chaque itération, ce même conteneur est réécrit avec un texte de plus en plus long, ce qui donne visuellement l'effet d'un texte qui s'écrit progressivement à l'écran, comme dans une interface de chat IA typique.

39 else:
→ Cas où le formulaire n'a pas été soumis avec toutes les informations requises.

40 st.write("No PDF file uploaded")
→ Affiche un message indiquant qu'aucun fichier PDF n'a été téléversé.

Projet 25 — 25-PriceAndPositioning

Analyse concurrentielle de prix Amazon (recherche web IA)

Objectif

Analyse le positionnement prix d'un produit sur Amazon en comparant son prix et ses caractéristiques à ceux de produits concurrents trouvés via une recherche web, puis génère une recommandation de positionnement tarifaire argumentée.

Code source, ligne par ligne

25-PriceAndPositioning/price_and_positioning.py

Workflow qui recherche des produits concurrents sur Amazon via Exa/OpenAI puis génère une recommandation de prix et de positionnement.

161 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard os pour lire les variables d'environnement.

2 from exa_py import Exa

→ Importe la classe Exa, client officiel de l'API de recherche Exa. Exa est un moteur de recherche pensé spécifiquement pour être utilisé par des applications d'IA : contrairement à un moteur classique orienté vers des mots-clés, il propose une recherche « neuronale » (sémantique), c'est-à-dire capable de retrouver des pages pertinentes par le sens de la requête plutôt que par la simple correspondance de mots, ce qui le rend adapté pour alimenter un agent avec des données web fraîches.

3 from textwrap import dedent

→ Importe la fonction dedent pour supprimer l'indentation commune des chaînes multi-lignes, utilisée ici pour écrire proprement les longs blocs de prompt destinés au modèle.

4 from typing import Iterator

→ Importe le type Iterator utilisé pour annoter les fonctions qui produisent un flux de réponses (streaming).

5 from dotenv import load_dotenv

→ Importe la fonction permettant de charger les variables depuis un fichier .env.

6 from agno.workflow import Workflow

→ Importe la classe de base Workflow du framework agno, utilisée ici pour orchestrer la recherche de concurrents puis la génération de la recommandation de prix comme deux étapes séquentielles.

7 from agno.agent import Agent, RunResponse

→ Importe les classes Agent et RunResponse du framework agno.

8 from agno.models.openai import OpenAIChat

→ Importe la classe de modèle OpenAIChat d'agno, un wrapper qui permet à un Agent agno d'utiliser un modèle de la famille GPT d'OpenAI via l'API de chat completions.

9 from agno.utils.pprint import pprint_run_response

→ Importe une fonction utilitaire d'affichage formaté des réponses en flux dans la console, utilisée plus bas pour l'exécution en ligne de commande du script.

10 from openai import OpenAI

→ Importe le client officiel de l'API OpenAI (le SDK "openai" standard, distinct du wrapper agno) : il est utilisé ici en appel

direct, hors du framework agno, pour la fonction `exa_search` qui analyse les résultats de recherche sans passer par un Agent agno.

12 `load_dotenv()`

→ Charge les variables d'environnement définies dans le fichier `.env`.

14 `openai_api_key = os.getenv("OPENAI_API_KEY")`

→ Lit la clé API OpenAI depuis les variables d'environnement.

15 `exa_api_key = os.getenv("EXA_API_KEY")`

→ Lit la clé API Exa depuis les variables d'environnement.

17 `# Initialiser les clients`

→ Commentaire indiquant que les lignes suivantes initialisent les clients API.

18 `exa = Exa(api_key=exa_api_key) if exa_api_key else None`

→ Crée le client Exa uniquement si une clé API est disponible, sinon la variable vaut `None` : cette écriture conditionnelle évite que l'import ou l'instanciation du client ne plante immédiatement au démarrage du script si la clé n'est pas encore configurée, l'erreur étant gérée plus tard, au moment de l'utilisation réelle.

19 `openai_client = OpenAI(api_key=openai_api_key) if openai_api_key else None`

→ Crée le client OpenAI si une clé API est disponible, sinon la variable vaut `None`, selon la même logique défensive que pour Exa.

21 `def exa_search(search_term: str):`

→ Définit la fonction qui recherche des produits Amazon concurrents en combinant Exa (pour trouver et lire des pages web réelles) et OpenAI (pour analyser et synthétiser ces résultats bruts en fiches produit exploitables) : c'est un exemple de pipeline "recherche puis synthèse", une variante du principe de RAG (Retrieval-Augmented Generation) où l'on va chercher de l'information externe fraîche avant de la faire traiter par un modèle de langage, plutôt que de compter uniquement sur les connaissances internes du modèle.

22 `"""Fonction de recherche Exa """`

→ Docstring décrivant brièvement le rôle de la fonction `exa_search`.

23 `try:`

→ Démarre un bloc pour intercepter les erreurs de recherche ou d'analyse ; toute cette fonction est protégée car elle dépend de deux services externes (Exa et OpenAI) qui peuvent chacun échouer indépendamment.

24 `if not exa:`

→ Vérifie que le client Exa a bien été initialisé (donc qu'une clé API Exa était disponible).

25 `raise Exception("Clé API Exa non configurée")`

→ Lève une exception explicite si la clé API Exa n'est pas configurée, interceptée juste après par le bloc `except` qui bascule alors vers un mode de secours.

27 `# Utilisation de l'API Exa standard pour la recherche`

→ Commentaire indiquant que le bloc suivant appelle l'API Exa standard.

28 `search_results = exa.search(`

→ Débute l'appel de recherche à l'API Exa, qui va interroger le web pour trouver des pages pertinentes selon la requête donnée.

29 `query=f"Amazon {search_term} product reviews price",`

→ Construit la requête de recherche en combinant le terme produit fourni par l'utilisateur avec des mots-clés ciblant Amazon, les avis et le prix, pour orienter la recherche vers des pages produit exploitables.

30 `type="neural", # Recherche sémantique`

→ Précise que le type de recherche utilisé est la recherche neuronale (sémantique) d'Exa, qui comprend l'intention de la requête plutôt que de ne faire correspondre que des mots-clés exacts, contrairement à une recherche par mots-clés classique (`type="keyword"`).

```

31         num_results=10,
    → Demande à récupérer un maximum de dix résultats de recherche.

32         include_domains=["amazon.com", "amazon.fr"],
    → Restreint la recherche aux domaines amazon.com et amazon.fr, pour ne récupérer que des pages produit réellement issues
d'Amazon plutôt que des blogs ou comparateurs tiers.

33         use_autoprompt=True
    → Active la reformulation automatique de la requête par Exa, une fonctionnalité qui laisse le moteur de recherche optimiser
lui-même la formulation de la requête pour de meilleurs résultats.

34     )
    → Ferme l'appel de recherche à l'API Exa.

36     # Obtenir le contenu des pages
    → Commentaire indiquant que le bloc suivant récupère le contenu textuel des pages trouvées.

37     content_results = exa.get_contents(
    → Débute l'appel récupérant le contenu textuel réel des pages trouvées : la recherche (search) ne renvoie que des
métadonnées (titre, URL, score), il faut un second appel pour obtenir le texte de la page elle-même.

38         urls=[result.url for result in search_results.results[:5]],
    → Construit la liste des URL des cinq premiers résultats de recherche, limitant volontairement le nombre de pages à récupérer
pour ne pas générer une quantité excessive de texte à traiter ensuite.

39         text=True
    → Demande à récupérer le contenu sous forme de texte brut (déjà extrait du HTML par Exa), plutôt que de devoir parser
soi-même la page.

40     )
    → Ferme l'appel de récupération du contenu des pages.

42     # Formater les résultats pour l'agent
    → Commentaire indiquant que le bloc suivant met en forme les résultats.

43     formatted_content = f"""
    → Débute la construction d'une chaîne multi-lignes formatée qui va résumer, sous une forme lisible par un modèle de langage,
les résultats de recherche bruts récupérés.

44     Résultats de recherche pour "{search_term}":
    → Ligne de texte insérant le terme recherché dans l'en-tête du résumé.

46     """
    → Ferme la chaîne multi-lignes initiale affectée à formatted_content.

48     for i, (result, content) in enumerate(zip(search_results.results[:5],
content_results.results)):
    → Parcourt en parallèle les cinq premiers résultats de recherche (métadonnées) et leur contenu associé (texte de page), grâce
à zip qui associe les deux listes élément par élément et enumerate qui fournit un index de position pour la numérotation.

49         formatted_content += f"""
    → Ajoute à formatted_content un nouveau bloc de texte multi-lignes pour ce résultat.

50         ## Produit {i+1}
    → Ligne de texte ajoutant un titre markdown numéroté pour ce produit.

51         **Titre**: {result.title}
    → Ligne de texte insérant le titre du résultat de recherche.

```

52 `**URL**: {result.url}`
→ Ligne de texte insérant l'URL du résultat de recherche.

53 `**Score**: {result.score}`
→ Ligne de texte insérant le score de pertinence attribué par Exa à ce résultat par rapport à la requête.

54 `**Contenu**: {content.text[:1000] if content.text else "Contenu non disponible"}...`
→ Ligne de texte insérant les mille premiers caractères du contenu de la page (troncature volontaire pour limiter la taille totale du texte envoyé ensuite au modèle) ou un message par défaut si aucun contenu n'a pu être extrait.

56 `"""`
→ Ferme la chaîne multi-lignes ajoutée pour ce résultat.

58 `# Utiliser OpenAI pour analyser les résultats Exa`
→ Commentaire indiquant que le bloc suivant utilise OpenAI pour analyser les résultats.

59 `if not openai_client:`
→ Vérifie que le client OpenAI a bien été initialisé, car l'étape de synthèse qui suit en dépend entièrement.

60 `raise Exception("Clé API OpenAI non configurée")`
→ Lève une exception si la clé API OpenAI n'est pas configurée.

62 `completion = openai_client.chat.completions.create(`
→ Débute l'appel à l'API de chat completions d'OpenAI, l'appel standard pour obtenir une réponse générée par un modèle de langage à partir d'une liste de messages ; ici il sert à transformer les résultats de recherche bruts en une synthèse structurée de trois produits concurrents.

63 `model="gpt-4o-mini",`
→ Précise le modèle OpenAI utilisé pour la génération : gpt-4o-mini est une version allégée et moins coûteuse de la famille GPT-4o, adaptée à des tâches de synthèse de texte qui ne nécessitent pas le modèle le plus puissant.

64 `messages=[`
→ Débute la liste des messages envoyés au modèle, structurée selon le format conversationnel standard des API de chat (une suite de messages avec un rôle chacun).

65 `{"role": "system", "content": """Tu es un expert en recherche de produits Amazon.`
→ Débute le message système, qui définit le rôle et le comportement général attendu du modèle pour toute la conversation (par opposition au message "user" qui porte la demande ponctuelle) : ici, jouer le rôle d'un expert en recherche de produits Amazon.

66 `Analyse les résultats de recherche fournis et extrait 3 produits pertinents avec:`
→ Ligne de texte du message système demandant d'extraire trois produits pertinents.

67 `- Nom du produit`
→ Ligne de texte du message système listant le nom du produit comme champ attendu.

68 `- Prix (si disponible)`
→ Ligne de texte du message système listant le prix comme champ attendu s'il est disponible.

69 `- Description courte`
→ Ligne de texte du message système listant une description courte comme champ attendu.

70 `- Points négatifs mentionnés (ou crée des exemples réalistes d'avis négatifs basés sur le type de produit)`
→ Ligne de texte du message système demandant des points négatifs, ou à défaut des exemples réalistes générés par le modèle si les avis réels sont absents ou insuffisants dans les pages récupérées.

72 `Format en markdown propre.""],`

→ Ferme le message système en demandant un format markdown propre, pour que la sortie soit directement affichable proprement dans l'interface finale.

```
73         {"role": "user", "content": f"""
```

→ Débute le message utilisateur (rôle "user"), qui porte la demande concrète et les données à traiter pour cet appel précis.

```
74         Analyse ces résultats de recherche pour "{search_term}" et extrait 3 produits Amazon pertinents:
```

→ Ligne de texte du message utilisateur demandant d'analyser les résultats pour le terme recherché.

```
76         {formatted_content}
```

→ Ligne de texte du message utilisateur insérant le contenu formaté des résultats de recherche construits plus haut, qui constitue le contexte factuel fourni au modèle (c'est le principe même du RAG : donner au modèle des données récupérées en direct plutôt que de compter sur sa mémoire).

```
78         Si les prix ne sont pas visibles, estime des prix réalistes pour ce type de produit.
```

→ Ligne de texte du message utilisateur demandant d'estimer les prix si non visibles.

```
79         Pour les avis négatifs, soit utilise ceux trouvés, soit crée des exemples réalistes basés sur les problèmes courants de ce type de produit.
```

→ Ligne de texte du message utilisateur précisant comment traiter les avis négatifs.

```
80         """}
```

→ Ferme le message utilisateur formaté.

```
81     ],
```

→ Ferme la liste des messages envoyés au modèle.

```
82     temperature=0.7
```

→ Fixe la température de génération à 0.7. La température est un paramètre qui contrôle le degré d'aléatoire/créativité des réponses d'un modèle de langage : une valeur basse (proche de 0) donne des réponses déterministes et répétitives, une valeur plus haute (proche de 1 ou plus) donne des réponses plus variées et créatives ; 0.7 est une valeur intermédiaire courante pour des tâches de rédaction qui doivent rester cohérentes tout en ayant un peu de naturel.

```
83 )
```

→ Ferme l'appel de création de complétion OpenAI.

```
85     return completion.choices[0].message.content
```

→ Retourne le contenu textuel de la première réponse générée par le modèle (choices[0], le modèle pouvant en théorie renvoyer plusieurs variantes de réponse si demandé, mais une seule est demandée ici par défaut).

```
87     except Exception as e:
```

→ Intercepte toute exception survenue pendant la recherche ou l'analyse (clé manquante, erreur réseau, quota dépassé...).

```
88         print(f"Erreur lors de la recherche Exa: {e}")
```

→ Affiche dans la console le détail de l'erreur rencontrée.

```
89         print(" Utilisation du mode fallback (OpenAI seulement)...")
```

→ Affiche un message indiquant le passage en mode de secours (fallback) utilisant uniquement OpenAI, sans la recherche web via Exa.

```
91     # Fallback : utiliser uniquement OpenAI avec des connaissances générales
```

→ Commentaire précisant que le bloc suivant utilise OpenAI seul en repli. Ce mécanisme de fallback est un choix de conception qui privilégie la disponibilité du service : plutôt que de faire échouer toute la fonctionnalité si la recherche web échoue, on dégrade la qualité de la réponse (connaissances générales du modèle plutôt que données web réelles) mais on continue à fournir un résultat utile à l'utilisateur.

```
92     if not openai_client:
```

→ Vérifie que le client OpenAI a bien été initialisé avant de tenter le repli, puisque tout le mode fallback en dépend.

93 return f" Erreur: Clé API OpenAI requise. Veuillez configurer OPENAI_API_KEY dans votre fichier .env"

→ Retourne un message d'erreur si le client OpenAI n'est pas configuré, seul cas où la fonction ne peut vraiment rien produire.

95 completion = openai_client.chat.completions.create(
→ Début l'appel de secours à l'API de chat completions d'OpenAI, cette fois sans aucune donnée de recherche web en entrée.

96 model="gpt-4o-mini",
→ Précise le modèle OpenAI utilisé pour la génération de secours.

97 messages=[
→ Début la liste des messages envoyés au modèle en mode de secours.

98 {"role": "user", "content": f""
→ Début le message utilisateur demandant une analyse de marché fictive, cette fois sans message système séparé (structure simplifiée pour ce cas de repli).

99 Crée une analyse de marché fictive mais réaliste pour le produit suivant: {search_term}
→ Ligne de texte demandant de créer une analyse de marché fictive pour le produit recherché, le modèle devant ici s'appuyer uniquement sur ses connaissances générales acquises à l'entraînement plutôt que sur des données récupérées en direct.

101 Inclus 3 produits concurrents avec:
→ Ligne de texte demandant d'inclure trois produits concurrents fictifs.

102 - Nom du produit
→ Ligne de texte listant le nom du produit comme champ attendu.

103 - Prix estimé réaliste
→ Ligne de texte listant un prix estimé réaliste comme champ attendu.

104 - Description courte
→ Ligne de texte listant une description courte comme champ attendu.

105 - 3 avis clients négatifs réalistes pour chaque produit
→ Ligne de texte demandant trois avis clients négatifs réalistes par produit.

107 Base-toi sur ta connaissance des produits similaires sur Amazon.
→ Ligne de texte demandant de se baser sur la connaissance générale des produits similaires.

108 Format en markdown propre et détaillé.
→ Ligne de texte demandant un format markdown propre et détaillé.

109 """}
→ Ferme le message utilisateur du mode de secours.

110],
→ Ferme la liste des messages du mode de secours.

111 temperature=0.7
→ Fixe la température de génération à 0.7 pour le mode de secours, comme pour l'appel principal.

112)
→ Ferme l'appel de création de complétion OpenAI en mode de secours.

114 return completion.choices[0].message.content
→ Retourne le contenu textuel de la réponse générée en mode de secours.

116 `class ProductPriceAndPositioning(Workflow):`

→ Définit la classe du workflow de tarification et positionnement en héritant de Workflow d'agno, la seconde grande étape du pipeline : elle prend les données de marché collectées par `exa_search` et les transforme en recommandation stratégique.

117 `pricing_and_positioning_strategist: Agent = Agent(`

→ Débute la déclaration de l'agent stratège chargé d'analyser les données de marché et de produire la recommandation de tarification, cette fois via un Agent agno complet (avec instructions et format de sortie) plutôt qu'un appel direct à l'API OpenAI comme dans `exa_search`.

118 `name='Pricing Strategist',`

→ Attribue le nom "Pricing Strategist" à cet agent.

119 `model=OpenAIChat(id='gpt-4o-mini'),`

→ Configure l'agent pour utiliser le modèle OpenAI gpt-4o-mini via le wrapper agno OpenAIChat.

120 `instructions=dedent("""`

→ Débute la chaîne multi-lignes contenant les instructions système (le prompt) de l'agent stratège.

121 `You are a pricing and product strategy expert.`

→ Ligne de texte des instructions décrivant le rôle d'expert en tarification et stratégie produit.

123 `You will receive structured market research data about 3 competitor products, including for each:`

→ Ligne de texte des instructions précisant la nature des données de recherche reçues (la sortie de `exa_search`, transmise en entrée du `run()` plus bas).

124 `- Competitor name`

→ Ligne de texte des instructions listant le nom du concurrent comme donnée reçue.

125 `- Price`

→ Ligne de texte des instructions listant le prix comme donnée reçue.

126 `- Product description summary`

→ Ligne de texte des instructions listant le résumé de description produit comme donnée reçue.

127 `- 3 negative customer reviews highlighting product complaints`

→ Ligne de texte des instructions listant les trois avis négatifs comme donnée reçue.

129 `Your tasks:`

→ Ligne de texte des instructions introduisant la liste des tâches à accomplir.

131 `1. Analyze the competitor prices and product positioning (budget, midrange, premium).`

→ Ligne de texte des instructions demandant d'analyser les prix et le positionnement des concurrents.

132 `2. Pay special attention to the competitors' negative reviews to identify common product issues and customer pain points.`

→ Ligne de texte des instructions demandant d'étudier les avis négatifs pour identifier les problèmes récurrents.

133 `3. Based on the competitive pricing landscape and product differentiation, propose a recommended price range for your product:`

→ Ligne de texte des instructions demandant de proposer une fourchette de prix recommandée.

134 `- Classify it as Budget (< $30), Midrange ($30-$50), or Premium (> $50).`

→ Ligne de texte des instructions demandant de classer le produit selon trois catégories de prix précisément définies par des seuils chiffrés, ce qui contraint le modèle à un résultat structuré et comparable plutôt qu'une catégorisation vague.

135 `- Justify why this price range is appropriate, referencing competitors and market conditions.`

→ Ligne de texte des instructions demandant de justifier la fourchette de prix choisie.

136 4. Under **Key Competitors**, list each competitor with:
→ Ligne de texte des instructions demandant de lister chaque concurrent sous une section dédiée.

137 - Name and price
→ Ligne de texte des instructions demandant d'inclure le nom et le prix du concurrent.

138 - A brief note on how it compares to your product
→ Ligne de texte des instructions demandant une courte comparaison avec le produit propre.

139 - Include the 3 bad reviews associated with that competitor, quoting customer complaints
→ Ligne de texte des instructions demandant de citer les trois avis négatifs du concurrent.

140 5. Based on the negative reviews, suggest 2-4 tactical recommendations to improve your product and stand out, such as product improvements, bundles, discounts, or unique positioning strategies. Tie each recommendation directly to the competitor complaints you analyzed.
→ Ligne de texte des instructions demandant de proposer des recommandations tactiques liées aux avis négatifs, pour que la sortie finale reste ancrée dans des données concrètes plutôt que des conseils génériques.

141 """),
→ Ferme la chaîne multi-lignes des instructions et l'appel à dedent.

142 expected_output=dedent("""
→ Débute une seconde chaîne multi-lignes, cette fois passée au paramètre expected_output plutôt qu'instructions : ce paramètre agno sert spécifiquement à décrire le gabarit exact de sortie attendu (structure, titres, emojis), séparément des instructions de raisonnement, pour renforcer la conformité du format produit par le modèle.

143 Return your output exactly in this markdown format:
→ Ligne de texte demandant de respecter exactement le format markdown fourni.

145 ---
→ Ligne de texte insérant un séparateur horizontal markdown dans le modèle de sortie.

147 ## Recommended Price Range
→ Ligne de texte insérant le titre de la section fourchette de prix recommandée.

149 **Range**: ` \$XX - \$YY`
→ Ligne de texte du modèle de sortie indiquant l'emplacement de la fourchette de prix.

150 **Tier**: _Budget / Midrange / Premium_
→ Ligne de texte du modèle de sortie indiquant l'emplacement de la catégorie de prix.

152 ---
→ Ligne de texte insérant un second séparateur horizontal markdown.

154 ## Key Competitors
→ Ligne de texte insérant le titre de la section des concurrents clés.

156 - **[Competitor Name]** - ` \$XX.XX` : _Short comparison note_
→ Ligne de texte du modèle de sortie illustrant le format d'une entrée de concurrent.

157 - Bad Reviews:
→ Ligne de texte du modèle de sortie introduisant la liste des mauvais avis du concurrent.

158 1. "..."
→ Ligne de texte du modèle de sortie illustrant l'emplacement du premier avis négatif.

159 2. "..."
→ Ligne de texte du modèle de sortie illustrant l'emplacement du deuxième avis négatif.

160 3. "..."

→ Ligne de texte du modèle de sortie illustrant l'emplacement du troisième avis négatif.

162 - ****[Competitor Name]**** - `\$\$X.XX`: _Short comparison note_

→ Ligne de texte du modèle de sortie illustrant une seconde entrée de concurrent.

163 - Bad Reviews:

→ Ligne de texte du modèle de sortie introduisant à nouveau la liste des mauvais avis.

164 1. "..."

→ Ligne de texte du modèle de sortie illustrant l'emplacement du premier avis négatif du second concurrent.

165 2. "..."

→ Ligne de texte du modèle de sortie illustrant l'emplacement du deuxième avis négatif du second concurrent.

166 3. "..."

→ Ligne de texte du modèle de sortie illustrant l'emplacement du troisième avis négatif du second concurrent.

168 - _(Add more competitors if applicable)_

→ Ligne de texte du modèle de sortie indiquant la possibilité d'ajouter d'autres concurrents.

170 ---

→ Ligne de texte insérant un troisième séparateur horizontal markdown.

172 ## Rationale

→ Ligne de texte insérant le titre de la section justification.

174 _Explain why this price range fits your product and market, referencing competitor prices, positioning, and demand._

→ Ligne de texte demandant d'expliquer la pertinence de la fourchette de prix choisie.

176 ---

→ Ligne de texte insérant un quatrième séparateur horizontal markdown.

178 ## Tactical Recommendations

→ Ligne de texte insérant le titre de la section recommandations tactiques.

180 - ****[Recommendation 1]****: _Based on competitor complaints, e.g. "Improve pump mechanism to fix frequent failures."_

→ Ligne de texte du modèle de sortie illustrant un exemple de première recommandation.

181 - ****[Recommendation 2]****: _E.g. "Offer a cleaning brush bundle addressing cleaning difficulties customers mention."_

→ Ligne de texte du modèle de sortie illustrant un exemple de deuxième recommandation.

182 - _(Add more as relevant)_

→ Ligne de texte du modèle de sortie indiquant la possibilité d'ajouter d'autres recommandations.

184 ---

→ Ligne de texte insérant un cinquième séparateur horizontal markdown.

186 Keep your analysis specific, actionable, and tied closely to the competitor data and customer feedback.

→ Ligne de texte demandant de garder l'analyse spécifique et directement liée aux données concurrentielles.

187 """),

→ Ferme la chaîne multi-lignes du format de sortie attendu et l'appel à dedent.

```

188         markdown=True
    → Active le rendu markdown pour la sortie de l'agent côté agno, cohérent avec le format demandé dans expected_output.

189     )
    → Ferme l'appel de construction de l'agent pricing_and_positioning_strategist.

191     def run(self, product_details: str):
    → Définit la méthode run du workflow, point d'entrée qui reçoit la description du produit dont on veut positionner le prix.

192         print(f" Recherche de données de marché pour: {product_details}")
    → Affiche dans la console le lancement de la recherche de données de marché.

193         research_response = exa_search(product_details)
    → Appelle la fonction exa_search définie plus haut pour obtenir les données de marché (recherche Exa + synthèse OpenAI) sur le produit ; c'est la première étape du workflow, la collecte de données.

194         print('=== DONNÉES DE RECHERCHE ===')
    → Affiche dans la console un séparateur annonçant les données de recherche.

195         print(research_response)
    → Affiche dans la console le contenu complet des données de recherche obtenues, utile pour déboguer visuellement ce que reçoit l'agent stratégie.

196         print('=====')
    → Affiche dans la console un séparateur de fin de bloc de données.

197         yield from self.pricing_and_positioning_strategist.run(research_response, stream=True)
    → Délègue au générateur de l'agent stratégie la production en flux (streaming) de l'analyse finale, à partir des données de recherche collectées ; c'est la seconde étape du workflow, la synthèse stratégique, exécutée à partir du résultat de la première.

199 if __name__ == '__main__':
    → Vérifie que le script est exécuté directement et non importé, pour permettre un test en ligne de commande du workflow.

201     from rich.prompt import Prompt
    → Importe la classe Prompt de la bibliothèque rich, une bibliothèque tierce d'affichage enrichi dans le terminal (couleurs, mise en forme). Prompt.ask fournit une invite de saisie utilisateur plus soignée qu'un simple input() natif.

202     product = Prompt.ask('Enter your product details')
    → Demande à l'utilisateur de saisir les détails du produit dans le terminal, via l'invite stylisée de rich.

203     if product:
    → Vérifie qu'une valeur non vide a bien été saisie par l'utilisateur avant de lancer le workflow.

204         workflow = ProductPriceAndPositioning()
    → Instancie la classe du workflow de tarification et de positionnement.

205         response: Iterator[RunResponse] = workflow.run(product_details=product)
    → Lance le workflow avec les détails du produit et récupère l'itérateur de réponses en streaming.

206         pprint_run_response(response, markdown=True)
    → Affiche joliment dans le terminal le flux de réponses au format markdown, en utilisant la fonction utilitaire d'agno importée en tête de fichier, qui gère l'affichage progressif et la mise en forme markdown directement dans la console.

```

Projet 26 — 26-PlateDetection

Detection de plaques d'immatriculation + alerte SMS

Objectif

Analyse une image pour détecter et lire les plaques d'immatriculation des véhicules qui y apparaissent, puis envoie une alerte SMS automatique si une plaque correspond à une liste de véhicules surveillés.

Code source, ligne par ligne

26-PlateDetection/plate_detect.py

Application Streamlit qui analyse une vidéo pour détecter des plaques d'immatriculation, les confronte à une base d'infractions et envoie une alerte SMS via un agent.

122 lignes de code annotées ci-dessous.

1 import streamlit as st

→ Importe la bibliothèque Streamlit, utilisée ici pour afficher l'interface web de l'application (titre, bouton de démarrage, messages de statut, résultats JSON).

2 import json

→ Importe le module standard json pour lire la base de données d'infractions et manipuler les données échangées avec l'agent.

3 import os

→ Importe le module standard os pour lire les variables d'environnement (clés API, identifiants Twilio).

4 import warnings

→ Importe le module standard warnings, qui permet de contrôler l'affichage des avertissements Python. Certaines bibliothèques de vision par ordinateur ou de machine learning affichent des avertissements verbeux (dépréciations, etc.) qui peuvent polluer la sortie d'une application ; ce module permet de les masquer.

5 from textwrap import dedent

→ Importe la fonction dedent pour supprimer l'indentation commune des chaînes multi-lignes contenant le prompt de l'agent.

6 from agno.agent import Agent

→ Importe la classe Agent du framework agno, utilisée ici pour construire un petit agent chargé de mettre en forme et d'envoyer les alertes SMS à partir des données d'infraction détectées.

7 from agno.models.openai import OpenAIChat

→ Importe la classe de modèle OpenAIChat d'agno, qui permet à l'agent d'utiliser un modèle de la famille GPT d'OpenAI.

8 from inference import InferencePipeline

→ Importe la classe InferencePipeline du SDK Roboflow "inference". Roboflow est une plateforme de vision par ordinateur permettant d'entraîner et de déployer des modèles de détection d'objets ; InferencePipeline permet d'exécuter localement un "workflow" Roboflow (une chaîne de modèles, ici détection de véhicules/plaques puis lecture du texte de la plaque) directement sur un flux vidéo, image par image, sans avoir à gérer soi-même le découpage en frames.

9 from twilio.rest import Client

→ Importe la classe Client du SDK Twilio. Twilio est un service tiers permettant d'envoyer des SMS (et d'autres types de communications) par programmation via une API ; il est utilisé ici pour envoyer l'alerte SMS lorsqu'une plaque en infraction est détectée.

```
11 warnings.filterwarnings("ignore")
```

→ Configure le programme pour ignorer tous les avertissements affichés, afin de garder une sortie console/Streamlit propre malgré les bibliothèques de vision par ordinateur sous-jacentes.

```
13 # Load environment
```

→ Commentaire indiquant que les lignes suivantes chargent l'environnement.

```
14 from dotenv import load_dotenv
```

→ Importe la fonction permettant de charger les variables depuis un fichier .env.

```
15 load_dotenv()
```

→ Charge les variables d'environnement définies dans le fichier .env.

```
17 # Global setup
```

→ Commentaire indiquant que les lignes suivantes configurent les variables globales.

```
18 roboflow_key = os.getenv('ROBOFLOW_API_KEY')
```

→ Lit la clé API Roboflow depuis les variables d'environnement, nécessaire pour exécuter le workflow de détection hébergé sur la plateforme Roboflow.

```
19 account_sid = os.getenv('TWILIO_ACCOUNT_SID')
```

→ Lit l'identifiant de compte Twilio (Account SID) depuis les variables d'environnement, l'identifiant unique du compte Twilio utilisé.

```
20 auth_token = os.getenv('TWILIO_AUTH_TOKEN')
```

→ Lit le jeton d'authentification Twilio depuis les variables d'environnement, utilisé conjointement avec l'Account SID pour authentifier les appels à l'API Twilio.

```
21 client = Client(account_sid, auth_token)
```

→ Crée le client Twilio authentifié avec l'identifiant de compte et le jeton, l'objet utilisé ensuite pour envoyer des SMS.

```
23 MAX_FRAMES = 60
```

→ Définit le nombre maximal de frames vidéo à traiter avant l'arrêt automatique du pipeline, une limite pratique pour ne pas laisser tourner indéfiniment le traitement vidéo (utile en démonstration ou en test).

```
24 PLATES_FOUND = []
```

→ Initialise une liste vide destinée à stocker les plaques détectées (déclarée mais non alimentée directement dans le code visible, probablement prévue pour un suivi plus poussé).

```
25 DONE_PLATES = []
```

→ Initialise une liste vide destinée à stocker les plaques déjà traitées, afin d'éviter d'envoyer plusieurs alertes pour la même plaque détectée sur plusieurs frames consécutives.

```
26 data = []
```

→ Initialise une liste vide destinée à contenir les données de violations chargées depuis le fichier JSON local.

```
28 # Load the violations database
```

→ Commentaire indiquant que le bloc suivant charge la base de données des infractions.

```
29 try:
```

→ Démarre un bloc pour intercepter les erreurs de chargement du fichier de violations (absence de fichier, JSON corrompu).

```
30     with open('vehicle_violations.json', 'r') as file:
```

→ Ouvre en lecture le fichier JSON local contenant les infractions déjà enregistrées pour certains véhicules, qui sert de base de comparaison pour les plaques détectées dans la vidéo.

```
31         data = json.load(file)
```

→ Charge le contenu JSON du fichier et le convertit en structure Python (liste de dictionnaires) dans la variable data.

32 except FileNotFoundError:

→ Intercepte l'erreur si le fichier de violations n'existe pas à l'emplacement attendu.

33 st.error(" 'vehicle_violations.json' not found.")

→ Affiche un message d'erreur dans l'interface Streamlit (encadré rouge) signalant l'absence du fichier.

34 except json.JSONDecodeError:

→ Intercepte l'erreur si le contenu du fichier n'est pas un JSON syntaxiquement valide.

35 st.error(" JSON format error in 'vehicle_violations.json'.")

→ Affiche un message d'erreur Streamlit signalant un format JSON invalide.

37 # Send alert via Twilio

→ Commentaire indiquant que la fonction suivante envoie une alerte via Twilio.

38 def send_alert(plate_info: str):

→ Définit la fonction qui envoie un message d'alerte SMS via Twilio ; cette fonction est conçue pour être utilisée comme "tool" (outil) par l'agent défini plus bas, c'est-à-dire une fonction Python que le modèle de langage pourra appeler lui-même quand il jugera nécessaire d'envoyer réellement l'alerte.

39 st.info(f" Sending alert: {plate_info}")

→ Affiche dans l'interface Streamlit un message informatif annonçant l'envoi de l'alerte.

40 message = client.messages.create(

→ Débute l'envoi d'un message via l'API Twilio, l'appel qui déclenche réellement l'envoi du SMS.

41 from_=os.getenv('TWILIO_FROM_NUMBER'),

→ Précise le numéro de téléphone expéditeur du message (le numéro Twilio configuré pour ce compte), lu depuis l'environnement.

42 to=os.getenv('TWILIO_TO_NUMBER'),

→ Précise le numéro de téléphone destinataire du message (par exemple celui d'un agent de sécurité ou d'un service concerné), lu depuis l'environnement.

43 body=plate_info

→ Définit le contenu textuel du message d'alerte envoyé.

44)

→ Ferme l'appel de création et d'envoi du message Twilio.

45 return message.sid

→ Retourne le SID (identifiant unique) du message envoyé par Twilio, une confirmation traçable de l'envoi.

47 # Agent setup

→ Commentaire indiquant que le bloc suivant configure l'agent d'alerte.

48 agent = Agent(

→ Débute la déclaration de l'agent chargé de recevoir les données d'infraction, formater le message d'alerte, et l'envoyer via l'outil send_alert.

49 model=OpenAIChat(id='gpt-4o'),

→ Configure l'agent pour utiliser le modèle OpenAI gpt-4o (une version complète, plus capable que gpt-4o-mini) via le wrapper agno OpenAIChat.

50 instructions=dedent("""

→ Débute la chaîne multi-lignes contenant les instructions système (le prompt) de l'agent.

51 You are a license plate alert system. You receive JSON input with full violation data for a detected vehicle.

→ Ligne de texte des instructions décrivant le rôle de système d'alerte de plaques d'immatriculation.

53 Your task is:

→ Ligne de texte des instructions introduisant la description de la tâche à accomplir.

54 1. Extract the actual values from the JSON input:

→ Ligne de texte des instructions demandant d'extraire les valeurs du JSON reçu.

55 - license_plate

→ Ligne de texte des instructions listant le champ license_plate à extraire.

56 - crime

→ Ligne de texte des instructions listant le champ crime à extraire.

57 - date

→ Ligne de texte des instructions listant le champ date à extraire.

58 - location

→ Ligne de texte des instructions listant le champ location à extraire.

59 - severity

→ Ligne de texte des instructions listant le champ severity à extraire.

60 - fine_amount

→ Ligne de texte des instructions listant le champ fine_amount à extraire.

62 2. Format a single alert message exactly like this:

→ Ligne de texte des instructions demandant de formater un unique message d'alerte selon un modèle précis, pour garantir un format de SMS cohérent quelle que soit l'infraction détectée.

63 " ALERT: Vehicle [license_plate] detected on [location] | VIOLATION RECORD: [crime] | Original incident: [date] | Severity: [severity] | Fine: £[fine_amount] | Vehicle with known violation history spotted"

→ Ligne de texte fournissant le gabarit exact du message d'alerte à produire.

65 3. Use the `send\_alert` tool to send this message as a string.

→ Ligne de texte des instructions demandant d'utiliser l'outil send_alert pour envoyer le message : c'est l'instruction clé qui indique explicitement au modèle qu'il doit appeler la fonction outil plutôt que de simplement décrire ce qu'il ferait, exploitant le mécanisme de function calling (appel d'outil) sur lequel repose l'agent.

67 Return a JSON response in this format:

→ Ligne de texte des instructions introduisant le format de la réponse JSON attendue en cas de succès.

68 {

→ Ligne de texte des instructions ouvrant l'objet JSON du format de succès.

69 "status": "success",

→ Ligne de texte des instructions indiquant le champ status à success dans le format attendu.

70 "message": "Alert sent for license plate [license_plate]",

→ Ligne de texte des instructions indiquant le champ message avec la plaque concernée.

71 "alert_sent": "[full_alert_message]"

→ Ligne de texte des instructions indiquant le champ contenant le message d'alerte complet.

72 }

→ Ligne de texte des instructions fermant l'objet JSON du format de succès.

73 If anything fails, return:

→ Ligne de texte des instructions introduisant le format de réponse JSON en cas d'échec.

74 {

→ Ligne de texte des instructions ouvrant l'objet JSON du format d'échec.

75 "status": "failed",

→ Ligne de texte des instructions indiquant le champ status à failed dans le format d'échec.

76 "message": "Failed to send alert",

→ Ligne de texte des instructions indiquant le message d'échec générique.

77 "error": "[error_details]"

→ Ligne de texte des instructions indiquant le champ destiné aux détails de l'erreur.

78 }

→ Ligne de texte des instructions fermant l'objet JSON du format d'échec.

80 IMPORTANT:

→ Ligne de texte des instructions introduisant les remarques importantes finales.

81 - Do NOT invent or guess any data. Only use what's provided in the input.

→ Ligne de texte des instructions interdisant d'inventer des données non fournies en entrée : une contrainte de prompt engineering essentielle ici, car une plaque erronée ou une infraction inventée dans une vraie alerte SMS aurait des conséquences concrètes, contrairement au contexte créatif toléré dans d'autres projets.

82 - NEVER use placeholder values like ABC123 or Speeding.

→ Ligne de texte des instructions interdisant l'usage de valeurs d'exemple factices, pour empêcher que le modèle ne recopie par erreur les exemples donnés dans son propre prompt d'entraînement ou de démonstration au lieu des vraies données reçues.

83 """),

→ Ferme la chaîne multi-lignes des instructions et l'appel à dedent.

84 tools=[send_alert]

→ Fournit à l'agent la fonction send_alert comme outil disponible : c'est ce paramètre qui active concrètement le fonction calling, en exposant au modèle la signature de la fonction Python pour qu'il puisse décider de l'invoquer avec les bons arguments pendant son raisonnement.

85)

→ Ferme l'appel de construction de l'agent.

87 # Lookup violation

→ Commentaire indiquant que la fonction suivante recherche une infraction correspondant à une plaque.

88 def plate_detection(plate: str):

→ Définit la fonction qui recherche, dans la base de données locale chargée en mémoire, les données d'infraction associées à une plaque détectée par le pipeline de vision.

89 for d in data:

→ Parcourt chaque enregistrement d'infraction chargé depuis le fichier JSON.

90 if plate == d['license_plate'] and plate not in DONE_PLATES:

→ Vérifie que la plaque correspond à l'enregistrement et n'a pas déjà été traitée (présente dans DONE_PLATES), pour éviter de redéclencher une alerte déjà envoyée pour le même véhicule.

91 return d

→ Retourne l'enregistrement d'infraction correspondant à la plaque trouvée.

92 return False

→ Retourne False si aucune infraction correspondante non traitée n'a été trouvée.

94 # Display + process detected plates

→ Commentaire indiquant que la fonction suivante affiche et traite les plaques détectées.

95 def my_sink(result, video_frame):

→ Définit la fonction de callback ("sink", littéralement point de collecte) appelée automatiquement par InferencePipeline à chaque frame traité. C'est le patron de conception "callback" typique des pipelines de traitement de flux vidéo : au lieu d'écrire une boucle manuelle sur les frames, on fournit une fonction que la bibliothèque appelle elle-même pour chaque résultat produit.

96 if video_frame.frame_id == MAX_FRAMES:

→ Vérifie si le numéro de la frame courante atteint la limite maximale définie plus haut.

97 pipeline.terminate()

→ Arrête le pipeline d'inférence en cours d'exécution une fois la limite de frames atteinte.

98 return

→ Quitte immédiatement la fonction de callback après avoir arrêté le pipeline.

100 frame_plates = set()

→ Initialise un ensemble (set) vide destiné à stocker les plaques détectées dans la frame courante ; un set élimine automatiquement les doublons, utile si la même plaque est détectée plusieurs fois dans une seule frame.

101 for group in result['open_ai']:

→ Parcourt les groupes de résultats renvoyés par l'étape du workflow Roboflow qui utilise un modèle OpenAI (vraisemblablement de la vision, pour lire le texte de la plaque détectée par les étapes précédentes du workflow).

102 for res in group:

→ Parcourt chaque résultat individuel à l'intérieur d'un groupe.

103 if res.get('output'):

→ Vérifie que le résultat contient un champ output non vide (le texte de plaque lu par le modèle).

104 frame_plates.add(res['output'].strip())

→ Ajoute à l'ensemble le texte de la plaque détectée, débarrassé des espaces superflus en début/fin de chaîne (strip).

106 for plate in frame_plates:

→ Parcourt chaque plaque unique détectée dans la frame courante.

107 plate_info = plate_detection(plate)

→ Recherche les données d'infraction correspondant à la plaque détectée, via la fonction définie plus haut.

108 if plate_info:

→ Vérifie qu'une infraction correspondante a bien été trouvée pour cette plaque.

109 if plate_info['license_plate'] not in DONE_PLATES:

→ Vérifie que cette plaque n'a pas déjà été traitée précédemment, deuxième garde-fou contre les alertes en double après celui déjà présent dans plate_detection.

110 DONE_PLATES.append(plate_info['license_plate'])

→ Ajoute la plaque à la liste des plaques déjà traitées, avant même de savoir si l'agent réussira à envoyer l'alerte, pour empêcher tout nouvel essai sur cette plaque dans les frames suivantes.

111 st.success(f" Match found: {plate}")

→ Affiche dans l'interface Streamlit un message confirmant la correspondance trouvée entre la plaque détectée et la base d'infractions.

112 agent_response = agent.run(json.dumps(plate_info))

→ Exécute l'agent avec les données d'infraction converties en chaîne JSON (json.dumps sérialise le dictionnaire Python en texte), puisque l'agent attend une entrée textuelle et non un objet Python.

```

113         final_agent_output = None
    → Initialise à None la variable destinée à contenir la sortie finale de l'agent, pour pouvoir détecter le cas où aucune réponse exploitable n'aurait été trouvée.

114         for msg in reversed(agent_response.messages):
    → Parcourt les messages de la réponse de l'agent en partant du dernier (reversed) : un agent avec des outils peut produire plusieurs messages intermédiaires (appel d'outil, résultat d'outil) avant son message de conclusion final, donc parcourir à l'envers permet de trouver directement la dernière réponse de l'assistant sans avoir à filtrer tout l'historique.

115             if msg.role == 'assistant' and msg.content:
    → Vérifie que le message provient de l'assistant (et non d'un outil ou du système) et contient du texte.

116                 final_agent_output = msg.content
    → Enregistre le contenu de ce message comme sortie finale de l'agent.

117         break
    → Interrompt la boucle dès que le dernier message de l'assistant a été trouvé, inutile de continuer à parcourir le reste de l'historique.

118     if final_agent_output:
    → Vérifie qu'une sortie finale de l'agent a bien été obtenue.

119     try:
    → Démarre un bloc pour intercepter une erreur de décodage JSON de la sortie, au cas où le modèle n'aurait pas respecté le format JSON demandé dans ses instructions.

120         st.json(json.loads(final_agent_output))
    → Convertit la sortie textuelle de l'agent en objet JSON puis l'affiche dans l'interface Streamlit avec st.json, qui offre un rendu visuel structuré et dépliant du JSON plutôt qu'un simple bloc de texte.

121     except json.JSONDecodeError as e:
    → Intercepte l'erreur si la sortie de l'agent n'est pas un JSON valide, un cas possible malgré les instructions strictes données au modèle.

122         st.error(f" Agent returned non-JSON content:\n{final_agent_output}")
    → Affiche un message d'erreur Streamlit avec le contenu non JSON reçu, pour permettre de diagnostiquer visuellement ce que le modèle a réellement produit.

123     else:
    → Cas où aucune sortie de l'assistant n'a été trouvée dans la réponse de l'agent.

124         st.warning(" No assistant response received.")
    → Affiche un avertissement Streamlit signalant l'absence de réponse de l'assistant.

128 st.title(" License Plate Violation Detection")
    → Affiche le titre principal de la page Streamlit de l'application.

130 if st.button("■ Start Detection from Video"):
    → Vérifie si l'utilisateur a cliqué sur le bouton de démarrage de la détection vidéo, point de départ manuel de tout le pipeline.

131     with st.spinner("Processing video..."):
    → Affiche un indicateur de chargement pendant toute la durée du traitement de la vidéo par le pipeline.

132     pipeline = InferencePipeline.init_with_workflow(
    → Débute la création du pipeline d'inférence Roboflow, configuré pour exécuter un workflow prédéfini (une chaîne de modèles de vision, ici probablement détection de véhicule, détection de plaque, puis lecture du texte) plutôt qu'un unique modèle isolé.

```

133 `api_key=roboflow_key,`

→ Fournit la clé API Roboflow nécessaire à l'exécution du workflow hébergé sur la plateforme.

134 `workspace_name="tp-qpwtk",`

→ Précise le nom de l'espace de travail (workspace) Roboflow contenant le workflow, l'équivalent d'un espace de projet dans lequel sont organisés les modèles et workflows d'un compte Roboflow.

135 `workflow_id="custom-workflow",`

→ Précise l'identifiant du workflow Roboflow personnalisé à exécuter, celui défini dans l'interface Roboflow pour cette chaîne de détection spécifique.

136 `video_reference='video/traffic-3.mp4',`

→ Indique le chemin du fichier vidéo local à analyser image par image.

137 `max_fps=30,`

→ Limite la fréquence de traitement du pipeline à trente images par seconde, un plafond qui évite de surcharger le matériel si la vidéo source a un débit plus élevé.

138 `on_prediction=my_sink`

→ Définit la fonction `my_sink` comme callback appelé automatiquement par `InferencePipeline` à chaque nouvelle prédiction produite, le point de connexion entre la bibliothèque Roboflow et la logique métier du fichier.

139 `)`

→ Ferme l'appel de création du pipeline d'inférence.

140 `pipeline.start()`

→ Démarre l'exécution du pipeline d'inférence sur la vidéo, généralement dans un thread interne géré par la bibliothèque.

141 `pipeline.join()`

→ Bloque le thread principal jusqu'à la fin du traitement du pipeline (méthode "join", classique des API de threading, qui attend la terminaison du traitement avant de continuer le script).

142 `st.success(" Finished processing video.")`

→ Affiche un message de succès Streamlit signalant la fin du traitement de la vidéo.

Projet 27 — 27-ImportExportSpecialist

Conformite import/export UK-Maroc, verification et analyse de risque

Objectif

Vérifie la conformité réglementaire d'une transaction commerciale entre le Royaume-Uni et le Maroc en la confrontant à des textes de loi import/export indexés dans une base de connaissances, puis évalue et explique le niveau de risque de la transaction, via une interface web.

Code source, ligne par ligne

27-ImportExportSpecialist/import_export_agent.py

Définit les fonctions d'OCR et de lecture des lois, ainsi que les quatre agents agno (traduction, spécialiste, vérification, risques) du système de conformité import-export.

190 lignes de code annotées ci-dessous.

1 `import os`

→ Importe le module standard ``os``, qui permet ici de lire des variables d'environnement (les clés API) et de vérifier l'existence de fichiers sur le disque. C'est la façon standard en Python d'interagir avec le système de fichiers sans dépendre d'une bibliothèque tierce.

2 `from textwrap import dedent`

→ Importe ``dedent``, un utilitaire du module standard ``textwrap``. Quand on écrit une chaîne de caractères multi-lignes directement dans du code Python indenté (par exemple à l'intérieur d'un appel de fonction), chaque ligne hérite de l'indentation du code environnant ; ``dedent`` retire cette indentation commune pour que le texte envoyé au modèle de langage soit propre, sans espaces parasites en début de ligne qui pourraient perturber sa lecture.

3 `from dotenv import load_dotenv`

→ Importe ``load_dotenv`` de la bibliothèque ``python-dotenv``. Le principe : au lieu d'écrire les clés API en clair dans le code source (risque de sécurité si le code est partagé ou versionné), on les place dans un fichier ``.env`` local non partagé, et cette fonction les charge dans l'environnement du processus au démarrage.

4 `from mistralai.client import Mistral`

→ Importe le client officiel de l'API Mistral AI. Il servira ici uniquement pour son service d'OCR (reconnaissance optique de caractères) hébergé dans le cloud, qui convertit des PDF scannés ou natifs en texte structuré.

5 `from agno.models.groq import Groq`

→ Importe la classe ``Groq`` du framework agno. Agno est un framework Python open-source pour construire des agents IA : il orchestre l'appel au modèle de langage, la gestion des instructions, l'utilisation d'outils externes et la récupération de connaissances. Cette classe ``Groq`` est un adaptateur qui permet à agno de dialoguer avec l'API d'inférence de Groq plutôt qu'avec OpenAI directement.

6 `from agno.agent import Agent`

→ Importe la classe ``Agent``, brique centrale d'agno. Un « agent » ici n'est pas juste un appel ponctuel à un modèle de langage : c'est un objet configurable qui associe un modèle, des instructions systèmes, éventuellement des outils (recherche web, fichiers, etc.) et une source de connaissances, pour produire un comportement spécialisé et répétable.

7 `from agno.tools.duckduckgo import DuckDuckGoTools`

→ Importe l'outil de recherche web DuckDuckGo fourni par agno. Un « tool » (outil) dans ce contexte est une fonction que le modèle de langage peut décider d'appeler lui-même pendant sa réflexion (function calling) pour aller chercher une information qu'il ne connaît pas ou qui a pu changer depuis son entraînement — ici, une recherche web classique sans clé API à fournir.

9 # Load API keys from .env

→ Commentaire indiquant que les clés API vont être chargées depuis le fichier .env.

10 load_dotenv()

→ Exécute le chargement effectif : lit le fichier `.env` situé dans le répertoire du projet et copie chacune de ses variables (GROQ_API_KEY, MISTRAL_API_KEY, etc.) dans l'environnement du processus Python, comme si elles avaient été définies avec `os.environ`.

11 # Le script original utilisait OpenAI (gpt-4o/gpt-4o-mini) ; remplacé par Groq

→ Commentaire expliquant que le modèle OpenAI d'origine a été remplacé par Groq.

12 # pour réutiliser la clé GROQ_API_KEY déjà disponible.

→ Suite du commentaire : Groq est ici choisi non pas pour une raison technique de qualité de réponse, mais parce qu'il permet d'appeler des modèles open-weight performants (comme les modèles gpt-oss) via une API rapide et gratuite dans une certaine limite, en réutilisant une clé déjà en possession du projet plutôt que d'en payer une nouvelle chez OpenAI.

13 groq_key = os.getenv("GROQ_API_KEY")

→ Récupère la valeur de la variable d'environnement GROQ_API_KEY (chargée juste avant par `load_dotenv`) et la stocke dans `groq_key`, prête à être passée à chaque agent utilisant Groq.

14 mistral_key = os.getenv("MISTRAL_API_KEY")

→ Récupère la valeur de la variable d'environnement MISTRAL_API_KEY et la stocke dans `mistral_key`, qui servira à authentifier les appels au service d'OCR Mistral.

16 # Initialize Mistral client

→ Commentaire annonçant l'initialisation du client Mistral.

17 client = Mistral(api_key=mistral_key)

→ Crée une instance du client Mistral authentifiée avec la clé API récupérée. Cet objet `client` sera réutilisé pour tous les appels au service d'OCR (upload de fichier, obtention d'URL signée, lancement du traitement).

19 # OCR PDF to Markdown

→ Commentaire annonçant la fonction qui convertit un PDF en Markdown via OCR.

20 def ocr_pdf(pdf_path, pdf_name):

→ Définit la fonction `ocr_pdf`, qui encapsule tout le pipeline OCR : upload du fichier, traitement, puis écriture du résultat en Markdown. L'OCR (Optical Character Recognition, reconnaissance optique de caractères) est la technique qui transforme une image ou un PDF (pixels) en texte exploitable (caractères), ce qui est indispensable ici puisque les lois d'import/export sont fournies sous forme de PDF et que le modèle de langage a besoin de texte brut pour les lire.

21 if not os.path.exists(pdf_path):

→ Vérifie si le fichier PDF indiqué par `pdf_path` n'existe pas sur le disque, pour éviter de tenter un appel API sur un fichier absent.

22 return False

→ Arrête la fonction et renvoie False si le fichier PDF n'a pas été trouvé, permettant à l'appelant de détecter l'échec sans lever d'exception.

24 uploaded_pdf = client.files.upload(

→ Démarre l'appel qui téléverse le fichier PDF vers l'infrastructure Mistral. C'est une étape obligatoire du service cloud : le document doit d'abord résider sur les serveurs de Mistral avant de pouvoir être traité par le moteur OCR.

25 file={"file_name": pdf_path, "content": open(pdf_path, "rb")},

→ Fournit le nom du fichier et son contenu binaire (ouvert en mode lecture binaire `"rb"`, nécessaire car un PDF n'est pas du texte) comme paramètre `file` de l'appel d'upload.

26 purpose="ocr"

→ Indique à l'API que le fichier est téléversé dans le but de faire de l'OCR, ce qui conditionne le traitement que Mistral appliquera ensuite au fichier stocké.

27)
→ Ferme l'appel à `client.files.upload` commencé plus haut.

29 `signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)`
→ Demande une URL signée temporaire permettant d'accéder au fichier téléversé via son identifiant. Une URL signée est un lien à durée de vie limitée qui donne un accès contrôlé à une ressource privée, sans exposer publiquement le fichier ni les identifiants du compte.

31 `ocr_response = client.ocr.process(`
→ Démarre l'appel qui lance le traitement OCR proprement dit du document sur l'API Mistral.

32 `model="mistral-ocr-latest",`
→ Spécifie le modèle OCR à utiliser, ici la version la plus récente de `mistral-ocr` (un modèle spécialisé dans l'extraction de texte structuré, distinct des modèles de conversation classiques de Mistral).

33 `document={"type": "document_url", "document_url": signed_url.url},`
→ Indique que le document à traiter est désigné par l'URL signée obtenue précédemment plutôt que par un envoi direct du binaire.

34 `include_image_base64=True`
→ Demande à ce que les images du document soient également renvoyées encodées en base64, ce qui permettrait de les réintégrer dans le Markdown de sortie si besoin (même si ce script ne les exploite pas ensuite).

35)
→ Ferme l'appel à `client.ocr.process` commencé plus haut.

37 `with open(f"Documents/{pdf_name}.md", "w") as f:`
→ Ouvre en écriture un fichier Markdown nommé d'après `pdf_name` dans le dossier `Documents`, afin de persister le résultat de l'OCR sur disque et éviter de refaire l'appel payant à chaque exécution.

38 `f.write("\n".join([page.markdown for page in ocr_response.pages]))`
→ Concatène le Markdown de chaque page OCRisée (l'API renvoie le document page par page) avec des sauts de ligne, puis écrit le tout dans le fichier — reconstituant ainsi le document complet en un seul fichier texte structuré.

40 `return True`
→ Renvoie `True` pour signaler que l'OCR et l'écriture du fichier se sont déroulés avec succès.

42 # Load OCR outputs
→ Commentaire annonçant les fonctions qui chargent les résultats OCR déjà générés.

43 `def export_law_document(agent, query, num_documents=None, **kwargs):`
→ Définit une fonction `retrieve` (récupérateur) qui lit le document de loi d'exportation. Sa signature `(agent, query, num_documents=None, **kwargs)` n'est pas arbitraire : c'est le contrat attendu par `agno` pour ``knowledge_retrieve`` personnalisé, qui est appelé automatiquement par l'agent avec la requête de l'utilisateur et éventuellement d'autres paramètres.

44 `with open("Documents/uk-export-law.md", "r") as f:`
→ Ouvre en lecture le fichier Markdown contenant la loi d'exportation britannique, généré au préalable par la fonction ``ocr_pdf``.

45 `return f.read()`
→ Lit et renvoie l'intégralité du contenu du fichier ouvert.

47 `def import_law_document(agent, query, num_documents=None, **kwargs):`
→ Définit une fonction `retrieve` qui lit le document de loi d'importation, avec la même signature attendue par `agno` que la fonction précédente.

48 `with open("Documents/morocco-import-law.md", "r") as f:`
→ Ouvre en lecture le fichier Markdown contenant la loi d'importation marocaine.

49 return f.read()

→ Lit et renvoie l'intégralité du contenu du fichier ouvert.

51 # knowledge_retriever n'accepte qu'une seule fonction callable (pas une liste) :

→ Commentaire expliquant la contrainte technique qui motive la fonction combinée suivante.

52 # on combine les deux documents en un seul retriever. Les documents complets

→ Suite du commentaire, précisant que les deux documents sont fusionnés dans un seul retriever.

53 # (~180 Ko à eux deux) dépassaient largement la limite Groq (8000 tokens/minute

→ Suite du commentaire : ce chiffre illustre une contrainte réelle de l'inférence LLM — les fournisseurs comme Groq limitent le débit de tokens (unités de texte, environ 4 caractères en moyenne) traités par minute, surtout sur les offres gratuites, et un texte de loi complet consomme ce quota à lui seul.

54 # en tier gratuit) rien qu'en contexte, avant même la génération de la réponse ;

→ Suite du commentaire, précisant que la limite est dépassée par le seul contexte envoyé au modèle, avant même qu'il ne produise sa propre réponse (qui consomme elle aussi des tokens).

55 # on les tronque à un extrait représentatif.

→ Fin du commentaire, annonçant que les documents seront tronqués à un extrait représentatif plutôt que d'être envoyés en entier — un compromis pragmatique entre exhaustivité et respect des limites du fournisseur.

56 MAX_LAW_EXCERPT_CHARS = 4000

→ Définit la constante fixant à 4000 caractères la longueur maximale d'extrait conservé par document juridique, afin de rester sous la limite de tokens Groq tout en conservant un extrait substantiel de chaque loi.

58 def combined_law_documents(agent, query, num_documents=None, **kwargs):

→ Définit le retriever combiné effectivement utilisé par l'agent spécialiste (voir ligne 145), avec la même signature attendue par agno que les deux fonctions précédentes. C'est ce mécanisme de « retrieval » qui constitue la brique RAG (Retrieval-Augmented Generation) du projet : plutôt que de laisser le modèle deviner le contenu des lois à partir de sa mémoire d'entraînement (avec le risque d'hallucination et d'obsolescence), on va chercher les textes réels et on les injecte dans le prompt pour que la réponse s'appuie sur des sources vérifiables.

59 return [

→ Commence la construction et le renvoi d'une liste contenant les deux extraits de documents, format attendu par agno pour un retriever (une liste de morceaux de contexte).

60 export_law_document(agent, query)[:MAX_LAW_EXCERPT_CHARS],

→ Récupère le document d'exportation puis le tronque aux 4000 premiers caractères pour limiter la consommation de tokens envoyés au modèle.

61 import_law_document(agent, query)[:MAX_LAW_EXCERPT_CHARS],

→ Récupère le document d'importation puis le tronque aux 4000 premiers caractères pour la même raison.

62]

→ Ferme la liste renvoyée par la fonction, contenant les deux extraits tronqués qui seront injectés dans le contexte du modèle par agno.

64 # Translation Agent

→ Commentaire annonçant la définition de l'agent de traduction.

65 translation_agent = Agent(

→ Commence la création de l'agent nommé translation_agent, instance de la classe Agent d'agno. Son rôle est isolé (traduire) plutôt que noyé dans un agent généraliste : c'est le principe de décomposition en agents spécialisés, chacun avec un rôle et des instructions précis, plus fiable qu'un unique agent chargé de tout faire.

66 name="Translation Agent",

→ Définit le nom affiché de cet agent comme "Translation Agent", utile pour le débogage et les logs (notamment avec show_tool_calls ou debug_mode utilisés ailleurs dans le projet).

67 `model=Groq(id="openai/gpt-oss-120b", api_key=groq_key),`
 → Configure le modèle utilisé par cet agent : `gpt-oss-120b`, un modèle open-weight de 120 milliards de paramètres, exécuté via l'infrastructure d'inférence de Groq (connue pour sa très faible latence grâce à du matériel dédié, les LPU) plutôt que via OpenAI directement, et authentifié avec la clé Groq.

68 `instructions=dedent("""`
 → Commence la définition des instructions de l'agent sous forme de chaîne multi-lignes désindentée par `dedent` (voir import ligne 2). Les « instructions » forment le prompt système : le texte qui définit durablement le rôle et le comportement de l'agent, distinct du message ponctuel de l'utilisateur.

69 `You are an expert in translation. You are given a text in markdown format in French and you need to translate it to English.`
 → Ligne d'instruction indiquant à l'agent son rôle de traducteur du français vers l'anglais.

70 `IMPORTANT: Keep the same structure and content of the original text.`
 → Ligne d'instruction demandant à l'agent de préserver la structure et le contenu du texte original, pour éviter que la traduction ne reformule ou omette des clauses juridiques.

71 `"""),`
 → Ferme la chaîne d'instructions dedent et l'appel dedent, puis termine l'argument instructions.

72 `)`
 → Ferme l'appel au constructeur Agent, terminant la définition de translation_agent.

74 `# Export-Import Compliance Expert`
 → Commentaire annonçant la définition de l'agent spécialiste de conformité import-export.

75 `specialist_agent = Agent(`
 → Commence la création de l'agent nommé specialist_agent, celui qui exploite le RAG mis en place plus haut pour produire le rapport de conformité principal.

76 `name="Specialist Agent",`
 → Définit le nom affiché de cet agent comme "Specialist Agent".

77 `# Le retriever combiné injecte le texte intégral des deux lois dans le contexte,`
 → Commentaire expliquant que le retriever combiné insère le texte des deux lois dans le contexte du modèle.

78 `# ce qui dépasse la limite Groq du modèle 120B (8000 tokens/minute) ; le modèle`
 → Suite du commentaire, précisant que cela dépasse la limite de tokens du modèle 120B.

79 `# 20B a un quota plus élevé sur le tier gratuit.`
 → Fin du commentaire, justifiant le choix du modèle 20B : c'est un arbitrage taille/débit — un modèle plus petit (20 milliards de paramètres contre 120) bénéficie d'un quota de tokens/minute plus généreux chez Groq en offre gratuite, ce qui est nécessaire ici puisque cet agent reçoit un contexte volumineux via le RAG.

80 `model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),`
 → Configure le modèle Groq gpt-oss-20b utilisé par cet agent, authentifié avec la clé Groq, choisi spécifiquement pour son quota de tokens plus élevé comme expliqué dans le commentaire ci-dessus.

81 `instructions=dedent("""`
 → Commence la définition des instructions de l'agent spécialiste sous forme de chaîne multi-lignes désindentée.

82 `You are an Export-Import Compliance Expert with access to two knowledge sources:`
 → Ligne d'instruction présentant le rôle d'expert en conformité export-import et ses deux sources de connaissance — ces « sources » sont précisément les documents que le retriever RAG va injecter dans le contexte.

83 `- The official UK export law document for electronics (including licences, restrictions, taxes, fees).`

→ Ligne d'instruction décrivant la première source, le document officiel de loi d'exportation britannique.

84 - The official Moroccan import law document for electronics (including permits, quantity limits, import duties, VAT, prohibited items, and customs procedures).

→ Ligne d'instruction décrivant la seconde source, le document officiel de loi d'importation marocaine.

86 You also have access to GoogleSearchTools() to find up-to-date, authoritative data when it is missing or unclear.

→ Ligne d'instruction mentionnant l'accès à un outil de recherche pour compléter les données manquantes ; en pratique l'agent est configuré plus bas (ligne 146) avec DuckDuckGoTools plutôt que GoogleSearchTools, le prompt gardant une référence générique à « un outil de recherche » qui n'a pas été mise à jour lors du changement d'outil.

88 Your task is: given detailed product info (type, quantity, raw materials) and the shipment route (UK to Morocco), provide a **full, precise, step-by-step explanation** of:

→ Ligne d'instruction énonçant la tâche principale, produire une explication détaillée étape par étape.

90 1. **UK export process**:

→ Ligne d'instruction introduisant la première partie attendue de la réponse, le processus d'exportation britannique.

91 - Exact licence requirements by name and code,

→ Ligne d'instruction demandant les exigences précises de licence avec nom et code.

92 - Clear restrictions on product types or materials,

→ Ligne d'instruction demandant de préciser les restrictions sur les types de produits ou matériaux.

93 - Specific export taxes, fees, or levies with exact amounts or percentages,

→ Ligne d'instruction demandant les taxes et frais d'exportation avec montants ou pourcentages exacts.

94 - Necessary documentation, including official form names or codes,

→ Ligne d'instruction demandant la liste des documents nécessaires avec noms ou codes officiels des formulaires.

95 - Compliance rules and penalties with numbers and legal references.

→ Ligne d'instruction demandant les règles de conformité et pénalités avec chiffres et références légales.

97 2. **Moroccan import process**:

→ Ligne d'instruction introduisant la seconde partie attendue, le processus d'importation marocain.

98 - Precise import permits and certifications required,

→ Ligne d'instruction demandant les permis et certifications d'importation requis.

99 - Exact quantity restrictions or exemptions,

→ Ligne d'instruction demandant les restrictions ou exemptions de quantité exactes.

100 - Detailed import duties and VAT rates with percentages or fixed amounts,

→ Ligne d'instruction demandant le détail des droits de douane et taux de TVA.

101 - List of prohibited or restricted items by name and code,

→ Ligne d'instruction demandant la liste des articles interdits ou restreints avec nom et code.

102 - Required documents with official titles,

→ Ligne d'instruction demandant les documents requis avec leurs titres officiels.

103 - Customs clearance steps with timelines and fees.

→ Ligne d'instruction demandant les étapes de dédouanement avec délais et frais.

105 Use only **facts and figures** extracted from your knowledge base or from live, authoritative sources. Do not guess or be vague.

→ Ligne d'instruction imposant de n'utiliser que des faits vérifiés et d'éviter les approximations — c'est le cœur de l'approche RAG : contraindre le modèle à s'ancrer sur les documents fournis plutôt que sur des connaissances générales potentiellement

fausses ou périmées.

107 If any data is missing in your knowledge, immediately perform a `GoogleSearchTools()` lookup for official government or customs sites and incorporate exact data found.

→ Ligne d'instruction demandant une recherche web complémentaire en cas de donnée manquante, recherche qui sera en pratique effectuée via l'outil DuckDuckGo configuré à l'agent.

109 Present the response in clear, professional language, with no filler text or disclaimers.

→ Ligne d'instruction demandant un style de réponse clair, professionnel, sans remplissage ni avertissements.

111 Provide the output as a flowing narrative, but include concrete numbers, names, codes, and references wherever applicable.

→ Ligne d'instruction demandant une réponse rédigée sous forme narrative tout en incluant des données concrètes.

113 The goal is to give the user a complete, actionable export-import compliance report with real numbers and official requirements.

→ Ligne d'instruction résumant l'objectif final, un rapport de conformité complet et actionnable.

115 If data is unavailable, explicitly state that you researched and could not find updated official info.

→ Ligne d'instruction demandant de signaler explicitement l'absence de donnée trouvée après recherche, pour éviter que l'agent n'invente une réponse plausible mais fausse.

116 `"""),`

→ Ferme la chaîne d'instructions dedent et l'appel dedent, terminant l'argument instructions.

117 `expected_output=dedent("""`

→ Commence la définition du gabarit de sortie attendu sous forme de chaîne multi-lignes désindentée. Le paramètre ``expected_output`` d'agno fournit au modèle un moule Markdown à remplir, ce qui rend la sortie prévisible et facilement réutilisable en aval (affichage, conversion, etc.) plutôt que laissée à la libre forme du modèle.

118 `### UK Export Process`

→ Ligne du gabarit définissant le titre de section Markdown pour le processus d'exportation britannique.

120 `[Explain if an export licence is required for the product. Specify licence type and official code if available.]`

→ Ligne du gabarit indiquant l'espace réservé pour expliquer l'exigence de licence d'exportation.

121 `[Mention any restrictions on product types, raw materials, or dual-use classifications found in the UK export law document.]`

→ Ligne du gabarit indiquant l'espace réservé pour les restrictions de produits ou matériaux.

122 `[Provide specific export taxes or fees applicable, with exact amounts or percentages, citing the document.]`

→ Ligne du gabarit indiquant l'espace réservé pour les taxes et frais d'exportation.

123 `[Describe required paperwork and compliance steps before shipment.]`

→ Ligne du gabarit indiquant l'espace réservé pour les démarches et étapes de conformité avant expédition.

125 `### Moroccan Import Process`

→ Ligne du gabarit définissant le titre de section Markdown pour le processus d'importation marocain.

127 `[Detail import permits, licences, or certifications required for the product according to the Moroccan import law document.]`

→ Ligne du gabarit indiquant l'espace réservé pour les permis et certifications d'importation.

128 `[State any quantity limits or restrictions, quoting exact figures or thresholds.]`

→ Ligne du gabarit indiquant l'espace réservé pour les limites de quantité.

129 `[Specify prohibited items or materials mentioned in the document.]`

→ Ligne du gabarit indiquant l'espace réservé pour les articles ou matériaux interdits.

130 [Give import duties and VAT rates with percentages or fixed amounts, as documented.]

→ Ligne du gabarit indiquant l'espace réservé pour les droits de douane et taux de TVA.

131 [List all required customs documents and procedures.]

→ Ligne du gabarit indiquant l'espace réservé pour la liste des documents et procédures douanières.

133 ### Taxes and Compliance Details

→ Ligne du gabarit définissant le titre de section Markdown pour les détails de taxes et conformité.

135 [Summarize how taxes and fees are calculated and at which stage (export or import) they apply, based on the findings.]

→ Ligne du gabarit indiquant l'espace réservé pour le résumé du calcul des taxes.

136 [Explain compliance obligations and potential penalties for non-compliance extracted from the documents.]

→ Ligne du gabarit indiquant l'espace réservé pour les obligations de conformité et pénalités.

137 [Include any additional practical advice referenced in the documents for smooth customs clearance.]

→ Ligne du gabarit indiquant l'espace réservé pour des conseils pratiques additionnels.

139 ### Summary and Recommendations

→ Ligne du gabarit définissant le titre de section Markdown pour le résumé et les recommandations.

141 [Provide a concise summary of the entire export-import process highlighting key points and actionable next steps based solely on the document contents.]

→ Ligne du gabarit indiquant l'espace réservé pour le résumé final et les prochaines étapes.

142 """),

→ Ferme la chaîne du gabarit dedent et l'appel dedent, terminant l'argument expected_output.

143 knowledge=None,

→ Indique qu'aucune base de connaissance native agno (par exemple une base vectorielle) n'est attachée à cet agent : la récupération de contexte passe entièrement par le `knowledge\_retriever` personnalisé défini plus bas, pas par le système de knowledge base intégré d'agno.

144 search_knowledge=True,

→ Active la recherche dans la base de connaissance, ce qui déclenche l'appel automatique au knowledge_retriever avant chaque réponse — sans ce booléen, la fonction combined_law_documents ne serait jamais invoquée.

145 knowledge_retriever=combined_law_documents,

→ Associe la fonction combined_law_documents comme retriever fournissant le contexte juridique tronqué à l'agent. C'est le point de branchement concret du RAG : à chaque requête, agno appelle cette fonction, récupère les extraits de lois, et les insère dans le prompt envoyé au modèle avant qu'il ne rédige sa réponse.

146 tools=[DuckDuckGoTools()],

→ Fournit à l'agent l'outil de recherche web DuckDuckGo pour compléter les informations manquantes des documents statiques : le RAG couvre les lois déjà OCRisées, tandis que cet outil permet d'aller chercher en direct des données plus récentes ou absentes des PDF (taux de change, mises à jour réglementaires, etc.).

147 compress_tool_results=True,

→ Active la compression des résultats d'outils avant leur insertion dans le contexte du modèle : quand DuckDuckGo renvoie des pages de résultats volumineuses, agno les résume/réduit automatiquement pour économiser des tokens, dans le même esprit que la troncature MAX_LAW_EXCERPT_CHARS appliquée au RAG.

148)

→ Ferme l'appel au constructeur Agent, terminant la définition de specialist_agent.

150 # Agent 4 – Compliance Verification

→ Commentaire annonçant la définition de l'agent de vérification de conformité.

151 verification_agent = Agent(

→ Commence la création de l'agent nommé verification_agent, un agent de relecture qui n'a pas d'accès direct aux lois (pas de retrieveur, pas d'outil) : son rôle est d'auditer le texte produit par specialist_agent, pas de refaire la recherche documentaire.

152 name="Compliance Verification Agent",

→ Définit le nom affiché de cet agent comme "Compliance Verification Agent".

153 model=Groq(id="openai/gpt-oss-120b", api_key=groq_key),

→ Configure le modèle Groq gpt-oss-120b utilisé par cet agent, authentifié avec la clé Groq. Le modèle 120B (plus grand que le 20B) est réutilisable ici car cet agent ne reçoit pas le contexte volumineux des lois, seulement le rapport déjà généré, donc il reste sous la limite de tokens/minute malgré sa taille.

154 instructions=dedent("""

→ Commence la définition des instructions de l'agent de vérification sous forme de chaîne multi-lignes désindentée.

155 You are a regulatory compliance auditor specializing in international trade.

→ Ligne d'instruction présentant le rôle d'auditeur de conformité réglementaire spécialisé en commerce international.

157 Your job is to audit an export-import compliance report that was generated by another expert agent. You must carefully review the content and assess:

→ Ligne d'instruction décrivant la mission d'audit du rapport produit par l'agent spécialiste — cette chaîne d'agents (spécialiste puis vérificateur) illustre un pattern de relecture croisée entre agents IA, où un second modèle contrôle la sortie du premier.

159 1. **Legal consistency**:

→ Ligne d'instruction introduisant le premier critère d'évaluation, la cohérence légale.

160 - Are the legal steps and procedures correctly sequenced?

→ Ligne d'instruction posant la question de l'ordre correct des étapes légales.

161 - Are licenses, permits, and document names aligned with real trade practice?

→ Ligne d'instruction posant la question de la conformité des noms de licences et permis à la pratique réelle.

163 2. **Accuracy of references**:

→ Ligne d'instruction introduisant le second critère d'évaluation, l'exactitude des références.

164 - Are any numbers, codes, rates or penalties properly supported?

→ Ligne d'instruction posant la question du support factuel des chiffres, codes, taux et pénalités.

165 - Are there unsupported claims, vague terms, or approximations?

→ Ligne d'instruction posant la question de l'existence d'affirmations non étayées ou vagues.

167 3. **Completeness**:

→ Ligne d'instruction introduisant le troisième critère d'évaluation, l'exhaustivité.

168 - Are all sections fully covered: UK export, Moroccan import, taxes, and summary?

→ Ligne d'instruction posant la question de la couverture complète de toutes les sections attendues.

169 - Are essential steps or legal documents missing?

→ Ligne d'instruction posant la question de l'absence d'étapes ou documents légaux essentiels.

171 Your output should:

→ Ligne d'instruction introduisant les exigences de format de la sortie de l'agent.

172 - Clearly identify each issue, grouped by section,

→ Ligne d'instruction demandant d'identifier clairement chaque problème, regroupé par section.

173 - Suggest accurate corrections where needed,
→ Ligne d'instruction demandant de proposer des corrections précises si nécessaire.

174 - Confirm when a section is compliant.
→ Ligne d'instruction demandant de confirmer explicitement quand une section est conforme.

176 You must use markdown to organize your response under:
→ Ligne d'instruction imposant l'usage du Markdown pour structurer la réponse selon les titres suivants.

177 - UK Export Section
→ Ligne d'instruction listant le titre de section attendu pour l'export britannique.

178 - Moroccan Import Section
→ Ligne d'instruction listant le titre de section attendu pour l'import marocain.

179 - Taxation Section
→ Ligne d'instruction listant le titre de section attendu pour la fiscalité.

180 - General Observations
→ Ligne d'instruction listant le titre de section attendu pour les observations générales.

182 Be precise, legal, and concise – no filler or speculation.
→ Ligne d'instruction demandant un style précis, juridique et concis, sans remplissage ni spéculation.

183 """),
→ Ferme la chaîne d'instructions dedent et l'appel dedent, terminant l'argument instructions.

184 expected_output=dedent("""
→ Commence la définition du gabarit de sortie attendu de l'agent de vérification.

185 ### UK Export Section
→ Ligne du gabarit définissant le titre de section Markdown pour la vérification de l'export britannique.

186 [or X Summary of findings with details and line references]
→ Ligne du gabarit indiquant l'espace réservé pour le résumé des constats de la section export.

188 ### Moroccan Import Section
→ Ligne du gabarit définissant le titre de section Markdown pour la vérification de l'import marocain.

189 [or X Summary of findings with suggested corrections]
→ Ligne du gabarit indiquant l'espace réservé pour le résumé des constats et corrections de la section import.

191 ### Taxation Section
→ Ligne du gabarit définissant le titre de section Markdown pour la vérification de la fiscalité.

192 [or X Breakdown of duties, VAT accuracy, correction if needed]
→ Ligne du gabarit indiquant l'espace réservé pour le détail des droits et de l'exactitude de la TVA.

194 ### General Observations
→ Ligne du gabarit définissant le titre de section Markdown pour les observations générales.

195 [Highlight formatting, clarity, completeness, and red flags if any]
→ Ligne du gabarit indiquant l'espace réservé pour les remarques sur la forme et les signaux d'alerte.

196 """),
→ Ferme la chaîne du gabarit dedent et l'appel dedent, terminant l'argument expected_output.

197 knowledge=None,

→ Indique qu'aucune base de connaissance native agno n'est attachée à cet agent, cohérent avec le fait qu'il ne fait qu'auditer du texte déjà produit, sans consulter de source externe.

198 `search_knowledge=False,`

→ Désactive la recherche dans une base de connaissance pour cet agent, puisqu'aucun retriever n'est de toute façon défini ici.

199 `tools=[],`

→ Ne fournit aucun outil supplémentaire à cet agent, qui ne travaille que sur le texte reçu en entrée, sans recherche web ni fichier.

200)

→ Ferme l'appel au constructeur Agent, terminant la définition de `verification_agent`.

202 # Agent 5 – Risk Analysis Agent

→ Commentaire annonçant la définition de l'agent d'analyse de risques.

203 `risk_agent = Agent(`

→ Commence la création de l'agent nommé `risk_agent`, troisième maillon de la chaîne après le spécialiste et le vérificateur : il transforme le rapport de conformité en une analyse de risques orientée décision.

204 `name="Risk Analysis Agent",`

→ Définit le nom affiché de cet agent comme "Risk Analysis Agent".

205 `model=Groq(id="openai/gpt-oss-120b", api_key=groq_key),`

→ Configure le modèle Groq gpt-oss-120b utilisé par cet agent, authentifié avec la clé Groq, pour la même raison que `verification_agent` : pas de contexte RAG volumineux à traiter, donc le modèle plus grand reste utilisable.

206 `instructions=dedent("""`

→ Commence la définition des instructions de l'agent d'analyse de risques sous forme de chaîne multi-lignes désindentée.

207 `You are a senior risk analysis advisor for international trade operations between the UK and Morocco.`

→ Ligne d'instruction présentant le rôle de conseiller senior en analyse de risques pour le commerce UK-Maroc.

209 `Based on a complete export-import compliance report, your task is to identify and evaluate potential risks across three main areas:`

→ Ligne d'instruction décrivant la mission d'identification et d'évaluation des risques à partir du rapport de conformité.

211 `1. **Legal Risks**:`

→ Ligne d'instruction introduisant la première catégorie de risques, les risques légaux.

212 `- Missing or ambiguous license requirements`

→ Ligne d'instruction citant comme risque légal les exigences de licence manquantes ou ambiguës.

213 `- Penalties for non-compliance`

→ Ligne d'instruction citant comme risque légal les pénalités en cas de non-conformité.

214 `- Contradictions or lack of jurisdiction clarity`

→ Ligne d'instruction citant comme risque légal les contradictions ou le manque de clarté juridictionnelle.

216 `2. **Operational Risks**:`

→ Ligne d'instruction introduisant la deuxième catégorie de risques, les risques opérationnels.

217 `- Risks of shipment delay due to customs, documents, unclear timelines`

→ Ligne d'instruction citant comme risque opérationnel les retards d'expédition liés aux douanes et documents.

218 `- Problems with quantity limits, permit validation, or procedural bottlenecks`

→ Ligne d'instruction citant comme risque opérationnel les problèmes de limites de quantité et de validation de permis.

220 3. **Financial Risks**:

→ Ligne d'instruction introduisant la troisième catégorie de risques, les risques financiers.

221 - Unexpected taxes, fees, or duties

→ Ligne d'instruction citant comme risque financier les taxes, frais ou droits inattendus.

222 - Currency conversion, VAT surprises, or payment processing complications

→ Ligne d'instruction citant comme risque financier les complications de change, TVA ou paiement.

224 You must respond in clear markdown format, with bold headers for each section.

→ Ligne d'instruction imposant une réponse en Markdown clair avec des titres en gras par section.

225 Use concrete details from the report to justify your findings.

→ Ligne d'instruction demandant de justifier les constats à l'aide de détails concrets tirés du rapport.

226 Do not invent risks – base your output only on what is found in the report.

→ Ligne d'instruction interdisant d'inventer des risques et imposant de se baser uniquement sur le rapport, même logique anti-hallucination que celle appliquée au `specialist_agent`, mais ici la « source de vérité » est le rapport généré en amont plutôt que les documents de loi eux-mêmes.

228 You may recommend mitigation strategies if applicable.

→ Ligne d'instruction autorisant la proposition de stratégies d'atténuation des risques si pertinent.

229 """),

→ Ferme la chaîne d'instructions dedent et l'appel dedent, terminant l'argument instructions.

230 expected_output=dedent("""

→ Commence la définition du gabarit de sortie attendu de l'agent d'analyse de risques.

231 ### Legal Risks

→ Ligne du gabarit définissant le titre de section Markdown pour les risques légaux.

232 [Bullet list of legal vulnerabilities based on the report]

→ Ligne du gabarit indiquant l'espace réservé pour la liste à puces des vulnérabilités légales.

234 ### Operational Risks

→ Ligne du gabarit définissant le titre de section Markdown pour les risques opérationnels.

235 [Bullet list of logistics, customs or documentation-related risks]

→ Ligne du gabarit indiquant l'espace réservé pour la liste à puces des risques logistiques et documentaires.

236 """),

→ Ferme la chaîne du gabarit dedent et l'appel dedent, terminant l'argument expected_output.

237 knowledge=None,

→ Indique qu'aucune base de connaissance native agno n'est attachée à cet agent.

238 search_knowledge=False,

→ Désactive la recherche dans une base de connaissance pour cet agent.

239 tools=[],

→ Ne fournit aucun outil supplémentaire à cet agent, qui ne travaille que sur le texte reçu.

240)

→ Ferme l'appel au constructeur Agent, terminant la définition de risk_agent.

Interface Streamlit qui orchestre l'OCR des lois, la traduction, puis l'exécution successive des agents spécialiste, vérification et risques, et affiche leurs résultats.

27 lignes de code annotées ci-dessous.

```
1 import streamlit as st
```

→ Importe la bibliothèque Streamlit, sous l'alias `st`. Streamlit permet de transformer un script Python linéaire en application web interactive (champs de saisie, boutons, affichage formaté) sans avoir à écrire de HTML, CSS ou JavaScript : chaque appel à une fonction `st.\*` ajoute un composant visuel à la page, et le script entier est ré-exécuté à chaque interaction de l'utilisateur.

```
2 from import_export_agent import *
```

→ Importe tous les noms définis dans le module `import_export_agent` (import étoile `\*`), c'est-à-dire les quatre agents (`translation_agent`, `specialist_agent`, `verification_agent`, `risk_agent`), la fonction `ocr_pdf`, et les fonctions `retrieve` comme `import_law_document`, pour les rendre directement utilisables dans ce fichier sans préfixe de module.

```
4 # Streamlit UI
```

→ Commentaire annonçant le début du code de l'interface utilisateur Streamlit.

```
5 if __name__ == "__main__":
```

→ Vérifie que le script est exécuté directement (via `streamlit run app.py`), et non importé comme module, avant de lancer l'interface — convention standard Python pour distinguer exécution directe et import.

```
6     st.title("Import Export Specialist Multi-Agent System")
```

→ Affiche le titre principal de la page Streamlit, en haut de l'interface.

```
8     product_details = st.text_input("Enter product and shipment details")
```

→ Affiche un champ de saisie texte et stocke la valeur entrée par l'utilisateur dans `product_details` ; Streamlit relie automatiquement l'état du widget à cette variable Python, sans gestion manuelle d'événements.

```
10     if st.button("Submit"):
```

→ Affiche un bouton "Submit" et exécute le bloc suivant uniquement quand il est cliqué — en Streamlit, chaque interaction relance le script du haut, et `st.button` renvoie True seulement pendant l'exécution qui suit le clic.

```
11         with st.spinner("Running multi-agent workflow..."):
```

→ Affiche un indicateur de chargement animé pendant l'exécution du bloc englobé, pour donner un retour visuel à l'utilisateur pendant que les appels aux agents (potentiellement longs, plusieurs secondes à quelques minutes) sont en cours.

```
13             ocr_pdf("Documents/uk-export-law.pdf", "uk-export-law")
```

→ Lance l'OCR du PDF de loi d'exportation britannique et génère le fichier Markdown correspondant, ré-exécuté à chaque clic sur Submit (le script ne met pas en cache ce résultat entre deux soumissions).

```
14             ocr_pdf("Documents/morocco-import-law.pdf", "morocco-import-law")
```

→ Lance l'OCR du PDF de loi d'importation marocaine et génère le fichier Markdown correspondant.

```
16             # Translation step
```

→ Commentaire annonçant l'étape de traduction du document d'importation.

```
17             translation_response = translation_agent.run(import_law_document(None, None))
```

→ Exécute l'agent de traduction : `run()` est la méthode agno qui envoie le message au modèle et attend sa réponse complète (par opposition à un mode streaming token par token), ici appelée avec le contenu brut du document d'importation (récupéré directement par appel de la fonction `retrieve`, sans passer par le mécanisme `knowledge_retriever` puisque `translation_agent` n'en a pas).

```
18             with open("Documents/morocco-import-law-en.md", "w") as f:
```

→ Ouvre en écriture un fichier destiné à recevoir la traduction anglaise du document d'importation.

```
19                 f.write(translation_response.content)
```

→ Écrit le contenu traduit renvoyé par l'agent dans le fichier ouvert ; `.content` est l'attribut agno qui contient le texte final généré par le modèle, dans l'objet de réponse renvoyé par `run()`.

21 # Specialist analysis (le retriever combiné est déjà configuré à la construction de l'agent)
→ Commentaire précisant que le retriever combiné a déjà été configuré lors de la création de l'agent.

22 specialist_response = specialist_agent.run(product_details)
→ Exécute l'agent spécialiste sur les détails du produit saisis par l'utilisateur ; en interne, agno appelle automatiquement combined_law_documents (le RAG) pour injecter les extraits de lois avant de générer la réponse, et utilise DuckDuckGoTools si le modèle juge nécessaire de compléter par une recherche web.

24 # Display specialist report
→ Commentaire annonçant l'affichage du rapport de l'agent spécialiste.

25 st.subheader(" Specialist Compliance Report")
→ Affiche un sous-titre Streamlit annonçant le rapport de conformité du spécialiste.

26 st.markdown(specialist_response.content)
→ Affiche le contenu Markdown de la réponse du spécialiste dans la page Streamlit : `st.markdown` interprète le texte (titres, gras, listes) et le rend visuellement, plutôt que de l'afficher comme texte brut.

28 # Verification step
→ Commentaire annonçant l'étape de vérification du rapport.

29 verification_response = verification_agent.run(specialist_response.content)
→ Exécute l'agent de vérification sur le contenu du rapport spécialiste et stocke la réponse — c'est ici que la chaîne d'agents se matérialise concrètement : la sortie d'un agent devient l'entrée du suivant.

30 st.subheader(" Verification Feedback")
→ Affiche un sous-titre Streamlit annonçant les retours de vérification.

31 st.markdown(verification_response.content)
→ Affiche le contenu Markdown de la réponse de vérification dans la page Streamlit.

33 # Risk analysis
→ Commentaire annonçant l'étape d'analyse des risques.

34 risk_response = risk_agent.run(specialist_response.content)
→ Exécute l'agent d'analyse de risques, non pas sur le retour du vérificateur mais directement sur le rapport du spécialiste : verification_agent et risk_agent travaillent tous deux en parallèle logique sur la même source (specialist_response.content), chacun avec un angle différent.

35 st.subheader(" Risk Analysis")
→ Affiche un sous-titre Streamlit annonçant l'analyse de risques.

36 st.markdown(risk_response.content)
→ Affiche le contenu Markdown de la réponse d'analyse de risques dans la page Streamlit.

Projet 29 — 29-Investment_insight

Profil client + recommandation d'investissement

Objectif

Construit un profil d'investisseur a partir des reponses d'un utilisateur (objectifs, tolerance au risque, horizon de placement), recherche des informations de marche actuelles sur le web, et genere une recommandation d'investissement personnalisee et argumentee.

Code source, ligne par ligne

29-Investment_insight/main.py

Définit une équipe et cinq agents agno (recherche interne, recherche web, équipe de recherche client, analyse financière, recommandations) formant un pipeline de conseil en investissement.

257 lignes de code annotées ci-dessous.

1 from agno.agent import Agent

→ Importe la classe Agent du framework agno. Agno est un framework Python open-source pour construire des agents IA : il gère l'appel au modèle de langage, les instructions systèmes, l'utilisation d'outils externes (function calling) et, comme on le verra plus bas, la coordination de plusieurs agents entre eux via une Team.

2 from agno.team.team import Team

→ Importe la classe Team d'agno, qui permet de faire collaborer plusieurs agents spécialisés sous la supervision d'un agent coordinateur : au lieu qu'un seul agent généraliste tente de tout faire, chaque membre garde un rôle étroit (ici recherche interne vs recherche web), et la Team décide à qui déléguer la tâche et agrège les réponses.

3 from agno.models.openai import OpenAIChat

→ Importe la classe modèle OpenAIChat d'agno, un adaptateur qui permet à un Agent ou une Team de dialoguer avec les modèles de chat de l'API OpenAI (ici gpt-4o-mini, utilisé plus bas pour tous les agents de ce fichier).

4 from agno.models.google import Gemini

→ Importe la classe modèle Gemini d'agno, l'équivalent d'OpenAIChat mais pour les modèles Gemini de Google ; elle est disponible mais reste non utilisée activement dans ce fichier (on la retrouve seulement en commentaire, comme alternative envisagée).

5 from agno.tools.scraper import ScrapeGraphTools

→ Importe l'outil ScrapeGraphTools d'agno, une intégration du service ScrapeGraphAI : contrairement à un scraper classique qui extrait du HTML brut, ScrapeGraphAI utilise lui-même un modèle de langage pour interpréter une page web et en extraire un contenu structuré et pertinent, ce qui simplifie l'extraction d'informations depuis des pages hétérogènes.

6 from agno.tools.duckduckgo import DuckDuckGoTools

→ Importe l'outil de recherche web DuckDuckGo d'agno, bien que non instancié plus loin dans ce fichier (les agents utilisent ExaTools et ScrapeGraphTools à la place pour la recherche web).

7 from agno.tools.file import FileTools

→ Importe l'outil FileTools d'agno, qui donne à un agent la capacité d'écrire et lire des fichiers sur le disque local (par exemple financial_data.md ou recommandation.md plus bas) : sans cet outil, un agent ne peut que renvoyer du texte dans sa réponse, il ne peut pas produire de fichier persistant de lui-même.

8 from agno.tools.yfinance import YFinanceTools

→ Importe l'outil YFinanceTools d'agno, qui encapsule la bibliothèque `yfinance` pour donner à un agent un accès structuré aux données financières de Yahoo Finance (cours de bourse, informations sur l'entreprise, ratios financiers, recommandations

d'analystes), sans que le modèle ait besoin d'aller chercher ces chiffres par recherche web générique.

```
9 from agno.tools.exa import ExaTools
```

→ Importe l'outil ExaTools d'agno, une intégration de l'API Exa : un moteur de recherche dit « neuronal » ou sémantique, conçu spécifiquement pour les agents IA, qui comprend le sens d'une requête plutôt que de faire un simple appariement de mots-clés comme un moteur de recherche classique, ce qui donne des résultats souvent plus pertinents pour des recherches complexes.

```
11 from dotenv import load_dotenv
```

→ Importe `load_dotenv` de python-dotenv, qui charge les clés API stockées dans un fichier `.env` local plutôt que codées en dur dans le script, pour des raisons de sécurité et de portabilité.

```
12 import os
```

→ Importe le module standard `os`, utilisé ici pour lire les variables d'environnement contenant les clés API.

```
14 # Configuration
```

→ Commentaire annonçant la section de configuration du script.

```
15 load_dotenv()
```

→ Charge les variables définies dans le fichier `.env` dans l'environnement du processus.

```
17 sgai_api_key = os.getenv("SGAI_API_KEY")
```

→ Récupère la valeur de la variable d'environnement `SGAI_API_KEY` (la clé du service ScrapeGraphAI) et la stocke dans `sgai_api_key`.

```
18 exa_api_key = os.getenv("EXA_API_KEY")
```

→ Récupère la valeur de la variable d'environnement `EXA_API_KEY` (la clé du service de recherche Exa) et la stocke dans `exa_api_key`.

```
19 api_key_openai = os.getenv("OPENAI_API_KEY")
```

→ Récupère la valeur de la variable d'environnement `OPENAI_API_KEY` et la stocke dans `api_key_openai`, utilisée pour authentifier tous les modèles OpenAIChat de ce fichier.

```
21 # Agent 1: Internal Search Agent
```

→ Commentaire annonçant la définition du premier agent, chargé de la recherche interne.

```
22 internal_search = Agent(
```

→ Commence la création de l'agent nommé `internal_search`, dont le rôle est étroitement limité à interroger une base de données locale de clients — un exemple de spécialisation d'agent : il ne fait qu'une chose, ce qui rend son comportement plus prévisible qu'un agent généraliste.

```
23     name="internal search agent",
```

→ Définit le nom affiché de cet agent comme "internal search agent".

```
24     # model=Gemini(id=id, api_key=api_key),
```

→ Ligne de code commentée montrant une configuration alternative avec le modèle Gemini, laissée en place comme trace d'une option envisagée mais non retenue, sans effet sur l'exécution.

```
25     model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
```

→ Configure le modèle utilisé par cet agent : `gpt-4o-mini`, une version allégée et moins coûteuse de GPT-4o adaptée à des tâches simples comme la synthèse de notes de réunion, authentifiée avec la clé OpenAI.

```
26     tools=[FileTools()],
```

→ Fournit à l'agent l'outil `FileTools`, qui lui permet de lire le fichier local de données clients (`29-Investment_insight/Data/data.json`) directement, plutôt que de recevoir ce contenu déjà inséré dans le prompt par le code Python.

```
27     description="""
```

→ Commence la chaîne de description multi-lignes de l'agent, sans utiliser `dedent` (contrairement au fichier 27, ici l'indentation résiduelle du texte n'est pas retirée — différence de style entre les deux projets, sans impact fonctionnel majeur car le modèle tolère bien des espaces en trop).

- 28 A data analyst agent that retrieves and summarizes historical client meeting records
→ Ligne de description indiquant que l'agent récupère et résume les comptes rendus historiques de réunions clients.
- 29 from the internal database ('29-Investment_insight/Data/data.json').
→ Ligne de description précisant le chemin du fichier JSON servant de base de données interne.
- 30 The agent provides concise, insight-driven summaries based on each meeting entry.
→ Ligne de description précisant que l'agent fournit des résumés concis et riches en insights pour chaque réunion.
- 31 "",
→ Ferme la chaîne de description de l'agent. Le paramètre `description` d'agno sert de contexte général sur le rôle de l'agent, complémentaire aux `instructions` qui donnent des consignes opérationnelles plus précises.
- 32 instructions=""
→ Commence la chaîne d'instructions multi-lignes de l'agent, sans utiliser dedent.
- 33 You are given access to a JSON file containing client information, including detailed meeting notes.
→ Ligne d'instruction précisant que l'agent a accès à un fichier JSON contenant les informations clients et notes de réunion.
- 35 Your task is to:
→ Ligne d'instruction introduisant la liste des tâches à accomplir.
- 36 1. Search for the specified client by full name in the '29-Investment_insight/Data/data.json' file.
→ Ligne d'instruction demandant de rechercher le client par nom complet dans le fichier JSON indiqué.
- 37 2. If the client is found, focus on the 'meeting_notes' field.
→ Ligne d'instruction demandant de se concentrer sur le champ meeting_notes si le client est trouvé.
- 38 3. For each note, write a **professional, well-structured summary, and actions** using markdown bullet format like this:
→ Ligne d'instruction demandant de rédiger un résumé structuré et des actions au format à puces Markdown.
- 39 # Internal Search:
→ Ligne d'instruction montrant le titre Markdown attendu pour la section de recherche interne.
- 40 - <Date>: Summary
→ Ligne d'instruction montrant le format attendu d'une puce résumant une réunion datée.
- 41 - **Action Items**:
→ Ligne d'instruction montrant le format attendu d'une puce listant les actions à mener.
- 43 Notes:
→ Ligne d'instruction introduisant des remarques complémentaires de rédaction.
- 44 - Summarize clearly; do not just copy/paste raw content.
→ Ligne d'instruction demandant un résumé clair plutôt qu'une copie brute du contenu.
- 45 - Group related ideas. Be concise but informative.
→ Ligne d'instruction demandant de regrouper les idées liées tout en restant concis et informatif.
- 46 - Ensure the tone is suitable for executive review.
→ Ligne d'instruction demandant un ton adapté à une lecture par des dirigeants.
- 48 Final response must be:
→ Ligne d'instruction introduisant les critères de forme de la réponse finale.
- 49 - Well-structured using Markdown

→ Ligne d'instruction exigeant une réponse bien structurée en Markdown.

50 - Professional in tone

→ Ligne d'instruction exigeant un ton professionnel.

51 - Only include one of the two sections depending on the results.

→ Ligne d'instruction exigeant de n'inclure qu'une seule des deux sections possibles selon le résultat.

54 If the client is not found, respond exactly with: CLIENT NOT FOUND.

→ Ligne d'instruction imposant une réponse exacte et normalisée quand le client n'est pas trouvé — ce texte fixe sert de signal machine-à-machine : la Team définie plus bas (ligne 117) détecte cette chaîne précise pour décider de déléguer à l'agent de recherche web, un pattern simple de routage conditionnel entre agents basé sur le contenu de la réponse.

55 """

→ Ferme la chaîne d'instructions de l'agent.

56 stream=True

→ Active le mode streaming : plutôt que d'attendre la réponse complète du modèle avant de la renvoyer, agno la transmet token par token au fur et à mesure de sa génération, ce qui réduit la latence perçue par l'utilisateur (comme un texte qui s'écrit progressivement) même si le temps de génération total est identique.

57)

→ Ferme l'appel au constructeur Agent, terminant la définition de internal_search.

59 # Agent 2: Web Search Agent

→ Commentaire annonçant la définition du deuxième agent, chargé de la recherche web.

60 web_search = Agent(

→ Commence la création de l'agent nommé web_search, activé uniquement en secours quand la recherche interne échoue (voir l'instruction ligne 71) : une logique de repli (fallback) entre deux sources d'information de fiabilité différente.

61 name="web search agent",

→ Définit le nom affiché de cet agent comme "web search agent".

62 # model=Gemini(id=id, api_key=api_key),

→ Ligne de code commentée montrant une configuration alternative avec le modèle Gemini, non utilisée.

63 model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),

→ Configure le modèle OpenAIChat gpt-4o-mini utilisé par cet agent, authentifié avec la clé OpenAI.

64 tools=[ScrapeGraphTools(api_key=sgai_api_key, markdownify=True), ExaTools(api_key=exa_api_key)],

→ Fournit à l'agent deux outils complémentaires pour la recherche web : ExaTools pour trouver des pages pertinentes via recherche sémantique, puis ScrapeGraphTools pour en extraire le contenu utile — l'option `markdownify=True` demande que le contenu scrapé soit renvoyé directement en Markdown, format facilement réinjectable dans un prompt ou un rapport.

65 description="""

→ Commence la chaîne de description multi-lignes de l'agent.

66 A web intelligence agent activated only when a client is not found in the internal records.

→ Ligne de description précisant que l'agent n'est activé que si le client est absent des dossiers internes.

67 It searches the web to uncover publicly available, up-to-date information about the client,

→ Ligne de description précisant que l'agent recherche sur le web des informations publiques et à jour sur le client.

68 focusing on professional activity and recent public appearances.

→ Ligne de description précisant que la recherche se concentre sur l'activité professionnelle et les apparitions publiques récentes.

69 """

→ Ferme la chaîne de description de l'agent.

70 `instructions="""`

→ Commence la chaîne d'instructions multi-lignes de l'agent.

71 `Only respond if the internal search agent confirms: "CLIENT NOT FOUND".`

→ Ligne d'instruction conditionnant l'exécution de l'agent à la confirmation "CLIENT NOT FOUND" de l'agent interne, reprenant le signal textuel défini plus haut (ligne 54).

73 `Step 1: Use ExaTools to search the web for the client's full name.`

→ Ligne d'instruction décrivant la première étape, rechercher le nom complet du client via ExaTools.

74 `Step 2: Identify relevant links such as:`

→ Ligne d'instruction décrivant la deuxième étape, identifier des liens pertinents listés ci-dessous.

75 `- Personal websites`

→ Ligne d'instruction citant les sites web personnels comme type de lien pertinent à identifier.

76 `- LinkedIn, GitHub, or other professional profiles`

→ Ligne d'instruction citant les profils professionnels comme type de lien pertinent à identifier.

77 `- News articles or blog posts`

→ Ligne d'instruction citant les articles de presse ou de blog comme type de lien pertinent à identifier.

78 `- Conference appearances, guest speaking events, or publications`

→ Ligne d'instruction citant les conférences et publications comme type de lien pertinent à identifier.

80 `Step 3: Use ScrapeGraphTools to scrape **only the most relevant and recent pages**.`

→ Ligne d'instruction décrivant la troisième étape, ne scraper que les pages les plus pertinentes et récentes — une consigne qui limite le nombre d'appels à l'outil de scraping, chaque appel ayant un coût en temps et en tokens.

81 `Step 4: Summarize any recent or notable findings into a **clear, executive-level markdown summary**.`

→ Ligne d'instruction décrivant la quatrième étape, résumer les découvertes en un résumé Markdown de niveau exécutif.

83 `Formatting guidelines:`

→ Ligne d'instruction introduisant les consignes de mise en forme de la réponse.

84 `- Use bullet points per finding.`

→ Ligne d'instruction demandant d'utiliser une puce par découverte.

85 `- Each point should include a concise description of the event, post, or appearance.`

→ Ligne d'instruction demandant à chaque puce d'inclure une description concise de l'événement.

86 `- Keep it factual, neutral, and informative – suitable for a professional briefing.`

→ Ligne d'instruction demandant un contenu factuel, neutre et adapté à un briefing professionnel.

87 `- If no relevant information is found, respond with: "No public information found."`

→ Ligne d'instruction imposant une réponse normalisée en l'absence d'information publique trouvée.

88 `""",`

→ Ferme la chaîne d'instructions de l'agent.

89 `show_tool_calls=True,`

→ Active l'affichage des appels d'outils effectués par l'agent dans les logs/console : utile en développement pour voir exactement quelles requêtes ExaTools ou ScrapeGraphTools ont été déclenchées et avec quels arguments, ce qui aide à déboguer un comportement inattendu de l'agent.

90 `stream=True,`

→ Active le mode streaming pour que les réponses de l'agent soient renvoyées progressivement.

91)

→ Ferme l'appel au constructeur Agent, terminant la définition de web_search.

93 # Team: Client Research Team

→ Commentaire annonçant la définition de l'équipe coordonnant les deux agents de recherche.

94 client_research = Team(

→ Commence la création de l'équipe nommée client_research, instance de la classe Team d'agno : contrairement à un Agent isolé, une Team reçoit une requête, décide (via son propre modèle) à quel(s) membre(s) la déléguer, et peut composer une réponse finale à partir des réponses des membres.

95 name="Client Research Agent",

→ Définit le nom affiché de cette équipe comme "Client Research Agent".

96 # model=Gemini(id=id, api_key=api_key),

→ Ligne de code commentée montrant une configuration alternative avec le modèle Gemini, non utilisée.

97 model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),

→ Configure le modèle OpenAIChat gpt-4o-mini utilisé pour coordonner l'équipe (raisonner sur qui déléguer et comment agréger), authentifié avec la clé OpenAI ; c'est un modèle distinct de ceux des membres, qui ont chacun le leur.

98 members=[internal_search, web_search],

→ Définit les membres de l'équipe comme étant les agents internal_search et web_search déjà construits plus haut : la Team ne redéfinit pas leur comportement, elle les réutilise tels quels comme sous-agents spécialisés.

99 # model="coordinate",

→ Ligne de code commentée montrant un mode de coordination alternatif non utilisé (agno propose différents modes de fonctionnement pour une Team, comme la délégation coordonnée par un modèle superviseur ou l'exécution collaborative en parallèle).

100 tools=[],

→ Ne fournit aucun outil supplémentaire à l'équipe elle-même, au-delà de ceux de ses membres — la Team ne fait qu'orchestrer, elle ne recherche rien elle-même directement.

101 description="""

→ Commence la chaîne de description multi-lignes de l'équipe.

102 A high-level research agent that coordinates two specialized sub-agents:

→ Ligne de description précisant que l'équipe coordonne deux sous-agents spécialisés.

103 - An internal search agent that queries local client records.

→ Ligne de description citant le sous-agent de recherche interne interrogeant les dossiers locaux.

104 - A web search agent that performs external research if internal data is missing.

→ Ligne de description citant le sous-agent de recherche web utilisé si les données internes manquent.

106 This agent orchestrates both and produces a professional, structured summary about the client.

→ Ligne de description précisant que l'équipe orchestre les deux agents et produit un résumé structuré du client.

107 """

→ Ferme la chaîne de description de l'équipe.

108 instructions="""

→ Commence la chaîne d'instructions multi-lignes de l'équipe.

109 Your goal is to gather insights about a client using two research sources:

→ Ligne d'instruction énonçant l'objectif de rassembler des informations sur un client via deux sources.

111 1. First, delegate the task to the ****internal search agent**** to locate any existing internal meeting records about the client.

→ Ligne d'instruction demandant de déléguer d'abord la tâche à l'agent de recherche interne.

112 2. If the internal agent returns information, format it under the section titled:

→ Ligne d'instruction demandant de formater le résultat de l'agent interne sous un titre de section précis.

113 # Internal Search:

→ Ligne d'instruction montrant le titre Markdown attendu pour la section de recherche interne.

114 - Date: Summary

→ Ligne d'instruction montrant le format attendu d'une puce résumant une réunion datée.

115 - ****Action Items****: if any

→ Ligne d'instruction montrant le format attendu d'une puce listant les actions à mener, si applicable.

117 3. If the internal agent replies "CLIENT NOT FOUND", delegate the task to the ****web search agent****.

→ Ligne d'instruction demandant de déléguer la tâche à l'agent de recherche web si le client n'est pas trouvé en interne, appliquant concrètement la logique de repli déjà préparée dans les instructions des deux agents membres.

118 4. If the web search agent returns anything, format it under the section:

→ Ligne d'instruction demandant de formater le résultat de l'agent web sous un titre de section précis.

119 # Web Search:

→ Ligne d'instruction montrant le titre Markdown attendu pour la section de recherche web.

120 - Summarized bullet points from recent or relevant online content

→ Ligne d'instruction montrant le format attendu d'une puce résumant du contenu en ligne pertinent.

122 Final response must be:

→ Ligne d'instruction introduisant les critères de forme de la réponse finale de l'équipe.

123 - Well-structured using Markdown

→ Ligne d'instruction exigeant une réponse bien structurée en Markdown.

124 - Professional in tone

→ Ligne d'instruction exigeant un ton professionnel.

125 - Only include one of the two sections depending on the results.

→ Ligne d'instruction exigeant de n'inclure qu'une seule des deux sections possibles selon le résultat.

126 "",

→ Ferme la chaîne d'instructions de l'équipe.

127 stream=True,

→ Active le mode streaming pour que les réponses de l'équipe soient renvoyées progressivement.

128 show_members_responses=True,

→ Active l'affichage des réponses individuelles de chaque agent membre de l'équipe, en plus de la réponse finale agrégée : permet de voir séparément ce qu'a répondu internal_search et ce qu'a répondu web_search, utile pour comprendre ou déboguer la décision de délégation de la Team.

129)

→ Ferme l'appel au constructeur Team, terminant la définition de client_research.

131 # Agent 3: Financial Data Agent

→ Commentaire annonçant la définition du troisième agent, chargé des données financières.

```

132 financial_data_agent = Agent(
    → Commence la création de l'agent nommé financial_data_agent, qui combine plusieurs outils (données de marché, fichiers,
recherche web) pour produire une analyse d'investissement étayée.

133     name="Financial Intelligence Agent",
    → Définit le nom affiché de cet agent comme "Financial Intelligence Agent".

134     model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
    → Configure le modèle OpenAIChat gpt-4o-mini utilisé par cet agent, authentifié avec la clé OpenAI.

135     tools=[
    → Commence la liste des outils fournis à cet agent.

136         YFinanceTools(
    → Commence la configuration de l'outil YFinanceTools, qui donne à l'agent un accès direct et structuré aux données de
marché de Yahoo Finance, plus fiable pour ce type de données précises qu'une recherche web générique qui pourrait renvoyer
des chiffres obsolètes ou mal formatés.

137             stock_price=True,
    → Active la récupération du prix de l'action par l'outil YFinanceTools.

138             company_info=True,
    → Active la récupération des informations générales sur l'entreprise par l'outil YFinanceTools.

139             stock_fundamentals=True,
    → Active la récupération des données fondamentales de l'action (par exemple capitalisation, ratios de valorisation) par l'outil
YFinanceTools.

140             key_financial_ratios=True,
    → Active la récupération des ratios financiers clés (par exemple P/E) par l'outil YFinanceTools.

141             analyst_recommendations=True,
    → Active la récupération des recommandations d'analystes par l'outil YFinanceTools.

142         ),
    → Ferme la configuration de l'outil YFinanceTools.

143         FileTools(),
    → Ajoute l'outil FileTools à la liste, permettant à l'agent de lire et écrire des fichiers comme financial_data.md directement, sans
passer par le code Python appelant.

144         ExaTools(api_key=exa_api_key),
    → Ajoute l'outil ExaTools à la liste, authentifié avec la clé Exa, pour effectuer des recherches web sémantiques (par exemple
trouver des articles d'analystes ou des actualités sur l'entreprise).

145         ScrapeGraphTools(api_key=sgai_api_key, markdownify=True),
    → Ajoute l'outil ScrapeGraphTools à la liste, authentifié avec la clé SGAI, pour extraire le contenu utile des pages web trouvées
par ExaTools, converti directement en Markdown.

146     ],
    → Ferme la liste des outils fournis à cet agent.

147     description="""
    → Commence la chaîne de description multi-lignes de l'agent.

148 A financial analysis expert agent responsible for researching specific investment opportunities based
on client context.
    → Ligne de description précisant que l'agent est expert en analyse financière des opportunités d'investissement.

149 This agent can extract financial data, summarize earnings calls, and provide balanced investment

```

recommendations.

→ Ligne de description précisant les capacités de l'agent, extraction de données, résumé d'earnings calls, recommandations équilibrées.

151 It uses insights from the client's research and combines them with real-time financial data

→ Ligne de description précisant que l'agent combine les informations du client avec des données financières en temps réel.

152 and qualitative sources to support decision-making.

→ Ligne de description précisant que l'agent utilise aussi des sources qualitatives pour appuyer la prise de décision.

153 """,

→ Ferme la chaîne de description de l'agent.

154 instructions="""

→ Commence la chaîne d'instructions multi-lignes de l'agent.

155 You are a financial analyst agent tasked with evaluating a stock as an investment opportunity. Your output must strictly follow the structure outlined below.

→ Ligne d'instruction présentant le rôle d'analyste financier et l'obligation de suivre une structure stricte.

157 Start by understanding the client's investment preferences, goals, and risk tolerance. Tailor your recommendations to align with that profile.

→ Ligne d'instruction demandant d'adapter l'analyse aux préférences et à la tolérance au risque du client.

159 End by Saving the output in a file called 'financial_data.md'.

→ Ligne d'instruction demandant de sauvegarder la sortie finale dans un fichier nommé financial_data.md, action que l'agent réalise lui-même via l'outil FileTools plutôt que via le code Python appelant.

161 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

162 Analyze [Stock Name] as an investment opportunity using this exact structure:

→ Ligne d'instruction introduisant le gabarit exact à suivre pour analyser l'action.

164 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

165 ****[Stock Name] Overview****

→ Ligne du gabarit définissant le titre en gras de la section de présentation de l'action.

166 [1-2 sentences summarizing the company's core business and differentiation.]

→ Ligne du gabarit indiquant l'espace réservé pour un résumé de l'activité principale de l'entreprise.

168 ****Pros of Investing in [Stock Name]****

→ Ligne du gabarit définissant le titre en gras de la section des avantages d'investissement.

169 - ****Innovative Technology****: [Detail unique tech/advantages, if applicable.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les avantages technologiques uniques.

170 - ****Strategic Partnerships****: [List key partnerships/alliances.]

→ Ligne du gabarit indiquant l'espace réservé pour lister les partenariats stratégiques clés.

171 - ****Market Demand****: [Highlight growth sectors/tailwinds.]

→ Ligne du gabarit indiquant l'espace réservé pour mettre en avant les secteurs de croissance porteurs.

172 - ****Regulatory Support****: [Mention favorable policies/risks.]

→ Ligne du gabarit indiquant l'espace réservé pour mentionner les politiques favorables ou les risques réglementaires.

173 - ****Revenue Potential****: [Identify high-margin segments or opportunities.]

→ Ligne du gabarit indiquant l'espace réservé pour identifier les segments à forte marge.

175 ****Cons of Investing in [Stock Name]****

→ Ligne du gabarit définissant le titre en gras de la section des inconvénients d'investissement.

176 - ****Regulatory Challenges****: [Permitting delays/policy risks.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les défis réglementaires et retards de permis.

177 - ****Financial Health****: [Losses/debt/cash burn concerns.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les préoccupations sur la santé financière.

178 - ****Market Risks****: [Supply chain/competition/volatility.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les risques de marché.

179 - ****Execution Risks****: [Management hurdles/operational uncertainties.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les risques d'exécution liés à la gestion.

181 ****Overall****: [Stock Name] presents [summary of opportunity], but investors should weigh [key risks].

→ Ligne du gabarit définissant la phrase de conclusion globale résumant l'opportunité et les risques.

183 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

185 Use the following data sources to inform your analysis (in priority order):

→ Ligne d'instruction introduisant la liste ordonnée des sources de données à utiliser.

186 1. Latest earnings calls (Q2/Q3 2024).

→ Ligne d'instruction citant les derniers earnings calls comme première source de données prioritaire.

187 2. SEC filings (10-K/10-Q).

→ Ligne d'instruction citant les dépôts réglementaires SEC comme deuxième source de données prioritaire.

188 3. Analyst reports (e.g., Bloomberg, Reuters).

→ Ligne d'instruction citant les rapports d'analystes comme troisième source de données prioritaire.

189 4. Industry trends related to the company (e.g., AI, energy transition, EV, etc.).

→ Ligne d'instruction citant les tendances sectorielles comme quatrième source de données prioritaire.

190 5. Financial metrics using YFinance (stock fundamentals, key ratios, analyst recommendations).

→ Ligne d'instruction citant les métriques financières via YFinance comme cinquième source de données prioritaire, reliant explicitement cette consigne à l'outil YFinanceTools configuré plus haut.

192 Important:

→ Ligne d'instruction signalant une consigne importante à suivre avant la sauvegarde.

193 Before saving your output to 'financial_data.md', perform this final step:

→ Ligne d'instruction demandant d'effectuer une étape finale avant l'enregistrement du fichier.

194 - Replace all smart punctuation (like '‘', '’', '■', '“', '–') with standard ASCII characters (e.g., '‘', '’', '-').

→ Ligne d'instruction demandant de remplacer la ponctuation typographique (guillemets courbes, tirets longs, souvent générés par les modèles de langage) par des caractères ASCII standards, afin d'éviter des problèmes d'affichage ou d'encodage lors de la relecture du fichier par d'autres outils.

195 - Save the output in UTF-8 encoding using FileTools.

→ Ligne d'instruction demandant de sauvegarder le fichier en encodage UTF-8 via FileTools, un encodage universel qui garantit que tous les caractères (y compris accents et symboles) sont préservés correctement.

197 Your output must be:

→ Ligne d'instruction introduisant les critères finaux attendus pour la sortie.

198 - Clean, well-formatted Markdown

→ Ligne d'instruction exigeant un Markdown propre et bien formaté.

199 - Free of encoding issues

→ Ligne d'instruction exigeant l'absence de problèmes d'encodage.

200 - Saved in a file called 'financial_data.md'

→ Ligne d'instruction exigeant que le fichier soit sauvegardé sous le nom financial_data.md.

201 """

→ Ferme la chaîne d'instructions de l'agent.

202 success_criteria="Your task is only complete when the output is saved to 'financial_data.md'.",

→ Définit le critère de succès de l'agent : ce paramètre agno indique au modèle qu'il ne doit pas s'arrêter après une simple réponse textuelle, mais continuer à agir (ici, appeler réellement FileTools pour écrire le fichier) tant que la condition énoncée n'est pas remplie — c'est ce qui transforme l'agent d'un simple générateur de texte en un agent véritablement « agissant ».

203 stream=True,

→ Active le mode streaming pour que les réponses de l'agent soient renvoyées progressivement.

204)

→ Ferme l'appel au constructeur Agent, terminant la définition de financial_data_agent.

206 # Agent 4: Recommendation Agent

→ Commentaire annonçant la définition du quatrième agent, chargé des recommandations.

207 recommendation_agent = Agent(

→ Commence la création de l'agent nommé recommendation_agent, qui lit le fichier produit par financial_data_agent et le transforme en un rapport de recommandation formel, illustrant un pipeline où chaque agent consomme la sortie fichier du précédent plutôt qu'un simple passage de texte en mémoire.

208 name="Financial Advisor Agent",

→ Définit le nom affiché de cet agent comme "Financial Advisor Agent".

209 model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),

→ Configure le modèle OpenAIChat gpt-4o-mini utilisé par cet agent, authentifié avec la clé OpenAI.

210 tools=[FileTools(), YFinanceTools(

→ Commence la liste des outils fournis à cet agent en ajoutant FileTools (pour lire financial_data.md et écrire recommendation.md) puis le début de la configuration de YFinanceTools (pour compléter avec des données de marché à jour).

211 stock_price=True,

→ Active la récupération du prix de l'action par l'outil YFinanceTools.

212 company_info=True,

→ Active la récupération des informations générales sur l'entreprise par l'outil YFinanceTools.

213 stock_fundamentals=True,

→ Active la récupération des données fondamentales de l'action par l'outil YFinanceTools.

214 key_financial_ratios=True,

→ Active la récupération des ratios financiers clés par l'outil YFinanceTools.

215 analyst_recommendations=True,

→ Active la récupération des recommandations d'analystes par l'outil YFinanceTools.

216), ExaTools(api_key=exa_api_key)],

→ Ferme la configuration de YFinanceTools et ajoute ExaTools authentifié, terminant la liste des outils.

217 description="""

→ Commence la chaîne de description multi-lignes de l'agent.

218 A financial advisory agent responsible for issuing structured investment recommendations based on prior financial analysis.

→ Ligne de description précisant que l'agent émet des recommandations d'investissement structurées basées sur l'analyse préalable.

219 It reads data from 'financial_data.md' and produces a detailed recommendation report using a fixed template,

→ Ligne de description précisant que l'agent lit financial_data.md et produit un rapport selon un gabarit fixe.

220 including both qualitative insights and quantitative metrics.

→ Ligne de description précisant que le rapport combine des observations qualitatives et des métriques quantitatives.

221 """ ,

→ Ferme la chaîne de description de l'agent.

222 instructions="""

→ Commence la chaîne d'instructions multi-lignes de l'agent.

223 You are tasked with generating a professional stock recommendation report and save it as a 'recommendation.md' file.

→ Ligne d'instruction énonçant la tâche de générer un rapport de recommandation et de le sauvegarder sous recommendation.md.

225 Step 1: Load and read the file 'financial_data.md'. It contains a prior detailed analysis of the stock's performance, risks, and potential.

→ Ligne d'instruction décrivant la première étape, charger et lire le fichier financial_data.md — via l'outil FileTools, qui donne littéralement à l'agent une capacité de lecture de fichier qu'un simple appel de modèle de langage n'a pas nativement.

227 Step 2: Based on the data in that file and YFinance tools, complete the following structured markdown template with realistic and accurate values:

→ Ligne d'instruction décrivant la deuxième étape, compléter le gabarit Markdown avec des valeurs réalistes.

229 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

230 # **Company Details**:

→ Ligne du gabarit définissant le titre Markdown de la section détails de l'entreprise.

232 | Name | Rating | Target | ISIN | Last Price | Currency | Sector | Sector Rating |

→ Ligne du gabarit définissant l'en-tête du tableau Markdown des détails de l'entreprise.

233 |-----|-----|-----|-----|-----|-----|-----|-----|

→ Ligne du gabarit définissant la ligne de séparation du tableau Markdown (syntaxe standard des tableaux Markdown, entre l'en-tête et les données).

234 | [Company] | [Rating] | [Price] | [ISIN] | \$XX.XX | [Ccy] | [Sector] | [Sector Rating] |

→ Ligne du gabarit définissant la ligne de données du tableau avec les espaces réservés à remplir.

236 Notes:

→ Ligne d'instruction introduisant des remarques complémentaires pour remplir le gabarit.

237 - Make sure all values are consistent with the financial insights in the input file.

→ Ligne d'instruction demandant la cohérence des valeurs avec les données du fichier d'entrée.

238 - Maintain a professional tone suitable for investment committees or client reports.

→ Ligne d'instruction demandant un ton professionnel adapté aux comités d'investissement.

239 - Use markdown formatting to ensure the output is clean and readable.

→ Ligne d'instruction demandant un formatage Markdown propre et lisible.

241 Step 3: Save the final output as a 'recommendation.md' file.

→ Ligne d'instruction décrivant la troisième étape, sauvegarder la sortie finale sous recommendation.md.

242 """ ,

→ Ferme la chaîne d'instructions de l'agent.

243 stream=True,

→ Active le mode streaming pour que les réponses de l'agent soient renvoyées progressivement.

244)

→ Ferme l'appel au constructeur Agent, terminant la définition de recommendation_agent.

246 # Agent 5: Financial Advisor Agent (Alternative Template)

→ Commentaire annonçant la définition du cinquième agent, variante du précédent avec un autre gabarit.

247 recommendation_agent_v2 = Agent(

→ Commence la création de l'agent nommé recommendation_agent_v2, identique en configuration à recommendation_agent mais avec un gabarit de sortie différent (rating Buy/Hold/Sell et tableau de métriques boursières plutôt qu'un tableau d'entreprise) : une variante de présentation offerte au choix plutôt qu'une nécessité fonctionnelle.

248 name="Financial Advisor Agent",

→ Définit le nom affiché de cet agent comme "Financial Advisor Agent".

249 model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),

→ Configure le modèle OpenAIChat gpt-4o-mini utilisé par cet agent, authentifié avec la clé OpenAI.

250 tools=[FileTools(), YFinanceTools(

→ Commence la liste des outils fournis à cet agent en ajoutant FileTools puis le début de la configuration de YFinanceTools.

251 stock_price=True,

→ Active la récupération du prix de l'action par l'outil YFinanceTools.

252 company_info=True,

→ Active la récupération des informations générales sur l'entreprise par l'outil YFinanceTools.

253 stock_fundamentals=True,

→ Active la récupération des données fondamentales de l'action par l'outil YFinanceTools.

254 key_financial_ratios=True,

→ Active la récupération des ratios financiers clés par l'outil YFinanceTools.

255 analyst_recommendations=True,

→ Active la récupération des recommandations d'analystes par l'outil YFinanceTools.

256), ExaTools(api_key=exa_api_key)],

→ Ferme la configuration de YFinanceTools et ajoute ExaTools authentifié, terminant la liste des outils.

257 description="""

→ Commence la chaîne de description multi-lignes de l'agent.

258 A financial advisory agent responsible for issuing structured investment recommendations based on prior financial analysis.

→ Ligne de description précisant que l'agent émet des recommandations d'investissement structurées basées sur l'analyse préalable.

259 It reads data from 'financial_data.md' and produces a detailed recommendation report using a fixed template,

→ Ligne de description précisant que l'agent lit financial_data.md et produit un rapport selon un gabarit fixe.

260 including both qualitative insights and quantitative metrics.

→ Ligne de description précisant que le rapport combine des observations qualitatives et des métriques quantitatives.

261 """,

→ Ferme la chaîne de description de l'agent.

262 instructions="""

→ Commence la chaîne d'instructions multi-lignes de l'agent.

263 You are tasked with generating a professional stock recommendation report and save it as a 'recommendation.md' file.

→ Ligne d'instruction énonçant la tâche de générer un rapport de recommandation et de le sauvegarder sous recommendation.md.

265 Step 1: Load and read the file 'financial_data.md'. It contains a prior detailed analysis of the stock's performance, risks, and potential.

→ Ligne d'instruction décrivant la première étape, charger et lire le fichier financial_data.md.

267 Step 2: Based on the data in that file and YFinance tools, complete the following structured markdown template with realistic and accurate values:

→ Ligne d'instruction décrivant la deuxième étape, compléter le gabarit Markdown alternatif avec des valeurs réalistes.

269 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

270 [Company Name] Stock Recommendation

→ Ligne du gabarit définissant le titre de la recommandation avec l'espace réservé pour le nom de l'entreprise.

272 **Rating**: [Buy / Hold / Sell]

→ Ligne du gabarit définissant le champ de notation avec les valeurs possibles Buy, Hold ou Sell.

273 **Target Price**: [\$XX.XX]

→ Ligne du gabarit définissant le champ du prix cible avec l'espace réservé au montant.

275 **Rationale**:

→ Ligne du gabarit définissant le titre en gras de la section justification.

277 - **Financial Health**: [Brief summary of financials - EPS, revenue trends, profitability]

→ Ligne du gabarit indiquant l'espace réservé pour le résumé de la santé financière.

278 - **Regulatory Environment**: [Summary of policy or compliance issues]

→ Ligne du gabarit indiquant l'espace réservé pour le résumé de l'environnement réglementaire.

279 - **Growth Initiatives**: [Acquisitions, partnerships, product pipelines]

→ Ligne du gabarit indiquant l'espace réservé pour les initiatives de croissance.

280 - **Market Position**: [Competitive strengths or threats]

→ Ligne du gabarit indiquant l'espace réservé pour la position concurrentielle.

281 - **Risk Factors**: [Short/long-term risks or volatility factors]

→ Ligne du gabarit indiquant l'espace réservé pour les facteurs de risque à court et long terme.

283 **Overall**: [1-2 sentence summary - should reflect the company's outlook and your advised action]

→ Ligne du gabarit définissant le résumé global final avec la perspective et l'action conseillée.

285 ****Key Metrics**:**

→ Ligne du gabarit définissant le titre en gras de la section des indicateurs clés.

287 | Metric | Value |

→ Ligne du gabarit définissant l'en-tête du tableau Markdown des indicateurs clés.

288 | ----- | ----- |

→ Ligne du gabarit définissant la ligne de séparation du tableau des indicateurs clés.

289 | Open Price | \$XX.XX |

→ Ligne du gabarit définissant la ligne du tableau pour le prix d'ouverture.

290 | Market Cap | \$X.XXB |

→ Ligne du gabarit définissant la ligne du tableau pour la capitalisation boursière.

291 | 52-Week High | \$XX.XX |

→ Ligne du gabarit définissant la ligne du tableau pour le plus haut sur 52 semaines.

292 | 52-Week Low | \$XX.XX |

→ Ligne du gabarit définissant la ligne du tableau pour le plus bas sur 52 semaines.

293 | P/E Ratio | XX.XX |

→ Ligne du gabarit définissant la ligne du tableau pour le ratio cours/bénéfice.

294 | Volume | X,XXX,XXX |

→ Ligne du gabarit définissant la ligne du tableau pour le volume échangé.

295 | Volatility (30D) | XX% |

→ Ligne du gabarit définissant la ligne du tableau pour la volatilité sur 30 jours.

296 "",

→ Ferme la chaîne d'instructions de l'agent.

297 stream=True,

→ Active le mode streaming pour que les réponses de l'agent soient renvoyées progressivement.

298)

→ Ferme l'appel au constructeur Agent, terminant la définition de `recommandation_agent_v2`.

300 # Exemples d'utilisation

→ Commentaire annonçant la section d'exemples d'utilisation exécutée en fin de script.

301 if __name__ == "__main__":

→ Vérifie que le script est exécuté directement, et non importé comme module, avant de lancer les exemples — convention Python standard, ici sans interface graphique (contrairement au fichier 27), les tests s'exécutent directement en console.

302 # Test des agents

→ Commentaire annonçant les appels de test des différents agents.

303 internal_search.print_response("give me info about Nicole Robinson", markdown=True)

→ Exécute l'agent de recherche interne sur une requête test et affiche la réponse formatée en Markdown dans la console ; `print_response` est une méthode pratique d'agno qui combine l'appel au modèle et l'affichage lisible du résultat, utile pour des tests rapides sans construire d'interface.

305 client_research.print_response("I'm Nicole Robinson, I'm a Dental Hygienist, do you have any info about me?", stream=True, markdown=True)

→ Exécute l'équipe de recherche client sur une requête test, en streaming, et affiche la réponse en Markdown.

307 financial_data_agent.print_response("Summarize what Binance does and provide the pros and cons of investing into it based on the earnings call Q2/Q3 2024", stream=True, markdown=True)

→ Exécute l'agent d'analyse financière sur une requête test concernant Binance, en streaming, et affiche la réponse.

309 recommendation_agent.print_response("Nicole Robinson is interested in Binance stock. What is our recommendation for this stock?", stream=True, markdown=True)

→ Exécute l'agent de recommandation sur une requête test concernant Binance, en streaming, et affiche la réponse.

30-compliance_agent/main.py

Définit la classe AnalyseurConformiteUE qui exécute un agent agno d'analyse de conformité réglementaire UE, puis convertit son rapport Markdown en document Word formaté.

252 lignes de code annotées ci-dessous.

1 import sys

→ Importe le module standard sys pour accéder aux flux d'entrée/sortie du programme, ici utilisé pour reconfigurer l'encodage de la console.

3 # La console Windows utilise par défaut l'encodage cp1252, qui ne supporte pas

→ Commentaire expliquant que la console Windows par défaut ne supporte pas certains caractères.

4 # les emojis utilisés dans les messages de ce script (ex: ,).

→ Suite du commentaire, précisant que ce sont les émojis utilisés dans les messages du script qui posent problème : sans ce correctif, l'affichage d'un emoji dans un print() planterait le script avec une UnicodeEncodeError sur certains terminaux Windows.

5 for _stream in (sys.stdout, sys.stderr):

→ Itère sur les flux standard de sortie et d'erreur pour les reconfigurer tous les deux de la même façon.

6 if hasattr(_stream, "reconfigure"):

→ Vérifie que le flux courant possède bien la méthode reconfigure avant de l'appeler, cette méthode n'étant disponible que sur certaines versions de Python (3.7+) et certains types de flux, d'où cette vérification défensive.

7 _stream.reconfigure(encoding="utf-8", errors="replace")

→ Reconfigure le flux en encodage UTF-8 (qui supporte les emojis), en remplaçant (errors="replace") les caractères non représentables plutôt que de faire planter le programme.

9 import os

→ Importe le module standard pour interagir avec le système de fichiers et les variables d'environnement.

10 import re

→ Importe le module standard d'expressions régulières (regex), utilisé plus loin pour repérer et transformer les marqueurs Markdown (titres, gras, italique) lors de la conversion vers Word.

11 from datetime import datetime

→ Importe la classe datetime pour générer des horodatages dans les noms de fichiers et le rapport.

12 from pathlib import Path

→ Importe la classe Path du module pathlib, une API orientée objet pour manipuler des chemins de fichiers de façon portable (plutôt que de manipuler des chaînes de caractères brutes avec os.path).

13 from typing import Optional

→ Importe le type Optional utilisé pour annoter les paramètres facultatifs (indiquant qu'un argument peut valoir None), utile pour la lisibilité et la vérification statique du code.

14 from agno.agent import Agent

→ Importe la classe Agent du framework agno. Agno est un framework Python open-source pour construire des agents IA, qui

associe un modèle de langage, des instructions et des outils externes pour produire un comportement spécialisé, ici l'analyse de conformité réglementaire.

```
15 from agno.models.groq import Groq
```

→ Importe la classe modèle Groq d'agno, un adaptateur permettant d'appeler l'API d'inférence de Groq, connue pour sa très faible latence grâce à du matériel dédié (LPU), et qui sert ici de fournisseur pour un modèle open-weight.

```
16 from agno.tools.exa import ExaTools
```

→ Importe l'outil ExaTools d'agno, une intégration de l'API Exa : un moteur de recherche sémantique conçu pour les agents IA, capable de comprendre le sens d'une requête plutôt que de faire un simple appariement de mots-clés, ce qui le rend adapté à des recherches réglementaires précises.

```
17 from agno.tools.reasoning import ReasoningTools
```

→ Importe l'outil ReasoningTools d'agno, qui donne à l'agent la capacité de raisonner explicitement étape par étape avant de répondre (une forme de chaîne de pensée structurée, « chain of thought ») : le modèle peut noter ses étapes de réflexion intermédiaires comme des actions d'outil, ce qui améliore la qualité et la traçabilité d'analyses complexes comme celle-ci.

```
18 from docx import Document
```

→ Importe la classe Document de la bibliothèque python-docx, qui permet de créer et manipuler des fichiers Word (.docx) directement en Python : ajout de titres, paragraphes, mise en forme, sans avoir besoin de Microsoft Word installé.

```
19 from docx.shared import Pt
```

→ Importe l'unité Pt (points) de python-docx, utilisée pour définir des tailles de police dans le document Word de façon cohérente avec les unités typographiques standards.

```
20 from docx.enum.text import WD_PARAGRAPH_ALIGNMENT
```

→ Importe l'énumération des alignements de paragraphe de python-docx, utilisée pour centrer certains textes (titre, sous-titre, date) dans le document généré.

```
21 from dotenv import load_dotenv
```

→ Importe la fonction load_dotenv de python-dotenv, qui charge les clés API depuis un fichier .env local plutôt que de les coder en dur dans le script.

```
23 class AnalyseurConformiteUE:
```

→ Définit la classe AnalyseurConformiteUE qui encapsule toute la logique d'analyse de conformité : c'est un choix de conception orienté objet (plutôt que des fonctions isolées) qui regroupe la configuration de l'agent, les méthodes d'analyse et la génération du rapport Word sous un même état partagé (self.agent, self.groq_api_key, etc.).

```
24     """Système d'Analyse de Conformité UE"""
```

→ Docstring de la classe décrivant son rôle de système d'analyse de conformité UE.

```
26     def __init__(self, config_path: Optional[str] = None):
```

→ Définit le constructeur de la classe, acceptant un chemin de configuration optionnel (typiquement un chemin vers un fichier .env alternatif).

```
27     """Initialise l'analyseur de conformité"""
```

→ Docstring de la méthode décrivant son rôle d'initialisation de l'analyseur.

```
28         load_dotenv(config_path)
```

→ Charge les variables d'environnement depuis le fichier .env par défaut, ou depuis config_path si un chemin spécifique est fourni — cette flexibilité permet de pointer vers une configuration différente sans modifier le code.

```
30         # Récupération des clés API
```

→ Commentaire annonçant la récupération des clés API nécessaires.

```
31         self.exa_api_key = os.getenv("EXA_API_KEY")
```

→ Récupère la clé API Exa depuis les variables d'environnement et la stocke sur l'instance (self), pour qu'elle reste accessible dans les autres méthodes de la classe.

```
32         # Le script original utilisait OpenAI (gpt-4-turbo) ; remplacé par Groq
```

→ Commentaire expliquant que le modèle OpenAI d'origine a été remplacé par Groq.

33 # pour réutiliser la clé GROQ_API_KEY déjà disponible.

→ Suite du commentaire, justifiant le choix de réutiliser la clé Groq déjà disponible, plutôt que de payer un accès supplémentaire à OpenAI, tout en conservant un modèle capable de raisonnement réglementaire complexe.

34 self.groq_api_key = os.getenv("GROQ_API_KEY")

→ Récupère la clé API Groq depuis les variables d'environnement et la stocke sur l'instance.

36 # Validation des clés

→ Commentaire annonçant la validation de la présence des clés API.

37 if not self.groq_api_key:

→ Vérifie si la clé API Groq est absente ou vide.

38 raise ValueError("Clé API Groq manquante")

→ Lève une exception ValueError interrompant l'initialisation si la clé Groq manque, car sans elle aucun appel au modèle n'est possible : mieux vaut échouer immédiatement et clairement à la construction plutôt que plus tard avec une erreur d'API opaque.

40 # Initialisation de l'agent

→ Commentaire annonçant l'appel à la méthode de configuration de l'agent.

41 self._configurer_agent()

→ Appelle la méthode interne qui construit et configure l'agent agno, séparant la logique de configuration (potentiellement longue) du constructeur lui-même pour plus de lisibilité.

43 def _configurer_agent(self):

→ Définit la méthode privée (préfixée d'un underscore par convention Python pour signaler un usage interne) qui construit et configure l'agent de conformité.

44 """Configure l'agent d'analyse de conformité"""

→ Docstring de la méthode décrivant son rôle de configuration de l'agent.

45 outils = [ReasoningTools(add_instructions=True)]

→ Initialise la liste des outils de l'agent avec ReasoningTools, l'option `add\_instructions=True` demandant à agno d'injecter automatiquement dans le prompt système des consignes expliquant au modèle comment et quand utiliser cet outil de raisonnement explicite.

47 # Ajout d'ExaTools si la clé est disponible

→ Commentaire annonçant l'ajout conditionnel de l'outil ExaTools.

48 if self.exa_api_key:

→ Vérifie si une clé API Exa est disponible avant d'ajouter l'outil correspondant, rendant la recherche web optionnelle : l'agent reste fonctionnel avec le seul raisonnement s'il n'y a pas de clé Exa, mais gagne en capacité de recherche documentaire si elle est présente.

49 outils.append(ExaTools(

→ Ajoute une nouvelle instance d'ExaTools à la liste des outils, en ouvrant ses paramètres.

50 api_key=self.exa_api_key,

→ Passe la clé API Exa récupérée à l'instance d'ExaTools.

51 include_domains=["eur-lex.europa.eu", "ec.europa.eu"]

→ Restreint les recherches d'ExaTools aux domaines officiels eur-lex.europa.eu (le portail légal de l'Union européenne) et ec.europa.eu (le site de la Commission européenne) : cette restriction de domaine est une garde-fou de fiabilité, qui force l'agent à ne s'appuyer que sur des sources officielles plutôt que sur n'importe quel site trouvé par une recherche web ouverte, réduisant le risque de citer une information erronée ou non officielle.

52))

→ Ferme la configuration d'ExaTools et l'appel à outils.append.

54 self.agent = Agent(

→ Commence la création de l'agent agno et l'assigne à l'attribut self.agent, pour qu'il reste accessible depuis les autres méthodes de la classe (analyser_entreprise, etc.).

55 name="Analyseur de Conformité UE",

→ Définit le nom affiché de l'agent comme "Analyseur de Conformité UE".

56 model=Groq(

→ Commence la configuration du modèle Groq utilisé par l'agent.

57 id="openai/gpt-oss-120b",

→ Spécifie l'identifiant du modèle Groq gpt-oss-120b à utiliser : un modèle open-weight de 120 milliards de paramètres, choisi pour sa capacité à traiter un raisonnement réglementaire complexe et multi-étapes.

58 api_key=self.groq_api_key

→ Passe la clé API Groq récupérée à la configuration du modèle.

59),

→ Ferme la configuration du modèle Groq.

60 tools=outils,

→ Associe à l'agent la liste d'outils construite précédemment (ReasoningTools et éventuellement ExaTools).

61 debug_mode=True,

→ Active le mode debug de l'agent, qui affiche des informations supplémentaires (appels d'outils, étapes de raisonnement internes) lors de l'exécution dans la console, utile pour observer et déboguer le comportement de l'agent pendant le développement.

62 description="""

→ Commence la chaîne de description multi-lignes de l'agent.

63 Agent spécialisé dans l'analyse de la conformité réglementaire européenne.

→ Ligne de description précisant la spécialisation de l'agent en conformité réglementaire européenne.

64 Fournit des évaluations de conformité structurées pour les entreprises

→ Ligne de description précisant que l'agent fournit des évaluations structurées pour les entreprises.

65 opérant sur le marché européen.

→ Fin de la ligne de description, précisant que les entreprises visées opèrent sur le marché européen.

66 """,

→ Ferme la chaîne de description de l'agent.

67 instructions=self._obtenir_instructions_agent(),

→ Associe à l'agent les instructions renvoyées par la méthode _obtenir_instructions_agent : ces instructions sont extraites dans une méthode séparée plutôt qu'écrites en ligne, pour garder le constructeur de l'agent lisible malgré un prompt système volumineux.

68 markdown=True,

→ Active le formatage Markdown pour les réponses générées par l'agent, cohérent avec l'étape suivante de conversion du rapport en document Word qui repose justement sur la syntaxe Markdown (titres #, gras **, etc.) pour structurer le texte.

69)

→ Ferme l'appel au constructeur Agent, terminant l'assignation à self.agent.

71 def _obtenir_instructions_agent(self) -> str:

→ Définit la méthode privée qui renvoie la chaîne d'instructions détaillées de l'agent, annotée comme renvoyant un `str`.

72 """Instructions détaillées pour l'agent de conformité"""

→ Docstring de la méthode décrivant son rôle de fournisseur des instructions détaillées.

73 return """

→ Commence le renvoi de la chaîne d'instructions multi-lignes de l'agent.

74 Vous êtes un expert en conformité UE. Votre mission est de fournir des analyses

→ Ligne d'instruction présentant le rôle d'expert en conformité UE et sa mission d'analyse.

75 réglementaires complètes pour les entreprises opérant sur les marchés européens.

→ Fin de la ligne d'instruction, précisant que les analyses concernent les entreprises sur les marchés européens.

77 CADRE D'ANALYSE :

→ Ligne d'instruction introduisant le cadre d'analyse structuré à suivre.

78 1. PROFIL DE L'ENTREPRISE

→ Ligne d'instruction introduisant la première étape du cadre, le profil de l'entreprise.

79 - Identifier les activités principales

→ Ligne d'instruction demandant d'identifier les activités principales de l'entreprise.

80 - Déterminer la classification sectorielle

→ Ligne d'instruction demandant de déterminer la classification sectorielle de l'entreprise.

81 - Évaluer l'étendue géographique des opérations UE

→ Ligne d'instruction demandant d'évaluer l'étendue géographique des opérations dans l'UE.

83 2. CARTOGRAPHIE RÉGLEMENTAIRE

→ Ligne d'instruction introduisant la deuxième étape du cadre, la cartographie réglementaire.

84 - Rechercher les directives et règlements applicables

→ Ligne d'instruction demandant de rechercher les directives et règlements applicables — c'est cette étape qui déclenchera typiquement l'usage d'ExaTools par le modèle, restreint aux domaines officiels configurés plus haut.

85 - Identifier les exigences sectorielles spécifiques

→ Ligne d'instruction demandant d'identifier les exigences sectorielles spécifiques.

86 - Suivre les calendriers de mise en œuvre

→ Ligne d'instruction demandant de suivre les calendriers de mise en œuvre des règlements.

88 3. ÉVALUATION DES RISQUES DE CONFORMITÉ

→ Ligne d'instruction introduisant la troisième étape du cadre, l'évaluation des risques de conformité.

89 - Catégoriser les règlements par criticité (Élevée/Moyenne/Faible)

→ Ligne d'instruction demandant de catégoriser les règlements par niveau de criticité.

90 - Identifier les lacunes potentielles de conformité

→ Ligne d'instruction demandant d'identifier les lacunes potentielles de conformité.

91 - Évaluer les sanctions et pénalités

→ Ligne d'instruction demandant d'évaluer les sanctions et pénalités associées.

93 4. DOMAINES RÉGLEMENTAIRES PRINCIPAUX :

→ Ligne d'instruction introduisant la quatrième étape du cadre, les domaines réglementaires principaux.

94 - Protection des données : RGPD, ePrivacy

→ Ligne d'instruction citant le domaine de la protection des données avec le RGPD et ePrivacy.

95 - IA & Numérique : Loi sur l'IA, Loi sur les services numériques
→ Ligne d'instruction citant le domaine de l'IA et du numérique avec les lois correspondantes.

96 - Environnement : Pacte vert, CSRD, Taxonomie
→ Ligne d'instruction citant le domaine environnemental avec le Pacte vert, la CSRD et la Taxonomie.

97 - Sectoriel : Automobile, Services financiers
→ Ligne d'instruction citant les domaines sectoriels automobile et services financiers.

98 - Sécurité des produits : Marquage CE, Cybersécurité
→ Ligne d'instruction citant le domaine de la sécurité des produits avec le marquage CE et la cybersécurité.

100 5. STRUCTURE DU RAPPORT :

→ Ligne d'instruction introduisant la cinquième étape du cadre, la structure du rapport attendu.

101 - Résumé exécutif avec priorités clés
→ Ligne d'instruction demandant un résumé exécutif présentant les priorités clés.

102 - Calendrier réglementaire
→ Ligne d'instruction demandant l'inclusion d'un calendrier réglementaire dans le rapport.

103 - Matrice des risques
→ Ligne d'instruction demandant l'inclusion d'une matrice des risques dans le rapport.

104 - Feuille de route de conformité
→ Ligne d'instruction demandant l'inclusion d'une feuille de route de conformité dans le rapport.

105 - Exigences en ressources
→ Ligne d'instruction demandant l'inclusion des exigences en ressources dans le rapport.

107 STRATÉGIE DE RECHERCHE :
→ Ligne d'instruction introduisant la stratégie de recherche à suivre pour collecter les informations.

108 - Utiliser uniquement des sources officielles UE
→ Ligne d'instruction imposant l'utilisation exclusive de sources officielles de l'UE, cohérente avec la restriction `include\_domains` appliquée à ExaTools : la consigne textuelle et la contrainte technique se renforcent mutuellement pour garantir la fiabilité des sources.

109 - Inclure des références spécifiques aux articles
→ Ligne d'instruction demandant d'inclure des références précises aux articles réglementaires.

110 - Fournir les dates de publication et d'entrée en vigueur
→ Ligne d'instruction demandant de fournir les dates de publication et d'entrée en vigueur des textes.

111 - Vérifier la cohérence entre les sources
→ Ligne d'instruction demandant de vérifier la cohérence des informations entre les différentes sources.

112 """
→ Ferme la chaîne d'instructions renvoyée par la méthode.

```
114 def analyser_entreprise(self, nom_entreprise: str, secteur: str,
    → Définit la méthode analyser_entreprise avec les paramètres nom de l'entreprise et secteur d'activité, point d'entrée public
principal de la classe pour lancer une analyse.
```

115 contexte_supplementaire: str = "",
→ Poursuit la signature de la méthode avec un paramètre de contexte supplémentaire optionnel, vide par défaut.

```
116      chemin_markdown: str = "conformite_automobile.md") -> str:
```

→ Termine la signature avec le chemin de sortie Markdown par défaut, et précise le type de retour str.

```
117         """Analyse les exigences de conformité pour une entreprise spécifique"""
```

→ Docstring de la méthode décrivant son rôle d'analyse des exigences de conformité d'une entreprise.

```
119         requete = f"""
```

→ Commence la construction d'une chaîne formatée (f-string) multi-lignes contenant la requête envoyée à l'agent, les f-strings permettant d'insérer directement la valeur des variables Python entre accolades dans le texte.

```
120         Effectuez une analyse complète de conformité réglementaire UE pour :
```

→ Ligne de la requête demandant une analyse complète de conformité réglementaire UE.

```
122         Entreprise : {nom_entreprise}
```

→ Ligne de la requête insérant dynamiquement le nom de l'entreprise à analyser.

```
123         Secteur : {secteur}
```

→ Ligne de la requête insérant dynamiquement le secteur d'activité de l'entreprise.

```
124         Contexte supplémentaire : {contexte_supplementaire}
```

→ Ligne de la requête insérant dynamiquement le contexte supplémentaire fourni.

```
126         Identifiez tous les règlements UE applicables, leurs exigences de conformité,
```

→ Ligne de la requête demandant d'identifier tous les règlements UE applicables et leurs exigences.

```
127         les délais de mise en œuvre et les impacts potentiels sur l'activité.
```

→ Ligne de la requête demandant de préciser les délais de mise en œuvre et impacts sur l'activité.

```
128         Fournissez des conseils de conformité spécifiques et actionnables.
```

→ Ligne de la requête demandant de fournir des conseils de conformité spécifiques et actionnables.

```
129         """
```

→ Ferme la chaîne formatée contenant la requête envoyée à l'agent.

```
131         print(f" Début de l'analyse de conformité pour {nom_entreprise}...")
```

→ Affiche dans la console un message de démarrage de l'analyse incluant le nom de l'entreprise, rendu possible sans planter grâce à la reconfiguration UTF-8 des flux effectuée en tête de fichier.

```
132         # print_response() n'imprime que dans la console et ne retourne rien
```

→ Commentaire expliquant que print_response ne fait qu'afficher sans rien retourner d'exploitable.

```
133         # d'exploitable ; run() donne accès au contenu pour le sauvegarder en
```

→ Suite du commentaire, précisant que run() permet d'accéder au contenu pour le sauvegarder.

```
134         # markdown, nécessaire pour l'étape de génération du rapport Word.
```

→ Fin du commentaire, précisant que cet accès au contenu est nécessaire pour générer le rapport Word : c'est pourquoi ce script utilise `run()` plutôt que `print\_response()`, contrairement à main.py du projet 29 qui n'a pas besoin de récupérer le texte pour un traitement ultérieur.

```
135         reponse = self.agent.run(requete)
```

→ Exécute l'agent avec la requête construite et stocke la réponse obtenue ; `run()` est la méthode agno qui déclenche l'ensemble du cycle agentique (appel du modèle, éventuels appels d'outils comme ExaTools ou ReasoningTools, puis réponse finale) et renvoie un objet exploitable en Python plutôt que de se contenter d'un affichage console.

```
136         print(reponse.content)
```

→ Affiche dans la console le contenu textuel de la réponse de l'agent.

```
138         with open(chemin_markdown, "w", encoding="utf-8") as fichier:
```

→ Ouvre en écriture le fichier Markdown de sortie, explicitement encodé en UTF-8 pour préserver tous les caractères accentués et spéciaux du texte généré.

139 `fichier.write(reponse.content)`
→ Écrit le contenu de la réponse de l'agent dans le fichier Markdown ouvert.

141 `return reponse.content`
→ Renvoie le contenu textuel de la réponse de l'agent à l'appelant, pour un usage ultérieur en mémoire sans avoir à relire le fichier.

143 `def generer_rapport_word(self, chemin_markdown: str,`
→ Définit la méthode `generer_rapport_word` avec le chemin du fichier Markdown source en paramètre : cette méthode réalise la conversion Markdown vers Word « à la main », en interprétant elle-même la syntaxe (titres, listes, gras, italique) plutôt que de s'appuyer sur une bibliothèque de conversion externe.

144 `chemin_sortie: Optional[str] = None) -> str:`
→ Termine la signature avec un chemin de sortie Word optionnel et précise le type de retour `str`.

145 `"""Convertit un rapport markdown en document Word formaté"""`
→ Docstring de la méthode décrivant son rôle de conversion du Markdown en document Word.

147 `# Lecture du contenu markdown`
→ Commentaire annonçant la lecture du contenu du fichier Markdown source.

148 `try:`
→ Démarre un bloc `try` pour gérer l'absence éventuelle du fichier Markdown, plutôt que de laisser l'exception standard remonter sans message clair.

149 `with open(chemin_markdown, "r", encoding="utf-8") as fichier:`
→ Ouvre en lecture le fichier Markdown source, encodé en UTF-8.

150 `contenu = fichier.read()`
→ Lit l'intégralité du contenu du fichier Markdown et le stocke dans `contenu`.

151 `except FileNotFoundError:`
→ Intercepte l'exception levée si le fichier Markdown est introuvable.

152 `raise FileNotFoundError(f"Fichier markdown introuvable : {chemin_markdown}")`
→ Relève une exception `FileNotFoundError` avec un message plus explicite précisant le chemin manquant, pour faciliter le diagnostic par rapport au message d'erreur générique par défaut.

154 `# Initialisation du document Word`
→ Commentaire annonçant la création du document Word.

155 `doc = Document()`
→ Crée un nouveau document Word vide en mémoire via `python-docx`, basé sur un modèle de document par défaut intégré à la bibliothèque, sur lequel les méthodes suivantes vont progressivement ajouter du contenu.

157 `# Ajout de la page de titre`
→ Commentaire annonçant l'ajout de la page de titre au document.

158 `self._ajouter_page_titre(doc)`
→ Appelle la méthode qui ajoute la page de titre au document Word.

160 `# Traitement du contenu`
→ Commentaire annonçant la conversion du contenu Markdown en contenu Word.

161 `self._traiter_contenu_markdown(doc, contenu)`
→ Appelle la méthode qui convertit le contenu Markdown lu en paragraphes et titres Word.

163 `# Ajout du pied de page`

→ Commentaire annonçant l'ajout du pied de document contenant les métadonnées.

```
164         self._ajouter_pied_document(doc)
```

→ Appelle la méthode qui ajoute le pied de document avec les métadonnées au document Word.

```
166         # Sauvegarde du document
```

→ Commentaire annonçant la sauvegarde finale du document Word sur le disque.

```
167         if not chemin_sortie:
```

→ Vérifie si aucun chemin de sortie n'a été fourni par l'appelant.

```
168         horodatage = datetime.now().strftime("%Y%m%d_%H%M%S")
```

→ Génère un horodatage au format année-mois-jour-heure-minute-seconde pour nommer le fichier, évitant ainsi d'écraser un rapport précédent si aucun nom explicite n'est donné.

```
169         chemin_sortie = f"Rapport_Conformite_UE_{horodatage}.docx"
```

→ Construit un nom de fichier par défaut incluant l'horodatage généré.

```
171         doc.save(chemin_sortie)
```

→ Sauvegarde le document Word sur le disque au chemin déterminé, méthode fournie par python-docx qui sérialise l'objet Document en un vrai fichier .docx lisible par Microsoft Word ou équivalent.

```
172         print(f" Rapport Word sauvegardé : {chemin_sortie}")
```

→ Affiche dans la console un message confirmant la sauvegarde du rapport Word.

```
174         return chemin_sortie
```

→ Renvoie le chemin du fichier Word généré à l'appelant.

```
176     def _ajouter_page_titre(self, doc: Document):
```

→ Définit la méthode privée qui ajoute une page de titre au document Word passé en paramètre.

```
177         """Ajoute une page de titre professionnelle"""
```

→ Docstring de la méthode décrivant son rôle d'ajout d'une page de titre professionnelle.

```
178         # Titre
```

→ Commentaire annonçant l'ajout du titre principal du document.

```
179         titre = doc.add_heading("Analyse de Conformité Réglementaire UE", 0)
```

→ Ajoute un titre de niveau 0 (le plus haut niveau, réservé au titre principal du document) au document via l'API python-docx, et stocke la référence à ce paragraphe dans titre pour pouvoir le modifier ensuite.

```
180         titre.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER
```

→ Centre l'alignement du paragraphe de titre principal, en utilisant l'énumération importée en tête de fichier plutôt qu'une valeur numérique arbitraire.

```
182         # Sous-titre
```

→ Commentaire annonçant l'ajout du sous-titre du document.

```
183         sous_titre = doc.add_paragraph("Rapport d'Intelligence Réglementaire")
```

→ Ajoute un paragraphe de sous-titre au document et stocke la référence dans sous_titre.

```
184         sous_titre.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER
```

→ Centre l'alignement du paragraphe de sous-titre.

```
185         sous_titre_format = sous_titre.runs[0].font
```

→ Récupère l'objet de police du premier « run » du paragraphe de sous-titre. Dans le modèle de python-docx, un paragraphe est composé d'un ou plusieurs « runs » (segments de texte partageant une même mise en forme) : il faut accéder au run pour changer taille ou style, l'alignement seul ne suffisant pas.

186 `sous_titre_format.size = Pt(14)`
→ Fixe la taille de la police du sous-titre à 14 points, en utilisant l'unité Pt importée en tête de fichier.

187 `sous_titre_format.italic = True`
→ Met en italique le texte du sous-titre.

189 `# Date`
→ Commentaire annonçant l'ajout de la date de génération du document.

190 `date_para = doc.add_paragraph(f"Généré le : {datetime.now().strftime('%d %B %Y')})")`
→ Ajoute un paragraphe affichant la date courante, formatée en jour, nom du mois et année via `strftime`.

191 `date_para.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER`
→ Centre l'alignement du paragraphe de date.

193 `# Saut de page`
→ Commentaire annonçant l'insertion d'un saut de page après la page de titre.

194 `doc.add_page_break()`
→ Insère un saut de page dans le document Word, pour que le contenu du rapport commence sur une nouvelle page distincte de la page de titre.

196 `def _traiter_contenu_markdown(self, doc: Document, contenu: str):`
→ Définit la méthode privée qui convertit le texte Markdown en éléments du document Word, cœur du convertisseur maison de ce script.

197 `"""Traite le contenu markdown et l'ajoute au document Word"""`
→ Docstring de la méthode décrivant son rôle de traitement du contenu Markdown.

198 `lignes = contenu.splitlines()`
→ Découpe le contenu Markdown en une liste de lignes individuelles, pour les traiter une par une.

200 `for ligne in lignes:`
→ Itère sur chaque ligne du contenu Markdown découpé.

201 `ligne = ligne.strip()`
→ Supprime les espaces en début et fin de la ligne courante, pour fiabiliser la détection des marqueurs Markdown qui suit (une ligne commençant par un espace avant "# " ne serait pas reconnue sinon).

202 `if not ligne:`
→ Vérifie si la ligne, une fois nettoyée, est vide.

203 `continue`
→ Passe à l'itération suivante de la boucle sans traiter cette ligne vide.

205 `# En-têtes`
→ Commentaire annonçant le traitement des lignes de titres Markdown.

206 `if ligne.startswith("# "):`
→ Vérifie si la ligne commence par un titre Markdown de niveau 1 (syntaxe standard : un dièse suivi d'un espace).

207 `doc.add_heading(ligne[2:], level=1)`
→ Ajoute un titre Word de niveau 1 avec le texte de la ligne débarrassé du marqueur "# " (les deux premiers caractères).

208 `elif ligne.startswith("## "):`
→ Vérifie si la ligne commence par un titre Markdown de niveau 2.

209 `doc.add_heading(ligne[3:], level=2)`
→ Ajoute un titre Word de niveau 2 avec le texte de la ligne débarrassé du marqueur "##".

210 `elif ligne.startswith("### "):`
→ Vérifie si la ligne commence par un titre Markdown de niveau 3.

211 `doc.add_heading(ligne[4:], level=3)`
→ Ajoute un titre Word de niveau 3 avec le texte de la ligne débarrassé du marqueur "###".

212 `elif ligne.startswith("#### "):`
→ Vérifie si la ligne commence par un titre Markdown de niveau 4.

213 `doc.add_heading(ligne[5:], level=4)`
→ Ajoute un titre Word de niveau 4 avec le texte de la ligne débarrassé du marqueur "####".

215 `# Listes`
→ Commentaire annonçant le traitement des lignes de listes Markdown.

216 `elif ligne.startswith("* ") or ligne.startswith("- "):`
→ Vérifie si la ligne est un élément de liste à puces Markdown, dont la syntaxe standard accepte indifféremment "*" ou "-" comme marqueur de puce.

217 `self._ajouter_puce(doc, ligne[2:].strip())`
→ Appelle la méthode qui ajoute une puce Word avec le texte de la ligne débarrassé de son marqueur.

218 `elif re.match(r'^\d+\. ', ligne):`
→ Vérifie via une expression régulière si la ligne est un élément de liste numérotée Markdown (un ou plusieurs chiffres suivis d'un point et d'un espace, par exemple "1."): une simple vérification `startswith` ne suffirait pas ici car le numéro varie, d'où l'usage d'une regex.

219 `self._ajouter_numero(doc, re.sub(r'^\d+\. ', '', ligne))`
→ Appelle la méthode qui ajoute un élément numéroté Word avec le texte débarrassé de son numéro, la suppression étant elle aussi faite par regex (`re.sub`) puisque la longueur du numéro à retirer varie.

221 `# Paragraphes normaux`
→ Commentaire annonçant le traitement des lignes de texte normal, sans marqueur Markdown particulier.

222 `else:`
→ Cas par défaut appliqué quand la ligne n'est ni un titre ni un élément de liste.

223 `self._ajouter_paragraphe_formate(doc, ligne)`
→ Appelle la méthode qui ajoute la ligne comme paragraphe normal formaté dans le document Word.

225 `def _ajouter_puce(self, doc: Document, texte: str):`
→ Définit la méthode privée qui ajoute un élément de liste à puces au document Word.

226 `"""Ajoute une puce formatée"""`
→ Docstring de la méthode décrivant son rôle d'ajout d'une puce formatée.

227 `p = doc.add_paragraph(style="List Bullet")`
→ Crée un nouveau paragraphe utilisant le style Word intégré "List Bullet" (qui gère automatiquement l'indentation et le symbole de puce) et le stocke dans p.

228 `self._ajouter_texte_formate(p, texte)`
→ Appelle la méthode qui insère le texte avec gras et italique dans le paragraphe de puce créé.

230 `def _ajouter_numero(self, doc: Document, texte: str):`
→ Définit la méthode privée qui ajoute un élément de liste numérotée au document Word.

231 `"""Ajoute un élément numéroté"""`

→ Docstring de la méthode décrivant son rôle d'ajout d'un élément numéroté.

```
232 p = doc.add_paragraph(style="List Number")
```

→ Crée un nouveau paragraphe utilisant le style Word intégré "List Number" (qui gère automatiquement la numérotation séquentielle) et le stocke dans p.

```
233 self._ajouter_texte_formate(p, texte)
```

→ Appelle la méthode qui insère le texte avec gras et italique dans le paragraphe numéroté créé.

```
235 def _ajouter_paragraphe_formate(self, doc: Document, texte: str):
```

→ Définit la méthode privée qui ajoute un paragraphe de texte normal formaté au document Word.

```
236 """Ajoute un paragraphe avec formatage"""
```

→ Docstring de la méthode décrivant son rôle d'ajout d'un paragraphe avec formatage.

```
237 p = doc.add_paragraph()
```

→ Crée un nouveau paragraphe vide (sans style de liste particulier) dans le document Word et le stocke dans p.

```
238 self._ajouter_texte_formate(p, texte)
```

→ Appelle la méthode qui insère le texte avec gras et italique dans le paragraphe créé.

```
240 def _ajouter_texte_formate(self, paragraphe, texte: str):
```

→ Définit la méthode privée partagée qui insère du texte avec mise en forme gras et italique dans un paragraphe, utilisée par les trois méthodes précédentes (puce, numéro, paragraphe normal) pour ne pas dupliquer cette logique de parsing.

```
241 """Ajoute du texte avec formatage gras/italique"""
```

→ Docstring de la méthode décrivant son rôle d'ajout de texte avec formatage gras et italique.

```
242 # Traitement du texte en gras (**texte**)
```

→ Commentaire annonçant le traitement du texte en gras délimité par des doubles astérisques, syntaxe Markdown standard pour le gras.

```
243 parties = re.split(r"(\*.*?\*)", texte)
```

→ Découpe le texte en segments avec une regex non gourmande (`\*.*?\*`), en conservant les portions entourées de doubles astérisques comme éléments séparés de la liste résultante (grâce aux parenthèses capturantes dans le motif de split), ce qui permet ensuite de distinguer texte en gras et texte normal.

```
244 for partie in parties:
```

→ Itère sur chaque segment de texte obtenu après découpage.

```
245 if partie.startswith("**") and partie.endswith("**"):
```

→ Vérifie si le segment courant est entouré de doubles astérisques, marquant du texte en gras.

```
246 run = paragraphe.add_run(partie[2:-2])
```

→ Ajoute au paragraphe un « run » (segment de texte à mise en forme homogène, concept python-docx déjà rencontré ligne 185) contenant le texte sans les astérisques de gras.

```
247 run.bold = True
```

→ Met en gras le run de texte ajouté.

```
248 else:
```

→ Cas où le segment n'est pas entouré de doubles astérisques.

```
249 # Traitement du texte en italique (*texte*)
```

→ Commentaire annonçant le traitement du texte en italique délimité par un simple astérisque.

```
250 parties_italique = re.split(r"(\*.*\*)", partie)
```

→ Découpe le segment courant en isolant, de la même façon, les portions entourées d'un simple astérisque.

251 for partie_italique in parties_italique:
→ Itère sur chaque sous-segment obtenu après ce second découpage.

252 if (partie_italique.startswith("*") and partie_italique.endswith("*"))
→ Vérifie si le sous-segment commence et se termine par un astérisque simple, marquant de l'italique.

253 and not partie_italique.startswith("***")):
→ Vérifie également que ce n'est pas un marqueur de gras déjà traité (un texte en gras "***x**" commence aussi techniquement par un simple "*"), condition nécessaire pour éviter une confusion entre les deux syntaxes lors de ce second passage de découpage.

254 run = paragraphe.add_run(partie_italique[1:-1])
→ Ajoute au paragraphe un run contenant le texte sans les astérisques d'italique.

255 run.italic = True
→ Met en italique le run de texte ajouté.

256 else:
→ Cas où le sous-segment n'est ni gras ni italique, du texte brut.

257 paragraphe.add_run(partie_italique)
→ Ajoute au paragraphe un run contenant le texte brut sans mise en forme particulière.

259 def _ajouter_pied_document(self, doc: Document):
→ Définit la méthode privée qui ajoute le pied de document avec les métadonnées au document Word.

260 """Ajoute un pied de document avec métadonnées"""
→ Docstring de la méthode décrivant son rôle d'ajout du pied de document avec métadonnées.

261 doc.add_page_break()
→ Insère un saut de page avant le pied de document, pour le séparer visuellement du corps du rapport.

262 doc.add_heading("Informations du Document", level=2)
→ Ajoute un titre de niveau 2 intitulé "Informations du Document".

264 infos_pied = [
→ Commence la construction de la liste des lignes d'information à afficher dans le pied de document.

265 f"Généré le : {datetime.now().strftime('%d-%m-%Y %H:%M:%S')}",
→ Ajoute à la liste une chaîne indiquant la date et l'heure précises de génération du rapport, avec les secondes contrairement à la date affichée en page de titre (ligne 190), pour une traçabilité plus fine.

266 "Source : Sources officielles UE (eur-lex.europa.eu, ec.europa.eu)",
→ Ajoute à la liste une chaîne mentionnant les sources officielles UE utilisées, rappelant explicitement au lecteur la restriction de domaine appliquée à ExaTools.

267 "Avertissement : Ce rapport est à des fins d'information uniquement.",
→ Ajoute à la liste une chaîne d'avertissement précisant la portée purement informative du rapport.

268 "Recommandation : Consultez un conseil juridique pour des questions de conformité spécifiques."
→ Ajoute à la liste une chaîne recommandant de consulter un conseil juridique pour des questions spécifiques.

269]
→ Ferme la liste des lignes d'information du pied de document.

271 for info in infos_pied:
→ Itère sur chaque ligne d'information de la liste construite précédemment.

272 `p = doc.add_paragraph(info)`
→ Ajoute un paragraphe contenant la ligne d'information courante au document.

273 `p.style = "Caption"`
→ Applique le style Word intégré "Caption" au paragraphe d'information ajouté, un style typographique généralement plus petit et discret, adapté à des mentions légales ou des métadonnées.

275 `def main():`
→ Définit la fonction main, point d'entrée principal du script, séparée de la classe AnalyseurConformiteUE pour ne pas mélanger la logique métier réutilisable et le scénario d'exécution concret.

276 `"""Fonction d'exécution principale"""`
→ Docstring de la fonction décrivant son rôle de point d'exécution principal.

277 `try:`
→ Démarre un bloc try englobant l'ensemble du déroulement principal pour capturer les erreurs et éviter un plantage brut sans message clair.

278 `# Initialisation de l'analyseur`
→ Commentaire annonçant la création de l'instance de l'analyseur de conformité.

279 `analyseur = AnalyseurConformiteUE()`
→ Instancie la classe AnalyseurConformiteUE, déclenchant en chaîne le chargement des clés API et la configuration complète de l'agent agno (via `__init__` puis `_configurer_agent`).

281 `# Exemple d'utilisation`
→ Commentaire annonçant les valeurs d'exemple utilisées pour la démonstration.

282 `nom_entreprise = "Renault"`
→ Définit le nom d'entreprise d'exemple utilisé pour l'analyse, ici Renault.

283 `secteur = "Industrie Automobile"`
→ Définit le secteur d'exemple utilisé pour l'analyse, ici l'industrie automobile.

284 `contexte_supplementaire = "Focus sur les systèmes IA, réglementations d'émissions et reporting de durabilité"`
→ Définit le contexte supplémentaire d'exemple précisant les axes d'analyse souhaités.

286 `# Lancement de l'analyse`
→ Commentaire annonçant le lancement effectif de l'analyse de conformité.

287 `print(" Début de l'analyse de conformité UE...")`
→ Affiche dans la console un message annonçant le démarrage de l'analyse de conformité UE, rendu possible sans erreur grâce à la reconfiguration UTF-8 des flux effectuée en tête de script.

288 `reponse = analyseur.analyser_entreprise(`
→ Commence l'appel à la méthode `analyser_entreprise` sur l'instance `analyseur`, en ouvrant ses arguments.

289 `nom_entreprise=nom_entreprise,`
→ Passe le nom de l'entreprise d'exemple comme argument nommé à la méthode.

290 `secteur=secteur,`
→ Passe le secteur d'exemple comme argument nommé à la méthode.

291 `contexte_supplementaire=contexte_supplementaire`
→ Passe le contexte supplémentaire d'exemple comme argument nommé à la méthode.

292 `)`
→ Ferme l'appel à `analyser_entreprise` et stocke son résultat dans `reponse`.

294 # Conversion en Word (en supposant qu'un fichier markdown a été généré)
→ Commentaire annonçant l'étape de conversion du rapport Markdown en document Word.

295 chemin_markdown = "conformite_automobile.md"
→ Définit le chemin du fichier Markdown généré à convertir en Word, correspondant au nom par défaut utilisé par analyser_entreprise.

296 if Path(chemin_markdown).exists():
→ Vérifie si le fichier Markdown attendu existe bien sur le disque, en utilisant l'API orientée objet de pathlib.Path importée en tête de fichier plutôt que os.path.exists.

297 chemin_rapport_word = analyseur.generer_rapport_word(chemin_markdown)
→ Appelle la méthode qui génère le rapport Word à partir du fichier Markdown et stocke le chemin obtenu.

298 print(f" Analyse terminée ! Rapport sauvegardé : {chemin_rapport_word}")
→ Affiche dans la console un message confirmant la fin de l'analyse et l'emplacement du rapport.

299 else:
→ Cas où le fichier Markdown attendu n'a pas été trouvé.

300 print(" Fichier markdown introuvable. Veuillez vérifier le chemin.")
→ Affiche dans la console un message d'avertissement signalant l'absence du fichier Markdown.

302 except Exception as e:
→ Intercepte toute exception survenue pendant le bloc try, la stockant dans la variable e, garde-fou générique qui englobe aussi bien une clé API manquante qu'une erreur réseau ou de génération du document.

303 print(f" Erreur lors de l'analyse : {e}")
→ Affiche dans la console un message d'erreur incluant le détail de l'exception survenue.

304 raise
→ Relance l'exception interceptée après l'avoir affichée, pour ne pas masquer l'échec : le message est logué proprement, mais le programme se termine bien en erreur plutôt que de continuer silencieusement dans un état incohérent.

306 if __name__ == "__main__":
→ Vérifie que le script est exécuté directement, et non importé comme module, avant d'appeler main.

307 main()
→ Appelle la fonction main, déclenchant l'exécution complète du script.

Projet 30 — 30-compliance_agent

Rapport de conformité réglementaire UE, export Word

Objectif

Génère un rapport de conformité réglementaire pour l'Union Européenne à partir des informations fournies sur une entreprise ou une activité, et exporte ce rapport dans un document Word prêt à être partagé ou archivé.

Code source, ligne par ligne

29-Investment_insight/main.py

Définit une équipe et cinq agents agno (recherche interne, recherche web, équipe de recherche client, analyse financière, recommandations) formant un pipeline de conseil en investissement.

257 lignes de code annotées ci-dessous.

1 from agno.agent import Agent

→ Importe la classe Agent du framework agno. Agno est un framework Python open-source pour construire des agents IA : il gère l'appel au modèle de langage, les instructions systèmes, l'utilisation d'outils externes (function calling) et, comme on le verra plus bas, la coordination de plusieurs agents entre eux via une Team.

2 from agno.team.team import Team

→ Importe la classe Team d'agno, qui permet de faire collaborer plusieurs agents spécialisés sous la supervision d'un agent coordinateur : au lieu qu'un seul agent généraliste tente de tout faire, chaque membre garde un rôle étroit (ici recherche interne vs recherche web), et la Team décide à qui déléguer la tâche et agrège les réponses.

3 from agno.models.openai import OpenAIChat

→ Importe la classe modèle OpenAIChat d'agno, un adaptateur qui permet à un Agent ou une Team de dialoguer avec les modèles de chat de l'API OpenAI (ici gpt-4o-mini, utilisé plus bas pour tous les agents de ce fichier).

4 from agno.models.google import Gemini

→ Importe la classe modèle Gemini d'agno, l'équivalent d'OpenAIChat mais pour les modèles Gemini de Google ; elle est disponible mais reste non utilisée activement dans ce fichier (on la retrouve seulement en commentaire, comme alternative envisagée).

5 from agno.tools.scraper import ScrapeGraphTools

→ Importe l'outil ScrapeGraphTools d'agno, une intégration du service ScrapeGraphAI : contrairement à un scraper classique qui extrait du HTML brut, ScrapeGraphAI utilise lui-même un modèle de langage pour interpréter une page web et en extraire un contenu structuré et pertinent, ce qui simplifie l'extraction d'informations depuis des pages hétérogènes.

6 from agno.tools.duckduckgo import DuckDuckGoTools

→ Importe l'outil de recherche web DuckDuckGo d'agno, bien que non instancié plus loin dans ce fichier (les agents utilisent ExaTools et ScrapeGraphTools à la place pour la recherche web).

7 from agno.tools.file import FileTools

→ Importe l'outil FileTools d'agno, qui donne à un agent la capacité d'écrire et lire des fichiers sur le disque local (par exemple financial_data.md ou recommendation.md plus bas) : sans cet outil, un agent ne peut que renvoyer du texte dans sa réponse, il ne peut pas produire de fichier persistant de lui-même.

8 from agno.tools.yfinance import YFinanceTools

→ Importe l'outil YFinanceTools d'agno, qui encapsule la bibliothèque `yfinance` pour donner à un agent un accès structuré aux données financières de Yahoo Finance (cours de bourse, informations sur l'entreprise, ratios financiers, recommandations d'analystes), sans que le modèle ait besoin d'aller chercher ces chiffres par recherche web générique.

```
9 from agno.tools.exa import ExaTools
```

→ Importe l'outil ExaTools d'agno, une intégration de l'API Exa : un moteur de recherche dit « neuronal » ou sémantique, conçu spécifiquement pour les agents IA, qui comprend le sens d'une requête plutôt que de faire un simple appariement de mots-clés comme un moteur de recherche classique, ce qui donne des résultats souvent plus pertinents pour des recherches complexes.

```
11 from dotenv import load_dotenv
```

→ Importe `load\_dotenv` de python-dotenv, qui charge les clés API stockées dans un fichier .env local plutôt que codées en dur dans le script, pour des raisons de sécurité et de portabilité.

```
12 import os
```

→ Importe le module standard `os`, utilisé ici pour lire les variables d'environnement contenant les clés API.

```
14 # Configuration
```

→ Commentaire annonçant la section de configuration du script.

```
15 load_dotenv()
```

→ Charge les variables définies dans le fichier .env dans l'environnement du processus.

```
17 sgai_api_key = os.getenv("SGAI_API_KEY")
```

→ Récupère la valeur de la variable d'environnement SGAI_API_KEY (la clé du service ScrapeGraphAI) et la stocke dans sgai_api_key.

```
18 exa_api_key = os.getenv("EXA_API_KEY")
```

→ Récupère la valeur de la variable d'environnement EXA_API_KEY (la clé du service de recherche Exa) et la stocke dans exa_api_key.

```
19 api_key_openai = os.getenv("OPENAI_API_KEY")
```

→ Récupère la valeur de la variable d'environnement OPENAI_API_KEY et la stocke dans api_key_openai, utilisée pour authentifier tous les modèles OpenAIChat de ce fichier.

```
21 # Agent 1: Internal Search Agent
```

→ Commentaire annonçant la définition du premier agent, chargé de la recherche interne.

```
22 internal_search = Agent(
```

→ Commence la création de l'agent nommé internal_search, dont le rôle est étroitement limité à interroger une base de données locale de clients — un exemple de spécialisation d'agent : il ne fait qu'une chose, ce qui rend son comportement plus prévisible qu'un agent généraliste.

```
23     name="internal search agent",
```

→ Définit le nom affiché de cet agent comme "internal search agent".

```
24     # model=Gemini(id=id, api_key=api_key),
```

→ Ligne de code commentée montrant une configuration alternative avec le modèle Gemini, laissée en place comme trace d'une option envisagée mais non retenue, sans effet sur l'exécution.

```
25     model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
```

→ Configure le modèle utilisé par cet agent : gpt-4o-mini, une version allégée et moins coûteuse de GPT-4o adaptée à des tâches simples comme la synthèse de notes de réunion, authentifiée avec la clé OpenAI.

```
26     tools=[FileTools()],
```

→ Fournit à l'agent l'outil FileTools, qui lui permet de lire le fichier local de données clients (29-Investment_insight/Data/data.json) directement, plutôt que de recevoir ce contenu déjà inséré dans le prompt par le code Python.

```
27     description="""
```

→ Commence la chaîne de description multi-lignes de l'agent, sans utiliser dedent (contrairement au fichier 27, ici l'indentation résiduelle du texte n'est pas retirée — différence de style entre les deux projets, sans impact fonctionnel majeur car le modèle tolère bien des espaces en trop).

```
28 A data analyst agent that retrieves and summarizes historical client meeting records
```

→ Ligne de description indiquant que l'agent récupère et résume les comptes rendus historiques de réunions clients.

29 from the internal database ('29-Investment_insight/Data/data.json').

→ Ligne de description précisant le chemin du fichier JSON servant de base de données interne.

30 The agent provides concise, insight-driven summaries based on each meeting entry.

→ Ligne de description précisant que l'agent fournit des résumés concis et riches en insights pour chaque réunion.

31 "",

→ Ferme la chaîne de description de l'agent. Le paramètre `description` d'agno sert de contexte général sur le rôle de l'agent, complémentaire aux `instructions` qui donnent des consignes opérationnelles plus précises.

32 instructions=""

→ Commence la chaîne d'instructions multi-lignes de l'agent, sans utiliser dedent.

33 You are given access to a JSON file containing client information, including detailed meeting notes.

→ Ligne d'instruction précisant que l'agent a accès à un fichier JSON contenant les informations clients et notes de réunion.

35 Your task is to:

→ Ligne d'instruction introduisant la liste des tâches à accomplir.

36 1. Search for the specified client by full name in the '29-Investment_insight/Data/data.json' file.

→ Ligne d'instruction demandant de rechercher le client par nom complet dans le fichier JSON indiqué.

37 2. If the client is found, focus on the 'meeting_notes' field.

→ Ligne d'instruction demandant de se concentrer sur le champ meeting_notes si le client est trouvé.

38 3. For each note, write a **professional, well-structured summary, and actions** using markdown bullet format like this:

→ Ligne d'instruction demandant de rédiger un résumé structuré et des actions au format à puces Markdown.

39 # Internal Search:

→ Ligne d'instruction montrant le titre Markdown attendu pour la section de recherche interne.

40 - <Date>: Summary

→ Ligne d'instruction montrant le format attendu d'une puce résumant une réunion datée.

41 - **Action Items**:

→ Ligne d'instruction montrant le format attendu d'une puce listant les actions à mener.

43 Notes:

→ Ligne d'instruction introduisant des remarques complémentaires de rédaction.

44 - Summarize clearly; do not just copy/paste raw content.

→ Ligne d'instruction demandant un résumé clair plutôt qu'une copie brute du contenu.

45 - Group related ideas. Be concise but informative.

→ Ligne d'instruction demandant de regrouper les idées liées tout en restant concis et informatif.

46 - Ensure the tone is suitable for executive review.

→ Ligne d'instruction demandant un ton adapté à une lecture par des dirigeants.

48 Final response must be:

→ Ligne d'instruction introduisant les critères de forme de la réponse finale.

49 - Well-structured using Markdown

→ Ligne d'instruction exigeant une réponse bien structurée en Markdown.

50 - Professional in tone

→ Ligne d'instruction exigeant un ton professionnel.

51 - Only include one of the two sections depending on the results.

→ Ligne d'instruction exigeant de n'inclure qu'une seule des deux sections possibles selon le résultat.

54 If the client is not found, respond exactly with: CLIENT NOT FOUND.

→ Ligne d'instruction imposant une réponse exacte et normalisée quand le client n'est pas trouvé — ce texte fixe sert de signal machine-à-machine : la Team définie plus bas (ligne 117) détecte cette chaîne précise pour décider de déléguer à l'agent de recherche web, un pattern simple de routage conditionnel entre agents basé sur le contenu de la réponse.

55 """

→ Ferme la chaîne d'instructions de l'agent.

56 stream=True

→ Active le mode streaming : plutôt que d'attendre la réponse complète du modèle avant de la renvoyer, agno la transmet token par token au fur et à mesure de sa génération, ce qui réduit la latence perçue par l'utilisateur (comme un texte qui s'écrit progressivement) même si le temps de génération total est identique.

57)

→ Ferme l'appel au constructeur Agent, terminant la définition de internal_search.

59 # Agent 2: Web Search Agent

→ Commentaire annonçant la définition du deuxième agent, chargé de la recherche web.

60 web_search = Agent(

→ Commence la création de l'agent nommé web_search, activé uniquement en secours quand la recherche interne échoue (voir l'instruction ligne 71) : une logique de repli (fallback) entre deux sources d'information de fiabilité différente.

61 name="web search agent",

→ Définit le nom affiché de cet agent comme "web search agent".

62 # model=Gemini(id=id, api_key=api_key),

→ Ligne de code commentée montrant une configuration alternative avec le modèle Gemini, non utilisée.

63 model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),

→ Configure le modèle OpenAIChat gpt-4o-mini utilisé par cet agent, authentifié avec la clé OpenAI.

64 tools=[ScrapeGraphTools(api_key=sgai_api_key, markdownify=True), ExaTools(api_key=exa_api_key)],

→ Fournit à l'agent deux outils complémentaires pour la recherche web : ExaTools pour trouver des pages pertinentes via recherche sémantique, puis ScrapeGraphTools pour en extraire le contenu utile — l'option `markdownify=True` demande que le contenu scrapé soit renvoyé directement en Markdown, format facilement réinjectable dans un prompt ou un rapport.

65 description="""

→ Commence la chaîne de description multi-lignes de l'agent.

66 A web intelligence agent activated only when a client is not found in the internal records.

→ Ligne de description précisant que l'agent n'est activé que si le client est absent des dossiers internes.

67 It searches the web to uncover publicly available, up-to-date information about the client,

→ Ligne de description précisant que l'agent recherche sur le web des informations publiques et à jour sur le client.

68 focusing on professional activity and recent public appearances.

→ Ligne de description précisant que la recherche se concentre sur l'activité professionnelle et les apparitions publiques récentes.

69 """

→ Ferme la chaîne de description de l'agent.

70 instructions="""

→ Commence la chaîne d'instructions multi-lignes de l'agent.

71 Only respond if the internal search agent confirms: "CLIENT NOT FOUND".

→ Ligne d'instruction conditionnant l'exécution de l'agent à la confirmation "CLIENT NOT FOUND" de l'agent interne, reprenant le signal textuel défini plus haut (ligne 54).

73 Step 1: Use ExaTools to search the web for the client's full name.

→ Ligne d'instruction décrivant la première étape, rechercher le nom complet du client via ExaTools.

74 Step 2: Identify relevant links such as:

→ Ligne d'instruction décrivant la deuxième étape, identifier des liens pertinents listés ci-dessous.

75 - Personal websites

→ Ligne d'instruction citant les sites web personnels comme type de lien pertinent à identifier.

76 - LinkedIn, GitHub, or other professional profiles

→ Ligne d'instruction citant les profils professionnels comme type de lien pertinent à identifier.

77 - News articles or blog posts

→ Ligne d'instruction citant les articles de presse ou de blog comme type de lien pertinent à identifier.

78 - Conference appearances, guest speaking events, or publications

→ Ligne d'instruction citant les conférences et publications comme type de lien pertinent à identifier.

80 Step 3: Use ScrapeGraphTools to scrape **only the most relevant and recent pages**.

→ Ligne d'instruction décrivant la troisième étape, ne scraper que les pages les plus pertinentes et récentes — une consigne qui limite le nombre d'appels à l'outil de scraping, chaque appel ayant un coût en temps et en tokens.

81 Step 4: Summarize any recent or notable findings into a **clear, executive-level markdown summary**.

→ Ligne d'instruction décrivant la quatrième étape, résumer les découvertes en un résumé Markdown de niveau exécutif.

83 Formatting guidelines:

→ Ligne d'instruction introduisant les consignes de mise en forme de la réponse.

84 - Use bullet points per finding.

→ Ligne d'instruction demandant d'utiliser une puce par découverte.

85 - Each point should include a concise description of the event, post, or appearance.

→ Ligne d'instruction demandant à chaque puce d'inclure une description concise de l'événement.

86 - Keep it factual, neutral, and informative – suitable for a professional briefing.

→ Ligne d'instruction demandant un contenu factuel, neutre et adapté à un briefing professionnel.

87 - If no relevant information is found, respond with: "No public information found."

→ Ligne d'instruction imposant une réponse normalisée en l'absence d'information publique trouvée.

88 """

→ Ferme la chaîne d'instructions de l'agent.

89 show_tool_calls=True,

→ Active l'affichage des appels d'outils effectués par l'agent dans les logs/console : utile en développement pour voir exactement quelles requêtes ExaTools ou ScrapeGraphTools ont été déclenchées et avec quels arguments, ce qui aide à déboguer un comportement inattendu de l'agent.

90 stream=True,

→ Active le mode streaming pour que les réponses de l'agent soient renvoyées progressivement.

91)

→ Ferme l'appel au constructeur Agent, terminant la définition de web_search.

93 # Team: Client Research Team

→ Commentaire annonçant la définition de l'équipe coordonnant les deux agents de recherche.

94 client_research = Team(

→ Commence la création de l'équipe nommée client_research, instance de la classe Team d'agno : contrairement à un Agent isolé, une Team reçoit une requête, décide (via son propre modèle) à quel(s) membre(s) la déléguer, et peut composer une réponse finale à partir des réponses des membres.

95 name="Client Research Agent",

→ Définit le nom affiché de cette équipe comme "Client Research Agent".

96 # model=Gemini(id=id, api_key=api_key),

→ Ligne de code commentée montrant une configuration alternative avec le modèle Gemini, non utilisée.

97 model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),

→ Configure le modèle OpenAIChat gpt-4o-mini utilisé pour coordonner l'équipe (raisonner sur qui déléguer et comment agréger), authentifié avec la clé OpenAI ; c'est un modèle distinct de ceux des membres, qui ont chacun le leur.

98 members=[internal_search, web_search],

→ Définit les membres de l'équipe comme étant les agents internal_search et web_search déjà construits plus haut : la Team ne redéfinit pas leur comportement, elle les réutilise tels quels comme sous-agents spécialisés.

99 # model="coordinate",

→ Ligne de code commentée montrant un mode de coordination alternatif non utilisé (agno propose différents modes de fonctionnement pour une Team, comme la délégation coordonnée par un modèle superviseur ou l'exécution collaborative en parallèle).

100 tools=[],

→ Ne fournit aucun outil supplémentaire à l'équipe elle-même, au-delà de ceux de ses membres — la Team ne fait qu'orchestrer, elle ne recherche rien elle-même directement.

101 description="""

→ Commence la chaîne de description multi-lignes de l'équipe.

102 A high-level research agent that coordinates two specialized sub-agents:

→ Ligne de description précisant que l'équipe coordonne deux sous-agents spécialisés.

103 - An internal search agent that queries local client records.

→ Ligne de description citant le sous-agent de recherche interne interrogeant les dossiers locaux.

104 - A web search agent that performs external research if internal data is missing.

→ Ligne de description citant le sous-agent de recherche web utilisé si les données internes manquent.

106 This agent orchestrates both and produces a professional, structured summary about the client.

→ Ligne de description précisant que l'équipe orchestre les deux agents et produit un résumé structuré du client.

107 """

→ Ferme la chaîne de description de l'équipe.

108 instructions="""

→ Commence la chaîne d'instructions multi-lignes de l'équipe.

109 Your goal is to gather insights about a client using two research sources:

→ Ligne d'instruction énonçant l'objectif de rassembler des informations sur un client via deux sources.

111 1. First, delegate the task to the ****internal search agent**** to locate any existing internal meeting records about the client.

→ Ligne d'instruction demandant de déléguer d'abord la tâche à l'agent de recherche interne.

112 2. If the internal agent returns information, format it under the section titled:

→ Ligne d'instruction demandant de formater le résultat de l'agent interne sous un titre de section précis.

113 # Internal Search:

→ Ligne d'instruction montrant le titre Markdown attendu pour la section de recherche interne.

114 - Date: Summary

→ Ligne d'instruction montrant le format attendu d'une puce résumant une réunion datée.

115 - **Action Items**: if any

→ Ligne d'instruction montrant le format attendu d'une puce listant les actions à mener, si applicable.

117 3. If the internal agent replies "CLIENT NOT FOUND", delegate the task to the **web search agent**.

→ Ligne d'instruction demandant de déléguer la tâche à l'agent de recherche web si le client n'est pas trouvé en interne, appliquant concrètement la logique de repli déjà préparée dans les instructions des deux agents membres.

118 4. If the web search agent returns anything, format it under the section:

→ Ligne d'instruction demandant de formater le résultat de l'agent web sous un titre de section précis.

119 # Web Search:

→ Ligne d'instruction montrant le titre Markdown attendu pour la section de recherche web.

120 - Summarized bullet points from recent or relevant online content

→ Ligne d'instruction montrant le format attendu d'une puce résumant du contenu en ligne pertinent.

122 Final response must be:

→ Ligne d'instruction introduisant les critères de forme de la réponse finale de l'équipe.

123 - Well-structured using Markdown

→ Ligne d'instruction exigeant une réponse bien structurée en Markdown.

124 - Professional in tone

→ Ligne d'instruction exigeant un ton professionnel.

125 - Only include one of the two sections depending on the results.

→ Ligne d'instruction exigeant de n'inclure qu'une seule des deux sections possibles selon le résultat.

126 "",

→ Ferme la chaîne d'instructions de l'équipe.

127 stream=True,

→ Active le mode streaming pour que les réponses de l'équipe soient renvoyées progressivement.

128 show_members_responses=True,

→ Active l'affichage des réponses individuelles de chaque agent membre de l'équipe, en plus de la réponse finale agrégée : permet de voir séparément ce qu'a répondu internal_search et ce qu'a répondu web_search, utile pour comprendre ou déboguer la décision de délégation de la Team.

129)

→ Ferme l'appel au constructeur Team, terminant la définition de client_research.

131 # Agent 3: Financial Data Agent

→ Commentaire annonçant la définition du troisième agent, chargé des données financières.

132 financial_data_agent = Agent(

→ Commence la création de l'agent nommé financial_data_agent, qui combine plusieurs outils (données de marché, fichiers,

recherche web) pour produire une analyse d'investissement étayée.

133 `name="Financial Intelligence Agent",`

→ Définit le nom affiché de cet agent comme "Financial Intelligence Agent".

134 `model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),`

→ Configure le modèle OpenAIChat gpt-4o-mini utilisé par cet agent, authentifié avec la clé OpenAI.

135 `tools=[`

→ Commence la liste des outils fournis à cet agent.

136 `YFinanceTools(`

→ Commence la configuration de l'outil YFinanceTools, qui donne à l'agent un accès direct et structuré aux données de marché de Yahoo Finance, plus fiable pour ce type de données précises qu'une recherche web générique qui pourrait renvoyer des chiffres obsolètes ou mal formatés.

137 `stock_price=True,`

→ Active la récupération du prix de l'action par l'outil YFinanceTools.

138 `company_info=True,`

→ Active la récupération des informations générales sur l'entreprise par l'outil YFinanceTools.

139 `stock_fundamentals=True,`

→ Active la récupération des données fondamentales de l'action (par exemple capitalisation, ratios de valorisation) par l'outil YFinanceTools.

140 `key_financial_ratios=True,`

→ Active la récupération des ratios financiers clés (par exemple P/E) par l'outil YFinanceTools.

141 `analyst_recommendations=True,`

→ Active la récupération des recommandations d'analystes par l'outil YFinanceTools.

142 `),`

→ Ferme la configuration de l'outil YFinanceTools.

143 `FileTools(),`

→ Ajoute l'outil FileTools à la liste, permettant à l'agent de lire et écrire des fichiers comme financial_data.md directement, sans passer par le code Python appelant.

144 `ExaTools(api_key=exa_api_key),`

→ Ajoute l'outil ExaTools à la liste, authentifié avec la clé Exa, pour effectuer des recherches web sémantiques (par exemple trouver des articles d'analystes ou des actualités sur l'entreprise).

145 `ScrapeGraphTools(api_key=sgai_api_key, markdownify=True),`

→ Ajoute l'outil ScrapeGraphTools à la liste, authentifié avec la clé SGAI, pour extraire le contenu utile des pages web trouvées par ExaTools, converti directement en Markdown.

146 `],`

→ Ferme la liste des outils fournis à cet agent.

147 `description="""`

→ Commence la chaîne de description multi-lignes de l'agent.

148 `A financial analysis expert agent responsible for researching specific investment opportunities based on client context.`

→ Ligne de description précisant que l'agent est expert en analyse financière des opportunités d'investissement.

149 `This agent can extract financial data, summarize earnings calls, and provide balanced investment recommendations.`

→ Ligne de description précisant les capacités de l'agent, extraction de données, résumé d'earnings calls, recommandations

équilibrées.

151 It uses insights from the client's research and combines them with real-time financial data

→ Ligne de description précisant que l'agent combine les informations du client avec des données financières en temps réel.

152 and qualitative sources to support decision-making.

→ Ligne de description précisant que l'agent utilise aussi des sources qualitatives pour appuyer la prise de décision.

153 """,

→ Ferme la chaîne de description de l'agent.

154 instructions="""

→ Commence la chaîne d'instructions multi-lignes de l'agent.

155 You are a financial analyst agent tasked with evaluating a stock as an investment opportunity. Your output must strictly follow the structure outlined below.

→ Ligne d'instruction présentant le rôle d'analyste financier et l'obligation de suivre une structure stricte.

157 Start by understanding the client's investment preferences, goals, and risk tolerance. Tailor your recommendations to align with that profile.

→ Ligne d'instruction demandant d'adapter l'analyse aux préférences et à la tolérance au risque du client.

159 End by Saving the output in a file called 'financial_data.md'.

→ Ligne d'instruction demandant de sauvegarder la sortie finale dans un fichier nommé financial_data.md, action que l'agent réalise lui-même via l'outil FileTools plutôt que via le code Python appelant.

161 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

162 Analyze [Stock Name] as an investment opportunity using this exact structure:

→ Ligne d'instruction introduisant le gabarit exact à suivre pour analyser l'action.

164 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

165 ****[Stock Name] Overview****

→ Ligne du gabarit définissant le titre en gras de la section de présentation de l'action.

166 [1-2 sentences summarizing the company's core business and differentiation.]

→ Ligne du gabarit indiquant l'espace réservé pour un résumé de l'activité principale de l'entreprise.

168 ****Pros of Investing in [Stock Name]****

→ Ligne du gabarit définissant le titre en gras de la section des avantages d'investissement.

169 - ****Innovative Technology****: [Detail unique tech/advantages, if applicable.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les avantages technologiques uniques.

170 - ****Strategic Partnerships****: [List key partnerships/alliances.]

→ Ligne du gabarit indiquant l'espace réservé pour lister les partenariats stratégiques clés.

171 - ****Market Demand****: [Highlight growth sectors/tailwinds.]

→ Ligne du gabarit indiquant l'espace réservé pour mettre en avant les secteurs de croissance porteurs.

172 - ****Regulatory Support****: [Mention favorable policies/risks.]

→ Ligne du gabarit indiquant l'espace réservé pour mentionner les politiques favorables ou les risques réglementaires.

173 - ****Revenue Potential****: [Identify high-margin segments or opportunities.]

→ Ligne du gabarit indiquant l'espace réservé pour identifier les segments à forte marge.

175 ****Cons of Investing in [Stock Name]****

→ Ligne du gabarit définissant le titre en gras de la section des inconvénients d'investissement.

176 - ****Regulatory Challenges****: [Permitting delays/policy risks.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les défis réglementaires et retards de permis.

177 - ****Financial Health****: [Losses/debt/cash burn concerns.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les préoccupations sur la santé financière.

178 - ****Market Risks****: [Supply chain/competition/volatility.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les risques de marché.

179 - ****Execution Risks****: [Management hurdles/operational uncertainties.]

→ Ligne du gabarit indiquant l'espace réservé pour détailler les risques d'exécution liés à la gestion.

181 ****Overall****: [Stock Name] presents [summary of opportunity], but investors should weigh [key risks].

→ Ligne du gabarit définissant la phrase de conclusion globale résumant l'opportunité et les risques.

183 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

185 Use the following data sources to inform your analysis (in priority order):

→ Ligne d'instruction introduisant la liste ordonnée des sources de données à utiliser.

186 1. Latest earnings calls (Q2/Q3 2024).

→ Ligne d'instruction citant les derniers earnings calls comme première source de données prioritaire.

187 2. SEC filings (10-K/10-Q).

→ Ligne d'instruction citant les dépôts réglementaires SEC comme deuxième source de données prioritaire.

188 3. Analyst reports (e.g., Bloomberg, Reuters).

→ Ligne d'instruction citant les rapports d'analystes comme troisième source de données prioritaire.

189 4. Industry trends related to the company (e.g., AI, energy transition, EV, etc.).

→ Ligne d'instruction citant les tendances sectorielles comme quatrième source de données prioritaire.

190 5. Financial metrics using YFinance (stock fundamentals, key ratios, analyst recommendations).

→ Ligne d'instruction citant les métriques financières via YFinance comme cinquième source de données prioritaire, reliant explicitement cette consigne à l'outil YFinanceTools configuré plus haut.

192 Important:

→ Ligne d'instruction signalant une consigne importante à suivre avant la sauvegarde.

193 Before saving your output to 'financial_data.md', perform this final step:

→ Ligne d'instruction demandant d'effectuer une étape finale avant l'enregistrement du fichier.

194 - Replace all smart punctuation (like '‘', '’', '■', '„', '–') with standard ASCII characters (e.g., '"', "'", '-').

→ Ligne d'instruction demandant de remplacer la ponctuation typographique (guillemets courbes, tirets longs, souvent générés par les modèles de langage) par des caractères ASCII standards, afin d'éviter des problèmes d'affichage ou d'encodage lors de la relecture du fichier par d'autres outils.

195 - Save the output in UTF-8 encoding using FileTools.

→ Ligne d'instruction demandant de sauvegarder le fichier en encodage UTF-8 via FileTools, un encodage universel qui garantit que tous les caractères (y compris accents et symboles) sont préservés correctement.

197 Your output must be:

→ Ligne d'instruction introduisant les critères finaux attendus pour la sortie.

198 - Clean, well-formatted Markdown

→ Ligne d'instruction exigeant un Markdown propre et bien formaté.

199 - Free of encoding issues

→ Ligne d'instruction exigeant l'absence de problèmes d'encodage.

200 - Saved in a file called 'financial_data.md'

→ Ligne d'instruction exigeant que le fichier soit sauvegardé sous le nom financial_data.md.

201 """

→ Ferme la chaîne d'instructions de l'agent.

202 success_criteria="Your task is only complete when the output is saved to 'financial_data.md'.",

→ Définit le critère de succès de l'agent : ce paramètre agno indique au modèle qu'il ne doit pas s'arrêter après une simple réponse textuelle, mais continuer à agir (ici, appeler réellement FileTools pour écrire le fichier) tant que la condition énoncée n'est pas remplie — c'est ce qui transforme l'agent d'un simple générateur de texte en un agent véritablement « agissant ».

203 stream=True,

→ Active le mode streaming pour que les réponses de l'agent soient renvoyées progressivement.

204)

→ Ferme l'appel au constructeur Agent, terminant la définition de financial_data_agent.

206 # Agent 4:Recommendation Agent

→ Commentaire annonçant la définition du quatrième agent, chargé des recommandations.

207 recommendation_agent = Agent(

→ Commence la création de l'agent nommé recommendation_agent, qui lit le fichier produit par financial_data_agent et le transforme en un rapport de recommandation formel, illustrant un pipeline où chaque agent consomme la sortie fichier du précédent plutôt qu'un simple passage de texte en mémoire.

208 name="Financial Advisor Agent",

→ Définit le nom affiché de cet agent comme "Financial Advisor Agent".

209 model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),

→ Configure le modèle OpenAIChat gpt-4o-mini utilisé par cet agent, authentifié avec la clé OpenAI.

210 tools=[FileTools(), YFinanceTools(

→ Commence la liste des outils fournis à cet agent en ajoutant FileTools (pour lire financial_data.md et écrire recommendation.md) puis le début de la configuration de YFinanceTools (pour compléter avec des données de marché à jour).

211 stock_price=True,

→ Active la récupération du prix de l'action par l'outil YFinanceTools.

212 company_info=True,

→ Active la récupération des informations générales sur l'entreprise par l'outil YFinanceTools.

213 stock_fundamentals=True,

→ Active la récupération des données fondamentales de l'action par l'outil YFinanceTools.

214 key_financial_ratios=True,

→ Active la récupération des ratios financiers clés par l'outil YFinanceTools.

215 analyst_recommendations=True,

→ Active la récupération des recommandations d'analystes par l'outil YFinanceTools.

216), ExaTools(api_key=exa_api_key)],

→ Ferme la configuration de YFinanceTools et ajoute ExaTools authentifié, terminant la liste des outils.

217 description="""

→ Commence la chaîne de description multi-lignes de l'agent.

218 A financial advisory agent responsible for issuing structured investment recommendations based on prior financial analysis.

→ Ligne de description précisant que l'agent émet des recommandations d'investissement structurées basées sur l'analyse préalable.

219 It reads data from 'financial_data.md' and produces a detailed recommendation report using a fixed template,

→ Ligne de description précisant que l'agent lit financial_data.md et produit un rapport selon un gabarit fixe.

220 including both qualitative insights and quantitative metrics.

→ Ligne de description précisant que le rapport combine des observations qualitatives et des métriques quantitatives.

221 """ ,

→ Ferme la chaîne de description de l'agent.

222 instructions="""

→ Commence la chaîne d'instructions multi-lignes de l'agent.

223 You are tasked with generating a professional stock recommendation report and save it as a 'recommendation.md' file.

→ Ligne d'instruction énonçant la tâche de générer un rapport de recommandation et de le sauvegarder sous recommendation.md.

225 Step 1: Load and read the file 'financial_data.md'. It contains a prior detailed analysis of the stock's performance, risks, and potential.

→ Ligne d'instruction décrivant la première étape, charger et lire le fichier financial_data.md — via l'outil FileTools, qui donne littéralement à l'agent une capacité de lecture de fichier qu'un simple appel de modèle de langage n'a pas nativement.

227 Step 2: Based on the data in that file and YFinance tools, complete the following structured markdown template with realistic and accurate values:

→ Ligne d'instruction décrivant la deuxième étape, compléter le gabarit Markdown avec des valeurs réalistes.

229 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

230 # **Company Details**:

→ Ligne du gabarit définissant le titre Markdown de la section détails de l'entreprise.

232 | Name | Rating | Target | ISIN | Last Price | Currency | Sector | Sector Rating |

→ Ligne du gabarit définissant l'en-tête du tableau Markdown des détails de l'entreprise.

233 |-----|-----|-----|-----|-----|-----|-----|-----|

→ Ligne du gabarit définissant la ligne de séparation du tableau Markdown (syntaxe standard des tableaux Markdown, entre l'en-tête et les données).

234 | [Company] | [Rating] | [Price] | [ISIN] | \$XX.XX | [Ccy] | [Sector] | [Sector Rating] |

→ Ligne du gabarit définissant la ligne de données du tableau avec les espaces réservés à remplir.

236 Notes:

→ Ligne d'instruction introduisant des remarques complémentaires pour remplir le gabarit.

237 - Make sure all values are consistent with the financial insights in the input file.

→ Ligne d'instruction demandant la cohérence des valeurs avec les données du fichier d'entrée.

238 - Maintain a professional tone suitable for investment committees or client reports.

→ Ligne d'instruction demandant un ton professionnel adapté aux comités d'investissement.

239 - Use markdown formatting to ensure the output is clean and readable.

→ Ligne d'instruction demandant un formatage Markdown propre et lisible.

241 Step 3: Save the final output as a 'recommendation.md' file.

→ Ligne d'instruction décrivant la troisième étape, sauvegarder la sortie finale sous recommendation.md.

242 """ ,

→ Ferme la chaîne d'instructions de l'agent.

243 stream=True,

→ Active le mode streaming pour que les réponses de l'agent soient renvoyées progressivement.

244)

→ Ferme l'appel au constructeur Agent, terminant la définition de recommendation_agent.

246 # Agent 5: Financial Advisor Agent (Alternative Template)

→ Commentaire annonçant la définition du cinquième agent, variante du précédent avec un autre gabarit.

247 recommendation_agent_v2 = Agent(

→ Commence la création de l'agent nommé recommendation_agent_v2, identique en configuration à recommendation_agent mais avec un gabarit de sortie différent (rating Buy/Hold/Sell et tableau de métriques boursières plutôt qu'un tableau d'entreprise) : une variante de présentation offerte au choix plutôt qu'une nécessité fonctionnelle.

248 name="Financial Advisor Agent",

→ Définit le nom affiché de cet agent comme "Financial Advisor Agent".

249 model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),

→ Configure le modèle OpenAIChat gpt-4o-mini utilisé par cet agent, authentifié avec la clé OpenAI.

250 tools=[FileTools(), YFinanceTools(

→ Commence la liste des outils fournis à cet agent en ajoutant FileTools puis le début de la configuration de YFinanceTools.

251 stock_price=True,

→ Active la récupération du prix de l'action par l'outil YFinanceTools.

252 company_info=True,

→ Active la récupération des informations générales sur l'entreprise par l'outil YFinanceTools.

253 stock_fundamentals=True,

→ Active la récupération des données fondamentales de l'action par l'outil YFinanceTools.

254 key_financial_ratios=True,

→ Active la récupération des ratios financiers clés par l'outil YFinanceTools.

255 analyst_recommendations=True,

→ Active la récupération des recommandations d'analystes par l'outil YFinanceTools.

256), ExaTools(api_key=exa_api_key)],

→ Ferme la configuration de YFinanceTools et ajoute ExaTools authentifié, terminant la liste des outils.

257 description="""

→ Commence la chaîne de description multi-lignes de l'agent.

258 A financial advisory agent responsible for issuing structured investment recommendations based on prior financial analysis.

→ Ligne de description précisant que l'agent émet des recommandations d'investissement structurées basées sur l'analyse préalable.

259 It reads data from 'financial_data.md' and produces a detailed recommendation report using a fixed

template,

→ Ligne de description précisant que l'agent lit financial_data.md et produit un rapport selon un gabarit fixe.

260 including both qualitative insights and quantitative metrics.

→ Ligne de description précisant que le rapport combine des observations qualitatives et des métriques quantitatives.

261 """,

→ Ferme la chaîne de description de l'agent.

262 instructions="""

→ Commence la chaîne d'instructions multi-lignes de l'agent.

263 You are tasked with generating a professional stock recommendation report and save it as a 'recommendation.md' file.

→ Ligne d'instruction énonçant la tâche de générer un rapport de recommandation et de le sauvegarder sous recommendation.md.

265 Step 1: Load and read the file 'financial_data.md'. It contains a prior detailed analysis of the stock's performance, risks, and potential.

→ Ligne d'instruction décrivant la première étape, charger et lire le fichier financial_data.md.

267 Step 2: Based on the data in that file and YFinance tools, complete the following structured markdown template with realistic and accurate values:

→ Ligne d'instruction décrivant la deuxième étape, compléter le gabarit Markdown alternatif avec des valeurs réalistes.

269 ---

→ Ligne d'instruction contenant un séparateur Markdown horizontal dans le texte des instructions.

270 [Company Name] Stock Recommendation

→ Ligne du gabarit définissant le titre de la recommandation avec l'espace réservé pour le nom de l'entreprise.

272 **Rating**: [Buy / Hold / Sell]

→ Ligne du gabarit définissant le champ de notation avec les valeurs possibles Buy, Hold ou Sell.

273 **Target Price**: [\$XX.XX]

→ Ligne du gabarit définissant le champ du prix cible avec l'espace réservé au montant.

275 **Rationale**:

→ Ligne du gabarit définissant le titre en gras de la section justification.

277 - **Financial Health**: [Brief summary of financials - EPS, revenue trends, profitability]

→ Ligne du gabarit indiquant l'espace réservé pour le résumé de la santé financière.

278 - **Regulatory Environment**: [Summary of policy or compliance issues]

→ Ligne du gabarit indiquant l'espace réservé pour le résumé de l'environnement réglementaire.

279 - **Growth Initiatives**: [Acquisitions, partnerships, product pipelines]

→ Ligne du gabarit indiquant l'espace réservé pour les initiatives de croissance.

280 - **Market Position**: [Competitive strengths or threats]

→ Ligne du gabarit indiquant l'espace réservé pour la position concurrentielle.

281 - **Risk Factors**: [Short/long-term risks or volatility factors]

→ Ligne du gabarit indiquant l'espace réservé pour les facteurs de risque à court et long terme.

283 **Overall**: [1-2 sentence summary - should reflect the company's outlook and your advised action]

→ Ligne du gabarit définissant le résumé global final avec la perspective et l'action conseillée.

285 **Key Metrics**:

→ Ligne du gabarit définissant le titre en gras de la section des indicateurs clés.

287 | Metric | Value |

→ Ligne du gabarit définissant l'en-tête du tableau Markdown des indicateurs clés.

288 | ----- | ----- |

→ Ligne du gabarit définissant la ligne de séparation du tableau des indicateurs clés.

289 | Open Price | \$XX.XX |

→ Ligne du gabarit définissant la ligne du tableau pour le prix d'ouverture.

290 | Market Cap | \$X.XXB |

→ Ligne du gabarit définissant la ligne du tableau pour la capitalisation boursière.

291 | 52-Week High | \$XX.XX |

→ Ligne du gabarit définissant la ligne du tableau pour le plus haut sur 52 semaines.

292 | 52-Week Low | \$XX.XX |

→ Ligne du gabarit définissant la ligne du tableau pour le plus bas sur 52 semaines.

293 | P/E Ratio | XX.XX |

→ Ligne du gabarit définissant la ligne du tableau pour le ratio cours/bénéfice.

294 | Volume | X,XXX,XXX |

→ Ligne du gabarit définissant la ligne du tableau pour le volume échangé.

295 | Volatility (30D) | XX% |

→ Ligne du gabarit définissant la ligne du tableau pour la volatilité sur 30 jours.

296 "",

→ Ferme la chaîne d'instructions de l'agent.

297 stream=True,

→ Active le mode streaming pour que les réponses de l'agent soient renvoyées progressivement.

298)

→ Ferme l'appel au constructeur Agent, terminant la définition de recommendation_agent_v2.

300 # Exemples d'utilisation

→ Commentaire annonçant la section d'exemples d'utilisation exécutée en fin de script.

301 if __name__ == "__main__":

→ Vérifie que le script est exécuté directement, et non importé comme module, avant de lancer les exemples — convention Python standard, ici sans interface graphique (contrairement au fichier 27), les tests s'exécutent directement en console.

302 # Test des agents

→ Commentaire annonçant les appels de test des différents agents.

303 internal_search.print_response("give me info about Nicole Robinson", markdown=True)

→ Exécute l'agent de recherche interne sur une requête test et affiche la réponse formatée en Markdown dans la console ; `print_response` est une méthode pratique d'agno qui combine l'appel au modèle et l'affichage lisible du résultat, utile pour des tests rapides sans construire d'interface.

305 client_research.print_response("I'm Nicole Robinson, I'm a Dental Hygienist, do you have any info about me?", stream=True, markdown=True)

→ Exécute l'équipe de recherche client sur une requête test, en streaming, et affiche la réponse en Markdown.

307 financial_data_agent.print_response("Summarize what Binance does and provide the pros and cons of investing into it based on the earnings call Q2/Q3 2024", stream=True, markdown=True)

→ Exécute l'agent d'analyse financière sur une requête test concernant Binance, en streaming, et affiche la réponse.

309 recommendation_agent.print_response("Nicole Robinson is interested in Binance stock. What is our recommendation for this stock?", stream=True, markdown=True)
→ Exécute l'agent de recommandation sur une requête test concernant Binance, en streaming, et affiche la réponse.

30-compliance_agent/main.py

Définit la classe AnalyseurConformiteUE qui exécute un agent agno d'analyse de conformité réglementaire UE, puis convertit son rapport Markdown en document Word formaté.

252 lignes de code annotées ci-dessous.

1 import sys

→ Importe le module standard sys pour accéder aux flux d'entrée/sortie du programme, ici utilisé pour reconfigurer l'encodage de la console.

3 # La console Windows utilise par défaut l'encodage cp1252, qui ne supporte pas

→ Commentaire expliquant que la console Windows par défaut ne supporte pas certains caractères.

4 # les emojis utilisés dans les messages de ce script (ex: ,).

→ Suite du commentaire, précisant que ce sont les émojis utilisés dans les messages du script qui posent problème : sans ce correctif, l'affichage d'un emoji dans un print() planterait le script avec une UnicodeEncodeError sur certains terminaux Windows.

5 for _stream in (sys.stdout, sys.stderr):

→ Itère sur les flux standard de sortie et d'erreur pour les reconfigurer tous les deux de la même façon.

6 if hasattr(_stream, "reconfigure"):

→ Vérifie que le flux courant possède bien la méthode reconfigure avant de l'appeler, cette méthode n'étant disponible que sur certaines versions de Python (3.7+) et certains types de flux, d'où cette vérification défensive.

7 _stream.reconfigure(encoding="utf-8", errors="replace")

→ Reconfigure le flux en encodage UTF-8 (qui supporte les emojis), en remplaçant (errors="replace") les caractères non représentables plutôt que de faire planter le programme.

9 import os

→ Importe le module standard pour interagir avec le système de fichiers et les variables d'environnement.

10 import re

→ Importe le module standard d'expressions régulières (regex), utilisé plus loin pour repérer et transformer les marqueurs Markdown (titres, gras, italique) lors de la conversion vers Word.

11 from datetime import datetime

→ Importe la classe datetime pour générer des horodatages dans les noms de fichiers et le rapport.

12 from pathlib import Path

→ Importe la classe Path du module pathlib, une API orientée objet pour manipuler des chemins de fichiers de façon portable (plutôt que de manipuler des chaînes de caractères brutes avec os.path).

13 from typing import Optional

→ Importe le type Optional utilisé pour annoter les paramètres facultatifs (indiquant qu'un argument peut valoir None), utile pour la lisibilité et la vérification statique du code.

14 from agno.agent import Agent

→ Importe la classe Agent du framework agno. Agno est un framework Python open-source pour construire des agents IA, qui associe un modèle de langage, des instructions et des outils externes pour produire un comportement spécialisé, ici l'analyse de conformité réglementaire.

15 from agno.models.groq import Groq

→ Importe la classe modèle Groq d'agno, un adaptateur permettant d'appeler l'API d'inférence de Groq, connue pour sa très faible latence grâce à du matériel dédié (LPU), et qui sert ici de fournisseur pour un modèle open-weight.

16 from agno.tools.exa import ExaTools

→ Importe l'outil ExaTools d'agno, une intégration de l'API Exa : un moteur de recherche sémantique conçu pour les agents IA, capable de comprendre le sens d'une requête plutôt que de faire un simple appariement de mots-clés, ce qui le rend adapté à des recherches réglementaires précises.

17 from agno.tools.reasoning import ReasoningTools

→ Importe l'outil ReasoningTools d'agno, qui donne à l'agent la capacité de raisonner explicitement étape par étape avant de répondre (une forme de chaîne de pensée structurée, « chain of thought ») : le modèle peut noter ses étapes de réflexion intermédiaires comme des actions d'outil, ce qui améliore la qualité et la traçabilité d'analyses complexes comme celle-ci.

18 from docx import Document

→ Importe la classe Document de la bibliothèque python-docx, qui permet de créer et manipuler des fichiers Word (.docx) directement en Python : ajout de titres, paragraphes, mise en forme, sans avoir besoin de Microsoft Word installé.

19 from docx.shared import Pt

→ Importe l'unité Pt (points) de python-docx, utilisée pour définir des tailles de police dans le document Word de façon cohérente avec les unités typographiques standards.

20 from docx.enum.text import WD_PARAGRAPH_ALIGNMENT

→ Importe l'énumération des alignements de paragraphe de python-docx, utilisée pour centrer certains textes (titre, sous-titre, date) dans le document généré.

21 from dotenv import load_dotenv

→ Importe la fonction load_dotenv de python-dotenv, qui charge les clés API depuis un fichier .env local plutôt que de les coder en dur dans le script.

23 class AnalyseurConformiteUE:

→ Définit la classe AnalyseurConformiteUE qui encapsule toute la logique d'analyse de conformité : c'est un choix de conception orienté objet (plutôt que des fonctions isolées) qui regroupe la configuration de l'agent, les méthodes d'analyse et la génération du rapport Word sous un même état partagé (self.agent, self.groq_api_key, etc.).

24 """Système d'Analyse de Conformité UE"""

→ Docstring de la classe décrivant son rôle de système d'analyse de conformité UE.

26 def __init__(self, config_path: Optional[str] = None):

→ Définit le constructeur de la classe, acceptant un chemin de configuration optionnel (typiquement un chemin vers un fichier .env alternatif).

27 """Initialise l'analyseur de conformité"""

→ Docstring de la méthode décrivant son rôle d'initialisation de l'analyseur.

28 load_dotenv(config_path)

→ Charge les variables d'environnement depuis le fichier .env par défaut, ou depuis config_path si un chemin spécifique est fourni — cette flexibilité permet de pointer vers une configuration différente sans modifier le code.

30 # Récupération des clés API

→ Commentaire annonçant la récupération des clés API nécessaires.

31 self.exa_api_key = os.getenv("EXA_API_KEY")

→ Récupère la clé API Exa depuis les variables d'environnement et la stocke sur l'instance (self), pour qu'elle reste accessible dans les autres méthodes de la classe.

32 # Le script original utilisait OpenAI (gpt-4-turbo) ; remplacé par Groq

→ Commentaire expliquant que le modèle OpenAI d'origine a été remplacé par Groq.

```

33         # pour réutiliser la clé GROQ_API_KEY déjà disponible.
    → Suite du commentaire, justifiant le choix de réutiliser la clé Groq déjà disponible, plutôt que de payer un accès supplémentaire à OpenAI, tout en conservant un modèle capable de raisonnement réglementaire complexe.

34         self.groq_api_key = os.getenv("GROQ_API_KEY")
    → Récupère la clé API Groq depuis les variables d'environnement et la stocke sur l'instance.

36         # Validation des clés
    → Commentaire annonçant la validation de la présence des clés API.

37         if not self.groq_api_key:
    → Vérifie si la clé API Groq est absente ou vide.

38         raise ValueError("Clé API Groq manquante")
    → Lève une exception ValueError interrompant l'initialisation si la clé Groq manque, car sans elle aucun appel au modèle n'est possible : mieux vaut échouer immédiatement et clairement à la construction plutôt que plus tard avec une erreur d'API opaque.

40         # Initialisation de l'agent
    → Commentaire annonçant l'appel à la méthode de configuration de l'agent.

41         self._configurer_agent()
    → Appelle la méthode interne qui construit et configure l'agent agno, séparant la logique de configuration (potentiellement longue) du constructeur lui-même pour plus de lisibilité.

43         def _configurer_agent(self):
    → Définit la méthode privée (préfixée d'un underscore par convention Python pour signaler un usage interne) qui construit et configure l'agent de conformité.

44         """Configure l'agent d'analyse de conformité"""
    → Docstring de la méthode décrivant son rôle de configuration de l'agent.

45         outils = [ReasoningTools(add_instructions=True)]
    → Initialise la liste des outils de l'agent avec ReasoningTools, l'option `add_instructions=True` demandant à agno d'injecter automatiquement dans le prompt système des consignes expliquant au modèle comment et quand utiliser cet outil de raisonnement explicite.

47         # Ajout d'ExaTools si la clé est disponible
    → Commentaire annonçant l'ajout conditionnel de l'outil ExaTools.

48         if self.exa_api_key:
    → Vérifie si une clé API Exa est disponible avant d'ajouter l'outil correspondant, rendant la recherche web optionnelle : l'agent reste fonctionnel avec le seul raisonnement s'il n'y a pas de clé Exa, mais gagne en capacité de recherche documentaire si elle est présente.

49         outils.append(ExaTools(
    → Ajoute une nouvelle instance d'ExaTools à la liste des outils, en ouvrant ses paramètres.

50             api_key=self.exa_api_key,
    → Passe la clé API Exa récupérée à l'instance d'ExaTools.

51             include_domains=["eur-lex.europa.eu", "ec.europa.eu"]
    → Restreint les recherches d'ExaTools aux domaines officiels eur-lex.europa.eu (le portail légal de l'Union européenne) et ec.europa.eu (le site de la Commission européenne) : cette restriction de domaine est une garde-fou de fiabilité, qui force l'agent à ne s'appuyer que sur des sources officielles plutôt que sur n'importe quel site trouvé par une recherche web ouverte, réduisant le risque de citer une information erronée ou non officielle.

52         ))
    → Ferme la configuration d'ExaTools et l'appel à outils.append.

```

```

54         self.agent = Agent(
    → Commence la création de l'agent agno et l'assigne à l'attribut self.agent, pour qu'il reste accessible depuis les autres
    méthodes de la classe (analyser_entreprise, etc.).

55         name="Analyseur de Conformité UE",
    → Définit le nom affiché de l'agent comme "Analyseur de Conformité UE".

56         model=Groq(
    → Commence la configuration du modèle Groq utilisé par l'agent.

57             id="openai/gpt-oss-120b",
    → Spécifie l'identifiant du modèle Groq gpt-oss-120b à utiliser : un modèle open-weight de 120 milliards de paramètres, choisi
    pour sa capacité à traiter un raisonnement réglementaire complexe et multi-étapes.

58             api_key=self.groq_api_key
    → Passe la clé API Groq récupérée à la configuration du modèle.

59         ),
    → Ferme la configuration du modèle Groq.

60         tools=outils,
    → Associe à l'agent la liste d'outils construite précédemment (ReasoningTools et éventuellement ExaTools).

61         debug_mode=True,
    → Active le mode debug de l'agent, qui affiche des informations supplémentaires (appels d'outils, étapes de raisonnement
    internes) lors de l'exécution dans la console, utile pour observer et déboguer le comportement de l'agent pendant le
    développement.

62         description="""
    → Commence la chaîne de description multi-lignes de l'agent.

63             Agent spécialisé dans l'analyse de la conformité réglementaire européenne.
    → Ligne de description précisant la spécialisation de l'agent en conformité réglementaire européenne.

64             Fournit des évaluations de conformité structurées pour les entreprises
    → Ligne de description précisant que l'agent fournit des évaluations structurées pour les entreprises.

65             opérant sur le marché européen.
    → Fin de la ligne de description, précisant que les entreprises visées opèrent sur le marché européen.

66         """,
    → Ferme la chaîne de description de l'agent.

67         instructions=self._obtenir_instructions_agent(),
    → Associe à l'agent les instructions renvoyées par la méthode _obtenir_instructions_agent : ces instructions sont extraites
    dans une méthode séparée plutôt qu'écrites en ligne, pour garder le constructeur de l'agent lisible malgré un prompt système
    volumineux.

68         markdown=True,
    → Active le formatage Markdown pour les réponses générées par l'agent, cohérent avec l'étape suivante de conversion du
    rapport en document Word qui repose justement sur la syntaxe Markdown (titres #, gras **, etc.) pour structurer le texte.

69     )
    → Ferme l'appel au constructeur Agent, terminant l'assignation à self.agent.

71     def _obtenir_instructions_agent(self) -> str:
    → Définit la méthode privée qui renvoie la chaîne d'instructions détaillées de l'agent, annotée comme renvoyant un `str`.

72         """Instructions détaillées pour l'agent de conformité"""

```

→ Docstring de la méthode décrivant son rôle de fournisseur des instructions détaillées.

73 return ""

→ Commence le renvoi de la chaîne d'instructions multi-lignes de l'agent.

74 Vous êtes un expert en conformité UE. Votre mission est de fournir des analyses

→ Ligne d'instruction présentant le rôle d'expert en conformité UE et sa mission d'analyse.

75 réglementaires complètes pour les entreprises opérant sur les marchés européens.

→ Fin de la ligne d'instruction, précisant que les analyses concernent les entreprises sur les marchés européens.

77 CADRE D'ANALYSE :

→ Ligne d'instruction introduisant le cadre d'analyse structuré à suivre.

78 1. PROFIL DE L'ENTREPRISE

→ Ligne d'instruction introduisant la première étape du cadre, le profil de l'entreprise.

79 - Identifier les activités principales

→ Ligne d'instruction demandant d'identifier les activités principales de l'entreprise.

80 - Déterminer la classification sectorielle

→ Ligne d'instruction demandant de déterminer la classification sectorielle de l'entreprise.

81 - Évaluer l'étendue géographique des opérations UE

→ Ligne d'instruction demandant d'évaluer l'étendue géographique des opérations dans l'UE.

83 2. CARTOGRAPHIE RÉGLEMENTAIRE

→ Ligne d'instruction introduisant la deuxième étape du cadre, la cartographie réglementaire.

84 - Rechercher les directives et règlements applicables

→ Ligne d'instruction demandant de rechercher les directives et règlements applicables — c'est cette étape qui déclenchera typiquement l'usage d'ExaTools par le modèle, restreint aux domaines officiels configurés plus haut.

85 - Identifier les exigences sectorielles spécifiques

→ Ligne d'instruction demandant d'identifier les exigences sectorielles spécifiques.

86 - Suivre les calendriers de mise en œuvre

→ Ligne d'instruction demandant de suivre les calendriers de mise en œuvre des règlements.

88 3. ÉVALUATION DES RISQUES DE CONFORMITÉ

→ Ligne d'instruction introduisant la troisième étape du cadre, l'évaluation des risques de conformité.

89 - Catégoriser les règlements par criticité (Élevée/Moyenne/Faible)

→ Ligne d'instruction demandant de catégoriser les règlements par niveau de criticité.

90 - Identifier les lacunes potentielles de conformité

→ Ligne d'instruction demandant d'identifier les lacunes potentielles de conformité.

91 - Évaluer les sanctions et pénalités

→ Ligne d'instruction demandant d'évaluer les sanctions et pénalités associées.

93 4. DOMAINES RÉGLEMENTAIRES PRINCIPAUX :

→ Ligne d'instruction introduisant la quatrième étape du cadre, les domaines réglementaires principaux.

94 - Protection des données : RGPD, ePrivacy

→ Ligne d'instruction citant le domaine de la protection des données avec le RGPD et ePrivacy.

95 - IA & Numérique : Loi sur l'IA, Loi sur les services numériques

→ Ligne d'instruction citant le domaine de l'IA et du numérique avec les lois correspondantes.

96 - Environnement : Pacte vert, CSRD, Taxonomie

→ Ligne d'instruction citant le domaine environnemental avec le Pacte vert, la CSRD et la Taxonomie.

97 - Sectoriel : Automobile, Services financiers

→ Ligne d'instruction citant les domaines sectoriels automobile et services financiers.

98 - Sécurité des produits : Marquage CE, Cybersécurité

→ Ligne d'instruction citant le domaine de la sécurité des produits avec le marquage CE et la cybersécurité.

100 5. STRUCTURE DU RAPPORT :

→ Ligne d'instruction introduisant la cinquième étape du cadre, la structure du rapport attendu.

101 - Résumé exécutif avec priorités clés

→ Ligne d'instruction demandant un résumé exécutif présentant les priorités clés.

102 - Calendrier réglementaire

→ Ligne d'instruction demandant l'inclusion d'un calendrier réglementaire dans le rapport.

103 - Matrice des risques

→ Ligne d'instruction demandant l'inclusion d'une matrice des risques dans le rapport.

104 - Feuille de route de conformité

→ Ligne d'instruction demandant l'inclusion d'une feuille de route de conformité dans le rapport.

105 - Exigences en ressources

→ Ligne d'instruction demandant l'inclusion des exigences en ressources dans le rapport.

107 STRATÉGIE DE RECHERCHE :

→ Ligne d'instruction introduisant la stratégie de recherche à suivre pour collecter les informations.

108 - Utiliser uniquement des sources officielles UE

→ Ligne d'instruction imposant l'utilisation exclusive de sources officielles de l'UE, cohérente avec la restriction `include\_domains` appliquée à ExaTools : la consigne textuelle et la contrainte technique se renforcent mutuellement pour garantir la fiabilité des sources.

109 - Inclure des références spécifiques aux articles

→ Ligne d'instruction demandant d'inclure des références précises aux articles réglementaires.

110 - Fournir les dates de publication et d'entrée en vigueur

→ Ligne d'instruction demandant de fournir les dates de publication et d'entrée en vigueur des textes.

111 - Vérifier la cohérence entre les sources

→ Ligne d'instruction demandant de vérifier la cohérence des informations entre les différentes sources.

112 """

→ Ferme la chaîne d'instructions renvoyée par la méthode.

114 def analyser_entreprise(self, nom_entreprise: str, secteur: str,

→ Définit la méthode analyser_entreprise avec les paramètres nom de l'entreprise et secteur d'activité, point d'entrée public principal de la classe pour lancer une analyse.

115 contexte_supplementaire: str = "",

→ Poursuit la signature de la méthode avec un paramètre de contexte supplémentaire optionnel, vide par défaut.

116 chemin_markdown: str = "conformite_automobile.md") -> str:

→ Termine la signature avec le chemin de sortie Markdown par défaut, et précise le type de retour str.

117 `"""Analyse les exigences de conformité pour une entreprise spécifique"""`
→ Docstring de la méthode décrivant son rôle d'analyse des exigences de conformité d'une entreprise.

119 `requete = f"""`
→ Commence la construction d'une chaîne formatée (f-string) multi-lignes contenant la requête envoyée à l'agent, les f-strings permettant d'insérer directement la valeur des variables Python entre accolades dans le texte.

120 `Effectuez une analyse complète de conformité réglementaire UE pour :`
→ Ligne de la requête demandant une analyse complète de conformité réglementaire UE.

122 `Entreprise : {nom_entreprise}`
→ Ligne de la requête insérant dynamiquement le nom de l'entreprise à analyser.

123 `Secteur : {secteur}`
→ Ligne de la requête insérant dynamiquement le secteur d'activité de l'entreprise.

124 `Contexte supplémentaire : {contexte_supplementaire}`
→ Ligne de la requête insérant dynamiquement le contexte supplémentaire fourni.

126 `Identifiez tous les règlements UE applicables, leurs exigences de conformité,`
→ Ligne de la requête demandant d'identifier tous les règlements UE applicables et leurs exigences.

127 `les délais de mise en œuvre et les impacts potentiels sur l'activité.`
→ Ligne de la requête demandant de préciser les délais de mise en œuvre et impacts sur l'activité.

128 `Fournissez des conseils de conformité spécifiques et actionnables.`
→ Ligne de la requête demandant de fournir des conseils de conformité spécifiques et actionnables.

129 `"""`
→ Ferme la chaîne formatée contenant la requête envoyée à l'agent.

131 `print(f" Début de l'analyse de conformité pour {nom_entreprise}...")`
→ Affiche dans la console un message de démarrage de l'analyse incluant le nom de l'entreprise, rendu possible sans planter grâce à la reconfiguration UTF-8 des flux effectuée en tête de fichier.

132 `# print_response() n'imprime que dans la console et ne retourne rien`
→ Commentaire expliquant que `print_response` ne fait qu'afficher sans rien retourner d'exploitable.

133 `# d'exploitable ; run() donne accès au contenu pour le sauvegarder en`
→ Suite du commentaire, précisant que `run()` permet d'accéder au contenu pour le sauvegarder.

134 `# markdown, nécessaire pour l'étape de génération du rapport Word.`
→ Fin du commentaire, précisant que cet accès au contenu est nécessaire pour générer le rapport Word : c'est pourquoi ce script utilise ``run()`` plutôt que ``print_response()``, contrairement à `main.py` du projet 29 qui n'a pas besoin de récupérer le texte pour un traitement ultérieur.

135 `reponse = self.agent.run(requete)`
→ Exécute l'agent avec la requête construite et stocke la réponse obtenue ; ``run()`` est la méthode agno qui déclenche l'ensemble du cycle agentique (appel du modèle, éventuels appels d'outils comme ExaTools ou ReasoningTools, puis réponse finale) et renvoie un objet exploitable en Python plutôt que de se contenter d'un affichage console.

136 `print(reponse.content)`
→ Affiche dans la console le contenu textuel de la réponse de l'agent.

138 `with open(chemin_markdown, "w", encoding="utf-8") as fichier:`
→ Ouvre en écriture le fichier Markdown de sortie, explicitement encodé en UTF-8 pour préserver tous les caractères accentués et spéciaux du texte généré.

139 `fichier.write(reponse.content)`

→ Écrit le contenu de la réponse de l'agent dans le fichier Markdown ouvert.

```
141         return reponse.content
```

→ Renvoie le contenu textuel de la réponse de l'agent à l'appelant, pour un usage ultérieur en mémoire sans avoir à relire le fichier.

```
143     def generer_rapport_word(self, chemin_markdown: str,
```

→ Définit la méthode `generer_rapport_word` avec le chemin du fichier Markdown source en paramètre : cette méthode réalise la conversion Markdown vers Word « à la main », en interprétant elle-même la syntaxe (titres, listes, gras, italique) plutôt que de s'appuyer sur une bibliothèque de conversion externe.

```
144         chemin_sortie: Optional[str] = None) -> str:
```

→ Termine la signature avec un chemin de sortie Word optionnel et précise le type de retour `str`.

```
145         """Convertit un rapport markdown en document Word formaté"""
```

→ Docstring de la méthode décrivant son rôle de conversion du Markdown en document Word.

```
147         # Lecture du contenu markdown
```

→ Commentaire annonçant la lecture du contenu du fichier Markdown source.

```
148         try:
```

→ Démarre un bloc `try` pour gérer l'absence éventuelle du fichier Markdown, plutôt que de laisser l'exception standard remonter sans message clair.

```
149         with open(chemin_markdown, "r", encoding="utf-8") as fichier:
```

→ Ouvre en lecture le fichier Markdown source, encodé en UTF-8.

```
150         contenu = fichier.read()
```

→ Lit l'intégralité du contenu du fichier Markdown et le stocke dans `contenu`.

```
151         except FileNotFoundError:
```

→ Intercepte l'exception levée si le fichier Markdown est introuvable.

```
152         raise FileNotFoundError(f"Fichier markdown introuvable : {chemin_markdown}")
```

→ Relève une exception `FileNotFoundError` avec un message plus explicite précisant le chemin manquant, pour faciliter le diagnostic par rapport au message d'erreur générique par défaut.

```
154         # Initialisation du document Word
```

→ Commentaire annonçant la création du document Word.

```
155         doc = Document()
```

→ Crée un nouveau document Word vide en mémoire via `python-docx`, basé sur un modèle de document par défaut intégré à la bibliothèque, sur lequel les méthodes suivantes vont progressivement ajouter du contenu.

```
157         # Ajout de la page de titre
```

→ Commentaire annonçant l'ajout de la page de titre au document.

```
158         self._ajouter_page_titre(doc)
```

→ Appelle la méthode qui ajoute la page de titre au document Word.

```
160         # Traitement du contenu
```

→ Commentaire annonçant la conversion du contenu Markdown en contenu Word.

```
161         self._traiter_contenu_markdown(doc, contenu)
```

→ Appelle la méthode qui convertit le contenu Markdown lu en paragraphes et titres Word.

```
163         # Ajout du pied de page
```

→ Commentaire annonçant l'ajout du pied de document contenant les métadonnées.

164 `self._ajouter_pied_document(doc)`
→ Appelle la méthode qui ajoute le pied de document avec les métadonnées au document Word.

166 `# Sauvegarde du document`
→ Commentaire annonçant la sauvegarde finale du document Word sur le disque.

167 `if not chemin_sortie:`
→ Vérifie si aucun chemin de sortie n'a été fourni par l'appelant.

168 `horodatage = datetime.now().strftime("%Y%m%d_%H%M%S")`
→ Génère un horodatage au format année-mois-jour-heure-minute-seconde pour nommer le fichier, évitant ainsi d'écraser un rapport précédent si aucun nom explicite n'est donné.

169 `chemin_sortie = f"Rapport_Conformite_UE_{horodatage}.docx"`
→ Construit un nom de fichier par défaut incluant l'horodatage généré.

171 `doc.save(chemin_sortie)`
→ Sauvegarde le document Word sur le disque au chemin déterminé, méthode fournie par python-docx qui sérialise l'objet Document en un vrai fichier .docx lisible par Microsoft Word ou équivalent.

172 `print(f" Rapport Word sauvegardé : {chemin_sortie}")`
→ Affiche dans la console un message confirmant la sauvegarde du rapport Word.

174 `return chemin_sortie`
→ Renvoie le chemin du fichier Word généré à l'appelant.

176 `def _ajouter_page_titre(self, doc: Document):`
→ Définit la méthode privée qui ajoute une page de titre au document Word passé en paramètre.

177 `"""Ajoute une page de titre professionnelle"""`
→ Docstring de la méthode décrivant son rôle d'ajout d'une page de titre professionnelle.

178 `# Titre`
→ Commentaire annonçant l'ajout du titre principal du document.

179 `titre = doc.add_heading("Analyse de Conformité Réglementaire UE", 0)`
→ Ajoute un titre de niveau 0 (le plus haut niveau, réservé au titre principal du document) au document via l'API python-docx, et stocke la référence à ce paragraphe dans `titre` pour pouvoir le modifier ensuite.

180 `titre.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER`
→ Centre l'alignement du paragraphe de titre principal, en utilisant l'énumération importée en tête de fichier plutôt qu'une valeur numérique arbitraire.

182 `# Sous-titre`
→ Commentaire annonçant l'ajout du sous-titre du document.

183 `sous_titre = doc.add_paragraph("Rapport d'Intelligence Réglementaire")`
→ Ajoute un paragraphe de sous-titre au document et stocke la référence dans `sous_titre`.

184 `sous_titre.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER`
→ Centre l'alignement du paragraphe de sous-titre.

185 `sous_titre_format = sous_titre.runs[0].font`
→ Récupère l'objet de police du premier « run » du paragraphe de sous-titre. Dans le modèle de python-docx, un paragraphe est composé d'un ou plusieurs « runs » (segments de texte partageant une même mise en forme) : il faut accéder au run pour changer taille ou style, l'alignement seul ne suffisant pas.

186 `sous_titre_format.size = Pt(14)`
→ Fixe la taille de la police du sous-titre à 14 points, en utilisant l'unité Pt importée en tête de fichier.

187 `sous_titre_format.italic = True`
→ Met en italique le texte du sous-titre.

189 `# Date`
→ Commentaire annonçant l'ajout de la date de génération du document.

190 `date_para = doc.add_paragraph(f"Généré le : {datetime.now().strftime('%d %B %Y')})"`
→ Ajoute un paragraphe affichant la date courante, formatée en jour, nom du mois et année via `strftime`.

191 `date_para.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER`
→ Centre l'alignement du paragraphe de date.

193 `# Saut de page`
→ Commentaire annonçant l'insertion d'un saut de page après la page de titre.

194 `doc.add_page_break()`
→ Insère un saut de page dans le document Word, pour que le contenu du rapport commence sur une nouvelle page distincte de la page de titre.

196 `def _traiter_contenu_markdown(self, doc: Document, contenu: str):`
→ Définit la méthode privée qui convertit le texte Markdown en éléments du document Word, cœur du convertisseur maison de ce script.

197 `"""Traite le contenu markdown et l'ajoute au document Word"""`
→ Docstring de la méthode décrivant son rôle de traitement du contenu Markdown.

198 `lignes = contenu.splitlines()`
→ Découpe le contenu Markdown en une liste de lignes individuelles, pour les traiter une par une.

200 `for ligne in lignes:`
→ Itère sur chaque ligne du contenu Markdown découpé.

201 `ligne = ligne.strip()`
→ Supprime les espaces en début et fin de la ligne courante, pour fiabiliser la détection des marqueurs Markdown qui suit (une ligne commençant par un espace avant "# " ne serait pas reconnue sinon).

202 `if not ligne:`
→ Vérifie si la ligne, une fois nettoyée, est vide.

203 `continue`
→ Passe à l'itération suivante de la boucle sans traiter cette ligne vide.

205 `# En-têtes`
→ Commentaire annonçant le traitement des lignes de titres Markdown.

206 `if ligne.startswith("# "):`
→ Vérifie si la ligne commence par un titre Markdown de niveau 1 (syntaxe standard : un dièse suivi d'un espace).

207 `doc.add_heading(ligne[2:], level=1)`
→ Ajoute un titre Word de niveau 1 avec le texte de la ligne débarrassé du marqueur "# " (les deux premiers caractères).

208 `elif ligne.startswith("## "):`
→ Vérifie si la ligne commence par un titre Markdown de niveau 2.

209 `doc.add_heading(ligne[3:], level=2)`
→ Ajoute un titre Word de niveau 2 avec le texte de la ligne débarrassé du marqueur "## ".

210 elif ligne.startswith("### "):

→ Vérifie si la ligne commence par un titre Markdown de niveau 3.

211 doc.add_heading(ligne[4:], level=3)

→ Ajoute un titre Word de niveau 3 avec le texte de la ligne débarrassé du marqueur "### ".

212 elif ligne.startswith("#### "):

→ Vérifie si la ligne commence par un titre Markdown de niveau 4.

213 doc.add_heading(ligne[5:], level=4)

→ Ajoute un titre Word de niveau 4 avec le texte de la ligne débarrassé du marqueur "#### ".

215 # Listes

→ Commentaire annonçant le traitement des lignes de listes Markdown.

216 elif ligne.startswith("* ") or ligne.startswith("- "):

→ Vérifie si la ligne est un élément de liste à puces Markdown, dont la syntaxe standard accepte indifféremment "*" ou "-" comme marqueur de puce.

217 self._ajouter_puce(doc, ligne[2:].strip())

→ Appelle la méthode qui ajoute une puce Word avec le texte de la ligne débarrassé de son marqueur.

218 elif re.match(r'^\d+\. ', ligne):

→ Vérifie via une expression régulière si la ligne est un élément de liste numérotée Markdown (un ou plusieurs chiffres suivis d'un point et d'un espace, par exemple "1. ") : une simple vérification `startswith` ne suffirait pas ici car le numéro varie, d'où l'usage d'une regex.

219 self._ajouter_numero(doc, re.sub(r'^\d+\. ', '', ligne))

→ Appelle la méthode qui ajoute un élément numéroté Word avec le texte débarrassé de son numéro, la suppression étant elle aussi faite par regex (`re.sub`) puisque la longueur du numéro à retirer varie.

221 # Paragraphes normaux

→ Commentaire annonçant le traitement des lignes de texte normal, sans marqueur Markdown particulier.

222 else:

→ Cas par défaut appliqué quand la ligne n'est ni un titre ni un élément de liste.

223 self._ajouter_paragraphe_formate(doc, ligne)

→ Appelle la méthode qui ajoute la ligne comme paragraphe normal formaté dans le document Word.

225 def _ajouter_puce(self, doc: Document, texte: str):

→ Définit la méthode privée qui ajoute un élément de liste à puces au document Word.

226 """Ajoute une puce formatée"""

→ Docstring de la méthode décrivant son rôle d'ajout d'une puce formatée.

227 p = doc.add_paragraph(style="List Bullet")

→ Crée un nouveau paragraphe utilisant le style Word intégré "List Bullet" (qui gère automatiquement l'indentation et le symbole de puce) et le stocke dans p.

228 self._ajouter_texte_formate(p, texte)

→ Appelle la méthode qui insère le texte avec gras et italique dans le paragraphe de puce créé.

230 def _ajouter_numero(self, doc: Document, texte: str):

→ Définit la méthode privée qui ajoute un élément de liste numérotée au document Word.

231 """Ajoute un élément numéroté"""

→ Docstring de la méthode décrivant son rôle d'ajout d'un élément numéroté.

232 p = doc.add_paragraph(style="List Number")
→ Crée un nouveau paragraphe utilisant le style Word intégré "List Number" (qui gère automatiquement la numérotation séquentielle) et le stocke dans p.

233 self._ajouter_texte_formate(p, texte)
→ Appelle la méthode qui insère le texte avec gras et italique dans le paragraphe numéroté créé.

235 def _ajouter_paragraphe_formate(self, doc: Document, texte: str):
→ Définit la méthode privée qui ajoute un paragraphe de texte normal formaté au document Word.

236 """Ajoute un paragraphe avec formatage"""
→ Docstring de la méthode décrivant son rôle d'ajout d'un paragraphe avec formatage.

237 p = doc.add_paragraph()
→ Crée un nouveau paragraphe vide (sans style de liste particulier) dans le document Word et le stocke dans p.

238 self._ajouter_texte_formate(p, texte)
→ Appelle la méthode qui insère le texte avec gras et italique dans le paragraphe créé.

240 def _ajouter_texte_formate(self, paragraphe, texte: str):
→ Définit la méthode privée partagée qui insère du texte avec mise en forme gras et italique dans un paragraphe, utilisée par les trois méthodes précédentes (puce, numéro, paragraphe normal) pour ne pas dupliquer cette logique de parsing.

241 """Ajoute du texte avec formatage gras/italique"""
→ Docstring de la méthode décrivant son rôle d'ajout de texte avec formatage gras et italique.

242 # Traitement du texte en gras (**texte**)
→ Commentaire annonçant le traitement du texte en gras délimité par des doubles astérisques, syntaxe Markdown standard pour le gras.

243 parties = re.split(r"(*\.?\.?*)", texte)
→ Découpe le texte en segments avec une regex non gourmande (`\*\.?\.?\*`), en conservant les portions entourées de doubles astérisques comme éléments séparés de la liste résultante (grâce aux parenthèses capturantes dans le motif de split), ce qui permet ensuite de distinguer texte en gras et texte normal.

244 for partie in parties:
→ Itère sur chaque segment de texte obtenu après découpage.

245 if partie.startswith("***") and partie.endswith("***"):
→ Vérifie si le segment courant est entouré de doubles astérisques, marquant du texte en gras.

246 run = paragraphe.add_run(partie[2:-2])
→ Ajoute au paragraphe un « run » (segment de texte à mise en forme homogène, concept python-docx déjà rencontré ligne 185) contenant le texte sans les astérisques de gras.

247 run.bold = True
→ Met en gras le run de texte ajouté.

248 else:
→ Cas où le segment n'est pas entouré de doubles astérisques.

249 # Traitement du texte en italique (*texte*)
→ Commentaire annonçant le traitement du texte en italique délimité par un simple astérisque.

250 parties_italique = re.split(r"(*\.?*)", partie)
→ Découpe le segment courant en isolant, de la même façon, les portions entourées d'un simple astérisque.

251 for partie_italique in parties_italique:
→ Itère sur chaque sous-segment obtenu après ce second découpage.

252 if (partie_italique.startswith("*") and partie_italique.endswith("*"))
→ Vérifie si le sous-segment commence et se termine par un astérisque simple, marquant de l'italique.

253 and not partie_italique.startswith("***")):
→ Vérifie également que ce n'est pas un marqueur de gras déjà traité (un texte en gras "***x**" commence aussi techniquement par un simple "*"), condition nécessaire pour éviter une confusion entre les deux syntaxes lors de ce second passage de découpage.

254 run = paragraphe.add_run(partie_italique[1:-1])
→ Ajoute au paragraphe un run contenant le texte sans les astérisques d'italique.

255 run.italic = True
→ Met en italique le run de texte ajouté.

256 else:
→ Cas où le sous-segment n'est ni gras ni italique, du texte brut.

257 paragraphe.add_run(partie_italique)
→ Ajoute au paragraphe un run contenant le texte brut sans mise en forme particulière.

259 def _ajouter_pied_document(self, doc: Document):
→ Définit la méthode privée qui ajoute le pied de document avec les métadonnées au document Word.

260 """Ajoute un pied de document avec métadonnées"""
→ Docstring de la méthode décrivant son rôle d'ajout du pied de document avec métadonnées.

261 doc.add_page_break()
→ Insère un saut de page avant le pied de document, pour le séparer visuellement du corps du rapport.

262 doc.add_heading("Informations du Document", level=2)
→ Ajoute un titre de niveau 2 intitulé "Informations du Document".

264 infos_pied = [
→ Commence la construction de la liste des lignes d'information à afficher dans le pied de document.

265 f"Généré le : {datetime.now().strftime('%d-%m-%Y %H:%M:%S')}",
→ Ajoute à la liste une chaîne indiquant la date et l'heure précises de génération du rapport, avec les secondes contrairement à la date affichée en page de titre (ligne 190), pour une traçabilité plus fine.

266 "Source : Sources officielles UE (eur-lex.europa.eu, ec.europa.eu)",
→ Ajoute à la liste une chaîne mentionnant les sources officielles UE utilisées, rappelant explicitement au lecteur la restriction de domaine appliquée à ExaTools.

267 "Avertissement : Ce rapport est à des fins d'information uniquement.",
→ Ajoute à la liste une chaîne d'avertissement précisant la portée purement informative du rapport.

268 "Recommandation : Consultez un conseil juridique pour des questions de conformité
spécifiques."
→ Ajoute à la liste une chaîne recommandant de consulter un conseil juridique pour des questions spécifiques.

269]
→ Ferme la liste des lignes d'information du pied de document.

271 for info in infos_pied:
→ Itère sur chaque ligne d'information de la liste construite précédemment.

272 p = doc.add_paragraph(info)
→ Ajoute un paragraphe contenant la ligne d'information courante au document.

273 p.style = "Caption"

→ Applique le style Word intégré "Caption" au paragraphe d'information ajouté, un style typographique généralement plus petit et discret, adapté à des mentions légales ou des métadonnées.

275 def main():

→ Définit la fonction main, point d'entrée principal du script, séparée de la classe AnalyseurConformiteUE pour ne pas mélanger la logique métier réutilisable et le scénario d'exécution concret.

276 """Fonction d'exécution principale"""

→ Docstring de la fonction décrivant son rôle de point d'exécution principal.

277 try:

→ Démarre un bloc try englobant l'ensemble du déroulement principal pour capturer les erreurs et éviter un plantage brut sans message clair.

278 # Initialisation de l'analyseur

→ Commentaire annonçant la création de l'instance de l'analyseur de conformité.

279 analyseur = AnalyseurConformiteUE()

→ Instancie la classe AnalyseurConformiteUE, déclenchant en chaîne le chargement des clés API et la configuration complète de l'agent agno (via __init__ puis _configurer_agent).

281 # Exemple d'utilisation

→ Commentaire annonçant les valeurs d'exemple utilisées pour la démonstration.

282 nom_entreprise = "Renault"

→ Définit le nom d'entreprise d'exemple utilisé pour l'analyse, ici Renault.

283 secteur = "Industrie Automobile"

→ Définit le secteur d'exemple utilisé pour l'analyse, ici l'industrie automobile.

284 contexte_supplementaire = "Focus sur les systèmes IA, réglementations d'émissions et reporting de durabilité"

→ Définit le contexte supplémentaire d'exemple précisant les axes d'analyse souhaités.

286 # Lancement de l'analyse

→ Commentaire annonçant le lancement effectif de l'analyse de conformité.

287 print(" Début de l'analyse de conformité UE...")

→ Affiche dans la console un message annonçant le démarrage de l'analyse de conformité UE, rendu possible sans erreur grâce à la reconfiguration UTF-8 des flux effectuée en tête de script.

288 reponse = analyseur.analyser_entreprise(

→ Commence l'appel à la méthode analyser_entreprise sur l'instance analyseur, en ouvrant ses arguments.

289 nom_entreprise=nom_entreprise,

→ Passe le nom de l'entreprise d'exemple comme argument nommé à la méthode.

290 secteur=secteur,

→ Passe le secteur d'exemple comme argument nommé à la méthode.

291 contexte_supplementaire=contexte_supplementaire

→ Passe le contexte supplémentaire d'exemple comme argument nommé à la méthode.

292)

→ Ferme l'appel à analyser_entreprise et stocke son résultat dans reponse.

294 # Conversion en Word (en supposant qu'un fichier markdown a été généré)
→ Commentaire annonçant l'étape de conversion du rapport Markdown en document Word.

295 chemin_markdown = "conformite_automobile.md"
→ Définit le chemin du fichier Markdown généré à convertir en Word, correspondant au nom par défaut utilisé par analyser_entreprise.

296 if Path(chemin_markdown).exists():
→ Vérifie si le fichier Markdown attendu existe bien sur le disque, en utilisant l'API orientée objet de pathlib.Path importée en tête de fichier plutôt que os.path.exists.

297 chemin_rapport_word = analyseur.generer_rapport_word(chemin_markdown)
→ Appelle la méthode qui génère le rapport Word à partir du fichier Markdown et stocke le chemin obtenu.

298 print(f" Analyse terminée ! Rapport sauvegardé : {chemin_rapport_word}")
→ Affiche dans la console un message confirmant la fin de l'analyse et l'emplacement du rapport.

299 else:
→ Cas où le fichier Markdown attendu n'a pas été trouvé.

300 print(" Fichier markdown introuvable. Veuillez vérifier le chemin.")
→ Affiche dans la console un message d'avertissement signalant l'absence du fichier Markdown.

302 except Exception as e:
→ Intercepte toute exception survenue pendant le bloc try, la stockant dans la variable e, garde-fou générique qui englobe aussi bien une clé API manquante qu'une erreur réseau ou de génération du document.

303 print(f" Erreur lors de l'analyse : {e}")
→ Affiche dans la console un message d'erreur incluant le détail de l'exception survenue.

304 raise
→ Relance l'exception interceptée après l'avoir affichée, pour ne pas masquer l'échec : le message est logué proprement, mais le programme se termine bien en erreur plutôt que de continuer silencieusement dans un état incohérent.

306 if __name__ == "__main__":
→ Vérifie que le script est exécuté directement, et non importé comme module, avant d'appeler main.

307 main()
→ Appelle la fonction main, déclenchant l'exécution complète du script.

Projet 31 — 31-Procurement-agent

Recherche de fournisseurs pour une liste de produits

Objectif

Pour une liste de produits à acheter fournie par l'utilisateur, recherche automatiquement des fournisseurs potentiels sur le web et compile leurs informations (prix, conditions, coordonnées) afin de faciliter une décision d'achat.

Code source, ligne par ligne

31-Procurement-agent/procurement_agent.py

Ce fichier définit un agent d'approvisionnement (workflow agno) qui utilise l'API de recherche Exa pour trouver des fournisseurs, délègue l'analyse à un agent LLM outillé de PythonTools pour générer un CSV, puis produit un résumé exécutif des recommandations.

180 lignes de code annotées ci-dessous.

1 import os

→ os est le module standard de Python pour interagir avec le système d'exploitation (variables d'environnement, chemins de fichiers). Il est importé ici car le script doit lire des clés API sensibles (comme EXA_API_KEY) depuis l'environnement plutôt que de les écrire en dur dans le code, ce qui est la pratique standard pour ne jamais exposer de secrets dans le code source versionné.

2 from exa_py import Exa

→ Importe la classe Exa du SDK exa_py. Exa est un moteur de recherche web pensé pour les applications d'IA : contrairement à un moteur classique, il permet une recherche sémantique (par sens plutôt que par mots-clés exacts) et peut renvoyer directement le contenu texte des pages trouvées, évitant d'avoir à faire du scraping soi-même. Il donne ici à l'agent un accès à des informations fraîches sur le web (fournisseurs, prix) que le modèle de langage ne connaît pas nativement.

3 from textwrap import dedent

→ dedent est une fonction de la bibliothèque standard qui retire l'indentation commune au début de chaque ligne d'un bloc de texte. Elle est utilisée avec les chaînes multi-lignes (comme les instructions données à un agent), qui héritent sinon de l'indentation du code Python environnant.

4 from typing import Iterator

→ Importe le type générique Iterator, utilisé uniquement pour l'annotation de type (indiquer qu'une variable est un flux d'éléments que l'on peut parcourir un par un). Cela sert la lisibilité et les outils d'analyse statique, sans changer le comportement du programme.

5 from dotenv import load_dotenv

→ Importe la fonction load_dotenv de la librairie python-dotenv, qui charge des variables (comme les clés API) depuis un fichier .env local dans les variables d'environnement du processus. C'est la méthode standard pour séparer la configuration/les secrets du code source versionné dans Git.

6 from agno.workflow import Workflow

→ Importe la classe de base Workflow du framework agno, une bibliothèque Python dédiée à la construction d'agents et de systèmes multi-agents IA. Un Workflow permet d'orchestrer plusieurs étapes (ex: rechercher puis analyser) en une séquence contrôlée par du code Python classique, plutôt que de tout confier au raisonnement libre d'un seul agent.

7 from agno.agent import Agent, RunResponse

→ Importe la classe Agent, l'abstraction centrale d'agno représentant un agent IA (un modèle de langage doté d'instructions, d'outils et éventuellement d'une mémoire ou d'une base de connaissances), ainsi que RunResponse, le type représentant le

résultat retourné par l'exécution d'un agent.

```
8 from agno.models.openai import OpenAIChat
```

→ Importe la classe qui encapsule un modèle de chat OpenAI (GPT) pour l'utiliser comme "cerveau" d'un Agent agno. Elle sert de couche d'abstraction entre le framework agno et l'API OpenAI.

```
9 from agno.tools.python import PythonTools
```

→ Importe PythonTools, un "outil" (tool) qu'on peut donner à un agent. Dans une architecture d'agent IA, un outil est une fonction que le modèle de langage peut décider d'appeler lui-même pendant son raisonnement pour agir sur le monde réel (ici, écrire et exécuter du code Python) au lieu de se contenter de générer du texte. C'est ce mécanisme qui permet à l'agent de réellement créer le fichier CSV demandé.

```
10 from agno.utils.pprint import pprint_run_response
```

→ Importe une fonction utilitaire d'agno qui affiche de façon lisible (avec mise en forme) le flux de réponse renvoyé par l'exécution d'un workflow ou d'un agent.

```
12 load_dotenv()
```

→ Exécute le chargement effectif du fichier .env : à partir de cet appel, les variables qu'il contient (clés API, etc.) sont accessibles via os.environ ou os.getenv() dans le reste du script.

```
14 ### Exa Setup ###
```

→ Commentaire marquant le début de la section de configuration du client Exa.

```
15 exa_api_key = os.environ.get('EXA_API_KEY')
```

→ Récupère la clé API Exa depuis les variables d'environnement (chargées juste avant depuis .env). get() renvoie None si la variable est absente, plutôt que de lever une erreur immédiatement.

```
17 if not exa_api_key:
```

→ Vérifie si la clé API Exa est absente ou vide.

```
18     raise ValueError("EXA_API_KEY environment variable is required")
```

→ Arrête le programme en levant une erreur explicite si la clé est manquante, ce qui donne un message clair plutôt qu'un échec cryptique plus tard lors du premier appel réseau à Exa.

```
20 exa = Exa(api_key=exa_api_key)
```

→ Crée l'instance du client Exa authentifiée avec la clé API récupérée : cet objet exa sera utilisé pour effectuer toutes les recherches web dans le reste du fichier.

```
22 # Exa search function using basic search and contents
```

→ Commentaire décrivant le rôle de la fonction définie juste après.

```
23 def exa_search(product_list: str, location: str):
```

→ Définit la fonction de recherche prenant la liste de produits et la localisation en paramètres.

```
24     """
```

→ Ouvre la docstring décrivant la fonction.

```
25     Search for products and vendors using Exa's search and contents functionality
```

→ Décrit dans la docstring l'objectif de la fonction.

```
26     """
```

→ Ferme la docstring de la fonction.

```
27     try:
```

→ Démarre un bloc try/except : les appels réseau externes (ici vers Exa) peuvent échouer pour de nombreuses raisons (timeout, quota, panne), donc le code les entoure systématiquement d'une gestion d'erreur plutôt que de laisser le programme planter.

```
28         # Create a comprehensive search query
```

→ Commentaire annonçant la construction de la requête de recherche.

```

29     search_query = f"vendors suppliers {product_list} {location} prices purchasing procurement"
    → Construit la chaîne de requête en insérant la liste de produits et la localisation. Ajouter des mots-clés métier (vendors,
suppliers, prices...) autour de l'entrée utilisateur est une technique de recherche qui aide le moteur à cibler des pages
commerciales pertinentes plutôt que des résultats génériques.

31     print(f"Searching for: {search_query}")
    → Affiche dans la console la requête qui va être envoyée à Exa, utile pour le débogage.

33     # Perform search and get contents
    → Commentaire annonçant l'appel à l'API de recherche.

34     results = exa.search_and_contents(
    → Appelle la méthode search_and_contents du client Exa, qui combine en un seul appel la recherche web et la récupération
du contenu textuel des pages trouvées (au lieu de devoir faire une recherche puis scraper chaque URL séparément).

35         query=search_query,
    → Passe la requête de recherche construite précédemment comme argument.

36         num_results=15, # Get more results for better coverage
    → Demande 15 résultats : un nombre plus élevé augmente la probabilité de trouver de bons fournisseurs, au prix d'un contenu
plus volumineux à faire analyser ensuite par l'agent LLM.

37         text=True, # Include full text content
    → Demande à Exa d'inclure le texte complet extrait de chaque page, indispensable pour que l'agent puisse ensuite lire les prix
et descriptions sans requête supplémentaire.

38         type="auto" # Use auto type for best results
    → Laisse Exa choisir automatiquement entre recherche par mots-clés et recherche sémantique (neurale) selon ce qui donne
les meilleurs résultats pour cette requête.

39     )
    → Ferme l'appel à la méthode search_and_contents.

41     # Format the results for the procurement agent
    → Commentaire annonçant la mise en forme des résultats.

42     formatted_results = f"""
    → Commence la construction d'une chaîne multi-ligne au format Markdown qui contiendra les résultats. Présenter les données
brutes sous forme de texte structuré (titres, sections) facilite leur lecture et leur interprétation par le modèle de langage qui les
recevra ensuite.

43 # Procurement Research Results for {location}
    → Ligne de titre du gabarit de résultats incluant la localisation.

44 **Products searched:** {product_list}
    → Ligne du gabarit affichant la liste des produits recherchés.

45 **Location:** {location}
    → Ligne du gabarit affichant la localisation utilisée pour la recherche.

46 **Search performed:** {search_query}
    → Ligne du gabarit affichant la requête de recherche effectuée.

47 ## Search Results:
    → Ligne de sous-titre du gabarit annonçant la section des résultats.

51 """
    → Ferme la chaîne multi-ligne formatée assignée à formatted_results.

```

```

53         for i, result in enumerate(results.results, 1):
→ Itère sur chaque résultat de recherche renvoyé par Exa, avec un index commençant à 1 pour la numérotation affichée à l'utilisateur/l'agent.

54             formatted_results += f"""
→ Concatène à formatted_results le début d'un bloc de texte pour ce résultat.

55 ##### Result {i}: {result.title or 'No title'}
→ Ligne de gabarit affichant le numéro et le titre du résultat, avec un texte par défaut si Exa n'a pas trouvé de titre.

57 **URL:** {result.url}
→ Ligne de gabarit affichant l'URL du résultat, importante pour que l'agent puisse ensuite citer le site du fournisseur.

58 **Author:** {result.author or 'Not specified'}
→ Ligne de gabarit affichant l'auteur du résultat ou une valeur par défaut.

59 **Published Date:** {result.published_date or 'Not specified'}
→ Ligne de gabarit affichant la date de publication ou une valeur par défaut.

60 **Score:** {result.score or 'N/A'}
→ Ligne de gabarit affichant le score de pertinence attribué par Exa à ce résultat (plus il est élevé, plus la page est jugée correspondre à la requête).

62 **Content Preview:**
→ Ligne de gabarit introduisant l'aperçu du contenu du résultat.

63 {result.text[:1500] if result.text else 'No content available'}...
→ Insère seulement les 1500 premiers caractères du texte de la page : on tronque volontairement pour limiter la taille totale du texte transmis ensuite au modèle de langage (qui a une fenêtre de contexte et un coût par token limités), tout en gardant l'essentiel de l'information.

65 ---
→ Ligne de séparation visuelle Markdown entre deux résultats dans le gabarit.

66 """
→ Ferme le bloc de texte concaténé pour ce résultat.

68         print("----- EXA SEARCH COMPLETION
-----")
→ Affiche une ligne de séparation signalant la fin de la recherche Exa.

69         print(f"Found {len(results.results)} results")
→ Affiche le nombre total de résultats trouvés.

70         print("-----
-----")
→ Affiche une seconde ligne de séparation dans la console.

72         return formatted_results
→ Retourne la chaîne de résultats formatée à l'appelant : c'est ce texte qui servira de contexte à l'agent LLM pour son analyse.

74     except Exception as e:
→ Capture toute exception survenue pendant la recherche principale, pour permettre une tentative de secours plutôt que de faire planter tout le programme.

75         print(f"Error in exa_search: {str(e)}")
→ Affiche le message d'erreur de l'exception capturée, utile pour diagnostiquer un problème (quota dépassé, réseau, etc.).

```

```

76         # Fallback to a simpler search
    → Commentaire annonçant la tentative de recherche de secours.

77         try:
    → Démarre un bloc try imbriqué pour la recherche de secours : un pattern de robustesse à plusieurs niveaux (retry avec une
requête plus simple si la requête riche échoue).

78         print("Trying fallback search...")
    → Indique dans la console qu'une recherche de secours va être tentée.

79         results = exa.search(
    → Appelle la méthode search plus simple du client Exa (sans récupération de contenu), qui a moins de chances d'échouer car
elle demande moins de traitement à l'API.

80             query=f"{product_list} suppliers {location}",
    → Construit une requête simplifiée pour la recherche de secours.

81             num_results=10
    → Limite la recherche de secours à 10 résultats.

82         )
    → Ferme l'appel à la méthode search de secours.

84         fallback_results = f"""
    → Commence la construction d'une chaîne multi-ligne pour les résultats de secours, avec le même principe de mise en forme
Markdown que précédemment.

85 # Fallback Search Results for {location}
    → Ligne de titre du gabarit de secours incluant la localisation.

87 **Products searched:** {product_list}
    → Ligne du gabarit de secours affichant les produits recherchés.

88 **Location:** {location}
    → Ligne du gabarit de secours affichant la localisation.

90 ## Basic Search Results:
    → Ligne de sous-titre du gabarit de secours annonçant les résultats basiques.

92 """
    → Ferme la chaîne multi-ligne assignée à fallback_results.

94         for i, result in enumerate(results.results, 1):
    → Itère sur chaque résultat de la recherche de secours avec un index commençant à 1.

95             fallback_results += f"""
    → Concatène à fallback_results le début d'un bloc de texte pour ce résultat de secours.

96 ### Result {i}: {result.title or 'No title'}
    → Ligne de gabarit affichant le numéro et le titre du résultat de secours.

97 - **URL:** {result.url}
    → Ligne de gabarit affichant l'URL du résultat de secours sous forme de puce.

98 - **Score:** {result.score or 'N/A'}
    → Ligne de gabarit affichant le score du résultat de secours sous forme de puce.

99 - **Published:** {result.published_date or 'Not specified'}

```

→ Ligne de gabarit affichant la date de publication du résultat de secours.

101 """

→ Ferme le bloc de texte concaténé pour ce résultat de secours.

103 return fallback_results

→ Retourne la chaîne de résultats de secours à l'appelant : même en cas d'échec de la recherche riche, l'agent recevra tout de même des données exploitables.

105 except Exception as fallback_error:

→ Capture toute exception survenue pendant la recherche de secours elle-même (deuxième niveau de robustesse).

106 print(f"Fallback search also failed: {str(fallback_error)}")

→ Affiche le message d'erreur de la recherche de secours ayant échoué.

107 return f"""

→ Commence la construction et le retour d'un message d'erreur multi-ligne final, envoyé à l'agent à défaut de données réelles, afin qu'il puisse en informer l'utilisateur plutôt que de recevoir une chaîne vide incompréhensible.

108 # Search Error

→ Ligne de titre du message d'erreur final.

110 Unable to perform search for products: {product_list} in location: {location}

→ Ligne du message d'erreur précisant les produits et la localisation concernés.

112 Error details: {str(e)}

→ Ligne du message d'erreur affichant le détail de l'exception initiale.

114 Please check your Exa API key and try again.

→ Ligne du message d'erreur invitant l'utilisateur à vérifier sa clé API.

115 """

→ Ferme la chaîne de message d'erreur et termine la fonction `exa_search`.

117 class ProcurementAgent(Workflow):

→ Définit la classe `ProcurementAgent` héritant de `Workflow`. Hériter de `Workflow` permet de bénéficier des mécanismes internes d'agno (gestion du cycle de vie, du streaming, etc.) tout en définissant sa propre logique métier dans une méthode `run()`.

118 procurement_agent: Agent = Agent(

→ Déclare `procurement_agent` comme attribut de classe et commence à instancier un `Agent` : cet agent est l'analyste qui va recevoir les résultats de recherche Exa et produire le CSV et le résumé.

119 name='Procurement Agent',

→ Donne le nom 'Procurement Agent' à cet agent, utile pour l'identifier dans les logs ou une interface multi-agents.

120 model=OpenAIChat(id='gpt-4o'),

→ Configure l'agent pour utiliser le modèle OpenAI gpt-4o comme moteur de raisonnement : c'est ce modèle qui va lire les résultats de recherche, en extraire les données structurées et rédiger le résumé.

121 instructions=dedent("""

→ Commence à définir les instructions système de l'agent (le "prompt système"), désindentées avec `dedent` pour ne pas polluer le texte envoyé au modèle avec l'indentation du code Python.

122 You are a procurement analysis agent.

→ Ligne d'instruction indiquant à l'agent son rôle d'analyste d'achats.

124 Your goal is to:

→ Ligne d'instruction introduisant la liste des objectifs de l'agent.

125 1. Parse the search results from Exa containing information about vendors and suppliers.
→ Instruction demandant d'analyser les résultats de recherche Exa.

126 2. For each relevant product/vendor combination found, extract the following fields:
→ Instruction demandant d'extraire des champs pour chaque couple produit/fournisseur.

127 - Product Name
→ Énumère le champ Product Name à extraire.

128 - Vendor Name
→ Énumère le champ Vendor Name à extraire.

129 - Product Title
→ Énumère le champ Product Title à extraire.

130 - Price (if mentioned, convert to numeric if possible)
→ Énumère le champ Price à extraire, converti en nombre si possible.

131 - Currency (if mentioned)
→ Énumère le champ Currency à extraire si mentionné.

132 - Vendor Website or Purchase Link
→ Énumère le champ du site web ou lien d'achat du fournisseur.

133 - Short Product Description
→ Énumère le champ de description courte du produit.

134 - Minimum Order Quantity (if available)
→ Énumère le champ de quantité minimale de commande si disponible.

135 - Shipping Time (if available)
→ Énumère le champ de délai de livraison si disponible.

136 - Bulk Discounts or Deals (if available)
→ Énumère le champ des remises ou offres en gros si disponible.

137 - Vendor Location (if mentioned)
→ Énumère le champ de localisation du fournisseur si mentionné.

139 3. After extracting the data, write and execute a Python script that creates a file named
`data.csv`
→ Instruction demandant d'écrire et d'exécuter un script Python créant le fichier data.csv : c'est cette instruction qui déclenche l'usage de l'outil PythonTools par l'agent plutôt qu'une simple réponse textuelle.

140 - Use the standard `csv` module
→ Précise que le script doit utiliser le module standard csv.

141 - Make the columns in this order: Product Name, Vendor Name, Product Title, Price,
Currency, Bulk Discounts or Deals, Vendor Website, Short Product Description, Minimum Order Quantity,
Shipping Time, Vendor Location
→ Précise l'ordre exact des colonnes du CSV attendu.

142 - The script should write a header row followed by one row per vendor/product combination
→ Précise que le script doit écrire un en-tête puis une ligne par couple produit/fournisseur.

143 - IMPORTANT: When opening the file, always use encoding='utf-8' in open(), e.g.
open('data.csv', 'w', newline='', encoding='utf-8')
→ Précise l'encodage utf-8 obligatoire à utiliser lors de l'ouverture du fichier, pour éviter les erreurs d'encodage avec des caractères spéciaux (accents, symboles monétaires) dans les données extraites du web.

- 144 - Then use the PythonTools tool to run the script and save the data
→ Précise qu'il faut utiliser l'outil PythonTools pour exécuter le script et sauvegarder les données.
- 145 - Create rows for each relevant data point found in the search results
→ Précise de créer une ligne pour chaque donnée pertinente trouvée.
- 147 Then:
→ Ligne de transition annonçant l'étape suivante des instructions.
- 149 4. Analyze the data across all products and vendors.
→ Instruction demandant d'analyser les données de tous les produits et fournisseurs.
- 150 - Compare vendors based on pricing, shipping times, minimum quantities, and available deals.
→ Précise les critères de comparaison des fournisseurs.
- 151 - Prioritize vendors who:
→ Introduit la liste des critères de priorisation des fournisseurs.
- 152 - Deliver to the specified location
→ Critère de priorisation: livraison vers la localisation spécifiée.
- 153 - Offer competitive prices
→ Critère de priorisation: prix compétitifs.
- 154 - Have favorable shipping times or bulk deals
→ Critère de priorisation: délais de livraison ou offres en gros favorables.
- 155 - Appear reliable (from established marketplaces or verified sellers)
→ Critère de priorisation: fiabilité du vendeur ou de la marketplace.
- 157 5. Write an executive summary that includes:
→ Instruction demandant de rédiger un résumé exécutif et en liste le contenu attendu.
- 158 - Recommended vendor(s) per product (if clear recommendations can be made)
→ Précise d'inclure le ou les fournisseurs recommandés par produit.
- 159 - Reasons for the recommendation (price, location, delivery, etc.)
→ Précise d'inclure les raisons de la recommandation.
- 160 - Any noteworthy observations (e.g. pricing differences, available options)
→ Précise d'inclure les observations notables.
- 161 - Limitations of the search results (if search quality was limited)
→ Précise d'inclure les limites des résultats de recherche.
- 163 IMPORTANT:
→ Introduit un bloc de remarques importantes finales pour l'agent.
- 164 - Use PythonTools to write the extracted data to `data.csv`.
→ Rappelle d'utiliser PythonTools pour écrire les données dans data.csv.
- 165 - If search results are limited or don't contain specific pricing/vendor info, acknowledge this in your analysis.
→ Demande de signaler dans l'analyse si les résultats de recherche sont limités.
- 166 - Focus on extracting whatever relevant information is available from the search results.

→ Demande de se concentrer sur l'extraction des informations pertinentes disponibles.

167 `"""),`

→ Ferme la chaîne dedent des instructions et l'argument instructions de l'Agent.

168 `expected_output=dedent("""`

→ Commence à définir le format de sortie attendu de l'agent : donner explicitement un gabarit de sortie est une technique de prompt engineering qui augmente la cohérence et la lisibilité des réponses générées par le modèle.

169 `The output should include:`

→ Ligne introduisant la liste des éléments attendus dans la sortie.

171 `1. Data Summary:`

→ Introduit la section de résumé des données attendue en sortie.

172 `- Number of relevant results processed`

→ Précise d'inclure le nombre de résultats pertinents traités.

173 `- Total vendors/suppliers identified`

→ Précise d'inclure le nombre total de fournisseurs identifiés.

174 `- Location considered for delivery: <city>, <country>`

→ Précise d'inclure la localisation de livraison considérée.

175 `- Data quality assessment and any limitations`

→ Précise d'inclure une évaluation de la qualité des données et ses limites.

177 `2. Recommendations Per Product:`

→ Introduit la section de recommandations par produit attendue en sortie.

178 `For each product found (e.g., "banane", "pomme"), provide:`

→ Précise le format attendu pour chaque produit trouvé, avec des exemples.

180 `### Product: <Product Name>`

→ Ligne de gabarit montrant le titre attendu pour chaque produit.

182 `**Available Options Found:**`

→ Ligne de gabarit introduisant les options disponibles trouvées.

183 `- List the vendors/suppliers found with their key details`

→ Précise de lister les fournisseurs trouvés avec leurs détails clés.

185 `**Best Option (if determinable):** <Vendor Business Name>`

→ Ligne de gabarit indiquant où placer la meilleure option identifiée.

186 `**Why Recommended:**`

→ Ligne de gabarit introduisant les raisons de la recommandation.

187 `- Reason 1 (e.g. local supplier, competitive pricing mentioned)`

→ Exemple de première raison possible de recommandation.

188 `- Reason 2 (e.g. established marketplace presence)`

→ Exemple de deuxième raison possible de recommandation.

189 `- Reason 3 (e.g. good shipping options or bulk availability)`

→ Exemple de troisième raison possible de recommandation.

191 `**Alternative Options:**`

→ Ligne de gabarit introduisant les options alternatives.

192 - Mention other viable vendors if found
→ Précise de mentionner d'autres fournisseurs viables si trouvés.

194 ****Data Quality Note:****
→ Ligne de gabarit introduisant la note sur la qualité des données.

195 - Acknowledge any limitations in the search results
→ Précise de signaler les limites des résultats de recherche.

196 - Mention if specific pricing or delivery info was not available
→ Précise de signaler si des informations de prix ou livraison manquaient.

198 **IMPORTANT:** Confirm that the CSV file was created and contains the extracted data.
→ Rappelle de confirmer que le fichier CSV a bien été créé avec les données.

199 **"""),**
→ Ferme la chaîne dedent de `expected_output` et l'argument correspondant.

200 **tools=[PythonTools()],**
→ Fournit à l'agent l'outil `PythonTools` : sans cette ligne, l'agent ne pourrait que discuter du CSV à créer, il ne pourrait pas réellement exécuter de code pour le générer sur le disque.

201 **show_tool_calls=True,**
→ Active l'affichage des appels d'outils effectués par l'agent, ce qui rend visible dans la sortie quel code Python a été exécuté et avec quel résultat — utile pour la transparence et le débogage d'un agent autonome.

202 **markdown=True**
→ Indique que la sortie de l'agent doit être interprétée comme du markdown, pour un affichage correctement formaté (titres, listes, gras) plutôt que du texte brut.

203 **)**
→ Ferme l'appel au constructeur Agent définissant `procurement_agent`.

205 **def run(self, product_list: str, location: str):**
→ Définit la méthode `run` du workflow, le point d'entrée qui orchestre les étapes de l'agent : c'est cette méthode qui est appelée quand on exécute le workflow.

206 **research_response = exa_search(product_list, location)**
→ Appelle la fonction `exa_search` pour obtenir les résultats de recherche formatés : première étape du workflow (recherche), avant l'étape d'analyse par le LLM.

207 **yield from self.procurement_agent.run(research_response, stream=True)**
→ Délègue les résultats de recherche à l'agent LLM et retransmet son flux de réponse. `stream=True` active le streaming : au lieu d'attendre que l'agent ait fini de générer toute sa réponse, les morceaux de texte sont renvoyés au fur et à mesure qu'ils sont produits, ce qui donne une sensation de réactivité à l'utilisateur (comme dans ChatGPT). `yield from` propage ce flux du générateur interne de l'agent vers l'appelant du workflow.

209 **if __name__ == '__main__':**
→ Vérifie que le script est exécuté directement et non importé comme module.

210 **from rich.prompt import Prompt**
→ Importe localement la classe `Prompt` de la librairie `rich`, qui fournit des invites de saisie en ligne de commande avec une mise en forme soignée (couleurs, validation), plus agréable que le `input()` natif de Python.

212 **# Check if API key is available**
→ Commentaire annonçant la vérification de la présence de la clé API.

213 **if not os.environ.get('EXA_API_KEY'):**

→ Vérifie à nouveau que la clé API Exa est bien définie avant de continuer.

```
214         print(" Error: EXA_API_KEY environment variable is not set.")
```

→ Affiche un message d'erreur si la clé API est absente.

```
215         print("Please set your Exa API key in your .env file:")
```

→ Affiche une instruction pour définir la clé API dans le fichier .env.

```
216         print("EXA_API_KEY=your_api_key_here")
```

→ Affiche un exemple de ligne à ajouter dans le fichier .env.

```
217         exit(1)
```

→ Quitte le programme avec un code de sortie différent de zéro, convention indiquant une terminaison en erreur.

```
219     product_list = Prompt.ask('Enter your products list separated by commas')
```

→ Demande interactivement à l'utilisateur la liste des produits séparés par des virgules.

```
220     location = Prompt.ask('Enter your business location (city, country)')
```

→ Demande interactivement à l'utilisateur sa localisation professionnelle.

```
222     if product_list and location:
```

→ Vérifie que les deux saisies utilisateur ne sont pas vides avant de continuer.

```
223         try:
```

→ Démarre un bloc try pour capturer les erreurs d'exécution du workflow.

```
224             workflow = ProcurementAgent()
```

→ Instancie la classe ProcurementAgent définie plus haut.

```
225             response: Iterator[RunResponse] = workflow.run(product_list=product_list,
location=location)
```

→ Lance le workflow et récupère l'itérateur de réponses en streaming (typé Iterator[RunResponse] grâce à l'import de typing effectué plus haut).

```
226             pprint_run_response(response, markdown=True)
```

→ Affiche joliment le flux de réponse du workflow en interprétant le markdown, morceau par morceau au fur et à mesure qu'il arrive.

```
227         except Exception as e:
```

→ Capture toute exception survenue lors de l'exécution du workflow.

```
228             print(f" Error running procurement agent: {str(e)}")
```

→ Affiche le message d'erreur rencontré lors de l'exécution.

```
229             print("Please check your API keys and try again.")
```

→ Affiche une invitation à vérifier les clés API et réessayer.

31-Procurement-agent/create_data_csv.py

Ce script autonome génère un fichier data.csv d'exemple à partir de données de fournisseurs codées en dur dans le code, servant de jeu de données de démonstration ou de secours indépendant de l'agent de recherche.

62 lignes de code annotées ci-dessous.

```
1 import csv
```

→ Importe le module standard csv, qui fournit des outils pour lire et écrire des fichiers au format CSV (valeurs séparées par des virgules) en gérant correctement l'échappement des guillemets, virgules et retours à la ligne dans les champs.

3 data = [

→ Déclare la liste data et commence à définir ses éléments : chaque élément est un dictionnaire représentant une future ligne du fichier CSV, avec des données fixes plutôt qu'issues d'une recherche en temps réel.

4 {

→ Ouvre le premier dictionnaire de la liste représentant une entrée de données.

5 "Product Name": "Bananas",

→ Définit la clé Product Name avec la valeur "Bananas" pour cette entrée.

6 "Vendor Name": "United States",

→ Définit la clé Vendor Name avec la valeur "United States" pour cette entrée.

7 "Product Title": "Bananas",

→ Définit la clé Product Title avec la valeur "Bananas" pour cette entrée.

8 "Price": 1.50,

→ Définit la clé Price avec la valeur numérique 1.50 pour cette entrée.

9 "Currency": "USD",

→ Définit la clé Currency avec la valeur "USD" pour cette entrée.

10 "Vendor Website": "https://www.e-fresco.io/bananas-4dqa.html",

→ Définit la clé Vendor Website avec l'URL du site du fournisseur.

11 "Short Product Description": "Bananas for retailers, available through an online platform.",

→ Définit la clé Short Product Description avec une description du produit.

12 "Minimum Order Quantity": "",

→ Définit la clé Minimum Order Quantity avec une chaîne vide, valeur non renseignée.

13 "Shipping Time": "",

→ Définit la clé Shipping Time avec une chaîne vide, valeur non renseignée.

14 "Bulk Discounts or Deals": "",

→ Définit la clé Bulk Discounts or Deals avec une chaîne vide, valeur non renseignée.

15 "Vendor Location": "United States"

→ Définit la clé Vendor Location avec la valeur "United States" pour cette entrée.

16 },

→ Ferme le premier dictionnaire et sépare l'élément suivant de la liste.

17 {

→ Ouvre le deuxième dictionnaire de la liste représentant une nouvelle entrée.

18 "Product Name": "",

→ Définit la clé Product Name avec une chaîne vide pour cette entrée.

19 "Vendor Name": "Wali Trading",

→ Définit la clé Vendor Name avec la valeur "Wali Trading" pour cette entrée.

20 "Product Title": "",

→ Définit la clé Product Title avec une chaîne vide pour cette entrée.

21 "Price": "",

→ Définit la clé Price avec une chaîne vide, prix non renseigné.

22 "Currency": "",
→ Définit la clé Currency avec une chaîne vide, devise non renseignée.

23 "Vendor Website": "https://wali-trading.com/",
→ Définit la clé Vendor Website avec l'URL du site de Wali Trading.

24 "Short Product Description": "Offers a wide range of fruits, including bananas, from Morocco.",
→ Définit la clé Short Product Description avec une description du fournisseur.

25 "Minimum Order Quantity": "",
→ Définit la clé Minimum Order Quantity avec une chaîne vide, valeur non renseignée.

26 "Shipping Time": "",
→ Définit la clé Shipping Time avec une chaîne vide, valeur non renseignée.

27 "Bulk Discounts or Deals": "",
→ Définit la clé Bulk Discounts or Deals avec une chaîne vide, valeur non renseignée.

28 "Vendor Location": "Morocco"
→ Définit la clé Vendor Location avec la valeur "Morocco" pour cette entrée.

29 },
→ Ferme le deuxième dictionnaire et sépare l'élément suivant de la liste.

30 {
→ Ouvre le troisième dictionnaire de la liste représentant une nouvelle entrée.

31 "Product Name": "",
→ Définit la clé Product Name avec une chaîne vide pour cette entrée.

32 "Vendor Name": "Zenkal Trading SARL",
→ Définit la clé Vendor Name avec la valeur "Zenkal Trading SARL" pour cette entrée.

33 "Product Title": "",
→ Définit la clé Product Title avec une chaîne vide pour cette entrée.

34 "Price": "",
→ Définit la clé Price avec une chaîne vide, prix non renseigné.

35 "Currency": "",
→ Définit la clé Currency avec une chaîne vide, devise non renseignée.

36 "Vendor Website": "https://www.zenkaltrading.com/",
→ Définit la clé Vendor Website avec l'URL du site de Zenkal Trading SARL.

37 "Short Product Description": "Global exporter of fresh fruits and vegetables, offering various products from Morocco.",
→ Définit la clé Short Product Description avec une description de l'activité du fournisseur.

38 "Minimum Order Quantity": "",
→ Définit la clé Minimum Order Quantity avec une chaîne vide, valeur non renseignée.

39 "Shipping Time": "",
→ Définit la clé Shipping Time avec une chaîne vide, valeur non renseignée.

40 "Bulk Discounts or Deals": "",
→ Définit la clé Bulk Discounts or Deals avec une chaîne vide, valeur non renseignée.

41 "Vendor Location": "Casablanca, MA"

→ Définit la clé Vendor Location avec la valeur "Casablanca, MA" pour cette entrée.

42 },

→ Ferme le troisième dictionnaire et sépare l'élément suivant de la liste.

43 {

→ Ouvre le quatrième dictionnaire de la liste représentant une nouvelle entrée.

44 "Product Name": "Pomme products",

→ Définit la clé Product Name avec la valeur "Pomme products" pour cette entrée.

45 "Vendor Name": "Pomme de Pain Maroc",

→ Définit la clé Vendor Name avec la valeur "Pomme de Pain Maroc" pour cette entrée.

46 "Product Title": "",

→ Définit la clé Product Title avec une chaîne vide pour cette entrée.

47 "Price": "",

→ Définit la clé Price avec une chaîne vide, prix non renseigné.

48 "Currency": "",

→ Définit la clé Currency avec une chaîne vide, devise non renseignée.

49 "Vendor Website": "https://www.linkedin.com/company/pomme-de-pain-maroc",

→ Définit la clé Vendor Website avec l'URL de la page LinkedIn du fournisseur.

50 "Short Product Description": "Offers a variety of pomme (apple) products along with other fresh items.",

→ Définit la clé Short Product Description avec une description des produits proposés.

51 "Minimum Order Quantity": "",

→ Définit la clé Minimum Order Quantity avec une chaîne vide, valeur non renseignée.

52 "Shipping Time": "",

→ Définit la clé Shipping Time avec une chaîne vide, valeur non renseignée.

53 "Bulk Discounts or Deals": "",

→ Définit la clé Bulk Discounts or Deals avec une chaîne vide, valeur non renseignée.

54 "Vendor Location": "Casablanca, MA"

→ Définit la clé Vendor Location avec la valeur "Casablanca, MA" pour cette entrée.

55 }

→ Ferme le quatrième et dernier dictionnaire de la liste (pas de virgule finale, car c'est le dernier élément).

56]

→ Ferme la liste data contenant les quatre entrées définies.

58 # Create CSV

→ Commentaire annonçant la création du fichier CSV.

59 with open('data.csv', 'w', newline='', encoding='utf-8') as csvfile:

→ Ouvre (ou crée) le fichier data.csv en écriture, avec newline="" (recommandation standard du module csv pour éviter des lignes vides supplémentaires sous Windows) et l'encodage utf-8 pour bien gérer les caractères spéciaux, en l'associant à csvfile via un gestionnaire de contexte qui garantit sa fermeture propre.

60 fieldnames = ["Product Name", "Vendor Name", "Product Title", "Price", "Currency", "Bulk Discounts or Deals", "Vendor Website", "Short Product Description", "Minimum Order Quantity", "Shipping Time",

```
"Vendor Location"]
```

→ Définit la liste des noms de colonnes dans l'ordre attendu du CSV, qui doit correspondre aux clés utilisées dans les dictionnaires de data.

```
61 writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
```

→ Crée un objet DictWriter, une classe du module csv qui sait écrire directement des dictionnaires Python comme des lignes de CSV en respectant l'ordre de colonnes donné par fieldnames, plutôt que d'avoir à convertir manuellement chaque dictionnaire en liste ordonnée de valeurs.

```
63 writer.writeheader()
```

→ Écrit la ligne d'en-tête du CSV à partir des noms de colonnes de fieldnames.

```
64 for entry in data:
```

→ Itère sur chaque dictionnaire de la liste data.

```
65 writer.writerow(entry)
```

→ Écrit l'entrée courante comme une ligne du fichier CSV, en plaçant chaque valeur sous la colonne correspondant à sa clé.

Projet 32 — 32-Advance-Research-agent

Assistant de recherche academique avec base de connaissances PgVector

Objectif

Permet d'uploader un article academique au format PDF, en extrait le texte par OCR, l'indexe dans une base de connaissances vectorielle, puis genere un resume automatique et repond a des questions approfondies sur le document, en citant ses sources et en completant par une recherche web externe si necessaire.

Code source, ligne par ligne

32-Advance-Research-agent/research_assistant_agent.py

Ce fichier implémente une application web Streamlit qui fait l'OCR d'un PDF de recherche via l'API Mistral, indexe le contenu dans une base de connaissances vectorielle pour faire du RAG (Retrieval-Augmented Generation), puis expose un agent de résumé et un agent de questions-réponses basés sur des modèles Groq consultant cette base.

160 lignes de code annotées ci-dessous.

1 import sys

→ Importe le module standard sys, qui donne accès aux flux d'entrée/sortie du processus (stdin, stdout, stderr) et à d'autres éléments internes de l'interpréteur Python.

3 for _stream in (sys.stdout, sys.stderr):

→ Itère sur les flux de sortie standard et d'erreur standard pour les reconfigurer tous les deux de la même façon.

4 if hasattr(_stream, "reconfigure"):

→ Vérifie que le flux courant possède la méthode reconfigure (disponible depuis Python 3.7) avant de l'appeler, pour éviter une erreur si le flux a été redirigé vers un objet qui ne la supporte pas.

5 _stream.reconfigure(encoding="utf-8", errors="replace")

→ Reconfigure le flux pour utiliser l'encodage utf-8 et remplacer les caractères non décodables par un caractère de substitution plutôt que de lever une exception. Ceci évite des plantages liés à l'encodage quand le terminal Windows (souvent en cp1252) reçoit des caractères spéciaux issus de l'OCR ou des réponses du modèle.

7 import os

→ Importe le module standard os pour accéder aux variables d'environnement et au système de fichiers.

9 # sentence-transformers (utilisé pour les embeddings locaux) importe transformers,

→ Commentaire expliquant pourquoi la variable d'environnement suivante est nécessaire.

10 # qui essaie par défaut de charger TensorFlow en plus de PyTorch. La version de

→ Suite du commentaire explicatif sur le conflit avec TensorFlow.

11 # TensorFlow installée sur cette machine est incompatible avec la version de

→ Suite du commentaire explicatif sur l'incompatibilité de version.

12 # protobuf présente : on désactive TF avant tout import de la lib.

→ Fin du commentaire expliquant la désactivation de TensorFlow avant les imports.

13 os.environ.setdefault("USE_TF", "0")

→ Définit la variable d'environnement USE_TF à "0", uniquement si elle n'est pas déjà définie (setdefault). La librairie transformers, utilisée en interne par sentence-transformers pour générer les embeddings, détecte les frameworks de deep

learning installés (PyTorch, TensorFlow) et essaie de charger TensorFlow par défaut ; cette ligne force transformers à n'utiliser que PyTorch pour éviter un conflit de versions avec protobuf sur cette machine.

15 `import requests`

→ Importe la librairie requests, la bibliothèque tierce la plus utilisée en Python pour effectuer des appels HTTP (ici vers l'API Semantic Scholar).

16 `import streamlit as st`

→ Importe la librairie Streamlit sous l'alias st. Streamlit est un framework qui permet de construire une interface web interactive (upload de fichiers, boutons, zones de texte) en pur Python, sans écrire de HTML/CSS/JS, particulièrement populaire pour prototyper rapidement des applications autour de l'IA.

17 `from textwrap import dedent`

→ Importe dedent pour retirer l'indentation commune des chaînes de texte multi-lignes (utilisé pour les instructions des agents).

18 `from agno.agent import Agent`

→ Importe la classe Agent du framework agno, l'abstraction centrale représentant un agent IA (modèle + instructions + outils + base de connaissances).

19 `from mistralai.client import Mistral`

→ Importe le client officiel de l'API Mistral. Il est utilisé ici non pas pour du chat, mais pour son service d'OCR (reconnaissance optique de caractères) qui convertit un PDF (y compris scanné) en texte structuré au format Markdown.

20 `from agno.models.groq import Groq`

→ Importe la classe qui encapsule un modèle de chat hébergé sur Groq. Groq est une plateforme d'inférence pour modèles de langage réputée pour sa très grande vitesse de génération (grâce à du matériel dédié, les LPU), et propose un tier gratuit avec des limites de débit (tokens par minute) — ce qui explique plusieurs choix de configuration plus bas dans ce fichier.

21 `from agno.vectordb.pgvector import PgVector`

→ Importe PgVector, la classe qui permet d'utiliser PostgreSQL (via son extension pgvector) comme base de données vectorielle. Une base vectorielle stocke des documents sous forme de vecteurs numériques (embeddings) et permet de retrouver rapidement les documents dont le sens est le plus proche d'une requête, ce qui est la brique de stockage/recherche au cœur d'un système RAG.

22 `from agno.tools.reasoning import ReasoningTools`

→ Importe l'outil de raisonnement structuré fourni par agno. Il donne à l'agent une sorte de "bloc-notes" d'outils (réfléchir, planifier, faire le point) qu'il peut appeler explicitement pendant son raisonnement, ce qui améliore la qualité des réponses sur des tâches complexes en le forçant à décomposer son raisonnement en étapes visibles plutôt qu'en une seule réponse directe.

23 `from agno.tools.duckduckgo import DuckDuckGoTools`

→ Importe l'outil de recherche web basé sur DuckDuckGo, qui permet à l'agent de chercher de l'information sur le web en direct (contrairement à sa base de connaissances qui est figée au contenu du PDF indexé), sans nécessiter de clé API.

24 `from agno.knowledge.knowledge import Knowledge`

→ Importe la classe Knowledge, l'abstraction agno qui représente une base de connaissances consultable par un agent. C'est le composant central qui relie un stockage vectoriel (ici PgVector) et un embedder pour permettre à l'agent de retrouver des passages pertinents dans un corpus de documents avant de répondre — c'est le mécanisme de RAG (Retrieval-Augmented Generation, "génération augmentée par récupération") : au lieu de compter uniquement sur ce que le modèle a appris pendant son entraînement, on va chercher dans une base documentaire externe (ici le PDF OCRisé) les passages pertinents et on les injecte dans le contexte du modèle avant qu'il ne génère sa réponse, ce qui réduit les hallucinations et permet de répondre sur des documents que le modèle n'a jamais vus.

25 `from agno.knowledge.embedder.sentence_transformer import SentenceTransformerEmbedder`

→ Importe l'embedder basé sur la librairie open-source sentence-transformers, exécuté localement sur la machine. Un embedder est le composant qui transforme un texte en un vecteur numérique (une liste de nombres) capturant son sens ; c'est cette conversion qui permet ensuite de comparer des textes par similarité mathématique (distance entre vecteurs) plutôt que par correspondance exacte de mots. Utiliser un embedder local plutôt qu'une API payante (comme celle d'OpenAI) évite les coûts et les limites de débit lors de l'indexation.

26 `from dotenv import load_dotenv`

→ Importe la fonction qui charge les variables d'environnement depuis un fichier .env.

27 load_dotenv()

→ Charge les variables définies dans le fichier .env dans l'environnement du processus.

29 mistral_key = os.getenv("MISTRAL_API_KEY")

→ Récupère la clé API Mistral depuis les variables d'environnement, nécessaire pour utiliser le service d'OCR.

30 # Le script original utilisait OpenAI (chat + embeddings) ; remplacé par Groq

→ Commentaire expliquant le remplacement du modèle OpenAI par Groq.

31 # pour le chat (clé GROQ_API_KEY déjà disponible) et par un embedder local

→ Suite du commentaire expliquant l'usage d'un embedder local.

32 # (sentence-transformers) pour les embeddings, gratuit et sans limite de débit.

→ Fin du commentaire justifiant l'usage de sentence-transformers : contrairement à une API d'embeddings payante et soumise à des quotas, sentence-transformers tourne localement et n'a donc ni coût ni limite de requêtes par minute.

33 groq_key = os.getenv("GROQ_API_KEY")

→ Récupère la clé API Groq depuis les variables d'environnement, nécessaire pour interroger les modèles de chat hébergés sur Groq.

34 client = Mistral(api_key=mistral_key)

→ Crée l'instance du client Mistral authentifiée, utilisée ensuite pour lancer l'OCR des PDF.

36 ## OCR the pdf ###

→ Commentaire marquant le début de la section de traitement OCR.

37 def ocr_pdf(pdf_path):

→ Définit la fonction ocr_pdf prenant en paramètre le chemin du PDF à traiter.

38 # check if the file exists

→ Commentaire annonçant la vérification de l'existence du fichier.

39 if not os.path.exists(pdf_path):

→ Vérifie que le fichier PDF spécifié existe sur le disque.

40 st.error("PDF file not found")

→ Affiche un message d'erreur dans l'interface Streamlit (bandeau rouge) si le fichier est introuvable — st.error() est le mécanisme Streamlit pour remonter une erreur visuellement à l'utilisateur de l'application web.

41 return

→ Interrompt la fonction si le fichier PDF n'existe pas.

43 uploaded_pdf = client.files.upload(

→ Envoie le fichier PDF au service Mistral : l'API OCR de Mistral fonctionne en deux temps, il faut d'abord uploader le fichier pour obtenir une référence exploitable, avant de pouvoir lancer le traitement OCR dessus.

44 file={

→ Ouvre le dictionnaire décrivant le fichier à uploader.

45 "file_name": pdf_path,

→ Précise le nom de fichier à utiliser pour l'upload, ici le chemin complet.

46 "content": open(pdf_path, "rb"),

→ Ouvre le fichier PDF en mode binaire ("rb", read-binary) pour fournir son contenu brut à l'upload, nécessaire car un PDF n'est pas un fichier texte.

47 },

→ Ferme le dictionnaire décrivant le fichier.

```
48         purpose="ocr"
```

→ Indique explicitement à l'API Mistral que ce fichier est uploadé dans le but de faire de l'OCR, par opposition à d'autres usages possibles de l'API fichiers (fine-tuning, etc.).

```
49     )
```

→ Ferme l'appel à la méthode `files.upload`.

```
51     signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)
```

→ Récupère une URL signée temporaire permettant d'accéder au fichier uploadé : c'est cette URL, et non le fichier local, que l'endpoint OCR va aller lire.

```
53     ocr_response = client.ocr.process(
```

→ Lance le traitement OCR proprement dit sur le document distant.

```
54         model="mistral-ocr-latest",
```

→ Spécifie le modèle OCR à utiliser, ici la dernière version disponible du modèle spécialisé `mistral-ocr`, capable de reconnaître texte, mise en page et images dans un PDF (y compris scanné) et de restituer le tout en Markdown structuré.

```
55         document={
```

→ Ouvre le dictionnaire décrivant le document à traiter.

```
56             "type": "document_url",
```

→ Précise que le document est référencé par une URL plutôt que fourni en contenu brut dans la requête.

```
57             "document_url": signed_url.url,
```

→ Fournit l'URL signée du document à traiter.

```
58         },
```

→ Ferme le dictionnaire décrivant le document.

```
59         include_image_base64=True
```

→ Demande d'inclure dans la réponse les images extraites du PDF, encodées en base64, pour pouvoir éventuellement les réutiliser (figures, graphiques).

```
60     )
```

→ Ferme l'appel à la méthode `ocr.process`.

```
62     with open(f"DocumentMarkdown/ocr_document.md", "w", encoding="utf-8") as f:
```

→ Ouvre en écriture le fichier `ocr_document.md` dans le dossier `DocumentMarkdown` avec l'encodage `utf-8`, en utilisant un gestionnaire de contexte (`with`) qui garantit la fermeture propre du fichier même en cas d'erreur.

```
63         f.write("\n".join([page.markdown for page in ocr_response.pages]))
```

→ Le résultat OCR de Mistral est découpé par page ; cette ligne concatène le Markdown de chaque page (séparées par un saut de ligne) pour reconstituer le document complet, puis l'écrit sur le disque. C'est ce fichier Markdown qui sera ensuite indexé dans la base de connaissances vectorielle.

```
65 def knowledge_base_setup():
```

→ Définit la fonction qui construit et retourne la base de connaissances utilisée par les agents.

```
66     """ Setup Knowledge Base """
```

→ Commentaire marquant le début de la configuration de la base de connaissances.

```
67     # MarkdownKnowledgeBase(path=..., vector_db=...) a été remplacé par la classe
```

→ Commentaire expliquant le remplacement de l'ancienne classe par `Knowledge`.

68 # unifiée Knowledge : le contenu se charge via add_content(path=...) plutôt
→ Suite du commentaire expliquant le nouveau mode de chargement du contenu.

69 # qu'un argument du constructeur + .load(recreate=...).
→ Fin du commentaire comparant l'ancien et le nouveau mode de chargement.

70 knowledge_base = Knowledge(
→ Instancie la base de connaissances RAG et commence à passer ses arguments.

71 vector_db=PgVector(
→ Configure la base vectorielle utilisée et commence à instancier PgVector : c'est le composant "stockage + recherche" du RAG.

72 table_name="markdown_documents",
→ Spécifie le nom de la table PostgreSQL qui stockera les vecteurs et le texte des documents indexés.

73 db_url="postgresql+psycopg://ai:ai@localhost:5532/ai",
→ Spécifie l'URL de connexion à la base de données PostgreSQL (utilisateur, mot de passe, hôte, port et nom de base), ici une instance locale, généralement lancée via un conteneur Docker préconfiguré avec l'extension pgvector.

74 embedder=SentenceTransformerEmbedder(),
→ Configure l'embedder local sentence-transformers pour générer les vecteurs : c'est lui qui sera appelé à la fois pour indexer le document (au moment de add_content) et pour transformer chaque question de l'utilisateur en vecteur comparable lors d'une recherche.

75),
→ Ferme l'instanciation de PgVector.

76 # La valeur par default (10 documents) fait dépasser la limite Groq
→ Commentaire expliquant pourquoi max_results est réduit.

77 # (8000 tokens/minute en tier gratuit) en un seul appel.
→ Fin du commentaire précisant la limite de débit Groq concernée : les offres gratuites des API d'inférence limitent le nombre de tokens traités par minute, et renvoyer trop de passages récupérés (chunks) dans le contexte du modèle peut à elle seule dépasser ce quota.

78 max_results=3,
→ Limite à 3 le nombre de passages (chunks) les plus pertinents renvoyés par une recherche dans la base, pour rester sous la limite de tokens de Groq tout en gardant le contexte le plus pertinent.

79)
→ Ferme l'appel au constructeur Knowledge.

80 knowledge_base.add_content(path="DocumentMarkdown/ocr_document.md")
→ Ajoute le contenu du fichier markdown OCR à la base de connaissances : en interne, agno découpe ce texte en morceaux (chunking), calcule leur embedding via SentenceTransformerEmbedder, puis stocke le tout dans PgVector — c'est l'étape d'indexation qui rend le document interrogeable par similarité sémantique.

81 return knowledge_base
→ Retourne la base de connaissances construite à l'appelant.

83 def semantic_scholar_search(query):
→ Définit une fonction Python normale qui, du fait de sa signature simple et de son nom explicite, pourra être passée directement à un agent comme "outil" (function tool) : c'est un des mécanismes agno pour transformer n'importe quelle fonction Python en capacité que le LLM peut invoquer lui-même pendant son raisonnement.

84 url = "https://api.semanticscholar.org/graph/v1/paper/search"
→ Définit l'URL de l'endpoint de recherche de papiers de Semantic Scholar, un moteur de recherche académique gratuit qui indexe des millions de publications scientifiques.

```

85     params = {"query": query}
    → Construit le dictionnaire des paramètres de requête HTTP avec la requête fournie.

86     headers = {"x-api-key": "1234567890"}
    → Construit les en-têtes HTTP incluant une clé d'API pour Semantic Scholar (ici une valeur factice/placeholder — l'API Semantic Scholar reste utilisable sans clé valide sur son tier public, avec des limites de débit plus basses).

87     response = requests.get(url, params=params, headers=headers)
    → Envoie une requête GET à l'API Semantic Scholar avec les paramètres et en-têtes définis.

88     return response.json()
    → Retourne le corps de la réponse HTTP décodé en JSON, que l'agent pourra ensuite lire et synthétiser dans sa réponse.

90 def summary_agent():
    → Définit la fonction qui construit et retourne l'agent chargé de résumer le document.

91     knowledge_base = knowledge_base_setup()
    → Construit la base de connaissances RAG à utiliser par l'agent de résumé.

92     summary_agent = Agent(
    → Instancie l'agent de résumé et commence à passer ses arguments.

93         name="Summary Agent",
    → Donne le nom "Summary Agent" à cet agent.

94         model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),
    → Configure l'agent pour utiliser sur Groq le modèle open-weight openai/gpt-oss-20b (20 milliards de paramètres) — un modèle plus petit que les plus gros modèles disponibles, choisi ici pour son quota de tokens par minute plus généreux sur le tier gratuit de Groq plutôt que pour sa capacité brute.

95         instructions=dedent("""
    → Commence à définir les instructions de l'agent sous forme de texte désindenté.

96             You are a summary agent designed to summarize the document.
    → Ligne d'instruction indiquant à l'agent son rôle de résumé de document.

97             Read the entire document stored in the knowledge base (in markdown format) and produce a
concise summary in plain English. Keep it focused on the paper's objective, methodology, and key
findings.
    → Ligne d'instruction détaillant la consigne de lecture et de résumé du document.

98             """),
    → Ferme la chaîne dedent des instructions et l'argument instructions de l'Agent.

99         knowledge=knowledge_base,
    → Associe la base de connaissances RAG construite à l'agent : c'est ce qui lui donne accès, via recherche vectorielle, au contenu du PDF indexé.

100         search_knowledge=True,
    → Active la recherche automatique dans la base de connaissances : l'agent peut lui-même déclencher une requête de récupération (retrieval) dans PgVector avant de répondre, plutôt que de se limiter à ses connaissances générales.

101         # Sans limite, l'agent relance plusieurs recherches pour "tout lire", ce qui
    → Commentaire expliquant pourquoi tool_call_limit est fixé à 1.

102         # accumule le contexte et dépasse la limite Groq (8000 tokens/minute).
    → Fin du commentaire précisant la limite de débit Groq concernée.

103         tool_call_limit=1,
    → Limite l'agent à un seul appel d'outil (donc une seule recherche dans la base) par exécution, pour éviter qu'il ne multiplie les

```

requêtes de récupération et ne dépasse le quota de tokens par minute de Groq.

```
104     )
    → Ferme l'appel au constructeur Agent définissant summary_agent.

105     return summary_agent
    → Retourne l'agent de résumé construit à l'appelant.

107 def agent_setup():
    → Définit la fonction qui construit et retourne l'agent de recherche/questions-réponses principal.

108     knowledge_base = knowledge_base_setup()
    → Construit la base de connaissances RAG à utiliser par l'agent de recherche.

109     research_agent = Agent(
    → Instancie l'agent de recherche et commence à passer ses arguments.

110         name="Research Agent",
    → Donne le nom "Research Agent" à cet agent.

111         model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),
    → Configure l'agent pour utiliser le même modèle Groq openai/gpt-oss-20b que l'agent de résumé, pour les mêmes raisons de quota.

112         instructions=dedent("""
    → Commence à définir les instructions de l'agent sous forme de texte désindenté.

113             You are a research assistant designed to simplify and answer questions about academic papers.
Your behavior follows this logic:
    → Ligne d'instruction décrivant le rôle général de l'agent de recherche.

115             1. Answering Questions:
    → Introduit la première étape du comportement attendu, répondre aux questions.

116                 - When a user asks a question, try to answer it using the internal knowledge base first.
    → Précise de d'abord chercher la réponse dans la base de connaissances interne (RAG) avant tout autre moyen : c'est la stratégie type d'un agent RAG, prioriser les documents de référence fournis pour ancrer la réponse dans une source vérifiable.

117                 - If you find an answer, cite the exact source from the markdown document (quote the relevant sentence or paragraph).
    → Précise de citer la source exacte du document markdown en cas de réponse trouvée, ce qui permet à l'utilisateur de vérifier l'affirmation plutôt que de faire confiance aveuglément au modèle.

118                 - Format the answer clearly and showably: include the answer, the citation snippet, and its section or approximate heading if available.
    → Précise le format attendu de la réponse: réponse, citation, et section approximative.

120             2. Fallback to External Tools:
    → Introduit la deuxième étape du comportement, le recours à des outils externes.

121                 - If the knowledge base doesn't contain enough information to answer:
    → Précise la condition de déclenchement du recours aux outils externes : uniquement si le RAG interne ne suffit pas, pour éviter d'aller chercher sur le web des informations déjà disponibles localement.

122                 - Use `semantic_shcolar_search(query)` to look for external answers.
    → Précise d'utiliser la fonction outil de recherche Semantic Scholar en premier recours externe (le nom contient une faute de frappe dans le prompt d'origine, "shcolar" au lieu de "scholar", ce qui n'empêche pas le modèle de comprendre l'intention).

123                 - If that fails, retry once.
    → Précise de réessayer une fois en cas d'échec de la recherche externe.
```

124 - If it still fails, use ``GoogleSearchTool(query)`` as a backup.
→ Précise d'utiliser un outil de recherche Google comme solution de secours (bien que l'outil réellement fourni à l'agent plus bas soit DuckDuckGoTools, pas un GoogleSearchTool — une incohérence mineure entre le prompt et les outils réellement câblés).

125 - When using external tools, always show the `**source link**` or citation (title, author if possible, and source URL).
→ Précise d'afficher systématiquement la source lors de l'usage d'outils externes.

126 - Keep external answers clean, brief, and properly attributed.
→ Précise de garder les réponses externes claires, brèves et correctement attribuées.

128 3. `**Formatting**`:
→ Introduit la troisième étape du comportement, le formatage des réponses.

129 - Always use clean markdown formatting.
→ Précise d'utiliser toujours un formatage markdown propre.

130 - Highlight the `**source**` of every answer clearly.
→ Précise de toujours mettre en évidence la source de chaque réponse.

131 - Avoid hallucinating—if you don't know, say you'll look it up.
→ Précise d'éviter les hallucinations (le fait qu'un modèle de langage invente une information plausible mais fausse) et d'admettre ne pas savoir le cas échéant plutôt que d'inventer une réponse.

133 Your top priorities are accuracy, traceability (always show the source), and clarity.
→ Ligne d'instruction résumant les priorités de précision, traçabilité et clarté.

134 `"""),`
→ Ferme la chaîne dedent des instructions et l'argument instructions de l'Agent.

135 `knowledge=knowledge_base,`
→ Associe la base de connaissances RAG construite à l'agent de recherche.

136 `search_knowledge=True,`
→ Active la recherche dans la base de connaissances pour cet agent.

137 `tools=[ReasoningTools(add_instructions=True), semantic_scholar_search, DuckDuckGoTools()],`
→ Fournit à l'agent trois outils combinés : ReasoningTools pour structurer son raisonnement, la fonction semantic_scholar_search pour chercher des papiers académiques externes, et DuckDuckGoTools pour une recherche web générale — c'est la mise en pratique concrète de la logique de fallback décrite dans les instructions ci-dessus.

138 `compress_tool_results=True,`
→ Active la compression des résultats d'outils : les réponses brutes des outils (recherche web, base de connaissances) peuvent être volumineuses, cette option les condense avant de les injecter dans le contexte du modèle, ce qui réduit la consommation de tokens et aide à rester sous les limites de débit de Groq.

139 `)`
→ Ferme l'appel au constructeur Agent définissant research_agent.

141 `return research_agent`
→ Retourne l'agent de recherche construit à l'appelant.

143 `if __name__ == "__main__":`
→ Vérifie que le script est exécuté directement et non importé comme module. Dans le cas de Streamlit, le script est en réalité exécuté via la commande ``streamlit run``, qui réexécute tout le fichier de haut en bas à chaque interaction utilisateur.

144 `if "ocr_done" not in st.session_state:`

→ Vérifie si la clé `ocr_done` n'existe pas encore dans `st.session_state`, l'espace de stockage persistant de Streamlit qui survit aux ré-exécutions successives du script (contrairement aux variables Python normales qui seraient réinitialisées à chaque interaction).

```
145         st.session_state.ocr_done = False
```

→ Initialise l'indicateur `ocr_done` à `False` dans l'état de session.

```
147     if "knowledge_base" not in st.session_state:
```

→ Vérifie si la clé `knowledge_base` n'existe pas encore dans l'état de session.

```
148         st.session_state.knowledge_base = None
```

→ Initialise la clé `knowledge_base` à `None` dans l'état de session.

```
150     if "summary_agent" not in st.session_state:
```

→ Vérifie si la clé `summary_agent` n'existe pas encore dans l'état de session.

```
151         st.session_state.summary_agent = None
```

→ Initialise la clé `summary_agent` à `None` dans l'état de session.

```
153     if "agent" not in st.session_state:
```

→ Vérifie si la clé `agent` n'existe pas encore dans l'état de session.

```
154         st.session_state.agent = None
```

→ Initialise la clé `agent` à `None` dans l'état de session.

```
156     st.title("Advanced Research Assistant")
```

→ Affiche le titre principal de la page Streamlit.

```
157     uploaded_file = st.file_uploader("Upload your research paper", type=["pdf"])
```

→ Affiche le composant Streamlit de téléversement de fichier, restreint aux PDF, et récupère l'objet fichier chargé par l'utilisateur (ou `None` si rien n'a encore été téléversé).

```
158     if uploaded_file and not st.session_state.ocr_done:
```

→ Vérifie qu'un fichier a été téléversé et que l'OCR n'a pas déjà été effectué (grâce à l'état de session), pour éviter de relancer inutilement un traitement OCR coûteux à chaque ré-exécution du script.

```
159         with open(f"DocumentMarkdown/{uploaded_file.name}", "wb") as f:
```

→ Ouvre en écriture binaire un nouveau fichier local portant le nom du fichier téléversé.

```
160             f.write(uploaded_file.read())
```

→ Lit le contenu du fichier téléversé (reçu en mémoire par Streamlit) et l'écrit sur le disque, pour que `ocr_pdf()` puisse ensuite l'ouvrir comme un fichier normal.

```
161         with st.spinner("Performing OCR..."):
```

→ Affiche un indicateur de chargement animé (spinner) pendant le traitement OCR, pour informer l'utilisateur qu'une opération potentiellement longue est en cours.

```
162             ocr_pdf(f"DocumentMarkdown/{uploaded_file.name}")
```

→ Appelle la fonction `ocr_pdf` sur le fichier PDF nouvellement enregistré.

```
163         st.success("OCR completed successfully")
```

→ Affiche un message de succès (bandeau vert) une fois l'OCR terminé.

```
164         st.session_state.ocr_done = True
```

→ Marque l'OCR comme terminé dans l'état de session, pour ne pas le relancer aux prochaines ré-exécutions du script.

```
165         st.session_state.summary_agent = summary_agent()
```

→ Construit l'agent de résumé (ce qui construit aussi et indexe sa base de connaissances RAG) et le stocke dans l'état de session pour le réutiliser sans le reconstruire à chaque interaction.

166 `with st.spinner("Generating summary..."):`
→ Affiche un indicateur de chargement pendant la génération du résumé.

167 `summary_response = st.session_state.summary_agent.run("Summarize the document")`
→ Exécute l'agent de résumé avec la consigne de résumer le document : en interne, l'agent va d'abord chercher dans la base de connaissances RAG (recherche vectorielle) les passages pertinents, puis les injecter dans le contexte donné au modèle Groq pour générer la réponse.

168 `st.write(summary_response.content)`
→ Affiche dans l'interface le contenu textuel du résumé généré.

169 `# st.session_state.knowledge_base = knowledge_base_setup()`
→ Ligne de code commentée qui aurait stocké la base de connaissances dans l'état de session (redondant ici car chaque agent reconstruit déjà la sienne dans `knowledge_base_setup()`).

170 `st.session_state.agent = agent_setup()`
→ Construit l'agent de recherche/questions-réponses principal et le stocke dans l'état de session.

172 `if st.session_state.ocr_done:`
→ Vérifie que l'OCR a bien été effectué avant d'afficher la zone de question, pour ne pas permettre de poser une question avant qu'un document ait été indexé.

173 `user_input = st.text_input("Enter a question: ")`
→ Affiche un champ de saisie de texte pour poser une question et récupère la valeur saisie.

174 `if st.button("Submit"):`
→ Affiche un bouton Submit et vérifie s'il a été cliqué lors de cette exécution du script.

175 `with st.spinner('Response is being generated in your terminal'):`
→ Affiche un indicateur de chargement pendant la génération de la réponse.

176 `st.session_state.agent.print_response(`
→ Appelle la méthode `print_response` de l'agent de recherche, qui affiche la réponse directement dans le terminal (console) plutôt que de la retourner comme objet, et commence à passer ses arguments.

177 `user_input,`
→ Passe la question saisie par l'utilisateur comme message à l'agent.

178 `stream=True,`
→ Active le streaming de la réponse : le texte s'affiche au fur et à mesure de sa génération plutôt que d'un bloc une fois terminé.

179 `show_full_reasoning=True,`
→ Active l'affichage du raisonnement complet de l'agent (les étapes internes de réflexion, notamment via ReasoningTools), utile pour comprendre comment il est arrivé à sa réponse.

180 `stream_intermediate_steps=True`
→ Active le streaming des étapes intermédiaires (comme les appels d'outils et leurs résultats) au fur et à mesure qu'elles se produisent, et pas seulement la réponse finale.

181 `)`
→ Ferme l'appel à la méthode `print_response`.

182 `st.success('Response generated successfully')`
→ Affiche un message de succès une fois la réponse générée.

183 `# response = research_agent.run(user_input)`
→ Ligne de code commentée montrant une alternative directe d'appel de l'agent (qui retournerait la réponse comme objet au

lieu de l'imprimer dans le terminal).

184 `# st.write(response)`

→ Ligne de code commentée qui aurait affiché la réponse alternative dans l'interface Streamlit.

187 `## How the pipeline-based approach decomposes the document parsing workflow`

→ Ligne de titre isolée en fin de fichier, amorce d'une note ou section de documentation non développée par l'auteur du script.

Projet 33 — 33-FreightAgent

Estimation de couts de fret (UPS/FedEx/DHL) via WhatsApp

Objectif

Bot conversationnel joignable sur WhatsApp qui estime le cout d'expédition d'un colis en interrogeant en temps réel les API de plusieurs transporteurs à partir des dimensions, du poids et de la destination fournis par l'utilisateur, puis renvoie une comparaison des tarifs directement sur WhatsApp.

Code source, ligne par ligne

33-FreightAgent/freight_agent.py

Ce fichier configure un agent de fret (Freight Agent) qui effectue l'OCR de guides tarifaires PDF via Mistral, indexe leur contenu dans une base de connaissances vectorielle LanceDB pour faire du RAG, et répond aux demandes d'estimation de coûts d'expédition en s'appuyant sur un modèle Groq et une recherche web DuckDuckGo.

177 lignes de code annotées ci-dessous.

```
1 import os
  → Importe le module standard os pour accéder aux variables d'environnement et manipuler des chemins de fichiers.

3 # sentence-transformers (utilisé pour les embeddings locaux) importe transformers,
  → Commentaire expliquant pourquoi la variable d'environnement suivante est nécessaire.

4 # qui essaie par défaut de charger TensorFlow en plus de PyTorch. La version de
  → Suite du commentaire explicatif sur le conflit avec TensorFlow.

5 # TensorFlow installée sur cette machine est incompatible avec la version de
  → Suite du commentaire explicatif sur l'incompatibilité de version.

6 # protobuf présente : on désactive TF avant tout import de la lib.
  → Fin du commentaire expliquant la désactivation de TensorFlow avant les imports.

7 os.environ.setdefault("USE_TF", "0")
  → Définit la variable d'environnement USE_TF à "0" si elle n'est pas déjà définie. La librairie sentence-transformers (utilisée plus bas pour générer les embeddings) s'appuie sur transformers, qui essaie par défaut de charger TensorFlow en plus de PyTorch ; cette ligne force l'usage exclusif de PyTorch pour éviter un conflit de versions avec protobuf.

9 from textwrap import dedent
  → Importe dedent pour retirer l'indentation commune des chaînes de texte multi-lignes, utilisé pour écrire proprement les instructions de l'agent dans le code.

10 from agno.agent import Agent
  → Importe la classe Agent du framework agno, une bibliothèque de construction d'agents IA (modèle de langage + instructions + outils + base de connaissances).

11 from mistralai.client import Mistral
  → Importe le client officiel de l'API Mistral, utilisé ici pour son service d'OCR qui convertit les PDF de grilles tarifaires en texte Markdown exploitable.

12 from agno.models.groq import Groq
  → Importe la classe qui encapsule un modèle de chat hébergé sur Groq, une plateforme d'inférence connue pour sa rapidité et
```

son tier gratuit soumis à des limites de débit (tokens par minute), ce qui justifie plusieurs choix de configuration plus bas.

13 from agno.vectordb.lancedb import LanceDb, SearchType

→ Importe LanceDb, la classe qui permet d'utiliser LanceDB comme base de données vectorielle, ainsi que l'énumération SearchType qui définit les modes de recherche disponibles. Contrairement à une base vectorielle qui nécessiterait un serveur séparé, LanceDB est une base vectorielle embarquée : elle stocke ses données directement dans des fichiers locaux sur le disque, sans serveur à installer ni administrer, ce qui la rend pratique pour un projet autonome.

14 from agno.tools.reasoning import ReasoningTools

→ Importe l'outil de raisonnement structuré fourni par agno, qui donne à l'agent la capacité de décomposer explicitement son raisonnement en étapes avant de répondre.

15 from ddgs import DDGS

→ Importe la classe DDGS de la librairie ddgs (anciennement duckduckgo-search), qui permet d'effectuer des recherches web via DuckDuckGo par programmation, sans nécessiter de clé API.

16 from agno.knowledge.knowledge import Knowledge

→ Importe la classe Knowledge, l'abstraction agno représentant une base de connaissances consultable par un agent. C'est le composant central du RAG (Retrieval-Augmented Generation, "génération augmentée par récupération") : au lieu de répondre uniquement à partir de ce que le modèle a appris pendant son entraînement, l'agent va chercher dans cette base les passages pertinents des grilles tarifaires indexées et les utilise comme contexte pour générer une réponse ancrée dans les documents réels.

17 from agno.knowledge.embedder.sentence_transformer import SentenceTransformerEmbedder

→ Importe l'embedder basé sur la librairie open-source sentence-transformers, exécuté localement. Un embedder transforme un texte en vecteur numérique capturant son sens, ce qui permet de retrouver les passages les plus proches sémantiquement d'une question par calcul de similarité entre vecteurs plutôt que par simple recherche de mots-clés.

18 from dotenv import load_dotenv

→ Importe la fonction qui charge les variables d'environnement depuis un fichier .env.

19 load_dotenv()

→ Charge les variables définies dans le fichier .env dans l'environnement du processus.

21 # Dossier du script : les chemins sont résolus par rapport à lui, pas au

→ Commentaire expliquant que les chemins sont relatifs au dossier du script.

22 # répertoire courant, pour que le script fonctionne peu importe d'où il est lancé.

→ Fin du commentaire justifiant l'usage de chemins basés sur le script.

23 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

→ Calcule le chemin absolu du dossier contenant ce script, en résolvant d'abord le chemin absolu du fichier courant (__file__) puis en prenant son dossier parent : cette technique rend les chemins utilisés dans le script indépendants du répertoire de travail depuis lequel il est lancé.

24 MARKDOWN_DIR = os.path.join(SCRIPT_DIR, "Markdown")

→ Construit le chemin du sous-dossier Markdown relatif au dossier du script, où seront stockés les documents convertis par OCR.

25 DOCUMENT_DIR = os.path.join(SCRIPT_DIR, "Document")

→ Construit le chemin du sous-dossier Document relatif au dossier du script, où sont attendus les PDF sources à traiter.

26 LANCEDB_URI = os.path.join(SCRIPT_DIR, "tmp", "lancedb")

→ Construit le chemin de stockage local de la base vectorielle LanceDB : puisque LanceDB est embarquée, il s'agit simplement d'un dossier sur le disque plutôt que d'une URL de connexion réseau.

28 mistral_key = os.getenv("MISTRAL_API_KEY")

→ Récupère la clé API Mistral depuis les variables d'environnement, nécessaire pour l'OCR.

29 # Le script original utilisait OpenAI (chat + embeddings) ; remplacé par Groq
→ Commentaire expliquant le remplacement du modèle OpenAI par Groq.

30 # pour le chat (clé GROQ_API_KEY déjà disponible) et par l'embedder Mistral
→ Suite du commentaire expliquant le choix d'un autre embedder.

31 # pour les embeddings (Groq n'a pas d'API d'embeddings).
→ Fin du commentaire justifiant pourquoi Groq n'est pas utilisé pour les embeddings : Groq est une plateforme d'inférence pour modèles de chat/texte, elle n'expose pas de modèle d'embedding, d'où le recours à un embedder local séparé (sentence-transformers) pour cette partie du pipeline.

32 groq_key = os.getenv("GROQ_API_KEY")
→ Récupère la clé API Groq depuis les variables d'environnement.

34 if mistral_key:
→ Vérifie si la clé API Mistral est présente.

35 client = Mistral(api_key=mistral_key)
→ Crée l'instance du client Mistral authentifiée, utilisée ensuite pour lancer l'OCR des PDF.

36 else:
→ Démarre le cas où la clé API Mistral est absente.

37 print("ERREUR: MISTRAL_API_KEY requis pour OCR")
→ Affiche un message d'erreur signalant l'absence de la clé Mistral requise pour l'OCR.

38 exit()
→ Termine immédiatement le programme si la clé Mistral est absente, pour éviter un plantage plus tard avec un message d'erreur moins clair.

40 if not groq_key:
→ Vérifie si la clé API Groq est absente.

41 print("ERREUR: GROQ_API_KEY requis")
→ Affiche un message d'erreur signalant l'absence de la clé Groq requise.

42 exit()
→ Termine immédiatement le programme si la clé Groq est absente.

44 ## OCR Function améliorée ###
→ Commentaire marquant le début de la fonction OCR améliorée.

45 def ocr_pdf(pdf_path, output_name=None):
→ Définit la fonction ocr_pdf prenant le chemin du PDF et un nom de sortie optionnel (permettant de personnaliser le nom du fichier Markdown généré).

46 """Convertit un PDF en Markdown via OCR Mistral"""
→ Docstring décrivant le rôle de la fonction, convertir un PDF en Markdown via OCR Mistral.

47 if not os.path.exists(pdf_path):
→ Vérifie que le fichier PDF spécifié existe sur le disque.

48 print(f"ERREUR: PDF non trouvé - {pdf_path}")
→ Affiche un message d'erreur indiquant le chemin du PDF introuvable.

49 return False
→ Retourne False pour signaler l'échec si le fichier PDF n'existe pas, permettant à l'appelant (process_all_pdfs) de compter les échecs.

51 print(f"Traitement OCR de {pdf_path}...")

→ Affiche dans la console le chemin du PDF en cours de traitement.

53 `try:`

→ Démarre un bloc try/except pour capturer proprement les erreurs éventuelles du traitement OCR (appel réseau externe pouvant échouer).

54 `# Upload du PDF`

→ Commentaire annonçant l'upload du fichier PDF.

55 `uploaded_pdf = client.files.upload(`

→ Envoie le fichier PDF au service Mistral : l'API OCR fonctionne en deux temps (upload puis traitement), il faut d'abord obtenir une référence exploitable côté serveur.

56 `file={`

→ Ouvre le dictionnaire décrivant le fichier à uploader.

57 `"file_name": os.path.basename(pdf_path),`

→ Précise le nom de fichier à utiliser pour l'upload, réduit au nom de base (sans le chemin du dossier) via `os.path.basename`.

58 `"content": open(pdf_path, "rb"),`

→ Ouvre le fichier PDF en mode binaire pour fournir son contenu à l'upload.

59 `},`

→ Ferme le dictionnaire décrivant le fichier.

60 `purpose="ocr"`

→ Indique que le fichier est uploadé dans le but de faire de l'OCR.

61 `)`

→ Ferme l'appel à la méthode `files.upload`.

63 `# Obtenir URL signée`

→ Commentaire annonçant la récupération d'une URL signée.

64 `signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)`

→ Récupère une URL signée temporaire permettant à l'API OCR d'accéder au fichier uploadé.

66 `# Traitement OCR`

→ Commentaire annonçant le lancement du traitement OCR proprement dit.

67 `ocr_response = client.ocr.process(`

→ Lance le traitement OCR du document et commence à passer ses arguments.

68 `model="mistral-ocr-latest",`

→ Spécifie le modèle OCR à utiliser, la dernière version du modèle spécialisé `mistral-ocr` capable de convertir un PDF (texte et mise en page) en Markdown structuré.

69 `document={`

→ Ouvre le dictionnaire décrivant le document à traiter.

70 `"type": "document_url",`

→ Précise que le document est référencé par une URL.

71 `"document_url": signed_url.url,`

→ Fournit l'URL signée du document à traiter.

72 `},`

→ Ferme le dictionnaire décrivant le document.

```

73         include_image_base64=True
    → Demande d'inclure les images extraites (tableaux tarifaires, graphiques) encodées en base64 dans la réponse.

74     )
    → Ferme l'appel à la méthode ocr.process.

76     # Générer nom de sortie
    → Commentaire annonçant la détermination du nom de fichier de sortie.

77     if output_name is None:
    → Vérifie si aucun nom de sortie n'a été fourni par l'appelant.

78     base_name = os.path.splitext(os.path.basename(pdf_path))[0]
    → Extrait le nom de base du PDF sans son extension, en combinant basename (retire le chemin) et splitext (sépare nom et extension).

79     output_name = f"{base_name}.md"
    → Construit le nom de sortie en ajoutant l'extension .md au nom de base.

81     # Sauvegarder en Markdown
    → Commentaire annonçant la sauvegarde du résultat au format Markdown.

82     os.makedirs(MARKDOWN_DIR, exist_ok=True)
    → Crée le dossier Markdown s'il n'existe pas déjà ; exist_ok=True évite qu'une erreur soit levée si le dossier existe déjà (utile si la fonction est appelée plusieurs fois).

83     output_path = os.path.join(MARKDOWN_DIR, output_name)
    → Construit le chemin complet du fichier Markdown de sortie.

85     with open(output_path, "w", encoding="utf-8") as f:
    → Ouvre en écriture le fichier de sortie avec l'encodage utf-8, via un gestionnaire de contexte garantissant sa fermeture propre.

86     f.write("\n".join([page.markdown for page in ocr_response.pages]))
    → Concatène le Markdown de chaque page renvoyée par l'OCR (une page à la fois) et l'écrit dans le fichier, reconstituant le document complet.

88     print(f"SUCCESS: {output_path} créé ({len(ocr_response.pages)} pages)")
    → Affiche un message de succès précisant le chemin créé et le nombre de pages traitées.

89     return True
    → Retourne True pour signaler le succès du traitement OCR.

91     except Exception as e:
    → Capture toute exception survenue pendant le traitement OCR (échec réseau, quota, fichier corrompu, etc.).

92     print(f"ERREUR OCR {pdf_path}: {e}")
    → Affiche un message d'erreur précisant le PDF concerné et l'exception rencontrée.

93     return False
    → Retourne False pour signaler l'échec du traitement OCR.

94 class DDGSTools:
    → Définit une classe wrapper autour de DDGS : encapsuler la recherche DuckDuckGo dans une classe avec sa propre gestion d'erreur permet de la fournir à l'agent comme un outil robuste (qui ne fait pas planter l'agent si la recherche échoue), plutôt que d'exposer directement l'appel brut à la librairie.

95     def __init__(self):
    → Définit le constructeur de la classe DDGSTools.

```

```

96         self.ddgs = DDGS()
    → Crée et stocke une instance du client DDGS sur l'objet, réutilisée à chaque appel de recherche.

98     def search(self, query, max_results=5):
    → Définit la méthode search prenant une requête et un nombre maximal de résultats par défaut à 5.

99         try:
    → Démarre un bloc try pour capturer les erreurs éventuelles de la recherche.

100             results = list(self.ddgs.text(query, max_results=max_results))
    → Effectue une recherche textuelle DuckDuckGo (méthode .text) et convertit le générateur de résultats renvoyé en liste concrète.

101             return results
    → Retourne la liste des résultats de recherche obtenus.

102         except Exception as e:
    → Capture toute exception survenue pendant la recherche DuckDuckGo (par exemple un blocage temporaire ou un problème réseau).

103             print(f"Erreur recherche: {e}")
    → Affiche un message d'erreur précisant l'exception rencontrée.

104             return []
    → Retourne une liste vide en cas d'échec de la recherche, pour que l'agent reçoive un résultat exploitable (absence de résultats) plutôt qu'une exception qui interromprait son exécution.

106 # Dans votre agent
    → Commentaire indiquant que l'instance suivante sera utilisée par l'agent.

107 ddgs_tool = DDGSTools()
    → Crée une instance de DDGSTools qui sera fournie comme outil à l'agent de fret plus bas dans le fichier.

109 ## Traitement de tous vos PDFs ###
    → Commentaire marquant le début de la fonction de traitement en masse des PDF.

110 def process_all_pdfs():
    → Définit la fonction qui traite tous les PDF du dossier Document.

111     """Traite tous les PDFs du dossier Document/"""
    → Docstring décrivant le rôle de la fonction, traiter tous les PDF du dossier Document.

112     document_folder = DOCUMENT_DIR
    → Assigne le chemin du dossier Document à la variable locale document_folder.

114     if not os.path.exists(document_folder):
    → Vérifie que le dossier Document existe sur le disque.

115         print(f"ERREUR: Dossier {document_folder} non trouvé")
    → Affiche un message d'erreur indiquant que le dossier Document est introuvable.

116         return
    → Interrompt la fonction si le dossier Document n'existe pas.

118     # Lister tous les PDFs
    → Commentaire annonçant le listage des fichiers PDF du dossier.

119     pdf_files = [f for f in os.listdir(document_folder) if f.lower().endswith('.pdf')]

```


→ Construit, par compréhension de liste, l'ensemble des noms de fichiers du dossier dont le nom se termine par .pdf (comparaison insensible à la casse grâce à .lower()).

```
121     if not pdf_files:
```

→ Vérifie si aucun fichier PDF n'a été trouvé dans le dossier.

```
122         print("Aucun PDF trouvé dans Document/")
```

→ Affiche un message indiquant qu'aucun PDF n'a été trouvé.

```
123         return
```

→ Interrompt la fonction si aucun fichier PDF n'a été trouvé.

```
125     print(f"PDFs trouvés: {pdf_files}")
```

→ Affiche la liste des fichiers PDF trouvés dans la console.

```
127     success_count = 0
```

→ Initialise à zéro le compteur de traitements OCR réussis.

```
128     for pdf_file in pdf_files:
```

→ Itère sur chaque nom de fichier PDF trouvé.

```
129         pdf_path = os.path.join(document_folder, pdf_file)
```

→ Construit le chemin complet du fichier PDF courant.

```
130         if ocr_pdf(pdf_path):
```

→ Appelle ocr_pdf sur le fichier courant et vérifie si le traitement a réussi (grâce à la valeur booléenne renvoyée).

```
131         success_count += 1
```

→ Incrmente le compteur de succès si le traitement OCR a réussi.

```
133     print(f"\nTraitement terminé: {success_count}/{len(pdf_files)} PDFs convertis")
```

→ Affiche le bilan du nombre de PDF convertis avec succès sur le total.

```
135     # Lister les Markdown créés
```

→ Commentaire annonçant le listage des fichiers Markdown générés.

```
136     if os.path.exists(MARKDOWN_DIR):
```

→ Vérifie que le dossier Markdown existe avant de le lister.

```
137     md_files = [f for f in os.listdir(MARKDOWN_DIR) if f.endswith('.md')]
```

→ Construit la liste des fichiers du dossier Markdown se terminant par .md.

```
138     print(f"Fichiers Markdown créés: {md_files}")
```

→ Affiche la liste des fichiers Markdown créés dans la console.

```
140 ## Configuration base de connaissances ###
```

→ Commentaire marquant le début de la configuration de la base de connaissances.

```
141 # MarkdownKnowledgeBase(path=..., vector_db=...) a été remplacé par la classe
```

→ Commentaire expliquant le remplacement de l'ancienne classe par Knowledge.

```
142 # unifiée Knowledge : le contenu se charge via add_content(path=...) plutôt
```

→ Suite du commentaire expliquant le nouveau mode de chargement du contenu.

```
143 # qu'un argument du constructeur + .load(recreate=...).
```

→ Fin du commentaire comparant l'ancien et le nouveau mode de chargement.

```
144 knowledge_base = Knowledge(
```

→ Instancie la base de connaissances RAG globale (au niveau module, donc construite une seule fois à l'import du fichier) et

commence à passer ses arguments.

145 `vector_db=LanceDb(`

→ Configure la base vectorielle utilisée et commence à instancier LanceDb.

146 `table_name="freight_documents",`

→ Spécifie le nom de la table LanceDB qui stockera les vecteurs et le texte des documents de fret.

147 `uri=LANCEDB_URI,`

→ Spécifie le chemin de stockage local de la base LanceDB défini plus haut.

148 `search_type=SearchType.hybrid,`

→ Configure le type de recherche vectorielle sur hybride : la recherche hybride combine la recherche sémantique (par similarité de vecteurs, qui capture le sens) et la recherche par mots-clés classique (type BM25, qui capture les termes exacts comme des codes tarifaires ou des noms de transporteur), ce qui donne généralement de meilleurs résultats qu'une seule des deux approches, notamment sur des documents très structurés comme des grilles tarifaires.

149 `# L'API d'embeddings gratuite de Mistral est trop vite rate-limitée pour`

→ Commentaire expliquant pourquoi un embedder local est utilisé plutôt que celui de Mistral.

150 `# indexer plusieurs documents d'un coup ; embedder local à la place.`

→ Fin du commentaire justifiant le choix de l'embedder local.

151 `embedder=SentenceTransformerEmbedder(),`

→ Configure l'embedder local sentence-transformers pour générer les vecteurs, sans dépendre d'une API externe soumise à des limites de débit lors de l'indexation de plusieurs documents.

152 `),`

→ Ferme l'instanciation de LanceDb.

153 `# La valeur par défaut (10 documents) fait dépasser la limite Groq (8000`

→ Commentaire expliquant pourquoi max_results est réduit.

154 `# tokens/minute en tier gratuit) en un seul appel.`

→ Fin du commentaire précisant la limite de débit Groq concernée.

155 `max_results=3,`

→ Limite à 3 le nombre de passages les plus pertinents renvoyés par une recherche dans la base, pour que le contexte transmis au modèle reste sous la limite de tokens par minute du tier gratuit de Groq.

156 `)`

→ Ferme l'appel au constructeur Knowledge.

157 `if os.path.exists(MARKDOWN_DIR) and any(f.endswith(".md") for f in os.listdir(MARKDOWN_DIR)):`

→ Vérifie que le dossier Markdown existe et contient au moins un fichier .md, avant de tenter d'indexer quoi que ce soit.

158 `knowledge_base.add_content(path=MARKDOWN_DIR)`

→ Ajoute le contenu de tous les fichiers du dossier Markdown à la base de connaissances : cette étape déclenche le découpage en chunks, le calcul des embeddings et leur stockage dans LanceDB, rendant les grilles tarifaires interrogeables par recherche sémantique/hybride.

160 `freight_agent = Agent(`

→ Instancie l'agent de fret global (au niveau module) et commence à passer ses arguments.

161 `name="Freight Agent",`

→ Donne le nom "Freight Agent" à cet agent.

162 `# Le contexte (base de connaissances + ReasoningTools) dépasse régulièrement la`

→ Commentaire expliquant pourquoi le modèle 20B est choisi plutôt que le 120B.

163 # limite Groq du modèle 120B (8000 tokens/minute) ; le modèle 20B a un quota plus
→ Suite du commentaire comparant les quotas des deux modèles.

164 # élevé sur le tier gratuit.
→ Fin du commentaire justifiant le choix du modèle 20B.

165 model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),
→ Configure l'agent pour utiliser sur Groq le modèle open-weight openai/gpt-oss-20b (20 milliards de paramètres), choisi plutôt qu'un modèle plus gros (120B) car son quota de tokens par minute est plus généreux sur le tier gratuit, ce qui compte davantage ici que sa capacité de raisonnement brute vu le contexte volumineux (base de connaissances + outils de raisonnement) que l'agent manipule.

166 instructions=dedent("""
→ Commence à définir les instructions système de l'agent, désindentées avec dedent.

167 You are a Freight Cost Estimator Agent that helps small import/export businesses estimate and
compare shipping costs and timelines across major carriers.
→ Ligne d'instruction décrivant le rôle général de l'agent d'estimation de coûts de fret.

169 You have access to:
→ Ligne d'instruction introduisant la liste des ressources disponibles pour l'agent.

170 - A knowledge base built from 2024 rate guides and surcharge tables for UPS, FedEx, and DHL
(converted from PDF via OCR)
→ Précise que la base de connaissances contient les grilles tarifaires 2024 d'UPS, FedEx et DHL, converties en Markdown par OCR puis indexées dans LanceDB.

171 - DDGS search for live data and missing information
→ Précise que l'agent dispose de la recherche DDGS pour les données manquantes ou en temps réel, en complément de sa base de connaissances figée.

173 When analyzing shipping requests, search your knowledge base FIRST for official rates, then
supplement with web search if needed.
→ Précise l'ordre de priorité: consulter d'abord la base de connaissances (RAG, source fiable et à jour au moment de l'indexation) avant de recourir à la recherche web, moins fiable mais plus à jour.

175 Always provide detailed cost breakdowns, delivery estimates, and clear recommendations based on
the specific product dimensions and requirements.
→ Précise que l'agent doit toujours fournir un détail des coûts, des délais et des recommandations claires.

176 """),
→ Ferme la chaîne dedent des instructions et l'argument instructions de l'Agent.

177 tools=[ReasoningTools(add_instructions=True), ddgs_tool],
→ Fournit à l'agent deux outils : ReasoningTools pour structurer son raisonnement en étapes explicites, et l'instance ddgs_tool (l'objet DDGSTools créé plus haut) qui lui permet d'effectuer des recherches web DuckDuckGo quand la base de connaissances ne suffit pas.

178 knowledge=knowledge_base,
→ Associe la base de connaissances RAG globale à l'agent de fret.

179 search_knowledge=True,
→ Active la recherche automatique dans la base de connaissances pour cet agent.

180 compress_tool_results=True,
→ Active la compression des résultats d'outils pour réduire la quantité de tokens consommés par les réponses d'outils (recherche web, base de connaissances), et ainsi mieux respecter la limite de débit du tier gratuit de Groq.

181)

→ Ferme l'appel au constructeur Agent définissant freight_agent.

183 def load_knowledge_base():

→ Définit une fonction qui vérifie et signale l'état de la base de connaissances (le chargement effectif ayant déjà eu lieu plus haut, au niveau module).

184 """Le contenu est déjà chargé à la construction de knowledge_base via add_content()."""

→ Docstring précisant que le chargement effectif a déjà eu lieu à la construction du module, cette fonction ne fait donc qu'un contrôle de cohérence a posteriori.

185 if not (os.path.exists(MARKDOWN_DIR) and any(f.endswith('.md') for f in os.listdir(MARKDOWN_DIR))):

→ Vérifie si le dossier Markdown est absent ou ne contient aucun fichier .md.

186 print("Aucun fichier Markdown trouvé - utilisation de DDGS uniquement")

→ Affiche un avertissement indiquant que, faute de documents indexés, l'agent devra s'appuyer uniquement sur la recherche web DDGS plutôt que sur son RAG interne.

188 def full_response(message):

→ Définit la fonction qui exécute l'agent de fret sur un message et retourne sa réponse : c'est le point d'entrée utilisé par les autres modules du projet (comme whatsapp_utils.py) pour obtenir une estimation de fret.

189 print('-----Freight Agent Triggered-----')

→ Affiche dans la console un marqueur signalant le déclenchement de l'agent de fret.

190 try:

→ Démarre un bloc try pour capturer les erreurs éventuelles de l'exécution de l'agent.

191 response = freight_agent.run(message)

→ Exécute l'agent de fret avec le message fourni : en interne, l'agent consulte sa base de connaissances (RAG) et éventuellement DDGS avant de générer sa réponse.

192 return response.content

→ Retourne le contenu textuel de la réponse de l'agent.

193 except Exception as e:

→ Capture toute exception survenue pendant l'exécution de l'agent.

194 print(f"Erreur: {e}")

→ Affiche dans la console le message de l'exception rencontrée.

195 return f"Erreur lors du traitement: {str(e)}"

→ Retourne un message d'erreur formaté à l'appelant en cas d'échec, pour que le programme appelant puisse tout de même transmettre une réponse (par exemple à l'utilisateur WhatsApp) plutôt que de planter.

197 if __name__ == '__main__':

→ Vérifie que le script est exécuté directement et non importé comme module.

198 print("=== FREIGHT AGENT OCR SETUP ===")

→ Affiche un en-tête signalant le démarrage de la configuration OCR de l'agent de fret.

200 # Étape 1: Traiter les PDFs (décommentez pour exécuter)

→ Commentaire indiquant l'étape optionnelle de traitement des PDF.

201 #process_all_pdfs()

→ Ligne de code commentée qui, décommentée, lancerait le traitement OCR de tous les PDF du dossier Document (à exécuter une seule fois, ou quand de nouveaux tarifs sont ajoutés).

202 #exit() # Arrête après traitement OCR

→ Ligne de code commentée qui, décommentée, arrêterait le script juste après l'OCR, sans lancer le test de l'agent qui suit.

204 # Étape 2: Charger la base de connaissances

→ Commentaire annonçant l'étape de chargement de la base de connaissances.

205 load_knowledge_base()

→ Appelle la fonction qui vérifie et signale l'état de la base de connaissances.

207 # Test

→ Commentaire annonçant le lancement d'un test manuel de l'agent.

208 test_message = """

→ Commence à définir une chaîne multi-ligne de message de test.

209 User Question: Ship from Chicago, IL to Miami, FL

→ Ligne du message de test précisant l'origine et la destination de l'expédition.

210 Product Specifications: Machine parts, Weight: 300 lbs, Dimensions: 36"x24"x18", Industrial equipment

→ Ligne du message de test précisant les caractéristiques du produit à expédier.

211 """

→ Ferme la chaîne multi-ligne assignée à test_message.

213 print("\nTest agent...")

→ Affiche un message annonçant le lancement du test de l'agent.

214 result = full_response(test_message)

→ Appelle full_response avec le message de test et récupère le résultat, permettant de vérifier rapidement que le pipeline complet (RAG + Groq + DDGS) fonctionne en exécutant directement ce fichier.

215 print(f"\nRésultat:\n{result}")

→ Affiche le résultat retourné par l'agent dans la console.

33-FreightAgent/main.py

Ce fichier expose, via le framework web FastAPI, un webhook WhatsApp Business (vérification et réception de messages) ainsi que des routes de test pour l'envoi manuel de messages WhatsApp.

107 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard os pour accéder aux variables d'environnement (jetons et identifiants WhatsApp).

2 import json

→ Importe le module standard json, utilisé ici pour capturer l'exception JSONDecodeError levée quand le corps d'une requête entrante n'est pas un JSON valide.

3 import uvicorn

→ Importe uvicorn, un serveur ASGI (Asynchronous Server Gateway Interface) haute performance : c'est le programme qui écoute réellement les connexions réseau et exécute l'application FastAPI en réponse à chaque requête HTTP reçue.

4 import logging

→ Importe le module standard logging, utilisé pour journaliser les événements du webhook (vérifications, erreurs) de façon structurée plutôt qu'avec de simples print().

5 import requests

→ Importe la librairie requests pour effectuer des appels HTTP sortants vers l'API Graph de WhatsApp (envoi de messages).

6 from threading import Thread

→ Importe la classe Thread du module standard threading, qui permet d'exécuter une fonction en parallèle, dans un fil d'exécution séparé, sans bloquer le thread principal du serveur web.

7 from dotenv import load_dotenv

→ Importe la fonction qui charge les variables d'environnement depuis un fichier .env.

8 from fastapi import FastAPI, Request, HTTPException

→ Importe les classes centrales de FastAPI, un framework web Python moderne et asynchrone, largement utilisé pour construire des API HTTP : FastAPI est la classe de l'application, Request représente une requête HTTP entrante, et HTTPException permet de lever proprement une erreur avec un code de statut HTTP donné.

9 from fastapi.responses import PlainTextResponse, JSONResponse

→ Importe les classes de réponse PlainTextResponse (texte brut, requis par WhatsApp pour la vérification du webhook) et JSONResponse (réponses structurées en JSON) de FastAPI.

10 from whatsapp_utils import process_whatsapp_message, is_valid_whatsapp_message

→ Importe les fonctions utilitaires de traitement et de validation des messages WhatsApp, définies dans le module voisin whatsapp_utils.py.

12 load_dotenv()

→ Charge les variables définies dans le fichier .env dans l'environnement du processus.

14 VERSION = os.getenv('VERSION')

→ Récupère la version de l'API Graph WhatsApp (ex: "v18.0") depuis les variables d'environnement, utilisée pour construire les URL d'appel à l'API Meta.

15 PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')

→ Récupère l'identifiant du numéro de téléphone WhatsApp Business associé à l'application, depuis les variables d'environnement.

16 WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')

→ Récupère le jeton d'accès permettant d'authentifier les appels à l'API Graph de WhatsApp/Meta.

17 RECIPIENT_PHONE_NUMBER = os.getenv('RECIPIENT_PHONE_NUMBER')

→ Récupère un numéro de téléphone destinataire par défaut, utilisé par les routes de test ci-dessous.

18 VERIFY_TOKEN = os.getenv('VERIFY_TOKEN')

→ Récupère le jeton de vérification du webhook, une valeur secrète choisie par le développeur et configurée aussi côté Meta, qui sert à prouver que la requête de vérification provient bien de Meta.

20 app = FastAPI()

→ Crée l'instance de l'application FastAPI, l'objet central autour duquel toutes les routes (endpoints) du serveur sont déclarées.

22 @app.get("/")

→ Déclare, via un décorateur, une route HTTP GET sur le chemin racine de l'application. FastAPI utilise ce système de décorateurs pour associer une fonction Python à une URL et une méthode HTTP.

23 def read_root():

→ Définit la fonction de gestion de la route racine.

24 return {"message": "Welcome to the portal!"}

→ Retourne un dictionnaire Python, que FastAPI convertit automatiquement en réponse JSON pour la route racine (souvent utilisée comme simple test que le serveur répond).

26 # Required webhook verification for WhatsApp

→ Commentaire indiquant que la fonction suivante gère la vérification obligatoire du webhook WhatsApp.

27 `async def verify(request: Request):`

→ Définit la fonction asynchrone `verify` (mot-clé `async`, propre à la programmation asynchrone que permet FastAPI/ASGI, pour ne pas bloquer le serveur pendant les opérations d'E/S) prenant la requête HTTP en paramètre.

28 `params = dict(request.query_params)`

→ Convertit les paramètres de requête de l'URL (query string) en dictionnaire Python standard.

29 `mode = params.get("hub.mode")`

→ Récupère le paramètre `hub.mode` envoyé par WhatsApp pour la vérification. C'est Meta qui appelle cette route (webhook) pour vérifier que le serveur est bien celui attendu, selon un protocole de handshake standardisé.

30 `token = params.get("hub.verify_token")`

→ Récupère le paramètre `hub.verify_token` envoyé par WhatsApp pour la vérification, à comparer avec le `VERIFY_TOKEN` secret configuré côté serveur.

31 `challenge = params.get("hub.challenge")`

→ Récupère le paramètre `hub.challenge`, une valeur aléatoire que le webhook doit renvoyer telle quelle pour prouver qu'il contrôle bien le serveur.

33 `if mode and token:`

→ Vérifie que les paramètres `mode` et `token` sont bien présents dans la requête.

34 `if mode == "subscribe" and token == VERIFY_TOKEN:`

→ Vérifie que le mode est "subscribe" et que le token correspond exactement au jeton de vérification secret attendu.

35 `logging.info("WEBHOOK_VERIFIED")`

→ Journalise en niveau info que le webhook a été vérifié avec succès.

36 `return PlainTextResponse(content=challenge, status_code=200)`

→ Renvoie la valeur du challenge en texte brut avec un code de statut 200, exactement comme l'exige le protocole de vérification de webhook de Meta pour valider l'abonnement.

37 `else:`

→ Démarre le cas où le mode ou le token ne correspondent pas aux valeurs attendues.

38 `logging.warning("VERIFICATION_FAILED")`

→ Journalise en niveau avertissement l'échec de la vérification du webhook.

39 `raise HTTPException(status_code=403, detail="Verification failed")`

→ Lève une exception HTTP 403 (Forbidden) signalant l'échec de la vérification ; FastAPI la convertit automatiquement en réponse HTTP appropriée.

40 `else:`

→ Démarre le cas où le mode ou le token sont absents de la requête.

41 `logging.warning("MISSING_PARAMETER")`

→ Journalise en niveau avertissement l'absence de paramètres requis.

42 `raise HTTPException(status_code=400, detail="Missing parameters")`

→ Lève une exception HTTP 400 (Bad Request) signalant des paramètres manquants.

44 `async def handle_message(request: Request):`

→ Définit la fonction asynchrone qui traite le corps d'une requête webhook entrante (les messages WhatsApp reçus, envoyés par Meta en POST).

45 `try:`

→ Démarre un bloc try pour capturer les erreurs de décodage du corps JSON.

```

46         body = await request.json()
    → Attend (await, car la lecture du corps de requête est une opération asynchrone) et décode le corps de la requête HTTP au
    format JSON.

47     except json.JSONDecodeError:
    → Capture l'exception levée si le corps de la requête n'est pas un JSON valide.

48         logging.error("Failed to decode JSON")
    → Journalise en niveau erreur l'échec du décodage JSON.

49         return JSONResponse(status_code=400, content={"status": "error", "message": "Invalid JSON
provided"})
    → Retourne une réponse JSON d'erreur avec un code de statut 400.

51     if (
    → Commence une condition composée qui vérifie l'accès imbriqué à la structure du corps de la requête WhatsApp (qui suit un
    format d'enveloppe imbriqué standardisé par Meta).

52         body.get("entry", [{}])[0]
    → Accède au premier élément de la liste entry, ou à un dictionnaire vide par défaut si la clé est absente : ce motif ".get(clé,
    [{}])[0]" est une façon défensive d'accéder à une structure JSON imbriquée sans lever d'exception si un niveau manque.

53         .get("changes", [{}])[0]
    → Accède au premier élément de la liste changes de cet entry, ou à un dictionnaire vide par défaut.

54         .get("value", {})
    → Accède au dictionnaire value de ce changement, ou à un dictionnaire vide par défaut.

55         .get("statuses")
    → Accède à la clé statuses de value, présente uniquement lors d'une mise à jour de statut de livraison de message (envoyé,
    livré, lu) plutôt qu'un nouveau message entrant.

56     ):
    → Ferme la condition composée testant la présence d'une mise à jour de statut.

57         logging.info("Received a WhatsApp status update.")
    → Journalise en niveau info la réception d'une mise à jour de statut WhatsApp.

58         return JSONResponse(content={"status": "ok"})
    → Retourne une réponse JSON de statut ok sans traitement supplémentaire pour cette notification, qui n'est pas un message à
    traiter par l'agent.

60     if is_valid_whatsapp_message(body):
    → Vérifie via la fonction utilitaire que le corps de la requête est bien structuré comme un message WhatsApp valide.

61         try:
    → Démarre un bloc try pour capturer les erreurs de démarrage du traitement en arrière-plan.

62             # Run message processing in a background thread
    → Commentaire annonçant l'exécution du traitement du message dans un thread séparé.

63             Thread(target=process_whatsapp_message, args=(body,)).start()
    → Crée et démarre un thread qui exécute process_whatsapp_message avec le corps de la requête. C'est un choix délibéré :
    traiter le message (OCR/agent de vision, agent de fret, appels LLM) peut prendre plusieurs secondes, alors que Meta impose un
    délai de réponse très court sur le webhook — en déléguant le traitement réel à un thread séparé, le serveur peut répondre
    immédiatement (ligne suivante) pendant que le travail long continue en arrière-plan.

64         except Exception as e:
    → Capture toute exception survenue lors du démarrage du thread.

```


65 logging.error(f"Error starting background message thread: {e}")
 → Journalise en niveau erreur l'échec du démarrage du thread de traitement.

66 return JsonResponse(content={"status": "ok"})
 → Retourne immédiatement une réponse JSON de statut ok, sans attendre la fin du traitement en arrière-plan, pour respecter le délai de réponse attendu par l'API WhatsApp.

67 else:
 → Démarre le cas où le corps de la requête n'est pas un message WhatsApp valide.

68 return JsonResponse(status_code=404, content={"status": "error", "message": "Not a WhatsApp API event"})
 → Retourne une réponse JSON d'erreur 404 signalant que l'événement n'est pas reconnu.

72 @app.get("/test")
 → Déclare une route GET sur le chemin /test.

73 def test():
 → Définit la fonction de gestion de la route de test d'envoi de message template.

74 url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
 → Construit l'URL de l'API Graph de Meta pour envoyer un message, en insérant la version de l'API et l'identifiant du numéro WhatsApp Business.

75 headers = {
 → Ouvre le dictionnaire des en-têtes HTTP de la requête.

76 'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
 → Définit l'en-tête d'autorisation avec le jeton d'accès WhatsApp, au format standard "Bearer <token>" utilisé par la plupart des API REST authentifiées.

77 'Content-Type': 'application/json'
 → Définit l'en-tête indiquant que le corps de la requête est au format JSON.

78 }
 → Ferme le dictionnaire des en-têtes HTTP.

79 data = {
 → Ouvre le dictionnaire du corps de la requête à envoyer à l'API WhatsApp.

80 'messaging_product': 'whatsapp',
 → Précise que le produit de messagerie utilisé est whatsapp, champ requis par l'API Graph.

81 'to': RECIPIENT_PHONE_NUMBER,
 → Précise le numéro de téléphone destinataire du message.

82 'type': 'template',
 → Précise que le type de message envoyé est un template : WhatsApp exige d'utiliser des messages "template" pré-approuvés par Meta pour initier une conversation avec un utilisateur qui n'a pas écrit en premier depuis moins de 24h.

83 'template': {
 → Ouvre le dictionnaire décrivant le template de message à envoyer.

84 'name': 'hello_world',
 → Précise le nom du template utilisé, ici hello_world, un template de démonstration fourni par défaut par Meta pour tester l'intégration.

```

85         'language': {
    → Ouvre le dictionnaire précisant la langue du template.

86         'code': 'en_US'
    → Précise le code de langue en_US pour le template.

87     }
    → Ferme le dictionnaire de la langue du template.

88 }
    → Ferme le dictionnaire décrivant le template.

89 }
    → Ferme le dictionnaire du corps de la requête data.

90 response = requests.post(url, headers=headers, json=data)
    → Envoie une requête POST à l'API WhatsApp avec les en-têtes et le corps JSON définis.

91 return {"message": "Test successful!", "response": response.json()}
    → Retourne un message de succès accompagné du corps JSON de la réponse de l'API.

93 @app.get("/send_message")
    → Déclare une route GET sur le chemin /send_message.

94 def send_message():
    → Définit la fonction de gestion de la route d'envoi d'un message texte simple.

95     url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
    → Construit l'URL de l'API Graph WhatsApp pour envoyer un message avec la version et l'ID de numéro.

96     headers = {
    → Ouvre le dictionnaire des en-têtes HTTP de la requête.

97         'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
    → Définit l'en-tête d'autorisation avec le jeton d'accès WhatsApp.

98         'Content-Type': 'application/json'
    → Définit l'en-tête indiquant que le corps de la requête est au format JSON.

99     }
    → Ferme le dictionnaire des en-têtes HTTP.

100     data = {
    → Ouvre le dictionnaire du corps de la requête à envoyer à l'API WhatsApp.

101         'messaging_product': 'whatsapp',
    → Précise que le produit de messagerie utilisé est whatsapp.

102         'to': RECIPIENT_PHONE_NUMBER,
    → Précise le numéro de téléphone destinataire du message.

103         'type': 'text',
    → Précise que le type de message envoyé est un texte libre : contrairement au template, ce type ne peut être utilisé que dans
    une fenêtre de 24h après que l'utilisateur a écrit en premier.

104         'text': {
    → Ouvre le dictionnaire décrivant le contenu textuel du message.

105             'body': 'Hello, how are you?'
    → Précise le corps du message texte envoyé.

```

```

106         }
    → Ferme le dictionnaire décrivant le contenu textuel du message.

107     }
    → Ferme le dictionnaire du corps de la requête data.

108     response = requests.post(url, headers=headers, json=data)
    → Envoie une requête POST à l'API WhatsApp avec les en-têtes et le corps JSON définis.

109     return {"message": "Message sent successfully!", "response": response.json()}
    → Retourne un message de succès accompagné du corps JSON de la réponse de l'API.

111 # @app.get("/webhook")
    → Ligne de code commentée montrant une ancienne déclaration de route GET sur /webhook.

112 # async def webhook(request: Request):
    → Ligne de code commentée montrant une ancienne définition de fonction webhook.

113 # return await verify(request)
    → Ligne de code commentée montrant l'ancien appel à la fonction verify.

115 @app.get("/webhook")
    → Déclare la route GET active sur le chemin /webhook : c'est cette URL, une fois configurée dans le tableau de bord développeur Meta, que Meta appelle pour effectuer la vérification initiale du webhook.

116 async def webhook_verify(request: Request):
    → Définit la fonction asynchrone de gestion de la vérification du webhook.

117     return await verify(request)
    → Attend et retourne le résultat de la fonction verify appliquée à la requête.

119 @app.post("/webhook")
    → Déclare la route POST sur le même chemin /webhook : c'est cette même URL que Meta appelle ensuite, en POST cette fois, pour transmettre chaque événement (message reçu, mise à jour de statut) au fil de l'eau — c'est le principe même d'un webhook : le serveur tiers (Meta) pousse les événements vers votre application dès qu'ils se produisent, plutôt que votre application n'aille les chercher (polling).

120 async def webhook_message(request: Request):
    → Définit la fonction asynchrone de gestion des messages entrants du webhook.

121     return await handle_message(request)
    → Attend et retourne le résultat de la fonction handle_message appliquée à la requête.

123 if __name__ == "__main__":
    → Vérifie que le script est exécuté directement et non importé comme module.

124     uvicorn.run(app, host="0.0.0.0", port=8000)
    → Démarre le serveur uvicorn, qui prend en charge l'application FastAPI et écoute les requêtes entrantes. host="0.0.0.0" signifie que le serveur accepte les connexions depuis n'importe quelle interface réseau (pas seulement localhost), nécessaire pour qu'un service externe comme Meta puisse l'atteindre (généralement via un tunnel ou un déploiement public), sur le port 8000.

```

33-FreightAgent/agent_service.py

Ce fichier définit un agent de vision multimodal qui analyse une image de produit (ou un texte seul) pour en extraire les spécifications (nom, dimensions, matériau) via un modèle OpenAI, utilisé en amont de l'agent de

fret.

50 lignes de code annotées ci-dessous.

1 `import os`

→ Importe le module standard `os` pour accéder aux variables d'environnement.

2 `import base64`

→ Importe le module standard `base64` (utilisé pour encoder des données binaires en texte), bien qu'il ne soit finalement pas utilisé directement dans le reste de ce fichier — probablement un reliquat d'une version antérieure où l'image était encodée manuellement avant d'être envoyée au modèle.

3 `# from letta_client import Letta`

→ Ligne de code commentée qui aurait importé le client `Letta`. `Letta` (anciennement `MemGPT`) est un framework concurrent d'`agno` pour construire des agents IA dotés d'une mémoire à long terme persistante ; cette piste a visiblement été explorée puis abandonnée au profit d'`agno`, dont l'usage se retrouve dans les autres fichiers du projet.

4 `from dotenv import load_dotenv`

→ Importe la fonction qui charge les variables d'environnement depuis un fichier `.env`.

5 `from agno.agent import Agent`

→ Importe la classe `Agent` du framework `agno`, l'abstraction centrale représentant un agent IA.

6 `from agno.media import Image`

→ Importe la classe `Image` d'`agno`, qui permet d'encapsuler des données d'image binaires pour les fournir en entrée à un agent. Elle est indispensable pour utiliser un modèle multimodal (capable de traiter à la fois texte et image) plutôt qu'un modèle purement textuel.

7 `from agno.models.openai import OpenAIChat`

→ Importe la classe qui encapsule un modèle de chat `OpenAI` utilisable par l'agent, ici pour sa capacité de vision (analyse d'image).

9 `load_dotenv()`

→ Charge les variables définies dans le fichier `.env` dans l'environnement du processus.

11 `openai_key = os.getenv("OPENAI_API_KEY")`

→ Récupère la clé API `OpenAI` depuis les variables d'environnement.

14 `# LETTA_TOKEN = os.getenv('LETTA_API_KEY')`

→ Ligne de code commentée qui aurait récupéré un jeton d'API `Letta`, non utilisé dans la version actuelle du fichier.

15 `# AGENT_ID = os.getenv('AGENT_ID')`

→ Ligne de code commentée qui aurait récupéré un identifiant d'agent `Letta` persistant, non utilisé.

17 `# client = Letta(`

→ Ligne de code commentée qui aurait commencé l'instanciation d'un client `Letta`.

18 `# token=LETTA_TOKEN,`

→ Ligne de code commentée qui aurait passé le jeton `Letta` au constructeur.

19 `# # timeout=15,`

→ Ligne de code commentée montrant un paramètre de `timeout` désactivé.

20 `# )`

→ Ligne de code commentée qui aurait fermé l'instanciation du client `Letta`.

22 `# create_agent`

→ Commentaire indiquant la section commentée de création d'agent `Letta`.

```

23 # response = client.templates.agents.create(
    → Ligne de code commentée qui aurait créé un agent à partir d'un template Letta (Letta propose un système de templates d'agents réutilisables côté serveur).

24 # project="default-project",
    → Ligne de code commentée précisant le projet Letta par défaut.

25 # template_version="Companion:latest",
    → Ligne de code commentée précisant la version du template Letta utilisée.

26 # )
    → Ligne de code commentée qui aurait fermé l'appel de création d'agent Letta.

28 # message_agent
    → Commentaire indiquant que la fonction suivante envoie un message à l'agent.

29 def get_response(message):
    → Définit la fonction get_response, le point d'entrée public de ce module : elle accepte soit une chaîne de texte simple, soit un dictionnaire décrivant une image à analyser.

30     message = message
    → Réaffecte le paramètre message à lui-même, ligne sans effet fonctionnel (probablement un reliquat de refactoring).

31     image_data = None
    → Initialise la variable image_data à None avant de savoir si une image est fournie, pour pouvoir tester sa présence plus loin.

32     print('-----Agent triggered-----')
    → Affiche dans la console un marqueur signalant le déclenchement de l'agent.

33     if isinstance(message, dict):
    → Vérifie si le paramètre message est un dictionnaire, ce qui, par convention dans ce projet, indique la présence d'une image (par opposition à une simple chaîne de texte).

34         with open(message.get("image_path"), "rb") as f:
    → Ouvre en lecture binaire le fichier image dont le chemin est fourni dans le dictionnaire message.

35             image_data = f.read()
    → Lit l'intégralité du contenu binaire du fichier image et le stocke dans image_data.

36             message = message.get('caption')
    → Remplace message par la légende (caption) associée à l'image, extraite du dictionnaire : à partir de cette ligne, message redevient une simple chaîne de texte (ou None si aucune légende).

38     print('---message---', message)
    → Affiche dans la console le contenu du message ou de la légende à traiter.

39     agent = Agent(
    → Instancie un nouvel Agent à chaque appel de get_response (plutôt qu'un agent global réutilisé) et commence à passer ses arguments.

40         model=OpenAIChat(id="gpt-4o-mini", api_key=openai_key),
    → Configure l'agent pour utiliser le modèle OpenAI gpt-4o-mini, une version plus légère et moins coûteuse de gpt-4o qui conserve des capacités de vision (analyse d'image), suffisante pour une tâche d'extraction de caractéristiques produit.

41         instructions="""
    → Commence à définir les instructions de l'agent sous forme de chaîne multi-ligne brute (sans dedent ici, contrairement aux autres fichiers du projet — l'indentation résiduelle n'affecte pas significativement la compréhension du modèle).

42         If an image is given, you are responsible for defining the object on the image, (if it's a
        product define what is it in details),

```

→ Ligne d'instruction demandant de décrire en détail l'objet présent sur l'image.

43 along with it's name, dimensions and material.
→ Ligne d'instruction précisant d'inclure le nom, les dimensions et le matériau de l'objet.

44 IMPORTANT: Do not ask back, only give product specifications, dimensions and material!
→ Ligne d'instruction interdisant à l'agent de poser des questions en retour : essentiel ici car l'agent est appelé de façon programmatique (via un webhook WhatsApp) sans possibilité réelle d'échange conversationnel avec l'utilisateur à cette étape.

45 """
→ Ferme la chaîne d'instructions multi-ligne et l'argument instructions de l'Agent.

46)
→ Ferme l'appel au constructeur Agent définissant la variable agent.

47 if image_data:
→ Vérifie si des données d'image ont été chargées précédemment.

48 response = agent.run('Please define the object on the image, along with it\'s name, dimensions and material.', images=[Image(content=image_data)])
→ Exécute l'agent avec une consigne textuelle fixe et l'image fournie encapsulée dans un objet Image : c'est ce paramètre images qui active le mode multimodal, envoyant à la fois du texte et des données visuelles binaires au modèle gpt-4o-mini.

49 else:
→ Démarre le cas où aucune image n'a été fournie.

50 response = agent.run(message)
→ Exécute l'agent uniquement avec le message texte fourni, sans image.

51 print('---response---', response.content)
→ Affiche dans la console le contenu de la réponse générée par l'agent.

52 full_response = f"""
→ Commence la construction d'une chaîne multi-ligne combinant la question et la réponse.

53 User Question: {message}
→ Ligne du gabarit affichant le message ou la légende d'origine de l'utilisateur.

54 Product Specifications: {response.content}
→ Ligne du gabarit affichant les spécifications produit générées par l'agent, qui seront ensuite transmises telles quelles à l'agent de fret comme contexte de la demande.

55 """
→ Ferme la chaîne multi-ligne assignée à full_response.

56 return full_response
→ Retourne la chaîne combinant question et spécifications à l'appelant.

59 if __name__ == '__main__':
→ Vérifie que le script est exécuté directement et non importé comme module.

60 print(get_response('hello'))
→ Appelle get_response avec le texte 'hello' (sans image) et affiche le résultat dans la console, un test manuel rapide du chemin texte seul.

Ce fichier fournit les fonctions utilitaires pour valider, envoyer et traiter les messages WhatsApp entrants, en orchestrant les appels successifs à l'agent de vision (agent_service) puis à l'agent de fret (freight_agent) avant de renvoyer la réponse finale à l'utilisateur.

82 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard os pour accéder aux variables d'environnement.

2 import re

→ Importe le module standard re (expressions régulières), bien que non utilisé directement dans ce fichier — probablement conservé d'une version antérieure ou pour un usage futur.

3 import json

→ Importe le module standard json, bien que non utilisé directement dans ce fichier (le décodage JSON des requêtes est géré côté FastAPI dans main.py).

4 import logging

→ Importe le module standard logging pour journaliser les événements liés à l'envoi et à la réception des messages WhatsApp.

5 import requests

→ Importe la librairie requests pour effectuer des appels HTTP vers l'API Graph de WhatsApp (envoi de messages et récupération de médias).

6 from time import sleep

→ Importe la fonction sleep du module time, bien que non utilisée directement dans ce fichier — probablement un reliquat prévu pour une temporisation entre tentatives d'envoi.

7 from dotenv import load_dotenv

→ Importe la fonction qui charge les variables d'environnement depuis un fichier .env.

8 from freight_agent import full_response

→ Importe la fonction full_response du module freight_agent : c'est l'appel à l'agent de fret (RAG + Groq) qui produit l'estimation finale de coût d'expédition.

9 from agent_service import get_response

→ Importe la fonction get_response du module agent_service : c'est l'appel à l'agent de vision (OpenAI gpt-4o-mini) qui analyse le texte ou l'image reçue pour en extraire les spécifications du produit à expédier.

10 from fastapi.responses import JSONResponse

→ Importe la classe JSONResponse de FastAPI pour construire des réponses d'erreur JSON directement depuis ce module utilitaire.

12 load_dotenv()

→ Charge les variables définies dans le fichier .env dans l'environnement du processus.

14 WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')

→ Récupère le jeton d'accès à l'API WhatsApp depuis les variables d'environnement.

15 VERSION = os.getenv('VERSION')

→ Récupère la version de l'API Graph WhatsApp depuis les variables d'environnement.

16 PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')

→ Récupère l'identifiant du numéro de téléphone WhatsApp depuis les variables d'environnement.

18 def log_http_response(response):

→ Définit une fonction utilitaire qui journalise les détails d'une réponse HTTP, réutilisée après chaque envoi de message pour faciliter le diagnostic en cas de problème.

19 logging.info(f"Status: {response.status_code}")

→ Journalise en niveau info le code de statut HTTP de la réponse.

```
20 logging.info(f"Content-type: {response.headers.get('content-type')}")
```

→ Journalise en niveau info le type de contenu de l'en-tête de la réponse.

```
21 logging.info(f"Body: {response.text}")
```

→ Journalise en niveau info le corps textuel de la réponse.

```
24 def get_text_message_input(recipient, text):
```

→ Définit une fonction qui construit le corps de requête JSON attendu par l'API Graph de WhatsApp pour envoyer un message texte.

```
25     return {
```

→ Commence à retourner un dictionnaire représentant la charge utile (payload) du message.

```
26         "messaging_product": "whatsapp",
```

→ Précise que le produit de messagerie utilisé est whatsapp, champ requis par l'API Graph.

```
27         "recipient_type": "individual",
```

→ Précise que le destinataire est un individu et non un groupe.

```
28         "to": recipient,
```

→ Précise le numéro de téléphone destinataire passé en paramètre.

```
29         "type": "text",
```

→ Précise que le type de message envoyé est un texte simple.

```
30         "text": {"preview_url": False, "body": text},
```

→ Définit le contenu du message texte, sans aperçu d'URL (preview_url: False), avec le texte fourni.

```
31     }
```

→ Ferme le dictionnaire retourné par la fonction.

```
34 def generate_response(message):
```

→ Définit une fonction de génération de réponse simple, non utilisée par le reste du fichier (probablement un reliquat conservé d'une version antérieure du code avant l'intégration des vrais agents).

```
35     # Dummy response for now
```

→ Commentaire indiquant que cette réponse est une implémentation temporaire.

```
36     return message.upper()
```

→ Retourne le message reçu converti en majuscules, une réponse factice de test.

```
39 def send_message(data):
```

→ Définit la fonction qui envoie effectivement un message via l'API Graph de WhatsApp.

```
40     headers = {
```

→ Ouvre le dictionnaire des en-têtes HTTP de la requête.

```
41         "Content-type": "application/json",
```

→ Définit l'en-tête indiquant que le corps de la requête est au format JSON.

```
42         "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
```

→ Définit l'en-tête d'autorisation avec le jeton d'accès WhatsApp.

```
43     }
```

→ Ferme le dictionnaire des en-têtes HTTP.


```

44     url = f"https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages"
    → Construit l'URL de l'API Graph WhatsApp pour l'envoi de messages.

46     try:
    → Démarre un bloc try pour capturer les erreurs éventuelles de l'envoi HTTP.

47         response = requests.post(url, json=data, headers=headers, timeout=10)
    → Envoie une requête POST à l'API WhatsApp avec les données, en-têtes et un délai d'expiration (timeout) de 10 secondes :
fixer un timeout explicite évite que la requête ne reste bloquée indéfiniment si l'API distante ne répond pas.

48         response.raise_for_status()
    → Lève une exception (requests.HTTPError) si le code de statut HTTP indique une erreur (4xx ou 5xx), pour que ces cas
soient traités par le bloc except plutôt que traités silencieusement.

49         log_http_response(response)
    → Journalise les détails de la réponse HTTP via la fonction utilitaire dédiée.

50         return response
    → Retourne l'objet réponse HTTP en cas de succès.

51     except requests.Timeout:
    → Capture spécifiquement l'exception de dépassement de délai de la requête (déclenchée par le timeout=10 défini plus haut).

52         logging.error("Timeout occurred while sending message")
    → Journalise en niveau erreur le dépassement de délai lors de l'envoi du message.

53         return JSONResponse({"status": "error", "message": "Request timed out"}, status_code=408)
    → Retourne une réponse JSON d'erreur avec le code de statut HTTP 408 (Request Timeout) pour le délai dépassé.

54     except requests.RequestException as e:
    → Capture toute autre exception liée à la requête HTTP (y compris celle levée par raise_for_status), la classe la plus générale
de la hiérarchie d'exceptions de requests.

55         logging.error(f"Request failed due to: {e}")
    → Journalise en niveau erreur le détail de l'exception de requête rencontrée.

56         return JSONResponse({"status": "error", "message": "Failed to send message"}, status_code=500)
    → Retourne une réponse JSON d'erreur avec le code de statut 500 pour l'échec d'envoi.

59 def is_valid_whatsapp_message(body: dict):
    → Définit la fonction qui vérifie que le corps de requête reçu suit bien la structure imbriquée attendue d'un message WhatsApp
valide, avant de le traiter.

60     return (
    → Commence à retourner une expression composée de vérifications enchaînées avec l'opérateur and : en Python, and
court-circuite dès la première valeur fausse (ou absente), donc chaque accès suivant n'est évalué que si le précédent a réussi, ce
qui évite une exception de clé manquante.

61         body.get("object")
    → Vérifie que la clé object est présente et non vide dans le corps.

62         and body.get("entry")
    → Vérifie que la clé entry est présente et non vide dans le corps.

63         and body["entry"][0].get("changes")
    → Vérifie que le premier élément de entry contient la clé changes.

64         and body["entry"][0]["changes"][0].get("value")
    → Vérifie que le premier changement contient la clé value.

```

```

65         and body["entry"][0]["changes"][0]["value"].get("messages")
    → Vérifie que value contient la clé messages.

66         and body["entry"][0]["changes"][0]["value"]["messages"][0]
    → Vérifie que le premier élément de la liste messages existe et n'est pas vide.

67     )
    → Ferme l'expression composée retournée par la fonction is_valid_whatsapp_message.

70 def process_whatsapp_message(body: dict):
    → Définit la fonction principale qui traite un message WhatsApp entrant déjà validé : c'est elle qui orchestre l'ensemble du
    pipeline (extraction du message, appel à l'agent de vision, appel à l'agent de fret, envoi de la réponse), exécutée dans le thread
    d'arrière-plan lancé par main.py.

71     value = body["entry"][0]["changes"][0]["value"]
    → Extrait le dictionnaire value contenant les informations du message et du contact, selon la structure imbriquée du payload
    WhatsApp.

72     wa_id = value["contacts"][0]["wa_id"]
    → Extrait l'identifiant WhatsApp (le numéro de téléphone) du contact expéditeur, nécessaire pour lui répondre ensuite.

73     name = value["contacts"][0]["profile"]["name"]
    → Extrait le nom de profil du contact expéditeur, non utilisé ensuite dans la fonction (récupéré mais non exploité dans cette
    version).

74     message = value["messages"][0]
    → Extrait le premier message de la liste des messages reçus.

75     if message["type"] == "text":
    → Vérifie si le type du message reçu est un texte simple.

76         message_body = message["text"]["body"]
    → Extrait le corps textuel du message.

77     elif message["type"] == "image":
    → Vérifie si le type du message reçu est une image.

78         message_body = message["image"]
    → Extrait le dictionnaire décrivant l'image reçue (WhatsApp ne transmet pas l'image directement dans le webhook, seulement
    une référence à télécharger).

79         id = message_body.get("id")
    → Extrait l'identifiant du média image fourni par WhatsApp, nécessaire pour interroger l'API Graph et obtenir son URL de
    téléchargement.

80         headers = {
    → Ouvre le dictionnaire des en-têtes HTTP utilisés pour récupérer les métadonnées de l'image.

81             "Content-type": "application/json",
    → Définit l'en-tête indiquant que la requête est au format JSON.

82             "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
    → Définit l'en-tête d'autorisation avec le jeton d'accès WhatsApp.

83         }
    → Ferme le dictionnaire des en-têtes HTTP.

84         metadata_res = requests.get(f"https://graph.facebook.com/{VERSION}/{id}", headers=headers)

```

→ Envoie une requête GET pour récupérer les métadonnées du média à partir de son identifiant : l'API WhatsApp fonctionne en deux étapes pour les médias, il faut d'abord résoudre l'identifiant en une URL de téléchargement temporaire.

```
85 image_url = metadata_res.json().get("url")
```

→ Extrait l'URL de téléchargement réelle de l'image depuis la réponse JSON des métadonnées.

```
86 image_res = requests.get(image_url, headers={"Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}"})
```

→ Télécharge le contenu binaire de l'image depuis son URL, en fournissant à nouveau le jeton d'autorisation (l'URL de média WhatsApp reste protégée et nécessite le même jeton).

```
87 with open(f"33-FreightAgent/images/{id}.jpg", "wb") as f:
```

→ Ouvre en écriture binaire un fichier nommé d'après l'identifiant du média dans le dossier images, pour sauvegarder localement l'image reçue.

```
88 f.write(image_res.content)
```

→ Écrit le contenu binaire téléchargé de l'image dans le fichier local.

```
89 if message_body.get("caption"):
```

→ Vérifie si le message image contient une légende (caption) non vide, c'est-à-dire un texte accompagnant l'image envoyée par l'utilisateur.

```
90 message_body = {'image_path': f'33-FreightAgent/images/{id}.jpg', 'caption':  
message_body.get("caption")}
```

→ Remplace message_body par un dictionnaire contenant le chemin de l'image et sa légende : c'est ce format (dict avec image_path) que la fonction get_response de agent_service.py sait reconnaître comme un message contenant une image, cf. le isinstance(message, dict) de ce fichier-là.

```
91 else:
```

→ Démarre le cas où l'image ne contient pas de légende.

```
92 message_body = {'image_path': f'33-FreightAgent/images/{id}.jpg', 'caption': ''}
```

→ Remplace message_body par un dictionnaire contenant le chemin de l'image et une légende vide.

```
93 response_text = get_response(message_body)
```

→ Appelle l'agent de vision (module agent_service) avec le message construit (texte simple ou dictionnaire image+légende) et récupère l'analyse des spécifications du produit : première étape du pipeline en deux agents.

```
94 full_text = full_response(response_text)
```

→ Transmet cette analyse de produit à l'agent de fret (module freight_agent), qui consulte sa base de connaissances RAG (grilles tarifaires) et éventuellement le web pour produire une estimation complète de coût d'expédition : deuxième étape du pipeline.

```
95 data = get_text_message_input(wa_id, full_text)
```

→ Construit la charge utile JSON du message WhatsApp de réponse à envoyer au contact, avec l'estimation de fret comme contenu.

```
96 send_message(data)
```

→ Envoie le message de réponse final au contact via l'API WhatsApp, bouclant ainsi le cycle complet : message entrant (texte/image) agent de vision agent de fret réponse WhatsApp sortante.

Projet 34 — 34-ImportExportSpecialist

Variante autonome (CLI) : conforme import/export, export Word direct

Objectif

Vérifie automatiquement la conformité d'une transaction d'import/export entre le Royaume-Uni et le Maroc face à un ensemble de textes réglementaires indexés, et évalue le niveau de risque associé. Fonctionne comme un script autonome en ligne de commande, sans interface web, et exporte directement son rapport final dans un document Word.

Code source, ligne par ligne

34-ImportExportSpecialist/full_import_export_agent.py

Agent Agno qui fait de l'OCR sur des documents PDF réglementaires, indexe leur contenu dans une base de connaissances vectorielle locale (RAG), puis génère un rapport de conformité import-export au format DOCX.

240 lignes de code annotées ci-dessous.

1 import sys

→ Importe le module standard sys, qui donne accès aux mécanismes internes de l'interpréteur Python : arguments de ligne de commande, code de sortie du processus, et surtout ici les flux d'entrée/sortie standard (stdout/stderr). Il est importé uniquement pour le bloc de reconfiguration d'encodage qui suit juste en dessous.

3 # La console Windows utilise par défaut l'encodage cp1252, qui ne supporte pas

→ Commentaire expliquant le problème à l'origine du bloc suivant : cp1252 est un ancien encodage de caractères occidental utilisé par défaut par les terminaux Windows, incapable de représenter les emojis (qui font partie du jeu de caractères Unicode étendu).

4 # les emojis utilisés dans les messages de ce script (ex: ,).

→ Suite du commentaire : ce script affiche volontairement des emojis dans ses messages de log/print (voir plus loin " Rapport sauvegardé"), d'où la nécessité de forcer un encodage compatible avant tout print().

5 for _stream in (sys.stdout, sys.stderr):

→ Boucle sur les deux flux de sortie du processus (sortie standard et sortie d'erreur) pour appliquer le même correctif d'encodage aux deux, car une erreur peut être écrite sur stderr tout autant qu'un message normal sur stdout.

6 if hasattr(_stream, "reconfigure"):

→ Vérifie que l'objet flux possède bien la méthode reconfigure avant de l'appeler. Cette méthode n'existe que sur les TextIOWrapper standards (Python 3.7+) ; dans certains contextes (flux redirigés, capturés par un IDE, etc.) l'objet peut être d'un autre type qui ne l'expose pas, d'où cette vérification défensive pour éviter un crash au démarrage.

7 _stream.reconfigure(encoding="utf-8", errors="replace")

→ Force l'encodage du flux en UTF-8 (qui couvre tous les caractères Unicode, dont les emojis) et indique que tout caractère malgré tout non encodable doit être remplacé par un caractère de substitution plutôt que de lever une exception UnicodeEncodeError qui arrêterait le script.

9 import os

→ Importe le module standard os, utilisé dans tout le script pour manipuler des chemins de fichiers, créer des dossiers et lire les variables d'environnement (clés API).

10 import markdown2

→ Importe la bibliothèque tierce markdown2, un convertisseur Markdown vers HTML. Elle est disponible dans le script mais n'est pas explicitement invoquée dans le code visible plus bas ; la conversion HTMLDOCX finale passe en réalité par html2docx

directement sur le HTML déjà produit par les agents.

11 `from textwrap import dedent`

→ Importe `dedent`, une fonction de la bibliothèque standard qui retire l'indentation commune à toutes les lignes d'une chaîne multi-lignes. Elle est essentielle ici car les blocs d'instructions des agents (prompts système) sont écrits indentés pour rester lisibles dans le code source, mais cette indentation ne doit pas se retrouver dans le texte réellement envoyé au modèle de langage.

12 `from agno.agent import Agent`

→ Importe la classe `Agent` du framework `Agno`. `Agno` est un framework Python open-source pour construire des agents IA : il encapsule un modèle de langage (LLM), lui associe des instructions (rôle, règles), éventuellement des outils (function calling) et une base de connaissances (RAG), puis expose une méthode `run()/print_response()` pour dialoguer avec lui. C'est la brique de base sur laquelle repose tout ce script.

13 `from mistralai.client import Mistral`

→ Importe le client officiel du SDK `Mistral AI`. Il permet d'appeler les API hébergées de `Mistral`, notamment ici son service d'OCR (reconnaissance optique de caractères), utilisé plus bas pour extraire le texte d'un PDF réglementaire.

14 `from html2docx import html2docx`

→ Importe la fonction `html2docx`, une bibliothèque qui convertit une chaîne HTML en un document Word (.docx) binaire. Elle permet de transformer directement la sortie HTML générée par les agents en un livrable bureautique, sans avoir à reconstruire manuellement le document avec une API comme `python-docx`.

15 `from agno.models.groq import Groq`

→ Importe la classe `Groq` du framework `Agno`. `Groq` est un fournisseur d'inférence qui exécute des modèles de langage open-weight (comme `Llama` ou `GPT-OSS`) sur du matériel spécialisé (des puces LPU) optimisé pour une génération de texte très rapide et peu coûteuse. Ici `Groq` sert de "modèle" pour le chat des agents, en alternative/complément à `OpenAI`.

16 `from agno.vectordb.lancedb import LanceDb, SearchType # Changement ici`

→ Importe `LanceDb`, une base de données vectorielle embarquée (fichier local, sans serveur externe à héberger) utilisée pour stocker des vecteurs d'embeddings et effectuer des recherches par similarité, ainsi que l'énumération `SearchType` qui définit la stratégie de recherche (vectorielle pure, texte intégral, ou hybride). Le commentaire signale qu'un composant a été remplacé récemment (probablement une autre base vectorielle plus lourde à héberger).

17 `from agno.tools.duckduckgo import DuckDuckGoTools`

→ Importe `DuckDuckGoTools`, un outil `Agno` qui donne à l'agent la capacité d'effectuer des recherches web via `DuckDuckGo`. C'est un exemple de "function calling" (ou tool use) : le modèle de langage peut décider, pendant sa génération, d'appeler cette fonction pour aller chercher une information manquante ou plus récente que ce que contient sa base de connaissances statique, plutôt que d'inventer une réponse (hallucination).

18 `from agno.knowledge.knowledge import Knowledge`

→ Importe `Knowledge`, la classe unifiée d'`Agno` qui implémente le pattern RAG (Retrieval-Augmented Generation, "génération augmentée par récupération") : les documents sources sont découpés, transformés en vecteurs (embeddings) et stockés dans une base vectorielle ; à chaque question, les passages les plus pertinents sont récupérés et injectés dans le contexte envoyé au modèle. Cela permet à l'agent de répondre en s'appuyant sur des documents réels plutôt que sur sa seule mémoire d'entraînement.

19 `from agno.knowledge.embedder.mistral import MistralEmbedder`

→ Importe `MistralEmbedder`, qui encapsule un modèle d'embedding de `Mistral`. Un embedding est une représentation numérique (un vecteur) d'un texte, construite de sorte que deux textes de sens proche aient des vecteurs proches : c'est ce mécanisme qui permet à la base de connaissances de retrouver un passage pertinent même s'il ne partage aucun mot exact avec la question posée.

20 `from dotenv import load_dotenv`

→ Importe `load_dotenv`, de la bibliothèque `python-dotenv`, qui charge des paires clé=valeur depuis un fichier `.env` local vers les variables d'environnement du processus. C'est la pratique standard pour garder les clés API et secrets hors du code source versionné.

21 `load_dotenv()`

→ Exécute effectivement ce chargement au démarrage du script, rendant les variables du fichier `.env` (clés `Mistral`, `Groq`, etc.)

accessibles via `os.getenv()` dans la suite du fichier.

23 # Dossier du script : les chemins sont résolus par rapport à lui, pas au

→ Commentaire introduisant la logique de résolution des chemins par rapport à l'emplacement du script plutôt qu'au répertoire d'exécution courant.

24 # répertoire courant, pour que le script fonctionne peu importe d'où il est lancé.

→ Suite du commentaire : si les chemins étaient relatifs au répertoire courant (cwd), lancer le script depuis un autre dossier casserait les accès aux fichiers Markdown/PDF ; ancrer les chemins sur `__file__` rend le script portable.

25 `SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))`

→ Calcule le chemin absolu du dossier contenant ce script (en résolvant d'abord le chemin absolu du fichier courant, puis en prenant son dossier parent) et le stocke dans une constante réutilisée pour construire tous les autres chemins.

26 `MARKDOWN_DIR = os.path.join(SCRIPT_DIR, "Markdown")`

→ Construit le chemin vers le sous-dossier "Markdown", qui sert de dépôt intermédiaire pour les documents déjà convertis en texte, avant leur indexation dans la base de connaissances.

27 `LANCEDB_URI = os.path.join(SCRIPT_DIR, "tmp", "lancedb")`

→ Construit le chemin vers le dossier local où LanceDB persistera physiquement ses fichiers de données (index vectoriels). Étant une base embarquée, LanceDB n'a besoin que d'un chemin de dossier, contrairement à une base vectorielle hébergée qui nécessiterait une URL de serveur et des identifiants réseau.

29 `mistral_key = os.getenv("MISTRAL_API_KEY")`

→ Lit la clé API Mistral depuis les variables d'environnement. `os.getenv` renvoie `None` si la variable est absente plutôt que de lever une exception, ce qui permet ensuite de gérer l'absence de clé de façon conditionnelle (fonctionnalité OCR désactivée) plutôt que de faire planter tout le script.

30 # Le script original utilisait OpenAI (chat + embeddings) ; remplacé par Groq

→ Commentaire expliquant un choix d'architecture : la version originale du script s'appuyait entièrement sur OpenAI pour le chat et les embeddings.

31 # pour le chat (clé `GROQ_API_KEY` déjà disponible) et par l'embedder Mistral

→ Suite du commentaire : Groq a été choisi pour le chat (inférence rapide et économique sur des modèles open-weight), et Mistral fournit les embeddings.

32 # pour les embeddings (Groq n'a pas d'API d'embeddings).

→ Fin du commentaire : Groq est un service dédié à l'inférence de génération de texte rapide, il ne propose pas de modèle d'embedding, ce qui justifie de faire appel à un second fournisseur (Mistral) uniquement pour cette tâche.

33 `groq_key = os.getenv("GROQ_API_KEY")`

→ Lit la clé API Groq depuis les variables d'environnement, nécessaire au fonctionnement de tout modèle de chat du script.

35 `if not groq_key:`

→ Vérifie si la clé Groq est absente ou vide.

36 `print("ERREUR: GROQ_API_KEY manquant")`

→ Affiche un message d'erreur indiquant que la clé Groq est manquante.

37 `exit()`

→ Arrête immédiatement l'exécution du script : contrairement à la clé Mistral (optionnelle, ne bloque que l'OCR), la clé Groq est requise par tous les agents définis plus bas, donc continuer sans elle mènerait à un échec plus tard, moins clair pour l'utilisateur ; on préfère échouer tôt et explicitement (pattern "fail fast").

39 `if mistral_key:`

→ Vérifie si une clé API Mistral a été fournie.

40 `client = Mistral(api_key=mistral_key)`

→ Instancie le client Mistral avec la clé API, ce qui permettra ensuite d'appeler ses services (upload de fichiers, OCR) via cet objet.

```
41 else:
    → Cas où aucune clé Mistral n'a été trouvée.

42     print("WARNING: MISTRAL_API_KEY manquant - OCR désactivé")
    → Affiche un avertissement (pas une erreur bloquante) : la clé Mistral ne gouverne que la fonctionnalité d'OCR, le reste du script (agents interrogeant la base de connaissances déjà construite) reste utilisable sans elle ; c'est un exemple de dégradation gracieuse plutôt que d'arrêt total.

44 ## OCR the pdf ###
    → Commentaire de section annonçant la fonction dédiée à l'OCR des PDF.

45 def ocr_pdf(pdf_path):
    → Définit la fonction qui effectue l'OCR (Optical Character Recognition, reconnaissance optique de caractères) d'un fichier PDF donné en argument : elle transforme un document scanné ou en image en texte exploitable par du code, étape indispensable avant de pouvoir l'indexer dans une base de connaissances textuelle/vectorielle.

46     if not mistral_key:
        → Vérifie à nouveau la présence de la clé Mistral avant de lancer l'OCR, garde locale à la fonction en plus de l'avertissement global.

47         print("MISTRAL_API_KEY requis pour OCR")
        → Affiche un message indiquant que la clé est requise pour cette opération.

48         return
        → Interrompt la fonction et retourne None si la clé Mistral est absente.

50     # check if the file exists
    → Commentaire indiquant que le bloc suivant vérifie l'existence du fichier PDF.

51     if not os.path.exists(pdf_path):
        → Vérifie que le chemin du fichier PDF fourni existe réellement sur le disque, pour échouer avec un message clair plutôt qu'une exception bas niveau si le chemin est incorrect.

52         print("PDF file not found")
        → Affiche un message indiquant que le fichier PDF est introuvable.

53         return
        → Interrompt la fonction si le fichier PDF n'existe pas.

55     uploaded_pdf = client.files.upload(
        → Démarre l'appel à l'API Mistral pour téléverser le fichier PDF vers ses serveurs : l'API OCR de Mistral fonctionne sur des fichiers déjà hébergés côté Mistral (référéncés par un identifiant), pas sur des flux binaires envoyés directement à l'endpoint d'OCR.

56         file={
            → Ouvre le dictionnaire décrivant le fichier à envoyer.

57             "file_name": os.path.basename(pdf_path),
            → Fournit le nom du fichier (sans le chemin complet) comme métadonnée pour l'upload.

58             "content": open(pdf_path, "rb"),
            → Ouvre le fichier PDF en mode binaire ("rb", read-binary) car un PDF est un format binaire et non du texte, puis fournit ce flux directement au SDK qui se charge de le lire pour l'upload.

59         },
        → Ferme le dictionnaire décrivant le fichier.

60         purpose="ocr"
```

→ Indique explicitement à l'API que ce fichier est téléversé dans le but d'un traitement OCR, ce qui conditionne le pipeline appliqué côté serveur Mistral.

61)

→ Ferme l'appel à client.files.upload commencé plus haut.

63 signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)

→ Demande à l'API une URL signée temporaire pointant vers le fichier téléversé. L'étape d'OCR suivante attend une URL de document (pas des octets bruts) ; une URL signée permet d'y accéder de façon authentifiée et limitée dans le temps, sans rendre le fichier public de façon permanente.

65 ocr_response = client.ocr.process(

→ Lance le traitement OCR proprement dit via l'API Mistral, en utilisant un modèle spécialisé distinct des modèles de chat.

66 model="mistral-ocr-latest",

→ Précise le modèle OCR à utiliser : un modèle dédié à la lecture de mise en page et de texte dans des documents scannés/images, différent des modèles conversationnels utilisés ailleurs dans le script.

67 document={

→ Ouvre le dictionnaire décrivant le document à analyser.

68 "type": "document_url",

→ Indique que le document est fourni sous forme d'URL plutôt que de contenu brut encodé dans la requête.

69 "document_url": signed_url.url,

→ Fournit l'URL signée récupérée précédemment comme source du document à traiter.

70 },

→ Ferme le dictionnaire décrivant le document.

71 include_image_base64=True

→ Demande à l'API d'inclure également les images extraites du document, encodées en base64, dans la réponse (utile si le document contient des schémas/tableaux graphiques à conserver, même si ce script n'exploite ici que le texte Markdown renvoyé).

72)

→ Ferme l'appel à client.ocr.process commencé plus haut.

74 # Créer le dossier s'il n'existe pas

→ Commentaire annonçant la création du dossier de sortie Markdown si nécessaire.

75 os.makedirs(MARKDOWN_DIR, exist_ok=True)

→ Crée le dossier MARKDOWN_DIR (et ses dossiers parents si besoin) ; exist_ok=True évite qu'une exception soit levée si le dossier existe déjà, rendant l'opération idempotente.

77 with open(os.path.join(MARKDOWN_DIR, "morocco-import-law.md"), "w", encoding="utf-8") as f:

→ Ouvre en écriture, avec un encodage UTF-8 explicite, un fichier Markdown de sortie nommé morocco-import-law.md ; le bloc with garantit la fermeture automatique du fichier même en cas d'erreur.

78 f.write("\n".join([page.markdown for page in ocr_response.pages]))

→ L'OCR renvoie le document page par page ; cette ligne concatène le Markdown de chaque page (séparé par des retours à la ligne) en un seul texte, transformant ainsi un PDF scanné en un document texte structuré prêt à être indexé par la base de connaissances RAG plus bas.

80 ## Run it only once ##

→ Commentaire indiquant que les lignes suivantes ne doivent être exécutées qu'une seule fois.

81 # ocr_pdf(os.path.join(SCRIPT_DIR, "Documents", "morocco-import-law.pdf"))

→ Appel à ocr_pdf laissé en commentaire : l'OCR (opération payante et coûteuse en temps) a été exécutée une fois

manuellement pour produire le fichier Markdown, puis désactivée pour ne pas la relancer à chaque exécution du script - un pattern courant pour des étapes de prétraitement de données coûteuses.

```
82 # quit()
```

→ Instruction quit() laissée en commentaire, qui aurait arrêté le script juste après l'OCR ponctuelle ; désactivée pour laisser le reste du script s'exécuter normalement.

```
84 ## Setup Knowledge Base avec LanceDB (LOCAL) ###
```

→ Commentaire de section annonçant la configuration de la base de connaissances RAG appuyée sur LanceDB.

```
85 def setup_knowledge_base():
```

→ Définit la fonction qui construit et retourne l'objet Knowledge (base de connaissances vectorielle) à partir des fichiers Markdown disponibles.

```
86 """Configure la base de connaissances avec LanceDB"""
```

→ Docstring décrivant le rôle de la fonction setup_knowledge_base.

```
87     markdown_path = MARKDOWN_DIR
```

→ Copie le chemin du dossier Markdown dans une variable locale.

```
89     if not os.path.exists(markdown_path):
```

→ Vérifie que le dossier Markdown existe avant de continuer.

```
90         print(f"Dossier {markdown_path} non trouvé")
```

→ Affiche un message indiquant que le dossier attendu est introuvable.

```
91         return None
```

→ Retourne None pour signaler l'échec de la configuration si le dossier manque ; les agents plus bas gèrent ce cas en désactivant la recherche documentaire plutôt qu'en plantant.

```
93     md_files = [f for f in os.listdir(markdown_path) if f.endswith('.md')]
```

→ Liste tous les fichiers du dossier Markdown dont le nom se termine par .md, en filtrant les autres types de fichiers éventuellement présents.

```
94     if not md_files:
```

→ Vérifie si la liste des fichiers Markdown trouvés est vide.

```
95         print(f"Aucun fichier Markdown dans {markdown_path}")
```

→ Affiche un message indiquant qu'aucun fichier Markdown n'a été trouvé.

```
96         return None
```

→ Retourne None pour signaler l'échec si aucun fichier Markdown n'est disponible.

```
98     print(f"Fichiers Markdown trouvés: {md_files}")
```

→ Affiche la liste des fichiers Markdown détectés dans le dossier, utile pour vérifier que l'ingestion va bien porter sur les bons documents.

```
100     # MarkdownKnowledgeBase(path=..., vector_db=...) a été remplacé par la classe
```

→ Commentaire expliquant la migration depuis l'ancienne classe MarkdownKnowledgeBase d'Agno.

```
101     # unifiée Knowledge : le contenu se charge maintenant via add_content(path=...)
```

→ Suite du commentaire : dans les versions récentes d'Agno, la classe générique Knowledge remplace les classes spécialisées par type de source (Markdown, PDF...), et le chargement du contenu se fait via une méthode add_content plutôt qu'un argument direct du constructeur.

```
102     # plutôt qu'un argument du constructeur + .load(recreate=...).
```

→ Fin du commentaire précisant l'ancienne API remplacée : avant, il fallait passer le chemin au constructeur puis appeler .load(recreate=...) séparément pour déclencher l'indexation.

```
103     knowledge_base = Knowledge(
```

→ Instancie l'objet Knowledge qui représentera la base de connaissances RAG de l'agent.

```
104         vector_db=LanceDb(
```

→ Configure la base vectorielle LanceDb utilisée en arrière-plan par Knowledge pour stocker et interroger les embeddings.

```
105         table_name="import_export_documents",
```

→ Nomme la table LanceDB qui stockera les vecteurs des documents, un peu comme le nom d'une table dans une base SQL classique - permet de séparer plusieurs jeux de documents dans le même fichier LanceDB si besoin.

```
106         uri=LANCEDB_URI, # Stockage local
```

→ Indique le chemin local où LanceDB persistera ses données sur disque ; le commentaire souligne que ce stockage est local (pas de service cloud à provisionner), ce qui simplifie le déploiement mais lie les données à la machine.

```
107         search_type=SearchType.hybrid,
```

→ Configure le type de recherche en mode hybride : cela combine une recherche par mots-clés (texte intégral, qui trouve les correspondances exactes de termes) avec une recherche vectorielle par similarité sémantique (qui trouve des passages au sens proche même sans les mêmes mots). La recherche hybride est généralement plus robuste que l'une ou l'autre seule, car elle rattrape les cas où l'une des deux approches rate la bonne information.

```
108         embedder=MistralEmbedder(api_key=mistral_key),
```

→ Fournit l'embedder Mistral, avec sa clé API, chargé de convertir chaque passage de texte (et chaque question utilisateur) en vecteur numérique pour permettre la comparaison de similarité au cœur du RAG.

```
109     ),
```

→ Ferme la configuration de l'objet LanceDb.

```
110 )
```

→ Ferme l'appel au constructeur de Knowledge.

```
111     knowledge_base.add_content(path=markdown_path)
```

→ Déclenche l'étape d'ingestion : lit les fichiers du dossier Markdown, les découpe en morceaux (chunks), calcule leur embedding via MistralEmbedder, puis les stocke dans la table LanceDB - c'est la construction effective de l'index qui permettra la recherche documentaire au moment des requêtes des agents.

```
113     return knowledge_base
```

→ Retourne l'objet Knowledge configuré et rempli à l'appelant.

```
115 # Configuration de la base de connaissances
```

→ Commentaire annonçant l'appel de configuration de la base de connaissances au niveau module.

```
116 knowledge_base = setup_knowledge_base()
```

→ Appelle la fonction précédente au chargement du script et stocke le résultat (ou None en cas d'échec) dans une variable globale réutilisée par les deux agents définis plus bas.

```
118 import_export_agent = Agent(
```

→ Commence la définition du premier agent Agno, chargé de produire le rapport de conformité import-export ; les paramètres suivants (model, instructions, knowledge, tools...) configurent entièrement son comportement.

```
119     name='Import Export Agent',
```

→ Donne un nom lisible à l'agent, utilisé notamment dans les logs.

```
120     model=Groq(id='openai/gpt-oss-120b', api_key=groq_key),
```

→ Sélectionne le modèle de langage qui alimente cet agent : ici "openai/gpt-oss-120b", un modèle ouvert de 120 milliards de paramètres, exécuté via l'infrastructure d'inférence rapide de Groq plutôt que directement chez OpenAI, avec la clé API Groq lue plus haut.

```
121     instructions=dedent("""
```

→ Ouvre une chaîne multi-lignes dédentée (grâce à dedent) assignée au paramètre instructions : c'est le "prompt système" de l'agent, le texte de cadrage envoyé au modèle à chaque requête pour définir son rôle et ses règles, distinct du message utilisateur

ponctuel.

122 You are an Export-Import Compliance Expert with access to two knowledge sources:

→ Ligne d'instruction définissant le rôle de l'agent en tant qu'expert conformité import-export.

123 - The official UK export law document for electronics (including licences, restrictions, taxes, fees),

→ Décrit la première source de connaissance : le droit britannique de l'export pour l'électronique.

124 - The official Moroccan import law document for electronics (including permits, quantity limits, import duties, VAT, prohibited items, and customs procedures).

→ Décrit la seconde source de connaissance : le droit marocain de l'import pour l'électronique.

126 You also have access to GoogleSearchTools() to find up-to-date, authoritative data when it is missing or unclear.

→ Précise dans le prompt que l'agent peut aussi utiliser une recherche web pour compléter des données manquantes ; cela mentionne GoogleSearchTools() alors que l'outil réellement câblé plus bas sur cet agent est DuckDuckGoTools() - une incohérence mineure entre la documentation du prompt et l'implémentation, sans impact fonctionnel car le modèle appelle simplement l'outil de recherche disponible quel que soit son nom exact dans le texte.

128 Your task is: given detailed product info (type, quantity, raw materials) and the shipment route (UK to Morocco), provide a **full, precise, step-by-step explanation** of:

→ Formule la tâche principale de l'agent à partir des informations produit et de la route d'expédition.

130 1. **UK export process**:

→ Annonce le premier point à traiter : le processus d'export britannique.

131 - Exact licence requirements by name and code,

→ Demande de préciser les exigences de licence par nom et code exacts.

132 - Clear restrictions on product types or materials,

→ Demande de lister clairement les restrictions sur les types de produits ou matériaux.

133 - Specific export taxes, fees, or levies with exact amounts or percentages,

→ Demande de donner les taxes et frais d'export avec montants ou pourcentages précis.

134 - Necessary documentation, including official form names or codes,

→ Demande de lister les documents nécessaires avec leurs noms ou codes officiels.

135 - Compliance rules and penalties with numbers and legal references.

→ Demande de préciser les règles de conformité et sanctions avec références légales chiffrées.

137 2. **Moroccan import process**:

→ Annonce le second point à traiter : le processus d'import marocain.

138 - Precise import permits and certifications required,

→ Demande de préciser les permis et certifications d'importation requis.

139 - Exact quantity restrictions or exemptions,

→ Demande de préciser les restrictions ou exemptions de quantité exactes.

140 - Detailed import duties and VAT rates with percentages or fixed amounts,

→ Demande de détailler les droits de douane et taux de TVA avec pourcentages ou montants fixes.

141 - List of prohibited or restricted items by name and code,

→ Demande de lister les articles interdits ou restreints par nom et code.

142 - Required documents with official titles,

→ Demande de lister les documents requis avec leurs intitulés officiels.

143 - Customs clearance steps with timelines and fees.

→ Demande de détailler les étapes de dédouanement avec délais et frais.

145 Use only ****facts and figures extracted from your knowledge base or from live, authoritative sources****. Do not guess or be vague.

→ Impose de n'utiliser que des faits vérifiés issus de la base de connaissances (RAG) ou de sources fiables trouvées en ligne, une consigne anti-hallucination classique dans un prompt système.

147 If any data is missing in your knowledge, immediately perform a `GoogleSearchTools()` lookup for official government or customs sites and incorporate exact data found.

→ Demande de lancer une recherche web en cas de donnée manquante dans la base de connaissances et d'intégrer les résultats exacts trouvés, illustrant le pattern RAG-plus-recherche-live pour combler les trous de la base documentaire statique.

149 Present the response in clear, professional language, with no filler text or disclaimers.

→ Demande un style de réponse clair et professionnel sans remplissage ni avertissements superflus.

151 Provide the output as a flowing narrative, but include concrete numbers, names, codes, and references wherever applicable.

→ Demande une réponse rédigée en prose continue tout en incluant des données concrètes.

153 The goal is to give the user a complete, actionable export-import compliance report with real numbers and official requirements.

→ Réaffirme l'objectif final : un rapport de conformité complet et exploitable avec des chiffres réels.

155 If data is unavailable, explicitly state that you researched and could not find updated official info.

→ Demande de signaler explicitement l'absence de données à jour lorsque la recherche échoue, plutôt que de laisser un vide ou d'inventer une réponse.

156 `"""),`

→ Ferme la chaîne dédentée et l'appel à `dedent()` pour le paramètre instructions.

157 `expected_output=dedent("""`

→ Ouvre une nouvelle chaîne multi-lignes dédentée pour le paramètre `expected_output` : ce second bloc de prompt ne décrit plus le comportement de l'agent mais le format exact attendu de sa réponse (un gabarit HTML), une technique de prompt engineering qui augmente la fiabilité du formatage de sortie par rapport à une simple description en langage naturel.

158 **IMPORTANT:** Output the final result in raw HTML, starting directly with `<h1>` and without wrapping it in Markdown code blocks like ``html ...``.

→ Exige que la sortie soit du HTML brut commençant par un `h1`, sans bloc de code Markdown - important car ce HTML est ensuite directement concaténé et passé à `html2docx` plus bas, donc tout balisage Markdown parasite casserait la conversion.

160 `<h1 style="text-align: center;"> Import-Export Compliance Report</h1>`

→ Modèle HTML donnant le titre principal centré du rapport attendu.

161 `<hr>`

→ Modèle HTML insérant une ligne de séparation horizontale.

163 `<h2>UK Export Process</h2>`

→ Modèle HTML donnant le titre de la section sur le processus d'export britannique.

164 `<p><strong>Export License:</strong> [Explain if an export licence is required for the product. Specify licence type and official code if available.]</p>`

→ Modèle de paragraphe attendu pour la sous-section licence d'export, avec placeholder d'instruction que le modèle doit remplacer par du contenu réel.

165 `<p><strong>Restrictions:</strong> [Mention any restrictions on product types, raw materials, or dual-use classifications found in the UK export law document.]</p>`

→ Modèle de paragraphe attendu pour la sous-section restrictions, avec placeholder d'instruction.

- 166 <p>Export Taxes: [Provide specific export taxes or fees applicable, with exact amounts or percentages, citing the document.]</p>
→ Modèle de paragraphe attendu pour la sous-section taxes d'export, avec placeholder d'instruction.
- 167 <p>Documentation & Compliance: [Describe required paperwork and compliance steps before shipment.]</p>
→ Modèle de paragraphe attendu pour la sous-section documentation et conformité.
- 169 <h2>Moroccan Import Process</h2>
→ Modèle HTML donnant le titre de la section sur le processus d'import marocain.
- 170 <p>Import Licences & Certifications: [Detail import permits, licences, or certifications required for the product according to the Moroccan import law document.]</p>
→ Modèle de paragraphe attendu pour les licences et certifications d'import.
- 171 <p>Quantity Limits: [State any quantity limits or restrictions, quoting exact figures or thresholds.]</p>
→ Modèle de paragraphe attendu pour les limites de quantité à l'import.
- 172 <p>Prohibited Materials: [Specify prohibited items or materials mentioned in the document.]</p>
→ Modèle de paragraphe attendu pour les matériaux interdits.
- 173 <p>Duties & VAT: [Give import duties and VAT rates with percentages or fixed amounts, as documented.]</p>
→ Modèle de paragraphe attendu pour les droits de douane et la TVA.
- 174 <p>Customs Procedures: [List all required customs documents and procedures.]</p>
→ Modèle de paragraphe attendu pour les procédures douanières.
- 176 <h2>Taxes and Compliance Details</h2>
→ Modèle HTML donnant le titre de la section détails fiscaux et conformité.
- 177 <p>Tax Timing & Calculation: [Summarize how taxes and fees are calculated and at which stage (export or import) they apply, based on the findings.]</p>
→ Modèle de paragraphe attendu sur le calcul et le moment d'application des taxes.
- 178 <p>Compliance Obligations: [Explain compliance obligations and potential penalties for non-compliance extracted from the documents.]</p>
→ Modèle de paragraphe attendu sur les obligations de conformité et sanctions.
- 179 <p>Practical Advice: [Include any additional practical advice referenced in the documents for smooth customs clearance.]</p>
→ Modèle de paragraphe attendu pour des conseils pratiques de dédouanement.
- 181 <h2>Summary and Recommendations</h2>
→ Modèle HTML donnant le titre de la section résumé et recommandations.
- 182 <p>[Provide a concise summary of the entire export-import process highlighting key points and actionable next steps based solely on the documents' contents.]</p>
→ Modèle de paragraphe attendu pour un résumé final concis avec recommandations.
- 183 """),
→ Ferme la chaîne dédentée et l'appel à dedent() pour le paramètre expected_output.
- 184 knowledge=knowledge_base,
→ Associe la base de connaissances RAG construite plus haut à l'agent : c'est ce paramètre qui donne concrètement à l'agent la capacité de rechercher dans les documents indexés (LanceDB) lorsqu'il répond.

185 `search_knowledge=True if knowledge_base else False,`

→ Active la recherche dans la base de connaissances uniquement si celle-ci a bien été créée ; cette condition évite qu'Agno essaie d'interroger un objet `knowledge_base` à `None` si l'ingestion a échoué plus haut (dossier Markdown manquant, par exemple).

186 `tools=[DuckDuckGoTools()]`

→ Fournit à l'agent l'outil de recherche web DuckDuckGo comme capacité supplémentaire de fonction calling, en complément du RAG, pour aller chercher une information manquante ou plus récente que les documents indexés.

187)

→ Ferme l'appel au constructeur Agent commencé à la ligne 118.

189 # Le contenu est déjà chargé dans `setup_knowledge_base()` via `add_content()` ;

→ Commentaire rappelant que le chargement du contenu se fait désormais dans `setup_knowledge_base`.

190 # l'ancienne étape séparée `.knowledge.load(recreate=...)` n'existe plus dans la

→ Suite du commentaire décrivant l'ancienne API de chargement supprimée avec la migration vers la classe Knowledge unifiée.

191 # nouvelle classe Knowledge.

→ Fin du commentaire sur la nouvelle classe Knowledge.

192 if not knowledge_base:

→ Vérifie si la base de connaissances n'a pas pu être créée (par exemple si aucun fichier Markdown n'était présent).

193 `print("Aucune base de connaissances - utilisation de GoogleSearch uniquement")`

→ Affiche un message indiquant que seule la recherche web (via l'outil configuré) sera utilisée en secours, l'agent perdant simplement l'accès au RAG documentaire sans que le script s'arrête.

195 `local_regulations_agent = Agent(`

→ Commence la définition d'un second agent Agno, spécialisé sur les réglementations locales du pays d'importation (distinct du premier agent qui traite l'export/import UK-Maroc au sens douanier).

196 `name='Local Regulations Agent',`

→ Donne un nom lisible à ce second agent.

197 `model=Groq(id='openai/gpt-oss-120b', api_key=groq_key),`

→ Configure le même modèle Groq que le premier agent, avec la clé API Groq, pour garder cohérence et coûts d'inférence maîtrisés.

198 `instructions=dedent("""`

→ Ouvre la chaîne d'instructions dédentée (prompt système) pour cet agent.

199 `You are a Local Regulation Checker Agent.`

→ Ligne d'instruction définissant le rôle de l'agent en tant que vérificateur de réglementation locale.

201 `Your task is to research local regulations in a specific country related to the **imported product**. Focus on the rules and requirements for legally selling that product in the local market after import.`

→ Décrit la tâche : rechercher les règles locales de vente du produit importé, une fois celui-ci arrivé sur le marché cible (complémentaire au premier agent, centré lui sur le passage de la frontière).

203 `Based on the provided product and target import country, gather clear and up-to-date information from official sources (e.g., government websites, trade portals, customs authorities). Use reliable data to answer the following:`

→ Demande de rassembler des informations à jour depuis des sources officielles pour répondre aux points suivants ; notez que cet agent n'a pas de base de connaissances RAG dédiée (pas de paramètre `knowledge=`) et s'appuie donc entièrement sur la recherche web en direct via son outil.

205 1. Product Legality:

→ Annonce le premier point à traiter : la légalité du produit.

206 - Is the product legal to sell in the country?

→ Question à traiter : la légalité de la vente du produit dans le pays.

207 - Are there any restrictions, bans, or conditions?

→ Question à traiter : l'existence de restrictions, interdictions ou conditions.

209 2. Labeling & Standards:

→ Annonce le deuxième point à traiter : étiquetage et normes.

210 - What are the mandatory labeling, packaging, and safety standards?

→ Question à traiter : les normes obligatoires d'étiquetage, d'emballage et de sécurité.

211 - Are there local language, cultural, or environmental compliance rules?

→ Question à traiter : les règles locales de langue, culture ou environnement.

213 3. Certification Requirements:

→ Annonce le troisième point à traiter : les exigences de certification.

214 - Is local testing, registration, or certification needed before the product can be sold?

→ Question à traiter : la nécessité de tests, d'enregistrement ou de certification locaux.

215 - Mention specific marks or agencies involved (e.g., CE, FDA, ISO, halal).

→ Demande de citer les labels ou organismes concernés, avec des exemples.

217 4. Licenses for Selling:

→ Annonce le quatrième point à traiter : les licences de vente.

218 - Are there any business or product-specific licenses required for sale?

→ Question à traiter : l'existence de licences commerciales ou produit requises.

219 - Which government bodies issue these, and what is the application process?

→ Question à traiter : les organismes émetteurs et le processus de demande.

221 5. Distribution Rules:

→ Annonce le cinquième point à traiter : les règles de distribution.

222 - Are there any constraints on how the product can be distributed (retail, online, marketplaces)?

→ Question à traiter : les contraintes sur les canaux de distribution du produit.

223 - Is a local distributor or legal entity required?

→ Question à traiter : la nécessité d'un distributeur local ou d'une entité légale.

225 6. Taxes & Duties:

→ Annonce le sixième point à traiter : taxes et droits.

226 - What import duties, VAT, or additional regulatory fees apply to this product?

→ Question à traiter : les droits d'import, la TVA et les frais réglementaires applicables.

227 - Are there trade agreements, exemptions, or thresholds that affect this?

→ Question à traiter : l'effet d'accords commerciaux, exemptions ou seuils.

229 Only return factual and country-specific results with references when possible. If something is unclear or unavailable, state that clearly.

→ Demande des réponses factuelles et référencées, avec mention explicite des incertitudes plutôt qu'une réponse inventée.

230 `"""),`

→ Ferme la chaîne d'instructions dédentée pour ce second agent.

231 `expected_output=dedent("""`

→ Ouvre la chaîne dédentée du paramètre `expected_output` pour ce second agent, définissant à nouveau un gabarit HTML strict pour la sortie.

232 `IMPORTANT: Output the final result in raw HTML, starting directly with <h1> and without wrapping it in Markdown code blocks like `html ...`.`

→ Exige une sortie HTML brute commençant par un `h1`, sans bloc de code Markdown, pour la même raison que le premier agent : cette sortie sera concaténée puis convertie en DOCX.

234 `<h1 style="text-align: center;"> Local Regulation Report</h1>`

→ Modèle HTML donnant le titre principal centré du rapport de réglementation locale.

235 `<hr>`

→ Modèle HTML insérant une ligne de séparation horizontale.

237 `<p><strong>Product:</strong> [Product name]</p>`

→ Modèle de paragraphe attendu affichant le nom du produit.

238 `<p><strong>Import Country:</strong> [Country name]</p>`

→ Modèle de paragraphe attendu affichant le pays d'importation.

240 `<h2>1. Product Legality</h2>`

→ Modèle HTML donnant le titre de la sous-section légalité du produit.

241 `<p>[Summary of whether the product is legal to sell, with any conditions, bans, or restricted status. Include reference links if available.]</p>`

→ Modèle de paragraphe attendu résumant la légalité du produit avec liens de référence.

243 `<h2>2. Labeling & Standards</h2>`

→ Modèle HTML donnant le titre de la sous-section étiquetage et normes.

244 `<p>[Details on required product labels (e.g., language, safety, expiry date), packaging rules, and any applicable national or international standards.]</p>`

→ Modèle de paragraphe attendu détaillant les exigences d'étiquetage et de normes.

246 `<h2>3. Certification Requirements</h2>`

→ Modèle HTML donnant le titre de la sous-section exigences de certification.

247 `<p>[List of certifications required to sell the product locally, the issuing authority, and any necessary pre-market approval/testing.]</p>`

→ Modèle de paragraphe attendu listant les certifications requises et l'autorité émettrice.

249 `<h2>4. Licenses for Selling</h2>`

→ Modèle HTML donnant le titre de la sous-section licences de vente.

250 `<p>[Information on whether a license is needed to sell the product and what type. Mention the issuing body and key application steps.]</p>`

→ Modèle de paragraphe attendu sur le besoin de licence et les étapes de demande.

252 `<h2>5. Distribution Rules</h2>`

→ Modèle HTML donnant le titre de la sous-section règles de distribution.

253 `<p>[Summary of any rules or limits on selling methods – e.g., retail, e-commerce, need for local presence or partnership.]</p>`

→ Modèle de paragraphe attendu résumant les règles de distribution du produit.

255 <h2>6. Taxes & Duties</h2>

→ Modèle HTML donnant le titre de la sous-section taxes et droits.

256 <p>[Overview of import taxes, VAT, excise duties, or regulatory fees. Note any applicable thresholds, exemptions, or relevant trade agreements.]</p>

→ Modèle de paragraphe attendu donnant un aperçu des taxes et droits applicables.

258 <h2>Sources</h2>

→ Modèle HTML donnant le titre de la section listant les sources utilisées, une pratique de traçabilité importante quand la réponse provient d'une recherche web en direct plutôt que d'une base documentaire figée.

259

→ Modèle HTML ouvrant une liste à puces pour les sources.

260 [Link 1]

→ Modèle HTML représentant un premier élément de lien source.

261 [Link 2]

→ Modèle HTML représentant un second élément de lien source.

262 ...

→ Modèle HTML indiquant que d'autres liens peuvent être ajoutés.

263

→ Ferme la liste à puces des sources.

264 """),

→ Ferme la chaîne dédentée et l'appel à `dedent()` pour le paramètre `expected_output`.

265 tools=[DuckDuckGoTools()],

→ Fournit à cet agent l'outil de recherche web DuckDuckGo, ici sa seule source d'information externe puisqu'il ne dispose pas de base de connaissances RAG dédiée.

266)

→ Ferme l'appel au constructeur Agent commencé à la ligne 195.

268 # Fonction principale

→ Commentaire annonçant la fonction principale de génération de rapport.

269 def generate_report(product_description):

→ Définit la fonction qui orchestre la génération complète du rapport pour un produit donné, en enchaînant les deux agents puis la conversion DOCX.

270 """Génère un rapport d'import-export complet"""

→ Docstring décrivant le rôle de la fonction `generate_report`.

271 try:

→ Ouvre un bloc try pour capturer toute exception survenant pendant la génération du rapport (appel LLM échoué, erreur de conversion, etc.).

272 print(f"Génération du rapport pour: {product_description}")

→ Affiche un message indiquant le produit pour lequel le rapport est généré.

274 output_html = ""

→ Initialise une chaîne vide qui accumulera le HTML final du rapport, section par section.

276 # Agent 1: Import-Export Compliance

→ Commentaire annonçant l'exécution du premier agent de conformité.

277 `print("Exécution de l'agent Import-Export...")`
→ Affiche un message indiquant le démarrage de l'agent Import-Export.

278 `agent1 = import_export_agent.run(product_description)`
→ Exécute l'agent de conformité import-export : la méthode `run()` envoie le texte du produit comme message utilisateur, déclenche le cycle complet (recherche RAG, éventuel appel d'outil de recherche web, génération de la réponse par le LLM) et retourne un objet de réponse.

279 `output_html += agent1.content`
→ Ajoute le contenu HTML retourné par le premier agent (le texte généré par le modèle, conforme au gabarit `expected_output` défini plus haut) à la chaîne de sortie.

280 `output_html += "<hr>"`
→ Ajoute une ligne de séparation HTML entre les deux sections du rapport.

282 `# Agent 2: Local Regulations`
→ Commentaire annonçant l'exécution du second agent de réglementations locales.

283 `print("Exécution de l'agent Local Regulations...")`
→ Affiche un message indiquant le démarrage de l'agent Local Regulations.

284 `agent2 = local_regulations_agent.run(product_description)`
→ Exécute le second agent avec la même description de produit, qui va cette fois chercher les règles de revente locales via recherche web.

285 `output_html += agent2.content`
→ Ajoute le contenu HTML retourné par le second agent à la chaîne de sortie.

287 `# Conversion en DOCX`
→ Commentaire annonçant la conversion du HTML accumulé en document Word.

288 `print("Conversion en DOCX...")`
→ Affiche un message indiquant le début de la conversion en DOCX.

289 `docx_bytes = html2docx(output_html, title="Import-Export Regulation Report")`
→ Convertit la chaîne HTML complète (produite par les deux agents) en un document DOCX en mémoire (objet de type flux d'octets), avec un titre de document donné - c'est ici que le contenu généré par les LLM devient un livrable bureautique standard.

291 `# Sauvegarde`
→ Commentaire annonçant l'écriture du fichier DOCX sur le disque.

292 `filename = "import_export_regulation_report.docx"`
→ Définit le nom du fichier de sortie pour le rapport généré.

293 `with open(filename, "wb") as f:`
→ Ouvre le fichier de sortie en écriture binaire ("wb"), nécessaire car un .docx est un format binaire (une archive ZIP contenant du XML), pas du texte brut.

294 `f.write(docx_bytes.getvalue())`
→ Écrit les octets du document DOCX généré (récupérés via `getvalue()` sur le flux en mémoire) dans le fichier ouvert.

296 `print(f" Rapport sauvegardé: {filename}")`
→ Affiche un message confirmant la sauvegarde réussie du rapport avec son nom de fichier ; c'est ce genre de print avec emoji qui justifiait la reconfiguration d'encodage en tout début de script.

297 `return filename`
→ Retourne le nom du fichier généré à l'appelant.

299 `except Exception as e:`
→ Capture toute exception survenue pendant le bloc try précédent (appel agent, conversion, écriture disque), pour éviter que le

script ne plante brutalement sans message clair.

300 `print(f"Erreur lors de la génération: {e}")`

→ Affiche le message d'erreur de l'exception survenue.

301 `return None`

→ Retourne None pour signaler l'échec de la génération du rapport.

303 # Exécution

→ Commentaire annonçant le point d'entrée exécutable du script.

304 `if __name__ == "__main__":`

→ Vérifie que le script est exécuté directement (et non importé comme module dans un autre fichier Python), un idiome standard pour séparer code réutilisable et code d'exécution.

305 `product = "An electronic air purifier exported from the UK and imported into Morocco"`

→ Définit une description de produit d'exemple utilisée pour lancer une génération de rapport de démonstration.

306 `generate_report(product)`

→ Appelle la fonction `generate_report` avec la description de produit d'exemple, déclenchant l'ensemble du pipeline (deux agents + conversion DOCX) lorsqu'on exécute le script directement.

Projet 35 — 35-HR-Assistant

Assistant RH (conges, projets, approbations) via Airtable

Objectif

Assistant RH conversationnel qui répond aux questions des employés sur leurs congés, leurs projets en cours et le statut de leurs demandes d'approbation, en s'appuyant sur des données stockées dans une base Airtable consultée en temps réel.

Code source, ligne par ligne

35-HR-Assistant/rhAgent.py

Backend Flask + agents Agno qui gèrent les congés, les projets et les approbations RH via Airtable, avec exposition d'une API HTTP et d'un tunnel ngrok public.

556 lignes de code annotées ci-dessous.

1 # Imports

→ Commentaire annonçant le bloc d'imports du fichier.

2 from agno.agent import Agent

→ Importe la classe Agent du framework Agno, la brique de base pour construire un agent IA (un LLM doté d'un rôle, d'instructions, d'outils et éventuellement d'une mémoire/persistance) - voir le fichier full_import_export_agent.py pour la définition complète d'Agno.

3 from agno.models.openai import OpenAIChat

→ Importe la classe OpenAIChat d'Agno, un wrapper autour de l'API Chat Completions d'OpenAI : elle sert ici de "modèle" pour tous les agents de ce fichier, contrairement au projet précédent qui utilisait Groq.

4 from dotenv import load_dotenv

→ Importe la fonction qui charge les variables d'environnement depuis un fichier .env, pratique standard pour garder les clés API hors du code source.

5 import os

→ Importe le module standard pour interagir avec le système de fichiers et les variables d'environnement.

6 from agno.tools.file import FileTools

→ Importe FileTools, un outil Agno prêt à l'emploi qui donne à un agent la capacité de lire/écrire des fichiers locaux via fonction calling (le LLM peut décider d'appeler ces fonctions pendant sa génération plutôt que de se limiter à produire du texte).

7 from agno.tools.calculator import CalculatorTools

→ Importe CalculatorTools, un outil Agno qui donne à l'agent des fonctions de calcul arithmétique fiables - utile car les LLM sont notoirement peu fiables pour faire des calculs exacts "de tête", donc on délègue ce travail à du code déterministe appelé en tool use.

8 from agno.tools.email import EmailTools

→ Importe EmailTools, un outil Agno permettant à l'agent d'envoyer des emails (par exemple pour notifier les RH ou un employé), toujours selon le même principe de fonction calling.

9 from agno.team import Team

→ Importe la classe Team du framework Agno, qui permet de regrouper plusieurs agents spécialisés sous un agent "manager" commun. C'est le pattern d'architecture multi-agent : plutôt qu'un seul agent généraliste qui essaie de tout faire, on a plusieurs agents experts d'un domaine (congés, projets, approbations) et un routeur qui aiguille chaque requête vers le bon agent selon

son intention.

10 from agno.storage.sqlite import SqliteStorage

→ Importe SqliteStorage, le backend de persistance SQLite d'Agno utilisé pour sauvegarder l'historique des sessions de conversation (mémoire) sur disque, afin qu'une équipe d'agents se souvienne du contexte entre deux appels plutôt que de repartir de zéro à chaque requête.

11 from pyairtable import Api

→ Importe la classe client de pyairtable, le SDK Python non-officiel pour l'API Airtable. Airtable est un service de type "tableur augmenté" (base de données relationnelle simplifiée avec interface tableur) utilisé ici comme système de stockage des données RH (employés, congés, projets).

12 from flask import Flask, request, render_template_string, jsonify

→ Importe les éléments de Flask nécessaires : Flask est un micro-framework web Python léger pour créer des applications et API HTTP. Flask (classe de l'application), request (accès aux données de la requête entrante), render_template_string (rendu de gabarits HTML avec injection de variables) et jsonify (sérialisation de réponses au format JSON) sont les briques utilisées plus bas pour exposer des routes.

13 from flask_cors import CORS

→ Importe l'extension Flask-CORS. CORS (Cross-Origin Resource Sharing) est un mécanisme de sécurité des navigateurs qui bloque par défaut les requêtes JavaScript d'un site vers un domaine/port différent de celui qui a servi la page ; cette extension permet d'autoriser explicitement certaines origines (ici le frontend Streamlit sur localhost:8000) à appeler cette API.

14 import uuid

→ Importe le module standard permettant de générer des identifiants uniques (UUID), utilisé plus bas pour créer un identifiant de demande de congé impossible à deviner ou à faire entrer en collision.

15 import threading

→ Importe le module standard permettant d'exécuter du code dans des threads séparés - utilisé ici pour lancer le serveur Flask en arrière-plan pendant que le thread principal reste libre pour d'autres tâches (garder le process vivant, gérer ngrok).

16 import time

→ Importe le module standard fournissant des fonctions liées au temps (pause, formatage de dates).

17 import logging

→ Importe le module standard de journalisation (logging), qui permet d'émettre des messages structurés avec un niveau de gravité (info, warning, error) plutôt que de simples print(), facilitant le diagnostic en production.

18 import requests

→ Importe la bibliothèque tierce requests, le client HTTP standard de facto en Python, utilisée plus bas pour appeler l'API externe Retell.

19 import pyngrok.ngrok as ngrok

→ Importe le module ngrok du package pyngrok. ngrok est un service qui crée un tunnel sécurisé entre une URL publique sur Internet et un serveur tournant en local (par exemple sur localhost:5000) : cela permet à des services externes (ici l'API vocale Retell, ou un email de RH cliqué depuis n'importe où) d'atteindre ce serveur Flask local sans avoir à le déployer sur un vrai hébergeur.

21 # Set up logging

→ Commentaire annonçant la configuration du système de journalisation.

22 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

→ Configure le logging global : le niveau INFO signifie que tous les messages de gravité info et supérieure (warning, error) seront affichés, et le format précise qu'chaque ligne de log inclura l'horodatage, le niveau de gravité et le message.

23 logger = logging.getLogger(__name__)

→ Crée un logger nommé d'après le module courant, l'instance utilisée dans tout le fichier pour émettre les messages de log (logger.info, logger.error, etc.).

25 load_dotenv()

→ Charge les variables d'environnement définies dans le fichier .env du projet (clés API, tokens) dans l'environnement du processus.

27 # Load environment variables

→ Commentaire annonçant la lecture des variables d'environnement utilisées par le script.

28 api_key_openai = os.getenv("OPENAI_API_KEY")

→ Lit la clé API OpenAI depuis les variables d'environnement, nécessaire pour tous les appels aux modèles OpenAIChat des agents définis plus bas.

29 AIRTABLE_TOKEN = os.getenv("AIRTABLE_TOKEN")

→ Lit le jeton d'authentification Airtable (Personal Access Token) depuis les variables d'environnement, nécessaire pour lire/écrire dans les tables Airtable.

30 BASE_ID = os.getenv("AIRTABLE_BASE_ID")

→ Lit l'identifiant de la base Airtable ciblée depuis les variables d'environnement (une "base" Airtable est l'équivalent d'une base de données, contenant plusieurs tables).

31 NGROK_AUTH_TOKEN = os.getenv("NGROK_AUTH_TOKEN")

→ Lit le jeton d'authentification ngrok depuis les variables d'environnement, nécessaire pour ouvrir un tunnel public associé au compte ngrok de l'utilisateur.

32 HR_EMAIL = os.getenv("HR_EMAIL")

→ Lit l'adresse email des ressources humaines depuis les variables d'environnement, destinataire des notifications de demande de congé.

33 db_file = os.getenv("SQLITE_DB_FILE", "hr_assistant_sessions.db")

→ Lit le nom du fichier SQLite depuis l'environnement, avec une valeur par défaut ("hr_assistant_sessions.db") si la variable n'est pas définie ; ce fichier stockera l'historique des sessions de l'équipe d'agents.

35 # Validation des variables d'environnement critiques

→ Commentaire annonçant la validation des variables d'environnement indispensables.

36 required_env_vars = {

→ Ouvre un dictionnaire recensant les variables d'environnement obligatoires et leurs valeurs, pour pouvoir les vérifier en bloc juste après plutôt que de multiplier les "if not X" séparés.

37 "OPENAI_API_KEY": api_key_openai,

→ Ajoute la clé OpenAI à la liste des variables requises à vérifier.

38 "AIRTABLE_TOKEN": AIRTABLE_TOKEN,

→ Ajoute le jeton Airtable à la liste des variables requises à vérifier.

39 "AIRTABLE_BASE_ID": BASE_ID,

→ Ajoute l'identifiant de base Airtable à la liste des variables requises à vérifier.

40 "NGROK_AUTH_TOKEN": NGROK_AUTH_TOKEN,

→ Ajoute le jeton ngrok à la liste des variables requises à vérifier.

41 "HR_EMAIL": HR_EMAIL

→ Ajoute l'email RH à la liste des variables requises à vérifier.

42 }

→ Ferme le dictionnaire des variables d'environnement requises.

44 missing_vars = [var for var, value in required_env_vars.items() if not value]

→ Construit, par compréhension de liste, la liste des noms de variables dont la valeur est vide ou absente, en une seule passe sur le dictionnaire.

45 if missing_vars:

→ Vérifie si au moins une variable d'environnement requise est manquante.

46 logger.error(f"Variables d'environnement manquantes: {' '.join(missing_vars)}")

→ Journalise en erreur la liste des variables d'environnement manquantes, pour un diagnostic rapide au démarrage.

47 raise ValueError(f"Variables d'environnement requises manquantes: {' '.join(missing_vars)}")

→ Lève une exception ValueError pour interrompre le script immédiatement si des variables critiques manquent : un pattern "fail fast" qui évite de laisser tourner un service RH mal configuré silencieusement.

49 # Airtable table configuration

→ Commentaire annonçant la configuration de la connexion aux tables Airtable.

50 try:

→ Ouvre un bloc try pour capturer les erreurs lors de la connexion à Airtable (jeton invalide, base introuvable, panne réseau...).

51 api = Api(AIRTABLE_TOKEN)

→ Instancie le client Airtable avec le jeton d'authentification, l'objet racine à partir duquel toutes les opérations Airtable sont effectuées.

52 base = api.base(BASE_ID)

→ Récupère la référence de la base Airtable ciblée à partir de son identifiant.

53 holiday_table = base.table("employee_holidays")

→ Récupère la référence de la table Airtable contenant les données de congés des employés (solde, jours pris...).

54 project_table = base.table("employee_projects")

→ Récupère la référence de la table Airtable contenant les données de projets des employés.

55 request_table = base.table("holiday_requests")

→ Récupère la référence de la table Airtable contenant les demandes de congés en attente de traitement.

56 logger.info("Connexion aux tables Airtable réussie")

→ Journalise en info la réussite de la connexion aux tables Airtable.

57 except Exception as e:

→ Capture toute exception survenue pendant la connexion à Airtable.

58 logger.error(f"Erreur de connexion Airtable: {e}")

→ Journalise en erreur le message de l'exception survenue lors de la connexion.

59 raise

→ Relève l'exception capturée (raise sans argument la propage telle quelle) pour interrompre le script en cas d'échec de connexion : Airtable étant la source de vérité de toutes les données RH, continuer sans connexion valide n'aurait pas de sens.

61 # Flask app for approval/disapproval

→ Commentaire annonçant la création de l'application Flask servant les pages d'approbation.

62 app = Flask(__name__)

→ Instancie l'application Flask en utilisant le nom du module courant, l'objet central autour duquel toutes les routes HTTP seront déclarées.

64 CORS(app, resources={r"/chat": {"origins": "http://localhost:8000"}})

→ Active CORS uniquement sur la route /chat, en n'autorisant que les requêtes provenant de l'origine http://localhost:8000 (le frontend Streamlit) : c'est une restriction volontairement étroite plutôt qu'un CORS ouvert à tous ("*"), pour limiter la surface d'attaque de cette API.

65 `CORS(app, resources={r"/text": {"origins": "http://localhost:8000"}})`

→ Active de la même façon CORS sur la route /text, avec la même origine autorisée.

67 `APPROVAL_PAGE = ""`

→ Ouvre une chaîne multi-lignes stockant le gabarit HTML de la page d'approbation de congé, qui sera servi directement par Flask via `render_template_string` plus bas - une approche simple où le HTML est écrit en dur dans le code Python plutôt que dans un fichier .html séparé.

68 `<!DOCTYPE html>`

→ Déclare le document HTML de la page d'approbation selon le standard HTML5.

69 `<html>`

→ Ouvre la balise racine du document HTML.

70 `<head>`

→ Ouvre la section d'en-tête du document HTML.

71 `<title>Holiday Request Approval</title>`

→ Définit le titre affiché dans l'onglet du navigateur.

72 `<style>`

→ Ouvre le bloc de styles CSS intégré à la page.

73 `body { font-family: Arial, sans-serif; margin: 40px; }`

→ Définit la police et la marge globale du corps de la page.

74 `.container { max-width: 600px; margin: auto; }`

→ Définit une largeur maximale et centre horizontalement le conteneur principal.

75 `.button { padding: 10px 20px; margin: 10px; font-size: 16px; cursor: pointer; }`

→ Définit l'apparence générale des boutons de la page.

76 `.approve { background-color: #4CAF50; color: white; border: none; }`

→ Définit la couleur de fond verte et le texte blanc du bouton d'approbation.

77 `.disapprove { background-color: #f44336; color: white; border: none; }`

→ Définit la couleur de fond rouge et le texte blanc du bouton de refus.

78 `</style>`

→ Ferme le bloc de styles CSS.

79 `</head>`

→ Ferme la section d'en-tête du document HTML.

80 `<body>`

→ Ouvre le corps visible de la page HTML.

81 `<div class="container">`

→ Ouvre la div conteneur principale de la page.

82 `<h2>Holiday Request Approval</h2>`

→ Affiche le titre de la page d'approbation de demande de congé.

83 `<p><strong>Employee ID:</strong> {{ employee_id }}</p>`

→ Affiche l'identifiant de l'employé via une variable de gabarit Jinja2 (`{{ ... }}`) : Jinja2 est le moteur de templating intégré à Flask, qui remplace ces marqueurs par la valeur réelle passée par `render_template_string` au moment du rendu, tout en échappant automatiquement les caractères spéciaux HTML pour éviter des failles d'injection.

84 `<p><strong>Name:</strong> {{ name }}</p>`

→ Affiche le nom de l'employé via une variable de gabarit Jinja.

85 <p>Requested Days: {{ requested_days }}</p>
→ Affiche le nombre de jours demandés via une variable de gabarit Jinja.

86 <p>Remaining Holiday Days: {{ remaining_days }}</p>
→ Affiche le solde de jours de congé restants via une variable de gabarit Jinja.

87 <p>Request Date: {{ request_date }}</p>
→ Affiche la date de la demande via une variable de gabarit Jinja.

89 <form method="POST" action="/process/{{ request_id }}">
→ Ouvre un formulaire qui soumettra en POST vers la route dynamique de traitement de la demande, l'identifiant de demande étant injecté directement dans l'URL cible.

90 <button type="submit" name="action" value="approved" class="button
approve">Approve</button>
→ Définit le bouton qui soumet le formulaire avec l'action "approved".

91 <button type="submit" name="action" value="disapproved" class="button
disapprove">Disapprove</button>
→ Définit le bouton qui soumet le formulaire avec l'action "disapproved".

92 </form>
→ Ferme le formulaire d'approbation ou de refus.

93 </div>
→ Ferme la div conteneur principale.

94 </body>
→ Ferme le corps de la page HTML.

95 </html>
→ Ferme la balise racine du document HTML.

96 ""
→ Ferme la chaîne multi-lignes contenant le gabarit APPROVAL_PAGE.

98 # Variable globale pour stocker l'URL publique
→ Commentaire annonçant la déclaration de la variable globale d'URL publique.

99 public_url = None
→ Initialise la variable globale public_url à None avant qu'elle ne soit définie plus bas par le tunnel ngrok (ou par une valeur de repli locale) ; cette URL sera insérée dans les emails envoyés aux RH pour pointer vers la page d'approbation.

101 @app.route('/request/<request_id>')
→ Déclare une route Flask GET dynamique : le segment <request_id> dans l'URL est capturé et transmis comme argument à la fonction de vue associée, un mécanisme de routage à paramètres standard des frameworks web.

102 def show_request(request_id):
→ Définit la fonction de vue affichant la page d'approbation pour une demande donnée.

103 try:
→ Ouvre un bloc try pour capturer les erreurs lors de la récupération de la demande.

104 logger.info(f"Attempting to retrieve request with request_id: {request_id}")
→ Journalise en info la tentative de récupération de la demande par son identifiant.

105 records = request_table.all(formula=f"{{request_id}}='{{request_id}}'")
→ Interroge Airtable avec une formule de filtre (la syntaxe de formules d'Airtable, similaire à des formules de tableur) pour

récupérer les enregistrements dont le champ `request_id` correspond exactement à la valeur recherchée.

```
106         if not records:
```

→ Vérifie si aucun enregistrement n'a été trouvé pour cet identifiant.

```
107         logger.error(f"No record found for request_id: {request_id}")
```

→ Journalise en erreur l'absence d'enregistrement trouvé.

```
108         return "Request not found", 404
```

→ Retourne une réponse HTTP avec le code de statut 404 (Not Found) et un message d'erreur, la convention Flask pour renvoyer un code de statut différent de 200 en retournant un tuple (corps, code).

```
109         record = records[0]
```

→ Sélectionne le premier enregistrement trouvé (censé être unique, `request_id` agissant comme clé).

```
110         fields = record['fields']
```

→ Extrait le dictionnaire des champs de l'enregistrement Airtable (chaque enregistrement Airtable est structuré en `{"id": ..., "fields": {...}}`).

```
111         logger.info(f"Retrieved record: {record}")
```

→ Journalise en info l'enregistrement complet récupéré.

```
112         return render_template_string(
```

→ Commence le rendu du gabarit HTML `APPROVAL_PAGE` avec les données de la demande injectées dans les variables Jinja.

```
113             APPROVAL_PAGE,
```

→ Fournit la chaîne de gabarit HTML à rendre comme premier argument.

```
114             employee_id=fields.get("employee_id"),
```

→ Passe l'identifiant de l'employé au contexte du gabarit.

```
115             name=fields.get("full_name"),
```

→ Passe le nom complet de l'employé au contexte du gabarit.

```
116             requested_days=fields.get("requested_days"),
```

→ Passe le nombre de jours demandés au contexte du gabarit.

```
117             remaining_days=fields.get("remaining_days"),
```

→ Passe le solde de jours restants au contexte du gabarit.

```
118             request_date=fields.get("request_date"),
```

→ Passe la date de la demande au contexte du gabarit.

```
119             request_id=request_id
```

→ Passe l'identifiant de la demande au contexte du gabarit pour construire l'URL du formulaire.

```
120         )
```

→ Ferme l'appel à `render_template_string` commencé plus haut.

```
121     except Exception as e:
```

→ Capture toute exception survenue pendant la récupération ou le rendu de la demande.

```
122         logger.error(f"Error retrieving request {request_id}: {str(e)}")
```

→ Journalise en erreur le message de l'exception survenue.

```
123         return f"Error retrieving request: {str(e)}", 500
```

→ Retourne une réponse HTTP 500 (Internal Server Error) avec le détail de l'erreur.

```
125 @app.route('/process/<request_id>', methods=['POST'])
```

→ Déclare une route Flask POST dynamique pour traiter la décision sur une demande ; methods=['POST'] restreint cette route aux requêtes de soumission de formulaire, contrairement à la route GET précédente qui ne fait qu'afficher la page.

```
126 def process_request(request_id):
```

→ Définit la fonction de vue traitant l'approbation ou le refus d'une demande.

```
127     try:
```

→ Ouvre un bloc try pour capturer les erreurs lors du traitement de la demande.

```
128         logger.info(f"Processing request with request_id: {request_id}")
```

→ Journalise en info le début du traitement de la demande.

```
129         action = request.form.get('action')
```

→ Récupère la valeur du champ "action" soumis par le formulaire HTML (approved/disapproved), via request.form qui donne accès aux données envoyées en POST classique (form-urlencoded), par opposition à request.get_json() utilisé plus bas pour les routes API JSON.

```
130         records = request_table.all(formula=f"{{request_id}}='{request_id}'")
```

→ Interroge Airtable pour récupérer les enregistrements correspondant à cet identifiant.

```
131         if not records:
```

→ Vérifie si aucun enregistrement n'a été trouvé pour cet identifiant.

```
132         logger.error(f"No record found for request_id: {request_id}")
```

→ Journalise en erreur l'absence d'enregistrement trouvé.

```
133         return "Request not found", 404
```

→ Retourne une réponse HTTP 404 si la demande est introuvable.

```
135         record = records[0]
```

→ Sélectionne le premier enregistrement trouvé.

```
136         request_table.update(record['id'], {'status': action})
```

→ Met à jour le statut de la demande dans Airtable avec l'action reçue, en utilisant l'identifiant interne Airtable de l'enregistrement (record['id'], différent du request_id métier).

```
137         logger.info(f"Updated request {request_id} with status: {action}")
```

→ Journalise en info la mise à jour du statut de la demande.

```
139         # Trigger approval agent to notify employee
```

→ Commentaire annonçant l'appel à l'agent d'approbation pour notifier l'employé.

```
140         response = approval_agent.run(f"Process holiday request {request_id} with action {action}",  
markdown=True)
```

→ Exécute l'agent d'approbation (défini plus bas) avec un prompt en langage naturel décrivant la demande et l'action à traiter ; c'est l'agent, et non du code métier classique, qui va lui-même décider quels outils appeler (mise à jour Airtable, envoi d'email) pour finaliser la notification - une délégation de logique métier au LLM plutôt qu'un enchaînement de fonctions écrit explicitement.

```
141         print(response.content)
```

→ Affiche dans la console le contenu de la réponse de l'agent d'approbation.

```
142         logger.info(f"Approval agent response: {response.content}")
```

→ Journalise en info le contenu de la réponse de l'agent d'approbation.

```
143         return f"Request {action} successfully. The employee will be notified."
```

→ Retourne un message de confirmation HTTP indiquant le succès du traitement.

144 except Exception as e:
→ Capture toute exception survenue pendant le traitement de la demande.

145 logger.error(f"Error processing request {request_id}: {str(e)}")
→ Journalise en erreur le message de l'exception survenue.

146 return f"Error processing request: {str(e)}", 500
→ Retourne une réponse HTTP 500 avec le détail de l'erreur.

148 @app.route('/chat', methods=['POST'])
→ Déclare une route Flask POST servant de point d'entrée du chat pour l'équipe d'agents (utilisée par le frontend Streamlit app.py).

149 def chat_handler():
→ Définit la fonction de vue traitant les requêtes de chat entrantes.

150 try:
→ Ouvre un bloc try pour capturer les erreurs lors du traitement du chat.

151 data = request.get_json()
→ Récupère le corps de la requête HTTP entrante en tant que JSON, l'équivalent de request.form mais pour des clients envoyant du JSON (comme une API REST classique) plutôt qu'un formulaire HTML.

152 if not data:
→ Vérifie si le corps JSON de la requête est vide ou absent.

153 return jsonify({"error": "No JSON data provided"}), 400
→ Retourne une réponse JSON avec le code 400 (Bad Request) indiquant l'absence de données JSON.

155 message = data.get('message')
→ Extrait le champ "message" du JSON reçu.

157 if not message:
→ Vérifie si le message est vide ou absent.

158 return jsonify({"error": "Missing message"}), 400
→ Retourne une réponse JSON 400 indiquant que le message est manquant.

160 # Call the team agent
→ Commentaire annonçant l'appel à l'équipe d'agents pour traiter le message.

161 response = ChatBot_Team.run(message, markdown=True)
→ Exécute l'équipe d'agents ChatBot_Team (définie plus bas) avec le message reçu : c'est le manager de l'équipe qui analyse l'intention du message et route la conversation vers l'agent spécialisé approprié (congé, projets ou approbation) avant de renvoyer une réponse.

163 return jsonify({"response": response.content})
→ Retourne le contenu de la réponse de l'équipe d'agents au format JSON, consommable par le frontend.

164 except Exception as e:
→ Capture toute exception survenue pendant le traitement du chat.

165 logger.error(f"Error in chat handler: {str(e)}")
→ Journalise en erreur le message de l'exception survenue.

166 return jsonify({"error": str(e)}), 500
→ Retourne une réponse JSON 500 avec le détail de l'erreur.

168 @app.route('/text', methods=['POST'])
→ Déclare une route Flask POST servant de point d'entrée pour les transcriptions vocales, destinée à être appelée par un

service externe de type assistant vocal.

```
169 def voice_handler():
```

→ Définit la fonction de vue traitant les requêtes de transcription vocale.

```
170     try:
```

→ Ouvre un bloc try pour capturer les erreurs lors du traitement de la transcription.

```
171         data = request.get_json()
```

→ Récupère le corps de la requête HTTP entrante en tant que JSON.

```
172         if not data:
```

→ Vérifie si le corps JSON de la requête est vide ou absent.

```
173         return jsonify({"error": "No JSON data provided"}), 400
```

→ Retourne une réponse JSON 400 indiquant l'absence de données JSON.

```
175         print(data)
```

→ Affiche dans la console l'intégralité des données JSON reçues, à des fins de débogage.

```
176         print(data.get("transcript"))
```

→ Affiche dans la console le champ "transcript" extrait des données, à des fins de débogage.

```
178         transcript = data.get("transcript")
```

→ Extrait le champ "transcript" du JSON reçu et le stocke dans une variable.

```
179         print(transcript)
```

→ Affiche à nouveau la transcription dans la console, à des fins de débogage.

```
181         if not transcript:
```

→ Vérifie si la transcription est vide ou absente.

```
182         return jsonify({"error": "Missing transcript"}), 400
```

→ Retourne une réponse JSON 400 indiquant que la transcription est manquante.

```
184         # Optional: customer_name = data.get("customer_name", "User")
```

→ Commentaire indiquant une extraction optionnelle du nom client, laissée en commentaire (fonctionnalité non activée dans cette version).

```
186         # Step 1: Create Chat Session
```

→ Commentaire annonçant la première étape : la création d'une session de chat auprès de l'API Retell.

```
187         try:
```

→ Ouvre un bloc try imbriqué pour capturer les erreurs de création de session Retell spécifiquement, séparément des autres erreurs de la fonction.

```
188         create_chat_response = requests.post(
```

→ Envoie une requête HTTP POST pour créer une session de chat auprès de l'API Retell, un service tiers spécialisé dans les agents conversationnels vocaux/texte : ici, la transcription vocale déjà obtenue en amont (par un autre système) est relayée à Retell pour obtenir une réponse d'agent.

```
189             "https://api.retellai.com/create-chat",
```

→ Spécifie l'URL de l'endpoint de création de chat de l'API Retell.

```
190             headers={
```

→ Ouvre le dictionnaire des en-têtes HTTP de la requête.

```
191                 "Authorization": f"Bearer key_1581ac824f79644c48f3f9c8fc51",
```

→ Définit l'en-tête d'autorisation HTTP au format standard "Bearer <clé>". La clé API Retell est ici codée en dur directement dans le code source plutôt que lue depuis une variable d'environnement : c'est une pratique de sécurité à éviter, car toute

personne ayant accès au dépôt de code (ou à un historique git) obtient la clé, qui devrait normalement être un secret géré via .env comme les autres clés de ce fichier.

```
192             "Content-Type": "application/json"
```

→ Indique que le corps de la requête est au format JSON.

```
193         },
```

→ Ferme le dictionnaire des en-têtes HTTP.

```
194         json={
```

→ Ouvre le dictionnaire du corps JSON de la requête ; le paramètre json= de requests sérialise automatiquement ce dictionnaire en JSON et positionne le bon Content-Type.

```
195             "agent_id": "agent_4cb8cf66474b91a09d19419",
```

→ Fournit l'identifiant de l'agent Retell (côté plateforme Retell, distinct des agents Agno définis dans ce fichier) à utiliser pour la session.

```
196             "metadata": {}
```

→ Fournit un objet metadata vide dans le corps de la requête, un champ libre que l'API Retell permet d'attacher à la session mais qui n'est pas exploité ici.

```
197         },
```

→ Ferme le dictionnaire du corps JSON de la requête.

```
198         timeout=30
```

→ Définit un délai maximal de 30 secondes pour la requête HTTP, pour éviter que le serveur Flask reste bloqué indéfiniment si l'API Retell ne répond pas.

```
199     )
```

→ Ferme l'appel à requests.post commencé plus haut.

```
200     except requests.RequestException as e:
```

→ Capture spécifiquement les exceptions de la bibliothèque requests (erreur réseau, timeout, DNS...) survenues lors de cet appel vers l'API Retell.

```
201         logger.error(f"Error creating chat: {e}")
```

→ Journalise en erreur le message de l'exception réseau survenue.

```
202         return jsonify({"error": "Failed to connect to Retell API"}), 500
```

→ Retourne une réponse JSON 500 indiquant l'échec de connexion à l'API Retell.

```
204     print(create_chat_response) # If response 402, means payment required!!!
```

→ Affiche l'objet réponse dans la console ; le commentaire précise qu'un code HTTP 402 (Payment Required) indiquerait un problème de facturation côté compte Retell, une note de diagnostic utile pour l'équipe plutôt qu'un vrai bug de code.

```
205     if create_chat_response.status_code != 200:
```

→ Vérifie si le code de statut HTTP de la réponse n'est pas 200 (succès).

```
206         return jsonify({"error": f"Failed to create chat: {create_chat_response.text}"}), 500
```

→ Retourne une réponse JSON 500 incluant le texte d'erreur renvoyé par l'API Retell.

```
208     chat_id = create_chat_response.json().get("chat_id")
```

→ Extraît l'identifiant de chat renvoyé par l'API Retell depuis la réponse JSON, nécessaire pour poursuivre la conversation dans l'appel suivant.

```
209     if not chat_id:
```

→ Vérifie si aucun identifiant de chat n'a été renvoyé.

```
210     return jsonify({"error": "No chat_id returned"}), 500
```

→ Retourne une réponse JSON 500 si l'identifiant de chat est absent.

212 # RETELL_API_KEY corrigé

→ Commentaire indiquant que la clé API Retell utilisée ci-dessous a été corrigée (probablement suite à un bug antérieur), sans changer le fait qu'elle reste codée en dur.

213 RETELL_API_KEY = "key_1581ac824f79644c48f3f9c8fc51"

→ Définit une variable locale contenant la même clé API Retell codée en dur dans le code source (au lieu d'être lue via `os.getenv` comme les autres clés du fichier), ce qui reste une mauvaise pratique de sécurité : une clé en dur peut fuiter via le contrôle de version ou en partageant le fichier.

214 try:

→ Ouvre un bloc try imbriqué pour capturer les erreurs lors de la génération de la complétion.

215 completion_response = requests.post(

→ Envoie une requête HTTP POST pour obtenir la réponse de l'agent Retell à la transcription, dans la session de chat créée à l'étape précédente.

216 "https://api.retellai.com/create-chat-completion",

→ Spécifie l'URL de l'endpoint de complétion de chat de l'API Retell.

217 headers={

→ Ouvre le dictionnaire des en-têtes HTTP de la requête.

218 "Authorization": f"Bearer {RETELL_API_KEY}",

→ Définit l'en-tête d'autorisation en utilisant la clé API Retell définie plus haut (via une f-string cette fois, contrairement à la ligne 191 où la clé était écrite littéralement).

219 "Content-Type": "application/json"

→ Indique que le corps de la requête est au format JSON.

220 },

→ Ferme le dictionnaire des en-têtes HTTP.

221 json={

→ Ouvre le dictionnaire du corps JSON de la requête.

222 "chat_id": chat_id,

→ Fournit l'identifiant de chat obtenu précédemment dans le corps de la requête, pour que Retell rattache ce message à la bonne conversation.

223 "content": transcript

→ Fournit la transcription vocale reçue comme contenu du message à envoyer à l'agent Retell.

224 },

→ Ferme le dictionnaire du corps JSON de la requête.

225 timeout=30

→ Définit un délai maximal de 30 secondes pour la requête HTTP.

226)

→ Ferme l'appel à `requests.post` commencé plus haut.

227 except requests.RequestException as e:

→ Capture toute exception réseau survenue lors de la requête de complétion.

228 logger.error(f"Error getting completion: {e}")

→ Journalise en erreur le message de l'exception réseau survenue.

229 return jsonify({"error": "Failed to get completion from Retell API"}), 500

→ Retourne une réponse JSON 500 indiquant l'échec d'obtention de la complétion.

```
231         if completion_response.status_code != 200:
```

→ Vérifie si le code de statut HTTP de la réponse de complétion n'est pas 200.

```
232         return jsonify({"error": f"Failed to get completion: {completion_response.text}"}, 500)
```

→ Retourne une réponse JSON 500 incluant le texte d'erreur renvoyé par l'API Retell.

```
234         messages = completion_response.json().get("messages", [])
```

→ Extrait la liste des messages renvoyés par l'API Retell depuis la réponse JSON, avec une liste vide par défaut si le champ est absent.

```
235         agent_reply = messages[-1]["content"] if messages else "No reply from agent."
```

→ Récupère le contenu du dernier message de la liste (la réponse la plus récente de l'agent Retell), ou un texte par défaut si la liste est vide.

```
237         return jsonify({
```

→ Commence la construction de la réponse JSON finale renvoyée au client.

```
238             "response": agent_reply,
```

→ Inclut la réponse de l'agent Retell dans le JSON de sortie.

```
239             "chat_id": chat_id
```

→ Inclut l'identifiant de chat dans le JSON de sortie, permettant au client de poursuivre la même session lors d'un prochain appel.

```
240         })
```

→ Ferme le dictionnaire passé à jsonify et l'appel à jsonify.

```
242     except Exception as e:
```

→ Capture toute exception non gérée survenue pendant le traitement de la requête vocale (au niveau externe, en plus des try imbriqués plus spécifiques).

```
243     logger.error(f"Error in voice handler: {str(e)}")
```

→ Journalise en erreur le message de l'exception survenue.

```
244     return jsonify({"error": str(e)}), 500
```

→ Retourne une réponse JSON 500 avec le détail de l'erreur.

```
246 def start_flask():
```

→ Définit la fonction chargée de démarrer le serveur Flask ; elle sera lancée dans un thread séparé plus bas, pour ne pas bloquer le reste du script (initialisation de ngrok et des agents) pendant que le serveur tourne indéfiniment.

```
247     logger.info("Starting Flask app on port 5000")
```

→ Journalise en info le démarrage du serveur Flask sur le port 5000.

```
248     try:
```

→ Ouvre un bloc try pour capturer les erreurs de démarrage du serveur.

```
249         app.run(port=5000, debug=False)
```

→ Démarre le serveur de développement Flask sur le port 5000, avec le mode debug désactivé (debug=False) : le mode debug active un rechargement automatique et un débogueur interactif pratiques en développement, mais dangereux à laisser actif car il peut exposer du code exécutable arbitraire si l'application est accessible publiquement (ce qui est justement le cas ici via ngrok).

```
250     except Exception as e:
```

→ Capture toute exception survenue pendant l'exécution du serveur Flask.


```

251         logger.error(f"Error starting Flask app: {e}")
→ Journalise en erreur le message de l'exception survenue lors du démarrage.

253 # Airtable tool functions
→ Commentaire annonçant les fonctions utilitaires exposées comme outils aux agents pour Airtable.

254 def get_employee_holiday(employee_id: int) -> dict:
→ Définit une fonction Python classique (typée avec des annotations employee_id: int et -> dict) qui récupère les données de congés d'un employé ; elle sera passée directement dans la liste tools=[...] d'un agent plus bas, Agno se chargeant d'exposer sa signature et sa docstring au LLM comme définition d'un outil callable (function calling).

255     """Retrieve employee holiday information from Airtable by employee_id."""
→ Docstring décrivant le rôle de la fonction : ce texte sert aussi de description de l'outil que le modèle de langage lira pour décider quand l'appeler.

256     try:
→ Ouvre un bloc try pour capturer les erreurs lors de la lecture Airtable.

257         records = holiday_table.all()
→ Récupère l'ensemble des enregistrements de la table des congés depuis Airtable en une seule requête.

258         for record in records:
→ Itère sur chaque enregistrement récupéré.

259             fields = record["fields"]
→ Extrait le dictionnaire des champs de l'enregistrement courant.

260             if fields.get("employee_id") == employee_id:
→ Vérifie si l'identifiant employé de l'enregistrement correspond à celui recherché (recherche linéaire côté client, faute de filtre serveur ici, contrairement aux formules Airtable utilisées plus haut dans les routes Flask).

261                 logger.info(f"Retrieved holiday data for employee_id: {employee_id}")
→ Journalise en info la récupération réussie des données de congés pour cet employé.

262                 return {
→ Commence la construction du dictionnaire de résultat à retourner (et donc du résultat que verra le LLM appelant cet outil).

263                     "name": fields.get("full_name"),
→ Inclut le nom complet de l'employé dans le résultat.

264                     "email": fields.get("email"),
→ Inclut l'adresse email de l'employé dans le résultat.

265                     "total": int(fields.get("total_holiday_days", 0)),
→ Inclut le nombre total de jours de congé alloués, converti en entier, avec 0 par défaut si le champ est absent (évite un TypeError si le champ Airtable est vide plutôt qu'un chiffre).

266                     "taken": int(fields.get("holidays_taken", 0)),
→ Inclut le nombre de jours de congé déjà pris, converti en entier, avec 0 par défaut.

267                     "last_date": fields.get("last_holiday_taken"),
→ Inclut la date du dernier congé pris dans le résultat.

268                 }
→ Ferme le dictionnaire de résultat retourné.

269                 logger.warning(f"No holiday data found for employee_id: {employee_id}")
→ Journalise un avertissement si aucun enregistrement ne correspond à l'employé après la boucle.

270         return None

```

→ Retourne None si aucune donnée de congé n'a été trouvée pour cet employé.

271 except Exception as e:

→ Capture toute exception survenue pendant la lecture des données Airtable.

272 logger.error(f"Error retrieving employee holiday data: {e}")

→ Journalise en erreur le message de l'exception survenue.

273 return None

→ Retourne None en cas d'erreur lors de la récupération des données.

275 def get_employee_project(employee_id: int):

→ Définit une fonction, également destinée à être exposée comme outil à un agent, qui récupère les données de projet d'un employé à partir de son identifiant.

276 """Retrieve employee project information from Airtable by employee_id."""

→ Docstring décrivant le rôle de la fonction get_employee_project, servant aussi de description d'outil pour le LLM.

277 try:

→ Ouvre un bloc try pour capturer les erreurs lors de la lecture Airtable.

278 records = project_table.all()

→ Récupère l'ensemble des enregistrements de la table des projets depuis Airtable.

279 for record in records:

→ Itère sur chaque enregistrement récupéré.

280 fields = record["fields"]

→ Extrait le dictionnaire des champs de l'enregistrement courant.

281 if fields.get("employee_id") == employee_id:

→ Vérifie si l'identifiant employé de l'enregistrement correspond à celui recherché.

282 logger.info(f"Retrieved project data for employee_id: {employee_id}")

→ Journalise en info la récupération réussie des données de projet pour cet employé.

283 return {

→ Commence la construction du dictionnaire de résultat à retourner.

284 "name": fields.get("employee_name"),

→ Inclut le nom de l'employé dans le résultat.

285 "project": fields.get("project_name"),

→ Inclut le nom du projet courant de l'employé dans le résultat.

286 "role": fields.get("role"),

→ Inclut le rôle de l'employé sur le projet dans le résultat.

287 "start_date": fields.get("start_date"),

→ Inclut la date de début du projet dans le résultat.

288 "end_date": fields.get("end_date"),

→ Inclut la date de fin du projet dans le résultat.

289 "next_project": {

→ Ouvre un sous-dictionnaire décrivant le prochain projet prévu de l'employé.

290 "project_name": fields.get("next_project_name", ""),

→ Inclut le nom du prochain projet, avec une chaîne vide par défaut.

291 "start_date": fields.get("next_project_start_date", ""),
→ Inclut la date de début du prochain projet, avec une chaîne vide par défaut.

292 "role": fields.get("next_project_role", "")
→ Inclut le rôle prévu sur le prochain projet, avec une chaîne vide par défaut.

293 }
→ Ferme le sous-dictionnaire next_project.

294 }
→ Ferme le dictionnaire de résultat retourné.

295 logger.warning(f"No project data found for employee_id: {employee_id}")
→ Journalise un avertissement si aucun enregistrement ne correspond à l'employé après la boucle.

296 return None
→ Retourne None si aucune donnée de projet n'a été trouvée pour cet employé.

297 except Exception as e:
→ Capture toute exception survenue pendant la lecture des données Airtable.

298 logger.error(f"Error retrieving employee project data: {e}")
→ Journalise en erreur le message de l'exception survenue.

299 return None
→ Retourne None en cas d'erreur lors de la récupération des données.

301 def update_next_project(employee_id: int, project_name: str, start_date: str, role: str):
→ Définit une fonction outil qui met à jour les informations du prochain projet d'un employé ; les paramètres typés définissent ici le "schéma" que le LLM devra respecter lorsqu'il appellera cet outil.

302 """Update the next project information for an employee in Airtable."""
→ Docstring décrivant le rôle de la fonction update_next_project.

303 try:
→ Ouvre un bloc try pour capturer les erreurs lors de la mise à jour Airtable.

304 records = project_table.all()
→ Récupère l'ensemble des enregistrements de la table des projets depuis Airtable.

305 for record in records:
→ Itère sur chaque enregistrement récupéré.

306 fields = record["fields"]
→ Extrait le dictionnaire des champs de l'enregistrement courant.

307 if fields.get("employee_id") == employee_id:
→ Vérifie si l'identifiant employé de l'enregistrement correspond à celui recherché.

308 project_table.update(record["id"], {
→ Commence la mise à jour de l'enregistrement Airtable correspondant à cet employé, en utilisant son identifiant interne Airtable.

309 "next_project_name": project_name,
→ Met à jour le champ du nom du prochain projet avec la valeur fournie.

```

310         "next_project_start_date": start_date,
    → Met à jour le champ de la date de début du prochain projet avec la valeur fournie.

311         "next_project_role": role
    → Met à jour le champ du rôle sur le prochain projet avec la valeur fournie.

312     })
    → Ferme le dictionnaire de champs et l'appel à project_table.update.

313     logger.info(f"Updated next project for employee_id: {employee_id}")
    → Journalise en info la réussite de la mise à jour pour cet employé.

314     return True
    → Retourne True pour signaler le succès de la mise à jour.

315     logger.warning(f"No project record found for employee_id: {employee_id}")
    → Journalise un avertissement si aucun enregistrement ne correspond à l'employé après la boucle.

316     return False
    → Retourne False si aucun enregistrement de projet n'a été trouvé pour cet employé.

317 except Exception as e:
    → Capture toute exception survenue pendant la mise à jour Airtable.

318     logger.error(f"Error updating next project: {e}")
    → Journalise en erreur le message de l'exception survenue.

319     return False
    → Retourne False en cas d'erreur lors de la mise à jour.

321 def create_holiday_request(employee_id: int, requested_days: int):
    → Définit une fonction outil qui crée une nouvelle demande de congé dans Airtable, callable par l'agent Holiday lorsqu'un employé formule une demande.

322     """Create a new holiday request in Airtable."""
    → Docstring décrivant le rôle de la fonction create_holiday_request.

323     try:
    → Ouvre un bloc try pour capturer les erreurs lors de la création de la demande.

324         employee = get_employee_holiday(employee_id)
    → Récupère les données de congés de l'employé en réutilisant la fonction définie plus haut, pour connaître son solde avant de créer la demande.

325         if not employee:
    → Vérifie si aucune donnée d'employé n'a été trouvée.

326             logger.error(f"No employee found for employee_id: {employee_id}")
    → Journalise en erreur l'absence d'employé trouvé pour cet identifiant.

327             return None
    → Retourne None si l'employé n'existe pas.

329             request_id = str(uuid.uuid4())
    → Génère un identifiant unique universel (UUID version 4, basé sur des nombres aléatoires) pour la nouvelle demande de congé : contrairement à un simple compteur auto-incrémenté, un UUID est pratiquement impossible à deviner, ce qui est important puisque cet identifiant se retrouve dans une URL d'approbation envoyée par email (voir plus bas) et ne doit pas pouvoir être deviné par un tiers.

330             remaining_days = employee['total'] - employee['taken']

```

→ Calcule le solde de jours restants en soustrayant les jours pris du total alloué.

```
332         request_table.create({
```

→ Commence la création d'un nouvel enregistrement dans la table des demandes de congé.

```
333             "request_id": request_id,
```

→ Enregistre l'identifiant unique de la demande générée.

```
334             "employee_id": employee_id,
```

→ Enregistre l'identifiant de l'employé concerné par la demande.

```
335             "full_name": employee['name'],
```

→ Enregistre le nom complet de l'employé dans la demande.

```
336             "requested_days": requested_days,
```

→ Enregistre le nombre de jours demandés.

```
337             "remaining_days": remaining_days,
```

→ Enregistre le solde de jours restants calculé précédemment.

```
338             "status": "pending",
```

→ Initialise le statut de la demande à "pending" (en attente), avant qu'une décision d'approbation ou de refus ne soit prise via la route /process/<request_id>.

```
339             "request_date": time.strftime("%Y-%m-%d")
```

→ Enregistre la date du jour formatée (année-mois-jour) comme date de la demande.

```
340         })
```

→ Ferme le dictionnaire de champs et l'appel à request_table.create.

```
341         logger.info(f"Created holiday request {request_id} for employee_id: {employee_id}")
```

→ Journalise en info la création réussie de la demande de congé.

```
342         return request_id
```

→ Retourne l'identifiant de la demande créée à l'appelant (l'agent), qui l'utilisera pour construire le lien d'approbation dans l'email envoyé aux RH.

```
343     except Exception as e:
```

→ Capture toute exception survenue pendant la création de la demande.

```
344         logger.error(f"Failed to create holiday request for employee_id {employee_id}: {str(e)}")
```

→ Journalise en erreur le message de l'exception survenue.

```
345         return None
```

→ Retourne None en cas d'échec de création de la demande.

```
347 def update_holiday_taken(employee_id: int, additional_days: int):
```

→ Définit une fonction outil qui met à jour le nombre de jours de congé pris par un employé, destinée à être appelée par l'agent Approval une fois une demande approuvée.

```
348     """Update holidays_taken for an employee in Airtable."""
```

→ Docstring décrivant le rôle de la fonction update_holiday_taken.

```
349     try:
```

→ Ouvre un bloc try pour capturer les erreurs lors de la mise à jour Airtable.

```
350         records = holiday_table.all()
```

→ Récupère l'ensemble des enregistrements de la table des congés depuis Airtable.

```

351         for record in records:
            → Itère sur chaque enregistrement récupéré.

352             fields = record["fields"]
            → Extrait le dictionnaire des champs de l'enregistrement courant.

353             if fields.get("employee_id") == employee_id:
            → Vérifie si l'identifiant employé de l'enregistrement correspond à celui recherché.

354                 current_taken = int(fields.get("holidays_taken", 0))
            → Récupère le nombre de jours déjà pris, converti en entier, avec 0 par défaut.

355                 holiday_table.update(record["id"], {
            → Commence la mise à jour de l'enregistrement Airtable correspondant à cet employé.

356                     "holidays_taken": current_taken + additional_days,
            → Met à jour le nombre de jours pris en y ajoutant les jours additionnels fournis (accumulation plutôt qu'écrasement de la
            valeur).

357                     "last_holiday_taken": time.strftime("%Y-%m-%d")
            → Met à jour la date du dernier congé pris avec la date du jour.

358                 })
            → Ferme le dictionnaire de champs et l'appel à holiday_table.update.

359                 logger.info(f"Updated holidays_taken for employee_id: {employee_id}")
            → Journalise en info la réussite de la mise à jour pour cet employé.

360                 return True
            → Retourne True pour signaler le succès de la mise à jour.

361                 logger.warning(f"No holiday record found for employee_id: {employee_id}")
            → Journalise un avertissement si aucun enregistrement ne correspond à l'employé après la boucle.

362                 return False
            → Retourne False si aucun enregistrement de congé n'a été trouvé pour cet employé.

363     except Exception as e:
            → Capture toute exception survenue pendant la mise à jour Airtable.

364         logger.error(f"Error updating holiday taken: {e}")
            → Journalise en erreur le message de l'exception survenue.

365         return False
            → Retourne False en cas d'erreur lors de la mise à jour.

367 # Configuration des credentials Gmail
            → Commentaire annonçant la résolution du chemin vers les identifiants Gmail.

368 gmail_credentials_path = os.path.join(os.path.dirname(__file__), "client_secret.json")
            → Construit le chemin vers le fichier client_secret.json (fichier d'identifiants OAuth2 Google typiquement utilisé pour l'envoi
            d'emails via l'API Gmail) situé dans le dossier du script.

369 if not os.path.exists(gmail_credentials_path):
            → Vérifie si le fichier d'identifiants Gmail n'existe pas à cet emplacement.

370     logger.warning(f"Gmail credentials file not found at {gmail_credentials_path}")
            → Journalise un avertissement indiquant que le fichier d'identifiants Gmail est introuvable.

```

371 `gmail_credentials_path = None`

→ Réinitialise la variable à None pour indiquer l'absence d'identifiants Gmail disponibles ; cette variable conditionnera plus bas la création (ou non) de l'outil EmailTools, selon le même principe de dégradation gracieuse que la clé Mistral du premier projet.

373 `# Start ngrok`

→ Commentaire annonçant le démarrage du tunnel ngrok.

374 `try:`

→ Ouvre un bloc try pour capturer les erreurs lors de la configuration de ngrok.

375 `ngrok.set_auth_token(NGROK_AUTH_TOKEN)`

→ Configure le jeton d'authentification ngrok pour autoriser la création de tunnels associés au compte ngrok correspondant.

376 `try:`

→ Ouvre un bloc try imbriqué pour tenter la connexion ngrok en HTTPS en priorité.

377 `public_url = ngrok.connect(5000, bind_tls=True).public_url`

→ Ouvre un tunnel ngrok en HTTPS (bind_tls=True) vers le port local 5000 où tourne Flask, et récupère l'URL publique générée : c'est cette URL qui rendra le serveur local accessible depuis Internet, par exemple pour qu'un lien d'approbation envoyé par email fonctionne même si le RH n'est pas sur le même réseau que la machine qui exécute le script.

378 `logger.info(f"ngrok tunnel \"{public_url}\" -> \"http://127.0.0.1:5000\")`

→ Journalise en info l'URL publique du tunnel et sa correspondance locale.

379 `except Exception as ngrok_error:`

→ Capture toute exception survenue lors de la tentative de connexion HTTPS.

380 `logger.warning(f"ngrok HTTPS connection failed: {ngrok_error}")`

→ Journalise un avertissement indiquant l'échec de la connexion ngrok en HTTPS.

381 `try:`

→ Ouvre un second bloc try imbriqué pour tenter la connexion ngrok en HTTP simple, en repli.

382 `public_url = ngrok.connect(5000).public_url`

→ Ouvre un tunnel ngrok (HTTP, sans TLS forcé) vers le port 5000 et récupère l'URL publique générée, comme solution de repli si le tunnel HTTPS a échoué.

383 `logger.info(f"ngrok tunnel (HTTP) \"{public_url}\" -> \"http://127.0.0.1:5000\")`

→ Journalise en info l'URL publique du tunnel HTTP obtenu en repli.

384 `except Exception:`

→ Capture toute exception survenue lors de la tentative de connexion HTTP de repli.

385 `logger.warning("ngrok completely failed, using localhost")`

→ Journalise un avertissement indiquant que ngrok a totalement échoué et que localhost sera utilisé à la place (le système reste alors accessible uniquement en local, les liens envoyés par email ne fonctionneront que depuis la même machine/réseau).

386 `public_url = "http://localhost:5000"`

→ Définit l'URL publique de repli sur l'adresse locale du serveur.

387 `except Exception as e:`

→ Capture toute exception survenue au niveau de la configuration globale de ngrok (par exemple un jeton d'authentification invalide).

388 `logger.error(f"Error setting up ngrok: {e}")`

→ Journalise en erreur le message de l'exception survenue lors de la configuration de ngrok.

389 `public_url = "http://localhost:5000"`

→ Définit l'URL publique de repli sur l'adresse locale en cas d'échec total de la configuration.

391 `threading.Thread(target=start_flask, daemon=True).start()`

→ Démarre le serveur Flask dans un thread démon séparé (`daemon=True` signifie que ce thread s'arrêtera automatiquement quand le programme principal se termine, sans bloquer sa fermeture), permettant au reste du script (initialisation des agents, boucle d'attente) de continuer à s'exécuter dans le thread principal en parallèle.

392 `time.sleep(2)`

→ Met le script en pause pendant 2 secondes pour laisser le temps au serveur Flask de terminer son démarrage dans l'autre thread avant que la suite du script ne suppose qu'il est déjà opérationnel.

394 `# Holiday Agent`

→ Commentaire annonçant la définition de l'agent de gestion des congés.

395 `try:`

→ Ouvre un bloc try pour capturer les erreurs lors de l'initialisation de l'agent Holiday.

396 `email_tools = EmailTools() if gmail_credentials_path else None`

→ Instancie l'outil d'envoi d'emails uniquement si des identifiants Gmail valides sont disponibles ; sinon `email_tools` reste à None, et les agents plus bas adaptent leur liste d'outils en conséquence plutôt que de planter à l'instanciation.

397 `holiday_agent = Agent(`

→ Commence la définition du premier agent spécialisé de l'équipe : l'agent Agno chargé du calcul et du traitement des jours de congé.

398 `name="Holiday Days Calculator Agent",`

→ Donne un nom lisible à l'agent de calcul des congés.

399 `model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),`

→ Configure le modèle OpenAI "gpt-4o-mini" (une version allégée et moins coûteuse de GPT-4o, adaptée à des tâches structurées comme celle-ci) utilisé par cet agent, avec sa clé API.

400 `markdown=True,`

→ Active le formatage Markdown pour les réponses de l'agent, afin que les listes, gras, etc. soient correctement interprétés côté client plutôt que renvoyés en texte brut.

401 `show_tool_calls=True,`

→ Active l'affichage des appels d'outils effectués par l'agent dans les logs/sortie, un réglage de débogage utile pour voir concrètement quelles fonctions (`get_employee_holiday`, `create_holiday_request`...) le LLM a choisi d'invoquer pour répondre.

402 `tools=[`

→ Ouvre la liste conditionnelle des outils disponibles pour l'agent (function calling) : c'est la liste qu'Agno traduit en définitions de fonctions présentées au modèle, qui peut alors décider d'en appeler une plutôt que de répondre directement en texte.

403 `FileTools(),`

→ Ajoute l'outil de manipulation de fichiers à la liste des outils de l'agent.

404 `CalculatorTools(),`

→ Ajoute l'outil de calcul à la liste des outils de l'agent, pour fiabiliser les opérations arithmétiques (comme le calcul de solde de congés) plutôt que de laisser le LLM les estimer.

405 `email_tools,`

→ Ajoute l'outil d'envoi d'emails (déjà instancié ligne 396) à la liste des outils de l'agent.

406 `get_employee_holiday,`

→ Ajoute la fonction Python `get_employee_holiday`, définie plus haut, comme outil disponible pour l'agent - une fonction "métier" personnalisée exposée en `tool use` au même titre qu'un outil Agno intégré.

407 `create_holiday_request`

→ Ajoute la fonction `create_holiday_request` comme outil disponible pour l'agent.

408] if email_tools else [

→ Ferme la première branche de la liste et ouvre la branche alternative utilisée si email_tools est indisponible (identifiants Gmail manquants).

409 FileTools(),

→ Ajoute l'outil de manipulation de fichiers dans la liste d'outils de repli (sans email).

410 CalculatorTools(),

→ Ajoute l'outil de calcul dans la liste d'outils de repli (sans email).

411 get_employee_holiday,

→ Ajoute la fonction get_employee_holiday dans la liste d'outils de repli (sans email).

412 create_holiday_request

→ Ajoute la fonction create_holiday_request dans la liste d'outils de repli (sans email).

413],

→ Ferme la liste conditionnelle des outils de l'agent.

414 description="""

→ Ouvre la chaîne multi-lignes décrivant le rôle général de l'agent Holiday ; ce paramètre description (distinct des instructions) sert surtout de résumé utilisé notamment par le Team manager plus bas pour savoir vers quel agent router une requête.

415 An internal company assistant that helps employees check their holiday status, submit holiday requests, and create new employee records.

→ Décrit l'agent comme un assistant interne aidant aux statuts de congé, demandes et enregistrements d'employés.

416 This agent manages holiday data from an Airtable using employee IDs, creates holiday request entries for HR approval, and can register new employees in the system.

→ Précise que l'agent gère les données Airtable par identifiant employé et crée des demandes pour la RH.

417 It sends emails to HR with links to approve or disapprove requests.

→ Précise que l'agent envoie des emails à la RH avec des liens d'approbation ou de refus.

418 """

→ Ferme la chaîne multi-lignes du paramètre description.

419 instructions=f"""

→ Ouvre une chaîne multi-lignes formatée (f-string, remarquable par le "f" avant les triples guillemets) définissant les instructions détaillées de l'agent : contrairement au bloc description ci-dessus, ces instructions sont le vrai prompt système, et l'usage d'une f-string permet d'y interpoler directement des variables Python (HR_EMAIL, public_url) comme on le voit plus bas.

420 - Your job is to retrieve holiday information or process holiday requests using the employee's unique ID.

→ Ligne d'instruction précisant la mission principale : traiter les congés par identifiant employé.

421 - Use 'get_employee_holiday' to fetch:

→ Ligne d'instruction indiquant d'utiliser l'outil get_employee_holiday pour récupérer des informations, en le désignant par son nom de fonction afin que le modèle sache explicitement quel outil appeler pour quel besoin.

422 - Full name

→ Précise que le nom complet fait partie des informations à récupérer.

423 - Email

→ Précise que l'email fait partie des informations à récupérer.

424 - Total allocated holiday days

→ Précise que le total de jours alloués fait partie des informations à récupérer.

425 - Number of holidays taken
→ Précise que le nombre de jours déjà pris fait partie des informations à récupérer.

426 - Last holiday date
→ Précise que la date du dernier congé fait partie des informations à récupérer.

427 - For balance queries:
→ Ligne d'instruction introduisant le traitement des demandes de solde de congés.

428 - Calculate: `remaining_days = total_holiday_days - holidays_taken`
→ Précise la formule de calcul du solde de jours restants directement dans le prompt, pour guider le raisonnement du modèle même s'il dispose aussi de CalculatorTools pour l'exécuter de façon fiable.

429 - Return the information clearly and professionally.
→ Demande de restituer l'information de façon claire et professionnelle.

430 - For holiday requests:
→ Ligne d'instruction introduisant le traitement des demandes de congé.

431 - Extract the number of requested days and employee ID from the query.
→ Demande d'extraire le nombre de jours demandés et l'identifiant employé de la requête utilisateur.

432 - Use 'create_holiday_request' to store the request in Airtable.
→ Demande d'utiliser l'outil `create_holiday_request` pour enregistrer la demande dans Airtable.

433 - Generate a unique request ID and include it in an email to HR (`{HR_EMAIL}`) with a link to the approval page: `{public_url}/request/<request_id>`
→ Demande de générer un identifiant unique et d'envoyer un email à la RH avec le lien d'approbation ; grâce à la f-string, `{HR_EMAIL}` et `{public_url}` sont ici remplacés par leur valeur réelle (lue depuis l'environnement et générée par ngrok) au moment de la construction du prompt, avant même que le modèle ne le reçoive - le modèle voit donc l'adresse email et l'URL publique concrètes, pas des placeholders littéraux.

434 - The email should include the employee's name, requested days, and remaining balance.
→ Précise le contenu attendu de l'email : nom, jours demandés et solde restant.

435 - Do not send holiday balance or request confirmation directly to the employee unless explicitly requested.
→ Interdit d'envoyer directement à l'employé le solde ou la confirmation sauf demande explicite.

436 - Only send data from Airtable. Do not guess or hallucinate employee info.
→ Impose de n'utiliser que les données issues d'Airtable, sans invention d'informations - une consigne anti-hallucination, cruciale ici puisque de mauvaises données RH auraient un impact réel sur un employé.

437 - If no matching `employee_id` is found, respond with: "No holiday data found for the provided employee ID."
→ Définit le message de réponse standard lorsqu'aucun employé ne correspond à l'identifiant.

438 ""
→ Ferme la chaîne multi-lignes du paramètre instructions.

439)
→ Ferme l'appel au constructeur Agent commencé à la ligne 397.

440 `logger.info("Holiday Agent initialized successfully")`
→ Journalise en info l'initialisation réussie de l'agent Holiday.

441 `except Exception as e:`
→ Capture toute exception survenue pendant l'initialisation de l'agent Holiday.

442 `logger.error(f"Error initializing Holiday Agent: {e}")`

→ Journalise en erreur le message de l'exception survenue.

443 `raise`

→ Relève l'exception pour interrompre le script en cas d'échec d'initialisation, car les agents suivants (Approval, Project, Team) dépendent tous du bon fonctionnement de ce socle.

445 `# Approval Agent`

→ Commentaire annonçant la définition de l'agent d'approbation des congés.

446 `try:`

→ Ouvre un bloc try pour capturer les erreurs lors de l'initialisation de l'agent Approval.

447 `approval_agent = Agent(`

→ Commence la définition du deuxième agent spécialisé de l'équipe : l'agent Agno chargé de traiter les décisions d'approbation ou de refus de congé (appelé notamment depuis la route `/process/<request_id>`).

448 `name="Holiday Approval Agent",`

→ Donne un nom lisible à l'agent d'approbation des congés.

449 `model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),`

→ Configure le modèle OpenAI gpt-4o-mini utilisé par cet agent, avec sa clé API.

450 `markdown=True,`

→ Active le formatage Markdown pour les réponses de l'agent.

451 `show_tool_calls=True,`

→ Active l'affichage des appels d'outils effectués par l'agent.

452 `tools=[`

→ Ouvre la liste conditionnelle des outils disponibles pour l'agent.

453 `email_tools,`

→ Ajoute l'outil d'envoi d'emails à la liste des outils de l'agent, pour notifier l'employé de la décision.

454 `get_employee_holiday,`

→ Ajoute la fonction `get_employee_holiday` comme outil disponible pour l'agent, pour retrouver l'email de l'employé à notifier.

455 `update_holiday_taken`

→ Ajoute la fonction `update_holiday_taken` comme outil disponible pour l'agent, pour mettre à jour le solde en cas d'approbation.

456 `] if email_tools else [`

→ Ferme la première branche de la liste et ouvre la branche alternative si `email_tools` est indisponible.

457 `get_employee_holiday,`

→ Ajoute la fonction `get_employee_holiday` dans la liste d'outils de repli (sans email).

458 `update_holiday_taken`

→ Ajoute la fonction `update_holiday_taken` dans la liste d'outils de repli (sans email).

459 `],`

→ Ferme la liste conditionnelle des outils de l'agent.

460 `description="""`

→ Ouvre la chaîne multi-lignes décrivant le rôle général de l'agent Approval, utilisée par le Team manager pour le routage.

461 `An internal agent that processes HR holiday request approvals or disapprovals.`

→ Décrit l'agent comme un agent interne traitant les approbations ou refus de demandes de congé.

462 It updates the employee's holiday data and notifies the employee of the decision via email.

→ Précise que l'agent met à jour les données de congé et notifie l'employé par email.

463 """

→ Ferme la chaîne multi-lignes du paramètre description.

464 instructions="""

→ Ouvre la chaîne multi-lignes définissant les instructions détaillées (prompt système) de l'agent Approval - ici une chaîne simple, pas une f-string, car aucune variable Python n'a besoin d'y être interpolée.

465 - Your job is to process holiday request outcomes (approve or disapprove) from the Airtable 'holiday_requests' table.

→ Ligne d'instruction précisant la mission : traiter les décisions d'approbation ou de refus.

466 - For each request:

→ Ligne d'instruction introduisant les étapes à suivre pour chaque demande.

467 - Retrieve the request by 'request_id' using a filter formula "({request_id})='{request_id}'" to get "employee_id", "full_name", "requested_days", and "status".

→ Demande de récupérer la demande via une formule de filtre Airtable pour obtenir ses champs, décrivant en langage naturel dans le prompt la même syntaxe de formule utilisée en dur dans le code Flask plus haut.

468 - Use 'get_employee_holiday' to get the employee's email for notifications.

→ Demande d'utiliser get_employee_holiday pour obtenir l'email de l'employé à notifier.

469 - If status is "approved":

→ Ligne d'instruction introduisant le cas où la demande est approuvée.

470 - Use 'update_holiday_taken' to increment "holidays_taken" by "requested_days" and update "last_holiday_taken".

→ Demande d'incrémenter les jours pris et de mettre à jour la date du dernier congé via update_holiday_taken.

471 - Send an email to the employee's email confirming approval.

→ Demande d'envoyer un email à l'employé confirmant l'approbation.

472 - If status is "disapproved":

→ Ligne d'instruction introduisant le cas où la demande est refusée.

473 - Send an email to the employee confirming disapproval.

→ Demande d'envoyer un email à l'employé confirmant le refus.

474 - Include the employee's name, requested days, and decision in the email.

→ Précise le contenu attendu de l'email : nom, jours demandés et décision.

475 - Do not modify Airtable data unless explicitly required (e.g., for approved requests).

→ Interdit de modifier Airtable sauf si explicitement nécessaire, comme pour une approbation - une consigne de prudence limitant les effets de bord non désirés du LLM sur des données réelles.

476 - If no matching request_id is found, log: "No request found for the provided ID."

→ Définit le message de log standard lorsqu'aucune demande ne correspond à l'identifiant.

477 """

→ Ferme la chaîne multi-lignes du paramètre instructions.

478)

→ Ferme l'appel au constructeur Agent commencé à la ligne 447.

479 `logger.info("Approval Agent initialized successfully")`
→ Journalise en info l'initialisation réussie de l'agent Approval.

480 `except Exception as e:`
→ Capture toute exception survenue pendant l'initialisation de l'agent Approval.

481 `logger.error(f"Error initializing Approval Agent: {e}")`
→ Journalise en erreur le message de l'exception survenue.

482 `raise`
→ Relève l'exception pour interrompre le script en cas d'échec d'initialisation.

484 `# Project Agent`
→ Commentaire annonçant la définition de l'agent de gestion de projets.

485 `try:`
→ Ouvre un bloc try pour capturer les erreurs lors de l'initialisation de l'agent Project.

486 `project_agent = Agent(`
→ Commence la définition du troisième agent spécialisé de l'équipe : l'agent Agno chargé de la gestion des affectations projet des employés.

487 `name="Project Management Agent",`
→ Donne un nom lisible à l'agent de gestion de projets.

488 `model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),`
→ Configure le modèle OpenAI gpt-4o-mini utilisé par cet agent, avec sa clé API.

489 `markdown=True,`
→ Active le formatage Markdown pour les réponses de l'agent.

490 `tools=[`
→ Ouvre la liste conditionnelle des outils disponibles pour l'agent.

491 `email_tools,`
→ Ajoute l'outil d'envoi d'emails à la liste des outils de l'agent.

492 `FileTools(),`
→ Ajoute l'outil de manipulation de fichiers à la liste des outils de l'agent.

493 `get_employee_project,`
→ Ajoute la fonction `get_employee_project` comme outil disponible pour l'agent.

494 `update_next_project`
→ Ajoute la fonction `update_next_project` comme outil disponible pour l'agent.

495 `] if email_tools else [`
→ Ferme la première branche de la liste et ouvre la branche alternative si `email_tools` est indisponible.

496 `FileTools(),`
→ Ajoute l'outil de manipulation de fichiers dans la liste d'outils de repli (sans email).

497 `get_employee_project,`
→ Ajoute la fonction `get_employee_project` dans la liste d'outils de repli (sans email).

498 `update_next_project`
→ Ajoute la fonction `update_next_project` dans la liste d'outils de repli (sans email).

499 `],`
→ Ferme la liste conditionnelle des outils de l'agent.

500 description="""

→ Ouvre la chaîne multi-lignes décrivant le rôle général de l'agent Project, utilisée par le Team manager pour le routage.

501 An internal company assistant that helps employees view and manage their current and future project assignments.

→ Décrit l'agent comme un assistant interne aidant à consulter et gérer les affectations de projet.

502 This agent reads employee project data from an Airtable table using the employee's unique ID.

→ Précise que l'agent lit les données de projet Airtable via l'identifiant employé.

503 It can also update planned future projects when new information is provided.

→ Précise que l'agent peut mettre à jour les futurs projets prévus lorsque de nouvelles informations sont fournies.

504 """ ,

→ Ferme la chaîne multi-lignes du paramètre description.

505 instructions="""

→ Ouvre la chaîne multi-lignes définissant les instructions détaillées de l'agent Project.

506 - Use the Airtable project table to manage employee project data by 'employee_id'.

→ Ligne d'instruction précisant l'usage de la table Airtable des projets par identifiant employé.

507 - For each employee, retrieve:

→ Ligne d'instruction introduisant la liste des informations à récupérer par employé.

508 - Current project name

→ Précise que le nom du projet courant fait partie des informations à récupérer.

509 - Role

→ Précise que le rôle fait partie des informations à récupérer.

510 - Start date

→ Précise que la date de début fait partie des informations à récupérer.

511 - End date

→ Précise que la date de fin fait partie des informations à récupérer.

512 - Optional future assignment details under "next_project"

→ Précise que les détails de la future affectation sous "next_project" sont optionnels.

513 - Respond to user questions like:

→ Ligne d'instruction introduisant des exemples de questions utilisateur à traiter.

514 - "What is my current project?"

→ Exemple de question utilisateur concernant le projet courant.

515 - "When does my project end?"

→ Exemple de question utilisateur concernant la date de fin de projet.

516 - "What's my role in the project?"

→ Exemple de question utilisateur concernant le rôle sur le projet.

517 - If the user shares new project info, update the employee's "next_project" fields:

→ Ligne d'instruction précisant la mise à jour des champs next_project si l'utilisateur fournit de nouvelles infos.

518 - next_project_name

→ Précise le champ next_project_name à mettre à jour.

519 - next_project_start_date

→ Précise le champ next_project_start_date à mettre à jour.

520 - next_project_role

→ Précise le champ next_project_role à mettre à jour.

521 - Always match using 'employee_id' only - do not use names for lookups.

→ Impose de toujours rechercher par employee_id uniquement, jamais par nom - une consigne de fiabilité car deux employés peuvent partager un nom, mais pas un identifiant.

522 - Respond clearly, briefly, and professionally.

→ Demande des réponses claires, brèves et professionnelles.

523 - If no matching employee_id is found, reply: "No project record found for the provided employee ID."

→ Définit le message de réponse standard lorsqu'aucun projet ne correspond à l'identifiant.

524 """

→ Ferme la chaîne multi-lignes du paramètre instructions.

525)

→ Ferme l'appel au constructeur Agent commencé à la ligne 486.

526 logger.info("Project Agent initialized successfully")

→ Journalise en info l'initialisation réussie de l'agent Project.

527 except Exception as e:

→ Capture toute exception survenue pendant l'initialisation de l'agent Project.

528 logger.error(f"Error initializing Project Agent: {e}")

→ Journalise en erreur le message de l'exception survenue.

529 raise

→ Relève l'exception pour interrompre le script en cas d'échec d'initialisation.

531 # ChatBot Team Manager

→ Commentaire annonçant la définition de l'équipe d'agents gestionnaire du chat.

532 try:

→ Ouvre un bloc try pour capturer les erreurs lors de l'initialisation du gestionnaire d'équipe.

533 sqlite_storage = SqliteStorage(

→ Commence l'instanciation du backend de stockage SQLite d'Agno, qui va persister sur disque l'historique des conversations de l'équipe : sans cela, chaque appel à ChatBot_Team.run() serait complètement amnésique d'un appel à l'autre.

534 table_name="agent_sessions",

→ Définit le nom de la table SQLite (au sens base de données relationnelle classique, distincte des tables Airtable) utilisée pour stocker les sessions.

535 db_file=db_file

→ Définit le fichier de base de données SQLite à utiliser, lu depuis l'environnement plus haut (avec une valeur par défaut).

536)

→ Ferme l'appel au constructeur SqliteStorage.

537 logger.info("SQLite storage initialized successfully")

→ Journalise en info l'initialisation réussie du stockage SQLite.

539 ChatBot_Team = Team(

→ Commence la définition de l'équipe Agno (pattern multi-agent) regroupant les trois agents spécialisés définis plus haut sous un manager commun qui décide, pour chaque message entrant, quel agent doit y répondre.

540 `name="HR Assistant Team Manager",`

→ Donne un nom lisible à l'équipe gestionnaire de l'assistant RH.

541 `members=[holiday_agent, project_agent, approval_agent],`

→ Définit les trois agents membres de l'équipe : congés, projets et approbation, chacun spécialisé sur son propre domaine et ses propres outils.

542 `storage=sqlite_storage,`

→ Associe le stockage SQLite précédemment configuré à l'équipe, pour la persistance de l'historique de conversation entre les appels.

543 `add_history_to_messages=True,`

→ Active l'ajout de l'historique de conversation aux messages envoyés au modèle à chaque nouvel appel : c'est ce qui permet à l'équipe de "se souvenir" du contexte d'une question précédente (mémoire conversationnelle), plutôt que de traiter chaque message comme totalement isolé.

544 `model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),`

→ Configure le modèle OpenAI gpt-4o-mini utilisé par le gestionnaire d'équipe lui-même (pour analyser l'intention et décider du routage), avec sa clé API.

545 `description="""`

→ Ouvre la chaîne multi-lignes décrivant le rôle général de l'équipe.

546 `An intelligent HR Assistant Team Manager responsible for handling employee queries related to holidays, project assignments, and holiday request approvals.`

→ Décrit l'équipe comme gestionnaire intelligent traitant les requêtes RH sur congés, projets et approbations.

547 `This team manages delegated tasks to the most appropriate specialist agent:`

→ Précise que l'équipe délègue les tâches à l'agent spécialiste le plus approprié.

548 - `The **holiday agent** manages vacation day tracking and holiday request submissions.`

→ Décrit le rôle de l'agent holiday au sein de l'équipe.

549 - `The **project agent** oversees project assignments, updates, and status inquiries.`

→ Décrit le rôle de l'agent project au sein de l'équipe.

550 - `The **approval agent** processes HR approvals or disapprovals and notifies employees.`

→ Décrit le rôle de l'agent approval au sein de l'équipe.

551 `The manager ensures that every incoming query is handled by the appropriate agent based on the user's intent.`

→ Précise que le gestionnaire s'assure que chaque requête est traitée par l'agent adapté à l'intention.

552 `""",`

→ Ferme la chaîne multi-lignes du paramètre description.

553 `instructions="""`

→ Ouvre la chaîne multi-lignes définissant les instructions détaillées (prompt système) du gestionnaire d'équipe, qui codifie sa logique de routage.

554 `You are an intelligent routing agent that serves as the HR Assistant Team Manager.`

→ Ligne d'instruction définissant le rôle du gestionnaire comme agent de routage intelligent.

555 `Your role is to:`

→ Ligne d'instruction introduisant la liste des responsabilités du gestionnaire.

556 - Receive user questions related to employee holidays, project assignments, or holiday request approvals.

→ Précise la réception des questions utilisateur sur congés, projets ou approbations.

557 - Analyze the intent of the query.

→ Demande d'analyser l'intention de la requête utilisateur.

558 - Forward the query to the correct agent:

→ Ligne d'instruction introduisant les règles de routage vers l'agent correct.

559 - If the query relates to holiday, time off, vacation leave balance, or holiday requests, assign it to the ****Holiday Days Calculator Agent****.

→ Règle de routage vers l'agent Holiday pour les requêtes de congé.

560 - If the query relates to project names, roles, start/end dates, or updates, assign it to the ****Project Management Agent****.

→ Règle de routage vers l'agent Project pour les requêtes de projet.

561 - If the query relates to holiday request approval or disapproval (e.g., from a web hook or internal trigger), assign it to the ****Holiday Approval Agent****.

→ Règle de routage vers l'agent Approval pour les requêtes d'approbation.

562 - Never try to answer the user directly - your job is to decide which agent is most suited to respond.

→ Interdit au gestionnaire de répondre lui-même, son rôle étant uniquement de router - c'est la caractéristique distinctive d'un agent "manager" dans le pattern multi-agent : il orchestre plutôt qu'il n'exécute.

563 - If a query could match multiple domains, use judgment based on parsing and specificity.

→ Demande d'utiliser son jugement en cas d'ambiguïté entre plusieurs domaines.

564 - If no clear match exists, return a polite message suggesting the user clarify their request.

→ Demande de renvoyer un message poli invitant à clarifier la demande si aucun agent ne correspond.

565 Always strive for clarity and efficiency, ensuring accurate responses from the appropriate domain expert.

→ Rappelle l'objectif général de clarté et d'efficacité dans le routage des réponses.

566 """

→ Ferme la chaîne multi-lignes du paramètre instructions.

567)

→ Ferme l'appel au constructeur Team commencé à la ligne 539.

568 logger.info("ChatBot Team Manager initialized successfully")

→ Journalise en info l'initialisation réussie du gestionnaire d'équipe.

569 except Exception as e:

→ Capture toute exception survenue pendant l'initialisation du gestionnaire d'équipe.

570 logger.error(f"Error initializing ChatBot Team Manager: {e}")

→ Journalise en erreur le message de l'exception survenue.

571 raise

→ Relève l'exception pour interrompre le script en cas d'échec d'initialisation.

573 # Main interaction loop (optional - can be uncommented for CLI testing)

→ Commentaire indiquant que la boucle d'interaction suivante est optionnelle, pour des tests en CLI.

574 def main_cli():

→ Définit une fonction optionnelle offrant une interface en ligne de commande (CLI) pour dialoguer directement avec l'équipe

d'agents sans passer par Flask ni un frontend web, pratique pour tester la logique de routage et les agents isolément.

```
575 """Optional CLI interface for testing"""
```

→ Docstring décrivant le rôle de la fonction main_cli.

```
576 print("HR Assistant System started. Type 'quit' to exit.")
```

→ Affiche un message de bienvenue indiquant comment quitter l'interface CLI.

```
577 while True:
```

→ Démarre une boucle infinie lisant les entrées utilisateur en continu.

```
578     try:
```

→ Ouvre un bloc try pour capturer les erreurs lors de chaque itération de la boucle.

```
579         user_input = input("You: ")
```

→ Lit une ligne de texte saisie par l'utilisateur dans la console (fonction standard input()).

```
580         if user_input.lower() in ['exit', 'quit']:
```

→ Vérifie si l'utilisateur a tapé "exit" ou "quit" (insensible à la casse, grâce à .lower()) pour quitter.

```
581             print("Bot: Goodbye!")
```

→ Affiche un message d'au revoir avant de quitter la boucle.

```
582             break
```

→ Sort de la boucle while pour terminer l'interface CLI.

```
583         response = ChatBot_Team.run(user_input, markdown=True)
```

→ Exécute l'équipe d'agents avec le texte saisi par l'utilisateur et récupère la réponse, de la même façon que la route /chat de l'API HTTP mais en local dans la console.

```
584         print("Bot:", response.content)
```

→ Affiche la réponse de l'équipe d'agents dans la console.

```
585     except KeyboardInterrupt:
```

→ Capture l'interruption clavier (Ctrl+C) survenue pendant la boucle.

```
586         print("\nBot: Goodbye!")
```

→ Affiche un message d'au revoir suite à l'interruption clavier.

```
587         break
```

→ Sort de la boucle while suite à l'interruption clavier.

```
588     except Exception as e:
```

→ Capture toute autre exception survenue pendant l'itération de la boucle.

```
589         logger.error(f"Error in main loop: {e}")
```

→ Journalise en erreur le message de l'exception survenue dans la boucle principale.

```
590         print(f"Error: {e}")
```

→ Affiche le message d'erreur dans la console pour l'utilisateur.

```
592 if __name__ == "__main__":
```

→ Vérifie que le script est exécuté directement (et non importé comme module).

```
593     try:
```

→ Ouvre un bloc try englobant l'exécution principale du script.

```
594         logger.info("HR System fully initialized and ready")
```

→ Journalise en info que le système RH est entièrement initialisé et prêt.

```

595     print(f"System running at: {public_url}")
    → Affiche dans la console l'URL publique (ngrok ou localhost de repli) à laquelle le système est accessible.

596     print("Press Ctrl+C to stop")
    → Affiche une instruction indiquant comment arrêter le script.

598     # Uncomment the next line if you want CLI interface
    → Commentaire indiquant comment activer l'interface CLI en décommentant la ligne suivante.

599     # main_cli()
    → Appel à main_cli() laissé en commentaire, désactivant l'interface CLI par défaut ; par défaut le script tourne donc uniquement comme serveur API (Flask + agents), sans boucle de saisie console.

601     # Keep the main thread alive
    → Commentaire annonçant le maintien en vie du thread principal.

602     while True:
    → Démarre une boucle infinie pour empêcher le script principal de se terminer, puisque Flask tourne dans un thread démon séparé qui s'arrêterait sinon avec le thread principal.

603         time.sleep(1)
    → Met le thread principal en pause d'une seconde à chaque itération de la boucle, une façon simple (bien que peu élégante) de garder le processus vivant sans consommer de CPU en continu.

604     except KeyboardInterrupt:
    → Capture l'interruption clavier (Ctrl+C) survenue pendant la boucle d'attente principale.

605         logger.info("Shutting down...")
    → Journalise en info l'arrêt du système suite à l'interruption clavier.

606         print("Shutting down...")
    → Affiche un message indiquant que le système est en cours d'arrêt.

607     except Exception as e:
    → Capture toute autre exception critique survenue au niveau principal du script.

608         logger.error(f"Critical error: {e}")
    → Journalise en erreur critique le message de l'exception survenue.

609         raise
    → Relève l'exception pour terminer le script en signalant l'erreur critique.

```

35-HR-Assistant/app.py

Interface web Streamlit servant de client de chat pour interroger l'API Flask de l'assistant RH (congés, projets).

90 lignes de code annotées ci-dessous.

```

1 import streamlit as st
    → Importe la bibliothèque Streamlit, un framework Python qui permet de construire une interface web interactive (formulaires, boutons, zones de texte) directement à partir d'un script Python, sans avoir à écrire de HTML/CSS/JavaScript séparés. Chaque interaction utilisateur (clic, saisie) réexécute l'intégralité du script de haut en bas, un modèle d'exécution particulier à garder en tête pour comprendre le reste du fichier.

2 import requests
    → Importe la bibliothèque tierce requests, utilisée ici comme client HTTP pour appeler l'API Flask du fichier rhAgent.py (ce script Streamlit est donc un simple frontend, toute la logique d'agents et d'accès Airtable restant côté serveur Flask).

```

3 import json

→ Importe le module standard de manipulation du format JSON (disponible mais pas explicitement utilisé plus loin dans ce fichier, requests gérant déjà la sérialisation/désérialisation JSON automatiquement).

4 from datetime import datetime

→ Importe la classe datetime du module standard pour manipuler des dates et heures (disponible mais non utilisée explicitement dans le code visible plus bas).

6 # Configuration de la page

→ Commentaire annonçant la configuration générale de la page Streamlit.

7 st.set_page_config(

→ Commence la configuration des paramètres globaux de la page Streamlit ; cet appel doit être la première commande Streamlit du script pour fonctionner correctement.

8 page_title="HR Assistant",

→ Définit le titre affiché dans l'onglet du navigateur.

9 page_icon="",

→ Définit l'icône (favicon) affichée dans l'onglet du navigateur, ici un emoji.

10 layout="centered"

→ Configure la mise en page de l'application en mode centré (par opposition à "wide" qui utiliserait toute la largeur de l'écran), adapté à une interface de chat simple.

11)

→ Ferme l'appel à st.set_page_config commencé plus haut.

13 # URL de votre API Flask

→ Commentaire indiquant que la constante suivante définit l'URL de l'API Flask.

14 API_BASE_URL = "http://127.0.0.1:5000"

→ Définit l'URL de base de l'API Flask locale (rhAgent.py, qui écoute sur le port 5000) que cette interface va interroger ; contrairement au serveur Flask, ce frontend n'a pas besoin d'être exposé via ngrok car c'est l'utilisateur qui l'ouvre lui-même dans son navigateur local.

16 # Titre principal

→ Commentaire annonçant l'affichage du titre principal de la page.

17 st.title("HR Assistant System")

→ Affiche le titre principal "HR Assistant System" en haut de la page, un composant Streamlit de haut niveau qui génère un titre HTML stylé sans code CSS à écrire.

19 # Zone Employee ID

→ Commentaire annonçant la zone de saisie de l'identifiant employé.

20 col1, col2 = st.columns([1, 2])

→ Crée deux colonnes de largeurs relatives 1 et 2 (la seconde étant deux fois plus large que la première) pour organiser la mise en page côte à côte, un système de grille simple fourni par Streamlit.

21 with col1:

→ Ouvre un bloc de contexte plaçant les éléments suivants dans la première colonne.

22 employee_id = st.number_input("Employee ID:", min_value=1, value=1)

→ Affiche un champ numérique pour saisir l'identifiant employé, avec une valeur minimale de 1 (empêchant les identifiants négatifs ou nuls) et une valeur par défaut de 1 ; la valeur saisie est directement stockée dans la variable Python employee_id.

23 with col2:

→ Ouvre un bloc de contexte plaçant les éléments suivants dans la seconde colonne.

```
24     st.write("") # Espace vide
```

→ Affiche une chaîne vide pour créer un espace visuel dans la seconde colonne, un artifice courant pour aligner verticalement des éléments entre colonnes.

```
26 # Fonction pour appeler l'API
```

→ Commentaire annonçant la définition de la fonction d'appel à l'API Flask.

```
27 def call_hr_api(message):
```

→ Définit la fonction qui envoie un message texte à l'API Flask (route /chat) et retourne la réponse de l'équipe d'agents, centralisant ainsi toute la logique d'appel réseau en un seul endroit réutilisé par les différents boutons plus bas.

```
28     try:
```

→ Ouvre un bloc try pour capturer les erreurs lors de l'appel HTTP.

```
29         response = requests.post(
```

→ Envoie une requête HTTP POST vers l'API Flask.

```
30             f"{API_BASE_URL}/chat",
```

→ Cible l'endpoint /chat de l'API Flask en concaténant l'URL de base, exactement la route définie côté serveur dans rhAgent.py qui délègue au ChatBot_Team.

```
31             json={"message": message},
```

→ Fournit le message utilisateur dans le corps JSON de la requête ; le paramètre json= de requests sérialise automatiquement ce dictionnaire.

```
32             timeout=10
```

→ Définit un délai maximal de 10 secondes pour la requête HTTP, pour que l'interface ne reste pas bloquée indéfiniment si le serveur Flask ne répond pas.

```
33         )
```

→ Ferme l'appel à requests.post commencé plus haut.

```
34         if response.status_code == 200:
```

→ Vérifie si la requête a réussi avec un code de statut HTTP 200.

```
35             return response.json().get("response", "Pas de réponse")
```

→ Retourne le champ "response" du JSON reçu (le contenu généré par l'équipe d'agents côté serveur), ou un texte par défaut si ce champ est absent.

```
36         else:
```

→ Cas où le code de statut HTTP n'est pas 200.

```
37             return f"Erreur API: {response.status_code}"
```

→ Retourne un message d'erreur incluant le code de statut HTTP reçu.

```
38     except requests.exceptions.ConnectionError:
```

→ Capture spécifiquement une erreur de connexion réseau (serveur injoignable, port fermé) vers l'API, distincte d'une erreur applicative renvoyée par le serveur.

```
39         return "Erreur: Serveur HR non accessible"
```

→ Retourne un message indiquant que le serveur RH est inaccessible - un message ciblé, plus clair pour l'utilisateur qu'une trace d'exception brute.

```
40     except Exception as e:
```

→ Capture toute autre exception survenue pendant l'appel à l'API.

```
41         return f"Erreur: {str(e)}"
```

→ Retourne un message d'erreur générique incluant le détail de l'exception.

43 # Zone de chat

→ Commentaire annonçant la section d'affichage du chat.

44 st.subheader("Chat")

→ Affiche un sous-titre "Chat" au-dessus de la zone de conversation.

46 # Initialiser l'historique

→ Commentaire annonçant l'initialisation de l'historique des messages.

47 if "messages" not in st.session_state:

→ Vérifie si la clé "messages" n'existe pas encore dans st.session_state : Streamlit réexécute tout le script à chaque interaction utilisateur, donc les variables Python normales seraient réinitialisées à chaque clic - session_state est le mécanisme de Streamlit qui persiste des données entre ces ré exécutions successives, indispensable pour conserver un historique de conversation.

48 st.session_state.messages = []

→ Initialise l'historique des messages à une liste vide dans l'état de session, uniquement lors du tout premier chargement de la page (grâce au if précédent).

50 # Afficher les messages

→ Commentaire annonçant l'affichage de l'historique des messages.

51 for message in st.session_state.messages:

→ Itère sur chaque message stocké dans l'historique de session, à chaque réexécution du script (donc à chaque interaction), pour réafficher toute la conversation depuis le début.

52 if message["role"] == "user":

→ Vérifie si le message courant provient de l'utilisateur.

53 st.write(f"**Vous:** {message['content']}")

→ Affiche le message utilisateur préfixé par "Vous:" en gras (la syntaxe **...** est interprétée comme du Markdown par st.write).

54 else:

→ Cas où le message ne provient pas de l'utilisateur (donc de l'assistant).

55 st.write(f"**Assistant:** {message['content']}")

→ Affiche le message de l'assistant préfixé par "Assistant:" en gras.

57 # Zone de saisie

→ Commentaire annonçant le champ de saisie du message utilisateur.

58 user_input = st.text_input("Votre message:", placeholder="Ex: Combien de jours de congés me reste-t-il ?")

→ Affiche un champ de texte pour saisir un message, avec un texte indicatif (placeholder) donnant un exemple de question à l'utilisateur.

60 # Boutons

→ Commentaire annonçant la rangée de boutons d'action.

61 col1, col2, col3 = st.columns(3)

→ Crée trois colonnes de largeurs égales pour organiser côte à côte les boutons d'action ci-dessous.

63 with col1:

→ Ouvre un bloc de contexte plaçant les éléments suivants dans la première colonne.

64 if st.button("Envoyer") and user_input:

→ Vérifie si le bouton "Envoyer" a été cliqué (Streamlit renvoie True le run qui suit immédiatement le clic) et qu'un texte a bien été saisi, pour éviter d'envoyer un message vide à l'API.

65 `# Ajouter Employee ID au message`
→ Commentaire annonçant la concaténation de l'identifiant employé au message.

66 `full_message = f"Employee ID: {employee_id}. {user_input}"`
→ Construit le message complet en préfixant l'identifiant employé au texte saisi, afin que l'équipe d'agents côté serveur (qui n'a pas de notion de session utilisateur authentifiée) sache pour quel employé elle doit chercher les données.

68 `# Ajouter à l'historique`
→ Commentaire annonçant l'ajout du message utilisateur à l'historique.

69 `st.session_state.messages.append({"role": "user", "content": user_input})`
→ Ajoute le message utilisateur (le texte brut saisi, sans le préfixe `employee_id`) à l'historique de session, pour un affichage plus lisible que le message technique réellement envoyé à l'API.

71 `# Appeler l'API`
→ Commentaire annonçant l'appel à l'API Flask avec le message complet.

72 `with st.spinner("En cours..."):`
→ Affiche un indicateur de chargement animé (spinner) pendant l'exécution du bloc suivant, pour donner un retour visuel à l'utilisateur pendant l'attente de la réponse de l'agent (qui peut prendre plusieurs secondes, un appel LLM n'étant pas instantané).

73 `response = call_hr_api(full_message)`
→ Appelle la fonction `call_hr_api` avec le message complet (incluant l'`employee_id`) et récupère la réponse.

75 `st.session_state.messages.append({"role": "assistant", "content": response})`
→ Ajoute la réponse de l'assistant à l'historique de session.

76 `st.rerun()`
→ Force Streamlit à relancer immédiatement l'exécution du script depuis le début pour rafraîchir l'affichage avec le nouvel historique, plutôt que d'attendre la prochaine interaction utilisateur.

78 `with col2:`
→ Ouvre un bloc de contexte plaçant les éléments suivants dans la deuxième colonne.

79 `if st.button("Mes congés"):`
→ Vérifie si le bouton raccourci "Mes congés" a été cliqué ; ce bouton envoie une question prédéfinie plutôt que de nécessiter une saisie manuelle, pour un accès rapide à une action courante.

80 `message = f"Employee ID: {employee_id}. Quel est mon solde de congés ?"`
→ Construit un message prédéfini demandant le solde de congés pour cet employé.

81 `st.session_state.messages.append({"role": "user", "content": "Mes congés"})`
→ Ajoute un message utilisateur simplifié "Mes congés" à l'historique de session (plutôt que la question complète réellement envoyée), pour un affichage plus concis correspondant au libellé du bouton.

83 `with st.spinner("En cours..."):`
→ Affiche un indicateur de chargement pendant l'exécution du bloc suivant.

84 `response = call_hr_api(message)`
→ Appelle la fonction `call_hr_api` avec le message prédéfini et récupère la réponse.

86 `st.session_state.messages.append({"role": "assistant", "content": response})`
→ Ajoute la réponse de l'assistant à l'historique de session.

87 `st.rerun()`
→ Force Streamlit à relancer l'exécution du script pour rafraîchir l'affichage.

89 `with col3:`

→ Ouvre un bloc de contexte plaçant les éléments suivants dans la troisième colonne.

```
90     if st.button("Mon projet"):
```

→ Vérifie si le bouton raccourci "Mon projet" a été cliqué.

```
91         message = f"Employee ID: {employee_id}. Quel est mon projet actuel ?"
```

→ Construit un message prédéfini demandant le projet actuel pour cet employé.

```
92         st.session_state.messages.append({"role": "user", "content": "Mon projet"})
```

→ Ajoute un message utilisateur simplifié "Mon projet" à l'historique de session.

```
94         with st.spinner("En cours..."):
```

→ Affiche un indicateur de chargement pendant l'exécution du bloc suivant.

```
95         response = call_hr_api(message)
```

→ Appelle la fonction `call_hr_api` avec le message prédéfini et récupère la réponse.

```
97         st.session_state.messages.append({"role": "assistant", "content": response})
```

→ Ajoute la réponse de l'assistant à l'historique de session.

```
98     st.rerun()
```

→ Force Streamlit à relancer l'exécution du script pour rafraîchir l'affichage.

```
100 # Bouton pour vider l'historique
```

→ Commentaire annonçant le bouton de réinitialisation de l'historique de chat.

```
101 if st.button("Vider le chat"):
```

→ Vérifie si le bouton "Vider le chat" a été cliqué.

```
102     st.session_state.messages = []
```

→ Réinitialise l'historique des messages à une liste vide, écrasant ainsi la conversation stockée en session.

```
103     st.rerun()
```

→ Force Streamlit à relancer l'exécution du script pour rafraîchir l'affichage (la liste de messages affichée redevient vide).

```
105 # Test de connexion
```

→ Commentaire annonçant la section de test de connexion à l'API.

```
106 st.subheader("Test de connexion")
```

→ Affiche un sous-titre "Test de connexion" au-dessus de la zone de test.

```
107 if st.button("Tester l'API"):
```

→ Vérifie si le bouton "Tester l'API" a été cliqué.

```
108     try:
```

→ Ouvre un bloc try pour capturer les erreurs lors du test de connexion.

```
109         response = requests.get(f"{API_BASE_URL}/", timeout=5)
```

→ Envoie une requête HTTP GET vers la racine de l'API Flask (une route non explicitement définie dans `rhAgent.py`, donc cet appel vise surtout à vérifier que le serveur répond, même par une erreur 404, plutôt qu'à récupérer un contenu précis) avec un délai de 5 secondes.

```
110         st.success("Connexion OK")
```

→ Affiche un message de succès en vert (composant Streamlit dédié) dès lors que la requête n'a pas levé d'exception, sans même vérifier le code de statut HTTP retourné.

```
111     except:
```

→ Capture toute exception survenue lors de la requête de test, sans préciser son type (except nu) - une pratique généralement

déconseillée en Python car elle masque aussi des erreurs inattendues, mais acceptable ici pour un simple bouton de diagnostic.

```
112         st.error("Connexion échouée - Vérifiez que le serveur Flask est démarré")
```

→ Affiche un message d'erreur en rouge invitant à vérifier que le serveur Flask est démarré.

Projet 36 — 36-rfp-Agent

Remplit un questionnaire d'appel d'offres via Google Sheets

Objectif

Remplit automatiquement un questionnaire d'appel d'offres (RFP) en recuperant les informations necessaires depuis une feuille Google Sheets et en generant des reponses adaptees a chaque question du questionnaire.

Code source, ligne par ligne

36-rfp-Agent/main.py

Agent Agno qui lit et complète automatiquement une feuille Google Sheets de réponses à un appel d'offres (RFP) à partir d'une base de connaissances documentaire (RAG).

189 lignes de code annotées ci-dessous.

1 import os

→ Importe le module standard pour interagir avec le système de fichiers et les variables d'environnement.

2 import pygsheets

→ Importe pygsheets, une bibliothèque tierce qui simplifie l'utilisation de l'API Google Sheets (elle encapsule l'authentification OAuth2/compte de service et expose une interface Python plus concise que le SDK officiel Google pour lire/écrire des feuilles de calcul).

3 import pandas as pd

→ Importe la bibliothèque pandas sous l'alias usuel pd. Pandas fournit la structure DataFrame, un tableau de données en mémoire (lignes/colonnes typées, façon feuille de calcul ou table SQL) qui sert ici de format intermédiaire entre une feuille Google Sheets et le JSON échangé avec l'agent.

4 from agno.tools import tool

→ Importe le décorateur tool du framework Agno. Ce décorateur transforme une fonction Python ordinaire en un outil (function calling) que l'agent peut appeler : il extrait automatiquement la signature typée et la docstring de la fonction pour générer la définition que le LLM voit et peut invoquer, exactement le même principe que passer directement une fonction dans tools=[...] mais avec un contrôle plus fin (nom personnalisé, description explicite, affichage du résultat).

5 from textwrap import dedent

→ Importe dedent, qui retire l'indentation commune d'une chaîne multi-lignes - utilisé plus bas pour écrire proprement le prompt d'instructions de l'agent sans que l'indentation du code Python ne s'y retrouve.

6 from agno.agent import Agent

→ Importe la classe Agent du framework Agno, la brique de base pour construire l'agent RFP défini plus bas.

7 from agno.models.openai import OpenAIChat

→ Importe la classe OpenAIChat d'Agno, le wrapper autour de l'API Chat Completions d'OpenAI utilisé comme modèle de langage de cet agent.

8 from agno.vectordb.pgvector import PgVector

→ Importe PgVector, une classe Agno permettant d'utiliser PostgreSQL (avec l'extension pgvector) comme base de données vectorielle. Cet import est présent dans le fichier mais PgVector n'est en réalité pas instancié plus bas (c'est Qdrant qui est utilisé à la place) : il s'agit probablement d'un vestige d'une version antérieure du script ou d'une alternative envisagée, sans impact sur l'exécution puisqu'un import inutilisé ne casse rien en Python.

9 from agno.tools.reasoning import ReasoningTools

→ Importe ReasoningTools, un outil Agno qui donne à l'agent une capacité de raisonnement explicite en plusieurs étapes (une sorte de "brouillon" structuré que le modèle peut utiliser pour planifier son approche avant d'agir) : c'est un outil comme un autre au sens function calling, mais dont l'effet est d'améliorer la fiabilité du modèle sur des tâches complexes à plusieurs étapes, comme remplir un tableur en croisant plusieurs sources.

10 from agno.knowledge.markdown import MarkdownKnowledgeBase

→ Importe MarkdownKnowledgeBase, une classe Agno (plus ancienne que la classe Knowledge unifiée vue dans le projet Import-Export) spécifiquement dédiée à construire une base de connaissances RAG à partir d'un dossier de fichiers Markdown.

11 from agno.vectordb.qdrant import Qdrant

→ Importe Qdrant, une classe Agno permettant d'utiliser Qdrant comme base de données vectorielle : Qdrant est un moteur de recherche vectorielle open-source, ici utilisé en mode hébergé (via une URL et une clé API, contrairement à LanceDB qui était un fichier local dans le premier projet), ce qui permet de partager la même base de connaissances entre plusieurs exécutions ou machines.

12 from dotenv import load_dotenv

→ Importe la fonction qui charge les variables d'environnement depuis un fichier .env.

13 import json

→ Importe le module standard de manipulation du format JSON, utilisé plus bas pour capturer spécifiquement les erreurs de parsing JSON.

14 from io import StringIO

→ Importe StringIO, une classe qui permet de traiter une chaîne de caractères en mémoire comme si c'était un fichier ouvert - utilisée plus bas pour satisfaire l'API de pandas (pd.read_json) qui, dans les versions récentes, préfère recevoir un objet fichier plutôt qu'une chaîne brute, afin d'éviter un avertissement de dépréciation (FutureWarning).

16 # Charger les variables d'environnement

→ Commentaire annonçant le chargement des variables d'environnement.

17 load_dotenv()

→ Charge les variables d'environnement définies dans le fichier .env du projet.

19 # Récupération des variables d'environnement

→ Commentaire annonçant la lecture des variables d'environnement utilisées par le script.

20 qdrant_api_key = os.getenv("QDRANT_API_KEY")

→ Lit la clé API du service Qdrant hébergé depuis les variables d'environnement.

21 qdrant_url = os.getenv("QDRANT_URL")

→ Lit l'URL du service Qdrant depuis les variables d'environnement (typiquement l'adresse d'une instance Qdrant Cloud ou auto-hébergée).

22 openai_api_key = os.getenv("OPENAI_API_KEY")

→ Lit la clé API OpenAI depuis les variables d'environnement.

24 # Autorisation Google Sheets

→ Commentaire annonçant l'authentification auprès de l'API Google Sheets.

25 gc = pygsheets.authorize(service_account_file='36-rfp-Agent/client_secret.json')

→ Authentifie le client Google Sheets via un compte de service (service account), un mécanisme d'authentification serveur-à-serveur de Google Cloud où un fichier JSON de clés privées (client_secret.json) fait office d'identité applicative, sans intervention humaine ni fenêtre de connexion interactive - adapté à un script automatisé qui doit s'exécuter sans supervision.

27 #### Setup Knowledge Base ####

→ Commentaire de section annonçant la configuration de la base de connaissances RAG.

28 vector_db = Qdrant(

→ Commence l'instanciation de la base vectorielle Qdrant utilisée par la base de connaissances : c'est ici la brique de

stockage/recherche par similarité qui sous-tend le RAG (Retrieval-Augmented Generation) de cet agent.

29 `collection="docs_markdown",`

→ Définit le nom de la collection Qdrant (l'équivalent d'une table) qui stockera les vecteurs des documents de l'entreprise.

30 `api_key=qdrant_api_key,`

→ Fournit la clé API Qdrant lue précédemment pour l'authentification auprès du service hébergé.

31 `url=qdrant_url,`

→ Fournit l'URL du service Qdrant lue précédemment.

32)

→ Ferme l'appel au constructeur Qdrant commencé plus haut.

34 `knowledge_base = MarkdownKnowledgeBase(`

→ Commence l'instanciation de la base de connaissances construite à partir de fichiers Markdown : cette classe combine la lecture des fichiers sources et la base vectorielle en un seul objet, à la différence de la classe Knowledge plus générique et plus récente vue dans le projet Import-Export.

35 `path="36-rfp-Agent/docs/markdown/",`

→ Définit le chemin du dossier contenant les fichiers Markdown de la base de connaissances de l'entreprise (documentation produit, réponses types...) à indexer.

36 `vector_db=vector_db,`

→ Associe la base vectorielle Qdrant configurée plus haut à cette base de connaissances, comme moteur de stockage/recherche des embeddings.

37 `num_documents=5,`

→ Configure le nombre de documents (chunks) les plus pertinents à récupérer lors d'une recherche RAG : ce paramètre contrôle la quantité de contexte injectée dans le prompt du modèle à chaque requête - un compromis entre couvrir suffisamment d'information pertinente et ne pas saturer le contexte du LLM avec du contenu peu utile.

38)

→ Ferme l'appel au constructeur MarkdownKnowledgeBase commencé plus haut.

40 `@tool(`

→ Applique le décorateur tool du framework Agno à la fonction suivante, pour l'exposer comme un outil que l'agent RFP pourra appeler pendant sa génération (function calling).

41 `name="list_worksheets",`

→ Définit le nom explicite sous lequel l'outil sera référencé par l'agent (et visible dans son prompt), plutôt que de reprendre automatiquement le nom de la fonction Python.

42 `description="List all available worksheets in the Google Sheet.",`

→ Définit la description de l'outil : c'est ce texte, plutôt que la docstring, que le modèle de langage lit en priorité pour décider quand et pourquoi appeler cet outil.

43 `show_result=True,`

→ Configure l'outil pour afficher son résultat dans les logs d'exécution de l'agent, utile pour suivre pas à pas ce que fait l'agent (notamment ici avec stream=True plus bas).

44)

→ Ferme l'appel au décorateur tool commencé plus haut.

45 `def list_worksheets(sheet_id: str) -> str:`

→ Définit la fonction outil qui liste les onglets (worksheets) disponibles d'une feuille Google Sheets donnée ; les annotations de type (sheet_id: str, retour -> str) participent au schéma exposé au modèle pour qu'il sache quel type d'argument fournir.

46 `"""`

→ Ouvre la docstring de la fonction list_worksheets.

47 List all available worksheets in a Google Sheet.
 → Ligne de docstring résumant le rôle de la fonction.

49 Args:
 → Ligne de docstring introduisant la description des paramètres.

50 sheet_id (str): The unique ID of the Google Sheet.
 → Ligne de docstring décrivant le paramètre sheet_id.

52 Returns:
 → Ligne de docstring introduisant la description de la valeur de retour.

53 str: List of available worksheet titles.
 → Ligne de docstring décrivant la valeur de retour de la fonction.

54 """
 → Ferme la docstring de la fonction list_worksheets.

55 try:
 → Ouvre un bloc try pour capturer les erreurs lors de l'accès à la feuille Google Sheets.

56 sh = gc.open_by_key(sheet_id)
 → Ouvre la feuille Google Sheets correspondant à l'identifiant fourni, via le client pygsheets authentifié précédemment.

57 worksheets = sh.worksheets()
 → Récupère la liste des objets onglet (worksheets) présents dans la feuille.

58 titles = [ws.title for ws in worksheets]
 → Construit la liste des titres de chaque onglet récupéré.

59 return f"Available worksheets: {' , '.join(titles)}"
 → Retourne une chaîne listant les titres des onglets disponibles, séparés par des virgules - ce texte simple (plutôt qu'une structure JSON) est ce que le modèle recevra en retour d'appel d'outil et devra interpréter lui-même.

60 except Exception as e:
 → Capture toute exception survenue lors de l'accès à la feuille.

61 return f"Error listing worksheets: {str(e)}"
 → Retourne un message d'erreur incluant le détail de l'exception survenue ; notez que l'erreur est retournée comme une chaîne normale plutôt que levée, car un outil d'agent doit renvoyer du texte que le modèle peut lire et sur lequel il peut raisonner, pas faire planter tout le programme.

63 @tool(
 → Applique le décorateur tool à la fonction suivante pour l'exposer comme second outil d'agent.

64 name="read_sheet_as_df",
 → Définit le nom sous lequel cet outil sera référencé par l'agent.

65 description="Read an entire Google Sheets worksheet as a pandas DataFrame.",
 → Définit la description de l'outil, utilisée par l'agent pour comprendre son usage.

66 show_result=True,
 → Configure l'outil pour afficher son résultat dans les logs d'exécution de l'agent.

67)
 → Ferme l'appel au décorateur tool commencé plus haut.

```

68 def read_sheet_as_df(sheet_id: str, worksheet_title: str) -> str:
    → Définit la fonction outil qui lit un onglet Google Sheets et le retourne sous forme de DataFrame sérialisé en JSON, la façon
    dont ce script fait transiter des données tabulaires vers/depuis le modèle de langage, qui ne manipule que du texte.

69     """
    → Ouvre la docstring de la fonction read_sheet_as_df.

70     Use this function to read an entire Google Sheets worksheet and return its data as a JSON-formatted
    DataFrame.
    → Ligne de docstring résumant le rôle de la fonction.

71     Args:
    → Ligne de docstring introduisant la description des paramètres.

72         sheet_id (str): The unique ID of the Google Sheet (from the URL).
    → Ligne de docstring décrivant le paramètre sheet_id.

73         worksheet_title (str): The tab name of the worksheet within the Google Sheet.
    → Ligne de docstring décrivant le paramètre worksheet_title.

74     Returns:
    → Ligne de docstring introduisant la description de la valeur de retour.

75         str: A JSON string representing the DataFrame containing the worksheet's data.
    → Ligne de docstring décrivant la valeur de retour de la fonction.

76     """
    → Ferme la docstring de la fonction read_sheet_as_df.

77     try:
    → Ouvre un bloc try pour capturer les erreurs lors de la lecture de l'onglet.

78         sh = gc.open_by_key(sheet_id)
    → Ouvre la feuille Google Sheets correspondant à l'identifiant fourni.

79         # Vérifier si l'onglet existe
    → Commentaire annonçant la vérification de l'existence de l'onglet demandé.

80         try:
    → Ouvre un bloc try imbriqué pour tenter d'accéder à l'onglet par son titre.

81             wks = sh.worksheet_by_title(worksheet_title)
    → Récupère l'objet onglet correspondant au titre fourni.

82         except:
    → Capture toute exception survenue si l'onglet demandé n'existe pas (except nu, moins précis mais suffisant ici puisque le seul
    cas géré est "onglet introuvable").

83         # Lister les onglets disponibles si celui demandé n'existe pas
    → Commentaire annonçant la construction de la liste des onglets disponibles en cas d'échec.

84         worksheets = sh.worksheets()
    → Récupère la liste des objets onglet présents dans la feuille.

85         available = [ws.title for ws in worksheets]
    → Construit la liste des titres de chaque onglet disponible.

86         return f"Worksheet '{worksheet_title}' not found. Available worksheets: {'',
    ''.join(available)}"

```

→ Retourne un message d'erreur indiquant l'onglet introuvable et listant les onglets disponibles - un message conçu pour être exploitable par le modèle, qui peut alors relancer l'outil avec le bon nom d'onglet plutôt que de rester bloqué.

```
91         df = wks.get_as_df()
```

→ Convertit le contenu de l'onglet Google Sheets en DataFrame pandas, transformant la grille de cellules en une structure tabulaire manipulable en Python.

```
92         return df.to_json(orient="records")
```

→ Sérialise le DataFrame en chaîne JSON avec orient="records", c'est-à-dire une liste d'objets où chaque ligne du tableau devient un dictionnaire {colonne: valeur} - le format le plus lisible pour un LLM qui doit ensuite raisonner ligne par ligne sur les questions du RFP.

```
93     except Exception as e:
```

→ Capture toute exception survenue lors de l'accès à la feuille au niveau externe.

```
94         return f"Error reading sheet: {str(e)}"
```

→ Retourne un message d'erreur incluant le détail de l'exception survenue.

```
96 @tool()
```

→ Applique le décorateur tool à la fonction suivante pour l'exposer comme troisième outil d'agent.

```
97     name="write_df_to_sheet",
```

→ Définit le nom sous lequel cet outil sera référencé par l'agent.

```
98     description="Write a DataFrame to a Google Sheets worksheet using the sheet ID.",
```

→ Définit la description de l'outil, utilisée par l'agent pour comprendre son usage.

```
99     show_result=True,
```

→ Configure l'outil pour afficher son résultat dans les logs d'exécution de l'agent.

```
100 )
```

→ Ferme l'appel au décorateur tool commencé plus haut.

```
101 def write_df_to_sheet()
```

→ Définit la fonction outil qui écrit un DataFrame dans un onglet Google Sheets, le dernier maillon du pipeline permettant à l'agent de renvoyer le tableau RFP complété vers Google Sheets.

```
102     df_json: str, sheet_id: str, worksheet_title: str, start_cell: list[int] = [1, 1]
```

→ Déclare les paramètres de la fonction : le JSON à écrire, l'identifiant de feuille, le titre d'onglet et la cellule de départ, avec une valeur par défaut [1, 1] (première ligne, première colonne) pour ce dernier paramètre optionnel.

```
103 ) -> str:
```

→ Termine la signature de la fonction en précisant qu'elle retourne une chaîne de caractères.

```
104     """
```

→ Ouvre la docstring de la fonction write_df_to_sheet.

```
105     Use this function to write a DataFrame (as JSON) to a Google Sheets worksheet.
```

→ Ligne de docstring résumant le rôle de la fonction.

```
107     Args:
```

→ Ligne de docstring introduisant la description des paramètres.

```
108         df_json (str): JSON string representing the DataFrame to write.
```

→ Ligne de docstring décrivant le paramètre df_json.

```
109         sheet_id (str): The unique ID of the Google Sheet (from the URL).
```

→ Ligne de docstring décrivant le paramètre sheet_id.

110 `worksheet_title (str):` The tab name of the worksheet.
→ Ligne de docstring décrivant le paramètre `worksheet_title`.

111 `start_cell (list[int], optional):` [row, column] location to start writing. Defaults to [1, 1].
→ Ligne de docstring décrivant le paramètre optionnel `start_cell` et sa valeur par défaut.

113 Returns:
→ Ligne de docstring introduisant la description de la valeur de retour.

114 `str:` Confirmation message upon successful write.
→ Ligne de docstring décrivant la valeur de retour de la fonction.

115 `"""`
→ Ferme la docstring de la fonction `write_df_to_sheet`.

116 `try:`
→ Ouvre un bloc `try` pour capturer les erreurs lors de l'écriture dans la feuille.

117 `# Gestion améliorée du JSON`
→ Commentaire annonçant le traitement du JSON reçu avant conversion en `DataFrame` ; ce nettoyage existe car un LLM peut parfois renvoyer son JSON entouré de balises de bloc de code Markdown au lieu de JSON pur, un artefact classique des sorties de modèles de langage qu'il faut anticiper côté code.

118 `if isinstance(df_json, str):`
→ Vérifie si l'argument `df_json` est bien une chaîne de caractères (le modèle pourrait en théorie fournir directement une structure déjà parsée selon la façon dont Agno transmet les arguments d'outil).

119 `# Nettoyer le JSON si nécessaire`
→ Commentaire annonçant le nettoyage éventuel de la chaîne JSON.

120 `df_json = df_json.strip()`
→ Retire les espaces et sauts de ligne en début et fin de la chaîne JSON.

121 `if df_json.startswith("```json"):`
→ Vérifie si la chaîne commence par un bloc de code Markdown ````json`, symptôme fréquent d'un LLM qui a enveloppé sa réponse JSON dans un formatage Markdown malgré les instructions.

122 `df_json = df_json.replace("```json", "").replace("```", "").strip()`
→ Retire les marqueurs de bloc de code Markdown (ouvrant et fermant) et les espaces superflus de la chaîne, pour obtenir du JSON strictement valide.

123 `# Parser le JSON avec StringIO pour éviter le warning`
→ Commentaire expliquant l'usage de `StringIO` pour éviter un avertissement de dépréciation émis par pandas quand on lui passe directement une chaîne littérale (par opposition à un chemin de fichier ou un objet fichier).

125 `df = pd.read_json(StringIO(df_json))`
→ Parse la chaîne JSON en `DataFrame` pandas, en l'enveloppant dans un objet `StringIO` pour se conformer à l'API attendue par les versions récentes de pandas.

126 `else:`
→ Cas où `df_json` n'est pas une chaîne de caractères.

127 `# Si ce n'est pas une string, essayer de convertir`
→ Commentaire annonçant une conversion directe si `df_json` n'est pas une chaîne.

128 `df = pd.DataFrame(df_json)`
→ Construit directement un `DataFrame` pandas à partir de l'objet `df_json` fourni (par exemple s'il s'agissait déjà d'une liste de dictionnaires).


```

130         sh = gc.open_by_key(sheet_id)
    → Ouvre la feuille Google Sheets correspondant à l'identifiant fourni.

131         try:
    → Ouvre un bloc try imbriqué pour tenter d'accéder à l'onglet cible par son titre.

132             wks = sh.worksheet_by_title(worksheet_title)
    → Récupère l'objet onglet correspondant au titre fourni.

133         except pygsheets.WorksheetNotFound:
    → Capture spécifiquement l'exception WorksheetNotFound de pygsheets, levée quand l'onglet demandé n'existe pas encore (contrairement à l'except nu utilisé plus haut dans read_sheet_as_df, ici l'exception est ciblée précisément, ce qui évite de masquer d'autres erreurs inattendues).

134             wks = sh.add_worksheet(worksheet_title)
    → Crée un nouvel onglet portant le titre demandé s'il n'existait pas, plutôt que d'échouer.

136             wks.clear()
    → Efface tout le contenu existant de l'onglet cible avant d'y écrire les nouvelles données, garantissant qu'aucune ancienne valeur résiduelle ne subsiste après l'écriture.

137             wks.set_dataframe(df, start=tuple(start_cell))
    → Écrit le DataFrame dans l'onglet en commençant à la cellule de départ spécifiée (convertie en tuple, le format attendu par pygsheets).

138             return f"Successfully wrote DataFrame to worksheet '{worksheet_title}' in sheet '{sheet_id}'. Rows: {len(df)}, Columns: {len(df.columns)}"
    → Retourne un message de confirmation précisant le nombre de lignes et colonnes écrites, une preuve textuelle de succès que le modèle peut relayer à l'utilisateur.

140         except json.JSONDecodeError as e:
    → Capture spécifiquement une exception d'échec de parsing JSON, distincte des autres erreurs possibles (accès réseau, feuille introuvable...).

141             return f"JSON parsing error: {str(e)}. Please provide valid JSON format."
    → Retourne un message d'erreur signalant un JSON invalide et demandant un format correct - un message directement actionnable par le modèle, qui peut alors reformuler son appel d'outil avec un JSON corrigé.

142         except Exception as e:
    → Capture toute autre exception survenue pendant l'écriture dans la feuille.

143             return f"Error writing to sheet: {str(e)}"
    → Retourne un message d'erreur incluant le détail de l'exception survenue.

145 rfp_agent = Agent(
    → Commence la définition de l'agent Agno chargé de compléter automatiquement les feuilles RFP (Request for Proposal, appel d'offres) à l'aide de la base de connaissances et des trois outils Google Sheets définis plus haut.

146     name="RFP Agent",
    → Donne un nom lisible à l'agent RFP.

147     model=OpenAIChat(id="gpt-4o", api_key=openai_api_key),
    → Configure le modèle OpenAI utilisé par l'agent : ici "gpt-4o" (le modèle complet, plus capable mais aussi plus coûteux que le "gpt-4o-mini" vu dans le projet RH), un choix cohérent avec une tâche qui demande de croiser plusieurs sources documentaires et de raisonner sur un tableau structuré.

148     knowledge=knowledge_base,
    → Associe la base de connaissances Markdown/Qdrant construite plus haut à l'agent, lui donnant accès au RAG documentaire de l'entreprise pour répondre aux questions du RFP.

```

149 `search_knowledge=True,`

→ Active la recherche dans la base de connaissances pour cet agent (contrairement au projet Import-Export, ici il n'y a pas de vérification conditionnelle car `knowledge_base` est toujours construit, sans clé optionnelle qui pourrait la désactiver).

150 `show_tool_calls=True,`

→ Active l'affichage des appels d'outils effectués par l'agent, pour suivre en direct quelles actions (lecture de feuille, recherche documentaire, écriture) l'agent choisit d'exécuter.

151 `tools=[ReasoningTools(add_instructions=True), list_worksheets, read_sheet_as_df, write_df_to_sheet],`

→ Fournit à l'agent l'outil de raisonnement explicite (avec `add_instructions=True` qui ajoute automatiquement au prompt système des consignes sur comment utiliser cet outil de raisonnement) ainsi que les trois outils Google Sheets définis plus haut ; c'est l'ensemble complet des capacités de fonction calling de cet agent.

152 `instructions=dedent(`

→ Commence l'appel à `dedent()` pour construire la chaîne d'instructions (prompt système) de l'agent, en supprimant l'indentation du code source de cette chaîne multi-lignes.

153 `"""`

→ Ouvre la chaîne multi-lignes contenant le texte des instructions.

154 `You are an RFP (Request-for-Proposal) agent.`

→ Ligne d'instruction définissant le rôle de l'agent comme agent de réponse à appel d'offres.

156 `Your role is to assist with completing RFP spreadsheets. You receive partially filled RFP sheets and use the company knowledge base to fill in the missing fields accurately.`

→ Décrit la mission de l'agent : compléter des feuilles RFP à l'aide de la base de connaissances RAG, plutôt que d'inventer des réponses à des questions commerciales/techniques sur l'entreprise.

158 `---`

→ Séparateur Markdown délimitant visuellement les sections des instructions.

160 `Tools You Have Access To:`

→ Titre de section annonçant la liste des outils disponibles pour l'agent - documenter explicitement les outils dans le prompt (en plus de leur description technique via `@tool`) aide le modèle à choisir le bon outil au bon moment dans un scénario à plusieurs étapes.

162 `1. list_worksheets(sheet_id: str)`

→ Présente le premier outil disponible : `list_worksheets`, avec sa signature.

163 `- Lists all available worksheets in a Google Sheet`

→ Décrit le rôle de l'outil `list_worksheets`.

164 `- Use this FIRST to check available worksheet names`

→ Précise que cet outil doit être utilisé en premier pour vérifier les noms d'onglets, une consigne d'ordre d'exécution qui prévient les erreurs "onglet introuvable" en aval.

166 `2. read_sheet_as_df(sheet_id: str, worksheet_title: str)`

→ Présente le second outil disponible : `read_sheet_as_df`, avec sa signature.

167 `- Reads an entire Google Sheets worksheet and returns its contents as a pandas DataFrame (serialized as JSON).`

→ Décrit le rôle de l'outil `read_sheet_as_df`.

168 `- Use this to:`

→ Introduit la liste des usages prévus pour cet outil.

169 `• Load the current contents of an RFP sheet`

→ Précise un usage : charger le contenu actuel d'une feuille RFP.

170 • Inspect which fields are already filled and which are missing

→ Précise un usage : identifier les champs déjà remplis et ceux manquants.

172 3. write_df_to_sheet(df_json: str, sheet_id: str, worksheet_title: str, start_cell: List[int]
= [1, 1])

→ Présente le troisième outil disponible : write_df_to_sheet, avec sa signature.

173 - Writes a pandas DataFrame (provided as a JSON string) to a specific worksheet in a Google Sheet, replacing all existing content.

→ Décrit le rôle de l'outil write_df_to_sheet, qui remplace tout le contenu existant.

174 - IMPORTANT: df_json must be valid JSON format

→ Avertit que le paramètre df_json doit impérativement être un JSON valide, anticipant dans le prompt le nettoyage défensif déjà codé dans la fonction elle-même (lignes 117-125).

175 - Use this to:

→ Introduit la liste des usages prévus pour cet outil.

176 • Write the filled RFP sheet back to Google Sheets

→ Précise un usage : réécrire la feuille RFP complétée vers Google Sheets.

177 • Overwrite the worksheet with the completed version

→ Précise un usage : écraser l'onglet avec la version complétée.

179 Sheet Identification Notes:

→ Titre de section annonçant des précisions sur l'identification des feuilles.

180 - `sheet\_id` is the unique identifier found in the Google Sheets URL:

→ Explique que sheet_id correspond à l'identifiant présent dans l'URL Google Sheets.

181 Example: For

`https://docs.google.com/spreadsheets/d/1yg8MQ7boc0lgURLQqZcX2fcY7AqjVlfQM8ZntwtbizA/edit?gid=0#gid=0`,
the `sheet\_id` is `1yg8MQ7boc0lgURLQqZcX2fcY7AqjVlfQM8ZntwtbizA`

→ Donne un exemple concret d'URL et l'identifiant de feuille qui en est extrait, pour lever toute ambiguïté sur ce que le modèle doit fournir en argument sheet_id.

182 - `worksheet\_title` refers to the tab name within the sheet, such as "Sheet1" or "RFP".

→ Explique que worksheet_title correspond au nom de l'onglet dans la feuille.

184 ---

→ Séparateur Markdown délimitant visuellement les sections des instructions.

186 Your Objective:

→ Titre de section annonçant les étapes de l'objectif principal de l'agent.

188 1. FIRST: Use list_worksheets to see available worksheet names

→ Première étape de l'objectif : lister les onglets disponibles.

189 2. Read the data using read_sheet_as_df with the correct worksheet name

→ Deuxième étape de l'objectif : lire les données de l'onglet correct.

190 3. Analyze the user's instructions and the structure of the RFP sheet

→ Troisième étape de l'objectif : analyser les instructions utilisateur et la structure de la feuille.

191 4. Use the company knowledge base to fill in missing fields. If no relevant information is found in the knowledge base, leave the field blank.

→ Quatrième étape : utiliser la base de connaissances RAG pour compléter les champs manquants, ou les laisser vides sinon

- une consigne anti-hallucination qui préfère un champ vide à une réponse inventée sur un document commercial officiel.

192 5. Fill in missing values using logic derived from the knowledge base or prompts
→ Cinquième étape : compléter les valeurs manquantes à partir de la logique de la base de connaissances.

193 6. Write the updated data using write_df_to_sheet with proper JSON format
→ Sixième étape : écrire les données mises à jour via write_df_to_sheet dans un format JSON valide.

195 ---
→ Séparateur Markdown délimitant visuellement les sections des instructions.

197 How to Answer When Filling Fields:
→ Titre de section annonçant le format de réponse attendu lors du remplissage des champs.

199 For each field you attempt to fill, explain your result in **natural language** using one of the following formats:
→ Demande d'expliquer chaque champ rempli en langage naturel selon les formats suivants - une contrainte de formatage de sortie, similaire dans l'esprit au gabarit HTML expected_output du projet Import-Export, mais ici imposée au sein même des instructions plutôt que via un paramètre dédié.

201 - **Yes** followed by a short explanation.
→ Décrit le format de réponse "Yes" suivi d'une courte explication.

202 - Example: **Yes** We support integration with Salesforce and Slack out of the box.*
→ Donne un exemple concret du format de réponse "Yes".

204 - **No** followed by a short explanation.
→ Décrit le format de réponse "No" suivi d'une courte explication.

205 - Example: **No** Our platform does not currently support HIPAA compliance.*
→ Donne un exemple concret du format de réponse "No".

207 - **Partial** followed by a short explanation.
→ Décrit le format de réponse "Partial" suivi d'une courte explication.

208 - Example: **Partial** We offer 24/7 support via email and chat, but phone support is limited to business hours.*
→ Donne un exemple concret du format de réponse "Partial".

210 This phrasing should be used inline as part of your reasoning and field-filling process – not just as a summary label.
→ Précise que ce format doit être utilisé au fil du raisonnement, pas seulement en résumé final.

212 ---
→ Séparateur Markdown délimitant visuellement les sections des instructions.

214 IMPORTANT: Always start by listing available worksheets to avoid WorksheetNotFound errors.
→ Rappelle l'importance de toujours commencer par lister les onglets pour éviter les erreurs, une répétition volontaire (déjà dit ligne 164 et 188) pour renforcer une consigne jugée critique - une technique courante de prompt engineering.

216 Respond like a knowledgeable and trustworthy analyst. Be clear, helpful, and direct.
→ Demande d'adopter un ton d'analyste compétent et digne de confiance, clair et direct.

217 ---
→ Séparateur Markdown final délimitant la fin des instructions.

218 ""
→ Ferme la chaîne multi-lignes contenant le texte des instructions.

219),

→ Ferme l'appel à `dedent()` commencé à la ligne 152.

220)

→ Ferme l'appel au constructeur `Agent` commencé à la ligne 145.

222 `rfp_agent.knowledge.load(recreate=True)`

→ Déclenche explicitement le (re)chargement de la base de connaissances : `recreate=True` force une réindexation complète (nouvelle lecture des fichiers Markdown, recalcul des embeddings, réécriture de la collection Qdrant) plutôt que de réutiliser un index déjà existant - utile pendant le développement quand les documents sources changent, mais coûteux en appels d'API d'embedding si on l'exécute à chaque lancement en production.

224 # Démarrage avec vérification des onglets disponibles

→ Commentaire annonçant le lancement de l'agent avec vérification préalable des onglets.

225 `rfp_agent.print_response(`

→ Exécute l'agent avec un prompt donné et affiche directement sa réponse (formatée, y compris les étapes intermédiaires d'appel d'outils) dans la console - une méthode pratique d'Agno pour un usage en script/CLI, par opposition à `run()` qui retourne un objet réponse à traiter programmatiquement comme dans les deux autres projets.

226 """

→ Ouvre la chaîne multi-lignes contenant le prompt utilisateur envoyé à l'agent.

227 Answer all the questions in the RFP spreadsheet based on our company knowledge base. only the 'Response / Answer' column should be updated.

→ Ligne du prompt demandant de répondre à toutes les questions de la feuille RFP en ne modifiant que la colonne réponse, pour préserver les colonnes de questions déjà présentes dans le tableau.

228 Sheet ID: '1yg8MQ7boc0lgURLQqZcX2fcY7AqjVlfQM8Zntwtbiza'

→ Ligne du prompt fournissant l'identifiant concret de la feuille Google Sheets à traiter, en dur dans ce script de démonstration plutôt que passé en argument de ligne de commande.

230 IMPORTANT: First check what worksheets are available in this sheet, then use the correct worksheet name.

→ Ligne du prompt rappelant de vérifier d'abord les onglets disponibles avant d'agir.

231 """,

→ Ferme la chaîne multi-lignes du prompt utilisateur.

232 stream=True,

→ Active le mode streaming : la réponse de l'agent (et les traces de ses appels d'outils intermédiaires) s'affiche au fur et à mesure de sa génération plutôt que d'attendre la fin complète, comme un chat qui "s'écrit" progressivement à l'écran.

233)

→ Ferme l'appel à `rfp_agent.print_response` commencé à la ligne 225.